

PV-PP

PROGRAMMER'S REFERENCE

V2.1.167 PRODUCTIVE VALUE-PRODUCTIVE POWER
FRAMEWORK



Lance Amundsen | amundsenlance@gmail.com
<https://amundsenlance.github.io/pvpp-runtime-api/>

PV-PP Programmer's Reference

Edition 2.1, for PV-PP Runtime v2.1.167 (Framework 2.1, Build 167). 2026.

DOI: 10.5281/zenodo.23238871. Previous version, for runtime build 141: 10.5281/zenodo.23072506.

Copyright © 2026 Lance Amundsen. This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0): <https://creativecommons.org/licenses/by/4.0/>

PV-PP™ is a trademark of Lance Amundsen. The license covers the text of this book; it grants no rights in the trademark.

The code skeletons in Appendix B are also distributed in the PV-PP runtime repository and are licensed there under the Apache License, Version 2.0, the license of that repository. They are provided "as is", without warranties or conditions of any kind.

Companion volumes: PV-PP Getting Started, DOI 10.5281/zenodo.23238468; PV-PP Programmer's Guide, DOI 10.5281/zenodo.23238712.

Runtime, code, and all three volumes: <https://github.com/AmundsenLance/pvpp-runtime-api> and <https://amundsenlance.github.io/pvpp-runtime-api/>

Contact: Lance Amundsen, amundsenlance@gmail.com

About This Book

This book states exactly what the Productive Value–Productive Power (PV-PP) runtime provides: the types, methods, adapter contracts, statuses, invariants, and limits of the frozen runtime v2.1.167—the decision core it carries unchanged from build 141, and the supervision layer that build 167 adds—with each statement tied to the source that controls it. It is the third of three volumes. PV-PP Getting Started is the short entry point; the PV-PP Programmer's Guide explains how to understand, model, build, and test with the PV-PP framework. This Reference is exact and is ordered for lookup.

Authority. Every entry is subordinate to its sources. The PV-PP framework's V2.1 specification set controls what the framework requires. The frozen v2.1.167 source, its tests, and its hash manifest control what the runtime does. The Guide controls nothing; it explains. Where this Reference and a source disagree, the source controls and the difference is a defect in this Reference (1.11). Framework and runtime are separately versioned: the framework governs semantics and the runtime implements them (Section 1.4).

Citations. A citation such as C15/D4 §3 names section 3 of document 4 in compendium 15 of the framework's V2.1 specification set. The set is maintained by the author and is not published with this book; Appendix F lists every cited document with its standing and authority level. A citation such as Guide 6.11 names a section of the PV-PP Programmer's Guide. Doc 05 is the Runtime API Reference shipped in the runtime's docs folder; this Reference supersedes it.

Frozen runtime. Runtime v2.1.167 is frozen and is verified by the V0167_FILE_HASHES.sha256 manifest. Nothing in this Reference proposes or depends on a change to it. Where the framework requires more than v2.1.167 implements, the Reference says so and labels the gap. It never describes prospective behavior as present.

Which runtime. Runtime v2.1.167 is Framework Version 2.1, runtime build 167, frozen. It was developed under the identifier v0.167, its package metadata still reports version 0.167, and it is in the repository folder runtime-v2.1.167, beside the earlier runtimes. The frozen release archive unpacks to a folder named runtime-v2.2, and release records written before the naming decision call it Runtime 2.2 v0.167 (15.9, D10). All of these name the same files.

What changed from the previous edition. The previous edition described build 141, v0.141. Build 167 changes nothing in the framework and nothing in the decision core: 205 of the 206 files in build 141's manifest are byte-identical, and the one that differs, pyproject.toml, carries only package metadata. It adds one package, pvpp_runtime.supervision, documented in the new Chapter 16, with its limits in Chapter 15, its skeleton in B.7, and its audit in H.7. Chapters 2 through 14 describe the decision core and keep the name v0.141 for it (1.4). Nothing this book said about build 141 has become false, and nothing has been removed from the API.

Four boundaries. Every statement in the Reference belongs to exactly one of four boundaries, and the Reference keeps them apart. The Guide uses the same four columns.

Table 0.1 — The four boundaries

Boundary	What it covers	How it is marked
Framework (V2.1)	What the canonical architecture requires, whether or not the runtime implements it	Cited to a compendium

Boundary	What it covers	How it is marked
Runtime v2.1.167	What the frozen code does and does not do, established by source and, where available, frozen tests or identified confirmation runs; includes its limitations	Cited to module and symbol, and to the test or confirmation run
Host	What an application must supply: facts, adapters, stores, enforcement	Marked as a host obligation
Prospective	Capabilities a successor runtime or host instrumentation could provide	Marked as prospective; never presented as behavior

Relationship to the Guide

The two volumes divide the work. The Guide teaches and is ordered for learning. The Reference is exact and is ordered for lookup. The Guide may simplify; the Reference may not. The Guide points to the Reference for exact constructors, fields, and statuses. The Reference points back to the Guide for explanation and worked examples, and does not repeat them.

Table 0.2 — Division of work between the volumes

Question	Guide	Reference
Why does this stage exist?	Explains, with examples	States the contract and cites it
How do I model a system?	Chapters 3–5 and prompts	Construct definitions only
What does this constructor take?	Summarizes	Exact signature, fields, and validation
What statuses can this return?	Illustrates	Complete list with meanings
What does the runtime not do?	Flags where it matters	Complete limitation register
Worked examples	Coffee vending; agent operations	Minimal skeletons only (Appendix B)

API Classification

Python export visibility is not API authority.

The runtime package exports 335 names at its top level, and its supervision package, added in build 167, 51 more. Being importable does not make a symbol part of the supported surface. Each symbol receives exactly one of six labels; the sixth, PUBLIC / REFERENCE, is used only in the supervision package (1.5). The export-surface audit (Appendix H) assigns the labels from recorded evidence, not from names or memory. The audit also covers the public methods of the three core classes.

Table 0.3 — API classification labels

Label	Meaning	Reader obligation
PUBLIC / SUPPORTED	An intended entry point for applications, documented with a full entry and stable within v0.141	Use as the entry's contract states
PUBLIC / ADVANCED	An intended entry point for specialized needs such as re-entry, provenance, or diagnostics, documented with a full entry	Use only with the preconditions the entry states
IMPLEMENTATION / SUPPORT	Exported because public signatures or stage results carry it; applications receive or pass it but do not build on it	Treat as data; construct only where documented
INTERNAL	Exported but not intended for application use	Do not use
LEGACY / COMPATIBILITY	Retained from an earlier runtime generation, or superseded by a newer entry point	Do not use in new code; see Chapter 15
PUBLIC / REFERENCE	A supplied reference implementation of a contract that a deployment may implement differently; supervision package only	Use as an example, or as is; nothing requires it

Default rule. A symbol whose status the evidence does not settle defaults to IMPLEMENTATION / SUPPORT. It is entered in the reconciliation register (H.5) with the evidence on each side. A label changes only by a recorded reconciliation.

Evidence. Each label rests on four kinds of evidence, recorded per symbol: the export list and module of origin; the source, including callers and whether stages consume the symbol; the tests, including whether they call it as an entry point or reach it only through other calls; and the shipped API documentation. No single kind of evidence is sufficient.

How to Read an Entry

Entries come at two depths. Symbols labeled PUBLIC receive full entries. All other symbols appear as compact rows, generated from the source and grouped under the surface that uses them. Every symbol has exactly one primary entry. Other chapters refer to it by section number and do not repeat it.

Table 0.4 — Full entry template

Field	Content
Name and label	Symbol, module of origin, and classification label
Purpose	One sentence: what it is for, in framework terms
Framework basis	The construct or stage it implements, with a compendium citation
Signature or fields	Exact, generated from source, with defaults and types as written
Preconditions	What must hold before use, including host obligations

Field	Content
Result	Return type or resulting state, and every status it can carry
Invariants	What the runtime guarantees and checks
Failure behavior	Exceptions, failure statuses, and the stage that reports them
Does not	What it does not decide, authorize, or guarantee
Evidence	Tests and benchmarks that exercise it
See also	Guide sections and related entries

Table 0.5 — Compact row template

Column	Content
Name and kind	Symbol; dataclass, class, function, method, constant, or module
Label	Classification
Role and use	One line on its role, then the entries that produce or consume it
Source	Module and line

Generated and written layers. Signatures, field lists, and compact rows are generated from the frozen source by script and mechanically compared with it at every build, so any drift from v0.141 is detected. Prose—purpose, preconditions, and “does not” statements—is written by hand and reviewed against the source.

Editorial Rules

The Reference is written to these rules.

- State what is implemented, not what is intended. A framework requirement without implementation goes in the limitation register, not in an entry.
- Cite every behavioral claim to source and, where a frozen test exists, to that test. For behavior the frozen suite does not cover, say so and identify any confirmation run used (Table E.5). Cite every framework claim to a compendium.
- Keep proved, tested, and conjectured apart. State mathematical results with the standing the proof program gives them—for example “proved locally only” or “strong but conditional”—and never stronger.
- Never infer a symbol’s classification from its name. Classification is an audit result.
- Never describe host code as runtime behavior, or runtime behavior as a host responsibility.
- Use the framework’s terms as the Guide defines them. New terms enter through the glossary (Chapter 2) first.
- Point to the Guide’s explanations and examples rather than repeating them.
- Give each symbol one primary entry. Duplicated description is a defect.

Plan of the Book

The book has four parts and eight appendices. Part I fixes conventions and the architecture. Part II is the reference proper, organized by functional surface: each chapter gives the contract for its surface and the entries for the symbols that implement it. Part III collects what cuts across surfaces. Part IV, Chapter 16, documents the supervision layer that build 167 adds; it follows Part III so that the numbering of Chapters 1 through 15 is unchanged.

Table 0.6 — Plan of the book

Part	Chapters	Role
I. Conventions and architecture	1–3	Authority, classification, notation, stage contracts
II. Reference by surface	4–13	Registry and configuration; state; information; objectives; graph; decision cycle; adapters; execution; re-entry; testing
III. Cross-cutting	14–15	Statuses and errors; limitations, compatibility, and legacy entry points
IV. Supervision	16	The supervision layer that build 167 adds
Appendices	A–H	Index, skeletons, status index, versions, tests, sources, crosswalk, audit record

Measured scale. The top-level package exports 335 names, unchanged from build 141. They are 250 dataclasses; 21 other classes (15 adapter and service protocols, the runtime object, two registries, and three execution exceptions); 54 functions; one constant; and nine submodule names. PVPPRuntime has 89 public methods, PVPPRegistry has 30, and ExecutionBindingRegistry has 14. Some callable surface is reachable only through submodules; the joint-feasibility assessment in the feasibility module is one example. The supervision package adds 51 package-level names (38 of them in its `__all__`) and 13 submodules; its 19 classes have 100 public methods. The frozen suite reports 1,320 passing tests in 168 test files: build 141's 1,093 in 140 files, unchanged, and 227 for the supervision layer in 28. The 54 benchmark and scenario scripts are unchanged. 13.7–13.8 and Listing H.2 record the commands that reproduce these counts.

Contents

About This Book.....	2
Relationship to the Guide.....	3
API Classification.....	3
How to Read an Entry.....	4
Editorial Rules.....	5
Plan of the Book.....	5
Chapter 1 — Conventions, Authority, and API Classification.....	13
1.1 Purpose and Scope of the Reference.....	13
1.2 Source Hierarchy and Conflict Resolution.....	14
1.3 The Four Boundaries.....	15
1.4 Version Identity and the Frozen Runtime.....	16
1.5 API Classification Labels.....	17
1.6 Entry Formats.....	18
1.7 Conventions for Signatures, Types, and Statuses.....	20
1.8 Citation Conventions.....	20
1.9 Mathematical Standing.....	21
1.10 Conformance to Project Control.....	21
1.11 Reporting a Defect in This Reference.....	22
Chapter 2 — Notation and Glossary.....	24
2.1 The Canonical Stack.....	24
2.2 State and Value Symbols.....	25
2.3 Informational, Objective, and History Symbols.....	26
2.4 Decision-Cycle Terms.....	27
2.5 Execution Terms.....	29
2.6 Information, Change, and Re-entry Terms.....	30
2.7 Runtime Stage Labels.....	31
2.8 Notation Differences Across Sources.....	32
Chapter 3 — Canonical Architecture and Stage Contracts.....	34
3.1 The Canonical Order.....	34
3.2 Stage Contract Format.....	34
3.3 Φ and H : Pressure and Collapse Horizons.....	35
3.4 G and $\mathcal{R}$: Governing Domains and Regime.....	36
3.5 Graph, Seed, and Π : Policy Construction.....	37
3.6 Π Completeness.....	38
3.7 Constraints.....	39
3.8 Domain Framing.....	39
3.9 Joint-Recovery Check.....	40
3.10 Adequacy.....	41
3.11 Σ : Standard, Fallback, and Terminal Selection.....	41
3.12 ε and Layer 1: From Selection to Transition.....	42
3.13 Whole-Cycle Forms.....	43

3.14 Cross-Cutting Invariants.....	44
Chapter 4 — Runtime Object, Registry, and Configuration.....	45
4.1 The Runtime Object.....	45
4.2 The Registry.....	46
4.3 Domains and Domain Relations.....	48
4.4 Actions and Productive Powers.....	48
4.5 Constraints, Recovery Plans, and Recovery Necessities.....	50
4.6 Stage Configurations.....	52
4.7 Graph Instances, Transformations, and Compositions.....	56
4.8 Substrate and Configuration Identities.....	58
4.9 Configuration Provenance.....	59
4.10 Architecture-Adaptation Invariance.....	60
4.11 Where Each Core Method Is Documented.....	60
4.12 Classification Record and Compact Rows.....	62
Chapter 5 — State, Perception, and Capacity Authority.....	67
5.1 Actual Persistent State.....	67
5.2 The Represented World State.....	68
5.3 The Perceived Decision State.....	69
5.4 Possession, Access, Permission, Control, and Operability.....	70
5.5 Qualified Units and Productive Configurations.....	71
5.6 Recovery Resource State.....	72
5.7 Capacity Authority.....	73
5.8 Joint Feasibility of Active Recovery Obligations.....	74
5.9 Transition-Relevant State Sufficiency.....	76
5.10 Classification Record and Compact Rows.....	77
Chapter 6 — Information, Memory, Evidence, and Provenance.....	81
6.1 Information-Governance Metadata.....	81
6.2 Retained Memory and Licensed Retrieval.....	82
6.3 Transfer History.....	85
6.4 Replay Sufficiency.....	88
6.5 Prediction Error.....	90
6.6 Post-Execution Epistemic Update.....	92
6.7 Evidence Sources and Evidence Authority.....	94
6.8 Control-Provenance Topology.....	96
6.9 Security Evidence.....	98
6.10 Classification Record and Compact Rows.....	99
Chapter 7 — Objectives.....	104
7.1 The Objective Contract.....	104
7.2 Objective Sets.....	104
7.3 Where Objectives Come From.....	106
7.4 Objective Responsiveness.....	106
7.5 Objective-Directed Discovery.....	108
7.6 Objective Lifecycle and Invalidation.....	109
7.7 Classification Record and Compact Rows.....	110

Chapter 8 — Graph, Reachability, and Structural Admission.....	112
8.1 The Rules of the Structural Surface.....	112
8.2 Graph Construction.....	112
8.3 Reachability Statuses.....	115
8.4 Reachability Refresh.....	115
8.5 Structural Admission and Revision.....	116
8.6 Novel-Function Admission.....	118
8.7 Recovery-Necessity Admission and Revision.....	119
8.8 Structural Coverage.....	121
8.9 Capability Change and Trajectories.....	123
8.10 Capability-Driven Graph Change.....	125
8.11 Capability-Formation Prediction Error.....	127
8.12 Classification Record and Compact Rows.....	128
Chapter 9 — Decision-Cycle Operators and Results.....	133
9.1 How to Use This Chapter.....	133
9.2 The Cycle Request.....	133
9.3 Cycle Entry Points.....	135
9.4 Cycle Results.....	139
9.5 Sequences and Memory-Conditioned Cycles.....	140
9.6 Host Hooks the Cycle Calls.....	143
9.7 Stage Calls: Pressure, Horizons, Governing Domains, and Regime.....	147
9.8 Stage Calls: Policy Construction and Completeness.....	149
9.9 Stage Calls: Constraints and Joint Recovery.....	152
9.10 Projections.....	153
9.11 Stage Calls: Adequacy and Σ	155
9.12 Cycle-Integrity Audits.....	158
9.13 Paths Outside the Canonical Cycle.....	160
9.14 Classification Record and Compact Rows.....	161
Chapter 10 — Adapters and Services.....	170
10.1 The Adapter Rule.....	170
10.2 The World Adapter.....	172
10.3 The Projection Service.....	176
10.4 The Layer-1 Transition Service.....	177
10.5 Memory Retrieval and Epistemic Update Services.....	178
10.6 Transfer-History and Prediction-Error Services.....	179
10.7 The Capacity Authority Service.....	181
10.8 The Objective Responsiveness Adapter.....	181
10.9 Declared Protocols the Runtime Does Not Call.....	182
10.10 Conformance Checklist for Host Implementations.....	183
10.11 Classification Record.....	188
Chapter 11 — Execution Lineage.....	189
11.1 The Lineage in One Table.....	189
11.2 Execution License.....	191
11.3 Instantiation.....	193
11.4 ε Observation and Advancement.....	194

11.5 Binding Registration.....	197
11.6 Native Authorization.....	199
11.7 Invocation and Outcomes.....	201
11.8 From Native Result to ϵ	203
11.9 Control States and Reauthorization.....	204
11.10 Layer-1 Handoff and Transition.....	205
11.11 Transition Provenance and Recovery Bookkeeping.....	208
11.12 Identity and Evidence Integrity.....	210
11.13 Classification Record and Compact Rows.....	211
Chapter 12 — Change, Invalidation, and Re-entry.....	216
12.1 The Re-entry Model.....	216
12.2 Dependencies and Invalidation.....	217
12.3 Planning Re-entry.....	219
12.4 Reusable Artifacts and Executable Scope.....	222
12.5 Stage-Specific Re-entry Executors.....	224
12.6 Change Surfaces That Plan Re-entry.....	229
12.7 Epistemic Invalidation Orchestration.....	230
12.8 Joint-Recovery Re-entry.....	231
12.9 Classification Record and Compact Rows.....	232
Chapter 13 — Testing Utilities.....	237
13.1 The Role of the Testing Module.....	237
13.2 Sequence Diagnostics.....	237
13.3 Bounded Viability Oracle.....	238
13.4 Multidomain Stress Classification.....	239
13.5 Contaminated-Substrate Paired Assessment.....	240
13.6 Projection Attribution and Causal-State Comparison.....	242
13.7 Runtime Identity Verification.....	244
13.8 Running the Frozen Suite, Benchmarks, and Examples.....	244
13.9 Classification Record.....	246
Chapter 14 — Statuses, Errors, and Hostile Invariants.....	248
14.1 How Failure Is Reported.....	248
14.2 Stage Status Vocabulary.....	248
14.3 Whole-Cycle and Sequence Statuses.....	250
14.4 Execution Vocabulary.....	251
14.5 Structural, Information, and Change Statuses.....	252
14.6 Re-entry Statuses.....	252
14.7 Exceptions.....	253
14.8 Which Calls Raise and Which Return.....	254
14.9 Validation Failures.....	254
14.10 Hostile Invariants.....	255
14.11 Diagnosing a Refusal.....	258
Chapter 15 — Limitations, Compatibility, and Legacy Entry Points.....	259
15.1 Limitation Register.....	259
15.2 Known Defects.....	261

15.3 Host Obligations the Runtime Does Not Check.....	262
15.4 Legacy and Superseded Entry Points.....	264
15.5 Declared-Only Protocols.....	267
15.6 Runtime Constructs Without Dedicated Framework Ownership.....	267
15.7 Compatibility Across Runtime Generations.....	268
15.8 Evidence That Does Not Transfer.....	269
15.9 Known Documentation Discrepancies.....	269
15.10 Register Decisions and Classification Record.....	270
Chapter 16 — The Supervision Layer.....	273
16.1 Architecture, Authority, and Classification.....	273
16.2 The Host Trust Provider.....	276
16.3 The Canonical Bridge and Active Executions.....	278
16.4 Observation Intake and Admission.....	284
16.5 Dependencies, Fan-Out, and Snapshots.....	289
16.6 Continuation and Checkpoints.....	293
16.7 Ownership, Control, and Enforcement.....	299
16.8 Effect, Finality, and Layer 1.....	304
16.9 Recovery.....	310
16.10 The Store, Concurrency, and Deployment.....	314
16.11 Errors.....	316
16.12 Boundaries and Host Duties.....	318
16.13 Simulation and Operational Reference Adapters.....	320
16.14 Classification Record.....	320
Appendix A — Quick API Index.....	324
A.1 Reading the Index.....	324
A.2 Exported Symbols.....	324
A.3 Core Class Methods.....	335
A.4 Submodule Namespaces and Submodule-Only Surface.....	339
A.5 Supervision Surface.....	340
Appendix B — Code Skeletons.....	346
B.1 Registry Setup.....	346
B.2 One Decision Cycle.....	348
B.3 Execution Lineage.....	350
B.4 Layer-1 Transition Service.....	352
B.5 Re-entry After Change.....	352
B.6 Test Harness.....	354
B.7 Supervised Execution.....	356
Appendix C — Status and Error Index.....	361
C.1 Kinds.....	361
C.2 Families.....	361
C.3 Alphabetical Index.....	363
C.4 Exceptions.....	382
C.5 Validator Return Forms.....	383
C.6 How the Index Was Checked.....	384

C.7 Supervision Exceptions and Values.....	384
Appendix D — Version Matrix.....	387
D.1 Framework and Compendia.....	387
D.2 Runtime Generations and Builds.....	389
D.3 Σ Versions by Source.....	391
D.4 Proof Program.....	391
D.5 Companion Volumes and Retired Documents.....	392
D.6 Verification Environment.....	392
D.7 Version Conflicts.....	392
D.8 Feature Matrix by Build.....	393
Appendix E — Test and Benchmark Matrix.....	396
E.1 The Evidence Base.....	396
E.2 What Each Kind of Evidence Establishes.....	396
E.3 Test Files.....	397
E.4 Benchmarks.....	405
E.5 Behaviors No Test Covers.....	408
E.6 Resolution Check.....	408
E.7 Supervision-Layer Tests.....	409
Appendix F — Source and Authority Map.....	411
F.1 Authority Levels.....	411
F.2 Compendium Citations.....	411
F.3 Runtime and Repository Sources.....	413
F.4 Proof Program and Companion Sources.....	415
F.5 Retired Reader Documents.....	416
Appendix G — Guide-Reference Crosswalk.....	417
G.1 By Guide Chapter.....	417
G.2 By Guide Section.....	417
G.3 By Reference Chapter.....	419
G.4 The Guide's Code Files.....	420
G.5 Guide Pointers to the Reference.....	420
G.6 Legacy Surface.....	420
G.7 Supervision Crosswalk.....	421
Appendix H — Export-Surface Audit.....	422
H.1 Scope and Inputs.....	422
H.2 Evidence Collected.....	422
H.3 Decision Rules.....	422
H.4 Classification Record.....	423
H.5 Reconciliation Register.....	433
H.6 Reproduction.....	435
H.7 Supervision-Layer Classification.....	437

Chapter 1 — Conventions, Authority, and API Classification

This chapter sets the rules the rest of the book follows. It states what the Reference covers and for whom; how authority flows from project control, the canonical compendia, and the frozen runtime to each entry; how every statement is assigned to one of four boundaries; how the runtime is identified; how symbols are classified; what an entry looks like; and the conventions for signatures, statuses, citations, and mathematical standing. It ends with the checks every chapter must pass and the procedure for reporting a defect.

For any statement in this book, a reader must be able to tell which source controls it and which boundary it belongs to.

1.1 Purpose and Scope of the Reference

The Reference answers exact questions about runtime v2.1.167: what a symbol is, what it accepts, what it returns, what it checks, what it refuses, and what it leaves to the host. It is written so that an answer can be used as found—by a developer, by a reviewer checking an application, or by an AI assistant given the Reference as context for generating code (Guide 1.12). For that reason it uses exact identifiers, quotes signatures as the source writes them, and never paraphrases a status value.

The Reference documents every name the runtime package exports at its top level, every public method of the runtime object and the two registries, and the callable surface reachable through the exported submodules. Not all of these receive the same treatment: Section 1.5 explains how classification decides which symbols get full entries.

Table 1.1 — What the Reference covers and where the rest lives

Topic	In the Reference	Elsewhere
Runtime v2.1.167 surface	Complete: entries, statuses, limitations	—
Framework constructs	Definitions and contracts, cited	Canonical compendia; Guide Chapters 1–2
Modeling a system	Not covered	Guide Chapters 3–5
Worked examples	Minimal skeletons only (Appendix B)	Guide Chapters 6–10 and their code
Proofs	Standing of results only (1.9)	Proof program, V3.25 Final
Earlier runtimes	Compatibility only (Chapter 15)	Their own frozen trees and records
Runtime sidecar and agent prototypes	Not covered	Compendium 40
Changes to the runtime	Never proposed	A successor branch from v2.1.167

1.2 Source Hierarchy and Conflict Resolution

Project control assigns every document an authority level (C00/D3 §6). The Reference uses those levels, and adds one source that the levels do not cover: the frozen runtime itself. Framework and runtime are separately versioned and separately authoritative: framework owner files govern semantics, and the runtime implements them (C00/D9 §18). Table 1.2 shows what each source controls and what the Reference cites it for.

Table 1.2 — Sources and what they control

Source	Authority	Controls	Cited for
00 Project Control	Level 0	Status, versions, placement, reading order	Version identity; document status
Canonical owner files in 10, 15, 20	Levels 1–2	Architecture, stack order, operator definitions	The framework basis of each entry
Proof program (V3.25 Final)	Level 2	Theorem statements and their conditions	Mathematical standing
Frozen runtime v2.1.167: source, tests, manifest, release notes and records	Runtime version, controlled separately	What v2.1.167 does	Behavior, signatures, statuses, evidence
Supporting files in 10, 20, 30, 45	Levels 3–4	Explanation, testing, application	Context and evidence
The Guide and this Reference	Level 3	Nothing on their own	—
Retired reader documents 01–10	Superseded	Nothing	Continuity of terms only

The runtime controls what v2.1.167 does. It never controls what the framework requires. Conflicts are resolved by the following rules, and each resolution is recorded rather than made silently.

- **Framework and runtime differ.** Both are stated, each in its own boundary. If the runtime does less than the framework requires, the gap goes in the limitation register (15.1). If the runtime does something the framework forbids, the difference is logged as a discrepancy (15.6) and referred to project control; the Reference does not decide which is right.
- **Source and tests differ in reach.** The source says what runs; the tests say what is verified. Behavior that the source implements but no test exercises is stated as “implemented, not tested.”
- **Source and documentation differ.** The source prevails over docstrings, doc 05, and the release notes. Docstrings are used as evidence of intent, not of behavior.
- **Compendia differ.** An owning operator file prevails over a supporting file on that operator's internals (C00/D9 §3). Differences in cited versions are shown in Appendix D, not silently resolved.
- **The Reference differs from any source.** The source prevails and the difference is a defect in the Reference (1.11).

1.3 The Four Boundaries

Every statement belongs to one boundary: framework, runtime, host, or prospective (Table 0.1). A framework requirement that v0.141 does not implement produces up to three statements, never one that blurs them: "Framework V2.1 requires X" is a framework statement; "v0.141 does not implement X" is a runtime statement, recorded as a limitation; "a successor could implement X" is prospective. A current absence is a fact about v0.141, not a prospect. To assign a statement, ask these questions in order.

- **Could reading or running v0.141 show it false?** Then it is a runtime statement, and it must cite source and, where one exists, a test.
- **Does it say what an application must supply, do, or guarantee?** Then it is a host statement, marked as a host obligation.
- **Does it state what the canonical architecture requires or permits?** Then it is a framework statement, cited to a compendium. If v0.141 does not meet the requirement, add the runtime limitation.
- **Does it describe what a successor runtime or host instrumentation could provide?** Then it is prospective, and it is never presented as behavior.

Statements that are easy to misplace are the ones where a runtime check and a host duty sit next to each other. Table 1.3 lists the common cases.

Table 1.3 — Statements that are easy to misplace

Statement	Correct boundary	Why
"The runtime stops a bound function from being called outside the governed path."	Host	v0.141 issues single-use authorizations and refuses to invoke without one. It cannot stop host code from calling the function directly; keeping every call on the governed path is the host's duty (Guide 7.5).
"Actual state is updated when the function returns."	Host	The runtime never writes actual state. The host's Layer-1 transition does, from authoritative records, including after failure (Guide 7.8–7.9).
"An attempt already in flight can be cancelled."	Runtime and host; false as stated	The decision core is synchronous and gates before the call; it does not guarantee cancellation once a registered callable has begun (C00/D1 §23). Build 167's supervision layer lets the current owner request a stop, which the host's controller carries out and reports; whether a running action stops is the controller's, and nothing governs a native call once entered (16.7, 16.12). A pre-call gate or a stop request must not be described as guaranteed cancellation.

Statement	Correct boundary	Why
"v0.141 implements everything Framework V2.1 permits."	Neither, as stated	v0.141 is the one-for-one Version 2.1-aligned build, but framework permission does not establish runtime capability (C00/D3 §15). Each capability is settled in its entry or in Chapter 15.

1.4 Version Identity and the Frozen Runtime

The framework is Version 2.1, frozen as master file set MFS11; Version 2.0 is preserved as MFS10 (C00/D8 §1–2). The runtime has three frozen generations: Runtime V1 at v0.70; Runtime V2 at v0.131, which is preserved unchanged; and the Version 2.1 line, developed from v0.131 through builds v0.132 to v0.141 and frozen at v0.141. Project control records v0.141 as the frozen Version 2.1-aligned runtime implementation, completed as a one-for-one build after the framework freeze (C00/D1 §23; C00/D8 §10). The build is an implementation milestone. It does not transfer semantic authority from the framework to the software. Build 167 followed, developed from v0.141 through builds v0.142 to v0.167 and released as Runtime v2.1.167: it leaves the Version 2.1 decision core byte-identical and adds the supervision layer (Chapter 16). Its release records—the Official Release Record, the Owner Freeze Declaration, and the Frozen Source-of-Truth Manifest—identify it (Appendix F). The release notes say what each build added, and the Reference checks each claim against source, and against a test where one exists.

Naming in this book. In Chapters 2 through 15, v0.141 names the decision core as frozen at build 141 and carried byte-identical in v2.1.167; every statement those chapters make about v0.141 holds for v2.1.167's decision core. Where a statement concerns the runtime as a whole and build 167 changes it, the text says so. Chapter 16 describes what build 167 adds.

The synchronous boundary. The decision core is synchronous, in v2.1.167 as in v0.141. By itself it does not independently sense external changes, inject observations into an invocation already in progress, continuously supervise distributed execution, or guarantee cancellation after a registered callable has begun (C00/D1 §23; C00/D3 §15). Build 167's supervision layer lets host code govern a running action through checkpoints it calls: admitted observations, declared dependencies, continuation decisions made by the canonical runtime, and stop requests carried out by the host's controllers (Chapter 16). It senses, schedules, and stops nothing itself, and it does not govern a native call once entered (16.12).

A runtime tree is the frozen v2.1.167 tree only if every file its manifest covers matches its hash. Tree identity is not environment identity: the Python version, installed packages, and host configuration are recorded separately (4.9). V0167_FILE_HASHES.sha256 lists SHA-256 digests for 346 files. Build 141's manifest, V0141_FILE_HASHES.sha256, kept in the tree, matches 205 of its 206 files; the exception is pyproject.toml. Listing 1.1 shows the checks and the regression run; all were performed on a fresh extraction of the frozen archive for this edition. Test execution may create cache artifacts that the manifest does not cover. Do not infer from manifest exclusion that a cache file is untracked: the repository commits some `__pycache__` directories. Use repository status to tell generated artifacts from tracked files, and run the suite as in 13.8 so that it writes none.

Listing 1.1 — Verifying runtime identity

```
# From the runtime root (the directory holding V0167_FILE_HASHES.sha256)
sha256sum -c V0167_FILE_HASHES.sha256 --quiet && echo "runtime v2.1.167 verified"
```

```
# The decision core against build 141's manifest; the one expected difference is pyproject.toml
sha256sum -c V0141_FILE_HASHES.sha256 --quiet      # prints: pyproject.toml: FAILED

# Frozen regression suite; expected result: 1320 passed
PYTHONDONTWRITEBYTECODE=1 python -m pytest -q -p no:cacheprovider
```

Frozen means the following, for every chapter of this book.

- **Permitted:** importing and calling the runtime; implementing its adapter and service protocols; wrapping it in host code; running its tests and benchmarks.
- **Not permitted under the v2.1.167 name:** editing any file the manifest lists; patching runtime internals at run time; depending on underscore-prefixed attributes. An application that does any of these is not running v2.1.167, and results from it cannot be attributed to v2.1.167.
- **Successor work: branches from v2.1.167** and carries its own version (release notes). The Reference does not describe successor behavior.

1.5 API Classification Labels

Python export visibility is not API authority. Project control requires runtime documentation to classify the surface explicitly as supported API, advanced integration surface, implementation or support machinery, internal machinery, and legacy or compatibility surface (C00/D9 §18; C00/D6 §14). The labels in Table 0.3 implement that requirement. This section gives the criteria and the procedure that applies them. The labels classify code symbols. They are unrelated to project control's status labels for documents (C00/D3 §5), and the two vocabularies are never mixed.

Table 1.4 — Criteria and entry depth for each label

Label	Criteria	Entry
PUBLIC / SUPPORTED	Intended as an ordinary application entry point or required host protocol, shown by documentation and corroborated by use	Full
PUBLIC / ADVANCED	Intended for specialized host, integration, or diagnostic work: re-entry, provenance, integrity audits, explicit validation	Full, with preconditions stated first
IMPLEMENTATION / SUPPORT	Built by the runtime, or carried by public signatures or results; applications receive, inspect, or pass it, and construct it only where its contract says so	Compact row
INTERNAL	Exported, but used only inside its module or by tests of internals	Compact row, marked
LEGACY / COMPATIBILITY	Superseded by a newer entry point in the release lineage, or retained from V1 or V2	Compact row with replacement (Chapter 15)
PUBLIC / REFERENCE	A supplied implementation that illustrates a contract a deployment may implement differently (rule R7; supervision package only)	Summary table (16.13)

Scope of classification. Every top-level export is classified, and so is every public method of PVPPRuntime, PVPPRegistry, and ExecutionBindingRegistry. A class's label does not pass to its

methods; each method is classified on its own evidence. Callable surface reachable only through an exported submodule is classified under the same rules. Underscore-prefixed names are never API and are not classified.

Intended use and evidence. Intended use is the primary criterion. It is established from documentation—docstrings, interface definitions, doc 05, the release notes, and the owner files—and corroborated by four kinds of recorded evidence: the export and module of origin; the source, including callers and whether stages consume the symbol; direct use in examples, benchmarks, and tests; and the shipped API documentation. Usage counts support a label but never define one: an advanced entry with no direct test is still advanced, and a benchmark call does not by itself make a symbol supported. Records are classified by their construction contract—who is documented to build them—not by the label of whatever consumes them. When the evidence conflicts or does not settle the question, the symbol is labeled IMPLEMENTATION / SUPPORT and entered in the reconciliation register (H.5).

Reconciliation. A register entry records the symbol, the evidence on each side, the proposed label, the decision, who made it, and when. The Reference's author decides, subject to review by the project owner. A label changes only through a new register entry. Because v0.141 is frozen, a label is stable for v0.141; it says nothing about any successor runtime.

Build 167. The supervision package is classified by the same rules, with intended use read from the v0.167 API Classification Record v1.0. That record introduces the label PUBLIC / REFERENCE, which this book adopts as its sixth label under rule R7; the two classifications are reconciled in H.7. The labels of build 141's symbols are unchanged, because their code is unchanged.

1.6 Entry Formats

A full entry uses the fields of Table 0.4; a compact row uses the columns of Table 0.5. Signatures, field lists, source locations, and compact rows are generated from the frozen tree by script. Each build regenerates them and mechanically compares the result with the text, so drift from v0.141 is detected by the build. Purpose, preconditions, and "does not" statements are written by hand and reviewed against the source.

Table 1.5 is a complete full entry for a real method, written to the standard every later chapter follows. Its label is taken from the completed export-surface audit (Appendix H).

Table 1.5 — Example full entry: `PVPPRuntime.build_execution_license_from_cycle`

Field	Content
Name and label	PVPPRuntime.build_execution_license_from_cycle — method; pvpp_runtime/kernel.py:4150. PUBLIC / SUPPORTED (rule R1): documented in doc 05 as the licensing step; called directly by two shipped examples, four test files, and one benchmark.
Purpose	Builds the execution license that ε consumes, from one completed canonical cycle, preserving the mode in which Σ selected.
Framework basis	Realization Operator ε , canonical specification v1.7 (C15/D9): ε consumes upstream licensing and may not recompute or repair it.
Signature	build_execution_license_from_cycle(cycle: CanonicalDecisionCycleAssessment, frame: DomainFrame) → ExecutionLicenseEnvelope

Field	Content
Preconditions	The cycle stopped at Σ with a selected policy; its generated policy space is present; Π completeness passed; the governing assessment is present; and a fresh artifact-integrity audit of the cycle is valid. For standard and fallback modes, additionally: a Constraints assessment in which the selected policy is feasible; valid Domain Framing; a frame target for every governing domain; and an Adequacy assessment.
Result	An ExecutionLicenseEnvelope whose selection_mode is standard, recovery_unavailable_fallback, or terminal_infeasibility. entry_authorized is True, True, and False respectively. In fallback mode adequacy_sufficient is False. The runtime records the license as issued.
Invariants	The mode comes from the Σ result, never from the candidate. Standard mode requires the selected policy to be adequate; fallback mode requires the adequate set to be empty; terminal mode requires the constraint-feasible set to be empty. A license is accepted by instantiate_execution only if the runtime issued it and it has not already been instantiated.
Failure behavior	Each unmet precondition or invariant raises ValueError. The message names the condition; the message text is not part of the API (1.7).
Does not	Authorize a call (native authorization is separate, 11.6); decide entry sufficiency on current facts, which the host supplies to instantiate_execution; relabel a fallback policy as adequate; execute anything.
Evidence	tests/test_mode_aware_epsilon_license_v037.py; test_concurrent_recovery_execution_binding_v055.py; test_cycle_artifact_identity_integrity_v064.py; test_v2_authority_bound_callthrough_v0130.py; benchmarks/benchmark_epsilon_mode_license_v037.py; examples/minimal_battery_service; examples/authority_bound_callthrough_v0130.py.
See also	Guide 7.4; Reference 11.2 (primary entry), 11.3, 9.3; compact row for ExecutionLicenseEnvelope.

Table 1.6 shows compact rows for two dataclasses: the license the method above returns, and a change notice from the re-entry surface.

Table 1.6 — Example compact rows

Name and kind	Label	Role and use	Source
ExecutionLicenseEnvelope — frozen dataclass, 13 fields	IMPLEMENTAT ION / SUPPORT (R3)	Upstream licensing record consumed by ϵ . Built by build_execution_license_from_ cycle and build_execution_license; consumed by instantiate_execution.	models.py:19 89

Name and kind	Label	Role and use	Source
GovernanceDependencyChange — frozen dataclass, 8 fields	Assigned in Chapter 12	Notice that registered dependency facts changed. Consumed by <code>derive_dependency_invalidations</code> ; also built inside two other invalidation derivations.	<code>models.py</code> :2469

1.7 Conventions for Signatures, Types, and Statuses

- **Methods.** Shown qualified by their class, without self: `PVPPRuntime.instantiate_execution(episode_id: str, license: ExecutionLicenseEnvelope, *, entry_sufficient: bool, max_steps: int)`.
- **Annotations.** Quoted as the source writes them, including string forward references and the `X | None` form. Collections in stage records are tuples; the Reference does not restate them as lists.
- **Dataclasses.** Fields are listed in declaration order with defaults, and frozen classes are marked. Construct dataclasses with keyword arguments: field order is shown for reading, not as a calling convention.
- **Status values.** Shown in code type exactly as the runtime emits them—for example `recovery_unavailable_fallback`—and never paraphrased. Each status value has one definition, in Chapter 14 and Appendix C.
- **Exceptions.** The exception class and the condition that raises it are API; the message text is not. Applications must not branch on message text.
- **Stage names.** Greek letters in prose (Σ , Π , ϵ); the runtime's own spellings in code and in status fields (for example, a cycle whose `stopped_at` is "Sigma").
- **Notation.** The Reference writes `P_i(t)`, `S_i(t)`, and `O_i(t)`, as the Guide does. Several owner files write `P_l(t)`, `S_l(t)`, and `O_l(t)`; the two forms mean the same thing (Chapter 2).

1.8 Citation Conventions

Table 1.7 — Citation formats

Source	Format	Example
Compendium document	C<folder>/D<n>, with section where useful	C15/D9 (Realization Operator ϵ v1.7)
Project control	C00/D<n> §<section>	C00/D3 §14
Runtime symbol	module:line and qualified name	<code>kernel.py</code> :4150 <code>PVPPRuntime.build_execution_license_from_cycle</code>
Test or benchmark	Path in the frozen tree	<code>tests/test_mode_aware_epsilon_license_v037.py</code>
Proof result	V3.25 Final and the result's label	V3.25 Final, SC-GDM
Guide	Guide and section	Guide 7.4

Source	Format	Example
This Reference	Section, table, or listing number	11.2; Table 1.5

Compendium document numbers follow the tables of contents of the current V2.1 compendia; Appendix F lists them with full titles and versions. Line numbers refer to the frozen tree and are regenerated with the other generated layers.

1.9 Mathematical Standing

Some entries implement properties that the proof program addresses. The Reference states the standing of a result as the proof program gives it and never stronger. It restates no proofs.

Table 1.8 — Standing labels

Label	Meaning	Example
Proved	Established by the proof program (V3.25 Final) under the theorem's explicitly stated assumptions and scope; never unconditional	Results cited by identifier
Proved locally only	Established under local or bounded conditions, not globally	Direction 1 scalar containment (SC-Geo, SC-GDM, SC-X, $K_2 > 0$, SC-1 to SC-5)
Strong but conditional	Established under stated structural conditions that are not shown to hold generally	Direction 2 non-reduction (D2-E, SC-S2, F1 + F2)
Tested	Verified by v0.141 tests for the cases they cover	Regime-family activation (Table 3.6)
Benchmark evidence	Observed in a specified benchmark; the claim is bounded by its specification	Small Economy eight-case program
Open	Stated as open by project control or the proof program	Universal belief-revision and evidence-quality laws (C00/D1 §17)

Two rules follow. Runtime tests never raise the standing of a mathematical result: a passing test shows the code behaves as specified in the cases tested, not that a theorem holds. And an entry that implements a proved property says "implements," cites the result, and states which cases are tested.

1.10 Conformance to Project Control

Before a chapter is released, it is checked against the Unsafe Stale Reference Checklist (C00/D7): the sixteen checks of its §4, the Version 2.1 signals of §10, and the post-v0.141 signals of §12. The checklist exists to stop old or unlicensed architecture from re-entering active documents, and a runtime reference is especially exposed to two of its checks: runtime language replacing the theory stack, and treating every export as supported API. Runtime identifiers name implementations; they never become framework primitives.

Table 1.9 — Checklist items as they apply to the Reference

Checks	The Reference must never
§4: 1–2	Present the old four-operator stack, or a stack without G after H
§4: 3	Treat J or Up as stack operators
§4: 4–5	Describe Σ as the whole selection and filtering system, or put feasibility filtering inside Π
§4: 6–10	Omit the Graph and seed layer, Π completeness, Constraints, Domain Framing, or Adequacy
§4: 11	Describe a later stage as repairing an earlier stage's failure
§4: 12–14	Let applications, memory or cooperation constructs, or runtime language redefine the core
§4: 15	Use a risk score, utility score, or weighted sum as the governing decision rule
§4: 16	Cite an old folder or register as authority
§10: V2.1 signals	Describe the runtime as implementing a V2.1 capability it does not implement; treat replay evidence as current state; treat future capability as present reachability
§12: post-v0.141 signals	Describe v0.131 as the current frozen runtime or call-through execution as future work; treat every exported symbol as supported API; attribute asynchronous sensing, external instrumentation, distributed supervision, or guaranteed mid-flight cancellation to v0.141; treat the runtime build as replacing framework semantic authority

The canonical stack is stated in project control's own form (C00/D2 §2): the complete perceived decision state $P_i(t)$, with the objective interface $O_i(t)$ as an adjacent typed input rather than a stage, followed by $\Phi \rightarrow H \rightarrow G \rightarrow \mathcal{R} \rightarrow$ Graph layer and seed substrate $\rightarrow \Pi \rightarrow \Pi$ completeness validation $\rightarrow$ Constraints $\rightarrow$ Domain Framing $\rightarrow$ Adequacy $\rightarrow \Sigma \rightarrow \varepsilon$. Projection is a shared service used where licensed, not a decision operator.

The front matter and copyright page follow the Document Header and Status Standard (C00/D3).

1.11 Reporting a Defect in This Reference

A defect is any place where the Reference fails its own rules. Defects are logged, not fixed silently.

Table 1.10 — Kinds of defect

Kind	Example
Source mismatch	A signature, field, default, or status that differs from the frozen tree
Boundary error	A host obligation described as runtime behavior, or the reverse
Classification error	A label without recorded evidence, or one that contradicts the register
Standing overstatement	A result stated more strongly than the proof program states it

Kind	Example
Stale reference	Any failure of a check in Table 1.9
Omission	An exported symbol or public method with no primary entry or compact row
Duplication	A symbol described in more than one place

Each log entry records an identifier, the location, the kind, the source evidence, the correction, and the edition that carries the correction. Corrections are published as new editions under project control's change-control rules (C00/D9 §5, §8).

Chapter 2 — Notation and Glossary

This chapter gives every symbol and term the Reference uses in one place. Each entry states the framework meaning, the file that owns it, and the runtime identifier that implements or carries it, where one exists. Definitions here are short by design: the owning file controls the full meaning, and the Guide explains it. The chapter also records where the sources and the code use different names for the same thing, and which reading the Reference adopts.

A reader who meets any symbol or term in the book can find its meaning, its owner, and its runtime name here.

How to read the tables. Owners are cited in the form of Section 1.8 (for example, C15/D8 is document 8 of compendium 15). The runtime column gives the exported name, the field, or the exact string the code uses. A dash means the framework object has no runtime counterpart in v0.141; that is recorded as a fact, not as a defect, unless Chapter 15 lists it as a limitation. Symbols are written with a lowercase *i*, as in the Guide; several owner files write an uppercase *I* with the same meaning (2.8).

2.1 The Canonical Stack

The canonical stack begins from the complete perceived decision state $P_i(t)$, with $O_i(t)$ as an adjacent typed input, and runs $\Phi \rightarrow H \rightarrow G \rightarrow \mathcal{R} \rightarrow \text{Graph layer and seed substrate} \rightarrow \Pi \rightarrow \Pi$ completeness validation $\rightarrow$ Constraints $\rightarrow$ Domain Framing $\rightarrow$ Adequacy $\rightarrow \Sigma \rightarrow \varepsilon$ (C20/D3; C00/D2 §2). Each stage has one owning file, and only that file defines its internal logic (C20/D2). Table 2.1 lists the stages with their one-line roles from the stack reference and the labels v0.141 uses for them.

Table 2.1 — Stages of the canonical stack

Symbol	Role	Owner	Runtime label
$P_i(t)$	Structured perceived decision state, available after perception completes; $PPP_i(t)$ is one component	C10/D19 Perception v0.9	"PPP" (2.8)
$O_i(t)$	Currently applicable qualified objectives; adjacent input, not a stage	C10/D9 Objectives v0.1	—
Φ	Domain-structured pressure representation	C15/D8 Φ v0.7	"Phi"
H	Baseline collapse-horizon estimation	C15/D4 H v1.7	"H"
G	Dependency-closed governing-domain construction	C15/D2 G v0.6	"G"
$\mathcal{R}$	Regime classification from horizons, pressure, and governing structure	C15/D3 $\mathcal{R}$ v0.5	"R"

Symbol	Role	Owner	Runtime label
Graph layer / seed	Reachable instance-transformation substrate for policy generation	C15/D11 Graph v0.4	"Graph/Seed"
Π	Bounded candidate construction from graph-derived seeds	C15/D7 Π v0.8	"Pi"
Π completeness	Hard gate against a materially incomplete candidate space	C15/D12 v0.4	"Pi Completeness"
Constraints	Feasibility filtering only	C15/D5 v0.7	"Constraints"
Domain Framing	Explicit declaration of the preservation target being evaluated	C10/D7 v0.5	"Domain Framing"
Adequacy	Recovery-corridor sufficiency for the framed governing target	C15/D10 v0.8 (2.8)	"Adequacy"
Σ	Non-scalar selection, only after upstream validity conditions are met	C15/D6 Σ v1.14	"Sigma"
ϵ	Bounded execution-side realization and monitored completion control	C15/D9 ϵ v1.7	"epsilon"
T_i	Layer-1 transition; all actual persistent-state change occurs through it	C10/D2 Layer 1 v1.9.4	Host-implemented (Chapter 11)

Two further items appear beside the stack without being stages. The shared policy-conditional projection service (C10/D6) supplies projections where licensed; it is a service, not a decision operator. The runtime also has a joint-recovery feasibility check, labeled "Joint Recovery Feasibility", which runs after Domain Framing and before Adequacy over active recovery obligations only; Section 3.9 states its framework basis.

2.2 State and Value Symbols

The canonical state object and symbol table (C10/D1) owns these symbols. Actual persistent state and perceived decision state are structural crosswalks of each other:

- $S_i(t) = [PP_i(t), SPV_i(t), AVS_i(t), X_i(t)]$ — actual persistent state.
- $P_i(t) = [PPP_i(t), SPVhat_i(t), PVS_i(t), Xhat_i(t)]$ — perceived decision state.

The crosswalks do not make the components commensurable, and none of them needs to be a single number. v0.141 mirrors both structures: ActualPersistentStateEnvelope has fields pp, spv, avs, and context; PerceivedDecisionState has fields ppp, spv_hat, pvs, and x_hat, together with optional retrieval, expectation, confidence, and uncertainty interfaces.

Table 2.2 — State and value symbols

Symbol	Meaning (C10/D1)	Runtime
PV	Productive Value: goods, services, information, effects, or claims conveyed through an event or interaction. Importance alone does not make a variable PV.	Realized bundles in execution records
PP_i(t)	Productive Power: actual capability, under stated conditions, to produce PV	pp; ProductivePowerState
SPV_i(t)	Stored Productive Value: realized PV retained for later use; a holding, not a capability	spv
AVS_i(t)	Actual viability state under current thresholds, margins, reserves, access, commitments, and exposure. Not a utility score.	avs
X_i(t)	Actual context: environmental, relational, institutional, and physical conditions	context
PPP_i(t)	Perceived Productive Power: the agent's estimate of available capability; one component of P _i (t)	ppp
SPV_{hat}_i(t)	Perceived Stored Productive Value, including believed access and deployability	spv_hat
PVS_i(t)	Perceived viability state; may diverge from AVS	pvs
X_{hat}_i(t)	Perceived context	x_hat

Role discipline. PV and PP are role-defined. A thing is PV only when it is conveyed through an event, and PP only when it is actual capability to produce PV. Risk, safety, trust, evidence, authority, access, permission, and harm are not automatically either; they may be context, constraints, evidence, viability conditions, or consequences (C10/D1 §4A). The same object may change roles over time, and each change must be represented explicitly. Entries in this Reference use PV and PP only in these senses.

2.3 Informational, Objective, and History Symbols

These objects are canonical reference objects but are not components of S_i(t). Memory, retrieval, expectation, and confidence do not modify actual state directly, and no Layer-2 operator receives raw memory merely because memory exists (C10/D1 §3–4).

Table 2.3 — Informational, objective, and history symbols

Symbol	Meaning	Owner	Runtime
M_i(t)	Retained memory: the persistent historical informational substrate	C10/D16 Memory Interface	MemoryStateReference
R^M_i(t,q)	Retrieval: a cue-conditioned reconstruction drawn from M _i (t); not the stored memory itself	C10/D16	MemoryRetrievalRequest, MemoryRetrievalPackage

Symbol	Meaning	Owner	Runtime
K_i(t)	Expectation state used in bounded forward projection and expectation learning	C10/D1; projection-expectation interface	ExpectationStateReference
Confidence metadata	Object-local confidence, source, freshness, and similar qualifiers; not truth and not a global scalar	C10/D1 §3	RetrievalQualityMetadata, InformationGovernanceMetadata
TH_i(t)	Transfer history: recorded events, supporting explanation and reconstruction, not current truth	C10/D22 Transfer History v0.2	TransferHistoryEvent, TransferHistoryPackage
O_i(t)	Currently applicable qualified objectives, with source, scope, lifecycle, and provenance	C10/D9	ObjectiveSet, QualifiedObjectiveRecord
Resp(π,o)	Application-owned evaluation of how a candidate relates to an objective; not ranking or selection	C10/D9	ObjectiveResponsivenessRecord

Two symbols are deliberately not canonical: a persistent belief object B_i(t) and a single global confidence state C_i(t) (C10/D1 §8). The Reference does not use either. The confidence field on PerceivedDecisionState is an optional per-state interface, not C_i(t).

2.4 Decision-Cycle Terms

Table 2.4 — Decision-cycle terms

Term	Meaning	Runtime spelling
Pressure	Domain-structured representation of how close each domain is to its threshold (Φ)	PressureFieldAssessment
Collapse horizon	Estimated time to threshold breach. Baseline horizons come from H; policy-conditioned horizons come from projection and must not be swapped with them	HorizonAssessment
Governing domain	A domain in the dependency-closed governing set produced by G; written G _i (t) for the set	GoverningAssessment.governing_domain_ids

Term	Meaning	Runtime spelling
Regime	Classification from horizons, pressure, and governing structure: Mission, Stabilization, Survival, or Existential	"Mission", "Stabilization", "Survival", "Existential"
Policy family	One of four canonical families Π constructs from: continuation (mandatory), local adjustment, structural shift, emergency or escape (C15/D7 §6)	"continuation", "local_adjustment", "structural_shift", "emergency_escape"
Seed	A graph-derived starting point that makes a materially distinct policy family visible	PolicySeedDefinition
Candidate space $\Pi_i(t)$	The bounded set of candidates Π constructs	CandidatePolicySpace
Π completeness	The requirement that no materially relevant reachable family is omitted	PiCompletenessAssessment
Constraint class	Hard, conditional (with a stated relaxation rule), or soft	"hard", "conditional", "soft"
Π_{valid}	The constraint-feasible candidates	ConstraintsAssessment.feasible_policy_ids
Domain frame	The declared preservation target: each governing domain stated as a function to preserve	DomainFrame, DomainFrameTarget
Recovery corridor	A reachable path by which a framed domain can recover	RecoveryCorridorProjection, ActiveRecoveryCorridor
Adequacy	Whether a framed candidate reaches a sustainable recovery-capable region; a judgment, not a ranking	AdequacyAssessment
$\pi_{\text{sel}}(t)$	The policy Σ selects; $\pi^*_{\text{ord}}(t)$ in the ordinary case	SelectedPolicy; selected_policy_id

Term	Meaning	Runtime spelling
Selection mode	Standard; recovery-unavailable fallback; terminal infeasibility	"standard", "recovery_unavailable_fallback", "terminal_infeasibility"
Censored value	A projected value known only to exceed or fall below the projection limit	PolicyProjectionRecord.right_censored_domain_ids

2.5 Execution Terms

The ε owner file defines the realization relation $\varepsilon(\pi_{\text{sel}}(t), \text{context_exec}(t)) = (x_{\text{exec}}(t), E_{\text{exec}}(t, \tau), \text{ExecPath}(t), \text{status_exec}(t), B_{\text{real}}(t), \text{info_exec}(t))$ (C15/D9 §10). The runtime adds the authority objects that carry a selection safely to the host.

Table 2.5 — Execution terms

Term	Meaning	Runtime
x_exec(t)	The instantiated execution episode	instantiate_execution; ExecutionEpisode
E_exec(t, τ)	Bounded within-execution state	EpsilonStepResult
ExecPath(t)	What actually occurred within the licensed corridor	ExecutionTraceNode chain
status_exec(t)	Auditable execution status	"completed", "completed_staged", "partial_realization", "aborted_return", "failed", "emergency_return"
B_real(t)	The realized Productive-Value bundle; not the status and not the path	realized PV bundle in trace records
info_exec(t)	Execution-generated information for authorized update surfaces	information in trace records
Execution license	Record of the upstream licensing ε consumes and may not repair	ExecutionLicenseEnvelope
Binding	Registration of a host callable as the implementation of an action	ExecutionBindingRegistry.register
Native authorization	Single-use, ledger-backed permission for one attempt	NativeExecutionAuthorization
Invocation outcome	Result class of one attempt	"succeeded", "failed_cleanly", "indeterminate", "completed_invalid" in NativeExecutionResult.status

Term	Meaning	Runtime
Control state	Current control posture of an execution context	"ready", "paused", "aborted", "reauthorization_required" in NativeExecutionControlState.state
Layer-1 handoff	The package the host's transition consumes	Layer1TransitionHandoff
Transition provenance	Record linking a transition to the execution that caused it	ExecutionTransitionProvenance
Active execution	A canonical execution attempt registered for supervision	ActiveExecutionRecord (16.3)
Governed fact	The current view of one reported fact, admitted under source authority	GovernedFactView (16.4)
Dependency binding	A declared reliance of one active execution on one governed fact	ExecutionDependencyBinding (16.5)
Governance snapshot	The versions of every fact an execution depends on, at one moment	GovernanceSnapshotToken (16.5)
Continuation posture	The canonical runtime's answer at a checkpoint: continue, reauthorization_required, abort_return, emergency_return, or indeterminate	ContinuationAssessment (16.6)
Supervisory owner and fence	The one supervisor that may fence and request stops for an execution, and its generation number	SupervisoryOwnershipRecord (16.7)
Control, enforcement, effect	A requested stop; what the controller reported; what the world shows	16.7–16.8
Canonical finality	How the ϵ episode ended, recorded once and immutably	CanonicalEpisodeFinality (16.8)

2.6 Information, Change, and Re-entry Terms

Table 2.6 — Information, change, and re-entry terms

Term	Meaning	Runtime
Temporal provenance	When a fact was true (valid time) and when it was recorded or known (record time), kept apart where decision meaning depends on it	InformationTemporalProvenance

Term	Meaning	Runtime
Disposition	Whether information is current or non-current (rejected, superseded, defective, prohibited for use, historically valid but non-current, unresolved, or contradicted). Only current information is ordinary positive evidence; a non-current disposition is preserved, not promoted	InformationGovernanceMetadata.disposition
Evidence authority	Whether an evidence source may support a governance artifact	EvidenceSourceAuthorityState
Transition-relevant state sufficiency	Whether current state carries every causally active consequence of the past that a transition needs	TransitionRelevantStateSufficiencyAssessment
Replay sufficiency	Whether recorded material suffices to reconstruct a decision actually made, using only decision-time information	ReplaySufficiencyAssessment; DecisionReplayPackage
Governance dependency	A registered fact on which a decision artifact depends	GovernanceArtifactDependency
Dependency change	A notice that registered facts changed	GovernanceDependencyChange
Invalidation	A derived signal that an artifact no longer holds	GovernanceInvalidationSignal
Re-entry plan	The earliest affected stage and the results that may be reused	GovernanceReentryPlan; reentry_plan.recompute_from_stage
Reusable artifact	An upstream result that remains valid after a change	GovernanceReusableArtifactBundle
Prediction error	The difference between expectation and observed outcome, routed to authorized updates	PredictionErrorAssessment

2.7 Runtime Stage Labels

v0.141 names stages by strings in stage traces, in the stopped_at field of cycle results, and in two exported tuples. Table 2.7 maps each label to its framework stage.

Table 2.7 — Runtime stage labels

Runtime label	Framework stage	Note
"PPP"	$P_i(t)$ entry	Historical label; the runtime validates the full perceived decision state (2.8)
"Phi", "H", "G", "R"	$\Phi, H, G, \mathcal{R}$	ASCII spellings
"Graph/Seed"	Graph layer and seed substrate	—
"Pi", "Pi Completeness"	Π, Π completeness validation	—
"Constraints", "Domain Framing"	Constraints, Domain Framing	—
"Joint Recovery Feasibility"	Runtime joint-recovery check	Not a canonical stage (2.1, 3.9)
"Adequacy", "Sigma", "epsilon"	Adequacy, Σ, ε	ASCII spellings

CANONICAL_REENTRY_STAGE_ORDER lists all fourteen labels in order, including "Joint Recovery Feasibility". The kernel module's PIPELINE tuple lists thirteen, without it; it is used by the PVPPRuntime.assess path, which is LEGACY / COMPATIBILITY (15.4).

2.8 Notation Differences Across Sources

Where sources differ, the Reference records the difference, states the reading it adopts, and leaves the sources unchanged.

Table 2.8 — Recorded notation differences

Difference	Where	Reading adopted
Subscript written i or I ($P_i(t)$, $P_I(t)$)	Owner files in 15 and 20 mostly use I; the core compendium mixes both; the Guide uses i	Lowercase i throughout; the forms are equivalent
Σ owner version	Proof-program documents cite Σ at v1.11 and v1.13, and the v0.141 standard- Σ docstring cites v1.13; the owner file and ownership map give v1.14	Cite v1.14 as controlling. Review found no change to the ordinary-case rule across v1.11, v1.13, and v1.14; the runtime docstring is stale provenance, not a conformance defect (15.6)

Difference	Where	Reading adopted
Adequacy owner version	The ownership map (C20/D2) names v0.9; compendium 15 carries v0.8 (C15/D10)	v0.8 is current; no v0.9 exists (project owner, September 2026). The map entry is an error (15.6)
First runtime stage labeled "PPP"	Framework: the stack begins from the complete $P_i(t)$, of which PPP is one component	Read the label as the perceived-state entry stage
Two runtime stage tuples	PIPELINE omits "Joint Recovery Feasibility"; CANONICAL_REENTRY_STAGE_ORDER includes it	Table 2.7; the re-entry order is authoritative for re-entry
The letter B	$\mathcal{B}_i(t)$ is a policy-conditioned bundle in Layer 1; $B_{\text{real}}(t)$ is the realized bundle; $B_i(t)$ is a deferred, non-canonical belief object	Write $\mathcal{B}_i(t)$ and $B_{\text{real}}(t)$ in full; never $B_i(t)$
The letter R	$\mathcal{R}$ is the regime operator; $R^M_i(t,q)$ is retrieval; the runtime labels $\mathcal{R}$ as "R"	Write $\mathcal{R}$ for regime and R^M for retrieval in prose
The letter H	H is the horizon operator; transfer history is $TH_i(t)$	Never abbreviate transfer history as H

Chapter 3 — Canonical Architecture and Stage Contracts

This chapter states the canonical stack as a sequence of contracts. For each stage it gives the owner, what the stage consumes and produces, what it may decide, what it is forbidden to decide, how v0.141 implements it, and where the cycle stops if the stage fails. It then covers the whole-cycle entry points and the invariants that hold across stages. Full API entries for the calls named here are in Chapters 4–12; this chapter says what each stage is for and where its authority ends.

A reader must be able to place any runtime behavior in exactly one stage and know what that stage is forbidden to do.

3.1 The Canonical Order

The stack begins from the complete perceived decision state $P_i(t)$, with $O_i(t)$ as an adjacent typed input, and runs $\Phi \rightarrow H \rightarrow G \rightarrow \mathcal{R} \rightarrow \text{Graph layer and seed substrate} \rightarrow \Pi \rightarrow \Pi \text{ completeness validation} \rightarrow \text{Constraints} \rightarrow \text{Domain Framing} \rightarrow \text{Adequacy} \rightarrow \Sigma \rightarrow \epsilon$ (C00/D2 §2; C20/D3). Actual persistent state then changes only through the host's Layer-1 transition T_i (C10/D2). Order is part of the contract: each stage consumes the results of the stages before it and may not repair them. A failure is fixed where it arises, never where it becomes visible.

PVPPRuntime.evaluate_canonical_decision_cycle runs one host-invoked pass through Σ . Its trace follows the canonical order, with four implementation details a reader must know:

- **Objectives.** Objective responsiveness is evaluated after Π completeness passes, over the constructed candidates. It records relations between candidates and objectives; it removes, ranks, and authorizes nothing.
- **Joint recovery.** The active-obligation joint-recovery check is computed after Π completeness, and mandatory current-period recovery actions are bound into the candidates before Constraints. The cycle stops on its result only after Domain Framing, before Adequacy (3.9).
- **Terminal branch.** If Constraints leaves no feasible candidate, the cycle goes directly to terminal Σ over the constructed candidates. Domain Framing and Adequacy are not invoked on that branch.
- **ϵ .** The decision cycle never invokes ϵ and never changes host state. Execution is a separate, explicitly requested step (3.12, Chapter 11).

Before running, the cycle requires three things from the host and registry: a world adapter that supplies the strict Φ and H facts (pressure_factors, pressure_value, and expected_deterioration); a registered governing configuration or recovery-necessity structure; and a registered regime configuration. It also requires a registered "steady" action, from which it projects the baseline continuation used by Φ and H .

3.2 Stage Contract Format

Every contract in this chapter uses the fields of Table 3.1. The "may not decide" field comes from the owner file's own scope statements. The v0.141 field names the call that implements the stage in the canonical cycle; where the cycle calls a private helper, the public entry point with the same contract is named instead.

Table 3.1 — Stage contract fields

Field	Content
Owner	The one file permitted to define the stage's internal logic (C20/D2)
Consumes	Inputs from upstream stages and from the host
Produces	The result downstream stages may rely on
Decides	The questions this stage alone answers
May not decide	Questions that belong to other stages, and repairs it may not make
v0.141	The implementing call and its result type
Stops the cycle when	The condition under which the cycle returns at this stage, and the status it reports

3.3 Φ and H: Pressure and Collapse Horizons

Table 3.2 — Contract for Φ

Field	Contract
Owner	C15/D8, Pressure Field Operator Φ , v0.7
Consumes	Authorized perceived-state evidence for each domain; the baseline continuation
Produces	Domain-local pressure for every registered domain (PressureFieldAssessment)
Decides	How strongly each domain is represented as under strain
May not decide	Collapse horizons, the governing set, regime, candidate policies, or whether a domain will bind behavior. Φ is not a caring weight, utility term, or motivational score.
Invariant	Monotonicity, as C15/D8 states it: holding contextual and propagation terms fixed, if perceived distance to a domain's boundary decreases, represented pressure in that domain must not decrease.
v0.141	evaluate_pressure_field; validate_pressure_invariants checks the invariants
Stops the cycle when	Φ does not stop the cycle. Missing host facts raise errors before any result is returned.

Table 3.3 — Contract for H

Field	Contract
Owner	C15/D4, Collapse Horizon Operator H, v1.7
Consumes	Perceived state, the baseline continuation, and Φ
Produces	The baseline collapse horizon T_i^{base} for each domain (HorizonAssessment)
Decides	How long each domain has under continuation before breaching its threshold

Field	Contract
May not decide	Dominant domain, governing set, urgency ranking, regime, adequacy, or selection
Invariants	Monotonicity, as C15/D4 states it: with the expected net deterioration rate held fixed, a larger horizon-relevant perceived reserve margin gives a strictly longer baseline horizon. The baseline horizon (owned by H) and the policy-conditioned horizon $T_k^{\text{ext}}(t \mid \pi)$ (owned by the projection service) must never be substituted for one another.
v0.141	evaluate_collapse_horizons
Stops the cycle when	H does not stop the cycle.

3.4 G and $\mathcal{R}$: Governing Domains and Regime

Table 3.4 — Contract for G

Field	Contract
Owner	C15/D2, Governing Domain Identification Operator G, v0.6
Consumes	Baseline horizons for every registered domain; registered recovery-necessity structure
Produces	The dependency-closed governing set $G_i(t)$ (GoverningAssessment.governing_domain_ids)
Decides	Which domains must be preserved if recoverable viability is to remain possible
May not decide	Policy evaluation, ranking, or regime. Domain devaluation must not shrink G, and pressure that fails to bind behavior does not alter membership.
v0.141	identify_governing_domains: a horizon seed (every domain within the governing tolerance of the shortest horizon), then minimal closure over registered recovery necessities. Pressure, policy ranking, registration order, and domain labels cannot change membership.
Stops the cycle when	G does not stop the cycle. A horizon set that omits a registered domain or contains NaN raises an error.

Table 3.5 — Contract for $\mathcal{R}$

Field	Contract
Owner	C15/D3, Regime Classification Operator $\mathcal{R}$, v0.5
Consumes	Horizons and pressure over the governing set; $G_i(t)$ as supplied; the previous regime, if the host supplies it
Produces	A regime—Mission, Stabilization, Survival, or Existential—with the trigger that produced it (RegimeAssessment)
Decides	The behavioral posture that shapes which policy families Π generates

Field	Contract
May not decide	Governing membership, feasibility, adequacy, or selection. $\mathcal{R}$ does not treat local indifference as non-governance.
v0.141	classify_regime. Triggers are tested from most to least severe using registered thresholds on the shortest governing horizon, the highest governing pressure, and the number of governing domains; a contextual interruption forces Existential. Hysteresis uses only the previous regime the host passes in: a materially worsening horizon never de-escalates, de-escalation needs a registered improvement margin, and no hysteresis rule suppresses an Existential trigger.
Stops the cycle when	$\mathcal{R}$ does not stop the cycle. An empty governing set or a missing regime configuration raises an error.

Regime shapes Π only by activating policy families. Table 3.6 shows the activation v0.141 implements. A seed that is structurally required, or that belongs to a materially required policy class, is kept whatever its family. Under Survival and Existential, optional non-continuation seeds that touch no governing domain may be suppressed.

Table 3.6 — Policy families activated by regime in v0.141

Regime	Active families
Mission	continuation, local_adjustment
Stabilization	continuation, local_adjustment, structural_shift
Survival	continuation, local_adjustment, structural_shift, emergency_escape
Existential	continuation, local_adjustment, structural_shift, emergency_escape

The framework states the pattern in words: Mission favors continuation and local adjustment; Stabilization adds selected structural alternatives; Survival expands structural and escape families materially; Existential activates maximal bounded expansion across all families (C15/D7 §6). v0.141 implements Survival and Existential with the same family set. They differ in their triggers and in how far construction expands, not in which families are active.

3.5 Graph, Seed, and Π : Policy Construction

Table 3.7 — Contract for the Graph layer and seed substrate

Field	Contract
Owner	C15/D11, Instance-Transformation Graph Layer, v0.4; construction protocol C15/D14
Consumes	The preliminary preservation object, regime, governing set, and registered graph structure
Produces	A reachable instance-transformation substrate and the seeds Π builds from (GraphConstructionAssessment)
Decides	Which transformations are structurally realizable and reachable

Field	Contract
May not decide	Adequacy or selection. Structurally unrealizable transformations must not enter the graph; a transformation must not be excluded merely because it looks difficult. Graph validity does not guarantee Π completeness.
v0.141	construct_graph
Stops the cycle when	The graph status is anything other than PASS or CONDITIONAL PASS. The cycle returns at "Graph/Seed".

Table 3.8 — Contract for Π

Field	Contract
Owner	C15/D7, Policy Construction Operator Π , v0.8
Consumes	A validated graph handoff; the materially required policy classes the host declares
Produces	The bounded candidate space $\Pi_i(t)$ (PiConstructionAssessment, CandidatePolicySpace)
Decides	Which candidates exist, built from the four canonical families, with continuation mandatory
May not decide	Adequacy or selection. Π may remove strict duplicates and cosmetic variants, but must not run horizon dominance, adequacy-style comparison, or projection ranking; tradeoff policies and materially distinct variants must be kept.
v0.141	construct_pi_from_graph. Graph families map onto the four Π families; for example, substitution and exit are structural_shift, and escape is emergency_escape.
Stops the cycle when	No policy space is produced, for example when an invalid graph handoff blocks construction (pi_blocked_by_graph_invalid). The cycle returns at "Pi".

3.6 Π Completeness

Table 3.9 — Contract for Π completeness validation

Field	Contract
Owner	C15/D12, Π Completeness Validation Rule, v0.4
Consumes	The candidate space and its declared materially required classes
Produces	A pass or a failure naming the missing classes (PiCompletenessAssessment)
Decides	Whether the candidate space represents every materially required class
May not decide	Which classes are required—the constructor and graph layer declare them—or feasibility, adequacy, framing, or ranking. Bounded generation does not excuse omission of a structurally indicated class.

Field	Contract
v0.141	validate_pi_completeness; qualify_pi_completeness_for_substrate where the substrate is contaminated
Stops the cycle when	A required class is missing (pi_completeness_failed_missing_material_policy_classes). The cycle returns at "Pi Completeness". A completeness failure is a construction problem, not a selection problem: no later stage may absorb it.

3.7 Constraints

Table 3.10 — Contract for Constraints

Field	Contract
Owner	C15/D5, Constraint System, v0.7
Consumes	The candidates, with any mandatory recovery actions bound in; perceived constraint evidence; regime and governing set
Produces	The constraint-feasible set Π_{valid} and a structured violation profile per candidate (ConstraintsAssessment)
Decides	Feasibility, under the hard, conditional, and soft taxonomy, with necessity-driven relaxation of conditional constraints
May not decide	Adequacy, framing, or selection. Σ consumes the feasible set and must not relax constraints retroactively.
v0.141	evaluate_constraint_system; constraint evidence comes from the host through ConstraintEvidenceAdapter
Stops the cycle when	Constraints does not stop the cycle. An empty Π_{valid} sends the cycle to terminal Σ (3.11).

3.8 Domain Framing

Table 3.11 — Contract for Domain Framing

Field	Contract
Owner	C10/D7, Viability Domain Framing Rule, v0.5
Consumes	The host's domain frame; the governing set from G
Produces	A validated preservation target: each governing domain paired with the function to preserve (DomainFramingAssessment)
Decides	What exactly is being preserved in each governing domain
May not decide	Adequacy or selection, and it may not recompute $P_i(t)$, Φ , H , or G . Framing names a function, not a mechanism, unless the mechanism is itself non-substitutable in the viability structure (C10/D7).

Field	Contract
v0.141	Validated inside the canonical cycle against the canonical G. The frame must cover G exactly; each target must name a registered domain and a function, give its basis as "function" or "non_substitutable_mechanism", and be locked. The public <code>validate_domain_framing</code> applies the same rules but computes G by the prototype path, so it is not the cycle's check (9.13; Chapter 15).
Stops the cycle when	The frame is invalid (<code>domain_framing_failed</code>). The cycle returns at "Domain Framing".

3.9 Joint-Recovery Check

The framework makes joint sustainment across the governing set part of Adequacy: a policy is not adequate if one governing domain recovers by destroying another, or if relief never adds up to joint sustainment (C15/D10 §7.3). v0.141 adds a runtime check that protects a precondition of that judgment. Before judging any candidate, it asks whether the recovery obligations already active can all be met together under the shared capacity and deadlines the host has represented. The check covers active obligations only. Whether a given candidate sustains the governing set jointly is a projection fact, which Adequacy judges.

Table 3.12 — Contract for the joint-recovery check

Field	Contract
Basis	Adequacy's joint-sustainment requirement (C15/D10 §7.3). The check is a runtime stage label, "Joint Recovery Feasibility", not a canonical stack stage.
Consumes	The runtime's active recovery corridors, whose identity and deadlines the runtime owns; the capacity demands registered on their remaining actions; the <code>CapacityCalendar</code> the host supplies for this decision
Produces	Whether the active set is jointly feasible, with a witness when it is not; mandatory current-period recovery actions bound into candidates
May not decide	Which obligation to sacrifice. v0.141 uses no queue, registration order, pressure ranking, or sacrifice rule, because the canonical material supplies no universal sacrifice rule. A host projector may not substitute a Boolean for the runtime's computation.
v0.141	<code>assess_recovery_feasibility</code> ; the joint-feasibility assessment in the feasibility submodule
Stops the cycle when	Active obligations carry capacity demands but no <code>CapacityCalendar</code> was supplied (<code>joint_recovery_capacity_authority_required</code>), or the active set is jointly infeasible (<code>joint_recovery_feasibility_failed</code>). The cycle returns at "Joint Recovery Feasibility", after Domain Framing and before Adequacy.

3.10 Adequacy

Table 3.13 — Contract for Adequacy

Field	Contract
Owner	C15/D10, Restoration Adequacy Operator, v0.8 (the ownership map's v0.9 is an error; Table 2.8)
Consumes	The validated frame; the governing set; one authoritative projection record $Q_t(\pi)$ for each constraint-feasible candidate
Produces	The adequate subset, with per-domain results and reasons (AdequacyAssessment)
Decides	Whether a framed candidate preserves viability until recovery is reachable, enters a recovery-capable region, and reaches joint sustainment across the governing set
May not decide	Selection or ranking. Adequacy must not compensate for graph or Π omission, repair a failed completeness gate, or reinterpret the frame.
v0.141	evaluate_restoration_adequacy. Adequacy is conjunctive over G and uses only the projection records; it invents no trajectory, recovery region, substitution, closure, or reopening. The cycle checks that Adequacy did not alter any record.
Stops the cycle when	Adequacy does not stop the cycle. An empty adequate set sends the cycle to recovery-unavailable fallback Σ .

3.11 Σ : Standard, Fallback, and Terminal Selection

Σ selects only after every upstream validity condition has been settled. It has three modes, and the mode is fixed by what reached it, never chosen by Σ .

Table 3.14 — Σ modes in the canonical cycle

Mode	Entered when	Operates over	v0.141
Standard	The adequate set is non-empty	The adequate set	evaluate_sigma_standard; status sigma_standard_policy_selected
Recovery-unavailable fallback	Π_{valid} is non-empty but no candidate is adequate	Π_{valid}	evaluate_sigma_fallback
Terminal infeasibility	Π_{valid} is empty	The constructed candidates	evaluate_sigma_terminal; status sigma_terminal_policy_selected

Table 3.15 — Contract for Σ

Field	Contract
Owner	C15/D6, Σ Selection Operator, v1.14; fallback parameterization addendum C15/D17
Consumes	The set its mode operates over; the governing set; the projection records, including censoring flags
Produces	One selected policy $\pi_{\text{sel}}(t)$, with the stage at which it was resolved
Decides	Standard mode: Stage 1 keeps the governing-domain non-dominated frontier; Stage 2 refines by global dominance over all active domains; Stage 3 breaks remaining ties by a fixed, state-independent order index. All horizon comparisons are censor-aware.
May not decide	Governing membership, completeness, feasibility, framing, or adequacy. Σ repairs no upstream omission and launders no upstream invalidity. The Stage-3 order is a tie-break, not a preference.
v0.141	In terminal mode, Σ applies strict class priority—hard, then conditional, then soft—over structured violation profiles supplied upstream; improvement in a lower class never compensates for worsening in a higher one. The fallback implementation resolves singleton maximal sets and two-candidate non-dominated conflicts; for an unresolved conflict among three or more candidates it stops rather than inventing selection semantics (15.1). The standard-mode docstring cites Σ v1.13; v1.14 controls, and the ordinary-case rule is unchanged between them (Table 2.8, 15.6).
Stops the cycle when	Σ is the last stage of the decision cycle; the cycle returns at "Sigma".

3.12 ϵ and Layer 1: From Selection to Transition

Table 3.16 — Contract for ϵ and the Layer-1 transition

Field	Contract
Owner	C15/D9, Realization Operator ϵ , v1.7; Layer-1 transition C10/D2
Consumes	The selected policy with its upstream licensing; host facts about execution
Produces	ϵ : the realization outputs x_{exec} , E_{exec} , ExecPath , $\text{status}_{\text{exec}}$, B_{real} , and $\text{info}_{\text{exec}}$ for the execution bridge. ϵ does not itself perform state transition (C15/D9). Layer 1, owned by the host, consumes these outputs and performs the authoritative transition $S_i(t+1) = T_i(S_i(t), B_i(t))$ (C10/D2).
Decides	The permitted execution-side realization behavior within the licensed corridor, and the execution status it reports. It does not determine authoritative environmental effects or actual-state change.
May not decide	ϵ must not override or redo Σ , redefine governing domains, repair framing or adequacy, rank alternatives, or treat execution success as validating an invalid upstream path. Σ selects; ϵ realizes.

Field	Contract
v0.141	The execution lineage of Chapter 11, from <code>build_execution_license_from_cycle</code> to transition provenance. v0.141 is synchronous: it gates before a call and does not guarantee cancellation once a registered callable has begun (1.4).
Stops	Terminal-infeasibility selections are preserved for traceability, but entry to ϵ is blocked.

3.13 Whole-Cycle Forms

v0.141 offers several ways to run the cycle. None of them is an autonomous loop: every pass is invoked by the host, and every sequence is authored by the host in advance.

Table 3.17 — Whole-cycle entry points

Entry point	What it does
prepare_canonical_cycle_snapshot	Asks the host to expose a represented Layer-2 state for one actual state. An integration boundary, not a perception operator; the runtime never opens actual state itself.
evaluate_canonical_decision_cycle	One pass through Σ , as described in 3.1. ϵ is not invoked.
evaluate_integrated_canonical_cycle	One explicit pass that stops at Σ by default. With <code>execute=True</code> , the host requests ϵ and, if ϵ terminates, the registered Layer-1 transition. No retry, heartbeat, scheduling, or persistence.
evaluate_canonical_cycle_sequence	A finite, host-authored sequence for integration stress testing. The runtime threads validated Layer-1 state forward and records cross-cycle invariants.
prepare_memory_conditioned_cycle_snapshot	Builds a perception snapshot through licensed retrieval only: memory goes to the retrieval service, and perception receives $R^*M_i(t,q)$ and optionally $K_i(t)$, never raw $M_i(t)$.
evaluate_memory_conditioned_decision_cycle	The canonical cycle run from a memory-conditioned snapshot
evaluate_memory_conditioned_integrated_cycle	One retrieval-conditioned pass through an optional memory and expectation update; it adds no memory semantics, learning, persistence, or cycling
evaluate_memory_conditioned_cycle_sequence	A finite host-authored retrieval, decision, execution, and update sequence

3.14 Cross-Cutting Invariants

These are framework-level invariants that hold across stages. Table 3.18 names, row by row, the v0.141 check or structural mechanism that implements or tests each one where such enforcement exists; no blanket enforcement claim is made.

Table 3.18 — Cross-cutting invariants

Invariant	Framework source	v0.141 enforcement
No stage repairs an upstream failure	C00/D2 §2; owner scope statements	The cycle returns at the first failed gate
Projections are computed once and shared	C10/D6; C15/D6 §3.3	One record per feasible candidate; Adequacy and Σ are checked not to alter it
Objectives do not remove, rank, or authorize	C10/D9; C20/D3	Responsiveness is recorded, not used as a gate or selector
No scalar aggregation governs selection	C15/D6; C00/D7 §4 check 15	Σ uses dominance and a fixed tie-break order
Memory reaches decisions only through licensed retrieval	C10/D1 §4	Memory-conditioned snapshots pass R^M and K , never raw M
Actual state changes only through T_i	C10/D2	The runtime never writes host state; the host transition does
Cycle artifacts belong to one state	Supports C10/D6 (one shared record set)	An integrity audit runs before the cycle result is returned

Chapter 4 — Runtime Object, Registry, and Configuration

This chapter documents the entry points every application touches first: the runtime object, the registry and its registration surface, the identity fingerprints the registry computes, configuration provenance, and architecture-adaptation invariance. It is the first chapter with classified entries. Its labels come from the export-surface audit, whose rules and working record are in Appendix H; Section 4.12 lists every label assigned here with the rule it rests on.

Naming. In this chapter and through Chapter 14, v0.141 denotes the decision core that v2.1.167 carries unchanged (1.4), not a different installed package.

A reader must be able to construct a runtime and registry correctly and know what each registration commits the runtime to—and what it does not.

Generated fields. In every entry below, the name line, signature, argument-record fields, and evidence counts are generated from the frozen tree by the audit scripts and regenerated for each build. Evidence counts are the number of files in tests/, benchmarks/, and examples/ that call the symbol directly.

4.1 The Runtime Object

PVPPRuntime is the object through which an application runs cycles, crosses the execution boundary, and applies re-entry. It holds references to the registry, the host world adapter, and any optional services the application supplies. It also keeps runtime-private ledgers—issued execution licenses, execution episodes, and native authorizations—whose object identity carries authority (Chapter 11). Applications never read or write these ledgers.

Entry — PVPPRuntime

Field	Content
Name and label	PVPPRuntime — class; kernel.py:19. PUBLIC / SUPPORTED (rule R1).
Purpose	Creates the runtime that evaluates canonical cycles over a registry, through a host world adapter and optional host services.
Framework basis	The canonical stack (C20/D3) as implemented by v0.141; host services implement the application/adaptor boundary (C30/D3).
Signature	PVPPRuntime(registry: PVPPRegistry, world: WorldAdapter, projection_service: PolicyProjectionService None = None, layer1_transition_service: Layer1TransitionService None = None, memory_retrieval_service: MemoryRetrievalService None = None, epistemic_update_service: EpistemicUpdateService None = None, capacity_authority_service: CapacityAuthorityService None = None, prediction_error_service: PredictionErrorService None = None, transfer_history_service: TransferHistoryService None = None)
Preconditions	The registry contains an action with id "steady"; the runtime projects the baseline continuation from it. The world object implements WorldAdapter. WorldAdapter is not exported at the top level; import it from pvpp_runtime.interfaces (H.5-1). Canonical cycles additionally need the strict Φ/H world methods (3.1).

Field	Content
Result	A runtime with no active recovery corridors and empty private ledgers.
Invariants	The runtime never holds actual persistent state and never writes host state. Each service is optional at construction; a call that needs a missing service fails when it is made, not here.
Failure behavior	Raises ValueError("registered steady action required") if the registry has no "steady" action.
Does not	Validate the world adapter, copy the registry, or claim general thread safety. v0.141 serializes only native-authorization issuance with a lock.
Evidence	Direct use: test files 49, benchmarks 42, example files 14; named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.
See also	Guide 8.3; Table 4.1; Chapter 10 for each protocol.

Table 4.1 — Services accepted by the runtime constructor

Parameter	Protocol	Used by
world	WorldAdapter (required)	Perception, Φ/H facts, constraints, and Σ facts (9.6, 10.2)
projection_service	PolicyProjectionService	Authoritative $Q_t(\pi)$ records for Adequacy and Σ (9.10, 10.3)
layer1_transition_service	Layer1TransitionService	The host transition T_i (10.4, 11.10)
memory_retrieval_service	MemoryRetrievalService	Licensed retrieval $R^M_i(t,q)$ (6.2)
epistemic_update_service	EpistemicUpdateService	Post-execution epistemic update (6.6)
capacity_authority_service	CapacityAuthorityService	Capacity authority (5.7)
prediction_error_service	PredictionErrorService	Prediction error (6.5)
transfer_history_service	TransferHistoryService	Transfer history (6.3)

Three of these protocols—WorldAdapter, MemoryRetrievalService, and EpistemicUpdateService—are defined in `pvpn_runtime.interfaces` but are not exported from the package's top level, although the constructor accepts them. The audit records this in H.5-1. Chapter 10 documents all eighteen protocols the interfaces module defines.

4.2 The Registry

Entry — PVPPRegistry

Field	Content
Name and label	PVPPRegistry — class; registry.py:7. PUBLIC / SUPPORTED (rule R1).

Field	Content
Purpose	Holds the registered structure a runtime reads: domains, powers, actions, constraints, recovery plans and necessities, stage configurations, policy seeds, graph structure, and the Σ order.
Framework basis	Registered structure is the architecture-visible input to G , $\mathcal{R}$, Graph, Π , Constraints, and Σ (C15 owner files).
Signature	PVPPRegistry() $\rightarrow$ None
Preconditions	None.
Result	An empty registry.
Invariants	Registration records structure; it decides nothing. Every register method rejects a duplicate identifier and any reference to an unregistered domain, action, instance, or transformation.
Failure behavior	Register methods raise ValueError with a message naming the problem.
Does not	Evaluate, rank, or authorize anything. The registry is shared by reference with the runtime, so a later registration changes what later cycles see and changes the identities of 4.8.
Evidence	Direct use: test files 59, benchmarks 41, example files 13; named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.
See also	Table 4.2; 4.3–4.7.

Because references are checked when a record is registered, registration order matters. Table 4.2 gives the dependencies.

Table 4.2 — Registration dependencies

Register	Requires already registered
Domain	Nothing
Action; productive power; recovery necessity; domain relation; dependency requirement	The domains it names
Recovery plan	Its domain, trigger action, and continuation actions
Policy seed	Its actions and affected domains
Graph instance	Its domains
Graph transformation	Its source and destination instances, its action, and its affected domains
Graph composition	Its transformations
Configurations; Σ order; constraint rule	Nothing
Runtime construction	The "steady" action

4.3 Domains and Domain Relations

Entry — PVPPRegistry.register_domain

Field	Content
Name and label	PVPPRegistry.register_domain — method; registry.py:28. PUBLIC / SUPPORTED (rule R1).
Purpose	Registers a viability domain with its failure threshold.
Framework basis	Viability domains and thresholds (C10/D1 §2; C15/D4).
Signature	PVPPRegistry.register_domain(domain)
Argument record	DomainDefinition: frozen dataclass; id: str; description: str; threshold: float; governing_horizon: float None = None; dependencies: tuple[str, ...] = ()
Preconditions	The domain identifier is new.
Result	None. The domain becomes visible to Φ , H, G, and every later registration that names it.
Invariants	Domains are identified by id; G requires horizons for every registered domain (3.4).
Failure behavior	Raises ValueError if the identifier is already registered.
Does not	Declare a domain governing. Governing membership is computed by G each cycle.
Evidence	Direct use: test files 44, benchmarks 33, example files 13; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.
See also	Guide 6.3; 3.3–3.4.

Two further registrations, register_domain_relation and register_dependency_requirement, record prerequisite structure between domains. Their docstrings call them prototypes. A domain relation must have relation_type "dependency_floor"; a dependency requirement uses mode "all_of" or "any_of". In v0.141 both are consumed only by the policy-space evaluators (evaluate_policy_space, evaluate_recovery_policy_space, evaluate_framed_policy_space, and evaluate_policy_sets), not by evaluate_canonical_decision_cycle, where G's closure uses recovery necessities instead (4.5). Examples still use them. The evidence conflicted, so both were first labeled IMPLEMENTATION / SUPPORT by default (H.5-2). Chapter 15 settled the entry: all their consumers are LEGACY / COMPATIBILITY, so they are too. They appear as compact rows in 4.12.

4.4 Actions and Productive Powers

Entry — PVPPRegistry.register_action

Field	Content
Name and label	PVPPRegistry.register_action — method; registry.py:35. PUBLIC / SUPPORTED (rule R1).

Field	Content
Purpose	Registers an action that candidates, recovery plans, graph transformations, and execution bindings refer to.
Framework basis	Actions are the executable units of candidate policies (C15/D7); execution binds them to host callables (C15/D9).
Signature	PVPPRegistry.register_action(action)
Argument record	ActionDefinition: frozen dataclass; id: str; description: str; domains: tuple[str, ...]; metadata: Mapping[str, Any] = (empty)
Preconditions	The action identifier is new; every domain in domains is registered. One action must have id "steady" before a runtime is constructed.
Result	None.
Invariants	metadata is free-form but read by the runtime for specific keys; for example, capacity_demands is read by the joint-recovery check (3.9).
Failure behavior	Raises ValueError if the identifier is already registered. Raises ValueError if the action names unregistered domains.
Does not	Bind the action to an implementation (11.5) or make it feasible.
Evidence	Direct use: test files 55, benchmarks 43, example files 13; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.
See also	Guide 6.4 and 8.3 (the "steady" action); 11.5.

Entry — PVPPRegistry.register_power

Field	Content
Name and label	PVPPRegistry.register_power — method; registry.py:31. PUBLIC / SUPPORTED (rule R1).
Purpose	Registers a productive power: a capability in one domain that world states report values for.
Framework basis	Productive Power PP _i (t) (C10/D1 §2).
Signature	PVPPRegistry.register_power(power)
Argument record	ProductivePowerDefinition: frozen dataclass; id: str; domain_id: str; description: str; persistent: bool = True; metadata: Mapping[str, Any] = (empty)
Preconditions	The power identifier is new and its domain is registered.
Result	None.
Invariants	A power belongs to exactly one domain.
Failure behavior	Raises ValueError if the identifier is already registered. Raises ValueError if the domain is unregistered.

Field	Content
Does not	Hold the power's value. Values arrive in WorldState.powers from the host (Chapter 5).
Evidence	Direct use: test files 3, benchmarks 3, example files 12; not named in doc 05; e.g. minimal_battery_service.py, run_branching_dependency.py (legacy tiny_* example, 15.4).
See also	Chapter 5 (WorldState, ProductivePowerState).

4.5 Constraints, Recovery Plans, and Recovery Necessities

Entry — PVPPRegistry.register_constraint_rule

Field	Content
Name and label	PVPPRegistry.register_constraint_rule — method; registry.py:423. PUBLIC / SUPPORTED (rule R1).
Purpose	Registers a feasibility rule and its class.
Framework basis	Constraint System v0.7: hard, conditional, and soft classes with necessity-driven relaxation (C15/D5).
Signature	PVPPRegistry.register_constraint_rule(rule: ConstraintRuleDefinition)
Argument record	ConstraintRuleDefinition: frozen dataclass; id: str; constraint_class: str; conditional_class: str = ''; description: str = ''; notes: tuple[str, ...] = ()
Preconditions	The rule identifier is new. constraint_class is "hard", "conditional", or "soft". A conditional rule names its conditional_class; other classes must leave it empty.
Result	None.
Invariants	The class is fixed at registration; Constraints evaluates it each cycle from host evidence.
Failure behavior	Raises ValueError if the identifier is already registered. Raises ValueError on an unknown class or a misplaced conditional_class.
Does not	Evaluate the rule. Observations come from the host through the world adapter's constraint_profile hook (10.2).
Evidence	Direct use: test files 9, benchmarks 7, example files 2; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.
See also	3.7; Guide 6.8.

Entry — PVPPRegistry.register_recovery_plan

Field	Content
Name and label	PVPPRegistry.register_recovery_plan — method; registry.py:40. PUBLIC / SUPPORTED (rule R1).

Field	Content
Purpose	Registers a recovery plan: the trigger action that opens a recovery corridor for a domain's function, the actions that continue it, and its deadline.
Framework basis	Recovery corridors and obligations (C15/D10).
Signature	PVPPRegistry.register_recovery_plan(plan: RecoveryPlanDefinition)
Argument record	RecoveryPlanDefinition: frozen dataclass; id: str; domain_id: str; function_id: str; trigger_action: str; continuation_actions: tuple[str, ...]; deadline_offset: float
Preconditions	The plan identifier is new; its domain, trigger action, and continuation actions are registered.
Result	None. The plan is indexed by its trigger action.
Invariants	A plan belongs to one domain and one function.
Failure behavior	Raises ValueError if the identifier is already registered. Raises ValueError naming any unregistered domain or action.
Does not	Activate a corridor. Corridors become active through execution bookkeeping (11.11).
Evidence	Direct use: test files 3, benchmarks 4, example files 5; not named in doc 05; e.g. minimal_battery_service.py, run_crop.py (legacy tiny_* example, 15.4).
See also	3.9; Guide 6.10.

Entry — PVPPRegistry.register_recovery_necessity

Field	Content
Name and label	PVPPRegistry.register_recovery_necessity — method; registry.py:81. PUBLIC / SUPPORTED (rule R1).
Purpose	Registers the structural fact that one domain is strictly necessary for another's recovery.
Framework basis	G's dependency closure (C15/D2).
Signature	PVPPRegistry.register_recovery_necessity(relation: RecoveryNecessityDefinition)
Argument record	RecoveryNecessityDefinition: frozen dataclass; id: str; support_domain_id: str; target_domain_id: str; basis: str = 'registered_recovery_necessity'; notes: tuple[str, ...] = ()
Preconditions	The identifier is new; both domains are registered and are distinct.
Result	None.
Invariants	Whenever the target domain governs, G adds the support domain, transitively. Recovery necessities enter the governing substrate identity (4.8).

Field	Content
Failure behavior	Raises ValueError if the identifier is already registered. Raises ValueError for unregistered or identical domains.
Does not	Make either domain governing on its own account.
Evidence	Direct use: test files 2, benchmarks 1; not named in doc 05; e.g. benchmark_governing_operator_v027.py.
See also	3.4; Chapter 8 for admission and revision after construction.

4.6 Stage Configurations

Stage configurations make the thresholds and tolerances of G , H , $\mathcal{R}$, fallback Σ , and the Σ tie-break architecture-visible. Three of them—governing, regime, and fallback—may be registered only once. The horizon configuration is different: a later registration silently replaces an earlier one.

Entry — `PVPPRegistry.register_governing_configuration`

Field	Content
Name and label	<code>PVPPRegistry.register_governing_configuration</code> — method; registry.py:74. PUBLIC / SUPPORTED (rule R1).
Purpose	Sets G 's horizon proximity tolerance.
Framework basis	G 's horizon seed (C15/D2).
Signature	<code>PVPPRegistry.register_governing_configuration(config: GoverningConfiguration)</code>
Argument record	GoverningConfiguration: frozen dataclass; epsilon_h: float = 0.0
Preconditions	No governing configuration is registered; epsilon_h is not negative.
Result	None.
Invariants	Without a registration, G uses the default epsilon_h = 0.0. The canonical cycle requires either this registration or at least one recovery necessity (3.1).
Failure behavior	Raises ValueError if already registered or if epsilon_h is negative.
Does not	Change membership rules; only the seed tolerance.
Evidence	Direct use: test files 11, benchmarks 8, example files 2; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.
See also	3.4.

Entry — `PVPPRegistry.register_horizon_configuration`

Field	Content
Name and label	<code>PVPPRegistry.register_horizon_configuration</code> — method; registry.py:128. PUBLIC / SUPPORTED (rule R1).

Field	Content
Purpose	Sets H's numerical stabilizer for deterioration rates.
Framework basis	Baseline collapse horizon (C15/D4).
Signature	PVPPRegistry.register_horizon_configuration(config: HorizonConfiguration)
Argument record	HorizonConfiguration: frozen dataclass; epsilon_deterioration: float = 1e-12
Preconditions	epsilon_deterioration is positive.
Result	None. Replaces any earlier horizon configuration.
Invariants	Without a registration, H uses the default epsilon_deterioration = 1e-12.
Failure behavior	Raises ValueError if epsilon_deterioration is not positive.
Does not	Reject a second registration, unlike the other configurations.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_phi_h_v028.py.
See also	3.3.

Entry — PVPPRegistry.register_regime_configuration

Field	Content
Name and label	PVPPRegistry.register_regime_configuration — method; registry.py:92. PUBLIC / SUPPORTED (rule R1).
Purpose	Sets $\mathcal{R}$'s thresholds and hysteresis margins.
Framework basis	Regime Classification Operator $\mathcal{R}$ (C15/D3).
Signature	PVPPRegistry.register_regime_configuration(config: RegimeConfiguration)
Argument record	RegimeConfiguration: frozen dataclass; tau_h_existential: float; tau_h_survival: float; tau_h_stabilization: float; tau_phi_existential: float; tau_phi_survival: float; tau_phi_stabilization: float; tau_m_survival: int = 2; tau_m_stabilization: int = 2; tau_h_mult_survival: float None = None; tau_h_mult_stabilization: float None = None; delta_h_escalation: float = 0.0; delta_h_deescalation: float = 0.0
Preconditions	No regime configuration is registered, and the thresholds are ordered: horizon thresholds existential < survival < stabilization; pressure thresholds existential > survival > stabilization; multiplicity thresholds at least 2, survival not below stabilization; multiplicity-compression horizons no greater than their horizon thresholds; hysteresis margins not negative.
Result	None.
Invariants	The canonical cycle requires a regime configuration (3.1).
Failure behavior	Raises ValueError if already registered or if any ordering condition fails.
Does not	Classify anything; $\mathcal{R}$ applies the thresholds each cycle (3.4).

Field	Content
Evidence	Direct use: test files 11, benchmarks 7, example files 2; not named in doc 05; e.g. <code>authority_bound_callthrough_v0130.py</code> , <code>minimal_battery_service.py</code> .
See also	3.4; Guide 6.6.

Entry — `PVPPRegistry.register_fallback_configuration`

Field	Content
Name and label	<code>PVPPRegistry.register_fallback_configuration</code> — method; <code>registry.py:408</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Sets the override parameter of recovery-unavailable fallback Σ .
Framework basis	Σ fallback parameterization addendum (C15/D17).
Signature	<code>PVPPRegistry.register_fallback_configuration(config: FallbackConfiguration)</code>
Argument record	<code>FallbackConfiguration</code> : frozen dataclass; <code>alpha_base</code> : float = 0.05
Preconditions	No fallback configuration is registered; <code>alpha_base</code> lies strictly between 0 and 1.
Result	None.
Invariants	The default <code>alpha_base</code> = 0.05 comes from the addendum's initial frozen parameterization.
Failure behavior	Raises <code>ValueError</code> if already registered or if <code>alpha_base</code> is out of range.
Does not	Extend fallback beyond the cases v0.141 resolves (15.1).
Evidence	Direct use: test files 1, benchmarks 2; not named in doc 05; e.g. <code>three_domain_buffered_multistep_scenario_v048.py</code> , <code>three_domain_oscillation_scenario_v047.py</code> .
See also	3.11.

Entry — `PVPPRegistry.register_sigma_order`

Field	Content
Name and label	<code>PVPPRegistry.register_sigma_order</code> — method; <code>registry.py:415</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Assigns a policy its position in the fixed Stage-3 tie-break order.
Framework basis	Σ Stage 3: a fixed, state-independent, deterministic order index (C15/D6 §6.3).
Signature	<code>PVPPRegistry.register_sigma_order(definition: SigmaOrderDefinition)</code>
Argument record	<code>SigmaOrderDefinition</code> : frozen dataclass; <code>policy_id</code> : str; <code>order_index</code> : int
Preconditions	Neither the policy nor the index is already registered.
Result	None.

Field	Content
Invariants	Each policy has at most one index and each index at most one policy. The order breaks exact ties only.
Failure behavior	Raises ValueError on a duplicate policy or index.
Does not	Express preference. Using the order to rank is a misuse (Guide 6.12).
Evidence	Direct use: test files 15, benchmarks 7, example files 2; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.
See also	3.11.

Entry — `PVPPRegistry.register_policy_seed`

Field	Content
Name and label	<code>PVPPRegistry.register_policy_seed</code> — method; registry.py:133. PUBLIC / ADVANCED (rule R2, revised (H.5-4)).
Purpose	Registers a seed that Π may build a candidate from when the graph path is not used.
Framework basis	Policy seeds and families (C15/D7; C15/D14).
Signature	<code>PVPPRegistry.register_policy_seed(seed: PolicySeedDefinition)</code>
Argument record	<code>PolicySeedDefinition</code> : frozen dataclass; <code>id</code> : str; <code>action_ids</code> : tuple[str, ...]; <code>family</code> : str; <code>affected_domain_ids</code> : tuple[str, ...] = (); <code>policy_class_ids</code> : tuple[str, ...] = (); <code>structurally_required</code> : bool = False; <code>coherent</code> : bool = True; <code>visible</code> : bool = True; <code>metadata</code> : Mapping[str, Any] None = None; <code>source_family</code> : str = ""; <code>path_type</code> : str = ""; <code>source_ids</code> : tuple[str, ...] = (); <code>structural_distinctness_note</code> : str = ""
Preconditions	The seed identifier is new; family is one of the four canonical families; its actions and affected domains are registered.
Result	None.
Invariants	A seed is not a projected or ranked policy. Regime activates seeds by family; structurally required seeds and seeds in a materially required class are kept regardless (3.4).
Failure behavior	Raises ValueError if the identifier is already registered. Raises ValueError for an unknown family or an unregistered action or domain.
Does not	Guarantee the seed becomes a candidate.
Evidence	Direct use: test files 2, benchmarks 2; not named in doc 05; e.g. benchmark_mission_required_pi_v049.py, benchmark_pi_construction_v025.py.
See also	3.4–3.5; 9.8; H.5-4.

4.7 Graph Instances, Transformations, and Compositions

Entry — PVPPRegistry.register_graph_instance

Field	Content
Name and label	PVPPRegistry.register_graph_instance — method; registry.py:374. PUBLIC / SUPPORTED (rule R1).
Purpose	Registers an instance node of the instance-transformation graph.
Framework basis	Instance-Transformation Graph Layer (C15/D11).
Signature	PVPPRegistry.register_graph_instance(instance: GraphInstanceDefinition)
Argument record	GraphInstanceDefinition: frozen dataclass; id: str; instance_type: str; domain_ids: tuple[str, ...]; current_state: str; roles: tuple[str, ...] = (); reachability_relevance: str = ""; composition_relevance: str = ""; metadata: Mapping[str, Any] = (empty)
Preconditions	The identifier is new and its domains are registered.
Result	None.
Invariants	Instances enter the graph substrate identity (4.8).
Failure behavior	Raises ValueError if the identifier is already registered. Raises ValueError for unregistered domains.
Does not	Assert reachability; that belongs to transformations.
Evidence	Direct use: test files 24, benchmarks 10, example files 2; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.
See also	3.5; Chapter 8.

Entry — PVPPRegistry.register_graph_transformation

Field	Content
Name and label	PVPPRegistry.register_graph_transformation — method; registry.py:382. PUBLIC / SUPPORTED (rule R1).
Purpose	Registers a transformation between two instances, carried out by one action.
Framework basis	Graph layer and construction protocol (C15/D11; C15/D14).
Signature	PVPPRegistry.register_graph_transformation(transformation: GraphTransformationDefinition)

Field	Content
Argument record	GraphTransformationDefinition: frozen dataclass; id: str; source_instance_id: str; destination_instance_id: str; family: str; affected_domain_ids: tuple[str, ...]; reachability_status: str; action_id: str; policy_class_ids: tuple[str, ...] = (); enabling_conditions: tuple[str, ...] = (); blocking_conditions: tuple[str, ...] = (); relevant_horizon: float None = None; irreversible: bool = False; structurally_required: bool = False; uncertainty_note: str = ''; structural_effect: str = ''; metadata: Mapping[str, Any] = (empty); only_viable_path_candidate: bool = False
Preconditions	The identifier is new. family is one of the eight graph families (continuation, maintenance_local_adjustment, corrective_repair, structural_reconfiguration, exit_transfer_liquidation, substitution, release_restructuring, escape_emergency). reachability_status is "reachable", "unreachable", or "uncertain"; an uncertain transformation needs a non-empty uncertainty_note. Instances, action, and domains are registered.
Result	None.
Invariants	Graph families map onto the four Π families (3.5). "unreachable" is a real status and is kept, not dropped.
Failure behavior	Raises ValueError if the identifier is already registered. Raises ValueError for any failed condition above.
Does not	Make the transformation feasible; Constraints decides feasibility.
Evidence	Direct use: test files 19, benchmarks 10, example files 2; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.
See also	3.5; Chapter 8 for reachability refresh, admission, and revision.

Entry — PVPPRegistry.register_graph_composition

Field	Content
Name and label	PVPPRegistry.register_graph_composition — method; registry.py:400. PUBLIC / SUPPORTED (rule R1).
Purpose	Registers a composition of transformations that represents a policy family together.
Framework basis	Graph composition sufficiency (C15/D11).
Signature	PVPPRegistry.register_graph_composition(composition: GraphCompositionDefinition)
Argument record	GraphCompositionDefinition: frozen dataclass; id: str; transformation_ids: tuple[str, ...]; represented_family: str; affected_domain_ids: tuple[str, ...]; policy_class_ids: tuple[str, ...] = (); recovery_relevance: str = ''; distinctness_justification: str = ''; structurally_required: bool = False
Preconditions	The identifier is new and every transformation is registered.

Field	Content
Result	None.
Invariants	construct_graph treats a composition as valid only if its recovery relevance and distinctness justification are non-empty (8.2).
Failure behavior	Raises ValueError if the identifier is already registered. Raises ValueError for unregistered transformations.
Does not	Establish sufficiency on its own; Chapter 8 covers the checks.
Evidence	Direct use: test files 3, benchmarks 2; not named in doc 05; e.g. benchmark_graph_composition_sufficiency_v039.py, three_domain_buffered_multistep_scenario_v048.py.
See also	Chapter 8.

4.8 Substrate and Configuration Identities

The registry computes six identity fingerprints. Each is the SHA-256 digest of a canonical JSON serialization (sorted keys, compact separators) of the registered records it covers. They are integrity and version tokens: a fingerprint changes exactly when the structure it covers changes, which lets re-entry and integrity audits tell whether an earlier result still describes the current registry. They carry no semantic authority.

Table 4.3 — Identity methods (each returns a hexadecimal SHA-256 string)

Method	Covers	Evidence
action_registry_identity()	All registered action definitions	test files 1
graph_substrate_identity()	Domains, graph instances, transformations, and compositions	test files 15
governing_substrate_identity()	Domains, the governing configuration (or its default), and recovery necessities	test files 3
phi_substrate_identity()	Domain definitions	test files 1
regime_configuration_identity()	The regime configuration; an empty string if none	test files 1
horizon_configuration_identity()	The effective horizon configuration, including the default	test files 1

Entry, shared by all six. Label: PUBLIC / ADVANCED (rule R2). Signature: () → str. Preconditions: none. Invariant: deterministic for identical registered content, independent of registration order within each collection. Failure: none raised. Does not: detect changes in host state, world adapters, or services; verify runtime identity (use the hash manifest, 1.4). See also: Chapter 12 (re-entry dependencies); audit_cycle_artifact_integrity (9.12).

4.9 Configuration Provenance

Entry — `PVPPRuntime.validate_runtime_configuration_provenance`

Field	Content
Name and label	<code>PVPPRuntime.validate_runtime_configuration_provenance</code> — method; <code>kernel.py:5282</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Checks that a configuration record identifies itself completely enough to be reproduced.
Framework basis	Reproducibility requirements for runs (C45; Guide 9.9).
Signature	<code>PVPPRuntime.validate_runtime_configuration_provenance(record: RuntimeConfigurationProvenance) → RuntimeConfigurationProvenanceAssessment</code>
Argument record	<code>RuntimeConfigurationProvenance</code> : frozen dataclass; <code>configuration_id</code> : str; <code>framework_version</code> : str; <code>runtime_version</code> : str; <code>created_at</code> : float; <code>adapter_identities</code> : tuple[<code>AdapterIdentity</code> , ...] = (); <code>source_ids</code> : tuple[str, ...] = (); <code>metadata</code> : Mapping[str, Any] = (empty)
Preconditions	None.
Result	<code>RuntimeConfigurationProvenanceAssessment</code> : valid, the configuration id, and any violations.
Invariants	Required: non-empty configuration, framework, and runtime versions; <code>created_at</code> not negative; each adapter identity with a non-empty id and implementation version and a role from the five V2 adapter roles (<code>state_measurement</code> , <code>action_structure</code> , <code>projection_world_model</code> , <code>constraint_evidence</code> , <code>execution_transition</code>); adapter ids and roles unique; source ids unique.
Failure behavior	Returns violations; raises nothing for invalid content.
Does not	Check that <code>runtime_version</code> names the running runtime, or check the hash manifest; grant adapter identities any operator authority. v0.141 has no version attribute; its version is in <code>pyproject.toml</code> and its identity is the manifest (1.4).
Nested record	<code>AdapterIdentity</code> : frozen dataclass; <code>adapter_id</code> : str; <code>adapter_role</code> : str; <code>implementation_version</code> : str; <code>provenance_ids</code> : tuple[str, ...] = (); <code>metadata</code> : Mapping[str, Any] = (empty)
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_v2_adapter_configuration_v074.py</code> .
See also	1.4; Guide 9.9.

4.10 Architecture-Adaptation Invariance

Entry — `PVPPRuntime.assess_architecture_adaptation_invariance`

Field	Content
Name and label	<code>PVPPRuntime.assess_architecture_adaptation_invariance</code> — method; <code>kernel.py:5301</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Checks, across two or more configurations, that adapting the Productive Value–Productive Power (PV-PP) framework to different domains changed adapters and not the canonical architecture.
Framework basis	Portability without new primitives (C00/D1 §20, Milestone 17; C30/D3).
Signature	<code>PVPPRuntime.assess_architecture_adaptation_invariance(profile, configurations)</code>
Argument record	<code>ArchitectureInvarianceProfile</code> : frozen dataclass; <code>profile_id</code> : str; <code>canonical_stage_ids</code> : tuple[str, ...]; <code>required_adapter_roles</code> : tuple[str, ...]; <code>prohibited_adapter_roles</code> : tuple[str, ...] = ('selector', 'canonical_operator', 'sigma_override'); <code>notes</code> : tuple[str, ...] = ()
Preconditions	At least two configurations.
Result	<code>ArchitectureAdaptationAssessment</code> : validity, the configurations, the declared canonical stages, the adapter roles of each configuration, and violations.
Invariants	Each configuration must pass 4.9, include every required adapter role, and include no prohibited role—by default "selector", "canonical_operator", and "sigma_override". Profile stages, required roles, and configuration ids must be unique.
Failure behavior	Returns violations; raises nothing for invalid content.
Does not	Inspect the runtime's actual stages: the canonical stage list is taken from the profile as declared. It is instrumentation and confers no operator or execution authority.
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_v2_architecture_invariance_v0129.py</code> .
See also	Guide 1.10; 9.14.

4.11 Where Each Core Method Is Documented

Each public method of `PVPPRuntime` and `PVPPRegistry` has exactly one primary entry. Table 4.4 routes all 119 of them; the fourteen methods of `ExecutionBindingRegistry` are all in Chapter 11. The routing is generated from `audit/routing.py` and checked against the export list, so no method is missed or placed twice.

Table 4.4 — Primary-entry chapter for every PVPPRuntime and PVPPRegistry method

Chapter	Methods
4	assess_architecture_adaptation_invariance, registry.action_registry_identity, registry.governing_substrate_identity, registry.graph_substrate_identity, registry.horizon_configuration_identity, registry.phi_substrate_identity, registry.regime_configuration_identity, registry.register_action, registry.register_constraint_rule, registry.register_dependency_requirement, registry.register_domain, registry.register_domain_relation, registry.register_fallback_configuration, registry.register_governing_configuration, registry.register_graph_composition, registry.register_graph_instance, registry.register_graph_transformation, registry.register_horizon_configuration, registry.register_policy_seed, registry.register_power, registry.register_recovery_necessity, registry.register_recovery_plan, registry.register_regime_configuration, registry.register_sigma_order, validate_runtime_configuration_provenance
5	assess_transition_relevant_state_pair, build_capacity_authority_request, derive_capacity_authority, prepare_integrated_request_capacity, validate_access_operability_state, validate_actual_persistent_state, validate_capacity_authority_envelope, validate_perceived_decision_state, validate_productive_configuration_state, validate_qualified_unit_state, validate_recovery_resource_state, validate_transition_relevant_state_basis
6	apply_post_execution_epistemic_update, apply_prediction_error, assess_replay_sufficiency, build_decision_replay_package, build_post_execution_epistemic_update_request, build_prediction_error_request, record_transfer_history, validate_information_governance_metadata, validate_memory_retrieval_package, validate_memory_state_reference, validate_post_execution_epistemic_update_result, validate_prediction_error_assessment, validate_transfer_history_event, validate_transfer_history_package
7	evaluate_objective_responsiveness, validate_objective_discovery_hints, validate_objective_set
8	construct_graph, registry.admit_graph_transformation, registry.admit_novel_function, registry.admit_recovery_necessity, registry.assess_graph_structural_coverage, registry.refresh_graph_reachability, registry.revise_graph_transformation, registry.revise_recovery_necessity

Chapter	Methods
9	assess_recovery_feasibility, audit_cycle_artifact_integrity, audit_integrated_cycle_lifecycle_integrity, audit_post_selection_identity_chain, classify_regime, construct_pi, construct_pi_from_graph, evaluate_canonical_cycle_sequence, evaluate_canonical_decision_cycle, evaluate_collapse_horizons, evaluate_constraint_system, evaluate_integrated_canonical_cycle, evaluate_memory_conditioned_cycle_sequence, evaluate_memory_conditioned_decision_cycle, evaluate_memory_conditioned_integrated_cycle, evaluate_pressure_field, evaluate_restoration_adequacy, evaluate_sigma_fallback, evaluate_sigma_standard, evaluate_sigma_terminal, identify_governing_domains, prepare_canonical_cycle_snapshot, prepare_memory_conditioned_cycle_snapshot, qualify_pi_completeness_for_substrate, validate_pi_completeness, validate_pressure_invariants, validate_projection_causal_inputs, validate_projection_model_authority, validate_projection_record
11	advance_execution, apply_layer1_transition, assess_identity, build_execution_license_from_cycle, build_execution_transition_provenance, build_layer1_execution_commit_from_transition, build_layer1_handoff, build_layer1_transition_handoff, build_reauthorization_handoff, control_history, control_state, create_context, epsilon_observation, identity, identity_for_action, instantiate_execution, invalidate_native_execution_authorizations, invoke, invoke_authorized_native, issue_native_execution_authorization, native_execution_authorization_execution_id, native_execution_authorization_status, native_execution_authorizations, record_layer1_execution_commit, record_recovery_bookkeeping_from_transition, register, registered_identities, set_control_state, telemetry, validate_execution_transition_provenance, validate_layer1_transition_result
15	assess, build_execution_license, decide, evaluate_constraints, evaluate_framed_policy_space, evaluate_policy_sets, evaluate_policy_space, evaluate_proposal, evaluate_recovery_policy_space, record_execution, validate_domain_framing

The 11 methods routed to Chapter 15 are all LEGACY / COMPATIBILITY: compatibility paths kept for pre-canonical callers, with their replacements in 15.4 (H.5-5, H.5-8).

4.12 Classification Record and Compact Rows

Table 4.5 lists every label assigned in this chapter with the rule of Appendix H.3 it rests on. The definition and configuration records are documented for applications to construct as the argument of their register method, so they take that method's label (rule R6, construction contract confirmed); their primary entry is the entry that consumes them. Result records returned by this chapter's entries are IMPLEMENTATION / SUPPORT (rule R3).

Table 4.5 — Labels assigned in Chapter 4

Symbol	Label	Rule	Primary entry
PVPPRuntime	PUBLIC / SUPPORTED	R1	4.1
PVPPRegistry	PUBLIC / SUPPORTED	R1	4.2
interfaces.WorldAdapter	PUBLIC / SUPPORTED	R1 (required host protocol); export gap H.5-1	Chapter 10
PVPPRegistry.register_domain	PUBLIC / SUPPORTED	R1	4.3
PVPPRegistry.register_action	PUBLIC / SUPPORTED	R1	4.4
PVPPRegistry.register_power	PUBLIC / SUPPORTED	R1	4.4
PVPPRegistry.register_constraint_rule	PUBLIC / SUPPORTED	R1	4.5
PVPPRegistry.register_recovery_plan	PUBLIC / SUPPORTED	R1	4.5
PVPPRegistry.register_recovery_necessity	PUBLIC / SUPPORTED	R1	4.5
PVPPRegistry.register_governing_configuration	PUBLIC / SUPPORTED	R1	4.6
PVPPRegistry.register_horizon_configuration	PUBLIC / SUPPORTED	R1	4.6
PVPPRegistry.register_regime_configuration	PUBLIC / SUPPORTED	R1	4.6
PVPPRegistry.register_fallback_configuration	PUBLIC / SUPPORTED	R1	4.6
PVPPRegistry.register_sigma_order	PUBLIC / SUPPORTED	R1	4.6
PVPPRegistry.register_policy_seed	PUBLIC / ADVANCED	R2, revised (H.5-4)	4.6
PVPPRegistry.register_graph_instance	PUBLIC / SUPPORTED	R1	4.7

Symbol	Label	Rule	Primary entry
PVPPRegistry.register_graph_transformation	PUBLIC / SUPPORTED	R1	4.7
PVPPRegistry.register_graph_composition	PUBLIC / SUPPORTED	R1	4.7
PVPPRegistry.register_domain_relation	LEGACY / COMPATIBILITY	R5 (H.5-2, decided in Chapter 15)	4.12
PVPPRegistry.register_dependency_requirement	LEGACY / COMPATIBILITY	R5 (H.5-2, decided in Chapter 15)	4.12
PVPPRegistry.action_registry_identity	PUBLIC / ADVANCED	R2	4.8
PVPPRegistry.graph_substrate_identity	PUBLIC / ADVANCED	R2	4.8
PVPPRegistry.governing_substrate_identity	PUBLIC / ADVANCED	R2	4.8
PVPPRegistry.phi_substrate_identity	PUBLIC / ADVANCED	R2	4.8
PVPPRegistry.regime_configuration_identity	PUBLIC / ADVANCED	R2	4.8
PVPPRegistry.horizon_configuration_identity	PUBLIC / ADVANCED	R2	4.8
PVPPRuntime.validate_runtime_configuration_provenance	PUBLIC / ADVANCED	R2	4.9
PVPPRuntime.assess_architecture_adaptation_invariance	PUBLIC / ADVANCED	R2	4.10
DomainDefinition	PUBLIC / SUPPORTED	R6	4.3
ActionDefinition	PUBLIC / SUPPORTED	R6	4.4
ProductivePowerDefinition	PUBLIC / SUPPORTED	R6	4.4

Symbol	Label	Rule	Primary entry
ConstraintRuleDefinition	PUBLIC / SUPPORTED	R6	4.5
RecoveryPlanDefinition	PUBLIC / SUPPORTED	R6	4.5
RecoveryNecessityDefinition	PUBLIC / SUPPORTED	R6	4.5
GoverningConfiguration	PUBLIC / SUPPORTED	R6	4.6
HorizonConfiguration	PUBLIC / SUPPORTED	R6	4.6
RegimeConfiguration	PUBLIC / SUPPORTED	R6	4.6
FallbackConfiguration	PUBLIC / SUPPORTED	R6	4.6
SigmaOrderDefinition	PUBLIC / SUPPORTED	R6	4.6
PolicySeedDefinition	PUBLIC / ADVANCED	R6, revised (H.5-4)	4.6
GraphInstanceDefinition	PUBLIC / SUPPORTED	R6	4.7
GraphTransformationDefinition	PUBLIC / SUPPORTED	R6	4.7
GraphCompositionDefinition	PUBLIC / SUPPORTED	R6	4.7
DomainRelationDefinition	LEGACY / COMPATIBILITY	R6, following its entry (H.5-2)	4.12
DependencyRequirementDefinition	LEGACY / COMPATIBILITY	R6, following its entry (H.5-2)	4.12
RuntimeConfigurationProvenance	PUBLIC / ADVANCED	R6	4.9
AdapterIdentity	PUBLIC / ADVANCED	R6	4.9

Symbol	Label	Rule	Primary entry
ArchitectureInvarianceProfile	PUBLIC / ADVANCED	R6	4.10
RuntimeConfigurationProvenanceAssessment	IMPLEMENTATION / SUPPORT	R3	4.12
ArchitectureAdaptationAssessment	IMPLEMENTATION / SUPPORT	R3	4.12

Table 4.6 — Compact rows for records not given full entries

Name and kind	Label	Role and use	Source
register_domain_relation — method	LEGACY / COMPATIBILITY	Registers a "dependency_floor" relation; consumed only by legacy evaluators (H.5-2; 15.4)	registry.py:49
DomainRelationDefinition — frozen dataclass	LEGACY / COMPATIBILITY	Argument of register_domain_relation	models.py:1685
register_dependency_requirement — method	LEGACY / COMPATIBILITY	Registers an all_of or any_of prerequisite; consumed only by legacy evaluators (H.5-2; 15.4)	registry.py:60
DependencyRequirementDefinition — frozen dataclass	LEGACY / COMPATIBILITY	Argument of register_dependency_requirement	models.py:1702
RuntimeConfigurationProvenanceAssessment — frozen dataclass	IMPLEMENTATION / SUPPORT	Result of validate_runtime_configuration_provenance	models.py:2303
ArchitectureAdaptationAssessment — frozen dataclass	IMPLEMENTATION / SUPPORT	Result of assess_architecture_adaptation_invariance	models.py:3512

Chapter 5 — State, Perception, and Capacity Authority

This chapter covers the state surfaces an application hands to the runtime: actual persistent state $S_i(t)$, the represented world state the operators compute over, the perceived decision state $P_i(t)$, the Version 2 carriers for access, units, configurations, and recovery resources, capacity authority, joint feasibility of active recovery obligations, and transition-relevant state sufficiency. For each surface it states what the runtime validates and, just as important, what it deliberately does not interpret.

A reader must be able to supply state in the form v0.141 validates, know which validator rejects which error, and know which meanings stay with the host.

The governing rule. The host owns the meaning of state. The runtime validates identity, time, routing, and shape; it never opens PP, SPV, AVS, or context to derive anything (C10/D1 §4). Every validator in this chapter returns its violations rather than raising, unless a host hook returns an object of the wrong type.

5.1 Actual Persistent State

ActualPersistentStateEnvelope carries $S_i(t) = [PP_i(t), SPV_i(t), AVS_i(t), X_i(t)]$ as four opaque components with actor, state, and time identity. The runtime imposes no inner type, arithmetic, or scalar meaning on any component. It checks only that the outer structure is present.

Record — ActualPersistentStateEnvelope

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: the host constructs it as the argument of the integrated cycle, snapshot, and sequence entries, 9.3 and 9.5)
Fields	frozen dataclass; actor_id: str; state_id: str; time: float; pp: Any; spv: Any; avs: Any; context: Any; metadata: Mapping[str, Any] = (empty)
Source	models.py:428
Evidence	Direct use: test files 21, benchmarks 11, example files 1; named in doc 05; e.g. minimal_battery_service.py.

Entry — PVPPRuntime.validate_actual_persistent_state

Field	Content
Name and label	PVPPRuntime.validate_actual_persistent_state — method; kernel.py:4689. PUBLIC / ADVANCED (rule R2).
Purpose	Checks that an actual-state envelope has identity, a finite time, and all four canonical components.
Framework basis	Actual persistent state (C10/D1 §1–2; Layer 1, C10/D2).
Signature	PVPPRuntime.validate_actual_persistent_state(state: ActualPersistentStateEnvelope) → Layer1TransitionValidationAssessment
Preconditions	None.

Field	Content
Result	A Layer1TransitionValidationAssessment. The result type is shared with Layer-1 transition validation; here next_state_id is None.
Invariants	Required: non-empty actor_id and state_id; numeric, finite time; pp, spv, avs, and context each present (not None). Nothing inside a component is examined.
Failure behavior	Returns violations; raises nothing.
Does not	Interpret or type-check component contents, compare with earlier states, or verify that the state is current.
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_layer1_actual_state_boundary_v040.py, test_v2_state_interface_reconciliation_v072.py.
See also	Called by the snapshot, sequence, Layer-1, and capacity-authority entries; Guide 8.6.

5.2 The Represented World State

In v0.141 the operators do not read the typed perceived-state components directly. They compute over a represented world state—a `WorldState` of productive-power values—through the host's world adapter: `perceive`, `domain_value`, `project`, and the strict Φ/H hooks (3.1). The typed perceived decision state wraps this payload in its `represented_state` field, which the runtime's own docstring describes as a compatibility projection payload, not a new canonical state component. Readers should hold both facts together: $P_i(t)$ is the framework's decision input, and the `WorldState` inside it is what the v0.141 operators compute over.

Records — `WorldState` and `ProductivePowerState`

Record	Label	Fields
WorldState	PUBLIC / SUPPORTED (R6: returned by the host's <code>WorldAdapter</code> hooks)	frozen dataclass; time: float; powers: Mapping[str, <code>ProductivePowerState</code>]; metadata: Mapping[str, Any] = (empty)
ProductivePowerState	PUBLIC / SUPPORTED (R6: part of <code>WorldState</code>)	frozen dataclass; power_id: str; value: float; executable_value: float None = None; metadata: Mapping[str, Any] = (empty)

`powers` is keyed by the power identifiers registered in 4.4. `value` is the capability level; `executable_value`, where the host supplies it, is the part that can be used now. Evidence: `WorldState` — test files 29, benchmarks 19, example files 24; named in doc 05; e.g. `authority_bound_callthrough_v0130.py`, `minimal_battery_service.py`. `ProductivePowerState` — test files 9, benchmarks 4, example files 23; not named in doc 05; e.g. `minimal_battery_service.py`, `branching_world.py` (legacy `tiny_*` example, 15.4).

5.3 The Perceived Decision State

The host forms $P_i(t)$ through a perception hook on its world adapter. v0.141 looks for `perceived_decision_state_from_actual` first, and falls back to the older `represented_state_from_actual`. In the fallback, the runtime wraps the returned `WorldState` as the PPP component and marks the metadata "legacy_world_state_as_ppp_interface"; it does not infer SPV-hat, PVS, or X-hat. New integrations should implement the typed hook. The memory-conditioned path has matching hooks that receive current retrieval and expectation state, never raw memory (6.2).

Record — `PerceivedDecisionState`

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: returned by the host's <code>WorldAdapter</code> perception hook)
Fields	frozen dataclass; <code>actor_id</code> : str; <code>state_id</code> : str; <code>time</code> : float; <code>represented_state</code> : <code>WorldState</code> ; <code>ppp</code> : Any; <code>spv_hat</code> : Any = None; <code>pvs</code> : Any = None; <code>x_hat</code> : Any = None; <code>retrieval</code> : <code>MemoryRetrievalPackage</code> None = None; <code>expectation_state</code> : <code>ExpectationStateReference</code> None = None; <code>confidence</code> : Any = None; <code>uncertainty</code> : Any = None; <code>projection_causal_inputs</code> : tuple[<code>ProjectionCausalStateInput</code> , ...] = (); <code>metadata</code> : Mapping[str, Any] = (empty)
Source	models.py:394
Evidence	Direct use: test files 8, benchmarks 4, example files 1; named in doc 05; e.g. <code>minimal_battery_service.py</code> .

Entry — `PVPPRuntime.validate_perceived_decision_state`

Field	Content
Name and label	<code>PVPPRuntime.validate_perceived_decision_state</code> — method; <code>kernel.py</code> :2236. PUBLIC / ADVANCED (rule R2).
Purpose	Checks that a perceived decision state is attributed to the right actual state and routed only through licensed inputs.
Framework basis	$P_i(t)$ as the perception-supplied decision interface; PPP is one component (C10/D1; C10/D19).
Signature	<code>PVPPRuntime.validate_perceived_decision_state(perceived: PerceivedDecisionState, actual_state: ActualPersistentStateEnvelope, *, expected_retrieval: MemoryRetrievalPackage None = None, expected_expectation: ExpectationStateReference None = None) → PerceivedDecisionStateValidationAssessment</code>
Preconditions	None. Pass <code>expected_retrieval</code> and <code>expected_expectation</code> when the snapshot is memory-conditioned.
Result	<code>PerceivedDecisionStateValidationAssessment</code> : valid, actor and state identity, violations.

Field	Content
Invariants	Required: actor_id and state_id match the actual state; time is finite and equals the actual state's time; represented_state is a WorldState whose time matches and whose metadata state_id, if present, matches; ppp is present. Retrieval appears if and only if a licensed retrieval was supplied, and must match it; expectation state likewise.
Failure behavior	Returns violations. Raises ValueError if the object is not a PerceivedDecisionState.
Does not	Derive or check the meaning of PPP, SPV-hat, PVS, or X-hat, or compare them with actual state. It validates attribution and routing only.
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_perceived_decision_state_boundary_v056.py, test_v2_state_interface_reconciliation_v072.py.
See also	Called by both snapshot entries (9.3, 9.5); Guide 8.5.

5.4 Possession, Access, Permission, Control, and Operability

Version 2 separates five facts about a resource that everyday language runs together: whether it is possessed, available, permitted, controlled, and operable (C00/D1 §20, Milestone 17; Guide 3.6). The runtime keeps them independent: none is inferred from another. The four Version 2 carriers in 5.4–5.6 are validated on request; in v0.141 no decision-cycle stage consumes them.

Entry — `PVPPRuntime.validate_access_operability_state`

Field	Content
Name and label	<code>PVPPRuntime.validate_access_operability_state</code> — method; kernel.py:4538. PUBLIC / ADVANCED (rule R2).
Purpose	Checks the representation of the five access and operability facts for one resource or capability.
Framework basis	Possession, access, permission, control, and operability distinctions (C00/D1 §20; C10/D20).
Signature	<code>PVPPRuntime.validate_access_operability_state(state: AccessOperabilityState) → AccessOperabilityValidationAssessment</code>
Argument record	<code>AccessOperabilityState</code> : frozen dataclass; <code>resource_id</code> : str; <code>observed_time</code> : float; <code>possessed</code> : bool None = None; <code>available</code> : bool None = None; <code>permitted</code> : bool None = None; <code>controlled</code> : bool None = None; <code>operable</code> : bool None = None; <code>conditions</code> : Mapping[str, Any] = (empty); <code>provenance_ids</code> : tuple[str, ...] = (); <code>metadata</code> : Mapping[str, Any] = (empty)
Preconditions	None.
Result	<code>AccessOperabilityValidationAssessment</code> .
Invariants	Required: non-empty <code>resource_id</code> ; finite <code>observed_time</code> ; each of the five facts True, False, or None (unknown); no empty provenance identity.
Failure behavior	Returns violations. Raises ValueError if the object is not an <code>AccessOperabilityState</code> .

Field	Content
Does not	Infer one fact from another, or decide PP, feasibility, adequacy, authorization, or selection.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_state_interface_reconciliation_v072.py.
See also	Guide 3.6.

5.5 Qualified Units and Productive Configurations

Entry — `PVPPRuntime.validate_qualified_unit_state`

Field	Content
Name and label	<code>PVPPRuntime.validate_qualified_unit_state</code> — method; kernel.py:4610. PUBLIC / ADVANCED (rule R2).
Purpose	Checks identity and nesting metadata for a modeled unit, such as a household, team, or firm.
Framework basis	Collective entity qualification and nesting (C10/D11).
Signature	<code>PVPPRuntime.validate_qualified_unit_state(state: QualifiedUnitState) → QualifiedUnitValidationAssessment</code>
Argument record	QualifiedUnitState: frozen dataclass; unit_id: str; unit_type: str; member_ids: tuple[str, ...] = (); parent_unit_id: str None = None; boundary_id: str None = None; entity_state_id: str None = None; productive_role_qualified: bool = False; qualification_evidence_ids: tuple[str, ...] = (); metadata: Mapping[str, Any] = (empty)
Preconditions	None.
Result	QualifiedUnitValidationAssessment.
Invariants	Required: non-empty unit_id and unit_type; unique members; a unit is not its own member or parent; productive-role qualification is claimed only with evidence, and evidence identities are unique.
Failure behavior	Returns violations; raises nothing.
Does not	Confer qualification. Membership and nesting do not establish collective productive qualification.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_recovery_and_units_v073.py.
See also	Guide 3.11.

Entry — `PVPPRuntime.validate_productive_configuration_state`

Field	Content
Name and label	<code>PVPPRuntime.validate_productive_configuration_state</code> — method; kernel.py:4568. PUBLIC / ADVANCED (rule R2).

Field	Content
Purpose	Checks a configuration of components whose joint operation provides a capability.
Framework basis	Configuration-dependent and joint capability (C10/D20; C10/D10).
Signature	PVPPRuntime.validate_productive_configuration_state(state: ProductiveConfigurationState) → ProductiveConfigurationValidationAssessment
Argument record	ProductiveConfigurationState: frozen dataclass; configuration_id: str; component_ids: tuple[str, ...]; dependency_ids: tuple[str, ...] = (); jointly_feasible: bool None = None; evidence_ids: tuple[str, ...] = (); metadata: Mapping[str, Any] = (empty)
Preconditions	None.
Result	ProductiveConfigurationValidationAssessment.
Invariants	Required: non-empty configuration_id; at least one component; unique component and dependency identities; no empty identities; jointly_feasible True, False, or None.
Failure behavior	Returns violations. Raises ValueError if the object is not a ProductiveConfigurationState.
Does not	Aggregate components into a scalar PP, authorize an action, or select a policy.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_state_interface_reconciliation_v072.py.
See also	Guide 3.7.

5.6 Recovery Resource State

Entry — PVPPRuntime.validate_recovery_resource_state

Field	Content
Name and label	PVPPRuntime.validate_recovery_resource_state — method; kernel.py:4592. PUBLIC / ADVANCED (rule R2).
Purpose	Checks a recovery resource record that keeps reserve, recovery capability, and current reachability apart.
Framework basis	Reserve and recovery distinctions (C00/D1 §20); recovery as structure (Guide 3.10).
Signature	PVPPRuntime.validate_recovery_resource_state(state: RecoveryResourceState) → RecoveryResourceValidationAssessment
Argument record	RecoveryResourceState: frozen dataclass; recovery_id: str; observed_time: float; reserve: Any = None; recovery_capable: bool None = None; currently_reachable: bool None = None; prerequisite_ids: tuple[str, ...] = (); evidence_ids: tuple[str, ...] = (); metadata: Mapping[str, Any] = (empty)

Field	Content
Preconditions	None.
Result	RecoveryResourceValidationAssessment.
Invariants	Required: non-empty recovery_id; observed_time not negative; unique, non-empty prerequisite identities; unique evidence identities.
Failure behavior	Returns violations; raises nothing.
Does not	Infer recovery capability from reserve, or reachability from capability; establish adequacy, authorization, or selection.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_recovery_and_units_v073.py.
See also	Guide 3.10; 3.9–3.10 for how recovery obligations enter the cycle.

5.7 Capacity Authority

When active recovery obligations compete for shared capacity—people, equipment, money, time—the runtime needs a represented capacity calendar to check them jointly (3.9). Only the host can say what capacity exists, so v0.141 asks for it through an optional CapacityAuthorityService. The runtime supplies the structural demand context and validates the answer's shape; the host alone interprets its own individuals, skills, availability, commitments, and restoration rules.

Table 5.1 — The capacity-authority flow

Step	Call	What it establishes
1	build_capacity_authority_request	Active corridors, their remaining actions, the resources those actions demand (from capacity_demands in action metadata), and the latest deadline
2	CapacityAuthorityService.derive (host)	A CapacityAuthorityEnvelope with a calendar for this actual-state snapshot
3	validate_capacity_authority_envelope	Actor, state, and time attribution; calendar shape: period offsets are non-negative integers, resource identities non-blank, amounts finite and non-negative
4	derive_capacity_authority	Steps 1–3 in order; raises ValueError if no service is attached or the envelope is invalid
5	prepare_integrated_request_capacity	Used by evaluate_integrated_canonical_cycle: if the request has no calendar and active obligations carry capacity demands, fills the calendar from the service. An explicit calendar is never overridden.

Entry — `PVPPRuntime.build_capacity_authority_request`

Field	Content
Name and label	<code>PVPPRuntime.build_capacity_authority_request</code> — method; <code>kernel.py:3014</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Builds the structural demand context a host capacity service needs.
Framework basis	Joint sustainment across governing domains (C15/D10 §7.3); host-owned interpretation of actual state (C10/D1 §4).
Signature	<code>PVPPRuntime.build_capacity_authority_request(actual_state: ActualPersistentStateEnvelope) → CapacityAuthorityRequest</code>
Preconditions	None beyond an actual-state envelope for identity and time.
Result	<code>CapacityAuthorityRequest</code> : identity and time, active corridor ids in plan order, remaining action ids, demanded resource ids, latest deadline.
Invariants	Only corridors with status "active" and remaining actions are included. Resources come only from registered <code>capacity_demands</code> metadata.
Failure behavior	Raises nothing for well-formed input.
Does not	Read PP, SPV, AVS, or context, or estimate any capacity.
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_persistent_state_capacity_authority_v051.py</code> .
See also	Table 5.1; 3.9.

`derive_capacity_authority`, `validate_capacity_authority_envelope`, and `prepare_integrated_request_capacity` are labeled IMPLEMENTATION / SUPPORT by the default rule. Their docstrings describe explicit host invocation, but no test, benchmark, or example calls them directly; they are exercised only through `evaluate_integrated_canonical_cycle`. The audit recorded this as H.5-3 and closed it at this label in v0.20: no owner file requires the methods to be application-callable (15.10). One behavior belongs in the record: `derive_capacity_authority` calls `validate_actual_persistent_state` but does not act on its result; attribution is enforced afterwards by the envelope check.

5.8 Joint Feasibility of Active Recovery Obligations

`CapacityCalendar` maps period offsets 0 to N-1 to the capacity of each resource in that period. The joint-recovery check (3.9) uses it with the runtime's ledger of active corridors: it asks whether every active corridor's remaining actions can be scheduled, one period per action, within the calendar and before each deadline. The schedule search is exact; the code describes it as a small-prototype search, and its cost grows quickly with the number of corridors and periods (15.1). The search lives in the feasibility submodule, is not exported, and is called only by the runtime (INTERNAL, rule R4).

Table 5.2 — Joint-recovery statuses (`JointRecoveryFeasibilityAssessment.status`)

Status	Meaning	Cycle effect
no_active_recovery_corridors	Nothing to compose	Continues
active_corridors_have_no_represented_capacity_demands	Active obligations exist but demand no shared capacity	Continues
active_capacity_authority_required	Active obligations demand capacity but no calendar was supplied	Stops: joint_recovery_capacity_authority_required
active_jointly_feasible	A schedule exists for the whole active set	Continues
active_jointly_infeasible	No schedule exists	Stops: joint_recovery_feasibility_failed

When the active set is infeasible, the report names the conflict. In the default diagnostic mode, "witness", it gives one inclusion-minimal infeasible set of corridors with its bottleneck resources. In "all_minimal" mode, set through the cycle request's `joint_recovery_diagnostic_mode`, it enumerates every inclusion-minimal infeasible set. Neither mode applies a priority or sacrifice rule. A feasible result carries the schedule, deadline slack, and residual capacity.

Record — CapacityCalendar

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: argument of the cycle request, 9.2). Its docstring calls it a prototype calendar; the canonical cycle nonetheless requires it whenever active obligations demand capacity.

Field	Content
Fields	frozen dataclass; capacities: Mapping[int, Mapping[str, float]]
Source	models.py:2139
Evidence	Direct use: test files 6, benchmarks 4, example files 6; not named in doc 05; e.g. run_crossing_choice.py (legacy tiny_* example, 15.4), run_discretionary_competition.py (legacy tiny_* example, 15.4).

5.9 Transition-Relevant State Sufficiency

Version 2.1 requires that current state carry every causally active consequence of the past that a transition needs; history that has no present consequence need not be carried, and historical evidence is not live state unless its present consequence is represented (C10/D21; C00/D1 §21). Completeness is a host judgment. The runtime checks the host's declaration and offers a paired test for the invariant that the same complete state and context must yield the same transition structure.

Entry — `PVPPRuntime.validate_transition_relevant_state_basis`

Field	Content
Name and label	<code>PVPPRuntime.validate_transition_relevant_state_basis</code> — method; kernel.py:4631. PUBLIC / SUPPORTED (rule R1).
Purpose	Checks the host's declaration that an actual state is complete for its modeled purpose and admissible context.
Framework basis	Transition-relevant state sufficiency (C10/D21, Version 2.1).
Signature	<code>PVPPRuntime.validate_transition_relevant_state_basis(state: ActualPersistentStateEnvelope, basis: TransitionRelevantStateBasis) → TransitionRelevantStateSufficiencyAssessment</code>
Argument record	TransitionRelevantStateBasis: frozen dataclass; basis_id: str; actor_id: str; state_id: str; modeled_purpose: str; admissible_context_id: str; material_state_surface_ids: tuple[str, ...] = (); authoritative_reference_ids: tuple[str, ...] = (); source_version_ids: tuple[str, ...] = (); provenance_ids: tuple[str, ...] = (); host_validated_complete: bool = False; metadata: Mapping[str, Any] = (empty)
Preconditions	None.
Result	TransitionRelevantStateSufficiencyAssessment: sufficient, identities, violations.
Invariants	Required: basis_id; actor and state identity matching the actual state; modeled_purpose; admissible_context_id; unique surface and reference identities, with at least one of either; and host_validated_complete set to True.
Failure behavior	Returns violations; raises nothing.
Does not	Judge completeness itself or infer omitted state from history. Complete causal history is not required.

Field	Content
Evidence	Direct use: test files 1; named in doc 05; e.g. test_v2_1_state_sufficiency_v0138.py.
See also	Guide 4.10; StateSufficiencyAdapter (Chapter 10).

Entry — `PVPPRuntime.assess_transition_relevant_state_pair`

Field	Content
Name and label	<code>PVPPRuntime.assess_transition_relevant_state_pair</code> — method; kernel.py:4665. PUBLIC / ADVANCED (rule R2).
Purpose	Tests two declarations for the same-state, same-context invariant.
Framework basis	Transition-relevant state sufficiency (C10/D21).
Signature	<code>PVPPRuntime.assess_transition_relevant_state_pair(left: TransitionRelevantStateBasis, right: TransitionRelevantStateBasis, *, left_transition_structure_id: str, right_transition_structure_id: str) → TransitionRelevantStatePairAssessment</code>
Preconditions	Two bases, each with the identifier of the transition or reachability structure the host modeled from it.
Result	<code>TransitionRelevantStatePairAssessment</code> : consistent, the two basis identities, violations.
Invariants	If both bases are host-declared complete and share purpose, context, surfaces, and references, their transition structure identifiers must be equal; a difference means a causally active consequence was omitted from state.
Failure behavior	Returns violations; raises nothing.
Does not	Accept historical origin as a reason for different prospective structure. Its docstring calls it a hostile-test helper; it is a diagnostic, not a runtime gate.
Evidence	Direct use: test files 1; named in doc 05; e.g. test_v2_1_state_sufficiency_v0138.py.
See also	Guide 4.10, 9.4.

The `StateSufficiencyAdapter` protocol lets a host supply bases on request (Chapter 10). In v0.141 no runtime method calls it; hosts call the validator above with the basis they produce.

5.10 Classification Record and Compact Rows

The state records are PUBLIC / SUPPORTED on their own construction contract: an ordinary application builds the actual-state envelope and returns `WorldState` and `PerceivedDecisionState` from the `WorldAdapter` hooks that the runtime constructor requires. The validators are PUBLIC / ADVANCED because the ordinary path already runs them inside the snapshot entries; calling them directly is specialized integration work. The difference in labels reflects that division, not inheritance.

Table 5.3 — Labels assigned in Chapter 5

Symbol	Label	Rule
PVPPRuntime.validate_actual_persistent_state	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_perceived_decision_state	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_access_operability_state	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_qualified_unit_state	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_productive_configuration_state	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_recovery_resource_state	PUBLIC / ADVANCED	R2
PVPPRuntime.build_capacity_authority_request	PUBLIC / ADVANCED	R2
PVPPRuntime.derive_capacity_authority	IMPLEMENTATION / SUPPORT	Default (H.5-3)
PVPPRuntime.validate_capacity_authority_envelope	IMPLEMENTATION / SUPPORT	Default (H.5-3)
PVPPRuntime.prepare_integrated_request_capacity	IMPLEMENTATION / SUPPORT	Default (H.5-3)
PVPPRuntime.validate_transition_relevant_state_basis	PUBLIC / SUPPORTED	R1
PVPPRuntime.assess_transition_relevant_state_pair	PUBLIC / ADVANCED	R2
ActualPersistentStateEnvelope	PUBLIC / SUPPORTED	R6
PerceivedDecisionState	PUBLIC / SUPPORTED	R6
WorldState	PUBLIC / SUPPORTED	R6
ProductivePowerState	PUBLIC / SUPPORTED	R6
AccessOperabilityState	PUBLIC / ADVANCED	R6
QualifiedUnitState	PUBLIC / ADVANCED	R6
ProductiveConfigurationState	PUBLIC / ADVANCED	R6
RecoveryResourceState	PUBLIC / ADVANCED	R6
TransitionRelevantStateBasis	PUBLIC / SUPPORTED	R6
CapacityCalendar	PUBLIC / SUPPORTED	R6
CapacityAuthorityEnvelope	IMPLEMENTATION / SUPPORT	Default (H.5-3)

Table 5.4 — Compact rows

Name and kind	Label	Role and use	Source
derive_capacity_authority — method	IMPLEMENTATION / SUPPORT	Runs the capacity-authority flow through the host service (H.5-3)	kernel.py:3089
validate_capacity_authority_envelope — method	IMPLEMENTATION / SUPPORT	Checks a host capacity envelope's attribution and calendar shape (H.5-3)	kernel.py:3046
prepare_integrated_request_capacity — method	IMPLEMENTATION / SUPPORT	Fills a missing calendar for the integrated cycle (H.5-3)	kernel.py:3103
CapacityAuthorityEnvelope — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Returned by the host CapacityAuthorityService; default label (H.5-3, closed)	models.py:2163
CapacityAuthorityRequest — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Built by the runtime for the host service	models.py:2145
PerceivedDecisionStateValidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of validate_perceived_decision_state	models.py:419
AccessOperabilityValidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of validate_access_operability_state	models.py:53
QualifiedUnitValidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of validate_qualified_unit_state	models.py:132
ProductiveConfigurationValidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of validate_productive_configuration_state	models.py:77
RecoveryResourceValidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of validate_recovery_resource_state	models.py:104
CapacityAuthorityValidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of validate_capacity_authority_envelope	models.py:2174
TransitionRelevantStateSufficiencyAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of validate_transition_relevant_state_basis	models.py:468
TransitionRelevantStatePairAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of assess_transition_relevant_state_pair	models.py:478

Name and kind	Label	Role and use	Source
CorridorFeasibilityReport — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Schedule or conflict report from the joint-feasibility search	models.py: 2189
ScheduledRecoveryStep — dataclass (frozen)	IMPLEMENTATION / SUPPORT	One scheduled action in a feasible report	models.py: 2181
JointRecoveryFeasibilityAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Carried in cycle results; not exported (Table 5.2)	models.py: 2201
assess_joint_feasibility — function	INTERNAL	Exact schedule search; not exported; called only by the runtime	feasibility.py:222

Chapter 6 — Information, Memory, Evidence, and Provenance

This chapter covers the informational surface of v0.141: governance metadata carried with information, retained memory and licensed retrieval, transfer history, replay packages, prediction error, the post-execution update of memory and expectation, and the qualification of evidence sources, control topology, and security observations. Every item here is informational. None of it changes actual state, and none of it authorizes a decision or an execution. The chapter states, for each object, what the runtime accepts, what authority it grants, and what it refuses to infer.

A reader must know which informational object each call accepts, what authority the runtime grants it, and what the runtime refuses to infer from it.

Three rules hold throughout. Quality and confidence do not confer authority, and relevance does not confer permitted use (C10/D16). Memory, retrieval, and expectation are distinct objects, and no stage receives raw memory (C10/D1 §4). Evidence qualification is epistemic only: it never creates feasibility, adequacy, selection, execution, or re-entry authority. Mapping a change in evidence to the stages it invalidates is a separate, explicit step (Chapter 12).

Labels. Every entry in this chapter is PUBLIC / ADVANCED (rule R2). The surfaces serve optional integrations—memory-conditioned cycles, history, replay, post-execution learning, and evidence governance—that an ordinary application may never use. Section 6.10 lists all labels.

6.1 Information-Governance Metadata

Version 2.1 requires information to carry its source authority, applicability, permitted use, disposition, supersession, and temporal provenance where decision meaning depends on them (C00/D1 §21; C10/D16). v0.141 carries these on `InformationGovernanceMetadata`, attached to a retrieval package or other information object, and keeps valid time and record time apart in `InformationTemporalProvenance`.

Record — `InformationGovernanceMetadata`

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host retrieval service)
Fields	frozen dataclass; <code>source_authority</code> : Any = None; <code>applicability_scope</code> : Any = None; <code>permitted_use_scope</code> : Any = None; <code>authorized_surfaces</code> : tuple[str, ...] = (); <code>disposition</code> : str = 'current'; <code>supersedes_or_superseded_by</code> : Any = None; <code>temporal_provenance</code> : <code>InformationTemporalProvenance</code> None = None; <code>source_version</code> : Any = None; <code>provenance</code> : Any = None; <code>metadata</code> : Mapping[str, Any] = (empty)
Source	models.py:598
Evidence	Direct use: test files 2, benchmarks 1; named in doc 05; e.g. <code>benchmark_small_economy_v2_1_v0140.py</code> .

Record — InformationTemporalProvenance

Field	Content
Label	PUBLIC / ADVANCED (rule R6)
Fields	frozen dataclass; valid_time: Any = None; record_or_knowledge_time: Any = None; metadata: Mapping[str, Any] = (empty)
Source	models.py:584
Evidence	Direct use: test files 3; named in doc 05; e.g. test_v2_1_memory_governance_v0135.py, test_v2_1_replay_sufficiency_v0137.py.

Entry — PVPPRuntime.validate_information_governance_metadata

Field	Content
Name and label	PVPPRuntime.validate_information_governance_metadata — method; kernel.py:2169. PUBLIC / ADVANCED (rule R2).
Purpose	Checks governance metadata and reports whether the information may serve as current positive evidence.
Framework basis	Retrieval disposition, authority, applicability, permitted use, and temporal provenance (C10/D16, Version 2.1).
Signature	PVPPRuntime.validate_information_governance_metadata(governance: InformationGovernanceMetadata)
Preconditions	None.
Result	InformationGovernanceAssessment: valid, the normalized disposition, usable_as_current_positive_evidence, and violations.
Invariants	The disposition, after normalizing case, spaces, and hyphens, is one of: current, rejected, superseded, defective, prohibited_for_use, historically_valid_noncurrent, unresolved, contradicted. Only a valid "current" disposition is usable as current positive evidence. Authorized surfaces must be non-empty strings.
Failure behavior	Returns violations. Raises ValueError if the object is not InformationGovernanceMetadata.
Does not	Promote non-current information to current evidence; treat quality or confidence as source authority; treat relevance as permission to use.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_small_economy_v2_1_v0140.py.
See also	Guide 4.4–4.7; Table 2.6.

6.2 Retained Memory and Licensed Retrieval

Retained memory $M_i(t)$ is visible only to the host's MemoryRetrievalService. The runtime builds a retrieval request from the host's query and context, receives a MemoryRetrievalPackage $R^M_i(t,q)$, validates it, and passes it—with any expectation state $K_i(t)$ —to the host's memory-conditioned perception hook. Perception never receives raw memory. This path is run by

prepare_memory_conditioned_cycle_snapshot (9.5); the validators below are the checks it applies, available for direct use. MemoryRetrievalService is defined in pvpp_runtime.interfaces and is not exported at the top level (H.5-1); its primary entry is in Chapter 10.

Record — MemoryStateReference

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; actor_id: str; memory_state_id: str; time: float; payload: Any; metadata: Mapping[str, Any] = (empty)
Source	models.py:552
Evidence	Direct use: test files 7, benchmarks 3; not named in doc 05; e.g. benchmark_memory_conditioned_sequence_v045.py, benchmark_memory_retrieval_boundary_v043.py.

Record — MemoryRetrievalPackage

Field	Content
Label	PUBLIC / ADVANCED (rule R6: returned by the host retrieval service)
Fields	frozen dataclass; actor_id: str; retrieval_id: str; memory_state_id: str; time: float; content: Any; quality: RetrievalQualityMetadata; query_trace: Any = None; source_trace: Mapping[str, Any] = (empty); notes: tuple[str, ...] = (); governance: InformationGovernanceMetadata None = None
Source	models.py:639
Evidence	Direct use: test files 6, benchmarks 2; not named in doc 05; e.g. benchmark_memory_conditioned_sequence_v045.py, benchmark_memory_retrieval_boundary_v043.py.

Record — RetrievalQualityMetadata

Field	Content
Label	PUBLIC / ADVANCED (rule R6)
Fields	frozen dataclass; confidence: Any = None; source_basis: Any = None; completeness: Any = None; distortion_risk: Any = None; freshness: Any = None; corroboration: Any = None; relevance_scope: Any = None; metadata: Mapping[str, Any] = (empty)
Source	models.py:567
Evidence	Direct use: test files 6, benchmarks 2; not named in doc 05; e.g. benchmark_memory_conditioned_sequence_v045.py, benchmark_memory_retrieval_boundary_v043.py.

Record — ExpectationStateReference

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)

Field	Content
Fields	frozen dataclass; actor_id: str; expectation_state_id: str; time: float; payload: Any; metadata: Mapping[str, Any] = (empty)
Source	models.py:715
Evidence	Direct use: test files 7, benchmarks 2; not named in doc 05; e.g. benchmark_memory_conditioned_sequence_v045.py, benchmark_post_execution_epistemic_update_v044.py.

Entry — `PVPPRuntime.validate_memory_state_reference`

Field	Content
Name and label	<code>PVPPRuntime.validate_memory_state_reference</code> — method; kernel.py:2146. PUBLIC / ADVANCED (rule R2).
Purpose	Checks identity and time of a retained-memory reference, optionally against the current actual state.
Framework basis	Retained memory $M_i(t)$ is informational persistence, not actual state (C10/D1 §3).
Signature	<code>PVPPRuntime.validate_memory_state_reference(memory_state: MemoryStateReference, actual_state: ActualPersistentStateEnvelope None = None)</code>
Preconditions	None.
Result	A tuple of violation strings; empty means valid. Unlike most validators, it returns no assessment record.
Invariants	Required: actor_id, memory_state_id, and a finite numeric time. With an actual state: the actor matches, and the memory does not originate after the actual state's time.
Failure behavior	Returns violations. Raises <code>ValueError</code> if the object is not a <code>MemoryStateReference</code> .
Does not	Read or interpret the memory payload.
Evidence	Direct use: no direct use in tests, benchmarks, or examples; not named in doc 05.
See also	6.2; Guide 4.8.

Entry — `PVPPRuntime.validate_memory_retrieval_package`

Field	Content
Name and label	<code>PVPPRuntime.validate_memory_retrieval_package</code> — method; kernel.py:2200. PUBLIC / ADVANCED (rule R2).
Purpose	Checks that a retrieval package answers the request that produced it, at the current decision time.
Framework basis	Retrieval $R^*M_i(t,q)$ is a current reconstruction, distinct from stored memory (C10/D1 §3; C10/D16).

Field	Content
Signature	PVPPRuntime.validate_memory_retrieval_package(request: MemoryRetrievalRequest, package: MemoryRetrievalPackage, actual_state: ActualPersistentStateEnvelope None = None)
Preconditions	The request that produced the package.
Result	A tuple of violation strings; empty means valid.
Invariants	Required: actor and memory-state identity match the request; a retrieval_id; RetrievalQualityMetadata present; any governance metadata valid (6.1); a finite time equal to the request's current time, or the actual state's time when the request gives none.
Failure behavior	Returns violations. Raises ValueError if the object is not a MemoryRetrievalPackage.
Does not	Judge the retrieved content's truth or relevance; convert quality metadata into truth or a global score.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_1_memory_governance_v0135.py.
See also	6.1; Guide 4.8.

6.3 Transfer History

Transfer history TH_i(t) records PV- and PP-relevant events beneath retained memory: what was transferred, claimed, realized, or surrendered, and with whom (C10/D22). The host owns the history store through TransferHistoryService. The runtime validates attribution and order. An event record supports explanation and reconstruction; it does not create current PP, current transition-relevant state, or authority.

Record — TransferHistoryEvent

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; event_id: str; actor_id: str; participants: tuple[str, ...] = (); interaction_id: str None = None; sequence_index: int None = None; transfer_type: str None = None; signal_or_claim: Any = None; projected_pv: Any = None; projected_pp_effect: Any = None; realized_pv: Any = None; revealed_pp_effect: Any = None; surrendered_pv: Any = None; domain_effects: Mapping[str, Any] = (empty); commitment_effect: Any = None; interpretation: Any = None; memory_status: Any = None; expectation_update: Any = None; quality_markers: Any = None; temporal_provenance: InformationTemporalProvenance None = None; provenance: Any = None; metadata: Mapping[str, Any] = (empty)
Source	models.py:654
Evidence	Direct use: test files 1; named in doc 05; e.g. test_v2_1_transfer_history_v0136.py.

Record — TransferHistoryReference

Field	Content
Label	PUBLIC / ADVANCED (rule R6)
Fields	frozen dataclass; actor_id: str; transfer_history_id: str; time: float; version: Any = None; metadata: Mapping[str, Any] = (empty)
Source	models.py:685
Evidence	Direct use: test files 2; named in doc 05; e.g. test_v2_1_replay_sufficiency_v0137.py, test_v2_1_transfer_history_v0136.py.

Record — TransferHistoryPackage

Field	Content
Label	PUBLIC / ADVANCED (rule R6: returned by the host history service)
Fields	frozen dataclass; actor_id: str; prior_history: TransferHistoryReference None; next_history: TransferHistoryReference; events: tuple[TransferHistoryEvent, ...] = (); notes: tuple[str, ...] = ()
Source	models.py:695
Evidence	Direct use: test files 1; named in doc 05; e.g. test_v2_1_transfer_history_v0136.py.

Entry — PVPPRuntime.record_transfer_history

Field	Content
Name and label	PVPPRuntime.record_transfer_history — method; kernel.py:2129. PUBLIC / ADVANCED (rule R2).
Purpose	Validates new events, sends them to the host history service, and validates the package it returns.
Framework basis	Transfer History object (C10/D22, Version 2.1).
Signature	PVPPRuntime.record_transfer_history(prior_history: TransferHistoryReference None, events: tuple[TransferHistoryEvent, ...]) → tuple[TransferHistoryPackage, TransferHistoryValidationAssessment]
Preconditions	A TransferHistoryService is attached; at least one event; all events belong to the first event's actor; any prior history belongs to the same actor.
Result	A (TransferHistoryPackage, TransferHistoryValidationAssessment) pair.
Invariants	Events are validated before the host sees them, and the returned package is validated before it is returned.
Failure behavior	Raises RuntimeError if no service is attached; ValueError for an invalid event, actor mismatch, or invalid package.
Does not	Store history itself, or convert event existence into memory, PP, or authority.

Field	Content
Evidence	Direct use: test files 1; named in doc 05; e.g. test_v2_1_transfer_history_v0136.py.
See also	Guide 4.9.

Entry — `PVPPRuntime.validate_transfer_history_event`

Field	Content
Name and label	<code>PVPPRuntime.validate_transfer_history_event</code> — method; kernel.py:2064. PUBLIC / ADVANCED (rule R2).
Purpose	Checks one transfer-history event.
Framework basis	Transfer History object (C10/D22).
Signature	<code>PVPPRuntime.validate_transfer_history_event(event: TransferHistoryEvent, *, expected_actor_id: str None = None)</code>
Preconditions	None.
Result	A tuple of violation strings; empty means valid.
Invariants	Required: event_id and actor_id; the expected actor when one is given; a non-negative integer sequence index when supplied; temporal provenance of the right type when supplied.
Failure behavior	Returns violations. Raises <code>ValueError</code> if the object is not a <code>TransferHistoryEvent</code> .
Does not	Interpret the event's transfer, realization, or effect fields.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_1_transfer_history_v0136.py.
See also	6.3.

Entry — `PVPPRuntime.validate_transfer_history_package`

Field	Content
Name and label	<code>PVPPRuntime.validate_transfer_history_package</code> — method; kernel.py:2080. PUBLIC / ADVANCED (rule R2).
Purpose	Checks a history package returned by the host service.
Framework basis	Transfer History object (C10/D22).
Signature	<code>PVPPRuntime.validate_transfer_history_package(package: TransferHistoryPackage, *, expected_actor_id: str None = None, prior_history: TransferHistoryReference None = None) → TransferHistoryValidationAssessment</code>
Preconditions	None. Pass the expected actor and prior history for full checking.
Result	<code>TransferHistoryValidationAssessment</code> : valid, actor, next history id, event ids, violations.

Field	Content
Invariants	Required: a next-history reference with the package's actor, an id, and a finite time; the prior history unchanged; history time never moving backward; every event valid; event ids unique; supplied sequence indexes in order.
Failure behavior	Returns violations. Raises ValueError if the object is not a TransferHistoryPackage.
Does not	Treat recorded events as current state or as authorization.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_1_transfer_history_v0136.py.
See also	6.3.

6.4 Replay Sufficiency

A replay package reconstructs a decision actually made, from decision-time material only (C45/D9, Version 2.1). It is audit evidence. It never becomes live state, and it never licenses a later decision or execution.

Frozen-code trap: `build_decision_replay_package` defaults `runtime_version` to "0.137". `v0.141` callers must pass `runtime_version="0.141"`; otherwise the package can identify the wrong runtime generation (15.6).

Entry — [PVPPRuntime.build_decision_replay_package](#)

Field	Content
Name and label	PVPPRuntime.build_decision_replay_package — method; kernel.py:5253. PUBLIC / ADVANCED (rule R2).
Purpose	Assembles a replay package from artifacts that already exist for one decision episode.
Framework basis	Replay sufficiency and historical decision reconstruction (C45/D9).
Signature	PVPPRuntime.build_decision_replay_package(*, decision_episode_id: str, snapshot: CanonicalCycleSnapshot, decision: CanonicalDecisionCycleAssessment, framework_version: str = 2.1, runtime_version: str = 0.137, retrieval: MemoryRetrievalPackage None = None, expectation_state: ExpectationStateReference None = None, transfer_history_references=(), execution_license=None, epsilon_result=None, transition_result=None, final_actual_state=None, runtime_configuration=None, source_snapshot_ids=(), governing_rule_ids=(), stop_condition=None, temporal_provenance=None, metadata=None) → DecisionReplayPackage
Preconditions	A CanonicalCycleSnapshot and the CanonicalDecisionCycleAssessment it produced; optional retrieval, expectation, history references, license, ϵ result, transition result, final state, and configuration provenance.
Result	A DecisionReplayPackage that references the supplied artifacts.

Field	Content
Invariants	Actor and decision time come from the snapshot's actual state. The method copies references; it re-runs nothing.
Failure behavior	Raises ValueError if the snapshot or decision is of the wrong type.
Does not	Re-run the decision, fill gaps from current memory, or authorize anything.
Caution	The keyword <code>runtime_version</code> defaults to "0.137", the build in which replay was added, and <code>framework_version</code> defaults to "2.1". Pass <code>runtime_version="0.141"</code> explicitly. If a configuration provenance record is included, <code>assess_replay_sufficiency</code> flags a mismatch; if none is included, the package silently records the wrong runtime version. Recorded in 15.6.
Evidence	Direct use: test files 1; named in doc 05; e.g. <code>test_v2_1_replay_sufficiency_v0137.py</code> .
See also	Guide 4.11, 9.11.

Entry — `PVPPRuntime.assess_replay_sufficiency`

Field	Content
Name and label	<code>PVPPRuntime.assess_replay_sufficiency</code> — method; <code>kernel.py:5183</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Checks that a replay package contains the minimum decision-time material and nothing from later.
Framework basis	Replay sufficiency (C45/D9).
Signature	<code>PVPPRuntime.assess_replay_sufficiency(package: DecisionReplayPackage) → ReplaySufficiencyAssessment</code>
Preconditions	None.
Result	<code>ReplaySufficiencyAssessment</code> : sufficient, missing material, violations.
Invariants	Required: episode id, actor, decision time, framework and runtime versions; actual state or a reference to it; the perceived decision state; the decision. Nothing in the package may be later than the decision time or belong to another actor. ε evidence requires its license; transition evidence requires ε evidence. A cycle that stopped anywhere other than a completed Σ selection needs an explicit <code>stop_condition</code> .
Failure behavior	Returns missing material and violations; raises ValueError only for the wrong type.
Does not	Make retained material live state; substitute later knowledge for decision-time artifacts.
Evidence	Direct use: test files 1; named in doc 05; e.g. <code>test_v2_1_replay_sufficiency_v0137.py</code> .
See also	Guide 4.11–4.12, 9.11.

6.5 Prediction Error

After a terminal ϵ result and a validated Layer-1 transition, the host may compare what was expected with what happened. v0.141 builds a typed request and validates the host's classification. Mismatches keep their type—transition, source, policy outcome, timing, or interaction response—and no scalar surprise score is imposed. Prediction error about capability formation belongs to the change surface (Chapter 8).

Entry — `PVPPRuntime.build_prediction_error_request`

Field	Content
Name and label	<code>PVPPRuntime.build_prediction_error_request</code> — method; kernel.py:2493. PUBLIC / ADVANCED (rule R2).
Purpose	Builds a typed comparison request for one completed execution episode.
Framework basis	Projection, expectation, and prediction-error interface (C10/D1 §3; C10/D16).
Signature	<code>PVPPRuntime.build_prediction_error_request(prior_actual_state: ActualPersistentStateEnvelope, epsilon_result: EpsilonStepResult, transition_result: Layer1TransitionResult, prior_expectation_state: ExpectationStateReference None = None, selected_projection_record: PolicyProjectionRecord None = None, *, metadata=None) → PredictionErrorRequest</code>
Preconditions	A terminal ϵ result; a Layer-1 transition result that validates against the handoff; any prior expectation state belongs to the actor and is not future-dated; any projection record is for the selected policy.
Result	<code>PredictionErrorRequest</code> carrying identities, status, path, realized bundles, execution information, and execution-transition provenance.
Invariants	The request is built only from the validated transition and its provenance.
Failure behavior	Raises <code>ValueError</code> for each unmet precondition.
Does not	Compute error, update memory or expectation, or change state.
Evidence	Direct use: test files 3; not named in doc 05; e.g. <code>test_cross_object_execution_identity_v068.py</code> , <code>test_execution_evidence_integrity_v067.py</code> .
See also	11.10–11.11.

Entry — `PVPPRuntime.apply_prediction_error`

Field	Content
Name and label	<code>PVPPRuntime.apply_prediction_error</code> — method; kernel.py:2579. PUBLIC / ADVANCED (rule R2).
Purpose	Builds the request, calls the host <code>PredictionErrorService</code> , and validates its answer.
Framework basis	As above.

Field	Content
Signature	PVPPRuntime.apply_prediction_error(prior_actual_state: ActualPersistentStateEnvelope, epsilon_result: EpsilonStepResult, transition_result: Layer1TransitionResult, prior_expectation_state: ExpectationStateReference None = None, selected_projection_record: PolicyProjectionRecord None = None, *, metadata=None)
Preconditions	A PredictionErrorService is attached, and the preconditions of build_prediction_error_request hold.
Result	A (request, assessment, validation) triple.
Invariants	The host's assessment is returned only if it validates.
Failure behavior	Raises ValueError if no service is attached or the assessment is invalid.
Does not	Apply the error to memory, expectation, or any stage.
Evidence	Direct use: test files 2, benchmarks 1; not named in doc 05; e.g. benchmark_prediction_error_v052.py.
See also	6.6.

Entry — PVPPRuntime.validate_prediction_error_assessment

Field	Content
Name and label	PVPPRuntime.validate_prediction_error_assessment — method; kernel.py:2528. PUBLIC / ADVANCED (rule R2).
Purpose	Checks a host prediction-error assessment against its request.
Framework basis	As above.
Signature	PVPPRuntime.validate_prediction_error_assessment(request: PredictionErrorRequest, assessment: PredictionErrorAssessment) → PredictionErrorValidationAssessment
Preconditions	None.
Result	PredictionErrorValidationAssessment.
Invariants	Actor, episode, and policy match the request; the request's execution-transition provenance matches its identities, status, path, evidence signatures, and return posture; every component has class transition, source, policy_outcome, timing, or interaction_response; material_mismatch is true exactly when a component is material.
Failure behavior	Returns violations. Raises ValueError if the assessment has the wrong type.
Does not	Impose a scalar score or a learning formula.
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_execution_evidence_integrity_v067.py, test_execution_evidence_provenance_v066.py.
See also	6.5.

6.6 Post-Execution Epistemic Update

The host may update retained memory, expectation state, both, or neither from a realized outcome, through `EpistemicUpdateService`. The runtime requires a validated Layer-1 transition first, so execution information is never treated as a state change or as a silently committed memory update before the realized transition is known. It validates attribution and time continuity; selection, consolidation, forgetting, and expectation learning remain host semantics. `EpistemicUpdateService` is not exported at the top level (H.5-1).

Entry — [PVPPRuntime.build_post_execution_epistemic_update_request](#)

Field	Content
Name and label	<code>PVPPRuntime.build_post_execution_epistemic_update_request</code> — method; kernel.py:2598. PUBLIC / ADVANCED (rule R2).
Purpose	Builds the realized-outcome handoff for a host memory and expectation update.
Framework basis	Memory interface and pipeline integration (C10/D16); ε outputs feed update surfaces (C15/D9 §10).
Signature	<code>PVPPRuntime.build_post_execution_epistemic_update_request(prior_actual_state: ActualPersistentStateEnvelope, epsilon_result: EpsilonStepResult, transition_result: Layer1TransitionResult, prior_memory_state: MemoryStateReference, prior_expectation_state: ExpectationStateReference None = None, *, metadata=None, prediction_error: PredictionErrorAssessment None = None) → PostExecutionEpistemicUpdateRequest</code>
Preconditions	A terminal ε result; a transition result that validates; a valid prior memory reference for the actor; any prior expectation state belongs to the actor, is finite, and is not future-dated.
Result	<code>PostExecutionEpistemicUpdateRequest</code> , including execution-transition provenance and any typed prediction error.
Invariants	Built only after the Layer-1 transition validates.
Failure behavior	Raises <code>ValueError</code> for each unmet precondition.
Does not	Update memory or expectation itself.
Evidence	Direct use: test files 3; not named in doc 05; e.g. <code>test_cross_object_execution_identity_v068.py</code> , <code>test_execution_evidence_integrity_v067.py</code> .
See also	11.10–11.11.

Entry — [PVPPRuntime.apply_post_execution_epistemic_update](#)

Field	Content
Name and label	<code>PVPPRuntime.apply_post_execution_epistemic_update</code> — method; kernel.py:2751. PUBLIC / ADVANCED (rule R2).

Field	Content
Purpose	Builds the request, calls the host EpistemicUpdateService, and validates the result.
Framework basis	As above.
Signature	PVPPRuntime.apply_post_execution_epistemic_update(prior_actual_state: ActualPersistentStateEnvelope, epsilon_result: EpsilonStepResult, transition_result: Layer1TransitionResult, prior_memory_state: MemoryStateReference, prior_expectation_state: ExpectationStateReference None = None, *, metadata=None, prediction_error: PredictionErrorAssessment None = None)
Preconditions	An EpistemicUpdateService is attached.
Result	A (request, result, validation) triple.
Invariants	The host's result is returned only if it validates.
Failure behavior	Raises ValueError if no service is attached or the result is invalid.
Does not	Decide what is learned or forgotten.
Evidence	Direct use: test files 2, benchmarks 1; not named in doc 05; e.g. benchmark_post_execution_epistemic_update_v044.py.
See also	6.6.

Entry — [PVPPRuntime.validate_post_execution_epistemic_update_result](#)

Field	Content
Name and label	PVPPRuntime.validate_post_execution_epistemic_update_result — method; kernel.py:2658. PUBLIC / ADVANCED (rule R2).
Purpose	Checks a host update result for attribution and time continuity.
Framework basis	As above.
Signature	PVPPRuntime.validate_post_execution_epistemic_update_result(request: PostExecutionEpistemicUpdateRequest, result: PostExecutionEpistemicUpdateResult) → EpistemicUpdateValidationAssessment
Preconditions	None.
Result	EpistemicUpdateValidationAssessment.
Invariants	Identity and provenance match the request, including evidence signatures and any prediction-error attribution. If memory_updated, the next memory has a new id and the next actual state's time; if not, memory is unchanged. The same holds for expectation state. Memory and expectation may change independently.
Failure behavior	Returns violations. Raises ValueError if the result has the wrong type.
Does not	Define learning semantics.

Field	Content
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_execution_evidence_integrity_v067.py, test_execution_evidence_provenance_v066.py.
See also	6.6.

6.7 Evidence Sources and Evidence Authority

A governance fact can depend on an evidence source. If the source is later compromised, revoked, stale, expired, or controlled by the governed agent itself, the fact loses epistemic authority. These functions live in the re-entry module and are exported at the top level. They qualify evidence; they do not decide which stages are invalidated (Chapter 12).

Record — EvidenceSourceAuthorityState

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; source_id: str; provenance_id: str; assessed_at: float; authority_state: str; control_state: str; valid_until: float None = None; evidence: Mapping[str, Any] = (empty); notes: tuple[str, ...] = ()
Source	models.py:3019
Evidence	Direct use: test files 4; not named in doc 05; e.g. test_v0124_control_provenance_topology.py, test_v0125_control_provenance_topology_lifecycle.py.

Record — GovernanceEvidenceDependency

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; dependency_id: str; fact_id: str; observation_id: str; source_id: str; source_provenance_id: str; dependency_role: str = 'supporting'; notes: tuple[str, ...] = ()
Source	models.py:3031
Evidence	Direct use: test files 3; not named in doc 05; e.g. test_v0124_control_provenance_topology.py, test_v2_epistemic_orchestration_v0113.py.

Entry — assess_evidence_dependency_authority

Field	Content
Name and label	assess_evidence_dependency_authority — function; reentry.py:1608. PUBLIC / ADVANCED (rule R2).
Purpose	Decides whether a fact-to-source dependency still carries epistemic authority.

Field	Content
Framework basis	Source authority and provenance in perception and memory (C10/D19; C10/D16). Implemented in runtime builds v0.111–v0.112.
Signature	<code>assess_evidence_dependency_authority(dependency, source_state, *, as_of=None)</code>
Preconditions	The dependency and the current state of its source; optionally an <code>as_of</code> time.
Result	<code>EvidenceDependencyAuthorityAssessment</code> : <code>valid</code> , <code>retains_epistemic_authority</code> , and <code>loss_reason</code> .
Invariants	Loss reasons, in order of precedence: <code>source_compromised</code> , <code>source_revoked</code> , or <code>source_stale</code> ; <code>source_controlled_by_governed_agent</code> or <code>source_controlled_by_descendant</code> ; <code>source_expired</code> (<code>as_of</code> after <code>valid_until</code>); <code>source_authority_unresolved</code> (degraded authority or unknown control). Allowed values: role <code>primary</code> , <code>supporting</code> , or <code>corroborating</code> ; authority <code>trusted</code> , <code>degraded</code> , <code>compromised</code> , <code>revoked</code> , or <code>stale</code> ; control <code>independent</code> , <code>governed_agent</code> , <code>descendant</code> , or <code>unknown</code> .
Failure behavior	Returns violations; raises nothing.
Does not	Infer which canonical stage is invalidated, or create re-entry authority.
Evidence	Direct use: test files 2; not named in doc 05; e.g. <code>test_v0124_control_provenance_topology.py</code> , <code>test_v2_evidence_source_dependency_v0111.py</code> .
See also	12.2 (<code>derive_evidence_authority_governance_invalidations</code>).

Entry — `qualify_evidence_source_from_control_topology`

Field	Content
Name and label	<code>qualify_evidence_source_from_control_topology</code> — function; <code>reentry.py:2202</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Sets a source's control state from a validated control-provenance topology.
Framework basis	Source authority and provenance (C10/D19; C10/D16); runtime construct of builds v0.124–v0.126 (6.8).
Signature	<code>qualify_evidence_source_from_control_topology(source_state, topology_assessment, *, governed_agent_ids=())</code>
Preconditions	A valid topology assessment.
Result	A copy of the source state with <code>control_state</code> set to <code>"descendant"</code> , <code>"governed_agent"</code> , or <code>"independent"</code> , and a note saying why.
Invariants	Only <code>control_state</code> changes; authority and provenance are preserved.
Failure behavior	Raises <code>ValueError</code> if the topology assessment is invalid.
Does not	Infer control from names or anything outside the topology.

Field	Content
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_v0124_control_provenance_topology.py, test_v0125_control_provenance_topology_lifecycle.py.
See also	6.8.

6.8 Control-Provenance Topology

When an agent governed by the Productive Value–Productive Power (PV-PP) framework can create, control, or spawn the sources whose evidence governs it, that evidence is not independent. v0.141 lets the host state the relevant control and provenance edges explicitly, derives the bounded facts that follow—what the governed agent controls, what descends from it—and tracks changes to the topology as versioned snapshots. It discovers no edges and infers nothing from names. No V2.1 owner file defines control-provenance topology as a framework object: it is a runtime construct, introduced in builds v0.124–v0.126, that serves the source-authority rules of the perception and memory owner files. Chapter 15 lists it among runtime constructs without dedicated framework ownership.

Record — ControlProvenanceRelation

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; relation_id: str; subject_id: str; object_id: str; relation_type: str; observed_at: float; source_id: str; provenance_id: str; active: bool = True; evidence_ids: tuple[str, ...] = (); notes: tuple[str, ...] = ()
Source	models.py:3388
Evidence	Direct use: test files 3; not named in doc 05; e.g. test_v0124_control_provenance_topology.py, test_v0125_control_provenance_topology_lifecycle.py.

Record — ControlProvenanceTopologyUpdateRequest

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; update_id: str; prior_topology_identity: str; topology_id: str; as_of: float; relations: tuple[ControlProvenanceRelation, ...]; governed_agent_ids: tuple[str, ...]; configuration_version: str; source_id: str; provenance_id: str; rationale: str
Source	models.py:3415
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_v0125_control_provenance_topology_lifecycle.py, test_v0126_control_provenance_governance_dependencies.py.

Entry — `assess_control_provenance_topology`

Field	Content
Name and label	<code>assess_control_provenance_topology</code> — function; <code>reentry.py:2158</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Validates a set of control and provenance edges and derives what governed agents control.
Framework basis	Runtime construct of build v0.124 serving the source-authority rules of C10/D19 and C10/D16 (6.8).
Signature	<code>assess_control_provenance_topology(relations, *, topology_id, as_of, governed_agent_ids=())</code>
Preconditions	The relations, a topology id, an as_of time, and the governed agent ids.
Result	<code>ControlProvenanceTopologyAssessment</code> : valid, relations, objects controlled by a governed agent, descendant objects.
Invariants	Relation types: creates, controls, delegates, revokes, terminates, descends_from. No self-reference; no observation after as_of; unique relation and evidence ids. Among active relations, an object is controlled if a governed agent creates, controls, or delegates it; descendants close transitively over descends_from from the agents; every descendant counts as controlled.
Failure behavior	Returns violations; raises nothing.
Does not	Invalidate any stage or create execution authority.
Evidence	Direct use: test files 3; not named in doc 05; e.g. <code>test_v0124_control_provenance_topology.py</code> , <code>test_v0125_control_provenance_topology_lifecycle.py</code> .
See also	6.7; Guide 10.

Entry — `update_control_provenance_topology`

Field	Content
Name and label	<code>update_control_provenance_topology</code> — function; <code>reentry.py:2236</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Replaces a topology snapshot with a new one, bound to the exact prior identity, and reports the change.
Framework basis	As above.
Signature	<code>update_control_provenance_topology(prior_assessment, request)</code>
Preconditions	A valid prior assessment; a request whose <code>prior_topology_identity</code> equals the prior snapshot's identity, whose as_of does not move backward, and whose identifying fields are non-empty.
Result	<code>ControlProvenanceTopologyUpdateAssessment</code> with status "UPDATED", "NO_CHANGE", or "REJECTED", the new identity, and the relation ids added, removed, and changed.

Field	Content
Invariants	An update is accepted only against the exact current snapshot.
Failure behavior	A rejected update returns status REJECTED with violations; nothing is raised.
Does not	Invalidate anything. Callers route changed control through 6.7 and Chapter 12.
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_v0125_control_provenance_topology_lifecycle.py, test_v0126_control_provenance_governance_dependencies.py.
See also	12.2 (derive_control_provenance_governance_invalidations).

Entry — control_provenance_topology_identity

Field	Content
Name and label	control_provenance_topology_identity — function; reentry.py:2219. PUBLIC / ADVANCED (rule R2).
Purpose	Returns a deterministic fingerprint of a validated topology snapshot.
Framework basis	As above.
Signature	control_provenance_topology_identity(topology_assessment)
Preconditions	A valid topology assessment.
Result	A hexadecimal SHA-256 string.
Invariants	Relation order does not affect the fingerprint.
Failure behavior	Raises ValueError if the assessment is invalid.
Does not	Carry semantic authority; it is an integrity token.
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_v0125_control_provenance_topology_lifecycle.py, test_v0126_control_provenance_governance_dependencies.py.
See also	4.8 for the registry fingerprints.

6.9 Security Evidence

Record — SecurityEvidenceObservation

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; observation_id: str; fact_id: str; value: Any; source_id: str; provenance_id: str; observed_at: float; authority_status: str; confidence: float None = None; distortion_risk: str = 'unknown'; independent_confirmation_ids: tuple[str, ...] = (); evidence: Mapping[str, Any] = (empty); notes: tuple[str, ...] = ()
Source	models.py:2978

Field	Content
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_security_evidence_v0110.py.

Entry — `assess_security_evidence_observation`

Field	Content
Name and label	<code>assess_security_evidence_observation</code> — function; <code>reentry.py:1575</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Qualifies a security-critical observation by its epistemic metadata.
Framework basis	Evidence quality and source authority (C10/D16; C10/D19); runtime construct of build v0.110.
Signature	<code>assess_security_evidence_observation(observation)</code>
Preconditions	None.
Result	<code>SecurityEvidenceAssessment</code> : valid, <code>usable_as_governance_evidence</code> , <code>requires_independent_confirmation</code> , and the metadata.
Invariants	Only "authoritative" or "corroborated" evidence is usable. "provisional" evidence is never usable directly: without confirmation ids it requires independent confirmation, and with them it is still not usable, because confirmation ids are links, not self-validating authority. "disputed" and "untrusted" evidence is kept for traceability but not usable. Confidence, if given, lies in [0, 1]; distortion risk is low, medium, high, or unknown.
Failure behavior	Returns violations; raises nothing.
Does not	Create feasibility, adequacy, Σ , ε , execution, or re-entry authority.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_security_evidence_v0110.py.
See also	Guide 4.4.

6.10 Classification Record and Compact Rows

The four host services of this chapter—`MemoryRetrievalService`, `EpistemicUpdateService`, `TransferHistoryService`, and `PredictionErrorService`—are PUBLIC / ADVANCED as optional services for specialized paths; the constructor accepts them but no ordinary cycle requires them. Their primary entries are in Chapter 10. This settles the two protocols left open in H.5-1. Records the runtime builds are IMPLEMENTATION / SUPPORT (R3); records the host builds for these entries are PUBLIC / ADVANCED (R6). A PUBLIC label describes the runtime API, not framework status: control-provenance topology (6.8) and security-evidence qualification (6.9) are PUBLIC / ADVANCED runtime constructs, and neither label makes them framework primitives or gives them a V2.1 owner file.

Table 6.1 — Labels assigned in Chapter 6

Symbol	Label	Rule
<code>PVPPRuntime.validate_information_governance_metadata</code>	PUBLIC / ADVANCED	R2

Symbol	Label	Rule
PVPPRuntime.validate_memory_state_reference	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_memory_retrieval_package	PUBLIC / ADVANCED	R2
PVPPRuntime.record_transfer_history	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_transfer_history_event	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_transfer_history_package	PUBLIC / ADVANCED	R2
PVPPRuntime.build_decision_replay_package	PUBLIC / ADVANCED	R2
PVPPRuntime.assess_replay_sufficiency	PUBLIC / ADVANCED	R2
PVPPRuntime.build_prediction_error_request	PUBLIC / ADVANCED	R2
PVPPRuntime.apply_prediction_error	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_prediction_error_assessment	PUBLIC / ADVANCED	R2
PVPPRuntime.build_post_execution_epistemic_update_request	PUBLIC / ADVANCED	R2
PVPPRuntime.apply_post_execution_epistemic_update	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_post_execution_epistemic_update_result	PUBLIC / ADVANCED	R2
assess_evidence_dependency_authority	PUBLIC / ADVANCED	R2
qualify_evidence_source_from_control_topology	PUBLIC / ADVANCED	R2
assess_control_provenance_topology	PUBLIC / ADVANCED	R2
update_control_provenance_topology	PUBLIC / ADVANCED	R2
control_provenance_topology_identity	PUBLIC / ADVANCED	R2
assess_security_evidence_observation	PUBLIC / ADVANCED	R2

Symbol	Label	Rule
interfaces.MemoryRetrievalService	PUBLIC / ADVANCED	R2 (optional host service)
interfaces.EpistemicUpdateService	PUBLIC / ADVANCED	R2 (optional host service)
TransferHistoryService	PUBLIC / ADVANCED	R2 (optional host service)
PredictionErrorService	PUBLIC / ADVANCED	R2 (optional host service)
InformationGovernanceMetadata	PUBLIC / ADVANCED	R6
InformationTemporalProvenance	PUBLIC / ADVANCED	R6
MemoryStateReference	PUBLIC / ADVANCED	R6
MemoryRetrievalPackage	PUBLIC / ADVANCED	R6
RetrievalQualityMetadata	PUBLIC / ADVANCED	R6
ExpectationStateReference	PUBLIC / ADVANCED	R6
TransferHistoryEvent	PUBLIC / ADVANCED	R6
TransferHistoryReference	PUBLIC / ADVANCED	R6
TransferHistoryPackage	PUBLIC / ADVANCED	R6
PredictionErrorAssessment	PUBLIC / ADVANCED	R6
PredictionErrorComponent	PUBLIC / ADVANCED	R6
PostExecuteEpistemicUpdateResult	PUBLIC / ADVANCED	R6

Symbol	Label	Rule
SecurityEvidenceObservation	PUBLIC / ADVANCED	R6
EvidenceSourceAuthorityState	PUBLIC / ADVANCED	R6
GovernanceEvidenceDependency	PUBLIC / ADVANCED	R6
ControlProvenanceRelation	PUBLIC / ADVANCED	R6
ControlProvenanceTopologyUpdateRequest	PUBLIC / ADVANCED	R6
InformationGovernanceAssessment	IMPLEMENTATIO N / SUPPORT	R3
MemoryRetrievalRequest	IMPLEMENTATIO N / SUPPORT	R3
TransferHistoryValidationAssessment	IMPLEMENTATIO N / SUPPORT	R3
DecisionReplayPackage	IMPLEMENTATIO N / SUPPORT	R3
ReplaySufficiencyAssessment	IMPLEMENTATIO N / SUPPORT	R3
PredictionErrorRequest	IMPLEMENTATIO N / SUPPORT	R3
PredictionErrorValidationAssessment	IMPLEMENTATIO N / SUPPORT	R3
PostExecuteEpistemicUpdateRequest	IMPLEMENTATIO N / SUPPORT	R3
EpistemicUpdateValidationAssessment	IMPLEMENTATIO N / SUPPORT	R3
SecurityEvidenceAssessment	IMPLEMENTATIO N / SUPPORT	R3
EvidenceDependencyAuthorityAssessment	IMPLEMENTATIO N / SUPPORT	R3
ControlProvenanceTopologyAssessment	IMPLEMENTATIO N / SUPPORT	R3
ControlProvenanceTopologyUpdateAssessment	IMPLEMENTATIO N / SUPPORT	R3

Table 6.2 — Compact rows for runtime-built records

Name and kind	Label	Role and use	Source
InformationGovernanceAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>validate_information_governance_metadata</code>	<code>models.py</code> :618
MemoryRetrievalRequest — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Built by the memory-conditioned snapshot for the host retrieval service	<code>models.py</code> :627
TransferHistoryValidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>validate_transfer_history_package</code>	<code>models.py</code> :705
DecisionReplayPackage — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Built by <code>build_decision_replay_package</code> ; checked by <code>assess_replay_sufficiency</code>	<code>models.py</code> :2244
ReplaySufficiencyAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>assess_replay_sufficiency</code>	<code>models.py</code> :2277
PredictionErrorRequest — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Built by <code>build_prediction_error_request</code> for the host service	<code>models.py</code> :736
PredictionErrorValidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>validate_prediction_error_assessment</code>	<code>models.py</code> :768
PostExecuteEpistemicUpdateRequest — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Built by <code>build_post_execution_epistemic_update_request</code>	<code>models.py</code> :778
EpistemicUpdateValidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>validate_post_execution_epistemic_update_result</code>	<code>models.py</code> :816
SecurityEvidenceAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>assess_security_evidence_observation</code>	<code>models.py</code> :2998
EvidenceDependencyAuthorityAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>assess_evidence_dependency_authority</code>	<code>models.py</code> :3046
ControlProvenanceTopologyAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>assess_control_provenance_topology</code>	<code>models.py</code> :3402
ControlProvenanceTopologyUpdateAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>update_control_provenance_topology</code>	<code>models.py</code> :3428

Chapter 7 — Objectives

Version 2.1 gives purpose a typed owner: $O_i(t)$, the set of objectives an actor is currently trying to bring about, maintain, prevent, or complete (C10/D9). This chapter documents how v0.141 carries objectives, how it records the relation between candidate policies and objectives, how objectives may direct discovery, and how an objective change invalidates only what explicitly depended on it. Objectives are the most tempting place in the architecture to hide a utility function. The chapter is organized around the rules that prevent that.

A reader must be able to supply objectives without turning them into hidden utility, and know which calls reject an attempt to do so.

7.1 The Objective Contract

$O_i(t)$ is an adjacent typed input beside $P_i(t)$. It is not a component of perceived state, not part of actual state, and not a stage (C10/D9 §2, §8; C20/D3). The owner file lists the distinctions that keep it that way. Table 7.1 restates them with the v0.141 behavior that respects each one.

Table 7.1 — What an objective is not, and how v0.141 keeps it so

Distinction (C10/D9 §3)	v0.141 behavior
Not viability, capability, or a viability domain	Objectives never enter Φ , H , G , or $\mathcal{R}$; an objective-only change preserves those stages (7.6)
Not utility, a preference weight, or a scalar value	Objective records carry no weight or score field, and no stage reads objectives as a score
Relevance is not authorization or permitted information use	Discovery hints may reference only registered graph structure; retrieval cues carry no authority (7.5)
Responsiveness is not feasibility, adequacy, authority, viability, ranking, or selection	Responsiveness records are descriptive and are never fed back into Π , Constraints, Adequacy, Σ , ϵ , or authorization (7.4)
Attainment is not viability; representation is not endorsement	The validator checks shape, identity, lifecycle, and time only; it endorses nothing
No invented ordering among objectives	Conflicting active objectives may coexist; the runtime orders none

Objective-free operation remains valid: when no objective set is supplied, the objective validator passes, responsiveness is not required, and the cycle runs unchanged.

7.2 Objective Sets

An `ObjectiveSet` holds the active qualified objectives for one actor at one decision time. Each objective is a `QualifiedObjectiveRecord` with the eight elements the owner file requires: id, content, source, scope, status, temporal bounds, relations, and authority (C10/D9 §4). The host builds the set and passes it to a cycle entry (9.3). Only `prepare_canonical_cycle_snapshot`, and therefore `evaluate_integrated_canonical_cycle`, validates the set. A host that passes objectives to `evaluate_canonical_decision_cycle` directly must call `validate_objective_set` itself: that entry validates discovery hints against the set but not the set.

Record — ObjectiveSet

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: built by the host as an argument of the cycle entries)
Fields	frozen dataclass; actor_id: str; time: float; objectives: tuple[QualifiedObjectiveRecord, ...] = (); objective_set_id: str = ''; version: str = '1'; metadata: Mapping[str, Any] = (empty)
Source	models.py:317
Evidence	Direct use: test files 2, benchmarks 1; named in doc 05; e.g. benchmark_small_economy_v2_1_v0140.py.

Record — QualifiedObjectiveRecord

Field	Content
Label	PUBLIC / SUPPORTED (rule R6)
Fields	frozen dataclass; id: str; content: Any; source: str; scope: Any; status: str = 'active'; temporal_bounds: ObjectiveTemporalBounds None = None; relations: tuple[ObjectiveRelation, ...] = (); authority: Any = None; version: str = '1'; metadata: Mapping[str, Any] = (empty)
Source	models.py:298
Evidence	Direct use: test files 2, benchmarks 1; named in doc 05; e.g. benchmark_small_economy_v2_1_v0140.py.

Record — ObjectiveTemporalBounds

Field	Content
Label	PUBLIC / SUPPORTED (rule R6)
Fields	frozen dataclass; start: float None = None; deadline: float None = None; horizon: float None = None; expiration: float None = None; metadata: Mapping[str, Any] = (empty)
Source	models.py:275
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_1_objective_interface_v0133.py.

Record — ObjectiveRelation

Field	Content
Label	PUBLIC / SUPPORTED (rule R6)
Fields	frozen dataclass; related_objective_id: str; relation: str; metadata: Mapping[str, Any] = (empty)
Source	models.py:290
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_1_objective_interface_v0133.py.

Entry — `PVPPRuntime.validate_objective_set`

Field	Content
Name and label	<code>PVPPRuntime.validate_objective_set</code> — method; <code>kernel.py:2295</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Checks the shape, identity, lifecycle, and timing of an objective set against the current actual state.
Framework basis	Actor Purpose and Objective Interface v0.1 (C10/D9 §4–5, §13).
Signature	<code>PVPPRuntime.validate_objective_set(objectives: ObjectiveSet None, actual_state: ActualPersistentStateEnvelope) → ObjectiveSetValidationAssessment</code>
Preconditions	None. A set of None is valid and means objective-free operation.
Result	<code>ObjectiveSetValidationAssessment</code> : valid, actor, set id, violations.
Invariants	The set's actor matches the actual state and its time equals the decision time; the set has an id and a version. Each objective has a unique id, content, source, scope, authority, and version. Status must be one of active, suspended, completed, abandoned, or expired, and only "active" may remain in $O_i(t)$: a suspended, completed, abandoned, or expired objective in the set is a violation. Temporal bounds, when given, are finite; an objective whose start lies in the future, or whose expiration has passed, is a violation. Each relation names a related objective and a label. Labels are not limited to the four the owner file names, which it treats as non-exhaustive.
Failure behavior	Returns violations. Raises <code>ValueError</code> if the object is neither an <code>ObjectiveSet</code> nor <code>None</code> .
Does not	Interpret objective content, endorse objectives, rank them, resolve conflicts among them, or infer motivation.
Evidence	Direct use: test files 1, benchmarks 1; named in doc 05; e.g. <code>benchmark_small_economy_v2_1_v0140.py</code> .
See also	Guide 3.8.

7.3 Where Objectives Come From

The interfaces module defines `ObjectiveInterfaceAdapter`, a host protocol that supplies an objective set for an actual state. In v0.141 no runtime method calls it: the host produces the set, by this protocol or any other means, and passes it to the cycle. The same holds for `ObjectiveDiscoveryAdapter` (7.5). Both protocols are documented, and labeled, in Chapter 10. Historical objectives may persist in memory or transfer history, but their history must not reactivate them in $O_i(t)$ (C10/D9 §5); the lifecycle check of 7.2 enforces this for whatever set the host supplies.

7.4 Objective Responsiveness

For a candidate π and an objective o , the application may evaluate $\text{Resp}(\pi, o)$ as supports, neutral, conflicts, or unresolved. The framework owns the four labels; the application owns the

evidence (C10/D9 §7). A candidate may support an objective indirectly—by building capability, gathering information, waiting, or satisfying a prerequisite—so no rule may prune a candidate for not attaining an objective immediately. v0.141 evaluates responsiveness inside the canonical cycle, after Π completeness passes (3.1), through an `ObjectiveResponsivenessAdapter` the host passes to the cycle entry.

Record — `ObjectiveResponsivenessRecord`

Field	Content
Label	PUBLIC / ADVANCED (rule R6: returned by the host responsiveness adapter)
Fields	frozen dataclass; policy_id: str; objective_id: str; relation: str; evidence: Any = None; provenance: Any = None; metadata: Mapping[str, Any] = (empty)
Source	models.py:342
Evidence	Direct use: test files 1, benchmarks 1; named in doc 05; e.g. benchmark_small_economy_v2_1_v0140.py.

Entry — `PVPPRuntime.evaluate_objective_responsiveness`

Field	Content
Name and label	<code>PVPPRuntime.evaluate_objective_responsiveness</code> — method; kernel.py:3163. PUBLIC / ADVANCED (rule R2).
Purpose	Collects the host's $\text{Resp}(\pi, o)$ record for every candidate–objective pair and validates each one.
Framework basis	Objective Responsiveness (C10/D9 §7).
Signature	<code>PVPPRuntime.evaluate_objective_responsiveness(state, policy_space, objectives: ObjectiveSet None, adapter=None) → ObjectiveResponsivenessAssessment</code>
Preconditions	None. With no active objectives, or no adapter, the result is valid and empty.
Result	<code>ObjectiveResponsivenessAssessment</code> : valid, the records, violations.
Invariants	With an adapter, every pair of candidate and active objective is evaluated. Each record must name the right policy and objective and carry one of the four relations.
Failure behavior	Returns violations. Inside the canonical cycle, an invalid result raises <code>ValueError</code> .
Does not	Admit, exclude, rank, authorize, prove adequacy, establish viability, or select. The records are never read by any later stage.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_small_economy_v2_1_v0140.py.
See also	3.1; Guide 6.4.

7.5 Objective-Directed Discovery

Objectives may direct bounded discovery—which parts of the existing reachable structure are examined, and which retrieval cues are used—without admitting new structure (C10/D9 §8–9; C15/D11). v0.141 accepts discovery hints with the cycle entries and validates them before any stage runs. A hint may refer only to graph instances and transformations already registered, so desired structure cannot enter the graph through an objective (C10/D9 §10).

Record — ObjectiveDiscoveryHint

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: built by the host as an argument of the cycle entries)
Fields	frozen dataclass; objective_id: str; graph_transformation_ids: tuple[str, ...] = (); graph_instance_ids: tuple[str, ...] = (); retrieval_cues: tuple[Any, ...] = (); provenance: Any = None; metadata: Mapping[str, Any] = (empty)
Source	models.py:366
Evidence	Direct use: test files 1; named in doc 05; e.g. test_v2_1_objective_responsiveness_discovery_v0134.py.

Entry — PVPPRuntime.validate_objective_discovery_hints

Field	Content
Name and label	PVPPRuntime.validate_objective_discovery_hints — method; kernel.py:3126. PUBLIC / ADVANCED (rule R2).
Purpose	Checks that discovery hints refer to active objectives and to registered graph structure only.
Framework basis	Bounded objective relevance (C10/D9 §8–10).
Signature	PVPPRuntime.validate_objective_discovery_hints(objectives: ObjectiveSet None, hints=()) → ObjectiveDiscoveryAssessment
Preconditions	None.
Result	ObjectiveDiscoveryAssessment: valid, the set id, the number of hints, the referenced transformations and instances, the retrieval cues, and violations.
Invariants	Hints require an active objective set. Each hint names an active objective, and every referenced transformation and instance must already be registered. Retrieval cues are carried as application data.
Failure behavior	Returns violations. Inside the canonical cycle, invalid hints raise ValueError.
Does not	Admit graph structure, establish reachability, rank policies, or confer information-use authority on the cues.
Evidence	Direct use: test files 1; named in doc 05; e.g. test_v2_1_objective_responsiveness_discovery_v0134.py.
See also	4.7; Chapter 8.

7.6 Objective Lifecycle and Invalidation

Objectives change—they are completed, suspended, abandoned, replaced—independently of state. An objective-only change therefore cannot by itself invalidate $P_i(t)$ or the state-reading stages Φ , H , G , and $\mathcal{R}$. It can invalidate only artifacts the host registered as depending on the changed objective identities: discovery results, graph and seed visibility, Π artifacts, responsiveness records, and what follows from them. The general dependency and re-entry machinery is in Chapter 12; this entry is its objective-specific front door.

Record — ObjectiveLifecycleChange

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; change_id: str; actor_id: str; prior_objective_set_id: str; prior_objective_set_version: str; current_objective_set_id: str; current_objective_set_version: str; changed_objective_ids: tuple[str, ...]; reason: str; actual_state_changed: bool = False; perceived_state_changed: bool = False; evidence_ids: tuple[str, ...] = (); notes: tuple[str, ...] = ()
Source	models.py:2481
Evidence	Direct use: test files 1, benchmarks 1; named in doc 05; e.g. benchmark_small_economy_v2_1_v0140.py.

Entry — derive_objective_lifecycle_invalidations

Field	Content
Name and label	derive_objective_lifecycle_invalidations — function; reentry.py:2363. PUBLIC / ADVANCED (rule R2).
Purpose	Derives invalidations caused by an objective-only change, from explicitly registered dependencies.
Framework basis	Objective lifecycle (C10/D9 §5); re-entry is a runtime construct (15.6).
Signature	derive_objective_lifecycle_invalidations(dependencies: "tuple[GovernanceArtifactDependency, ...]", change: "ObjectiveLifecycleChange") → "ObjectiveLifecycleInvalidationAssessment"
Preconditions	The dependencies the host registered, and the change. The change must carry ids, a reason, prior and current set identities and versions that differ, and unique changed-objective and evidence ids. It must not report an actual or perceived state change as well.
Result	ObjectiveLifecycleInvalidationAssessment: the objective fact ids, matched dependencies, invalidation signals, the re-entry plan, and the preserved stages ("PPP", "Phi", "H", "G", "R").

Field	Content
Invariants	The change is translated into fact ids "objective_set:<id>:<version>" for the prior and current sets and "objective:<id>" for each changed objective, with fact kind "objective", and matched only against dependencies registered on those ids. A host that wants an artifact invalidated by objective change must register it on these ids when the artifact is produced.
Failure behavior	Returns violations. A change that also reports a state change is refused: that change must go through the ordinary dependency path (12.2).
Does not	Recompute any operator, create objective-derived authority, or infer invalidation from state.
Evidence	Direct use: test files 1, benchmarks 1; named in doc 05; e.g. benchmark_small_economy_v2_1_v0140.py.
See also	12.2–12.3; Guide 8.7.

7.7 Classification Record and Compact Rows

The objective records the host builds for the cycle entries are PUBLIC / SUPPORTED. The basis is the documented host contract, not the bare fact that the host constructs them: the owner file's Application Contract requires applications that model purpose to declare exactly these elements (C10/D9 §12); doc 05 documents `ObjectiveSet`, `QualifiedObjectiveRecord`, and `ObjectiveDiscoveryHint` as application inputs; and the cycle entries take them as named parameters. Objectives are optional, but these are the records an application builds when it models purpose. `ObjectiveTemporalBounds` and `ObjectiveRelation` are parts of `QualifiedObjectiveRecord` and share its contract. The validators and the lifecycle function are PUBLIC / ADVANCED; the responsiveness adapter is an optional host service, PUBLIC / ADVANCED, and its record takes that label. The two protocols that no runtime method calls are labeled in Chapter 10.

Table 7.2 — Labels assigned in Chapter 7

Symbol	Label	Rule
PVPPRuntime.validate_objective_set	PUBLIC / ADVANCED	R2
PVPPRuntime.evaluate_objective_responsiveness	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_objective_discovery_hints	PUBLIC / ADVANCED	R2
derive_objective_lifecycle_invalidations	PUBLIC / ADVANCED	R2
ObjectiveSet	PUBLIC / SUPPORTED	R6
QualifiedObjectiveRecord	PUBLIC / SUPPORTED	R6
ObjectiveTemporalBounds	PUBLIC / SUPPORTED	R6
ObjectiveRelation	PUBLIC / SUPPORTED	R6
ObjectiveDiscoveryHint	PUBLIC / SUPPORTED	R6

Symbol	Label	Rule
ObjectiveResponsivenessAdapter	PUBLIC / ADVANCED	R2 (optional host service)
ObjectiveResponsivenessRecord	PUBLIC / ADVANCED	R6
ObjectiveLifecycleChange	PUBLIC / ADVANCED	R6
ObjectiveSetValidationAssessment	IMPLEMENTATION / SUPPORT	R3
ObjectiveResponsivenessAssessment	IMPLEMENTATION / SUPPORT	R3
ObjectiveDiscoveryAssessment	IMPLEMENTATION / SUPPORT	R3
ObjectiveLifecycleInvalidationAssessment	IMPLEMENTATION / SUPPORT	R3

Table 7.3 — Compact rows for runtime-built records

Name and kind	Label	Role and use	Source
ObjectiveSetValidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>validate_objective_set</code>	models.py: 333
ObjectiveResponsivenessAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>evaluate_objective_responsiveness</code> ; carried in cycle results	models.py: 358
ObjectiveDiscoveryAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>validate_objective_discovery_hints</code>	models.py: 382
ObjectiveLifecycleInvalidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>derive_objective_lifecycle_invalidations</code>	models.py: 2503

Chapter 8 — Graph, Reachability, and Structural Admission

This chapter covers how structure enters and changes in v0.141: construction of the instance-transformation graph for a cycle, the reachability statuses a transformation can carry, refresh of reachability, controlled admission and revision of graph structure, admission of novel functions and recovery necessities, qualification of structural coverage, and the path from evidence about changing capability to graph change. It is the surface where structure, reachability, capability, and authorization are easiest to collapse into one another, and the chapter keeps them apart.

A reader must know how structure enters the runtime, what each reachability status means, and how new structure is admitted without being invented.

8.1 The Rules of the Structural Surface

- **Structure is supplied, never discovered.** Every instance, transformation, composition, action, and recovery necessity comes from the host with its provenance. No call in this chapter infers structure from behavior, topology, names, or objectives (C15/D14; C10/D9 §10).
- **Reachability is not feasibility or authority.** A reachable transformation may still be infeasible under Constraints and inadequate under Adequacy, and admitting it authorizes nothing (C15/D11; 3.5–3.10).
- **Future capability is not present reachability.** Capability that does not yet exist cannot be represented as reachable now. A capability-development policy may change reachability only after its realization and the resulting state transition, in a later cycle (C10/D9 §10; C20/D3).
- **Every change is exact and atomic.** Refresh, admission, and revision name the exact registry identity they expect (4.8). A stale identity rejects the whole request, and a rejected request changes nothing.
- **Changing structure is separate from re-entry.** The registry methods here change the registry and return an assessment. They do not invalidate earlier cycle results. When cycles have already run, use the wrappers of 12.6, which call these methods and also plan re-entry from the Graph/Seed stage.

8.2 Graph Construction

construct_graph runs the bounded Graph Construction Protocol for one cycle: scope, domains, instances, taxonomy, reachability, composition, coverage, seed reduction, validity, and handoff to Π , in that order (C15/D14). The canonical cycle calls it after $\mathcal{R}$ (3.5). It validates registered structure; it does not search future states, project policies, or evaluate viability. The regime is recorded with the result, but regime-based family activation happens at Π (Table 3.6), not here.

Record — PreliminaryPreservationObject

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: built by the host for the cycle request, 9.2)
Fields	frozen dataclass; id: str; function_statement: str; mechanism_independence_note: str = ""; mechanism_identity_exception: str = ""; rationale: str = "
Source	models.py:1163

Field	Content
Evidence	Direct use: test files 14, benchmarks 10, example files 2; not named in doc 05; e.g. <code>authority_bound_callthrough_v0130.py</code> , <code>minimal_battery_service.py</code> .

Record — `GraphConstructionConfig`

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: optional part of the cycle request)
Fields	frozen dataclass; <code>max_seeds: int = 64</code>
Source	<code>models.py:1219</code>
Evidence	Direct use: test files 3, benchmarks 3; not named in doc 05; e.g. <code>benchmark_canonical_cycle_v032.py</code> , <code>benchmark_graph_composition_sufficiency_v039.py</code> .

Entry — `PVPPRuntime.construct_graph`

Field	Content
Name and label	<code>PVPPRuntime.construct_graph</code> — method; <code>kernel.py:1017</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Builds and validates the graph substrate for one cycle and hands policy seeds to Π .
Framework basis	Instance-Transformation Graph Layer v0.4 and Graph Construction Protocol v0.4 (C15/D11; C15/D14).
Signature	<code>PVPPRuntime.construct_graph(preservation_object: PreliminaryPreservationObject, regime: str, governing_domain_ids, required_family_ids=(), required_path_class_ids=(), *, config: GraphConstructionConfig None = None) → GraphConstructionAssessment</code>
Preconditions	The preservation object has an id and a function statement; the governing domains are registered; at least one graph instance is registered; any required graph families are among the eight graph families; <code>max_seeds</code> is positive.
Result	<code>GraphConstructionAssessment</code> : status "PASS", "CONDITIONAL PASS", or "FAIL"; the transformations validated, excluded, and uncertain; valid and invalid compositions; represented and missing families; missing path classes; seeds handed to Π ; failure codes (Table 8.1); and <code>safe_to_handoff</code> .

Field	Content
Invariants	Reachable transformations and valid compositions become seeds; unreachable transformations are recorded but never admitted. A composition is valid only if it has at least two transformations, all registered and reachable, chained destination to source, with a non-empty recovery relevance and distinctness justification. Seeds that are structurally required, or that carry a required family or path class, are marked required for Π . Seed reduction merges only seeds with identical actions, Π family, affected domains, classes, and path type. If seeds exceed <code>max_seeds</code> , handoff is blocked rather than truncated by registration order.
Failure behavior	Scope and declaration failures return FAIL with code G-010. An unknown required family or a non-positive <code>max_seeds</code> raises <code>ValueError</code> .
Does not	Project, evaluate adequacy, select, or invent path depth. Graph validity does not guarantee Π completeness.
Evidence	Direct use: test files 6, benchmarks 4; not named in doc 05; e.g. <code>benchmark_graph_composition_sufficiency_v039.py</code> , <code>benchmark_graph_construction_v026.py</code> .
See also	3.5; Guide 5.

Table 8.1 — Graph failure codes in v0.141

Code	Condition	Effect
G-001	A registered transformation has a family outside the eight graph families	FAIL
G-002	A required graph family has no reachable transformation or valid composition	FAIL
G-003	An uncertain transformation is structurally required, or is the only source of a required family	FAIL
G-004	Uncertainty decides whether a family is present, or an uncertain transformation is flagged as the only viable path	FAIL
G-005	A required path class is not represented, or a registered composition is invalid	FAIL
G-007	Seeds exceed <code>max_seeds</code> (boundedness overflow)	FAIL; no handoff
G-010	Preservation object undeclared, unknown governing domains, or no graph instances	FAIL

With no failure code, the status is PASS if no transformation is uncertain and CONDITIONAL PASS otherwise. The canonical cycle continues on either and stops on FAIL (Table 3.7). Because an invalid composition fails the whole graph (G-005), removing a transformation that a composition uses makes the next construction fail until the composition is revised (8.5).

8.3 Reachability Statuses

Table 8.2 — Reachability statuses of a graph transformation

Status	Meaning	Treatment in <code>construct_graph</code>
"reachable"	Structurally realizable from the current state under the current configuration	Admitted to the substrate; may become a seed
"unreachable"	Represented, but not realizable now	Recorded as excluded; never admitted; kept so its later change can be refreshed and audited
"uncertain"	Reachability unresolved; an uncertainty note is required	Not admitted as a seed. Allowed only in a CONDITIONAL PASS, and only if it neither decides family presence nor is the only viable path (G-003, G-004)

Unknown is therefore neither silently present nor silently absent (C15/D14 §14.4). Reachability says nothing about feasibility, which Constraints decides, or about adequacy.

8.4 Reachability Refresh

Record — `GraphReachabilityObservation`

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; <code>transformation_id</code> : str; <code>reachability_status</code> : str; <code>observed_at</code> : float; <code>source_id</code> : str; <code>provenance_id</code> : str; <code>uncertainty_note</code> : str = ""; <code>evidence</code> : Mapping[str, Any] = (empty)
Source	models.py:2750
Evidence	Direct use: test files 3; not named in doc 05; e.g. <code>test_v2_graph_reachability_execution_v0103.py</code> , <code>test_v2_graph_reachability_governance_trigger_v0102.py</code> .

Record — `GraphReachabilityRefreshRequest`

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; <code>expected_graph_substrate_identity</code> : str; <code>observations</code> : tuple[GraphReachabilityObservation, ...]; <code>refresh_id</code> : str; <code>notes</code> : tuple[str, ...] = ()
Source	models.py:2765
Evidence	Direct use: test files 3; not named in doc 05; e.g. <code>test_v2_graph_reachability_execution_v0103.py</code> , <code>test_v2_graph_reachability_governance_trigger_v0102.py</code> .

Entry — [PVPPRegistry.refresh_graph_reachability](#)

Field	Content
Name and label	PVPPRegistry.refresh_graph_reachability — method; registry.py:182. PUBLIC / ADVANCED (rule R2).
Purpose	Updates the reachability status of transformations already registered, atomically.
Framework basis	Reachability refresh of represented structure (C15/D11; C10/D18).
Signature	PVPPRegistry.refresh_graph_reachability(request: GraphReachabilityRefreshRequest) → GraphReachabilityRefreshAssessment
Preconditions	The request names the exact current graph substrate identity (4.8); a stale identity rejects the whole request. The request carries explicit source, provenance, configuration version, and rationale. Each observation names a registered transformation once, with a valid status, a note when uncertain, and explicit source and provenance.
Result	GraphReachabilityRefreshAssessment: status "APPLIED" or "REJECTED", the prior and resulting substrate identities, and the changed and unchanged transformations.
Invariants	All observations apply or none do. Each changed transformation records the refresh in its metadata. An observation that repeats the current status and note counts as unchanged.
Failure behavior	A rejected request returns REJECTED with violations and changes nothing; nothing is raised.
Does not	Admit new structure: an unregistered transformation is rejected, not created. Use 8.5 or 8.6 for new structure.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_graph_reachability_refresh_v0101.py.
See also	12.6 (refresh_graph_reachability_and_plan_reentry).

8.5 Structural Admission and Revision

Record — [GraphTransformationAdmissionRequest](#)

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; admission_id: str; expected_graph_substrate_identity: str; transformation: GraphTransformationDefinition; configuration_version: str; source_id: str; provenance_id: str; rationale: str; evidence: Mapping[str, Any] = (empty)
Source	models.py:2802

Field	Content
Evidence	Direct use: test files 4; not named in doc 05; e.g. test_v2_graph_structural_admission_execution_v0106.py, test_v2_graph_structural_admission_governance_v0105.py.

Entry — `PVPPRegistry.admit_graph_transformation`

Field	Content
Name and label	<code>PVPPRegistry.admit_graph_transformation</code> — method; registry.py:304. PUBLIC / ADVANCED (rule R2).
Purpose	Admits one new graph transformation as a versioned configuration change.
Framework basis	Controlled structural admission (C15/D11; C15/D14).
Signature	<code>PVPPRegistry.admit_graph_transformation(request: GraphTransformationAdmissionRequest) → GraphTransformationAdmissionAssessment</code>
Preconditions	The request names the exact current graph substrate identity (4.8); a stale identity rejects the whole request. The request carries explicit source, provenance, configuration version, and rationale. The transformation is new and passes the same checks as registration (4.7).
Result	<code>GraphTransformationAdmissionAssessment</code> : status "ADMITTED" or "REJECTED", identities, the transformation id, and provenance.
Invariants	The admission stamp—id, version, source, provenance, rationale, evidence—is written into the transformation's metadata.
Failure behavior	A rejected request changes nothing; nothing is raised.
Does not	Discover structure, authorize execution of the action, or select anything.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_graph_structural_admission_v0104.py.
See also	12.6 (<code>admit_graph_transformation_and_plan_reentry</code>).

Record — `GraphTransformationRevisionRequest`

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; revision_id: str; expected_graph_substrate_identity: str; operation: str; transformation_id: str; configuration_version: str; source_id: str; provenance_id: str; rationale: str; replacement: <code>GraphTransformationDefinition None = None</code> ; evidence: <code>Mapping[str, Any] = (empty)</code>
Source	models.py:2827
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_graph_structural_revision_v0127.py.

Entry — [PVPPRegistry.revise_graph_transformation](#)

Field	Content
Name and label	PVPPRegistry.revise_graph_transformation — method; registry.py:338. PUBLIC / ADVANCED (rule R2).
Purpose	Replaces or removes one registered graph transformation.
Framework basis	Controlled structural revision (C15/D11).
Signature	PVPPRegistry.revise_graph_transformation(request: GraphTransformationRevisionRequest) → GraphTransformationRevisionAssessment
Preconditions	The request names the exact current graph substrate identity (4.8); a stale identity rejects the whole request. The request carries explicit source, provenance, configuration version, and rationale. operation is "replace" or "remove". A replacement keeps the transformation id and passes the registration checks; a removal supplies no replacement.
Result	GraphTransformationRevisionAssessment: status "REVISED" or "REJECTED", identities, and provenance.
Invariants	A replacement cannot substitute a different structural object under the same id without passing the checks.
Failure behavior	A rejected request changes nothing; nothing is raised.
Does not	Check registered compositions or clean up dependencies. Removing a transformation that a composition uses leaves the composition invalid, and the next construct_graph fails with G-005 until the composition is revised. Revision is not reachability refresh; use 8.4 for status changes.
Host obligation	Before removing a transformation, find every registered composition that references it and revise or remove that composition first. v0.141 performs no cascade and no rejection on this ground. Listed in Chapter 15 among host obligations the runtime does not check.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_graph_structural_revision_v0127.py.
See also	12.6 (revise_graph_transformation_and_plan_reentry); Chapter 15.

8.6 Novel-Function Admission

A novel function is a new action together with the first graph transformation that uses it. v0.141 admits the two atomically, so an action never enters without represented structure and a transformation never refers to an action that does not exist.

Record — [NovelFunctionAdmissionRequest](#)

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)

Field	Content
Fields	frozen dataclass; admission_id: str; expected_action_registry_identity: str; expected_graph_substrate_identity: str; action: ActionDefinition; transformation: GraphTransformationDefinition; configuration_version: str; source_id: str; provenance_id: str; rationale: str; evidence: Mapping[str, Any] = (empty)
Source	models.py:3274
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_novel_function_admission_v0121.py.

Entry — PVPPRegistry.admit_novel_function

Field	Content
Name and label	PVPPRegistry.admit_novel_function — method; registry.py:219. PUBLIC / ADVANCED (rule R2).
Purpose	Admits one new action and its first graph transformation together.
Framework basis	Controlled structural admission (C15/D11); actions as executable units (C15/D7).
Signature	PVPPRegistry.admit_novel_function(request: NovelFunctionAdmissionRequest) → NovelFunctionAdmissionAssessment
Preconditions	The request names the exact current action-registry and graph-substrate identities. The action is new and its domains are registered. The transformation is new, uses that action, and passes the registration checks. Provenance fields are explicit.
Result	NovelFunctionAdmissionAssessment: status "ADMITTED" or "REJECTED", prior and resulting identities for both registries, the action and transformation ids, and provenance.
Invariants	Both are admitted or neither is; both carry the same admission stamp.
Failure behavior	A rejected request changes nothing; nothing is raised.
Does not	Infer a function from observed behavior, bind the action to an implementation (11.5), or authorize its execution.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_novel_function_admission_v0121.py.
See also	12.6 (admit_novel_function_and_plan_reentry).

8.7 Recovery-Necessity Admission and Revision

Recovery necessities define G's dependency closure (4.5, 3.4). Admitting or revising one changes which domains govern in later cycles, so it is guarded by the governing-substrate identity.

Record — RecoveryNecessityAdmissionRequest

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; admission_id: str; expected_governing_substrate_identity: str; relation: RecoveryNecessityDefinition; configuration_version: str; source_id: str; provenance_id: str; rationale: str; evidence: Mapping[str, Any] = (empty)
Source	models.py:3314
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_recovery_necessity_admission_v0122.py.

Entry — PVPPRegistry.admit_recovery_necessity

Field	Content
Name and label	PVPPRegistry.admit_recovery_necessity — method; registry.py:253. PUBLIC / ADVANCED (rule R2).
Purpose	Adds one recovery-necessity relation as a governed configuration change.
Framework basis	G's recovery-necessity closure (C15/D2).
Signature	PVPPRegistry.admit_recovery_necessity(request: RecoveryNecessityAdmissionRequest) → RecoveryNecessityAdmissionAssessment
Preconditions	The request names the exact current governing substrate identity (4.8); a stale identity rejects the whole request. The request carries explicit source, provenance, configuration version, and rationale. The relation is new, connects two distinct registered domains, and states its basis.
Result	RecoveryNecessityAdmissionAssessment: status "ADMITTED" or "REJECTED", identities, the relation id, and provenance.
Invariants	Add-only: admission never removes or changes another relation.
Failure behavior	A rejected request changes nothing; nothing is raised.
Does not	Recompute G; the next cycle does.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_recovery_necessity_admission_v0122.py.
See also	12.6 (admit_recovery_necessity_and_plan_reentry).

Record — RecoveryNecessityRevisionRequest

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)

Field	Content
Fields	frozen dataclass; revision_id: str; expected_governing_substrate_identity: str; operation: str; relation_id: str; replacement: RecoveryNecessityDefinition None; configuration_version: str; source_id: str; provenance_id: str; rationale: str; evidence: Mapping[str, Any] = (empty)
Source	models.py:3350
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_recovery_necessity_revision_v0123.py.

Entry — PVPPRegistry.revise_recovery_necessity

Field	Content
Name and label	PVPPRegistry.revise_recovery_necessity — method; registry.py:272. PUBLIC / ADVANCED (rule R2).
Purpose	Replaces or removes one registered recovery-necessity relation.
Framework basis	As above.
Signature	PVPPRegistry.revise_recovery_necessity(request: RecoveryNecessityRevisionRequest) → RecoveryNecessityRevisionAssessment
Preconditions	The request names the exact current governing substrate identity (4.8); a stale identity rejects the whole request. The request carries explicit source, provenance, configuration version, and rationale. operation is "replace" or "remove"; a replacement keeps the relation id and passes the admission checks.
Result	RecoveryNecessityRevisionAssessment: status "REVISED" or "REJECTED", identities, the relation id, and provenance.
Invariants	A replacement cannot substitute a different relation under the same id.
Failure behavior	A rejected request changes nothing; nothing is raised.
Does not	Recompute G.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_recovery_necessity_revision_v0123.py.
See also	12.6 (revise_recovery_necessity_and_plan_reentry).

8.8 Structural Coverage

Coverage asks a different question from reachability: whether the represented graph is complete enough for the decision at all. v0.141 keeps coverage uncertainty separate from ordinary reachability or projection uncertainty. The host reports coverage as sufficient, incomplete (with named missing families, path classes, or compositions), or uncertain (with named uncertainty markers). The runtime qualifies the report and then decides what may happen next.

Record — GraphStructuralCoverageObservation

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; observation_id: str; expected_graph_substrate_identity: str; coverage_status: str; source_id: str; provenance_id: str; observed_at: float; missing_family_ids: tuple[str, ...] = (); missing_path_class_ids: tuple[str, ...] = (); missing_composition_ids: tuple[str, ...] = (); uncertainty_markers: tuple[str, ...] = (); evidence: Mapping[str, Any] = (empty); notes: tuple[str, ...] = ()
Source	models.py:2881
Evidence	Direct use: test files 3; not named in doc 05; e.g. test_v2_graph_structural_coverage_disposition_v0109.py, test_v2_graph_structural_coverage_reentry_v0108.py.

Entry — PVPPRegistry.assess_graph_structural_coverage

Field	Content
Name and label	PVPPRegistry.assess_graph_structural_coverage — method; registry.py:452. PUBLIC / ADVANCED (rule R2).
Purpose	Qualifies a host coverage report against the exact current graph substrate.
Framework basis	Graph composition sufficiency and coverage (C15/D14); Π completeness depends on it (C15/D12).
Signature	PVPPRegistry.assess_graph_structural_coverage(observation: GraphStructuralCoverageObservation) → GraphStructuralCoverageAssessment
Preconditions	The request names the exact current graph substrate identity (4.8); a stale identity rejects the whole request. The request carries explicit source, provenance, configuration version, and rationale. coverage_status is "sufficient", "incomplete", or "uncertain".
Result	GraphStructuralCoverageAssessment: validity, status flags, the named gaps and markers, and provenance.
Invariants	The status must match the evidence: sufficient declares no gaps or markers; incomplete names at least one gap; uncertain names at least one marker and no known gap—known missing structure must be reported as incomplete.
Failure behavior	Invalid reports return valid False with violations; nothing is raised.
Does not	Mutate or admit structure, or plan re-entry.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_graph_structural_coverage_v0107.py.
See also	12.6 (assess_graph_structural_coverage_and_plan_reentry).

Entry — disposition_graph_structural_coverage

Field	Content
Name and label	disposition_graph_structural_coverage — function; reentry.py:1523. PUBLIC / ADVANCED (rule R2).
Purpose	Decides what a coverage trigger permits: nothing more, a handoff to admission, or a block.
Framework basis	As above.
Signature	disposition_graph_structural_coverage(trigger)
Preconditions	A coverage governance trigger from 12.6.
Result	GraphStructuralCoverageDisposition with disposition "coverage_sufficient", "admission_definition_required" (with a GraphStructuralCoverageAdmissionHandoff listing the gaps), "unsupported_uncertain", or "invalid", and whether graph reconstruction is blocked.
Invariants	Known incompleteness blocks reconstruction and hands the named gaps toward a separate definition and admission workflow. Unresolved uncertainty blocks reconstruction without a handoff, because the missing structure is not identified. Sufficient coverage blocks nothing.
Failure behavior	Returns the disposition; nothing is raised.
Does not	Reconstruct a graph known to be incomplete, or treat the handoff as an admission request. The handoff grants no structural or execution authority.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_graph_structural_coverage_disposition_v0109.py.
See also	12.6.

8.9 Capability Change and Trajectories

Capability changes over time—skills are gained, equipment lost, configurations rearranged. v0.141 represents such change as validated, non-scalar evidence about capability snapshots, and assembles ordered observations into a trajectory. Neither step computes PP, changes the graph, or invalidates anything. Graph change from capability evidence happens only through the explicit dependencies of 8.10.

Record — CapabilityChangeObservation

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; observation_id: str; entity_id: str; capability_id: str; observed_at: float; change_kind: str; prior_snapshot_id: str None; current_snapshot_id: str None; source_id: str; provenance_id: str; confidence: float None = None; evidence_ids: tuple[str, ...] = (); configuration_ids: tuple[str, ...] = (); notes: tuple[str, ...] = ()

Field	Content
Source	models.py:3118
Evidence	Direct use: test files 5; not named in doc 05; e.g. test_v2_capability_change_v0116.py, test_v2_capability_driven_graph_refresh_v0118.py.

Entry — `validate_capability_change_observation`

Field	Content
Name and label	<code>validate_capability_change_observation</code> — function; reentry.py:1851. PUBLIC / ADVANCED (rule R2).
Purpose	Qualifies one observation of capability change.
Framework basis	Execution-contingent capability revision and capability change (C10/D12; C10/D20).
Signature	<code>validate_capability_change_observation(observation)</code>
Preconditions	None.
Result	CapabilityChangeValidationAssessment: valid, identities, change kind, and <code>material_change_reported</code> .
Invariants	<code>change_kind</code> is gained, lost, increased, decreased, reconfigured, or uncertain. A definite change needs a current snapshot; increased, decreased, and reconfigured also need a prior snapshot, different from the current one. Confidence, if given, lies in [0, 1]; ids are non-empty and unique; source and provenance are explicit.
Failure behavior	Returns violations; raises nothing.
Does not	Infer PP magnitude, graph or Π change, invalidation, or execution authority.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_capability_change_v0116.py.
See also	Guide 5.

Entry — `project_capability_trajectory`

Field	Content
Name and label	<code>project_capability_trajectory</code> — function; reentry.py:1915. PUBLIC / ADVANCED (rule R2).
Purpose	Builds an ordered, non-scalar trajectory from validated observations of one capability of one entity.
Framework basis	As above.
Signature	<code>project_capability_trajectory(trajectory_id, observations, *, as_of=None)</code>

Field	Content
Preconditions	At least one observation; all valid, for the same entity and capability, with unique ids, strictly increasing observation times, and none after as_of. Consecutive definite observations must chain: each prior snapshot equals the previous current snapshot.
Result	CapabilityTrajectoryProjectionAssessment carrying a CapabilityTrajectoryProjection: the snapshot chain, ordered change kinds, sources, provenance, configurations, and uncertain observations.
Invariants	Uncertain observations neither break nor repair the snapshot chain; the current definite snapshot is the last in the chain.
Failure behavior	Returns violations and no trajectory on failure; raises nothing.
Does not	Aggregate capability into a scalar, or change the graph.
Evidence	Direct use: test files 4; not named in doc 05; e.g. test_v2_capability_driven_graph_refresh_v0118.py, test_v2_capability_graph_constructibility_v0128.py.
See also	8.10–8.11.

8.10 Capability-Driven Graph Change

Evidence that capability has changed may justify a graph change, but only through a dependency the host declares explicitly: this capability snapshot makes this transformation reachable, or supports admitting this transformation. Both functions below bind to the trajectory's current definite snapshot and then call the re-entry wrappers of 12.6, so a successful change also plans re-entry from Graph/Seed.

Record — CapabilityGraphReachabilityDependency

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; dependency_id: str; entity_id: str; capability_id: str; transformation_id: str; capability_snapshot_id: str; reachability_status: str; source_id: str; provenance_id: str; uncertainty_note: str = ''; evidence: Mapping[str, Any] = (empty); notes: tuple[str, ...] = ()
Source	models.py:3185
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_capability_driven_graph_refresh_v0118.py.

Entry — refresh_graph_from_capability_trajectory

Field	Content
Name and label	refresh_graph_from_capability_trajectory — function; reentry.py:1981. PUBLIC / ADVANCED (rule R2).
Purpose	Refreshes the reachability of registered transformations from declared capability dependencies.

Field	Content
Framework basis	Reachability change through capability change (C10/D12; C10/D18).
Signature	<code>refresh_graph_from_capability_trajectory(registry, trajectory_assessment, dependencies, *, refresh_id)</code>
Preconditions	A valid trajectory with a current definite snapshot; at least one dependency, each naming a transformation, a snapshot, a valid status, and explicit provenance; at least one matching the current snapshot.
Result	CapabilityDrivenGraphRefreshAssessment: validity, matched dependencies, the refresh request it built, and the re-entry trigger.
Invariants	Only dependencies bound to the current snapshot are applied, through <code>refresh_graph_reachability_and_plan_reentry</code> .
Failure behavior	Returns violations and changes nothing on failure; raises nothing.
Does not	Admit new structure or invent a transformation.
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_v2_capability_driven_graph_refresh_v0118.py</code> .
See also	8.4; 12.6.

Record — `CapabilityGraphConstructibilityDependency`

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; <code>dependency_id: str</code> ; <code>entity_id: str</code> ; <code>capability_id: str</code> ; <code>capability_snapshot_id: str</code> ; <code>transformation: GraphTransformationDefinition</code> ; <code>configuration_version: str</code> ; <code>source_id: str</code> ; <code>provenance_id: str</code> ; <code>rationale: str</code> ; <code>evidence: Mapping[str, Any] = (empty)</code> ; <code>notes: tuple[str, ...] = ()</code>
Source	<code>models.py:3471</code>
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_v2_capability_graph_constructibility_v0128.py</code> .

Entry — `construct_graph_from_capability_trajectory`

Field	Content
Name and label	<code>construct_graph_from_capability_trajectory</code> — function; <code>reentry.py:2320</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Admits one host-specified new transformation supported by capability evidence.
Framework basis	As above; controlled structural admission (C15/D11).
Signature	<code>construct_graph_from_capability_trajectory(registry, trajectory_assessment, dependency, *, admission_id)</code>

Field	Content
Preconditions	A valid trajectory with a current definite snapshot; a dependency bound to that snapshot, carrying the complete transformation and explicit provenance; an admission id.
Result	CapabilityDrivenGraphConstructibilityAssessment: validity, the admission request it built, and the re-entry trigger.
Invariants	Admission goes through admit_graph_transformation_and_plan_reentry. The admitted transformation is unselected and unlicensed until ordinary downstream governance runs.
Failure behavior	Returns violations and changes nothing on failure; raises nothing.
Does not	Invent a path or an action; the host supplies the complete transformation.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_capability_graph_constructibility_v0128.py.
See also	8.5; 12.6.

8.11 Capability-Formation Prediction Error

Record — CapabilityFormationExpectation

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; expectation_id: str; entity_id: str; capability_id: str; expected_at: float; expected_snapshot_id: str; source_id: str; provenance_id: str; configuration_ids: tuple[str, ...] = (); notes: tuple[str, ...] = ()
Source	models.py:3218
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_capability_prediction_error_v0119.py.

Record — CapabilityPredictionArtifactDependency

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; dependency_id: str; expectation_id: str; artifact_id: str; stage: str; provenance_ids: tuple[str, ...] = ()
Source	models.py:3249
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_capability_prediction_error_v0119.py.

Entry — `assess_capability_formation_prediction_error`

Field	Content
Name and label	<code>assess_capability_formation_prediction_error</code> — function; <code>reentry.py:2034</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Compares an expected capability snapshot with the one later evidence shows, and invalidates only artifacts declared to depend on the expectation.
Framework basis	Prediction error (6.5) applied to capability formation (C10/D12).
Signature	<code>assess_capability_formation_prediction_error(expectation, trajectory_assessment, artifact_dependencies, *, error_id)</code>
Preconditions	A valid trajectory that reaches the expectation's time, for the same entity and capability; dependencies naming artifacts and canonical stages from <code>CANONICAL_REENTRY_STAGE_ORDER</code> ; an error id.
Result	<code>CapabilityFormationPredictionErrorAssessment</code> : the error, if any; matched dependencies; invalidation signals; the re-entry assessment and plan.
Invariants	A match of snapshots means no error and no invalidation. A mismatch is recorded as a non-scalar snapshot mismatch and invalidates, at their declared stages, only the artifacts registered against this expectation.
Failure behavior	Returns violations on invalid input; raises nothing.
Does not	Infer PP magnitude, missing capability, graph structure, or undeclared dependencies.
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_v2_capability_prediction_error_v0119.py</code> .
See also	6.5; 12.2–12.3.

8.12 Classification Record and Compact Rows

The preservation object and graph configuration are PUBLIC / SUPPORTED as documented parts of the ordinary cycle request (9.2). `construct_graph` is PUBLIC / ADVANCED: the cycle runs it, and direct calls are specialized. All admission, revision, refresh, coverage, and capability surfaces are PUBLIC / ADVANCED, as are the requests and dependencies the host builds for them. The re-entry wrappers are documented in 12.6, and `GraphReachabilityAdapter` in Chapter 10.

Table 8.3 — Labels assigned in Chapter 8

Symbol	Label	Rule
<code>PVPPRuntime.construct_graph</code>	PUBLIC / ADVANCED	R2
<code>PVPPRegistry.refresh_graph_reachability</code>	PUBLIC / ADVANCED	R2
<code>PVPPRegistry.admit_graph_transformation</code>	PUBLIC / ADVANCED	R2
<code>PVPPRegistry.revise_graph_transformation</code>	PUBLIC / ADVANCED	R2

Symbol	Label	Rule
PVPPRegistry.assess_graph_structural_coverage	PUBLIC / ADVANCED	R2
PVPPRegistry.admit_novel_function	PUBLIC / ADVANCED	R2
PVPPRegistry.admit_recovery_necessity	PUBLIC / ADVANCED	R2
PVPPRegistry.revise_recovery_necessity	PUBLIC / ADVANCED	R2
disposition_graph_structural_coverage	PUBLIC / ADVANCED	R2
validate_capability_change_observation	PUBLIC / ADVANCED	R2
project_capability_trajectory	PUBLIC / ADVANCED	R2
construct_graph_from_capability_trajectory	PUBLIC / ADVANCED	R2
refresh_graph_from_capability_trajectory	PUBLIC / ADVANCED	R2
assess_capability_formation_prediction_error	PUBLIC / ADVANCED	R2
PreliminaryPreservationObject	PUBLIC / SUPPORTED	R6
GraphConstructionConfig	PUBLIC / SUPPORTED	R6
GraphReachabilityRefreshRequest	PUBLIC / ADVANCED	R6
GraphReachabilityObservation	PUBLIC / ADVANCED	R6
GraphTransformationAdmissionRequest	PUBLIC / ADVANCED	R6
GraphTransformationRevisionRequest	PUBLIC / ADVANCED	R6
NovelFunctionAdmissionRequest	PUBLIC / ADVANCED	R6
RecoveryNecessityAdmissionRequest	PUBLIC / ADVANCED	R6
RecoveryNecessityRevisionRequest	PUBLIC / ADVANCED	R6
GraphStructuralCoverageObservation	PUBLIC / ADVANCED	R6
CapabilityChangeObservation	PUBLIC / ADVANCED	R6
CapabilityGraphReachabilityDependency	PUBLIC / ADVANCED	R6
CapabilityGraphConstructibilityDependency	PUBLIC / ADVANCED	R6
CapabilityFormationExpectation	PUBLIC / ADVANCED	R6
CapabilityPredictionArtifactDependency	PUBLIC / ADVANCED	R6
GraphConstructionAssessment	IMPLEMENTATION / SUPPORT	R3
GraphReachabilityRefreshAssessment	IMPLEMENTATION / SUPPORT	R3
GraphTransformationAdmissionAssessment	IMPLEMENTATION / SUPPORT	R3
GraphTransformationRevisionAssessment	IMPLEMENTATION / SUPPORT	R3

Symbol	Label	Rule
NovelFunctionAdmissionAssessment	IMPLEMENTATION / SUPPORT	R3
RecoveryNecessityAdmissionAssessment	IMPLEMENTATION / SUPPORT	R3
RecoveryNecessityRevisionAssessment	IMPLEMENTATION / SUPPORT	R3
GraphStructuralCoverageAssessment	IMPLEMENTATION / SUPPORT	R3
GraphStructuralCoverageDisposition	IMPLEMENTATION / SUPPORT	R3
GraphStructuralCoverageAdmissionHandoff	IMPLEMENTATION / SUPPORT	R3
CapabilityChangeValidationAssessment	IMPLEMENTATION / SUPPORT	R3
CapabilityTrajectoryProjection	IMPLEMENTATION / SUPPORT	R3
CapabilityTrajectoryProjectionAssessment	IMPLEMENTATION / SUPPORT	R3
CapabilityDrivenGraphRefreshAssessment	IMPLEMENTATION / SUPPORT	R3
CapabilityDrivenGraphConstructibilityAssessment	IMPLEMENTATION / SUPPORT	R3
CapabilityFormationPredictionError	IMPLEMENTATION / SUPPORT	R3
CapabilityFormationPredictionErrorAssessment	IMPLEMENTATION / SUPPORT	R3

Table 8.4 — Compact rows for runtime-built records

Name and kind	Label	Role and use	Source
GraphConstructionAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>construct_graph</code> ; carried in cycle results	models.py :1224
GraphReachabilityRefreshAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>refresh_graph_reachability</code>	models.py :2773
GraphTransformationAdmissionAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>admit_graph_transformation</code>	models.py :2813

Name and kind	Label	Role and use	Source
GraphTransformationRevisionAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of revise_graph_transformation	models.py :2840
NovelFunctionAdmissionAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of admit_novel_function	models.py :3287
RecoveryNecessityAdmissionAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of admit_recovery_necessity	models.py :3325
RecoveryNecessityRevisionAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of revise_recovery_necessity	models.py :3363
GraphStructuralCoverageAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of assess_graph_structural_coverage	models.py :2901
GraphStructuralCoverageDisposition — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of disposition_graph_structural_coverage	models.py :2959
GraphStructuralCoverageAdmissionHandoff — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Named gaps handed toward admission; grants nothing	models.py :2942
CapabilityChangeValidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of validate_capability_change_observation	models.py :3142
CapabilityTrajectoryProjection — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Trajectory built by project_capability_trajectory	models.py :3155
CapabilityTrajectoryProjectionAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of project_capability_trajectory	models.py :3176
CapabilityDrivenGraphRefreshAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of refresh_graph_from_capability_trajectory	models.py :3206
CapabilityDrivenGraphConstructibilityAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of construct_graph_from_capability_trajectory	models.py :3492
CapabilityFormationPredictionError — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Recorded snapshot mismatch	models.py :3232

Name and kind	Label	Role and use	Source
CapabilityFormationPredictionErrorAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>assess_capability_formation_prediction_error</code>	<code>models.py</code> :3259

Chapter 9 — Decision-Cycle Operators and Results

This chapter documents the calls and records that implement the decision cycle. Chapter 3 states what each stage is for and what it may not decide; this chapter states how to run the cycle, what it returns, what it asks of the host, and how each stage can be called on its own. It also documents the integrity audits, and it separates the canonical cycle from older evaluation paths that remain in v0.141 but do not implement the canonical stack.

A reader must be able to run a cycle, read every field of its result, and trace any candidate to the stage that set it aside.

9.1 How to Use This Chapter

Applications run the cycle through three PUBLIC / SUPPORTED entry points: `prepare_canonical_cycle_snapshot`, `evaluate_canonical_decision_cycle`, and `evaluate_integrated_canonical_cycle` (9.3). Everything else in this chapter is PUBLIC / ADVANCED. The stage calls of 9.7–9.11 are the operations the cycle performs internally, exposed for stage-level testing and diagnostics. Calling them in sequence by hand does not reproduce the cycle: the cycle adds the joint-recovery binding and gate, the objective checks, projection licensing, the record-immutability checks, and the integrity audit, none of which the individual calls perform.

9.2 The Cycle Request

`CanonicalDecisionCycleRequest` carries every upstream input the host supplies for one cycle, other than state and objectives. Its fields map to the stages of Chapter 3.

Record — `CanonicalDecisionCycleRequest`

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: built by the host as the argument of the cycle entries)
Fields	frozen dataclass; <code>preservation_object</code> : <code>PreliminaryPreservationObject</code> ; <code>domain_frame</code> : <code>DomainFrame</code> ; <code>required_graph_family_ids</code> : <code>tuple[str, ...] = ()</code> ; <code>required_graph_path_class_ids</code> : <code>tuple[str, ...] = ()</code> ; <code>materially_required_policy_class_ids</code> : <code>tuple[str, ...] = ()</code> ; <code>contextual_interruption</code> : <code>bool = False</code> ; <code>previous_regime</code> : <code>PreviousRegimeState None = None</code> ; <code>graph_config</code> : <code>GraphConstructionConfig None = None</code> ; <code>pi_config</code> : <code>PiConstructionConfig None = None</code> ; <code>projection_horizon</code> : <code>float None = None</code> ; <code>projection_input_trace</code> : <code>Mapping[str, Any] = (empty)</code> ; <code>joint_recovery_capacity_calendar</code> : <code>CapacityCalendar None = None</code> ; <code>joint_recovery_diagnostic_mode</code> : <code>str = 'witness'</code> ; <code>projection_required_causal_input_ids</code> : <code>tuple[str, ...] = ()</code> ; <code>projection_model_authority</code> : <code>ProjectionModelAuthority None = None</code>
Source	<code>models.py</code> :1865

Field	Content
Evidence	Direct use: test files 9, benchmarks 6, example files 2; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.

Table 9.1 — Request fields and the stages that use them

Field	Stage	Meaning
preservation_object	Graph (8.2)	The preliminary preservation object
domain_frame	Domain Framing, Adequacy	The frame, one locked target per governing domain (3.8)
required_graph_family_ids, required_graph_path_class_ids	Graph	Families and path classes the graph must represent (Table 8.1)
materially_required_policy_class_ids	Π , Π completeness	Classes Π must represent (3.6)
contextual_interruption, previous_regime	$\mathcal{R}$	Forces Existential; supplies hysteresis input (3.4)
graph_config, pi_config	Graph, Π	max_seeds and max_candidates bounds
projection_horizon, projection_input_trace, projection_required_causal_input_ids, projection_model_authority	Projection (9.10)	The licensed horizon L_P and the projection licensing inputs
joint_recovery_capacity_calendar, joint_recovery_diagnostic_mode	Joint recovery (3.9, 5.8)	Shared capacity for active obligations; "witness" or "all_minimal"

Record — DomainFrame

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: part of the request)
Fields	frozen dataclass; targets: tuple[DomainFrameTarget, ...]; notes: tuple[str, ...] = ()
Source	models.py:1497
Evidence	Direct use: test files 13, benchmarks 9, example files 2; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.

Record — DomainFrameTarget

Field	Content
Label	PUBLIC / SUPPORTED (rule R6)

Field	Content
Fields	frozen dataclass; domain_id: str; function_id: str; basis: str = 'function'; locked: bool = True; notes: tuple[str, ...] = ()
Source	models.py:1481
Evidence	Direct use: test files 13, benchmarks 9, example files 2; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.

Record — PiConstructionConfig

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: part of the request)
Fields	frozen dataclass; max_candidates: int = 64
Source	models.py:1268
Evidence	Direct use: test files 2, benchmarks 3; not named in doc 05; e.g. benchmark_canonical_cycle_v032.py, benchmark_mission_required_pi_v049.py.

Record — PreviousRegimeState

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: part of the request; explicit input so hysteresis creates no hidden runtime state)
Fields	frozen dataclass; regime: str; governing_min_horizon: float
Source	models.py:1400
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_multicycle_sequence_v042.py, test_regime_operator_v021.py.

9.3 Cycle Entry Points**Entry — PVPPRuntime.evaluate_canonical_decision_cycle**

Field	Content
Name and label	PVPPRuntime.evaluate_canonical_decision_cycle — method; kernel.py:3611. PUBLIC / SUPPORTED (rule R1).
Purpose	Runs one host-invoked canonical decision cycle through Σ .
Framework basis	The canonical stack (C00/D2 §2; C20/D3); stage contracts of Chapter 3.
Signature	PVPPRuntime.evaluate_canonical_decision_cycle(state, request: CanonicalDecisionCycleRequest, *, perceived_decision_state: PerceivedDecisionState None = None, objectives: ObjectiveSet None = None, objective_responsiveness_adapter=None, objective_discovery_hints=()) → CanonicalDecisionCycleAssessment

Field	Content
Preconditions	The world adapter provides the strict Φ/H hooks and <code>constraint_profile</code> ; the registry has a governing configuration or recovery necessities, a regime configuration, constraint rules, and the "steady" action; a <code>PolicyProjectionService</code> is attached or the world provides the compatibility hook <code>project_policy_record</code> (10.3). If a perceived decision state is passed, it wraps the supplied <code>WorldState</code> at the same time. If objectives are passed, the caller has already validated them with <code>validate_objective_set</code> : this entry validates discovery hints against the set but not the set itself (7.2). The snapshot entries do this validation automatically.
Result	<code>CanonicalDecisionCycleAssessment</code> (9.4). The cycle returns at the first failed gate with that stage's status (Table 9.2), or at Σ with a selection.
Invariants	Stages run in canonical order (3.1). One perceived snapshot and one baseline continuation serve Φ through $\mathcal{R}$. One authoritative projection record is obtained per constraint-feasible candidate, and Adequacy and Σ are checked not to alter any record. An artifact-integrity audit (9.12) runs before the result is returned. ε is never invoked and host state is never changed.
Failure behavior	Stops return a result with a status. Precondition failures, invalid objective hints or responsiveness, projection licensing failures, record mutation, and an integrity failure raise <code>ValueError</code> .
Does not	Execute, apply a transition, or persist anything.
Evidence	Direct use: test files 22, benchmarks 4, example files 1; named in doc 05; e.g. <code>authority_bound_callthrough_v0130.py</code> .
See also	3.1–3.14; Guide 6.

Table 9.2 — Where the canonical cycle stops

stopped_at	Condition	Status
"Graph/Seed"	Graph status is FAIL	The graph status, "FAIL"
"Pi"	No policy space	The Π status, for example "pi_blocked_by_graph_invalid"
"Pi Completeness"	A required class is missing	"pi_completeness_failed_missing_material_policy_classes"
"Sigma" (terminal)	No constraint-feasible candidate	"sigma_terminal_policy_selected"
"Domain Framing"	The frame is invalid	"domain_framing_failed"

stopped_at	Condition	Status
"Joint Recovery Feasibility"	Capacity authority missing, or the active set infeasible	"joint_recovery_capacity_authority_required", "joint_recovery_feasibility_failed"
"Sigma"	Standard or fallback selection	The Σ status, for example "sigma_standard_policy_selected"

Entry — `PVPPRuntime.prepare_canonical_cycle_snapshot`

Field	Content
Name and label	<code>PVPPRuntime.prepare_canonical_cycle_snapshot</code> — method; kernel.py:3195. PUBLIC / SUPPORTED (rule R1).
Purpose	Asks the host to expose the perceived decision state for one actual state, and validates it with the objectives.
Framework basis	The perception boundary (C10/D19); objectives (C10/D9).
Signature	<code>PVPPRuntime.prepare_canonical_cycle_snapshot(actual_state: ActualPersistentStateEnvelope, objectives: ObjectiveSet None = None) → CanonicalCycleSnapshot</code>
Preconditions	The world adapter implements <code>perceived_decision_state_from_actual</code> , or the older <code>represented_state_from_actual</code> .
Result	<code>CanonicalCycleSnapshot</code> : the actual state, the represented <code>WorldState</code> , the perceived decision state, and the objective set.
Invariants	Validates the actual state (5.1), the perceived decision state (5.3), and the objective set (7.2).
Failure behavior	Raises <code>ValueError</code> for a missing hook or any failed validation.
Does not	Open PP, SPV, AVS, or context to build a <code>WorldState</code> .
Evidence	Direct use: test files 4, benchmarks 1, example files 1; named in doc 05; e.g. <code>test_minimal_battery_service.py</code> .
See also	5.2–5.3; 7.2.

Entry — `PVPPRuntime.evaluate_integrated_canonical_cycle`

Field	Content
Name and label	<code>PVPPRuntime.evaluate_integrated_canonical_cycle</code> — method; kernel.py:3231. PUBLIC / SUPPORTED (rule R1).
Purpose	Runs one explicit pass from actual state through the cycle and, if requested, through ε and the Layer-1 transition.
Framework basis	The canonical stack with the execution boundary (C15/D9; C10/D2).

Field	Content
Signature	PVPPRuntime.evaluate_integrated_canonical_cycle(actual_state: ActualPersistentStateEnvelope, request: CanonicalDecisionCycleRequest, *, objectives: ObjectiveSet None = None, domain_frame: DomainFrame None = None, objective_responsiveness_adapter=None, objective_discovery_hints=(), execute: bool = False, episode_id: str = canonical-cycle, max_execution_steps: int = 1, execution_observations=(), prior_transition_provenance: ExecutionTransitionProvenance None = None) → CanonicalIntegratedCycleResult
Preconditions	As for the two calls above. With execute=True: the host has established that entry is sufficient on current facts—setting execute=True is that assertion—a Layer-1 transition service is attached, and execution observations are supplied. Any prior transition provenance must validate against the actual state.
Result	CanonicalIntegratedCycleResult with a status from Table 9.3, the decision, and—when executed—the license, ϵ result, handoff, transition, validation, final state, and transition provenance.
Invariants	Runs the snapshot, fills a missing capacity calendar from the capacity service (5.7), runs the decision cycle, and attaches the objective assessments. With execute=True it builds the license from the cycle, instantiates ϵ , advances it through the supplied observations, and applies the transition only after a terminal ϵ result. No retry, scheduling, persistence, or autonomous loop.
Failure behavior	Raises ValueError for invalid prior provenance and for the errors of the calls it composes.
Does not	Decide entry sufficiency. With execute=True the method passes entry_sufficient=True to instantiate_execution, so execute=True is the host's assertion that entry is sufficient on current facts (Guide 7.4). A host that has not checked this must not set execute=True; it should run ϵ through Chapter 11 instead. The integrated path uses observation-driven ϵ , not native invocation (11.1, 11.4).
Evidence	Direct use: test files 12, benchmarks 6, example files 2; named in doc 05; e.g. minimal_battery_service.py, test_minimal_battery_service.py.
See also	11; Guide 7–8.

execute=True is an authority assertion, not a switch. The integrated entry passes entry_sufficient=True to ϵ on the host's behalf; v0.141 does not establish entry sufficiency itself.

Table 9.3 — Integrated-cycle statuses

Status	Meaning
"integrated_cycle_stopped_before_unique_sigma_selection"	The cycle stopped before a selection
"integrated_cycle_selected_not_executed"	Selected; execute was False

Status	Meaning
"integrated_cycle_execution_blocked"	The license blocks entry (terminal infeasibility)
"integrated_cycle_execution_active"	ε instantiated but not terminal, or no observation supplied
"integrated_cycle_transition_applied"	Terminal ε and a valid Layer-1 transition
"integrated_cycle_transition_invalid"	Declared but unreachable in v0.141: an invalid transition result raises ValueError in apply_layer1_transition before this status can be set (14.3)

9.4 Cycle Results

CanonicalDecisionCycleAssessment carries every stage's result, in stage order, so any candidate can be traced to the stage that set it aside: Π lists every candidate; Π completeness says whether the space is complete; Constraints lists feasible and infeasible candidates with their violation profiles; Adequacy lists adequate and inadequate candidates with per-domain reasons; the selection names the survivors of each Σ stage. Fields for stages after the stop point are None.

Table 9.4 — CanonicalDecisionCycleAssessment fields

Fields	Content
status, stopped_at, pipeline_trace, notes	Outcome, stop stage, the stage labels traversed (2.7)
pressure_field, horizon_assessment, governing_assessment, regime_assessment	Φ , H , G , $\mathcal{R}$ results
graph_assessment, pi_construction, pi_before_joint_recovery_binding, pi_completeness	Graph, Π (after and before binding mandatory recovery actions), completeness
constraints, domain_framing, joint_recovery_feasibility, joint_recovery_execution_binding	Constraints, framing, and the joint-recovery check (3.9)
projection_records, projection_consistency_audit, projection_input_license_assessment, projection_model_authority_assessment	The authoritative $Q_t(\pi)$ set and its licensing (9.10)
adequacy, selection	Adequacy; the PolicySelectionAssessment with the Σ result
objective_responsiveness, objective_discovery	Informational objective records (7.4–7.5)
cycle_artifact_integrity	The integrity audit (9.12)

The selection is a `PolicySelectionAssessment`. Its docstring still calls it a “prototype policy-level Σ surface”, from the build that introduced it; in the canonical cycle it carries the canonical standard, fallback, or terminal Σ result in its `sigma`, `fallback_sigma`, or `terminal_sigma` field. Its `selected_policy_id` is the selection.

9.5 Sequences and Memory-Conditioned Cycles

Entry — `PVPPRuntime.evaluate_canonical_cycle_sequence`

Field	Content
Name and label	<code>PVPPRuntime.evaluate_canonical_cycle_sequence</code> — method; <code>kernel.py:3345</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Runs a finite, host-authored sequence of integrated cycles for integration stress testing.
Framework basis	As 9.3.
Signature	<code>PVPPRuntime.evaluate_canonical_cycle_sequence(initial_actual_state: ActualPersistentStateEnvelope, directives) → CanonicalCycleSequenceAssessment</code>
Preconditions	A valid initial actual state and a sequence of <code>CanonicalCycleDirective</code> records.
Result	<code>CanonicalCycleSequenceAssessment</code> : status “sequence_completed” or “sequence_stopped_at_active_epsilon”, the ledger of <code>CanonicalCycleLedgerEntry</code> rows, the final state, and cross-cycle invariant violations.
Invariants	Validated Layer-1 state is threaded from one cycle to the next; each directive may state the state id it expects.
Failure behavior	Raises <code>ValueError</code> for an invalid initial state or directive type.
Does not	Loop autonomously, schedule, or persist.
Evidence	Direct use: test files 2, benchmarks 1; not named in doc 05; e.g. <code>benchmark_multicycle_sequence_v042.py</code> .
See also	3.13.

Record — `CanonicalCycleDirective`

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; request: <code>CanonicalDecisionCycleRequest</code> ; execute: bool = False; episode_id: str = 'canonical-cycle'; max_execution_steps: int = 1; execution_observations: tuple[<code>ExecutionObservation</code> , ...] = (); expected_initial_state_id: str None = None; label: str = ''
Source	<code>models.py:979</code>
Evidence	Direct use: test files 2, benchmarks 1; not named in doc 05; e.g. <code>benchmark_multicycle_sequence_v042.py</code> .

Entry — `PVPPRuntime.prepare_memory_conditioned_cycle_snapshot`

Field	Content
Name and label	<code>PVPPRuntime.prepare_memory_conditioned_cycle_snapshot</code> — method; <code>kernel.py:2401</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Builds a perception snapshot through licensed retrieval.
Framework basis	Memory interface (C10/D16); raw memory never reaches perception (C10/D1 §4).
Signature	<code>PVPPRuntime.prepare_memory_conditioned_cycle_snapshot(actual_state: ActualPersistentStateEnvelope, memory_state: MemoryStateReference, *, query=None, context=None, governing_concerns=(), expectation_state: ExpectationStateReference None = None, request_trace=None) → MemoryConditionedCycleSnapshot</code>
Preconditions	A <code>MemoryRetrievalService</code> is attached; the world adapter implements a memory-conditioned perception hook; valid actual, memory, and expectation states.
Result	<code>MemoryConditionedCycleSnapshot</code> .
Invariants	Only the retrieval service sees memory; perception receives the validated retrieval and optional expectation state.
Failure behavior	Raises <code>ValueError</code> for a missing service or hook or any failed validation.
Does not	Validate an objective set; this snapshot carries none.
Evidence	Direct use: test files 2, benchmarks 1; not named in doc 05; e.g. <code>benchmark_memory_retrieval_boundary_v043.py</code> .
See also	6.2.

Entry — `PVPPRuntime.evaluate_memory_conditioned_decision_cycle`

Field	Content
Name and label	<code>PVPPRuntime.evaluate_memory_conditioned_decision_cycle</code> — method; <code>kernel.py:2473</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Runs the canonical cycle from a memory-conditioned snapshot.
Framework basis	As above.
Signature	<code>PVPPRuntime.evaluate_memory_conditioned_decision_cycle(actual_state: ActualPersistentStateEnvelope, memory_state: MemoryStateReference, request: CanonicalDecisionCycleRequest, *, query=None, context=None, governing_concerns=(), expectation_state: ExpectationStateReference None = None, request_trace=None)</code>
Preconditions	As for the snapshot and the cycle.
Result	The snapshot and the <code>CanonicalDecisionCycleAssessment</code> .
Invariants	The decision cycle is unchanged; only perception differs.
Failure behavior	As for the calls it composes.

Field	Content
Does not	Add memory semantics.
Evidence	Direct use: test files 3; not named in doc 05; e.g. test_memory_retrieval_boundary_v043.py, test_perceived_decision_state_boundary_v056.py.
See also	6.2; 9.3.

Entry — `PVPPRuntime.evaluate_memory_conditioned_integrated_cycle`

Field	Content
Name and label	<code>PVPPRuntime.evaluate_memory_conditioned_integrated_cycle</code> — method; kernel.py:2771. PUBLIC / ADVANCED (rule R2).
Purpose	Runs one retrieval-conditioned pass through decision, optional execution, and optional memory and expectation update.
Framework basis	As above; post-execution update (6.6).
Signature	<code>PVPPRuntime.evaluate_memory_conditioned_integrated_cycle(actual_state : ActualPersistentStateEnvelope, memory_state: MemoryStateReference, request: CanonicalDecisionCycleRequest, *, query=None, retrieval_context=None, governing_concerns=(), expectation_state: ExpectationStateReference None = None, request_trace=None, execute: bool = False, update_epistemic_state: bool = False, episode_id: str = memory-conditioned-cycle, max_execution_steps: int = 1, execution_observations=()) → MemoryConditionedIntegratedCycleResult</code>
Preconditions	As for the calls it composes; an epistemic update needs execution and a terminal ε result.
Result	<code>MemoryConditionedIntegratedCycleResult</code> , with a status such as "memory_cycle_selected_not_executed" or "memory_cycle_execution_blocked", and every artifact produced.
Invariants	Composes 9.3, Chapter 11, and 6.5–6.6 in order.
Failure behavior	Raises <code>ValueError</code> when an update is requested without a terminal execution.
Does not	Learn, persist, or cycle on its own.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_prediction_error_boundary_v052.py.
See also	6.5–6.6.

Entry — `PVPPRuntime.evaluate_memory_conditioned_cycle_sequence`

Field	Content
Name and label	<code>PVPPRuntime.evaluate_memory_conditioned_cycle_sequence</code> — method; kernel.py:2916. PUBLIC / ADVANCED (rule R2).
Purpose	Runs a finite, host-authored sequence of memory-conditioned cycles.

Field	Content
Framework basis	As above.
Signature	PVPPRuntime.evaluate_memory_conditioned_cycle_sequence(initial_actual_state: ActualPersistentStateEnvelope, initial_memory_state: MemoryStateReference, directives, initial_expectation_state: ExpectationStateReference None = None) → MemoryConditionedCycleSequenceAssessment
Preconditions	Valid initial actual and memory states; MemoryConditionedCycleDirective records.
Result	MemoryConditionedCycleSequenceAssessment: "memory_sequence_completed" or "memory_sequence_stopped_at_active_epsilon", the ledger, and the final actual, memory, and expectation states.
Invariants	Each directive may state the actual, memory, and expectation state ids it expects; a mismatch raises.
Failure behavior	Raises ValueError for an invalid initial state, directive type, or expected-state mismatch.
Does not	Loop autonomously.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_memory_conditioned_sequence_v045.py.
See also	6.2.

Record — MemoryConditionedCycleDirective

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; request: CanonicalDecisionCycleRequest; query: Any = None; retrieval_context: Any = None; governing_concerns: tuple[str, ...] = (); request_trace: Mapping[str, Any] = (empty); execute: bool = False; update_epistemic_state: bool = False; episode_id: str = 'memory-conditioned-cycle'; max_execution_steps: int = 1; execution_observations: tuple[ExecutionObservation, ...] = (); expected_actual_state_id: str None = None; expected_memory_state_id: str None = None; expected_expectation_state_id: str None = None; label: str = ''
Source	models.py:865
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_memory_conditioned_sequence_v045.py.

9.6 Host Hooks the Cycle Calls

The canonical cycle reaches the host through the world adapter and the optional projection service. Table 9.5 lists each hook, when it is required, and the record it returns. The adapter itself is documented in Chapter 10.

Table 9.5 — Host hooks used by the canonical cycle

Hook	Required	Returns and use
perceive, domain_value	Always	The perceived WorldState and each domain's value
project (with the "steady" action)	Always	ActionProjection for the baseline continuation used by Φ and H
pressure_factors, pressure_value, expected_deterioration	Always (strict Φ/H)	PressureFactors and the host's domain-sensitive pressure and deterioration
constraint_profile	Always	PolicyConstraintProfile of ConstraintObservation records per candidate
PolicyProjectionService, or project_policy_record	Always, one or the other	PolicyProjectionRecord $Q_t(\pi)$ per feasible candidate, with RecoveryCorridorProjection facts
fallback_structure_profile	For fallback Σ with pairwise conflicts or ties	FallbackStructuralProfile
compare_constraint_violation_severity	For terminal Σ within-class comparison	An ordering of two violation profiles within one constraint class

Record — PressureFactors

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: returned by the world adapter)
Fields	frozen dataclass; domain_id: str; margin: float; local_trajectory: float; discontinuity: float = 0.0; persistence: float = 0.0; propagation: float = 0.0; context: Mapping[str, Any] = (empty)
Source	models.py:1029
Evidence	Direct use: test files 9, benchmarks 7, example files 2; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.

Record — ActionProjection

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: returned by the world adapter)
Fields	frozen dataclass; action_id: str; next_state: WorldState; feasible: bool; reasons: tuple[str, ...] = (); metadata: Mapping[str, Any] = (empty)
Source	models.py:1103

Field	Content
Evidence	Direct use: test files 26, benchmarks 26, example files 12; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.

Record — PolicyConstraintProfile

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: returned by the world adapter)
Fields	frozen dataclass; policy_id: str; observations: tuple[ConstraintObservation, ...]; notes: tuple[str, ...] = ()
Source	models.py:1448
Evidence	Direct use: test files 9, benchmarks 7, example files 2; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.

Record — ConstraintObservation

Field	Content
Label	PUBLIC / SUPPORTED (rule R6)
Fields	frozen dataclass; rule_id: str; violated: bool; active: bool = True; regime_relaxable: bool = False; necessity_authorized: bool = False; reasons: tuple[str, ...] = ()
Source	models.py:1432
Evidence	Direct use: test files 9, benchmarks 7, example files 2; not named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.

Record — PolicyProjectionRecord

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: returned by the projection service or world adapter)

Field	Content
Fields	frozen dataclass; policy_id: str; feasible: bool; terminal_state: WorldState None = None; projected_horizons: Mapping[str, float] = (empty); recovery_corridors: tuple[RecoveryCorridorProjection, ...] = (); closure_annotations: Mapping[str, Any] = (empty); reopening_annotations: Mapping[str, Any] = (empty); reasons: tuple[str, ...] = (); metadata: Mapping[str, Any] = (empty); projection_horizon: float None = None; right_censored_domain_ids: tuple[str, ...] = (); state_id: str None = None; state_time: float None = None; candidate_mode: str = ""; projected_domain_trajectories: Mapping[str, Any] = (empty); reachable_viable: Mapping[str, Any] = (empty); information_quality_trace: Mapping[str, Any] = (empty); information_quality_claims: tuple[ProjectionInformationQualityClaim, ...] = (); model_version: str = ""; projection_input_trace: Mapping[str, Any] = (empty); model_id: str = ""; model_configuration_id: str = ""; model_parameter_set_id: str = ""
Source	models.py:1554
Evidence	Direct use: test files 17, benchmarks 10, example files 2; named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.

Record — RecoveryCorridorProjection

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: part of PolicyProjectionRecord)
Fields	frozen dataclass; domain_id: str; function_id: str; pre_recovery_viability_preserved: bool; recovery_capable_region_reached: bool; joint_sustainment_supported: bool; recovery_entry_time: float None = None; projected_extinction_time: float None = None; notes: tuple[str, ...] = ()
Source	models.py:1521
Evidence	Direct use: test files 14, benchmarks 7, example files 2; named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.

Record — FallbackStructuralProfile

Field	Content
Label	PUBLIC / ADVANCED (rule R6: optional hook)
Fields	frozen dataclass; policy_id: str; critical_damage_structure_ids: tuple[str, ...] = (); critical_maneuver_structure_ids: tuple[str, ...] = (); irreversible_damage_rank: int = 0; noncritical_maneuver_rank: int = 0; spillover_damage_rank: int = 0; notes: tuple[str, ...] = ()
Source	models.py:1788
Evidence	Direct use: test files 2, benchmarks 2; not named in doc 05; e.g. three_domain_buffered_multistep_scenario_v048.py, three_domain_oscillation_scenario_v047.py.

9.7 Stage Calls: Pressure, Horizons, Governing Domains, and Regime

Entry — `PVPPRuntime.evaluate_pressure_field`

Field	Content
Name and label	<code>PVPPRuntime.evaluate_pressure_field</code> — method; kernel.py:73. PUBLIC / ADVANCED (rule R2).
Purpose	Computes Φ for every registered domain.
Framework basis	C15/D8.
Signature	<code>PVPPRuntime.evaluate_pressure_field(state) → PressureFieldAssessment</code>
Preconditions	The world provides <code>pressure_factors</code> and <code>pressure_value</code> .
Result	<code>PressureFieldAssessment</code> with per-domain results.
Invariants	Uses the baseline continuation from the "steady" action. The canonical cycle runs this stage itself; call it directly only for stage-level work such as tests or diagnostics.
Failure behavior	Raises <code>ValueError</code> if the hooks are missing.
Does not	Classify governing domains or regime.
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_phi_h_strict_v028.py</code> .
See also	3.3.

Entry — `PVPPRuntime.validate_pressure_invariants`

Field	Content
Name and label	<code>PVPPRuntime.validate_pressure_invariants</code> — method; kernel.py:81. PUBLIC / ADVANCED (rule R2).
Purpose	Probes Φ 's monotonicity for one domain over caller-supplied margins, holding the other inputs fixed.
Framework basis	C15/D8 monotonicity (Table 3.2).
Signature	<code>PVPPRuntime.validate_pressure_invariants(domain_id: str, reference_factors: PressureFactors, margins) → PressureInvariantAssessment</code>
Preconditions	At least two margins, none NaN; reference factors for the same domain; the world provides <code>pressure_value</code> .
Result	<code>PressureInvariantAssessment</code> .
Invariants	The caller supplies meaningful domain-scale margins; the runtime invents no probe scale.
Failure behavior	Raises <code>ValueError</code> for invalid input.
Does not	Prove monotonicity in general; it tests the margins given.
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_phi_h_strict_v028.py</code> .
See also	3.3.

Entry — `PVPPRuntime.evaluate_collapse_horizons`

Field	Content
Name and label	<code>PVPPRuntime.evaluate_collapse_horizons</code> — method; kernel.py:158. PUBLIC / ADVANCED (rule R2).
Purpose	Computes baseline collapse horizons.
Framework basis	C15/D4.
Signature	<code>PVPPRuntime.evaluate_collapse_horizons(state, pressure_field: PressureFieldAssessment None = None) → HorizonAssessment</code>
Preconditions	The world provides <code>expected_deterioration</code> .
Result	<code>HorizonAssessment</code> .
Invariants	The canonical cycle runs this stage itself; call it directly only for stage-level work such as tests or diagnostics.
Failure behavior	Raises <code>ValueError</code> if the hook is missing.
Does not	Compute policy-conditioned horizons; projection owns those.
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_phi_h_strict_v028.py</code> .
See also	3.3.

Entry — `PVPPRuntime.identify_governing_domains`

Field	Content
Name and label	<code>PVPPRuntime.identify_governing_domains</code> — method; kernel.py:226. PUBLIC / ADVANCED (rule R2).
Purpose	Computes the governing set from horizons and recovery necessities.
Framework basis	C15/D2.
Signature	<code>PVPPRuntime.identify_governing_domains(horizons) → GoverningAssessment</code>
Preconditions	Horizons for every registered domain, none NaN.
Result	<code>GoverningAssessment</code> : seed, governing set, added domains, the necessities used, and provenance.
Invariants	Horizon seed, then transitive recovery-necessity closure (Table 3.4). The canonical cycle runs this stage itself; call it directly only for stage-level work such as tests or diagnostics.
Failure behavior	Raises <code>ValueError</code> for missing or NaN horizons.
Does not	Consult pressure or ranking.
Evidence	Direct use: test files 2, benchmarks 1; not named in doc 05; e.g. <code>benchmark_governing_operator_v027.py</code> .
See also	3.4.

Entry — [PVPPRuntime.classify_regime](#)

Field	Content
Name and label	PVPPRuntime.classify_regime — method; kernel.py:277. PUBLIC / ADVANCED (rule R2).
Purpose	Classifies the regime from domain assessments, horizons, and the governing set.
Framework basis	C15/D3.
Signature	PVPPRuntime.classify_regime(domains, horizons, governing, *, contextual_interruption=False, previous: PreviousRegimeState None = None) → RegimeAssessment
Preconditions	A registered regime configuration and a non-empty governing set.
Result	RegimeAssessment with the regime, trigger, and hysteresis flag.
Invariants	Triggers and hysteresis as in Table 3.5. The canonical cycle runs this stage itself; call it directly only for stage-level work such as tests or diagnostics.
Failure behavior	Raises ValueError for a missing configuration, an empty set, or an unknown previous regime.
Does not	Change governing membership.
Evidence	Direct use: test files 2, benchmarks 1; not named in doc 05; e.g. benchmark_regime_operator_v021.py.
See also	3.4.

9.8 Stage Calls: Policy Construction and Completeness

Entry — [PVPPRuntime.construct_pi_from_graph](#)

Field	Content
Name and label	PVPPRuntime.construct_pi_from_graph — method; kernel.py:1291. PUBLIC / ADVANCED (rule R2).
Purpose	Constructs the candidate space from a graph that passed its validity boundary.
Framework basis	C15/D7.
Signature	PVPPRuntime.construct_pi_from_graph(graph: GraphConstructionAssessment, materially_required_class_ids, *, config: PiConstructionConfig None = None) → PiConstructionAssessment
Preconditions	A GraphConstructionAssessment; a positive max_candidates.
Result	PiConstructionAssessment with the policy space, or "pi_blocked_by_graph_invalid" and no space.
Invariants	Regime activates families (Table 3.6); required seeds are never suppressed. The canonical cycle runs this stage itself; call it directly only for stage-level work such as tests or diagnostics.

Field	Content
Failure behavior	Raises ValueError for a bad configuration or regime.
Does not	Repair or bypass an invalid graph handoff, test feasibility, or rank.
Evidence	Direct use: test files 3; not named in doc 05; e.g. test_graph_construction_v026.py, test_graph_uncertainty_integrity_v057.py.
See also	3.5.

Entry — [PVPPRuntime.construct_pi](#)

Field	Content
Name and label	PVPPRuntime.construct_pi — method; kernel.py:1232. PUBLIC / ADVANCED (rule R2).
Purpose	Constructs a candidate space from seeds registered directly with register_policy_seed.
Framework basis	C15/D7.
Signature	PVPPRuntime.construct_pi(regime: str, governing_domain_ids, materially_required_class_ids, *, config: PiConstructionConfig None = None) → PiConstructionAssessment
Preconditions	A supported regime; a positive max_candidates.
Result	PiConstructionAssessment.
Invariants	Applies the same family activation and suppression rules as construct_pi_from_graph. The canonical cycle does not call this method; it builds candidates from the graph.
Failure behavior	Raises ValueError for a bad configuration or regime.
Does not	Validate graph structure; registered seeds bypass the graph protocol.
Evidence	Direct use: test files 2, benchmarks 2; not named in doc 05; e.g. benchmark_mission_required_pi_v049.py, benchmark_pi_construction_v025.py.
See also	4.6; H.5-4.

Entry — [PVPPRuntime.validate_pi_completeness](#)

Field	Content
Name and label	PVPPRuntime.validate_pi_completeness — method; kernel.py:1373. PUBLIC / ADVANCED (rule R2).
Purpose	Checks that the candidate space represents every materially required class.
Framework basis	C15/D12.
Signature	PVPPRuntime.validate_pi_completeness(policy_space: CandidatePolicySpace) → PiCompletenessAssessment

Field	Content
Preconditions	None.
Result	PiCompletenessAssessment: "pi_completeness_passed" or "pi_completeness_failed_missing_material_policy_classes", with represented and missing classes.
Invariants	Required classes come from the space's declaration. The canonical cycle runs this stage itself; call it directly only for stage-level work such as tests or diagnostics.
Failure behavior	Returns the failure; raises nothing.
Does not	Discover required classes, test feasibility, or rank.
Evidence	Direct use: test files 5, benchmarks 1; not named in doc 05; e.g. benchmark_pi_completeness_v018.py.
See also	3.6.

Record — EpistemicSubstrateQualification

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; status: str; basis: str; warning_ids: tuple[str, ...] = (); notes: tuple[str, ...] = ()
Source	models.py:1320
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_qualified_pi_v054.py.

Entry — PVPPRuntime.qualify_pi_completeness_for_substrate

Field	Content
Name and label	PVPPRuntime.qualify_pi_completeness_for_substrate — method; kernel.py:1406. PUBLIC / ADVANCED (rule R2).
Purpose	Qualifies a completeness result by the epistemic status of the substrate it was computed on.
Framework basis	Π completeness on contaminated substrates (C45/D8; C15/D1).
Signature	PVPPRuntime.qualify_pi_completeness_for_substrate(local: PiCompletenessAssessment, qualification: EpistemicSubstrateQualification) → QualifiedPiCompletenessAssessment
Preconditions	A local completeness result and a qualification with status "clean", "uncertain", or "epistemically_contaminated" and an explicit basis.
Result	QualifiedPiCompletenessAssessment.
Invariants	Not invoked by the canonical cycle; its docstring describes it as a supporting protocol under hardening before promotion.
Failure behavior	Raises ValueError for a missing basis or unknown status.

Field	Content
Does not	Repair Π.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_qualified_pi_v054.py.
See also	3.6.

9.9 Stage Calls: Constraints and Joint Recovery

Entry — `PVPPRuntime.evaluate_constraint_system`

Field	Content
Name and label	<code>PVPPRuntime.evaluate_constraint_system</code> — method; kernel.py:1816. PUBLIC / ADVANCED (rule R2).
Purpose	Classifies every candidate under the hard, conditional, and soft constraint taxonomy.
Framework basis	C15/D5.
Signature	<code>PVPPRuntime.evaluate_constraint_system(state, candidates, *, regime: str, governing_domain_ids) → ConstraintsAssessment</code>
Preconditions	The world provides <code>constraint_profile</code> ; constraint rules are registered.
Result	<code>ConstraintsAssessment</code> : "constraints_passed_all_candidates", "constraints_filtered_candidates", or "constraints_no_feasible_candidates", with per-candidate results and violation profiles.
Invariants	Hard constraints are non-relaxable and regime-invariant. A conditional constraint is relaxed only when the observation carries both regime permission and explicit necessity authorization. Soft constraints never remove a candidate. The canonical cycle runs this stage itself; call it directly only for stage-level work such as tests or diagnostics.
Failure behavior	Raises <code>ValueError</code> for a missing hook, no rules, or a mismatched profile.
Does not	Fabricate the adequacy evidence that necessity requires.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_constraints_v031.py.
See also	3.7.

Entry — `PVPPRuntime.assess_recovery_feasibility`

Field	Content
Name and label	<code>PVPPRuntime.assess_recovery_feasibility</code> — method; kernel.py:4103. PUBLIC / ADVANCED (rule R2).
Purpose	Checks whether the runtime's active recovery corridors can all be scheduled within a capacity calendar.
Framework basis	C15/D10 §7.3 (3.9).

Field	Content
Signature	PVPPRuntime.assess_recovery_feasibility(calendar: CapacityCalendar, *, now: float = 0.0, diagnostic_mode: str = witness)
Preconditions	A CapacityCalendar.
Result	CorridorFeasibilityReport (5.8).
Invariants	Uses the runtime's active-corridor ledger; "witness" or "all_minimal" diagnostics. The canonical cycle performs this check and gates on it.
Failure behavior	Raises ValueError for an unknown diagnostic mode.
Does not	Apply a priority or sacrifice rule.
Evidence	Direct use: test files 2, benchmarks 2, example files 1; not named in doc 05; e.g. run_dual.py (legacy tiny_* example, 15.4).
See also	3.9; 5.8.

9.10 Projections

Projections are facts supplied by the shared projection service, one record per constraint-feasible candidate, shared unchanged by Adequacy and Σ (C10/D6; C15/D6 §3.3). v0.141 adds licensing: the model that produced a record may be declared with its version and licensed horizon, and the causal inputs a projection may use may be required explicitly. When a projection service is attached, a failed license stops the cycle.

Record — ProjectionModelAuthority

Field	Content
Label	PUBLIC / ADVANCED (rule R6: optional part of the request)
Fields	frozen dataclass; model_id: str; model_version: str; configuration_id: str; parameter_set_id: str; license_basis: str; licensed_projection_horizon: float; metadata: Mapping[str, Any] = (empty)
Source	models.py:246
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_projection_model_authority_v062.py.

Record — ProjectionCausalStateInput

Field	Content
Label	PUBLIC / ADVANCED (rule R6: part of PerceivedDecisionState)
Fields	frozen dataclass; input_id: str; kind: str; value: Any; license_basis: str; represented_time: float; source_ids: tuple[str, ...] = (); metadata: Mapping[str, Any] = (empty)
Source	models.py:219
Evidence	Direct use: test files 3, benchmarks 2; not named in doc 05; e.g. benchmark_projection_causal_input_licensing_v060.py, benchmark_projection_causal_state_cross_cycle_v061.py.

Record — ProjectionInformationQualityClaim

Field	Content
Label	PUBLIC / ADVANCED (rule R6: optional part of PolicyProjectionRecord)
Fields	frozen dataclass; subject_kind: str; subject_id: str; basis: str; confidence: Any; uncertainty_note: str = ''; source_ids: tuple[str, ...] = (); metadata: Mapping[str, Any] = (empty)
Source	models.py:1538
Evidence	Direct use: test files 2, benchmarks 1; not named in doc 05; e.g. benchmark_projection_information_quality_v058.py.

Entry — PVPPRuntime.validate_projection_model_authority

Field	Content
Name and label	PVPPRuntime.validate_projection_model_authority — method; kernel.py:1453. PUBLIC / ADVANCED (rule R2).
Purpose	Checks a declared model authority against the projection horizon.
Framework basis	Shared projection service (C10/D6).
Signature	PVPPRuntime.validate_projection_model_authority(authority: ProjectionModelAuthority None, projection_horizon) → ProjectionModelAuthorityAssessment
Preconditions	None; no authority is valid, for backward compatibility.
Result	ProjectionModelAuthorityAssessment.
Invariants	A declared authority must be well formed and license exactly the horizon L_P the cycle uses.
Failure behavior	Returns violations.
Does not	Interpret or synthesize a model identity.
Evidence	Direct use: no direct use in tests, benchmarks, or examples; not named in doc 05.
See also	9.10.

Entry — PVPPRuntime.validate_projection_causal_inputs

Field	Content
Name and label	PVPPRuntime.validate_projection_causal_inputs — method; kernel.py:1491. PUBLIC / ADVANCED (rule R2).
Purpose	Checks that required causal inputs are present and licensed on the perceived decision state.
Framework basis	As above.
Signature	PVPPRuntime.validate_projection_causal_inputs(perceived: PerceivedDecisionState None, required_input_ids=()) → ProjectionInputLicenseAssessment

Field	Content
Preconditions	None.
Result	ProjectionInputLicenseAssessment.
Invariants	Required input ids come from the host and model boundary.
Failure behavior	Returns violations.
Does not	Infer a missing commitment, fact, capacity, or expectation from graph, actual state, or metadata.
Evidence	Direct use: benchmarks 1; not named in doc 05; e.g. benchmark_projection_causal_input_licensing_v060.py.
See also	9.10.

Entry — `PVPPRuntime.validate_projection_record`

Field	Content
Name and label	PVPPRuntime.validate_projection_record — method; kernel.py:1586. PUBLIC / ADVANCED (rule R2).
Purpose	Checks one projection record against the request that produced it.
Framework basis	As above.
Signature	PVPPRuntime.validate_projection_record(request: ProjectionRequest, record: PolicyProjectionRecord) → PolicyProjectionRecord
Preconditions	The ProjectionRequest.
Result	The record, if valid.
Invariants	Identity, candidate mode, the licensed horizon L_P recorded exactly, censoring only for known domains, at most one corridor fact per domain and function, typed quality claims, and model identity and inputs matching any licenses.
Failure behavior	Raises ValueError on any violation.
Does not	Classify feasibility, adequacy, dominance, or preference.
Evidence	Direct use: test files 1, benchmarks 2; not named in doc 05; e.g. benchmark_projection_information_quality_v058.py, benchmark_projection_service_v036.py.
See also	9.10.

9.11 Stage Calls: Adequacy and Σ

Entry — `PVPPRuntime.evaluate_restoration_adequacy`

Field	Content
Name and label	PVPPRuntime.evaluate_restoration_adequacy — method; kernel.py:1721. PUBLIC / ADVANCED (rule R2).
Purpose	Judges each candidate's projected recovery corridors against the frame.

Field	Content
Framework basis	C15/D10.
Signature	PVPPRuntime.evaluate_restoration_adequacy(frame: DomainFrame, governing, records) → AdequacyAssessment
Preconditions	A frame covering the governing set exactly; the projection records.
Result	AdequacyAssessment with adequate and inadequate candidates and per-domain results.
Invariants	Conjunctive over the governing set; uses only the records. The canonical cycle runs this stage itself; call it directly only for stage-level work such as tests or diagnostics.
Failure behavior	Raises ValueError if the frame does not cover the governing set.
Does not	Invent trajectories, regions, substitutions, closures, or reopenings; rank.
Evidence	Direct use: benchmarks 1; not named in doc 05; e.g. benchmark_restoration_adequacy_v023.py.
See also	3.10.

Record — PolicySetEvaluation

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the caller for direct Σ calls; built by the cycle otherwise)
Fields	frozen dataclass; policy_id: str; action_ids: tuple[str, ...]; feasible: bool; adequate: bool; projected_horizons: Mapping[str, float] = (empty); reasons: tuple[str, ...] = (); projection_horizon: float None = None; right_censored_domain_ids: tuple[str, ...] = ()
Source	models.py:1673
Evidence	Direct use: test files 4, benchmarks 6; not named in doc 05; e.g. benchmark_branching_structure_v015.py, benchmark_dependency_requirements_v016.py.

Entry — PVPPRuntime.evaluate_sigma_standard

Field	Content
Name and label	PVPPRuntime.evaluate_sigma_standard — method; kernel.py:786. PUBLIC / ADVANCED (rule R2).
Purpose	Selects from the adequate set by Σ Stages 1–3.
Framework basis	C15/D6 v1.14 (3.11).
Signature	PVPPRuntime.evaluate_sigma_standard(evaluations, governing_domain_ids)

Field	Content
Preconditions	Evaluations that are feasible and adequate, with aligned consequence domains covering the governing set; a common licensed horizon when any value is censored; a registered SigmaOrderDefinition for every policy that may survive to Stage 3.
Result	SigmaAssessment: stage results, survivors, the selection, and its order index.
Invariants	Horizon comparisons are censor-aware. The canonical cycle runs this stage itself; call it directly only for stage-level work such as tests or diagnostics.
Failure behavior	Raises ValueError for each unmet precondition, including a Stage-3 tie among policies without a registered order.
Does not	Test feasibility or adequacy.
Evidence	Direct use: test files 3, benchmarks 2; not named in doc 05; e.g. benchmark_sigma_stage3_v029.py, benchmark_sigma_standard_v024.py.
See also	3.11; 4.6.

Entry — `PVPPRuntime.evaluate_sigma_fallback`

Field	Content
Name and label	PVPPRuntime.evaluate_sigma_fallback — method; kernel.py:567. PUBLIC / ADVANCED (rule R2).
Purpose	Selects from Π_{valid} when no candidate is adequate.
Framework basis	C15/D6; C15/D17.
Signature	PVPPRuntime.evaluate_sigma_fallback(evaluations, governing_domain_ids, structural_profiles=None)
Preconditions	Feasible, inadequate evaluations covering the governing set; structural profiles for pairwise conflicts and Stage-1 ties; a registered order for residual survivors.
Result	SigmaFallbackAssessment, for example "fallback_singleton_maximal_selected".
Invariants	Resolves singleton maximal sets and pairwise conflicts; stops on unresolved conflicts of three or more (15.1). The canonical cycle runs this stage itself; call it directly only for stage-level work such as tests or diagnostics.
Failure behavior	Raises ValueError for each unmet precondition, including while an adequate policy exists.
Does not	Invent a multiway rule.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_sigma_fallback_v034.py.
See also	3.11.

Entry — [PVPPRuntime.evaluate_sigma_terminal](#)

Field	Content
Name and label	PVPPRuntime.evaluate_sigma_terminal — method; kernel.py:748. PUBLIC / ADVANCED (rule R2).
Purpose	Selects when Π_{valid} is empty, by strict constraint-class priority.
Framework basis	C15/D6 (terminal infeasibility).
Signature	PVPPRuntime.evaluate_sigma_terminal(constraint_results) → SigmaTerminalAssessment
Preconditions	A structured violation profile per candidate; the world's within-class comparison hook where classes tie; a registered order for residual ties.
Result	SigmaTerminalAssessment (not exported at the top level).
Invariants	Hard before conditional before soft; a lower-class improvement never offsets a higher-class worsening. The canonical cycle runs this stage itself; call it directly only for stage-level work such as tests or diagnostics.
Failure behavior	Raises ValueError for missing profiles, duplicates, or unordered residual ties.
Does not	Make the selection executable: entry to ε is blocked (3.12).
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_sigma_terminal_v035.py.
See also	3.11.

9.12 Cycle-Integrity Audits

Entry — [PVPPRuntime.audit_cycle_artifact_integrity](#)

Field	Content
Name and label	PVPPRuntime.audit_cycle_artifact_integrity — method; kernel.py:3554. PUBLIC / ADVANCED (rule R2).
Purpose	Checks that one cycle's artifacts share one identity basis.
Framework basis	Shared projection record set (C10/D6).
Signature	PVPPRuntime.audit_cycle_artifact_integrity(cycle: CanonicalDecisionCycleAssessment, *, expected_state_id: str None = None, expected_state_time: float None = None) → CycleArtifactIntegrityAssessment
Preconditions	A cycle result; optionally the expected state id and time.
Result	CycleArtifactIntegrityAssessment.
Invariants	Graph, Π , and framing name the same governing set; Π seeds match the graph; Constraints, projection records, Adequacy, and Σ all refer to the current Π space; the projection set equals the feasible set; projection records share one state id and time.

Field	Content
Failure behavior	Returns violations. The cycle and license builder raise on a failed audit.
Does not	Infer semantic staleness or compare with persistent memory.
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_cycle_artifact_identity_integrity_v064.py, test_v2_governing_identity_ordering.py.
See also	9.3; 11.2.

Entry — `PVPPRuntime.audit_integrated_cycle_lifecycle_integrity`

Field	Content
Name and label	<code>PVPPRuntime.audit_integrated_cycle_lifecycle_integrity</code> — method; kernel.py:4839. PUBLIC / ADVANCED (rule R2).
Purpose	Checks that an integrated result's artifacts match its status.
Framework basis	Execution lineage (C15/D9).
Signature	<code>PVPPRuntime.audit_integrated_cycle_lifecycle_integrity(integrated: CanonicalIntegratedCycleResult) → IntegratedCycleLifecycleIntegrityAssessment</code>
Preconditions	An integrated result.
Result	<code>IntegratedCycleLifecycleIntegrityAssessment</code> .
Invariants	For example: a selected-not-executed result carries no execution artifacts; a transition follows a terminal ϵ result; the chain of license, ϵ , handoff, transition, and provenance is complete and consistent.
Failure behavior	Returns violations.
Does not	Repair a cycle or supply missing artifacts.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_integrated_cycle_lifecycle_integrity_v069.py.
See also	Table 9.3.

Entry — `PVPPRuntime.audit_post_selection_identity_chain`

Field	Content
Name and label	<code>PVPPRuntime.audit_post_selection_identity_chain</code> — method; kernel.py:4904. PUBLIC / ADVANCED (rule R2).
Purpose	Checks that objects after selection have not been spliced across episodes, policies, action bundles, or transition chains.
Framework basis	As above.

Field	Content
Signature	PVPPRuntime.audit_post_selection_identity_chain(integrated: CanonicalIntegratedCycleResult, prediction_error_request: PredictionErrorRequest None = None, prediction_error_assessment: PredictionErrorAssessment None = None, epistemic_update_request: PostExecutionEpistemicUpdateRequest None = None) → CrossObjectExecutionIdentityAssessment
Preconditions	An integrated result; optionally the prediction-error and epistemic-update objects built from it.
Result	CrossObjectExecutionIdentityAssessment.
Invariants	ε , handoff, and provenance match the license's policy and actions; prediction-error and update requests match the provenance.
Failure behavior	Returns violations.
Does not	Create authority or repair mismatches.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_cross_object_execution_identity_v068.py.
See also	6.5–6.6; 11.

9.13 Paths Outside the Canonical Cycle

v0.141 keeps several evaluation paths from builds before the canonical cycle (v0.32). They are not the canonical stack. The audit labels the four evaluators below LEGACY / COMPATIBILITY (H.5-5):

- **validate_domain_framing(state, frame)**. Checks the same target rules as the canonical cycle, but computes the governing set with the prototype operators of the older assess path, not with the canonical G of 3.4. It can therefore validate a frame against a different governing set from the one the cycle uses. The canonical cycle validates framing internally against its own G .
- **evaluate_policy_space, evaluate_recovery_policy_space**. Evaluate a caller-built policy space. Their domains come from the same prototype path, and when no authoritative adequacy is available they fall back to a compatibility horizon heuristic and a comparator their own notes describe as “not asserted canonical Σ mathematics”.
- **evaluate_constraints**. Checks feasibility through action projection rather than the host constraint profile of the canonical Constraint System.
- **The operator prototypes**. pressure_from_margin, collapse_horizon, governing_from_horizon, dependency_close, and regime_label in pvpp_runtime.operators.prototype serve only that older path. They are INTERNAL (rule R4) and are not the canonical Φ , H , G , or $\mathcal{R}$, which use the host's strict hooks.

These four entries are LEGACY / COMPATIBILITY under rule R5. They are V1-generation paths that the canonical cycle superseded in the release lineage at v0.32, and each departs from the canonical stack in a documented way. Their replacements: the canonical cycle's internal framing check for validate_domain_framing, and the cycle entries of 9.3 for the two policy-space evaluators; for evaluate_constraints, evaluate_constraint_system (9.9). Chapter 15 gives the full replacement mapping. The operator prototypes stay INTERNAL. The same review found that seeds registered with register_policy_seed feed only construct_pi, which the canonical cycle

does not call; register_policy_seed and PolicySeedDefinition are therefore relabeled PUBLIC / ADVANCED (H.5-4).

9.14 Classification Record and Compact Rows

Table 9.6 — Labels assigned in Chapter 9

Symbol	Label	Rule
PVPPRuntime.evaluate_canonical_decision_cycle	PUBLIC / SUPPORTED	R1
PVPPRuntime.evaluate_integrated_canonical_cycle	PUBLIC / SUPPORTED	R1
PVPPRuntime.prepare_canonical_cycle_snapshot	PUBLIC / SUPPORTED	R1
PVPPRuntime.evaluate_canonical_cycle_sequence	PUBLIC / ADVANCED	R2
PVPPRuntime.prepare_memory_conditioned_cycle_snapshot	PUBLIC / ADVANCED	R2
PVPPRuntime.evaluate_memory_conditioned_decision_cycle	PUBLIC / ADVANCED	R2
PVPPRuntime.evaluate_memory_conditioned_integrated_cycle	PUBLIC / ADVANCED	R2
PVPPRuntime.evaluate_memory_conditioned_cycle_sequence	PUBLIC / ADVANCED	R2
PVPPRuntime.evaluate_pressure_field	PUBLIC / ADVANCED	R2
PVPPRuntime.evaluate_collapse_horizons	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_pressure_invariants	PUBLIC / ADVANCED	R2
PVPPRuntime.identify_governing_domains	PUBLIC / ADVANCED	R2
PVPPRuntime.classify_regime	PUBLIC / ADVANCED	R2
PVPPRuntime.construct_pi	PUBLIC / ADVANCED	R2
PVPPRuntime.construct_pi_from_graph	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_pi_completeness	PUBLIC / ADVANCED	R2
PVPPRuntime.qualify_pi_completeness_for_substrate	PUBLIC / ADVANCED	R2

Symbol	Label	Rule
PVPPRuntime.evaluate_constraint_system	PUBLIC / ADVANCED	R2
PVPPRuntime.assess_recovery_feasibility	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_projection_record	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_projection_causal_inputs	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_projection_model_authority	PUBLIC / ADVANCED	R2
PVPPRuntime.evaluate_restoration_adequacy	PUBLIC / ADVANCED	R2
PVPPRuntime.evaluate_sigma_standard	PUBLIC / ADVANCED	R2
PVPPRuntime.evaluate_sigma_fallback	PUBLIC / ADVANCED	R2
PVPPRuntime.evaluate_sigma_terminal	PUBLIC / ADVANCED	R2
PVPPRuntime.audit_cycle_artifact_integrity	PUBLIC / ADVANCED	R2
PVPPRuntime.audit_integrated_cycle_lifecycle_integrity	PUBLIC / ADVANCED	R2
PVPPRuntime.audit_post_selection_identity_chain	PUBLIC / ADVANCED	R2
PVPPRuntime.evaluate_policy_space	LEGACY / COMPATIBILITY	R5 (H.5-5, revised in v0.14)
PVPPRuntime.evaluate_recovery_policy_space	LEGACY / COMPATIBILITY	R5 (H.5-5, revised in v0.14)
PVPPRuntime.evaluate_constraints	LEGACY / COMPATIBILITY	R5 (H.5-5, revised in v0.14)
PVPPRuntime.validate_domain_framing	LEGACY / COMPATIBILITY	R5 (H.5-5, revised in v0.14)
CanonicalDecisionCycleRequest	PUBLIC / SUPPORTED	R6

Symbol	Label	Rule
DomainFrame	PUBLIC / SUPPORTED	R6
DomainFrameTarget	PUBLIC / SUPPORTED	R6
PiConstructionConfig	PUBLIC / SUPPORTED	R6
PreviousRegimeState	PUBLIC / SUPPORTED	R6
PressureFactors	PUBLIC / SUPPORTED	R6
ActionProjection	PUBLIC / SUPPORTED	R6
PolicyProjectionRecord	PUBLIC / SUPPORTED	R6
RecoveryCorridorProjection	PUBLIC / SUPPORTED	R6
PolicyConstraintProfile	PUBLIC / SUPPORTED	R6
ConstraintObservation	PUBLIC / SUPPORTED	R6
ProjectionModelAuthority	PUBLIC / ADVANCED	R6
ProjectionCausalStateInput	PUBLIC / ADVANCED	R6
ProjectionInformationQualityClaim	PUBLIC / ADVANCED	R6
FallbackStructuralProfile	PUBLIC / ADVANCED	R6
EpistemicSubstrateQualification	PUBLIC / ADVANCED	R6
CanonicalCycleDirective	PUBLIC / ADVANCED	R6
MemoryConditionedCycleDirective	PUBLIC / ADVANCED	R6
PolicySetEvaluation	PUBLIC / ADVANCED	R6
CanonicalDecisionCycleAssessment	IMPLEMENTATIO N / SUPPORT	R3

Symbol	Label	Rule
CanonicalIntegratedCycleResult	IMPLEMENTATION / SUPPORT	R3
CanonicalCycleSnapshot	IMPLEMENTATION / SUPPORT	R3
CanonicalCycleLedgerEntry	IMPLEMENTATION / SUPPORT	R3
CanonicalCycleSequenceAssessment	IMPLEMENTATION / SUPPORT	R3
MemoryConditionedCycleSnapshot	IMPLEMENTATION / SUPPORT	R3
MemoryConditionedIntegratedCycleResult	IMPLEMENTATION / SUPPORT	R3
MemoryConditionedCycleLedgerEntry	IMPLEMENTATION / SUPPORT	R3
MemoryConditionedCycleSequenceAssessment	IMPLEMENTATION / SUPPORT	R3
PolicySelectionAssessment	IMPLEMENTATION / SUPPORT	R3
CycleArtifactIntegrityAssessment	IMPLEMENTATION / SUPPORT	R3
IntegratedCycleLifecycleIntegrityAssessment	IMPLEMENTATION / SUPPORT	R3
CrossObjectExecutionIdentityAssessment	IMPLEMENTATION / SUPPORT	R3
PressureFieldAssessment	IMPLEMENTATION / SUPPORT	R3
PressureDomainResult	IMPLEMENTATION / SUPPORT	R3
PressureInvariantAssessment	IMPLEMENTATION / SUPPORT	R3
HorizonAssessment	IMPLEMENTATION / SUPPORT	R3
HorizonDomainResult	IMPLEMENTATION / SUPPORT	R3
GoverningAssessment	IMPLEMENTATION / SUPPORT	R3
RegimeAssessment	IMPLEMENTATION / SUPPORT	R3

Symbol	Label	Rule
DomainAssessment	IMPLEMENTATIO N / SUPPORT	R3
PiConstructionAssessment	IMPLEMENTATIO N / SUPPORT	R3
CandidatePolicySpace	IMPLEMENTATIO N / SUPPORT	R3
CandidatePolicySet	IMPLEMENTATIO N / SUPPORT	R3
PiCompletenessAssessment	IMPLEMENTATIO N / SUPPORT	R3
QualifiedPiCompletenessAssessment	IMPLEMENTATIO N / SUPPORT	R3
ConstraintsAssessment	IMPLEMENTATIO N / SUPPORT	R3
ConstraintAssessment	IMPLEMENTATIO N / SUPPORT	R3
ConstraintViolationProfile	IMPLEMENTATIO N / SUPPORT	R3
DomainFramingAssessment	IMPLEMENTATIO N / SUPPORT	R3
AdequacyAssessment	IMPLEMENTATIO N / SUPPORT	R3
PolicyAdequacyResult	IMPLEMENTATIO N / SUPPORT	R3
DomainAdequacyResult	IMPLEMENTATIO N / SUPPORT	R3
SigmaAssessment	IMPLEMENTATIO N / SUPPORT	R3
SigmaStageAssessment	IMPLEMENTATIO N / SUPPORT	R3
SigmaFallbackAssessment	IMPLEMENTATIO N / SUPPORT	R3
ProjectionRequest	IMPLEMENTATIO N / SUPPORT	R3
ProjectionConsistencyAudit	IMPLEMENTATIO N / SUPPORT	R3
ProjectionInputLicenseAssessment	IMPLEMENTATIO N / SUPPORT	R3

Symbol	Label	Rule
ProjectionModelAuthorityAssessment	IMPLEMENTATION / SUPPORT	R3
JointRecoveryExecutionBindingAssessment	IMPLEMENTATION / SUPPORT	R3
models.SigmaTerminalAssessment	IMPLEMENTATION / SUPPORT	R3; submodule -only
operators.prototype.pressure_from_margin	INTERNAL	R4; submodule -only
operators.prototype.collapse_horizon	INTERNAL	R4; submodule -only
operators.prototype.governing_from_horizon	INTERNAL	R4; submodule -only
operators.prototype.dependency_close	INTERNAL	R4; submodule -only
operators.prototype.regime_label	INTERNAL	R4; submodule -only

Table 9.7 — Compact rows for runtime-built records

Name and kind	Label	Role and use	Source
CanonicalDecisionCycleAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of the decision cycle (Table 9.4)	models.py: 1896
CanonicalIntegratedCycleResult — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of the integrated cycle (Table 9.3)	models.py: 958
CanonicalCycleSnapshot — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of prepare_canonical_cycle_snapshot	models.py: 944
CanonicalCycleLedgerEntry — dataclass (frozen)	IMPLEMENTATION / SUPPORT	One row of a sequence ledger	models.py: 996
CanonicalCycleSequenceAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of evaluate_canonical_cycle_sequence	models.py: 1017

Name and kind	Label	Role and use	Source
MemoryConditionedCycleSnapshot — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of the memory-conditioned snapshot	models.py: 827
MemoryConditionedIntegratedCycleResult — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of the memory-conditioned integrated cycle	models.py: 839
MemoryConditionedCycleLedgerEntry — dataclass (frozen)	IMPLEMENTATION / SUPPORT	One row of a memory-conditioned ledger	models.py: 911
MemoryConditionedCycleSequenceAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of the memory-conditioned sequence	models.py: 931
PolicySelectionAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Selection record carrying the Σ result (9.4)	models.py: 1930
PressureFieldAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of Φ	models.py: 1049
PressureDomainResult — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Per-domain Φ result	models.py: 1041
PressureInvariantAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>validate_pressure_invariants</code>	models.py: 1059
HorizonAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of H	models.py: 1083
HorizonDomainResult — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Per-domain H result	models.py: 1075
GoverningAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of G	models.py: 1370
RegimeAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of $\mathcal{R}$	models.py: 1407
DomainAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Per-domain input to $\mathcal{R}$	models.py: 1093

Name and kind	Label	Role and use	Source
PiConstructionAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of Π	models.py: 1274
CandidatePolicySpace — dataclass (frozen)	IMPLEMENTATION / SUPPORT	The candidate space	models.py: 1306
CandidatePolicySet — dataclass (frozen)	IMPLEMENTATION / SUPPORT	One candidate	models.py: 1290
PiCompletenessAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of Π completeness	models.py: 1342
QualifiedPiCompletenessAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>qualify_pi_completeness_for_substrate</code>	models.py: 1329
ConstraintsAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of Constraints	models.py: 1473
ConstraintAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Per-candidate constraint result	models.py: 1463
ConstraintViolationProfile — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Violations by class for one candidate	models.py: 1455
DomainFramingAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of framing	models.py: 1510
AdequacyAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of Adequacy	models.py: 1664
PolicyAdequacyResult — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Per-candidate adequacy	models.py: 1656
DomainAdequacyResult — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Per-domain adequacy	models.py: 1644
SigmaAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of standard Σ	models.py: 1831

Name and kind	Label	Role and use	Source
SigmaStageAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	One Σ stage	models.py: 1821
SigmaFallbackAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of fallback Σ	models.py: 1806
SigmaTerminalAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of terminal Σ ; not exported	models.py: 1845
ProjectionRequest — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Built by the cycle for the projection service	models.py: 1592
ProjectionConsistencyAudit — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Record-immutability audit	models.py: 1614
ProjectionInputLicenseAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>validate_projection_causal_inputs</code>	models.py: 236
ProjectionModelAuthorityAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>validate_projection_model_authority</code>	models.py: 263
JointRecoveryExecutionBindingAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Binding of mandatory recovery actions (3.9)	models.py: 2218
CycleArtifactIntegrityAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>audit_cycle_artifact_integrity</code>	models.py: 1625
IntegratedCycleLifecycleIntegrityAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>audit_integrated_cycle_lifecycle_integrity</code>	models.py: 898
CrossObjectExecutionIdentityAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>audit_post_selection_identity_chain</code>	models.py: 886

Chapter 10 — Adapters and Services

The runtime owns the order of the stack, the gates, and the checks. The host owns every fact about its domain: what the state is, how pressure maps, what a policy would do, what the constraints observe, and how the world changes. This chapter documents the protocols through which the host supplies those facts. The interfaces module defines eighteen. v0.141 calls nine of them; the other nine are declared but no runtime method accepts or calls them. The chapter separates the two groups and gives each protocol a primary entry. It closes with a conformance checklist that a host author can run against an implementation.

A host author must be able to implement every protocol an application needs, know which ones v0.141 actually calls, and check the implementation against a conformance list.

10.1 The Adapter Rule

An adapter states facts and decides nothing. The application is authoritative for its domain mechanics; the framework is authoritative for the meanings those mechanics are mapped into and for the decision architecture. No runtime convenience may turn application data into framework authority, collapse actual and perceived state, treat feasibility as adequacy or authorization, or substitute application ranking for the operator sequence (C30/D4 §2A.2–2A.4; C30/D3 §6, §9). A domain that cannot be mapped without changing framework meanings is reported as a possible framework limitation, not repaired inside the adapter.

The runtime enforces part of this rule and cannot enforce the rest. On the supported service routes it generally validates what comes back, as each entry below specifies: identities match the request, times are finite and do not run backward, labels come from closed sets, records have the right type, and nothing the runtime owns has changed. Compatibility hooks and some world-adapter calls receive weaker checks or none: `project_policy_record` is checked only for its policy id (10.3), and `perceive` is not checked at all (10.2). It cannot check whether a fact is true, whether a pressure mapping is honest, whether a projection is a good model, or whether a constraint observation hides a ranking. Those are host obligations. The conformance checklist (10.10) marks which behaviors the runtime checks and which it only relies on.

The protocols are Python typing.Protocol classes and are not runtime-checkable. The runtime never tests an object with `isinstance` against them; it calls the method it needs, and for optional world hooks it first tests whether the attribute is callable. A host object therefore satisfies a protocol by having the right methods, and it may implement several protocols at once.

Table 10.1 lists all eighteen protocols by how the runtime reaches them. Four channels exist: the required world adapter; the optional services passed to the `PVPPRuntime` constructor; an adapter passed as an argument to the cycle entries; and the declared role protocols, which no runtime method reaches.

Table 10.1 — The eighteen host protocols and how v0.141 reaches them

Protocol	Channel	Called by	Label
WorldAdapter	Construct or, required	Perception, the stages Φ through Σ , and the pre-canonical paths (10.2)	PUBLIC / SUPPORTED

Protocol	Channel	Called by	Label
PolicyProjectionService	Construct or, optional	The canonical cycle, for each feasible candidate (10.3)	PUBLIC / SUPPORTED
Layer1TransitionService	Construct or, optional	apply_layer1_transition only (10.4)	PUBLIC / SUPPORTED
MemoryRetrievalService	Construct or, optional	prepare_memory_conditioned_cycle_snapshot (10.5)	PUBLIC / ADVANCED
EpistemicUpdateService	Construct or, optional	apply_post_execution_epistemic_update (10.5)	PUBLIC / ADVANCED
TransferHistoryService	Construct or, optional	record_transfer_history (10.6)	PUBLIC / ADVANCED
PredictionErrorService	Construct or, optional	apply_prediction_error (10.6)	PUBLIC / ADVANCED
CapacityAuthorityService	Construct or, optional	derive_capacity_authority, and the integrated cycle when capacity-bearing corridors are active (10.7)	PUBLIC / ADVANCED
ObjectiveResponsiveness Adapter	Cycle-entry argument	evaluate_objective_responsiveness, inside the cycle (10.8)	PUBLIC / ADVANCED
StateMeasurementAdapter	Declared only	Nothing in v0.141 (10.9)	IMPLEMENTATION / SUPPORT
StateSufficiencyAdapter	Declared only	Nothing in v0.141 (10.9)	IMPLEMENTATION / SUPPORT
ActionStructureAdapter	Declared only	Nothing in v0.141 (10.9)	IMPLEMENTATION / SUPPORT
GraphReachabilityAdapter	Declared only	Nothing in v0.141 (10.9)	IMPLEMENTATION / SUPPORT
ProjectionWorldModelAdapter	Declared only	Nothing in v0.141 (10.9)	IMPLEMENTATION / SUPPORT

Protocol	Channel	Called by	Label
ConstraintEvidenceAdapter	Declared only	Nothing in v0.141 (10.9)	IMPLEMENTATION / SUPPORT
ExecutionTransitionAdapter	Declared only	Nothing in v0.141 (10.9)	IMPLEMENTATION / SUPPORT
ObjectiveInterfaceAdapter	Declared only	Nothing in v0.141 (10.9)	IMPLEMENTATION / SUPPORT
ObjectiveDiscoveryAdapter	Declared only	Nothing in v0.141 (10.9)	IMPLEMENTATION / SUPPORT

Three of the nine called protocols—`WorldAdapter`, `MemoryRetrievalService`, and `EpistemicUpdateService`—are not exported at the top level; import them from `pvpp_runtime.interfaces` (H.5-1). Every declared-only protocol is exported. Export visibility and runtime use point in opposite directions here, which is why neither is taken as API authority (H.2).

10.2 The World Adapter

`WorldAdapter` is the only required host object. The constructor stores it without inspecting it; each method is looked up when a call needs it. Its methods fall into five groups: perception, domain values and baseline projection, strict Φ/H facts, constraint and Σ facts, and hooks that serve only older paths. Table 10.2 gives every method, when v0.141 needs it, and what the runtime checks. Methods marked optional are detected by testing whether the attribute is callable, so a host omits an optional hook simply by not defining it.

Entry — `WorldAdapter`

Field	Content
Name and label	<code>WorldAdapter</code> — protocol class; <code>interfaces.py:5</code> . PUBLIC / SUPPORTED (rule R1 (required host protocol); export gap H.5-1).
Purpose	Supplies the host's perception of actual state, domain values, projections of actions, the strict Φ/H facts, constraint observations, and the structural facts Σ needs.
Framework basis	Application/adaptor boundary (C30/D4 §2A.2–2A.4; C30/D3 §6, §9); perception (C10/D19); the stage owner files for the facts each hook supplies (Chapter 3).
Methods	Sixteen methods; see Table 10.2.
Reached through	The world argument of the <code>PVPPRuntime</code> constructor (4.1). The canonical cycle, the snapshot entries, the stage calls, and the pre-canonical evaluators call it.
Runtime checks on the answer	Per hook; see Table 10.2. The runtime never checks that a hook is free of side effects or that it reads only the state it is given.

Field	Content
Failure behavior	A missing required hook raises <code>ValueError</code> naming the hook when the call that needs it is made; a missing "steady" projection raises <code>RuntimeError("steady must be projectable")</code> .
Must not decide	Select, rank, or authorize; compute a stage the runtime owns; read memory the runtime has not licensed; or change host state. The world adapter reports; the runtime decides.
Evidence	Direct use: no direct use in tests, benchmarks, or examples; named in doc 05. Implemented by the world object of every test, benchmark, and example: perceive is defined in 26 test files, 26 benchmarks, and 12 example files.
See also	3.3–3.11; 9.6; Guide 8.3.

Table 10.2 — WorldAdapter methods in v0.141

Method	Needed by	Runtime check or use
perceived_decision_state_from_actual(state)	prepare_canonical_cycle_snapshot, preferred	Result validated as a <code>PerceivedDecisionState</code> attributed to the actual snapshot's actor, state id, and time (5.3)
represented_state_from_actual(state)	The same entry, when the typed hook is absent (compatibility)	Must return a <code>WorldState</code> , which the runtime wraps without inventing perceived SPV, PVS, or context (5.3)
perceived_decision_state_from_perception_inputs(state, retrieval, expectation)	prepare_memory_conditioned_cycle_snapshot, preferred	As above, plus the licensed retrieval and $K_i(t)$ must be the ones the runtime passed (6.2)
represented_state_from_perception_inputs(state, retrieval, expectation)	The same entry, when the typed hook is absent (compatibility)	Must return a <code>WorldState</code>

Method	Needed by	Runtime check or use
perceive(state)	Every cycle and stage call	None. The cycle applies it to the represented state even when a validated $P_i(t)$ was supplied; see the obligation below
domain_value(state, domain_id)	Φ , H , $\mathcal{R}$ inputs, and baseline drift	Converted to float
project(state, action)	Baseline continuation with "steady"; pre-canonical paths	The "steady" projection must be feasible
pressure_factors, pressure_value, expected_deterioration	Strict Φ and H ; required by the canonical cycle	The cycle refuses to run without all three (3.3); <code>validate_pressure_invariants</code> checks monotone compression on request. <code>expected_deterioration</code> is the signed change per period under the baseline continuation: negative when the domain is falling. H divides the margin by <code>max(epsilon_deterioration, -expected)</code> , so a positive return reads as improvement and yields an effectively infinite horizon without error (15.9 D9)

Method	Needed by	Runtime check or use
constraint_profile(state, policy_id, action_ids, regime, governing_ids)	Constraints in the canonical cycle; required	Must return PolicyConstraintProfile for the same policy id; unknown and duplicate rule observations fail closed (3.7)
project_policy_record(state, policy_id, action_ids)	$Q_t(\pi)$ when no projection service is attached (compatibility)	Policy id must match; an infeasible record for a constraint-feasible policy fails closed. No licensing or record validation (10.3)
fallback_structure_profile(state, policy_id, action_ids, governing_ids)	Fallback Σ , optional	Profiles feed evaluate_sigma_fallback; absent profiles leave structural comparison empty (3.11)
compare_constraint_violation_severity(a, profile_a, b, profile_b, class)	Terminal Σ ; required when terminal Σ must compare	Must return -1, 0, 1, or None, antisymmetric, and symmetric about incomparability
project_policy(state, action_ids)	The pre-canonical evaluators only (9.13)	None; without it they fall back to sequential project calls
execute(state, action_id)	Nothing in v0.141	Declared but never called. Execution runs through ε , native authorization, and the Layer-1 transition service (Chapter 11)

Three findings follow from the table. First, the canonical cycle calls perceive on the represented state even when the snapshot entry has already built and validated $P_i(t)$ from it. A perceive that alters an already-represented state makes Φ through Σ read a state other than the validated one, and the runtime does not detect the substitution. Every example in the tree implements perceive

as the identity. Second, execute is part of the protocol but is never called, so a host may leave it unimplemented; the adapter-configuration tests use a world object that defines execute and nothing else, and v0.141 never touches it. Third, the two compatibility perception hooks and `project_policy_record` are retained for older integrations. New integrations should implement the typed perception hooks and attach a `PolicyProjectionService`.

Requirement for v0.141 hosts: perceive must return its argument unchanged when given a `WorldState` that the host's own perception hooks produced. This is a conformance requirement, not a style preference (Table 10.5; Chapter 15).

10.3 The Projection Service

$Q_t(\pi)$ is the one authoritative projection record per constraint-feasible candidate, shared unchanged by Adequacy and Σ (C10/D6; C15/D6 §3.3). v0.141 has two ways to obtain it. The canonical route is the `PolicyProjectionService`, which receives a typed `ProjectionRequest`; the compatibility route is the world adapter's `project_policy_record` hook, which the runtime's own docstring describes as retained for historical integrations. The two routes are not checked equally.

New integrations should not use `WorldAdapter.project_policy_record`. It is a compatibility path and does not receive the finite-horizon, projection-authority, causal-input-licensing, and record-validation enforcement applied to `PolicyProjectionService`.

Entry — `PolicyProjectionService`

Field	Content
Name and label	<code>PolicyProjectionService</code> — protocol class; <code>interfaces.py:70</code> . PUBLIC / SUPPORTED (rule R1 (documented route for $Q_t(\pi)$ in the ordinary cycle)).
Purpose	Returns the authoritative policy-conditioned projection record $Q_t(\pi)$ for one candidate.
Framework basis	Shared projection service (C10/D6); a service, not a decision operator.
Methods	<code>project(request: ProjectionRequest) → PolicyProjectionRecord</code> .
Reached through	The <code>projection_service</code> argument of the constructor. The canonical cycle calls it once for each constraint-feasible candidate, in ordinary-adequacy mode, after Constraints and the projection licenses.
Runtime checks on the answer	<code>validate_projection_record</code> runs on every record: identity, candidate mode, the finite horizon L_P recorded exactly, censoring only for known domains, at most one corridor fact per domain and function, typed quality claims, and model identity and inputs matching any licenses (9.10). The cycle then verifies that Adequacy and Σ left every record unchanged.
Failure behavior	The request requires an externally supplied finite L_P ; without one the call raises <code>ValueError</code> . A failed model authority or causal-input license stops the cycle before any request is built. An invalid record raises <code>ValueError</code> .
Must not decide	Rank, select, classify feasibility or adequacy, or keep state between calls. The service is stateless and non-selective.

Field	Content
Evidence	Direct use: no direct use in tests, benchmarks, or examples; named in doc 05. Implemented in test_projection_service_v036, whose builder six other test files and three benchmarks reuse, and in the minimal_battery_service example.
See also	9.10.

Table 10.3 — Checks on the two projection routes

Check	PolicyProjectionService	project_policy_record hook
Finite L_P required	Yes	No
Projection model authority must be valid	Yes; failure stops the cycle	Assessed but not enforced
Required causal inputs must be licensed	Yes; failure stops the cycle	Assessed but not enforced
Record validated against its request	Yes (validate_projection_record)	Policy id only
Infeasible record for a feasible policy	Carried to Adequacy	Fails closed as a contradiction of Constraints
Record unchanged by Adequacy and Σ	Yes	Yes

A host that relies on the compatibility hook therefore runs without the projection licensing v0.141 added: the cycle computes the model-authority and input-license assessments but acts on them only when a service is attached. This is recorded among the compatibility paths of Chapter 15.

10.4 The Layer-1 Transition Service

Entry — Layer1TransitionService

Field	Content
Name and label	Layer1TransitionService — protocol class; interfaces.py:75. PUBLIC / SUPPORTED (rule R1 (host protocol the supported integrated entry needs to execute)).
Purpose	Applies the host's transition T_i to actual persistent state after ε has produced a terminal handoff.
Framework basis	Layer 1 (C10/D2); ε ends at realization outputs (C15/D9).
Methods	transition(current_state: ActualPersistentStateEnvelope, handoff: Layer1TransitionHandoff) → Layer1TransitionResult.
Reached through	The layer1_transition_service argument of the constructor. Only apply_layer1_transition calls it, directly or when the integrated cycle runs with execute=True. The decision cycle and ε never call it.

Field	Content
Runtime checks on the answer	Before the call: the current state is valid and the handoff names an episode and a selected policy. After: the result names the same episode, policy, and prior state; transition_applied is true; the next state is valid, keeps the actor, has a new state id, and does not move time backward; and every host invariant check is true.
Failure behavior	Raises ValueError("no Layer1TransitionService is attached") when absent, and ValueError on any violation. The runtime never mutates state itself.
Must not decide	Re-run the decision cycle, derive PP, SPV, AVS, or context semantics for the runtime, or treat the handoff as authority beyond the licensed actions.
Evidence	Direct use: test files 2, example files 1; named in doc 05; e.g. test_minimal_battery_service.py. Supplied as layer1_transition_service in 6 test files, 6 benchmarks, and 2 example files.
See also	3.12; Chapter 11.

10.5 Memory Retrieval and Epistemic Update Services

These two services bracket the memory-conditioned path. The runtime never sees retained memory $M_i(t)$: it passes a retrieval request to the host and validates the package that comes back, and after execution it passes the realized outcome to the host and validates the update. Both are documented functionally in Chapter 6.

Entry — MemoryRetrievalService

Field	Content
Name and label	MemoryRetrievalService — protocol class; interfaces.py:85. PUBLIC / ADVANCED (rule R2 (optional host service)).
Purpose	Reconstructs the licensed retrieval $R^*M_i(t,q)$ from retained memory for one cue and context.
Framework basis	Memory interface and licensed retrieval (C10/D16).
Methods	retrieve(request: MemoryRetrievalRequest) → MemoryRetrievalPackage.
Reached through	The memory_retrieval_service argument of the constructor; called by prepare_memory_conditioned_cycle_snapshot after the memory state and any $K_i(t)$ are checked for time.
Runtime checks on the answer	validate_memory_retrieval_package: identity and actor match the request, retrieval time equals the cycle time, and quality metadata is present (6.2). Only then is the perception hook called.
Failure behavior	Raises ValueError("no MemoryRetrievalService is attached") when absent; ValueError on an invalid package, before perception runs.
Must not decide	Pass raw memory into perception, turn quality metadata into a truth score, or update or persist memory.
Evidence	Direct use: no direct use in tests, benchmarks, or examples; not named in doc 05. Supplied as memory_retrieval_service in 3 test files and 2 benchmarks.

Field	Content
See also	6.2; 9.5.

Entry — EpistemicUpdateService

Field	Content
Name and label	EpistemicUpdateService — protocol class; interfaces.py:103. PUBLIC / ADVANCED (rule R2 (optional host service)).
Purpose	Updates retained memory, the expectation state $K_i(t)$, both, or neither from a realized outcome.
Framework basis	Memory and expectation updating (C10/D16); prediction error (C10/D1 §4).
Methods	update(request: PostExecutionEpistemicUpdateRequest) → PostExecutionEpistemicUpdateResult.
Reached through	The epistemic_update_service argument of the constructor; called by apply_post_execution_epistemic_update, directly or from the memory-conditioned integrated cycle with update_epistemic_state=True.
Runtime checks on the answer	validate_post_execution_epistemic_update_result: attribution to the episode and actor, a valid Layer-1 transition first, a new identity and the next-state time for any updated memory, and no silent rewrite when an update flag is false (6.6).
Failure behavior	Raises ValueError when absent—learning is never implicit—and ValueError on an invalid result.
Must not decide	Define a learning formula, treat execution information as a state update, or rewrite memory it reports as unchanged.
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_memory_conditioned_sequence_v045.py, test_post_execution_epistemic_update_v044.py. Supplied as epistemic_update_service in 3 test files and 2 benchmarks.
See also	6.6.

10.6 Transfer-History and Prediction-Error Services

Entry — TransferHistoryService

Field	Content
Name and label	TransferHistoryService — protocol class; interfaces.py:94. PUBLIC / ADVANCED (rule R2 (optional host service)).
Purpose	Persists transfer-history events and returns the resulting history package.
Framework basis	Transfer history (C10/D22).
Methods	record(prior_history: TransferHistoryReference None, events: tuple[TransferHistoryEvent, ...]) → TransferHistoryPackage.
Reached through	The transfer_history_service argument of the constructor; called only by record_transfer_history.

Field	Content
Runtime checks on the answer	Before the call: at least one event, each valid and attributed to one actor, and a prior history for the same actor. After: <code>validate_transfer_history_package</code> —unique event ids, preserved sequence order, time that does not run backward, and a prior identity matching the one supplied (6.3).
Failure behavior	Raises <code>RuntimeError("transfer_history_service required")</code> when absent; <code>ValueError</code> on invalid events or package.
Must not decide	Turn event existence into memory, current PP, current state, or execution authority.
Evidence	Direct use: no direct use in tests, benchmarks, or examples; named in doc 05. Supplied as <code>transfer_history_service</code> in 1 test file.
See also	6.3.

Entry — `PredictionErrorService`

Field	Content
Name and label	<code>PredictionErrorService</code> — protocol class; <code>interfaces.py:123</code> . PUBLIC / ADVANCED (rule R2 (optional host service)).
Purpose	Classifies the typed mismatch between an expected or projected stage and the realized outcome.
Framework basis	Prediction error (C10/D1 §4; C10/D16).
Methods	<code>evaluate(request: PredictionErrorRequest) → PredictionErrorAssessment</code> .
Reached through	The <code>prediction_error_service</code> argument of the constructor; called by <code>apply_prediction_error</code> , and by the memory-conditioned integrated cycle before the $K_i(t)$ update when the service is attached.
Runtime checks on the answer	<code>validate_prediction_error_assessment</code> : type and attribution, permitted error classes only, the material flag consistent with the typed components, and the right projection stage (6.5).
Failure behavior	Raises <code>ValueError</code> when absent—no prediction error is computed implicitly—and <code>ValueError</code> on an invalid assessment.
Must not decide	Impose a scalar surprise score or a learning formula.
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_prediction_error_boundary_v052.py</code> . Supplied as <code>prediction_error_service</code> in 1 test file and 1 benchmark.
See also	6.5.

10.7 The Capacity Authority Service

Entry — CapacityAuthorityService

Field	Content
Name and label	CapacityAuthorityService — protocol class; interfaces.py:112. PUBLIC / ADVANCED (rule R2 (optional host service)).
Purpose	Derives the represented shared capacity calendar from actual Layer-1 state.
Framework basis	Capacity authority and joint feasibility of active recovery obligations (C10/D20; C10/D10).
Methods	derive(actual_state: ActualPersistentStateEnvelope, request: CapacityAuthorityRequest) → CapacityAuthorityEnvelope.
Reached through	The capacity_authority_service argument of the constructor; called by derive_capacity_authority. The integrated cycle calls that method only when the request carries no capacity calendar and an active recovery corridor has capacity-bearing actions (5.7).
Runtime checks on the answer	validate_capacity_authority_envelope: identity and attribution to the actual state, and finite, non-negative capacity in a well-formed calendar.
Failure behavior	Raises ValueError when absent on a direct call; the integrated cycle instead proceeds without derived capacity, and joint feasibility fails closed as before. An explicit calendar in the request is never overridden.
Must not decide	Let the runtime open actual state to interpret people, skills, commitments, or resources; the host service owns those semantics.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_persistent_state_capacity_authority_v051.py. Supplied as capacity_authority_service in 1 test file and 1 benchmark.
See also	5.7–5.8; H.5–3.

10.8 The Objective Responsiveness Adapter

Entry — ObjectiveResponsivenessAdapter

Field	Content
Name and label	ObjectiveResponsivenessAdapter — protocol class; interfaces.py:41. PUBLIC / ADVANCED (rule R2 (optional host service)).
Purpose	Supplies the application's descriptive relation $\text{Resp}(\pi, o)$ for one candidate and one objective.
Framework basis	Objective responsiveness (C10/D9 §7).
Methods	objective_responsiveness(state, policy_id, action_ids, objective) → ObjectiveResponsivenessRecord.
Reached through	Not the constructor: the objective_responsiveness_adapter argument of the cycle entries. The cycle calls it for every candidate–objective pair after Π completeness.

Field	Content
Runtime checks on the answer	Each record has the right type, names the right policy and objective, and carries one of supports, neutral, conflicts, or unresolved (7.4).
Failure behavior	Invalid records raise ValueError inside the cycle. With no adapter, relations stay unresolved and nothing is fabricated.
Must not decide	Prune, rank, authorize, or select. No later stage reads the records.
Evidence	Direct use: no direct use in tests, benchmarks, or examples; named in doc 05. No test passes the adapter by name; the objective-responsiveness tests call evaluate_objective_responsiveness with a host object directly.
See also	7.4.

10.9 Declared Protocols the Runtime Does Not Call

Nine protocols define host roles that v0.141 does not use. Most date from the v0.74 portable-adapter separation, which defined role boundaries before the runtime had paths for them; three were added with Version 2.1 objectives and state sufficiency. No runtime method accepts an object of these types, calls their methods, or checks them. Where the same fact is used, the host supplies it by another route, shown in Table 10.4. Four of them have exactly the method signature of a protocol v0.141 does call, so an object written to the declared protocol also satisfies the called one.

Table 10.4 — Declared-only protocols and the route v0.141 actually uses

Protocol and method	Fact it would supply	Route in v0.141
StateMeasurementAdapter: represented_state_from_actual, perceived_decision_state_from_act ual	Perception of actual state	The same two methods on WorldAdapter (10.2)
StateSufficiencyAdapter: transition_relevant_state_basis(stat e, purpose, context_id)	A declared transition-relevant state basis	The host builds the basis and calls validate_transition_relevant_state_ba sis (5.9)
ActionStructureAdapter: action_definitions(state)	Reachable action definitions	The host registers actions with PVPPRegistry.register_action (4.4)
GraphReachabilityAdapter: graph_reachability_observations(st ate)	Current reachability of registered transformations	The host passes observations to PVPPRegistry.refresh_graph_reacha bility (Chapter 8)
ProjectionWorldModelAdapter: project(request)	$Q_t(\pi)$	PolicyProjectionService, identical signature (10.3)
ConstraintEvidenceAdapter: constraint_profile(...)	Constraint observations	WorldAdapter.constraint_profile, identical signature (10.2)

Protocol and method	Fact it would supply	Route in v0.141
ExecutionTransitionAdapter: transition(current_state, handoff)	The Layer-1 transition	Layer1TransitionService, identical signature (10.4)
ObjectiveInterfaceAdapter: objective_set(state)	O_i(t)	The host passes an ObjectiveSet to the cycle entry (7.2)
ObjectiveDiscoveryAdapter: objective_discovery_hints(state, objectives)	Objective discovery hints	The host passes hints to the cycle entry (7.5)

Five of these correspond to the adapter roles that `validate_runtime_configuration_provenance` accepts in an `AdapterIdentity`: `state_measurement`, `action_structure`, `projection_world_model`, `constraint_evidence`, and `execution_transition` (4.9). A host may use the declared protocols to organize its own code and to name those roles in its configuration record. Doing so documents the separation; it does not change what the runtime calls. `ExecutionTransitionAdapter`'s docstring still says that native call-through execution "remains a later phase". Native invocation has existed since v0.76 (Chapter 11), so that sentence is out of date; its other statement, that the protocol confers no invocation authority, still holds. The discrepancy is recorded for Chapter 15.

10.10 Conformance Checklist for Host Implementations

Table 10.5 lists the behaviors a host implementation must have. "Checked" means v0.141 rejects a violation and the named test demonstrates it. "Relied on" means the runtime cannot detect a violation; the host must guarantee the behavior and should test it in its own suite. Test names are from the v0.141 test tree.

Table 10.5 — Host conformance checklist

Behavior	Status	Demonstrating test
Typed perception hook is used when present; legacy hook only as fallback	Checked	test_perceived_decision_state_boundary_v056: test_integrated_cycle_prefers_typed_pds_hook_over_legacy_worldstate_hook
P_i(t) is attributed to the actual snapshot's actor, state id, and time	Checked	test_perceived_decision_state_boundary_v056: test_typed_pds_identity_and_time_are_attributed_to_actual_snapshot

Behavior	Status	Demonstrating test
The runtime does not derive perceived components from actual PP, SPV, AVS, or X	Checked	test_perceived_decision_state_boundary_v056: test_runtime_does_not_derive_perceived_components_from_actual_pp_spv_avs_x
perceive returns an already-represented state unchanged (required)	Relied on	None in the tree; add a host test
domain_value is deterministic for one state within a cycle	Relied on	None in the tree; add a host test
The "steady" action is projectable	Checked	None named; a non-feasible projection raises RuntimeError in every canonical-cycle run
Strict Φ: margin equals the $P_i(t)$ distance to threshold; compression is monotone	Checked on request	test_phi_h_strict_v028: test_phi_margin_must_match_ppp_distance_to_threshold, test_phi_monotonic_compression_validator_rejects_reversed_mapping
Constraint profile names the right policy; rule observations are known and unique	Checked	test_constraint_system_v031: test_profile_policy_mismatch_fails_closed, test_unknown_and_duplicate_rule_observations_fail_closed

Behavior	Status	Demonstrating test
Constraint observation reports facts, not a ranking	Relied on	test_constraint_system_v031 shows the runtime never counts violations as a scalar; honesty of the observations is the host's
Projection service receives a finite L_P and returns a record valid for its request	Checked	test_projection_service_v036: test_service_requires_explicit_finite_projection_horizon, test_record_rejects_exact_horizon_beyond_LP, test_record_rejects_candidate_mode_mismatch, test_duplicate_corridor_fact_fails_closed
A compatibility Q record does not contradict Constraints	Checked	test_canonical_decision_cycle_v032: test_q_contradiction_with_constraint_feasibility_fails_closed
Projection records describe a credible causal model	Relied on	None; licensing (9.10) records model identity but cannot judge it
Terminal comparator returns -1, 0, 1, or None, antisymmetric and symmetric about incomparability	Checked	test_sigma_terminal_v035: test_bad_comparator_contract_is_rejected, test_missing_comparator_fails_closed
Fallback structural profiles are supplied where Σ must compare	Checked	test_sigma_fallback_v034: test_multiway_stage2_composition_fails_closed_instead_of_inventing_tournament_rule

Behavior	Status	Demonstrating test
Layer-1 result keeps actor, gives a new state id, does not reverse time, passes host invariants	Checked	test_layer1_actual_state_boundary_v040: test_transition_result_cannot_change_actor_identity, test_transition_result_requires_new_state_identity, test_transition_time_may_not_move_backward, test_failed_host_invariant_check_blocks_transition_result
The transition service is called only through the explicit bridge	Checked	test_layer1_actual_state_boundary_v040: test_explicit_layer1_transition_service_is_called_only_by_explicit_bridge, test_epsilon_and_decision_methods_do_not_implicitly_call_layer1_service
Retrieval package matches its request, carries quality metadata, and aligns to the cycle time	Checked	test_memory_retrieval_boundary_v043: test_retrieval_identity_mismatch_fails_before_perception, test_retrieval_must_carry_quality_metadata, test_retrieval_time_must_align_to_current_cycle
Epistemic update is attributed, gives updated memory a new identity, and hides no rewrite	Checked	test_post_execution_epistemic_update_v044: test_episode_or_actor_mismatch_is_rejected, test_updated_memory_requires_new_identity_and_next_state_time, test_silent_memory_rewrite_with_update_flag_false_is_rejected

Behavior	Status	Demonstrating test
Transfer-history package keeps unique ids, order, forward time, and prior identity	Checked	test_v2_1_transfer_history_v0136: test_package_requires_unique_event_ids, test_package_preserves_sequence_order, test_package_time_cannot_move_backward, test_package_prior_identity_must_match_supplied_prior
Prediction error uses permitted classes and a consistent material flag	Checked	test_prediction_error_boundary_v052: test_unsupported_generic_error_class_fails_closed, test_material_flag_must_match_typed_components, test_policy_projection_stage_mismatch_is_rejected
Capacity envelope matches the state and has finite, non-negative capacity	Checked	test_persistent_state_capacity_authority_v051: test_capacity_envelope_identity_mismatch_is_rejected, test_negative_or_nonfinite_derived_capacity_is_rejected, test_explicit_request_calendar_is_not_overridden_by_derivation_service
Capacity derivation reflects the host's real commitments and people	Relied on	test_persistent_state_capacity_authority_v051 illustrates the host doing so; the runtime cannot check it
Resp records cover every pair, use closed labels, and name the right ids	Checked	test_v2_1_objective_responsiveness_discovery_v0134: test_resp_emits_every_candidate_objective_pair_without_pruning, test_resp_labels_are_closed_and_application_supplied, test_resp_identity_mismatch_is_rejected
No hook changes host state or keeps hidden state between calls	Relied on	None; the runtime checks only that its own Q records are unchanged

10.11 Classification Record

WorldAdapter keeps the label assigned in Chapter 4. PolicyProjectionService is PUBLIC / SUPPORTED: the canonical cycle needs a projection route, the service is the documented one, and the only alternative is a compatibility hook. Layer1TransitionService is PUBLIC / SUPPORTED because the supported integrated entry needs it to execute, and the minimal example uses it. CapacityAuthorityService joins the other optional services as PUBLIC / ADVANCED; its three flow methods keep their default label (H.5-3). The nine declared-only protocols take the default label IMPLEMENTATION / SUPPORT and are entered in the register as H.5-6: they are exported, but no runtime method accepts or calls them, no test, benchmark, or example implements them by name, and doc 05 does not name them. Chapter 15 decides whether any should be withdrawn, kept as reserved roles, or wired in by a successor build.

Table 10.6 — Labels assigned in Chapter 10

Symbol	Label	Rule
PolicyProjectionService	PUBLIC / SUPPORTED	R1 (documented route for $Q_t(\pi)$ in the ordinary cycle)
Layer1TransitionService	PUBLIC / SUPPORTED	R1 (host protocol the supported integrated entry needs to execute)
CapacityAuthorityService	PUBLIC / ADVANCED	R2 (optional host service)
StateMeasurementAdapter	IMPLEMENTATION / SUPPORT	Default (H.5-6)
ActionStructureAdapter	IMPLEMENTATION / SUPPORT	Default (H.5-6)
ProjectionWorldModelAdapter	IMPLEMENTATION / SUPPORT	Default (H.5-6)
ConstraintEvidenceAdapter	IMPLEMENTATION / SUPPORT	Default (H.5-6)
ExecutionTransitionAdapter	IMPLEMENTATION / SUPPORT	Default (H.5-6)
GraphReachabilityAdapter	IMPLEMENTATION / SUPPORT	Default (H.5-6)
StateSufficiencyAdapter	IMPLEMENTATION / SUPPORT	Default (H.5-6)
ObjectiveInterfaceAdapter	IMPLEMENTATION / SUPPORT	Default (H.5-6)
ObjectiveDiscoveryAdapter	IMPLEMENTATION / SUPPORT	Default (H.5-6)

Chapter 11 — Execution Lineage

A selection is not an action. Between Σ and a changed world, v0.141 requires a chain of objects, each issued by the runtime from the one before it: an execution license from a completed cycle, an ϵ episode from the license, observations that advance the episode to a terminal status, a handoff from that status, and a host transition validated against the handoff. Hosts that execute through registered native callables add a binding registry, a single-use authorization, and an invocation whose outcome is fed back into ϵ . This chapter documents every step, what each establishes, and what it leaves to the host.

A reader must be able to carry a selection through to a validated transition, and know at each step what is established and what is not.

11.1 The Lineage in One Table

Two routes run from a license to a transition. On the observation route, the host performs the action by its own means and reports what happened to ϵ as observations; the `minimal_battery_service` example and the integrated cycle with `execute=True` use this route. On the native route, the host registers a callable for each action, the runtime issues a single-use authorization, and the runtime enters the callable; the native result is then turned into an observation, so both routes pass through the same ϵ trace. Table 11.1 lists the steps.

Table 11.1 — The execution lineage

Step	Call	Object produced	Establis hes	Does not establish
License	<code>build_execution_license_from_cycle</code> (11.2)	<code>ExecutionLicenseEnv</code> <code>elope</code>	The selected policy, its actions, the governing targets, and the Σ mode	That entry is sufficient now
Instantiation	<code>instantiate_execution</code> (11.3)	<code>ExecutionEpisode</code> in an <code>EpsilonStepResult</code>	An active, bounded ϵ episode, or a blocked one	That anything has happened

Step	Call	Object produced	Establishes	Does not establish
Observation	advance_execution (11.4)	EpsilonStepResult	The ϵ status and trace	A state change
Binding (native)	ExecutionBindingRegistry.register (11.5)	ExecutionBindingAssessment	Which callable implements an action	Any authority to call it
Authorization (native)	issue_native_execution_authorization (11.6)	NativeExecutionAuthorization	One permission to enter one binding once	That the call will succeed
Invocation (native)	invoke_authorized_native (11.7)	NativeExecutionResult	The normalized technical outcome	Valid realization; a transition
Bridge (native)	ExecutionBindingRegistry.epsilon_observation (11.8)	ExecutionObservation	The native outcome as ϵ evidence	An ϵ status until advance_execution runs
Handoff	build_layer1_transition_handoff (11.10)	Layer1TransitionHandoff	The terminal ϵ outputs Layer 1 may consume	Next state
Transition	apply_layer1_transition (11.10)	Layer1TransitionResult	A validated host transition	Recovery bookkeeping or memory update

Step	Call	Object produced	Establishes	Does not establish
Provenance	build_execution_transition_provenance (11.11)	ExecutionTransitionProvenance	An audit record linking prior state, ε , and next state	Any decision or execution authority

Authority along this chain is carried by object identity, not by field values. The runtime keeps private ledgers of the licenses, episodes, and native authorizations it issued, keyed by the objects themselves. A copy made with `dataclasses.replace` compares equal to the original and is refused, and so is a record built by hand with the right fields. The ledgers live in memory in one `PVPPRuntime` instance: they are not persisted, and objects issued by one runtime carry no authority in another.

11.2 Execution License

The license is ε 's upstream envelope (C15/D9 §6, §10B). It records the selected policy and preserves the Σ mode rather than forcing every selection through ordinary adequacy. Table 11.2 gives the three modes.

Table 11.2 — License modes

Σ mode	Required of the cycle	License flags	Entry
standard	Selected policy constraint-feasible, framing valid, and in Π -adequate	<code>pi_complete</code> , <code>constraints_feasible</code> , <code>framing_valid</code> , <code>adequacy_sufficient</code> all true	Authorized
recovery_unavailable_fallback	Selected policy constraint-feasible, framing valid, and Π -adequate empty	<code>adequacy_sufficient</code> false, others true	Authorized by the fallback selection, not by adequacy
terminal_infeasibility	Π -valid empty; Σ selected from the infeasible set	Only <code>pi_complete</code> true; no framed targets	Blocked; the selection is kept for traceability

Entry — `PVPPRuntime.build_execution_license_from_cycle`

Field	Content
Name and label	<code>PVPPRuntime.build_execution_license_from_cycle</code> — method; <code>kernel.py:4150</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Builds ε 's licensing envelope from one completed canonical cycle.

Field	Content
Framework basis	Realization Operator ε v1.7 (C15/D9) §6, §10B; Σ modes (C15/D6; C15/D17).
Signature	PVPPRuntime.build_execution_license_from_cycle(cycle: CanonicalDecisionCycleAssessment, frame: DomainFrame) → ExecutionLicenseEnvelope
Preconditions	The cycle stopped at "Sigma" with a selected policy, a generated policy space, passed Π completeness, and a governing assessment. The frame names every governing domain. A fresh artifact-integrity audit (9.12) of the cycle passes.
Result	ExecutionLicenseEnvelope with selection_mode, entry_authorized, the selected candidate's actions, and any mandatory recovery actions bound in by the joint-recovery check (3.9). The license is entered in the runtime's license ledger.
Invariants	The mode is read from the selection: terminal if terminal Σ ran, fallback if fallback Σ ran, standard otherwise. A standard license requires the selection to be adequate; a fallback license requires $\Pi_{adequate}$ to be empty.
Failure behavior	Raises ValueError for a cycle not resolved by Σ , a stale or identity-inconsistent cycle, a selection missing from the policy space, missing framing targets, or a mode whose preconditions fail.
Does not	Decide entry sufficiency, instantiate ε , or limit how many licenses one cycle yields.
Evidence	Direct use: test files 4, benchmarks 1, example files 3; named in doc 05; e.g. authority_bound_callthrough_v0130.py, minimal_battery_service.py.
See also	3.12; 9.3; Guide 7.

Nothing in v0.141 stops the same cycle from yielding several licenses: each call returns a new license object with its own ledger entry, and each can be instantiated and executed. Authority is single-use per license and per native authorization, not per decision. Executing a selection at most once is a host obligation (Chapter 15). The older build_execution_license, which licenses from a pre-canonical selection result, is LEGACY / COMPATIBILITY and documented in Chapter 15.

In v0.141, single-use execution is enforced on issued authority objects, not on the underlying decision: execution authority is object-lineage authority, not decision-lineage authority. The host must enforce one execution per selected decision, unique episode ids, advancement of only the latest episode, and finality of the first terminal result (11.3–11.4; 15.3).

11.3 Instantiation

Entry — `PVPPRuntime.instantiate_execution`

Field	Content
Name and label	<code>PVPPRuntime.instantiate_execution</code> — method; <code>kernel.py:4382</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Instantiates the selected policy as a bounded ε episode $x_{\text{exec}}(t)$.
Framework basis	Realization Operator ε v1.7 (C15/D9) §5–6: ε instantiates $\pi_{\text{sel}}(t)$ and tracks entry sufficiency.
Signature	<code>PVPPRuntime.instantiate_execution(episode_id: str, license: ExecutionLicenseEnvelope, *, entry_sufficient: bool, max_steps: int) → EpsilonStepResult</code>
Preconditions	A license issued by this runtime and not yet instantiated; a non-empty episode id; <code>max_steps</code> at least 1. <code>entry_sufficient</code> is the host's judgment that entry is sufficient under current conditions (C15/D9 §6.1).
Result	<code>EpsilonStepResult</code> . Status "active" with an active episode when the license authorizes entry and entry is sufficient; otherwise status "aborted_return", terminal, with an inactive episode and nothing realized.
Invariants	A license can be instantiated once: its ledger entry moves from "issued" to "instantiated". The episode is recorded as current when active and inactive otherwise. The license flags are re-checked for the mode (Table 11.2), so a hand-edited flag cannot launder a blocked license.
Failure behavior	Raises <code>ValueError</code> for an unknown, copied, or already instantiated license, an empty episode id, <code>max_steps</code> below 1, or an unknown mode.
Does not	Judge entry sufficiency—the host asserts it—or enforce unique episode ids. Reusing an episode id replaces the earlier episode's ledger entry, which then no longer authorizes native execution.
Evidence	Direct use: test files 9, benchmarks 1, example files 3; named in doc 05; e.g. <code>authority_bound_callthrough_v0130.py</code> , <code>minimal_battery_service.py</code> .
See also	9.3 (<code>execute=True</code> asserts entry sufficiency); Guide 7.4.

Record — `ExecutionEpisode`

Field	Content
Label	IMPLEMENTATION / SUPPORT (rule R3: carried in <code>EpsilonStepResult</code>)
Fields	frozen dataclass; <code>episode_id: str</code> ; <code>license: ExecutionLicenseEnvelope</code> ; <code>max_steps: int</code> ; <code>step_count: int = 0</code> ; <code>entry_sufficient: bool = True</code> ; <code>trace_tail: ExecutionTraceNode None = None</code> ; <code>active: bool = True</code>
Source	<code>models.py:2040</code>
Evidence	Direct use: test files 5, benchmarks 4; named in doc 05; e.g. <code>benchmark_layer1_handoff_v033.py</code> , <code>benchmark_post_execution_bookkeeping_v038.py</code> .

11.4 ϵ Observation and Advancement

ϵ advances one observation at a time. Each observation is the host's report of what execution revealed; ϵ classifies it and never selects, reframes, or re-projects (C15/D9 §8).

EpsilonStepResult carries the framework's realization outputs: the episode is $x_exec(t)$ and its bounded state, $execution_path$ is $ExecPath(t)$, $status$ is $status_exec(t)$, $realized_pv_bundles$ is $B_real(t)$, and $information_events$ is $info_exec(t)$ (C15/D9 §10). Path, bundles, and information are materialized only when the result is terminal.

Record — ExecutionObservation

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: built by the host for <code>advance_execution</code>)
Fields	frozen dataclass; <code>event_id</code> : str; <code>continuation_sufficient</code> : bool = True; <code>completion_sufficient</code> : bool = True; <code>completed</code> : bool = False; <code>staged</code> : bool = False; <code>failed</code> : bool = False; <code>emergency</code> : bool = False; <code>material_policy_class_change_required</code> : bool = False; <code>realized_pv_bundle</code> : Mapping[str, Any] = (empty); <code>information</code> : Mapping[str, Any] = (empty)
Source	models.py:2016
Evidence	Direct use: test files 10, benchmarks 7, example files 2; named in doc 05; e.g. <code>minimal_battery_service.py</code> , <code>test_minimal_battery_service.py</code> .

Entry — PVPPRuntime.advance_execution

Field	Content
Name and label	PVPPRuntime.advance_execution — method; kernel.py:4432. PUBLIC / SUPPORTED (rule R1).
Purpose	Advances an active ϵ episode by one observation.
Framework basis	Realization Operator ϵ v1.7 (C15/D9) §6–9.
Signature	PVPPRuntime.advance_execution(episode: ExecutionEpisode, observation: ExecutionObservation) → EpsilonStepResult
Preconditions	An active episode and an observation with an event id.
Result	EpsilonStepResult with the next episode, the status, terminal and return_upstream flags, and—when terminal—the path, realized bundles, and information events.
Invariants	The observation is classified in the fixed order of Table 11.3; the first matching row decides. When the episode becomes terminal, every native authorization still issued for it is invalidated.
Failure behavior	Raises ValueError for an inactive episode or an observation without an event id.
Does not	Invoke Σ , Π , Domain Framing, Adequacy, or the Layer-1 transition; call WorldAdapter.execute; or re-project.

Field	Content
Host obligation	Advance only the episode returned by the latest call. <code>advance_execution</code> checks that the episode is active and that its license has a current lineage; it does not check that the object is the latest one recorded. Advancing an earlier episode object again produces a second, divergent trace, and an episode object that was still active can be advanced to a second terminal result after the ledger has marked the episode terminal. Treat the first terminal result as final.
Evidence	Direct use: test files 6, benchmarks 1, example files 2; named in doc 05; e.g. <code>minimal_battery_service.py</code> , <code>test_minimal_battery_service.py</code> .
See also	Guide 7.

Table 11.3 — How `advance_execution` classifies an observation

Order	Observation	Status	Return upstream
1	emergency	<code>emergency_return</code>	Yes
2	failed	<code>failed</code>	Yes
3	<code>material_policy_class_change_required</code>	<code>partial_realization</code> if a bundle was realized, else <code>aborted_return</code>	Yes
4	<code>continuation_sufficient</code> is false	<code>partial_realization</code> if a bundle was realized, else <code>aborted_return</code>	Yes
5	<code>completed</code> , <code>completion_sufficient</code>	<code>completed_staged</code> if staged, else <code>completed</code>	No
6	<code>completed</code> , <code>completion</code> not sufficient	<code>partial_realization</code> if a bundle was realized, else <code>aborted_return</code>	Yes
7	step limit reached	<code>partial_realization</code> if a bundle was realized, else <code>aborted_return</code>	Yes
8	staged, not completed	active (staging is not a new selection)	—
9	otherwise	active	—

Table 11.4 — ϵ statuses

Status	Meaning (C15/D9 §9)	Test
completed	The selected policy completed as licensed	test_epsilon_bounded_realization_v022: test_completion_status_and_realized_outputs_are_auditable_and_distinct
completed_staged	Completed through a licensed staged path	test_epsilon_bounded_realization_v022: test_staged_completion_is_status_not_new_selection
partial_realization	Some PV realized without full completion; returns upstream	test_epsilon_bounded_realization_v022: test_continuation_failure_with_partial_bundle_reports_partial_realization
aborted_return	Terminated with nothing realized; returns upstream, including blocked or insufficient entry	test_epsilon_bounded_realization_v022: test_entry_failure_returns_upstream_without_execution, test_bounded_step_limit_forces_return
failed	The licensed path was not realized	test_epsilon_bounded_realization_v022: test_emergency_and_failed_statuses_are_distinct
emergency_return	Execution exposed immediate governing risk	As above

Status	Meaning (C15/D9 §9)	Test
active	Not terminal; the runtime's working status, not a framework status class	—

Record — EpsilonStepResult

Field	Content
Label	IMPLEMENTATION / SUPPORT (rule R3: returned by <code>instantiate_execution</code> and <code>advance_execution</code>)
Fields	frozen dataclass; episode: <code>ExecutionEpisode</code> ; status: <code>str</code> ; terminal: <code>bool</code> ; return_upstream: <code>bool</code> ; realized_pv_bundles: <code>tuple[Mapping[str, Any], ...] = ()</code> ; information_events: <code>tuple[Mapping[str, Any], ...] = ()</code> ; execution_path: <code>tuple[str, ...] = ()</code> ; notes: <code>tuple[str, ...] = ()</code>
Source	<code>models.py:2056</code>
Evidence	Direct use: test files 4, benchmarks 4; not named in doc 05; e.g. <code>benchmark_layer1_handoff_v033.py</code> , <code>benchmark_post_execution_bookkeeping_v038.py</code> .

11.5 Binding Registration

The native route begins with an `ExecutionBindingRegistry`, a separate registry of host callables, one per registered action. Registration is not authorization: registering a binding enters nothing and grants nothing (C15/D9 §10B). The registry also holds the control state of each native execution (11.9).

Entry — ExecutionBindingRegistry

Field	Content
Name and label	<code>ExecutionBindingRegistry</code> — class; <code>execution.py:16</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Holds host callables for registered actions, their control states, and their control history.
Framework basis	Realization Operator ε v1.7 (C15/D9) §10B: runtime detail between $\pi_{\text{sel}}(t)$ and $x_{\text{exec}}(t)$ stays subordinate to ε .
Signature	<code>ExecutionBindingRegistry(action_ids: tuple[str, ...] list[str] set[str])</code>

Field	Content
Preconditions	Constructed with the ids of the registered actions, usually <code>tuple(runtime.registry.actions)</code> .
Result	An empty binding registry.
Invariants	At most one binding per action and per binding id. The class docstring still says the registry "deliberately has no invoke/execute method" and that a later build will invoke bindings; that is out of date—v0.141 invokes through <code>invoke_authorized_native</code> , and the registry's own <code>invoke</code> method fails closed (Chapter 15).
Failure behavior	None at construction.
Does not	Invoke a callable on its own, or authorize anything.
Evidence	Direct use: test files 7, example files 1; named in doc 05; e.g. <code>authority_bound_callthrough_v0130.py</code> .
See also	11.6–11.9.

Entry — `ExecutionBindingRegistry.register`

Field	Content
Name and label	<code>ExecutionBindingRegistry.register</code> — method; <code>execution.py:51</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Registers one callable as the implementation of one action.
Framework basis	As above.
Signature	<code>ExecutionBindingRegistry.register(identity: ExecutionBindingIdentity, target: Callable[..., Any]) → ExecutionBindingAssessment</code>
Argument record	<code>ExecutionBindingIdentity</code> : frozen dataclass; <code>binding_id</code> : str; <code>action_id</code> : str; <code>implementation_version</code> : str; <code>provenance_ids</code> : <code>tuple[str, ...] = ()</code> ; <code>metadata</code> : <code>Mapping[str, Any] = (empty)</code>
Preconditions	The identity names a binding id, a registered action, and an implementation version, with unique provenance ids; the target is callable; neither the binding id nor the action already has a binding.
Result	<code>ExecutionBindingAssessment</code> (valid, with the note that registration confers no execution authority).
Invariants	The callable is stored and not called. It will receive the <code>ExecutionContext</code> as its first argument when invoked.
Failure behavior	Raises <code>ValueError</code> for an invalid identity or a duplicate, and <code>TypeError</code> for a non-callable target.
Does not	Call the target or confer authority.
Evidence	Direct use: test files 7, example files 1; named in doc 05; e.g. <code>authority_bound_callthrough_v0130.py</code> .
See also	11.6.

Table 11.5 — Other `ExecutionBindingRegistry` methods

Method	Label	Use
<code>assess_identity(identity)</code>	PUBLIC / ADVANCED	Checks an identity without registering it
<code>identity_for_action(action_id), identity(binding_id), registered_identities()</code>	PUBLIC / ADVANCED	Read registered identities
<code>create_context(action_id, decision_cycle_id, ...)</code>	PUBLIC / ADVANCED	Builds an <code>ExecutionContext</code> ; a context carries no authority, and it generates a fresh execution id unless one is passed
<code>invoke(context, ...)</code>	LEGACY / COMPATIBILITY	Deprecated; always raises <code>RuntimeError</code> . Use <code>invoke_authorized_native</code>
<code>build_layer1_handoff(context, result, ...)</code>	LEGACY / COMPATIBILITY	Direct native-to-Layer-1 handoff that bypasses ϵ ; see 11.8
<code>telemetry, epsilon_observation</code>	PUBLIC / ADVANCED	11.8
<code>control_state, set_control_state, control_history, build_reauthorization_handoff</code>	PUBLIC / ADVANCED	11.9

11.6 Native Authorization

Entry — `PVPPRuntime.issue_native_execution_authorization`

Field	Content
Name and label	<code>PVPPRuntime.issue_native_execution_authorization</code> — method; <code>kernel.py:4303</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Issues one single-use permission to enter the binding for one licensed action.
Framework basis	Realization Operator ϵ v1.7 (C15/D9) §10B; ledger-backed native authority is a runtime construct serving it.
Signature	<code>PVPPRuntime.issue_native_execution_authorization(episode: ExecutionEpisode, action_id: str, binding_registry, *, decision_cycle_id: str, configuration_id: str None = None, attempt: int = 1) → NativeExecutionAuthorization</code>
Preconditions	The episode is the exact object this runtime recorded as the current episode, active and entry-sufficient; its license is the exact instantiated license; the license posture is consistent with its mode; the action is in the license; a binding is registered for it; <code>decision_cycle_id</code> is non-empty; attempt is at least 1.

Field	Content
Result	NativeExecutionAuthorization with status "issued", a sequence number, and the binding id and implementation version at issuance. A native execution id is allocated for it (native_execution_authorization_execution_id).
Invariants	Issuance is serialized by a lock; sequence numbers increase by one. The returned object is itself the ledger record, and only that object can be consumed.
Failure behavior	Raises TypeError for a non-episode, ValueError for a fabricated, copied, inactive, or invalidated episode, an unlicensed action, an inconsistent license, or bad identifiers, and KeyError for an action without a binding.
Does not	Invoke anything, or authorize a terminal-infeasibility license.
Evidence	Direct use: test files 2, example files 1; named in doc 05; e.g. authority_bound_callthrough_v0130.py.
See also	11.7; test_v2_canonical_lineage_authority_v0131.

Table 11.6 — Authorization lifecycle

Status	Reached by	Consequence
issued	issue_native_execution_authorization	May be consumed once
consumed	invoke_authorized_native, immediately before the callable is entered	Any later use raises RuntimeError, whatever the callable did
invalidated	invalidate_native_execution_authorizations; a non-ready control state at invocation; the episode becoming terminal	Any later use raises RuntimeError. Invalidation by episode or cycle also retires the episode, so no new authorization can be issued for it

Entry — PVPPRuntime.invalidate_native_execution_authorizations

Field	Content
Name and label	PVPPRuntime.invalidate_native_execution_authorizations — method; kernel.py:4335. PUBLIC / ADVANCED (rule R2).
Purpose	Invalidates every issued authorization in a scope, and retires the matching episodes.
Framework basis	Stale-lineage retirement for native authority; re-entry itself is Chapter 12.
Signature	PVPPRuntime.invalidate_native_execution_authorizations(*, episode_id: str None = None, decision_cycle_id: str None = None, action_id: str None = None, reason: str) → tuple[str, ...]
Preconditions	At least one of episode_id, decision_cycle_id, action_id, and a non-empty reason.
Result	The ids of the authorizations invalidated.

Field	Content
Invariants	Only "issued" records change. Episodes named by episode_id, or reached through decision_cycle_id, move from "current" to "invalidated".
Failure behavior	Raises ValueError without a scope or a reason.
Does not	Choose a re-entry stage or revoke a consumed authorization.
Evidence	Direct use: test files 2; named in doc 05; e.g. test_v2_authority_bound_callthrough_v0130.py, test_v2_canonical_lineage_authority_v0131.py.
See also	12.3.

Table 11.7 — Authorization queries

Method	Returns
native_execution_authorization_status(authorization_id)	The ledger status, or None
native_execution_authorizations(episode_id=None, decision_cycle_id=None)	Copies of the ledger records in issuance order. The copies are for audit and cannot be consumed
native_execution_authorization_execution_id(authorization_id)	The execution id the invocation will use; needed to set a control state before invocation (11.9)

11.7 Invocation and Outcomes

Entry — PVPPRuntime.invoke_authorized_native

Field	Content
Name and label	PVPPRuntime.invoke_authorized_native — method; kernel.py:4360. PUBLIC / SUPPORTED (rule R1).
Purpose	Consumes one authorization and enters its binding exactly once.
Framework basis	Realization Operator ε v1.7 (C15/D9) §10B.
Signature	PVPPRuntime.invoke_authorized_native(authorization: NativeExecutionAuthorization, binding_registry, *args, configuration_id: str None = None, **kwargs)
Preconditions	The exact authorization object issued by this runtime, still "issued"; configuration_id equal to the one it was issued with; the binding registry still maps the action to the same binding id and implementation version; the execution's control state is "ready".
Result	NativeExecutionResult, normalized as in Table 11.8. Exceptions from the callable are converted to results, not raised.

Field	Content
Invariants	All checks and the consumption happen under the issuance lock, so concurrent attempts enter the callable at most once. The callable receives an ExecutionContext with the native execution id, action, binding, decision cycle, attempt, and configuration id; its correlation id is None.
Failure behavior	Raises TypeError, KeyError, ValueError, or RuntimeError before the callable is entered for any failed check; the authorization then stays issued, except that a non-ready control state invalidates it.
Does not	Advance ϵ , apply a transition, or retry.
Evidence	Direct use: test_files 2, example files 1; named in doc 05; e.g. authority_bound_callthrough_v0130.py.
See also	11.8; test_v2_authority_bound_callthrough_v0130.

Table 11.8 — Native outcomes

Status	Cause	external_effect _possible / completed	Test
succeeded	The callable returned	True / True	test_v2_native_invocation_v076: test_success_invokes_once_and_preserves_execution_identity
failed_cleanly	The callable raised CleanExecutionFailure	False / False	test_v2_native_invocation_v076: test_clean_failure_requires_explicit_binding_signal
indeterminate	IndeterminateExecutionFailure, TimeoutError, or ConnectionError (failure_stage "external_unknown" or "transport"); or any other exception (failure_stage "local_callable")	True / False	test_v2_native_invocation_v076: test_timeout_is_indeterminate_not_clean_failure, test_arbitrary_exception_defaults_conservatively_to_indeterminate
completed_invalid	The callable raised CompletedInvalidExecution	True / True; transition_valid False	test_v2_native_invocation_v076: test_completed_invalid_is_distinct_from_technical_failure

The three exception classes are the binding's vocabulary. Only CleanExecutionFailure asserts that no external effect occurred, so only a binding that knows this should raise it; any unclassified exception is treated as possibly effective, because absence of a response is not evidence of absence of effect.

11.8 From Native Result to ϵ

A native result is technical evidence, not realization. The v0.79 bridge turns it into an ExecutionObservation for advance_execution, so the ϵ trace stays the single chain of execution evidence. invoke_authorized_native returns the result but not the context, so the host rebuilds the context from the result and the authorization: ExecutionContext(result.execution_id, a.action_id, a.binding_id, a.decision_cycle_id, a.attempt, configuration_id=a.configuration_id), or create_context with execution_id=result.execution_id.

Entry — ExecutionBindingRegistry.epsilon_observation

Field	Content
Name and label	ExecutionBindingRegistry.epsilon_observation — method; execution.py:321. PUBLIC / ADVANCED (rule R2).
Purpose	Translates one native result, with its telemetry, into an ϵ observation.
Framework basis	Realization Operator ϵ v1.7 (C15/D9) §6, §9.
Signature	ExecutionBindingRegistry.epsilon_observation(context: ExecutionContext, result, *, telemetry=None, event_id: str None = None, realized_pv_bundle: Mapping[str, Any] None = None, information: Mapping[str, Any] None = None)
Argument record	ExecutionContext: frozen dataclass; execution_id: str; action_id: str; binding_id: str; decision_cycle_id: str; attempt: int = 1; correlation_id: str None = None; configuration_id: str None = None; metadata: Mapping[str, Any] = (empty)
Preconditions	The result and telemetry match the context's execution, action, and binding ids, and the telemetry status equals the result status. Telemetry is built if not supplied.
Result	ExecutionObservation whose information records the native identity and outcome. The caller passes it to advance_execution.
Invariants	succeeded $\rightarrow$ completed and completion-sufficient; failed_cleanly $\rightarrow$ failed; indeterminate $\rightarrow$ continuation not sufficient; completed_invalid $\rightarrow$ completed but not completion-sufficient. Through Table 11.3, an indeterminate or completed-invalid result becomes partial_realization when the host supplies an observed realized bundle and aborted_return otherwise; an unknown effect is never reported as a known failure.
Failure behavior	Raises TypeError for wrong types and ValueError for identity or status mismatches.
Does not	Advance ϵ or change Layer-1 state.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_native_epsilon_bridge_v079.py.
See also	test_v2_native_epsilon_bridge_v079.

ExecutionBindingRegistry.telemetry(context, result) builds the NativeExecutionTelemetry record the bridge uses; telemetry is execution evidence, not a transition.

ExecutionBindingRegistry.build_layer1_handoff predates the bridge: it builds a Layer-1 handoff

straight from a native result, with the execution id as the episode id, and maps outcomes differently (indeterminate → partial_realization, completed_invalid → failed). It bypasses the ε trace that v0.79 made canonical, so it is labeled LEGACY / COMPATIBILITY (H.5-7).

11.9 Control States and Reauthorization

Each native execution has a control state in the binding registry, keyed by its execution id. The state is a synchronous gate checked before the callable is entered; it implies no asynchronous cancellation of a call already in progress. Every change is recorded as a NativeExecutionControlEvent.

Table 11.9 — Control states

State	Effect at invocation	Test
ready	Invocation proceeds (the default)	test_v2_native_telemetry_control_v078: test_ready_control_allows_invocation
paused	The authorization is invalidated and RuntimeError is raised	test_v2_authority_bound_callthrough_v0130: test_abct11_13_control_gate_invalidates_before_call
aborted	As for paused	As above
reauthorization_required	As for paused; a reauthorization handoff can be built	As above; test_v2_execution_bookkeeping_v080: test_handoff_requires_reauthorization_control_state

Entry — ExecutionBindingRegistry.set_control_state

Field	Content
Name and label	ExecutionBindingRegistry.set_control_state — method; execution.py:111. PUBLIC / ADVANCED (rule R2).
Purpose	Sets the control state of one native execution and records the change.
Framework basis	Runtime construct serving ε 's return-upstream rule (C15/D9 §7).
Signature	ExecutionBindingRegistry.set_control_state(execution_id: str, state: str, *, reason: str None = None, requested_by: str None = None, evidence_ids: tuple[str, ...] = ())
Preconditions	A state from Table 11.9, a non-empty execution id, and unique evidence ids.
Result	NativeExecutionControlState. A NativeExecutionControlEvent with prior and new state is appended to the history.
Invariants	A non-ready change without a reason is recorded with reason "unspecified". control_history(execution_id) returns the events in order.

Field	Content
Failure behavior	Raises ValueError for an unknown state, an empty execution id, or duplicate evidence ids.
Does not	Cancel a running call or choose a re-entry stage.
Evidence	Direct use: test files 3; not named in doc 05; e.g. test_v2_authority_bound_callthrough_v0130.py, test_v2_execution_bookkeeping_v080.py.
See also	11.6 (native_execution_authorization_execution_id).

Entry — ExecutionBindingRegistry.build_reauthorization_handoff

Field	Content
Name and label	ExecutionBindingRegistry.build_reauthorization_handoff — method; execution.py:136. PUBLIC / ADVANCED (rule R2).
Purpose	Packages an execution that requires reauthorization for return to governance.
Framework basis	As above.
Signature	ExecutionBindingRegistry.build_reauthorization_handoff(context: ExecutionContext, *, reason: str None = None, requested_by: str None = None, evidence_ids: tuple[str, ...] = (), handoff_id: str None = None)
Preconditions	The execution's control state is "reauthorization_required"; a reason is given or recorded; evidence ids are unique.
Result	NativeReauthorizationHandoff with the execution, action, binding, cycle, correlation, and configuration identities, and return_to_governance true.
Invariants	The handoff states that the earliest invalidated stage is not determined here.
Failure behavior	Raises ValueError otherwise.
Does not	Authorize anything or choose a re-entry stage; Chapter 12 does that.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_execution_bookkeeping_v080.py.
See also	12.

11.10 Layer-1 Handoff and Transition

Entry — PVPPRuntime.build_layer1_transition_handoff

Field	Content
Name and label	PVPPRuntime.build_layer1_transition_handoff — method; kernel.py:4497. PUBLIC / SUPPORTED (rule R1).
Purpose	Builds the immutable handoff from a terminal ϵ result to the host's Layer-1 transition.
Framework basis	Realization Operator ϵ v1.7 (C15/D9) §11.4; Layer 1 (C10/D2).

Field	Content
Signature	PVPPRuntime.build_layer1_transition_handoff(result: EpsilonStepResult) → Layer1TransitionHandoff
Preconditions	A terminal result with a canonical status and an inactive episode. Completed statuses do not request return upstream; aborted_return and emergency_return do.
Result	Layer1TransitionHandoff with the episode and policy ids, status, path, realized bundles, information events, return_upstream, the license's actions as licensed_action_ids, and its mandatory recovery actions.
Invariants	The handoff contains no next state. licensed_action_ids carries the complete execution authority for the period.
Failure behavior	Raises ValueError for any failed precondition.
Does not	Mutate state, call WorldAdapter.execute, or call the transition service.
Evidence	Direct use: test files 4, benchmarks 1, example files 2; named in doc 05; e.g. minimal_battery_service.py, test_minimal_battery_service.py.
See also	test_layer1_transition_handoff_v033.

Record — Layer1TransitionHandoff

Field	Content
Label	IMPLEMENTATION / SUPPORT (rule R3: built by the runtime and passed to the host service)
Fields	frozen dataclass; episode_id: str; selected_policy_id: str; execution_status: str; execution_path: tuple[str, ...]; realized_pv_bundles: tuple[Mapping[str, Any], ...]; execution_information: tuple[Mapping[str, Any], ...]; return_upstream: bool; notes: tuple[str, ...] = (); licensed_action_ids: tuple[str, ...] = (); mandatory_recovery_action_ids: tuple[str, ...] = ()
Source	models.py:2073
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_layer1_actual_state_boundary_v040.py.

Entry — PVPPRuntime.apply_layer1_transition

Field	Content
Name and label	PVPPRuntime.apply_layer1_transition — method; kernel.py:4715. PUBLIC / SUPPORTED (rule R1).
Purpose	Calls the host's Layer1TransitionService with the current actual state and the handoff, and validates the result.
Framework basis	Layer 1 (C10/D2).
Signature	PVPPRuntime.apply_layer1_transition(current_state: ActualPersistentStateEnvelope, handoff: Layer1TransitionHandoff) → Layer1TransitionResult

Field	Content
Preconditions	A Layer1TransitionService is attached (10.4); the current actual state is valid; the handoff names an episode and a selected policy.
Result	The validated Layer1TransitionResult.
Invariants	The decision cycle and ϵ never call this method; only the host or the integrated cycle with execute=True does.
Failure behavior	Raises ValueError when no service is attached, for an invalid current state or handoff, and for an invalid result.
Does not	Transition state itself, re-run the cycle, or record recovery bookkeeping.
Evidence	Direct use: test files 2, example files 2; named in doc 05; e.g. minimal_battery_service.py, test_minimal_battery_service.py.
See also	10.4; 11.11.

Record — Layer1TransitionResult

Field	Content
Label	PUBLIC / SUPPORTED (rule R6: returned by the host Layer1TransitionService)
Fields	frozen dataclass; episode_id: str; selected_policy_id: str; prior_state_id: str; next_state: ActualPersistentStateEnvelope; transition_applied: bool; invariant_checks: Mapping[str, bool] = (empty)); notes: tuple[str, ...] = ()
Source	models.py:488
Evidence	Direct use: test files 9, benchmarks 8, example files 1; not named in doc 05; e.g. minimal_battery_service.py.

Entry — PVPPRuntime.validate_layer1_transition_result

Field	Content
Name and label	PVPPRuntime.validate_layer1_transition_result — method; kernel.py:4736. PUBLIC / ADVANCED (rule R2).
Purpose	Checks a transition result against the prior state and the handoff.
Framework basis	As above.
Signature	PVPPRuntime.validate_layer1_transition_result(prior: ActualPersistentStateEnvelope, handoff: Layer1TransitionHandoff, result: Layer1TransitionResult) → Layer1TransitionValidationAssessment
Preconditions	None.
Result	Layer1TransitionValidationAssessment.
Invariants	The result names the handoff's episode and policy and the prior state id; transition_applied is true; the next state is valid, keeps the actor, has a new state id, and does not move time backward; every host invariant check is true.

Field	Content
Failure behavior	Returns violations; raises nothing.
Does not	Interpret PP, SPV, AVS, or context changes.
Evidence	Direct use: test files 2, benchmarks 1, example files 1; named in doc 05; e.g. minimal_battery_service.py.
See also	apply_layer1_transition runs this check itself.

11.11 Transition Provenance and Recovery Bookkeeping

After a validated transition, two things may follow. Provenance records the link from prior state through ϵ to next state so a later cycle can check that it starts from the recorded result. Recovery bookkeeping advances the runtime's record of active recovery corridors. Neither confers authority.

Entry — [PVPPRuntime.build_execution_transition_provenance](#)

Field	Content
Name and label	PVPPRuntime.build_execution_transition_provenance — method; kernel.py:4769. PUBLIC / SUPPORTED (rule R1).
Purpose	Builds an audit record of one validated transition.
Framework basis	Execution-evidence traceability (C45/D9); Layer 1 (C10/D2).
Signature	PVPPRuntime.build_execution_transition_provenance(prior: ActualPersistentStateEnvelope, handoff: Layer1TransitionHandoff, result: Layer1TransitionResult, validation: Layer1TransitionValidationAssessment) → ExecutionTransitionProvenance
Preconditions	A valid Layer1TransitionValidationAssessment.
Result	ExecutionTransitionProvenance: actor, episode, policy, licensed and mandatory actions, ϵ status and path, prior and next state ids and times, transition_applied, evidence signatures of the realized bundles and information, and the return posture.
Invariants	The runtime does not retain the record; the host passes it to a later cycle. The evidence signatures are repr-based in-process integrity checks, not cryptographic identities.
Failure behavior	Raises ValueError for an invalid validation.
Does not	Confer decision or execution authority.
Evidence	Direct use: test files 1, example files 1; not named in doc 05; e.g. minimal_battery_service.py.
See also	6.5–6.6 (evidence signatures); test_execution_transition_provenance_v065.

Entry — `PVPPRuntime.validate_execution_transition_provenance`

Field	Content
Name and label	<code>PVPPRuntime.validate_execution_transition_provenance</code> — method; <code>kernel.py:4798</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Checks that a host-supplied starting state is the recorded result of a prior transition.
Framework basis	As above.
Signature	<code>PVPPRuntime.validate_execution_transition_provenance(provenance: ExecutionTransitionProvenance, current_state: ActualPersistentStateEnvelope) → ExecutionTransitionProvenanceAssessment</code>
Preconditions	None.
Result	<code>ExecutionTransitionProvenanceAssessment</code> .
Invariants	The provenance names actor, episode, and policy; records an applied transition; matches the current state's actor, state id, and time; does not move time backward or reuse the prior state id; has unique licensed actions; and contains the mandatory recovery actions within them.
Failure behavior	Returns violations. <code>evaluate_integrated_canonical_cycle</code> raises on invalid prior provenance before perception runs.
Does not	Reconstruct how state changed.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. <code>benchmark_execution_transition_provenance_v065.py</code> .
See also	9.3.

Recovery corridors are runtime bookkeeping, held in `runtime.active_corridors` as `ActiveRecoveryCorridor` records. A corridor starts when its plan's trigger action is confirmed and advances as each continuation action is confirmed. The host must say which registered recovery actions were actually realized: the runtime does not infer action identity from ϵ event ids, path labels, or policy membership.

Record — `RecoveryActionConfirmation`

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; <code>action_id: str</code> ; <code>recovery_plan_id: str</code> None = None
Source	<code>models.py:2092</code>
Evidence	Direct use: test files 2, benchmarks 2; not named in doc 05; e.g. <code>benchmark_post_execution_bookkeeping_v038.py</code> , <code>benchmark_transition_authoritative_bookkeeping_v046.py</code> .

Entry — `PVPPRuntime.record_recovery_bookkeeping_from_transition`

Field	Content
Name and label	<code>PVPPRuntime.record_recovery_bookkeeping_from_transition</code> — method; <code>kernel.py:5112</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Updates active recovery corridors from a validated transition and the host's action confirmations.
Framework basis	Recovery plans and corridors (C15/D10); transition authority (C10/D2).
Signature	<code>PVPPRuntime.record_recovery_bookkeeping_from_transition(prior_actual_state: ActualPersistentStateEnvelope, result: EpsilonStepResult, transition_result: Layer1TransitionResult, action_confirmations=()) → ExecutionBookkeepingAssessment</code>
Preconditions	A terminal ϵ result whose handoff validates against the transition result; confirmations naming registered actions, with an explicit <code>recovery_plan_id</code> wherever an action could start or continue more than one corridor.
Result	<code>ExecutionBookkeepingAssessment</code> : corridors started, advanced, and completed, and actions ignored.
Invariants	The commit time is the next state's time. A trigger for an already active corridor raises; an action matching no corridor is recorded as ignored.
Failure behavior	Raises <code>ValueError</code> for an invalid transition, a non-terminal result, an unregistered action, an unknown plan, an ambiguous action, or an action that is not the plan's trigger or next step.
Does not	Deduplicate by episode. Record each validated transition once; recording it again can advance a corridor twice.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. <code>benchmark_transition_authoritative_bookkeeping_v046.py</code> .
See also	<code>test_transition_authoritative_bookkeeping_v046</code> .

`build_layer1_execution_commit_from_transition` derives the `Layer1ExecutionCommit` this method uses, as an audit view of the validated transition. `record_layer1_execution_commit`, the v0.38 surface that accepted a host-built commit with a `state_transition_applied` flag, is LEGACY / COMPATIBILITY by its own docstring; so is the pre- ϵ `record_execution` (Chapter 15).

11.12 Identity and Evidence Integrity

Execution identity is checked at each link: the license against the cycle (a fresh artifact audit), the episode and authorization against the ledgers, the native result and telemetry against the context, the handoff against the terminal ϵ result, the transition against the handoff, and the provenance against the next state. `audit_post_selection_identity_chain` (9.12) checks the chain across objects after the fact, and the prediction-error and epistemic-update requests carry the provenance's evidence signatures so that spliced or mutated execution evidence is rejected (6.5–6.6; `test_execution_evidence_integrity_v067`, `test_cross_object_execution_identity_v068`). What the chain does not bind is covered by the host obligations of 11.2–11.4 and 11.11: one execution per selection, unique episode ids, advancing only the latest episode, and recording each transition once.

11.13 Classification Record and Compact Rows

The observation route—license, instantiation, advancement, handoff, transition, and provenance—is PUBLIC / SUPPORTED: the minimal_battery_service example uses every step. The native route's registry, registration, authorization, and invocation are PUBLIC / SUPPORTED on the authority_bound_callthrough example. That example stops at invocation, so the bridge back into ϵ , telemetry, control, queries, invalidation, validation calls, and recovery bookkeeping are PUBLIC / ADVANCED. The registry's invoke method and direct Layer-1 handoff, and record_layer1_execution_commit, are LEGACY / COMPATIBILITY. Records the host builds take the label of the entry that consumes them; records the runtime issues are IMPLEMENTATION / SUPPORT.

Table 11.10 — Labels assigned in Chapter 11

Symbol	Label	Rule
PVPPRuntime.instantiate_execution	PUBLIC / SUPPORTED	R1
PVPPRuntime.advance_execution	PUBLIC / SUPPORTED	R1
PVPPRuntime.build_layer1_transition_handoff	PUBLIC / SUPPORTED	R1
PVPPRuntime.apply_layer1_transition	PUBLIC / SUPPORTED	R1
PVPPRuntime.build_execution_transition_provenance	PUBLIC / SUPPORTED	R1
PVPPRuntime.issue_native_execution_authorization	PUBLIC / SUPPORTED	R1
PVPPRuntime.invoke_authorized_native	PUBLIC / SUPPORTED	R1
PVPPRuntime.validate_layer1_transition_result	PUBLIC / ADVANCED	R2
PVPPRuntime.validate_execution_transition_provenance	PUBLIC / ADVANCED	R2
PVPPRuntime.native_execution_authorization_status	PUBLIC / ADVANCED	R2
PVPPRuntime.native_execution_authorizations	PUBLIC / ADVANCED	R2
PVPPRuntime.native_execution_authorization_execution_id	PUBLIC / ADVANCED	R2
PVPPRuntime.invalidate_native_execution_authorizations	PUBLIC / ADVANCED	R2
PVPPRuntime.record_recovery_bookkeeping_from_transition	PUBLIC / ADVANCED	R2

Symbol	Label	Rule
PVPPRuntime.build_layer1_execution_commit_from_transition	PUBLIC / ADVANCED	R2
PVPPRuntime.record_layer1_execution_commit	LEGACY / COMPATIBILITY	R5 (docstring: legacy v0.38 compatibility surface)
ExecutionBindingRegistry	PUBLIC / SUPPORTED	R1
ExecutionBindingRegistry.register	PUBLIC / SUPPORTED	R1
ExecutionBindingRegistry.assess_identity	PUBLIC / ADVANCED	R2
ExecutionBindingRegistry.create_context	PUBLIC / ADVANCED	R2
ExecutionBindingRegistry.epsilon_observation	PUBLIC / ADVANCED	R2
ExecutionBindingRegistry.identity	PUBLIC / ADVANCED	R2
ExecutionBindingRegistry.identity_for_action	PUBLIC / ADVANCED	R2
ExecutionBindingRegistry.registered_identities	PUBLIC / ADVANCED	R2
ExecutionBindingRegistry.control_state	PUBLIC / ADVANCED	R2
ExecutionBindingRegistry.set_control_state	PUBLIC / ADVANCED	R2
ExecutionBindingRegistry.control_history	PUBLIC / ADVANCED	R2
ExecutionBindingRegistry.build_reauthorization_handoff	PUBLIC / ADVANCED	R2
ExecutionBindingRegistry.telemetry	PUBLIC / ADVANCED	R2
ExecutionBindingRegistry.invoke	LEGACY / COMPATIBILITY	R5 (docstring: deprecated; fails closed)

Symbol	Label	Rule
ExecutionBindingRegistry.build_layer1_handoff	LEGACY / COMPATIBILITY	R5 (superseded by the ϵ bridge of v0.79; H.5- 7)
ExecutionObservation	PUBLIC / SUPPORTED	R6
ExecutionBindingIdentity	PUBLIC / SUPPORTED	R6
Layer1TransitionResult	PUBLIC / SUPPORTED	R6
CleanExecutionFailure	PUBLIC / SUPPORTED	R6 (raised by the host binding)
IndeterminateExecutionFailure	PUBLIC / SUPPORTED	R6 (raised by the host binding)
CompletedInvalidExecution	PUBLIC / SUPPORTED	R6 (raised by the host binding)
ExecutionContext	PUBLIC / ADVANCED	R6
RecoveryActionConfirmation	PUBLIC / ADVANCED	R6
ExecutionEpisode	IMPLEMENTATI ON / SUPPORT	R3
EpsilonStepResult	IMPLEMENTATI ON / SUPPORT	R3
ExecutionTraceNode	IMPLEMENTATI ON / SUPPORT	R3
NativeExecutionAuthorization	IMPLEMENTATI ON / SUPPORT	R3
NativeExecutionResult	IMPLEMENTATI ON / SUPPORT	R3
ExecutionBindingAssessment	IMPLEMENTATI ON / SUPPORT	R3
NativeExecutionTelemetry	IMPLEMENTATI ON / SUPPORT	R3

Symbol	Label	Rule
NativeExecutionControlState	IMPLEMENTATION / SUPPORT	R3
NativeExecutionControlEvent	IMPLEMENTATION / SUPPORT	R3
NativeReauthorizationHandoff	IMPLEMENTATION / SUPPORT	R3
Layer1TransitionHandoff	IMPLEMENTATION / SUPPORT	R3
Layer1TransitionValidationAssessment	IMPLEMENTATION / SUPPORT	R3
ExecutionTransitionProvenance	IMPLEMENTATION / SUPPORT	R3
ExecutionTransitionProvenanceAssessment	IMPLEMENTATION / SUPPORT	R3
ExecutionBookkeepingAssessment	IMPLEMENTATION / SUPPORT	R3
Layer1ExecutionCommit	IMPLEMENTATION / SUPPORT	R3
ActiveRecoveryCorridor	IMPLEMENTATION / SUPPORT	R3

Table 11.11 — Compact rows for runtime-built records

Name and kind	Label	Role and use	Source
ExecutionTraceNode — dataclass (frozen)	IMPLEMENTATION / SUPPORT	One node of the ϵ trace; linked through trace_tail	models.py: 2031
NativeExecutionAuthorization — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Issued by issue_native_execution_authorization (11.6)	models.py: 155
NativeExecutionResult — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of invoke_authorized_native (Table 11.8)	models.py: 2310
ExecutionBindingAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of register and assess_identity	models.py: 192
NativeExecutionTelemetry — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of ExecutionBindingRegistry.telemetry	models.py: 2348

Name and kind	Label	Role and use	Source
NativeExecutionControlState — dataclass (frozen)	IMPLEMENT ATION / SUPPORT	Result of set_control_state and control_state	models.py: 2372
NativeExecutionControlEvent — dataclass (frozen)	IMPLEMENT ATION / SUPPORT	One entry of control_history	models.py: 2385
NativeReauthorizationHandoff — dataclass (frozen)	IMPLEMENT ATION / SUPPORT	Result of build_reauthorization_handoff	models.py: 2402
Layer1TransitionValidationAssessment — dataclass (frozen)	IMPLEMENT ATION / SUPPORT	Result of validate_layer1_transition_result	models.py: 505
ExecutionTransitionProvenance — dataclass (frozen)	IMPLEMENT ATION / SUPPORT	Result of build_execution_transition_provenance	models.py: 515
ExecutionTransitionProvenanceAssessment — dataclass (frozen)	IMPLEMENT ATION / SUPPORT	Result of validate_execution_transition_provenance	models.py: 541
ExecutionBookkeepingAssessment — dataclass (frozen)	IMPLEMENT ATION / SUPPORT	Result of the bookkeeping methods	models.py: 2119
Layer1ExecutionCommit — dataclass (frozen)	IMPLEMENT ATION / SUPPORT	Audit view from build_layer1_execution_commit_from_transition	models.py: 2104
ActiveRecoveryCorridor — dataclass (frozen)	IMPLEMENT ATION / SUPPORT	Runtime bookkeeping in runtime.active_corridors	models.py: 1978

Chapter 12 — Change, Invalidation, and Re-entry

Facts change after a cycle has run: evidence loses authority, a reachability observation flips, a structure is admitted, an objective completes. Re-entry is the discipline that decides which earlier results may still be used and recomputes the rest from the earliest affected stage, without inventing invalidation and without silently restarting from the top. This chapter documents the v0.141 re-entry surface: how change becomes invalidation, how invalidation becomes a plan, and how a plan is executed by stage-specific executors.

A reader must be able to register decision dependencies, report a change, and re-enter at the correct stage without reusing an invalidated result.

12.1 The Re-entry Model

Re-entry has five steps, each a separate call with its own record. (1) The host registers dependencies: which artifact at which stage depends on which fact ids. (2) The host reports a change: which facts of which kind changed. (3) The runtime derives invalidation signals from explicit dependency matches only. (4) It forms a plan: the earliest invalidated stage, the stages before it that may be reused, and the stages from it onward that may not. (5) The host executes the plan through the executor for that boundary, supplying the reusable artifacts and any reconstruction inputs. Every function in this chapter is a module-level function of `pvpp_runtime`, exported at the top level; none is a `PVPPRuntime` method.

Framework standing. No V2.1 owner file defines dependency-scoped re-entry. The surface serves two framework rules—upstream invalidity is never repaired or laundered by a later stage (C20/D4), and the canonical stage order (C20/D3)—and is otherwise a runtime construct (15.6).

The runtime never infers invalidation. A change invalidates an artifact only if the host registered a dependency of that artifact on a changed fact id of the same kind. Nothing is invalidated because of names, stage order, or time. Stages at or after the earliest invalidated stage are never reused; stages before it are reusable only relative to the invalidations supplied, which is not a claim that they are valid under facts nobody reported.

Entry — `CANONICAL_REENTRY_STAGE_ORDER`

Field	Content
Name and label	<code>CANONICAL_REENTRY_STAGE_ORDER</code> — constant; <code>reentry.py:7</code> . PUBLIC / ADVANCED (rule R2).
Purpose	The stage order used to choose the earliest invalidated stage.
Framework basis	No V2.1 owner file; a runtime construct (15.6) serving the rule that upstream invalidity is never repaired or laundered downstream (C20/D4); canonical order (C20/D3).
Value	A tuple: "PPP", "Phi", "H", "G", "R", "Graph/Seed", "Pi", "Pi Completeness", "Constraints", "Domain Framing", "Joint Recovery Feasibility", "Adequacy", "Sigma", "epsilon".

Field	Content
Invariants	This is the canonical presentation order. It places Joint Recovery Feasibility after Domain Framing, although the executable cycle binds joint-recovery actions into Π before Constraints (3.9). Section 12.4 gives the correction.
Does not	Infer that a later observation invalidates an earlier stage.
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_v2_bounded_reentry_execution_v084.py, test_v2_reentry_plan_v083.py.
See also	3.1; 12.4.

12.2 Dependencies and Invalidation

Record — GovernanceArtifactDependency

Field	Content
Label	PUBLIC / ADVANCED (rule R6: registered by the host when an artifact is produced)
Fields	frozen dataclass; dependency_id: str; artifact_id: str; stage: str; fact_kind: str; fact_ids: tuple[str, ...]; provenance_ids: tuple[str, ...] = (); notes: tuple[str, ...] = ()
Source	models.py:2456
Evidence	Direct use: test files 2, benchmarks 1; not named in doc 05; e.g. benchmark_small_economy_v2_1_v0140.py.

Record — GovernanceDependencyChange

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host when facts change)
Fields	frozen dataclass; change_id: str; fact_kind: str; changed_fact_ids: tuple[str, ...]; reason: str; evidence_ids: tuple[str, ...] = (); execution_id: str None = None; configuration_id: str None = None; notes: tuple[str, ...] = ()
Source	models.py:2468
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_dependency_invalidation_v082.py.

Entry — derive_dependency_invalidations

Field	Content
Name and label	derive_dependency_invalidations — function; reentry.py:56. PUBLIC / ADVANCED (rule R2).
Purpose	Derives invalidation signals from explicit dependency matches, and assesses the resulting re-entry.

Field	Content
Framework basis	No V2.1 owner file; a runtime construct (15.6) serving the rule that upstream invalidity is never repaired or laundered downstream (C20/D4); canonical order (C20/D3).
Signature	derive_dependency_invalidations(dependencies: "tuple[GovernanceArtifactDependency, ...]", changes: "tuple[GovernanceDependencyChange, ...]") → "GovernanceDependencyInvalidationAssessment"
Preconditions	Unique dependency and change ids. Each dependency names an artifact, a canonical stage, a fact kind, and at least one fact id; each change names a kind, at least one changed fact id, and a reason. No duplicate fact, provenance, or evidence ids.
Result	GovernanceDependencyInvalidationAssessment: the matched dependencies, one GovernanceInvalidationSignal per match, and a GovernanceReentryAssessment. With no match, the assessment is valid and establishes no stage.
Invariants	A signal is produced only where the fact kinds are equal and the changed ids intersect the dependency's fact ids. Signal ids have the form "dep:<change_id>:<dependency_id>".
Failure behavior	Returns violations and no signals when the inputs are malformed.
Does not	Guess dependencies, recompute anything, or plan execution.
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_dependency_invalidation_v082.py.
See also	12.3; test_v2_dependency_invalidation_v082.

Four other derivations produce the same signal records from specialized sources. Each maps change to stages only through dependencies the host declared, and each is documented with its source chapter.

Table 12.1 — Invalidation derivations

Function	Source of change	Dependency record	Primary entry
derive_dependency_invalidations	Any explicit fact change	GovernanceArtifactDependency	12.2
derive_evidence_authority_governance_invalidations	Lost evidence authority	EvidenceCanonicalArtifactDependency	6.7; 12.7

Function	Source of change	Dependency record	Primary entry
derive_control_provenance_governance_invalidations	A control-provenance topology update	ControlProvenanceArtifactDependency	6.8
derive_objective_lifecycle_invalidations	An objective-only change	GovernanceArtifactDependency (fact kind "objective")	7.6

Record — GovernanceInvalidationSignal

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host or derived by the runtime)
Fields	frozen dataclass; signal_id: str; invalidated_stage: str; reason: str; source_kind: str; evidence_ids: tuple[str, ...] = (); artifact_ids: tuple[str, ...] = (); execution_id: str None = None; configuration_id: str None = None; notes: tuple[str, ...] = ()
Source	models.py:2423
Evidence	Direct use: test files 21; not named in doc 05; e.g. test_v2_adequacy_partial_reentry_v087.py, test_v2_bounded_reentry_execution_v084.py.

12.3 Planning Re-entry

Entry — assess_governance_reentry

Field	Content
Name and label	assess_governance_reentry — function; reentry.py:14. PUBLIC / ADVANCED (rule R2).
Purpose	Validates explicit invalidation signals and identifies the earliest invalidated stage.
Framework basis	No V2.1 owner file; a runtime construct (15.6) serving the rule that upstream invalidity is never repaired or laundered downstream (C20/D4); canonical order (C20/D3).
Signature	assess_governance_reentry(signals: tuple[GovernanceInvalidationSignal, ...]) → GovernanceReentryAssessment
Preconditions	At least one signal. Each has a unique non-empty id, a canonical stage, a reason, a source kind, and unique evidence and artifact ids.

Field	Content
Result	GovernanceReentryAssessment: validity, signal ids, invalidated stages, recompute_from_stage, and return_to_governance.
Invariants	The earliest stage is chosen only among the stages the signals name.
Failure behavior	Returns violations; raises nothing.
Does not	Infer causal invalidation or recompute any operator.
Evidence	Direct use: test files 21; not named in doc 05; e.g. test_v2_adequacy_partial_reentry_v087.py, test_v2_bounded_reentry_execution_v084.py.
See also	test_v2_governance_reentry_v081.

Entry — plan_governance_reentry

Field	Content
Name and label	plan_governance_reentry — function; reentry.py:125. PUBLIC / ADVANCED (rule R2).
Purpose	Turns a valid assessment into a decision-only recomputation scope.
Framework basis	No V2.1 owner file; a runtime construct (15.6) serving the rule that upstream invalidity is never repaired or laundered downstream (C20/D4); canonical order (C20/D3).
Signature	plan_governance_reentry(assessment: GovernanceReentryAssessment, signals: tuple[GovernanceInvalidationSignal, ...] = ()) → "GovernanceReentryPlan"
Preconditions	A valid assessment. If signals are supplied, their ids equal the assessment's signal ids, in order.
Result	GovernanceReentryPlan: recompute_from_stage, reusable_upstream_stages (the stages before it), nonreusable_stages (it and every later stage, including epsilon), the signal and invalidated artifact ids, and execute_reentry false. With no invalidated stage, the plan is valid and empty.
Invariants	The plan executes nothing.
Failure behavior	Returns an invalid plan with violations.
Does not	Assert that reusable stages are valid under unreported facts, or correct for executable dependencies (12.4).
Evidence	Direct use: test files 21; not named in doc 05; e.g. test_v2_1_objective_lifecycle_reentry_v0139.py, test_v2_adequacy_partial_reentry_v087.py.
See also	test_v2_reentry_plan_v083.

Record — GovernanceReentryPlan

Field	Content
Label	IMPLEMENTATION / SUPPORT (rule R3: returned by plan_governance_reentry and the change surfaces)
Fields	frozen dataclass; valid: bool; recompute_from_stage: str None; reusable_upstream_stages: tuple[str, ...]; nonreusable_stages: tuple[str, ...]; signal_ids: tuple[str, ...] = (); invalidated_artifact_ids: tuple[str, ...] = (); execute_reentry: bool = False; violations: tuple[str, ...] = (); notes: tuple[str, ...] = ()
Source	models.py:2528
Evidence	Direct use: test files 13; not named in doc 05; e.g. test_v2_bounded_reentry_execution_v084.py, test_v2_constraints_partial_reentry_v091.py.

Entry — execute_governance_reentry

Field	Content
Name and label	execute_governance_reentry — function; reentry.py:175. PUBLIC / ADVANCED (rule R2).
Purpose	Executes a plan through the generic entry, which supports only a PPP boundary.
Framework basis	No V2.1 owner file; a runtime construct (15.6) serving the rule that upstream invalidity is never repaired or laundered downstream (C20/D4); canonical order (C20/D3).
Signature	execute_governance_reentry(runtime, plan, actual_state, request)
Preconditions	A valid plan with execute_reentry false. For execution, recompute_from_stage is "PPP" and no stage is marked reusable.
Result	GovernanceReentryExecutionResult. "no_reentry_required" for an empty plan; "reentry_pass_completed" after one integrated cycle with execute=False; "partial_stage_reentry_not_yet_supported" for any other boundary.
Invariants	The PPP path runs evaluate_integrated_canonical_cycle once and stops before ϵ . Other boundaries are refused rather than restarted at PPP, which would recompute stages the plan marked reusable.
Failure behavior	Returns an invalid result with violations.
Does not	Dispatch to the stage executors. Its docstring still describes the v0.84 scope, and v0.141 has not widened it: for any boundary other than PPP, call the executor for that boundary (12.5).
Evidence	Direct use: test files 1; not named in doc 05; e.g. test_v2_bounded_reentry_execution_v084.py.
See also	test_v2_bounded_reentry_execution_v084.

Record — GovernanceReentryExecutionResult

Field	Content
Label	IMPLEMENTATION / SUPPORT (rule R3: returned by every executor)
Fields	frozen dataclass; valid: bool; recompute_from_stage: str None; status: str; decision: CanonicalDecisionCycleAssessment None = None; reused_stage_ids: tuple[str, ...] = (); recomputed_stage_ids: tuple[str, ...] = (); violations: tuple[str, ...] = (); notes: tuple[str, ...] = ()
Source	models.py:2545
Evidence	Direct use: no direct use in tests, benchmarks, or examples; not named in doc 05.

12.4 Reusable Artifacts and Executable Scope

A stage executor reuses earlier results only from a `GovernanceReusableArtifactBundle`: one payload per reusable stage, in the plan's order, tied to one cycle, one initial state, and one configuration. The payload for each stage is the result object the cycle produced for it (for example, the `GoverningAssessment` for "G"), or one of the reusable artifact records of Table 12.3 where an executor needs more than one object.

Record — GovernanceReusableArtifactBundle

Field	Content
Label	PUBLIC / ADVANCED (rule R6: assembled by the host from a prior cycle)
Fields	frozen dataclass; bundle_id: str; cycle_id: str; initial_state_id: str; configuration_id: str; stage_artifacts: tuple[tuple[str, Any], ...]; provenance_ids: tuple[str, ...] = (); notes: tuple[str, ...] = ()
Source	models.py:2580
Evidence	Direct use: test files 15; not named in doc 05; e.g. <code>test_v2_adequacy_partial_reentry_v087.py</code> , <code>test_v2_constraints_partial_reentry_v091.py</code> .

Entry — validate_reusable_artifact_bundle

Field	Content
Name and label	<code>validate_reusable_artifact_bundle</code> — function; <code>reentry.py</code> :212. PUBLIC / ADVANCED (rule R2).
Purpose	Checks a bundle's identity and stage coverage against a plan.
Framework basis	No V2.1 owner file; a runtime construct (15.6) serving the rule that upstream invalidity is never repaired or laundered downstream (C20/D4); canonical order (C20/D3).
Signature	<code>validate_reusable_artifact_bundle(plan, bundle, *, expected_cycle_id: str, expected_initial_state_id: str, expected_configuration_id: str)</code>
Preconditions	The expected cycle, initial-state, and configuration ids, all non-empty.
Result	<code>GovernanceReusableArtifactBundleAssessment</code> .

Field	Content
Invariants	The bundle's ids match the expected ids; its stages are unique, canonical, and exactly the plan's reusable stages in order; every payload is present; provenance ids are unique.
Failure behavior	Returns violations; raises nothing.
Does not	Check payload types or semantic validity. The bundle is matched by id strings only, and a payload for a stage the executor does not consume is never inspected. Each executor type-checks only the payloads it uses.
Evidence	Direct use: test files 15; not named in doc 05; e.g. test_v2_adequacy_partial_reentry_v087.py, test_v2_constraints_partial_reentry_v091.py.
See also	test_v2_reusable_artifact_bundle_v085.

A bundle is only as current as the host says it is. The runtime checks that the ids you declare match; it cannot check that the payloads came from that cycle or are still current. Assemble bundles from the result object of the cycle being re-entered, never from a different cycle.

Entry — [apply_executable_dependency_scope](#)

Field	Content
Name and label	apply_executable_dependency_scope — function; reentry.py:550. PUBLIC / ADVANCED (rule R2).
Purpose	Corrects a Joint Recovery Feasibility plan for the executable order of the cycle.
Framework basis	No V2.1 owner file; a runtime construct (15.6) serving the rule that upstream invalidity is never repaired or laundered downstream (C20/D4); canonical order (C20/D3); joint-recovery binding (3.9).
Signature	apply_executable_dependency_scope(plan)
Preconditions	A plan. Plans at any other boundary, and invalid plans, are returned unchanged.
Result	A GovernanceReentryPlan at the same boundary whose reusable stages end at Pi Completeness, and whose non-reusable stages are Joint Recovery Feasibility, Constraints, Domain Framing, Adequacy, Sigma, and epsilon.
Invariants	The executable cycle binds mandatory recovery actions into Π before Constraints, so a joint-recovery invalidation reaches Constraints and Domain Framing even though the presentation order lists them earlier.
Failure behavior	None.
Does not	Infer any other dependency.

Field	Content
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_v2_executable_dependency_scope_v089.py, test_v2_joint_recovery_reentry_v090.py.
See also	test_v2_joint_recovery_reentry_readiness_v088.

plan_governance_reentry does not apply this correction. A plan at "Joint Recovery Feasibility" produced by any derivation marks Constraints and Domain Framing reusable until the host passes it through apply_executable_dependency_scope; assess_joint_recovery_reentry_readiness (12.8) reports the unscoped plan as insufficient. The joint-recovery executor recomputes Constraints and Framing in either case, but a bundle built for an unscoped plan carries payloads for stages that are not in fact reusable. Apply the scope to every joint-recovery plan (Chapter 15).

12.5 Stage-Specific Re-entry Executors

v0.141 has one executor per boundary from PPP to Σ . Each checks the plan's boundary, the bundle assessment, and the payloads it needs; recomputes its own stage; and continues through the rest of the stack by delegating to the executor for the next boundary, so an early re-entry runs every later stage once. Every executor stops before ε and returns a GovernanceReentryExecutionResult whose decision is a CanonicalDecisionCycleAssessment. Early executors also need reconstruction inputs: the records of Table 12.3 that carry what the cycle request supplied for that stage, bound to the registry identity current when they were authorized.

Table 12.2 — Stage executors

Boundary	Executor	Bundle payloads used	Extra inputs	Continues to
Sigma	execute_sigma_reentry(runtime, plan, bundle, ba)	G, Pi, Pi Completeness, Constraints, Domain Framing, Adequacy (SigmaReusableAdequacyArtifact)	—	Standard Σ only; refuses when no policy is adequate
Adequacy	execute_adequacy_reentry(runtime, plan, bundle, ba)	G, Pi, Pi Completeness, Constraints, Domain Framing (AdequacyReusableDomainFramingArtifact), Joint Recovery Feasibility (AdequacyReusableProjectionArtifact)	—	Standard or fallback Σ ; Q is reused, not re-projected

Boundary	Executor	Bundle payloads used	Extra inputs	Continues to
Domain Framing	execute_domain_framing_reentry(runtime, plan, bundle, ba, state, request)	G, Graph/Seed, Pi, Pi Completeness, Constraints	—	Projection, Adequacy, standard or fallback Σ
Joint Recovery Feasibility	execute_joint_recovery_reentry(runtime, plan, bundle, ba, state, request)	G, R, Graph/Seed, Pi (pre-binding), Pi Completeness	—	Constraints, Framing, projection, Adequacy, Σ (terminal, fallback, or standard)
Constraints	execute_constraints_reentry(runtime, plan, bundle, ba, state, request)	Graph/Seed, Pi (ConstraintsReusablePi Artifact), Pi Completeness	—	Framing, projection, Adequacy, Σ
Pi Completeness	execute_pi_completeness_reentry(runtime, plan, bundle, ba, state, request)	G, R, Graph/Seed, Pi	—	Joint recovery and the rest; does not stop for an infeasible active set (defect K2)

Boundary	Executor	Bundle payloads used	Extra inputs	Continues to
Pi	execute_pi_reentry(..., pi_inputs)	Graph/Seed (G, R if present)	PiReusableConstructionInputs	Pi Completeness executor
Graph/Seed	execute_graph_reentry(.., graph_inputs, pi_inputs)	G, R	Graph and Pi inputs	Pi executor
R	execute_regime_reentry(..., regime_inputs, graph_inputs, pi_inputs)	Phi, H, G	Regime, Graph, and Pi inputs	Graph executor
G	execute_governing_reentry(..., governing_inputs, regime_inputs, graph_inputs, pi_inputs)	Phi, H	Governing, Regime, Graph, and Pi inputs	Regime executor
H	execute_horizon_reentry(..., horizon_inputs, governing_inputs, regime_inputs, graph_inputs, pi_inputs)	Phi	Horizon inputs and the above	Governing executor
Phi	execute_phi_reentry(..., phi_inputs, horizon_inputs, ...)	—	Phi inputs and the above	Horizon executor
PPP	execute_ppp_reentry(runtime, plan, bundle, ba, request, ppp_inputs, horizon_inputs, governing_inputs, regime_inputs, graph_inputs, pi_inputs)	—	PPPreentryInputs (the actual state) and the above	Re-runs the snapshot, then the Phi executor
PPP (memory-conditioned)	execute_memory_conditioned_ppp_reentry(..., perception_inputs, ...)	—	MemoryConditionedPPPreentryInputs and the above	Re-runs the memory-conditioned snapshot, then Phi

Tests, in the table's order: test_v2_sigma_partial_reentry_v086, test_v2_adequacy_partial_reentry_v087, test_v2_domain_framing_partial_reentry_v0115, test_v2_joint_recovery_reentry_v090, test_v2_constraints_partial_reentry_v091, test_v2_pi_completeness_reentry_v092, test_v2_pi_reentry_v093, test_v2_graph_reentry_v094, test_v2_regime_reentry_v095, test_v2_governing_reentry_v096, test_v2_horizon_reentry_v097, test_v2_phi_reentry_v098, test_v2_ppp_reentry_v099, test_v2_memory_conditioned_ppp_reentry_v0100.

Early executors (Phi through Pi) check that each reconstruction input matches the re-entry request—the preservation object, required families and paths, graph and Π configuration, material requirements—and that the registry identity recorded in the input (phi_substrate, horizon_configuration, governing_substrate, regime_configuration, graph_substrate) is still the registry's current identity. A changed registry is refused, not silently absorbed. The Phi and H executors reuse the perceived and baseline states recorded in their inputs; only the PPP executors perceive again.

Re-entry in v0.141 is selective recomputation, but it is not enforcement-equivalent to rerunning the complete canonical cycle. A re-entry decision has not passed everything a freshly generated canonical decision passes. This is demonstrated, not only cautioned: defect K2 below lets a re-entry decision through a joint-recovery gate that a full rerun enforces. Where full enforcement matters, rerun the canonical cycle instead.

The executors are not the canonical cycle, and three differences matter. First, the passes that re-project (Domain Framing, Joint Recovery, Constraints, Pi Completeness, and the earlier boundaries that reach them) check projection model authority but pass no perceived decision state or input-license assessment, so causal-input licensing (9.10) is not enforced during re-entry. Second, the decisions they return carry no artifact-integrity record; build_execution_license_from_cycle still accepts them, auditing only their internal consistency. A re-entry decision therefore can be licensed and executed; a run of the frozen code confirmed this for a Σ -only re-entry. Third, the Adequacy executor enters fallback Σ without a perceived state and so without the host's structural profiles. These are recorded in Chapter 15.

Known defect K2 in v0.141: execute_pi_completeness_reentry stops at the joint-recovery gate for "active_capacity_authority_required" and for "joint_recovery_infeasible", a status no record produces; an infeasible active set is reported as "active_jointly_infeasible". Re-entry at Pi Completeness, or at any earlier boundary—every earlier executor delegates to this one—therefore continues past an infeasible active recovery set to Σ , and the decision it returns can be licensed and executed. This is a governance bypass, not a diagnostic gap. Host obligation: after any such re-entry the host must inspect decision.joint_recovery_feasibility itself; if runtime_verified is true and jointly_feasible is false, the host must treat the pass as stopped and must not license or execute the decision. Recorded in Chapter 15.

A run of the frozen code confirmed the defect with the fixture of test_concurrent_recovery_execution_binding_v055 and a calendar too scarce for both active corridors. The full cycle stopped at Joint Recovery Feasibility with "joint_recovery_feasibility_failed", and the Joint Recovery Feasibility executor—which tests jointly_feasible, not the status—stopped too. The Pi Completeness executor completed with

"sigma_standard_policy_selected"; its decision still carried "active_jointly_infeasible" in joint_recovery_feasibility, yet build_execution_license_from_cycle licensed it and instantiate_execution opened an active episode. No test in the tree covers this case.

Where an executor does stop at the joint-recovery gate, the decision's status is the joint-recovery record's own status—"active_capacity_authority_required" or

"active_jointly_infeasible"—not the cycle's stop statuses

"joint_recovery_capacity_authority_required" and "joint_recovery_feasibility_failed" (Table 9.2).

Match on stopped_at, not on the status string, to recognize a joint-recovery stop in either path.

Table 12.3 — Reusable artifacts and reconstruction inputs (host-built)

Record	Used at	Carries
SigmaReusableAdequacyArtifact	Sigma	The AdequacyAssessment and the authoritative projection records
AdequacyReusableDomainFramingArtifact	Adequacy	The DomainFramingAssessment and the frame it validated
AdequacyReusableProjectionArtifact	Adequacy	The projection records and any joint-recovery artifact
ConstraintsReusablePiArtifact	Constraints	The bound Π , with its regime and governing assessments
PiReusableConstructionInputs	Pi and earlier	Materially required classes and Π configuration
GraphReusableConstructionInputs	Graph/Seed and earlier	Preservation object, required families and paths, graph configuration, and graph-substrate identity
RegimeReusableClassificationInputs	R and earlier	Domain assessments, interruption, previous regime, and regime-configuration identity
GoverningReusableIdentificationInputs	G and earlier	Governing-substrate identity
HorizonReusableEvaluationInputs	H and Phi	Perceived and baseline states and horizon-configuration identity
PhiReusableEvaluationInputs	Phi	Perceived and baseline states and Φ -substrate identity
PPPreentryInputs	PPP	The actual persistent state to perceive again
MemoryConditionedPPPreentryInputs	PPP (memory-conditioned)	Actual state, memory state, query, context, concerns, expectation state, and trace

Integration warning. Defects K1 and K2 (15.2) affect these executors, unchanged in v2.1.167. A supervised checkpoint plans re-entry but executes none of them (16.6); an application that executes the plan—supervised or not—must apply the K1 workaround and the K2 check of Table 15.3.

12.6 Change Surfaces That Plan Re-entry

Each registry change of Chapter 8 has a wrapper that applies the change and, when it is material, returns a trigger carrying the invalidation signal and plan. A rejected change creates no invalidation, and a no-op creates none. The wrappers change the registry; they execute nothing downstream. Two of the Graph triggers have execution bridges that check the trigger, bind the pre-change graph inputs to the prior substrate identity, verify that the registry now has the resulting identity, and delegate to `execute_graph_reentry`. The capability-driven graph functions of 8.10 route their changes through these wrappers.

Table 12.4 — Change wrappers

Wrapper	Registry change	Plans from	Execution
refresh_graph_reachability_and_plan_reentry(registry, request)	refresh_graph_reachability (8.4)	Graph/Seed, when a transformation's status changed	execute_graph_reachability_refresh_reentry
admit_graph_transformation_and_plan_reentry(registry, request)	admit_graph_transformation (8.5)	Graph/Seed	execute_graph_structural_admission_reentry
revise_graph_transformation_and_plan_reentry(registry, request)	revise_graph_transformation (8.5)	Graph/Seed	No bridge: call <code>execute_graph_reentry</code> with graph inputs bound to the new substrate identity
assess_graph_structural_coverage_and_plan_reentry(registry, observation)	assess_graph_structural_coverage (8.8)	Graph/Seed, when coverage is material	Through <code>disposition_graph_structural_coverage</code> (8.8)
admit_novel_function_and_plan_reentry(registry, request)	admit_novel_function (8.6)	Graph/Seed	No bridge; as for revision
admit_recovery_necessity_and_plan_reentry(registry, request)	admit_recovery_necessity (8.7)	G	No bridge: <code>execute_governing_reentry</code>
revise_recovery_necessity_and_plan_reentry(registry, request)	revise_recovery_necessity (8.7)	G	As above

Use the wrapper instead of the bare registry method whenever a cycle has already run: the bare method changes the registry but plans nothing, and the next re-entry's identity checks will then refuse inputs bound to the old substrate without saying why re-entry is needed.

12.7 Epistemic Invalidation Orchestration

Entry — `orchestrate_epistemic_invalidation_reentry`

Field	Content
Name and label	<code>orchestrate_epistemic_invalidation_reentry</code> — function; <code>reentry.py:1718</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Runs the chain from a changed evidence source to a re-entry plan.
Framework basis	No V2.1 owner file; a runtime construct (15.6) serving the rule that upstream invalidity is never repaired or laundered downstream (C20/D4); canonical order (C20/D3); evidence authority (6.7).
Signature	<code>orchestrate_epistemic_invalidation_reentry(source_state, evidence_dependencies, artifact_dependencies, *, as_of=None)</code>
Preconditions	A source state with source and provenance ids; unique evidence-dependency ids; the artifact dependencies.
Result	<code>EpistemicInvalidationOrchestrationAssessment</code> with every intermediate assessment and, when valid, the governance invalidation assessment carrying the plan.
Invariants	Only evidence dependencies naming this source and provenance are assessed; artifacts are reached only through declared <code>EvidenceCanonicalArtifactDependency</code> records.
Failure behavior	Returns violations.
Does not	Recompute or execute anything.
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_v2_epistemic_orchestration_v0113.py</code> .
See also	6.7; <code>test_v2_epistemic_orchestration_v0113</code> .

Entry — `execute_epistemic_reentry_plan`

Field	Content
Name and label	<code>execute_epistemic_reentry_plan</code> — function; <code>reentry.py:1759</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Dispatches the plan of a valid orchestration to the executor for its boundary.
Framework basis	As above.
Signature	<code>execute_epistemic_reentry_plan(runtime, orchestration, *, bundle=None, bundle_assessment=None, state=None, request=None, ppp_inputs=None, phi_inputs=None, horizon_inputs=None, governing_inputs=None, regime_inputs=None, graph_inputs=None, pi_inputs=None)</code>
Preconditions	A valid orchestration with a valid plan and a boundary; the inputs that boundary's executor needs, passed by keyword.

Field	Content
Result	EpistemicReentryExecutionAssessment wrapping the executor's result: "epistemic_reentry_dispatched", "epistemic_reentry_executor_rejected", or "epistemic_reentry_not_executable".
Invariants	Each boundary goes to exactly its executor of Table 12.2; there is no generic fallback.
Failure behavior	Returns violations for missing inputs. See the defect below for the PPP boundary.
Does not	Add ε or Layer-1 authority.
Evidence	Direct use: test files 2; not named in doc 05; e.g. test_v2_domain_framing_partial_reentry_v0115.py, test_v2_epistemic_reentry_execution_v0114.py.
See also	test_v2_epistemic_reentry_execution_v0114.

Known defect K1 in v0.141: the PPP branch of `execute_epistemic_reentry_plan` calls `execute_ppp_reentry` without `horizon_inputs`, so every argument after `ppp_inputs` shifts by one and the call raises `TypeError` (missing `pi_inputs`). Its missing-input check also omits `horizon_inputs`. For a PPP boundary, call `execute_ppp_reentry` directly with `orchestration.governance_invalidation.plan`. Recorded in Chapter 15.

The defect was confirmed by calling the frozen dispatcher with a PPP plan and every input supplied; no test in the tree exercises the PPP branch. The other twelve branches pass arguments in the executors' order.

12.8 Joint-Recovery Re-entry

Entry — [assess_joint_recovery_reentry_readiness](#)

Field	Content
Name and label	<code>assess_joint_recovery_reentry_readiness</code> — function; <code>reentry.py:515</code> . PUBLIC / ADVANCED (rule R2).
Purpose	Audits whether a plan and its originating cycle can support joint-recovery re-entry.
Framework basis	No V2.1 owner file; a runtime construct (15.6) serving the rule that upstream invalidity is never repaired or laundered downstream (C20/D4); canonical order (C20/D3); joint-recovery binding (3.9).
Signature	<code>assess_joint_recovery_reentry_readiness(plan, cycle)</code>
Preconditions	None.
Result	<code>JointRecoveryReentryReadinessAssessment</code> : ready, status, whether the cycle preserved the pre-binding Π , and whether the plan's scope is sufficient.

Field	Content
Invariants	Ready only if the plan is valid at the Joint Recovery Feasibility boundary, the cycle carries <code>pi_before_joint_recovery_binding</code> , and Constraints is not marked reusable.
Failure behavior	Returns violations.
Does not	Execute joint recovery or repair the plan; <code>apply_executable_dependency_scope</code> does that.
Evidence	Direct use: test files 2; not named in doc 05; e.g. <code>test_v2_executable_dependency_scope_v089.py</code> , <code>test_v2_joint_recovery_reentry_readiness_v088.py</code> .
See also	<code>test_v2_joint_recovery_reentry_readiness_v088</code> .

A joint-recovery re-entry must follow this sequence. It is a required host recipe, not a convenience: the second step reconciles the presentation order with the executable dependency order, and v0.141 does not do it for you.

- **1. Plan.** Derive the invalidation and form the plan at "Joint Recovery Feasibility" (12.2–12.3).
- **2. Scope.** Pass the plan through `apply_executable_dependency_scope`, so Constraints and every later stage are marked non-reusable (12.4).
- **3. Check readiness.** Call `assess_joint_recovery_reentry_readiness` with the scoped plan and the originating cycle; proceed only if it reports "joint_recovery_reentry_ready".
- **4. Execute.** Call `execute_joint_recovery_reentry` with a bundle for the scoped plan whose Pi payload is the originating cycle's `pi_before_joint_recovery_binding`.

The executor re-evaluates active joint feasibility, re-binds mandatory actions, and continues through Σ ; it stops at the joint-recovery gate when capacity authority is missing or the active set is infeasible. The executors at Pi Completeness and earlier do not stop for an infeasible set (defect K2, 12.5).

12.9 Classification Record and Compact Rows

The re-entry surface is PUBLIC / ADVANCED throughout: rule R2 names re-entry as specialized work, the tests exercise every executor, and no example or doc 05 entry uses it. The records the host builds—dependencies, changes, signals, bundles, reusable artifacts, and reconstruction inputs—take that label under R6. Assessments, plans, results, and triggers are IMPLEMENTATION / SUPPORT.

Table 12.5 — Labels assigned in Chapter 12

Symbol	Label	Rule
CANONICAL_REENTRY_STAGE_ORDER	PUBLIC / ADVANCED	R2
assess_governance_reentry	PUBLIC / ADVANCED	R2
derive_dependency_invalidations	PUBLIC / ADVANCED	R2
plan_governance_reentry	PUBLIC / ADVANCED	R2
execute_governance_reentry	PUBLIC / ADVANCED	R2
validate_reusable_artifact_bundle	PUBLIC / ADVANCED	R2

Symbol	Label	Rule
apply_executable_dependency_scope	PUBLIC / ADVANCED	R2
assess_joint_recovery_reentry_readiness	PUBLIC / ADVANCED	R2
execute_sigma_reentry	PUBLIC / ADVANCED	R2
execute_adequacy_reentry	PUBLIC / ADVANCED	R2
execute_domain_framing_reentry	PUBLIC / ADVANCED	R2
execute_joint_recovery_reentry	PUBLIC / ADVANCED	R2
execute_constraints_reentry	PUBLIC / ADVANCED	R2
execute_pi_completeness_reentry	PUBLIC / ADVANCED	R2
execute_pi_reentry	PUBLIC / ADVANCED	R2
execute_graph_reentry	PUBLIC / ADVANCED	R2
execute_regime_reentry	PUBLIC / ADVANCED	R2
execute_governing_reentry	PUBLIC / ADVANCED	R2
execute_horizon_reentry	PUBLIC / ADVANCED	R2
execute_phi_reentry	PUBLIC / ADVANCED	R2
execute_ppp_reentry	PUBLIC / ADVANCED	R2
execute_memory_conditioned_ppp_reentry	PUBLIC / ADVANCED	R2
refresh_graph_reachability_and_plan_reentry	PUBLIC / ADVANCED	R2
execute_graph_reachability_refresh_reentry	PUBLIC / ADVANCED	R2
admit_graph_transformation_and_plan_reentry	PUBLIC / ADVANCED	R2
revise_graph_transformation_and_plan_reentry	PUBLIC / ADVANCED	R2
execute_graph_structural_admission_reentry	PUBLIC / ADVANCED	R2
assess_graph_structural_coverage_and_plan_reentry	PUBLIC / ADVANCED	R2
admit_novel_function_and_plan_reentry	PUBLIC / ADVANCED	R2
admit_recovery_necessity_and_plan_reentry	PUBLIC / ADVANCED	R2
revise_recovery_necessity_and_plan_reentry	PUBLIC / ADVANCED	R2
derive_evidence_authority_governance_invalidations	PUBLIC / ADVANCED	R2
derive_control_provenance_governance_invalidations	PUBLIC / ADVANCED	R2
orchestrate_epistemic_invalidations_reentry	PUBLIC / ADVANCED	R2
execute_epistemic_reentry_plan	PUBLIC / ADVANCED	R2
GovernanceInvalidationSignal	PUBLIC / ADVANCED	R6

Symbol	Label	Rule
GovernanceArtifactDependency	PUBLIC / ADVANCED	R6
GovernanceDependencyChange	PUBLIC / ADVANCED	R6
GovernanceReusableArtifactBundle	PUBLIC / ADVANCED	R6
SigmaReusableAdequacyArtifact	PUBLIC / ADVANCED	R6
AdequacyReusableProjectionArtifact	PUBLIC / ADVANCED	R6
AdequacyReusableDomainFramingArtifact	PUBLIC / ADVANCED	R6
ConstraintsReusablePiArtifact	PUBLIC / ADVANCED	R6
PiReusableConstructionInputs	PUBLIC / ADVANCED	R6
GraphReusableConstructionInputs	PUBLIC / ADVANCED	R6
RegimeReusableClassificationInputs	PUBLIC / ADVANCED	R6
GoverningReusableIdentificationInputs	PUBLIC / ADVANCED	R6
HorizonReusableEvaluationInputs	PUBLIC / ADVANCED	R6
PhiReusableEvaluationInputs	PUBLIC / ADVANCED	R6
PPPReentryInputs	PUBLIC / ADVANCED	R6
MemoryConditionedPPPReentryInputs	PUBLIC / ADVANCED	R6
EvidenceCanonicalArtifactDependency	PUBLIC / ADVANCED	R6
ControlProvenanceArtifactDependency	PUBLIC / ADVANCED	R6
GovernanceReentryAssessment	IMPLEMENTATION / SUPPORT	R3
GovernanceDependencyInvalidationAssessment	IMPLEMENTATION / SUPPORT	R3
GovernanceReentryPlan	IMPLEMENTATION / SUPPORT	R3
GovernanceReentryExecutionResult	IMPLEMENTATION / SUPPORT	R3
GovernanceReusableArtifactBundleAssessment	IMPLEMENTATION / SUPPORT	R3
JointRecoveryReentryReadinessAssessment	IMPLEMENTATION / SUPPORT	R3
GraphReachabilityGovernanceTrigger	IMPLEMENTATION / SUPPORT	R3
GraphStructuralAdmissionGovernanceTrigger	IMPLEMENTATION / SUPPORT	R3
GraphStructuralRevisionGovernanceTrigger	IMPLEMENTATION / SUPPORT	R3

Symbol	Label	Rule
GraphStructuralCoverageGovernanceTrigger	IMPLEMENTATION / SUPPORT	R3
NovelFunctionAdmissionGovernanceTrigger	IMPLEMENTATION / SUPPORT	R3
RecoveryNecessityAdmissionGovernanceTrigger	IMPLEMENTATION / SUPPORT	R3
RecoveryNecessityRevisionGovernanceTrigger	IMPLEMENTATION / SUPPORT	R3
EpistemicInvalidationOrchestrationAssessment	IMPLEMENTATION / SUPPORT	R3
EpistemicReentryExecutionAssessment	IMPLEMENTATION / SUPPORT	R3
EvidenceAuthorityGovernanceInvalidationAssessment	IMPLEMENTATION / SUPPORT	R3
ControlProvenanceGovernanceInvalidationAssessment	IMPLEMENTATION / SUPPORT	R3

Table 12.6 — Compact rows for runtime-built records

Name and kind	Label	Role and use	Source
GovernanceReentryAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>assess_governance_reentry</code>	<code>models.py:2441</code>
GovernanceDependencyInvalidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>derive_dependency_invalidations</code>	<code>models.py:2517</code>
GovernanceReusableArtifactBundleAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>validate_reusable_artifact_bundle</code>	<code>models.py:2597</code>
JointRecoveryReentryReadinessAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>assess_joint_recovery_reentry_readiness</code>	<code>models.py:2564</code>
GraphReachabilityGovernanceTrigger — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>refresh_graph_reachability_and_plan_reentry</code>	<code>models.py:2785</code>
GraphStructuralAdmissionGovernanceTrigger — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>admit_graph_transformation_and_plan_reentry</code>	<code>models.py:2865</code>
GraphStructuralRevisionGovernanceTrigger — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>revise_graph_transformation_and_plan_reentry</code>	<code>models.py:2854</code>

Name and kind	Label	Role and use	Source
GraphStructuralCoverageGovernanceTrigger — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>assess_graph_structural_coverage_and_plan_reentry</code>	models.py:2926
NovelFunctionAdmissionGovernanceTrigger — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>admit_novel_function_and_plan_reentry</code>	models.py:3303
RecoveryNecessityAdmissionGovernanceTrigger — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>admit_recovery_necessity_and_plan_reentry</code>	models.py:3338
RecoveryNecessityRevisionGovernanceTrigger — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>revise_recovery_necessity_and_plan_reentry</code>	models.py:3377
EpistemicInvalidationOrchestrationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>orchestrate_epistemic_invalidations_reentry</code>	models.py:3084
EpistemicReentryExecutionAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>execute_epistemic_reentry_plan</code>	models.py:3102
EvidenceAuthorityGovernanceInvalidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>derive_evidence_authority_governance_invalidations</code>	models.py:3071
ControlProvenanceGovernanceInvalidationAssessment — dataclass (frozen)	IMPLEMENTATION / SUPPORT	Result of <code>derive_control_provenance_governance_invalidations</code>	models.py:3457

Chapter 13 — Testing Utilities

The testing module holds diagnostics for test worlds and integration harnesses. Its fifteen exports—seven functions and eight records—are exported at the top level of `pvpp_runtime`. Every one of them observes; none decides. This chapter gives each utility with the one statement that matters most for it: what it does not establish. It then documents how to verify a runtime tree against the frozen v0.141 hashes and how to run the frozen suite, the benchmarks, and the examples, with the results obtained for this Reference.

A reader must know what each utility measures and what it does not establish.

13.1 The Role of the Testing Module

The module's rule is diagnostics, not certification. Each utility compares, classifies, or searches under a model the caller supplies, and returns a frozen record whose notes restate its limits. A test world may use these results to decide whether a benchmark passed; the runtime never uses them to decide anything. A result that reaches a decision operator—for example, an oracle path used to seed Π —would break the separation between benchmark ground truth and the perceived state a decision may use (C45).

The supervision layer of build 167 has its own test harness—a deterministic scheduler, injected sources and controllers, a crash injector, and a read-only audit oracle—documented with the reference adapters in 16.13.

13.2 Sequence Diagnostics

Entry — `analyze_sequence`

Field	Content
Name and label	<code>analyze_sequence</code> — function; <code>testing.py:123</code> . PUBLIC / ADVANCED (rule R2 (diagnostic)).
Purpose	Counts switches and distinct items in a sequence and detects a repeating tail.
Framework basis	Testing and validation discipline (C45).
Signature	<code>analyze_sequence(items: Iterable[Any], *, max_period: int = 12, min_repetitions: int = 3) → SequenceDiagnostics</code>
Preconditions	Any iterable of hashable items. <code>max_period</code> (default 12) bounds the period searched; <code>min_repetitions</code> (default 3) sets how many repetitions count.
Result	<code>SequenceDiagnostics</code> : item count, switch count, distinct count, the shortest repeating period found at the end of the sequence and its repetitions, and <code>strict_cycle_detected</code> .
Invariants	The search starts at period 1 and stops at the first period that repeats at least <code>min_repetitions</code> times at the tail.
Failure behavior	None.

Field	Content
Does not	Call repetition pathological, penalize oscillation, or affect selection. Feed anything back into Graph, Π , Constraints, Adequacy, Σ , ε , or Layer 1. No decision operator reads its result.
Evidence	Direct use: test files 2, benchmarks 1; not named in doc 05; e.g. <code>three_domain_buffered_multistep_scenario_v048.py</code> .
See also	<code>test_three_domain_oscillation_stress_v047</code> .

Record — SequenceDiagnostics

Field	Content
Label	IMPLEMENTATION / SUPPORT (rule R3)
Fields	frozen dataclass; item_count: int; switch_count: int; distinct_item_count: int; repeated_period: int None = None; repeated_period_repetitions: int = 0; strict_cycle_detected: bool = False; notes: tuple[str, ...] = ()
Source	testing.py:22
Evidence	Direct use: no direct use in tests, benchmarks, or examples; not named in doc 05.

13.3 Bounded Viability Oracle

Entry — bounded_viability_oracle

Field	Content
Name and label	<code>bounded_viability_oracle</code> — function; testing.py:49. PUBLIC / ADVANCED (rule R2 (diagnostic)).
Purpose	Searches a small deterministic test world breadth-first for a path that stays viable through a bounded horizon.
Framework basis	Testing and validation discipline (C45).
Signature	<code>bounded_viability_oracle(initial_state: Any, action_ids: Sequence[str], transition: Callable[[Any, str], Any], viable: Callable[[Any], bool], horizon_steps: int, *, state_key: Callable[[Any], Any] None = None, success: Callable[[Any], bool] None = None, max_visited_states: int = 250000) → BoundedViabilityWitness</code>
Preconditions	An initial state, action ids, a transition function, a viability predicate, and a non-negative horizon. Optional: a state key for deduplication, a success predicate, and <code>max_visited_states</code> (default 250,000).
Result	<code>BoundedViabilityWitness</code> : feasible, the horizon, one action path and state path when feasible, and the visited count.
Invariants	States are deduplicated by (depth, key). A success predicate ends the search at the first state that satisfies it.
Failure behavior	Raises <code>ValueError</code> for a negative horizon and <code>RuntimeError</code> when <code>max_visited_states</code> is exceeded.

Field	Content
Does not	Certify a PV-PP policy, prove that the runtime should have found the path, or say anything outside the supplied test model. Feed anything back into Graph, Π , Constraints, Adequacy, Σ , ϵ , or Layer 1. No decision operator reads its result.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. three_domain_oscillation_scenario_v047.py.
See also	test_three_domain_oscillation_stress_v047; benchmark three_domain_oscillation_scenario_v047.

Record — BoundedViabilityWitness

Field	Content
Label	IMPLEMENTATION / SUPPORT (rule R3)
Fields	frozen dataclass; feasible: bool; horizon_steps: int; action_path: tuple[str, ...] = (); state_path: tuple[Any, ...] = (); visited_state_count: int = 0; notes: tuple[str, ...] = ()
Source	testing.py:7
Evidence	Direct use: no direct use in tests, benchmarks, or examples; not named in doc 05.

13.4 Multidomain Stress Classification

Entry — classify_multidomain_stress

Field	Content
Name and label	classify_multidomain_stress — function; testing.py:150. PUBLIC / ADVANCED (rule R2 (diagnostic)).
Purpose	Classifies the outcome of a multidomain stress run from the caller's flags and the run's history.
Framework basis	Testing and validation discipline (C45).
Signature	classify_multidomain_stress(*, structurally_feasible: bool, collapsed: bool, recovered: bool, margins: Sequence[Mapping[str, float]], policies: Sequence[str], governing_sets: Sequence[Sequence[str]]) → MultidomainStressClassification
Preconditions	Keyword arguments: structurally_feasible, collapsed, recovered, at least one margin snapshot, the selected policies, and the governing sets.
Result	MultidomainStressClassification: one of "recovery", "forced_or_observed_collapse", "collapse_despite_viable_alternative", "periodic_survival_cycling_with_viable_recovery_path", "structurally_forced_periodic_survival_cycling", or "bounded_nonterminal_survival"; the minimum initial and final margins; and sequence diagnostics for policies and governing sets.

Field	Content
Invariants	Oscillation is reported only when both the policy and governing sequences cycle and the run neither recovered nor collapsed. Whether an alternative was structurally feasible comes from the caller, typically from the oracle of 13.3.
Failure behavior	Raises ValueError with no margin snapshots.
Does not	Define a regime, or label oscillation an architectural failure without structural-feasibility evidence. Feed anything back into Graph, Π , Constraints, Adequacy, Σ , ε , or Layer 1. No decision operator reads its result.
Evidence	Direct use: test files 1, benchmarks 2; not named in doc 05; e.g. three_domain_buffered_multistep_scenario_v048.py, three_domain_oscillation_scenario_v047.py.
See also	Benchmarks three_domain_oscillation_scenario_v047 and three_domain_buffered_multistep_scenario_v048.

Record — MultidomainStressClassification

Field	Content
Label	IMPLEMENTATION / SUPPORT (rule R3)
Fields	frozen dataclass; outcome: str; structurally_feasible: bool; collapsed: bool; recovered: bool; oscillatory: bool; minimum_margin_initial: float; minimum_margin_final: float; policy_diagnostics: SequenceDiagnostics; governing_diagnostics: SequenceDiagnostics; notes: tuple[str, ...] = ()
Source	testing.py:34
Evidence	Direct use: no direct use in tests, benchmarks, or examples; not named in doc 05.

13.5 Contaminated-Substrate Paired Assessment

Entry — compare_contaminated_substrate_cycles

Field	Content
Name and label	compare_contaminated_substrate_cycles — function; testing.py:225. PUBLIC / ADVANCED (rule R2 (diagnostic)).
Purpose	Compares a reference cycle with a cycle run on epistemically contaminated inputs.
Framework basis	Testing and validation discipline (C45); Π completeness under substrate integrity (C15/D12).
Signature	compare_contaminated_substrate_cycles(reference_cycle, contaminated_cycle, *, materially_relevant_reference_class_ids=()) → ContaminatedSubstratePairedAssessment

Field	Content
Preconditions	Two completed or partial cycle results. The caller builds the pair: the same world, different epistemic state. Optionally, the reference policy classes that are materially relevant.
Result	ContaminatedSubstratePairedAssessment: policy classes and visible graph seeds on each side, omissions and extras, Π completeness on each side, whether adequacy, selection, or G diverged, and a status.
Invariants	The status is the first that applies: "material_omission_with_apparent_completeness" (a material reference class is missing yet the contaminated Π passed completeness), "contaminated_extra_structure_observed", "contaminated_omission_observed", "downstream_epistemic_divergence_observed", or "no_structural_or_downstream_divergence_observed".
Failure behavior	None; missing parts of a partial cycle compare as empty.
Does not	Treat the reference cycle as runtime input, prove a runtime error from an omission, or prove fabrication from an extra. Feed anything back into Graph, Π , Constraints, Adequacy, Σ , ϵ , or Layer 1. No decision operator reads its result.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_contaminated_pair_v053.py.
See also	test_contaminated_substrate_paired_diagnostics_v053; benchmark_contaminated_pair_v053.

Record — ContaminatedSubstratePairedAssessment

Field	Content
Label	IMPLEMENTATION / SUPPORT (rule R3)
Fields	frozen dataclass; status: str; reference_policy_class_ids: tuple[str, ...] = (); contaminated_policy_class_ids: tuple[str, ...] = (); omitted_reference_class_ids: tuple[str, ...] = (); extra_contaminated_class_ids: tuple[str, ...] = (); reference_graph_seed_ids: tuple[str, ...] = (); contaminated_graph_seed_ids: tuple[str, ...] = (); omitted_graph_seed_ids: tuple[str, ...] = (); extra_graph_seed_ids: tuple[str, ...] = (); reference_pi_complete: bool None = None; contaminated_pi_complete: bool None = None; apparent_completeness_launders: bool = False; adequacy_diverged: bool = False; selection_diverged: bool = False; governing_diverged: bool = False; reference_selected_policy_id: str None = None; contaminated_selected_policy_id: str None = None; notes: tuple[str, ...] = ()
Source	testing.py:183
Evidence	Direct use: no direct use in tests, benchmarks, or examples; not named in doc 05.

13.6 Projection Attribution and Causal-State Comparison

Two utilities compare projection-related state across two cycles. Both report co-occurrence, not cause.

Entry — `compare_projection_causal_state_across_cycles`

Field	Content
Name and label	<code>compare_projection_causal_state_across_cycles</code> — function; <code>testing.py:324</code> . PUBLIC / ADVANCED (rule R2 (diagnostic)).
Purpose	Compares the projection causal inputs of two perceived decision states.
Framework basis	Projection causal-input licensing (9.10); Testing and validation discipline (C45).
Signature	<code>compare_projection_causal_state_across_cycles(prior, current, *, declaration=None) → ProjectionCausalStateCrossCycleAssessment</code>
Preconditions	Two <code>PerceivedDecisionState</code> objects with numeric times; optionally a <code>ProjectionCausalStateConsistencyDeclaration</code> naming inputs the host knows are stale and groups it knows are contradictory.
Result	<code>ProjectionCausalStateCrossCycleAssessment</code> : appeared, disappeared, updated, and unchanged inputs; inputs whose own represented time went backward; inputs whose value changed at the same represented time; declared stale inputs present; declared contradictions present; and a status joining the flags with "+".
Invariants	Violations: a different actor, time running backward, or duplicate input ids. Staleness and contradiction are reported only when declared by the host; values are compared by repr.
Failure behavior	Raises <code>ValueError</code> for non-numeric times; otherwise returns violations.
Does not	Infer staleness or contradiction from opaque values, or treat appearance, disappearance, or a timestamped change as an error. Feed anything back into Graph, Π , Constraints, Adequacy, Σ , ϵ , or Layer 1. No decision operator reads its result.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. <code>benchmark_projection_causal_state_cross_cycle_v061.py</code> .
See also	<code>test_projection_causal_state_cross_cycle_v061</code> .

Record — `ProjectionCausalStateConsistencyDeclaration`

Field	Content
Label	PUBLIC / ADVANCED (rule R6: built by the host)
Fields	frozen dataclass; <code>stale_input_ids: tuple[str, ...] = ()</code> ; <code>contradiction_groups: tuple[tuple[str, ...], ...] = ()</code> ; <code>notes: tuple[str, ...] = ()</code>
Source	<code>testing.py:293</code>
Evidence	Direct use: test files 1; not named in doc 05; e.g. <code>test_projection_causal_state_cross_cycle_v061.py</code> .

Entry — `capture_projection_attribution_snapshot`

Field	Content
Name and label	<code>capture_projection_attribution_snapshot</code> — function; testing.py:458. PUBLIC / ADVANCED (rule R2 (diagnostic)).
Purpose	Captures comparison signatures of one completed cycle's projection inputs and outputs.
Framework basis	Testing and validation discipline (C45).
Signature	<code>capture_projection_attribution_snapshot(perceived, cycle) → ProjectionAttributionSnapshot</code>
Preconditions	The perceived decision state used for the cycle and a cycle result with projection records.
Result	<code>ProjectionAttributionSnapshot</code> : signatures of the represented state, causal inputs, graph context, candidate set, horizons, model authority, and projected content.
Invariants	The projected-content signature excludes state identity, model, and horizon, so a content change is separated from a provenance change. Signatures are opaque structural comparisons, not semantic hashes.
Failure behavior	Raises <code>ValueError</code> without a perceived state or without projection records.
Does not	Measure model quality or state truth. Feed anything back into Graph, Π , Constraints, Adequacy, Σ , ϵ , or Layer 1. No decision operator reads its result.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. <code>benchmark_projection_attribution_v063.py</code> .
See also	<code>test_projection_cross_cycle_attribution_v063</code> ; <code>benchmark_projection_attribution_v063</code> .

Entry — `compare_projection_attribution_across_cycles`

Field	Content
Name and label	<code>compare_projection_attribution_across_cycles</code> — function; testing.py:522. PUBLIC / ADVANCED (rule R2 (diagnostic)).
Purpose	Reports which authoritative-input classes changed alongside a change in projections.
Framework basis	Testing and validation discipline (C45).
Signature	<code>compare_projection_attribution_across_cycles(prior, current) → ProjectionCrossCycleAttributionAssessment</code>
Preconditions	Two snapshots, the second not earlier than the first.
Result	<code>ProjectionCrossCycleAttributionAssessment</code> : whether projections changed, which of represented state, causal inputs, graph context, candidate set, horizon, and model authority changed, and a status.

Field	Content
Invariants	A projection change with no observed input or model change is flagged as a determinism anomaly, because Q is deterministic for identical authoritative inputs and model version.
Failure behavior	Raises ValueError if the current snapshot precedes the prior one.
Does not	Infer cause from co-change, or require that a changed input change Q. Feed anything back into Graph, Π , Constraints, Adequacy, Σ , ϵ , or Layer 1. No decision operator reads its result.
Evidence	Direct use: test files 1, benchmarks 1; not named in doc 05; e.g. benchmark_projection_attribution_v063.py.
See also	As above.

13.7 Runtime Identity Verification

V0167_FILE_HASHES.sha256, at the root of runtime-v2.1.167, lists SHA-256 hashes for 346 files: the 25 files of the pvpp_runtime package (build 141's 11 and the 14 of pvpp_runtime/supervision), pyproject.toml, the 170 files in tests (168 test files and two fixtures), the 54 benchmark files, and, unlike build 141's manifest, the 27 example files, the 10 shipped docs, and 59 release records at the root. From the runtime-v2.1.167 directory:

Table 13.1 — Verifying the frozen tree

Step	Command	Expected result
Hashes	sha256sum -c V0167_FILE_HASHES.sha256 --quiet	No output and exit status 0
Decision core	sha256sum -c V0141_FILE_HASHES.sha256 --quiet	One line, pyproject.toml: FAILED; every other build-141 file matches
Version	grep version pyproject.toml	version = "0.167"; requires-python = ">=3.11"

Build 167's hash set covers the examples, docs, README.md, RELEASE_NOTES.md, and the build notes and records, which build 141's did not; their v2.1.167 copies are part of the frozen artifact, though they remain documentation and host code, not runtime behavior (15.8). README.md still names the lineage record V2_1_BASELINE_AND_LINEAGE.md; the file in the tree is BASELINE_AND_LINEAGE.md.

For this edition the hashes were checked on 7 October 2026, on a fresh extraction of the frozen archive (SHA-256

7359f88ba83004af078e1f0cd26f1bb045bbb137c96e70044c3e0eb93087a85e): all 346 matched, and 205 of the 206 build-141 entries matched, as expected.

13.8 Running the Frozen Suite, Benchmarks, and Examples

The suite needs Python 3.11 or later and pytest. Run it without writing caches into the frozen tree: PYTHONDONTWRITEBYTECODE=1 suppresses __pycache__, and -p no:cacheprovider suppresses .pytest_cache. Benchmarks and examples are scripts; run them with python3 -B and

with the runtime root, and for benchmarks the tests directory, on PYTHONPATH, because several benchmarks import test fixtures.

Table 13.2 — Reproduction commands and results (7 October 2026, Python 3.13.16)

Run	Command, from runtime-v2.1.167	Result
Frozen suite	PYTHONDONTWRITEBYTECODE=1 python3 -m pytest -q -p no:cacheprovider	1320 passed, matching README.md and RELEASE_NOTES.md; also 1320 passed with -W error
Small Economy benchmark	PYTHONPATH=.:tests python3 -B benchmarks/benchmark_small_economy_v2_1_v0140.py	Output identical to BENCHMARK_SMALL_ECONOMY_v0_140.txt, ending "PASS: V2.1 Small Economy objective perturbation benchmark"
All benchmark scripts	PYTHONPATH=.:tests python3 -B benchmarks/<file>.py, for each of the 54 files	51 exit 0; 3 fail (Table 13.3)
Example scripts	PYTHONPATH=. python3 -B examples/<path>.py	All 17 run scripts exit 0
Supervision tests	PYTHONDONTWRITEBYTECODE=1 python3 -m pytest -q -p no:cacheprovider tests/test_r22_*.py tests/test_r3_*.py	227 passed in 28 files (Table E.6)
Example tests	PYTHONDONTWRITEBYTECODE=1 python3 -m pytest -q -p no:cacheprovider examples	6 passed

Three benchmark scripts no longer run against v0.141, or against v2.1.167. Each predates a guard that a later build added; the guard refuses the benchmark's setup, which is the runtime behaving as documented. They are in the hash set, so they are part of the frozen tree, but they are not part of the regression suite and their historical results do not transfer to v0.141 (Chapter 15).

Table 13.3 — Benchmarks that fail against v0.141

Script	Error	Later guard
benchmark_epsilon_v022.py	instantiate_execution: requires a runtime-recognized current canonical license	License lineage (v0.131; 11.1): hand-built licenses are refused
benchmark_sigma_standard_v024.py	evaluate_sigma_standard: Stage 3 requires an explicit OrderIndex for every survivor	Σ order requirement (3.11; 9.11)

Script	Error	Later guard
benchmark_epsilon_mode_license_v037.py	build_execution_license_from_cycle: stale or identity-inconsistent cycle artifacts	Fresh artifact-integrity audit at licensing (v0.64; 9.12, 11.2)

Running any of these commands left the tree unchanged: git status in the repository showed no modification afterward. The repository also tracks some `__pycache__` directories; they are part of its committed state.

13.9 Classification Record

The seven functions are PUBLIC / ADVANCED under R2 as diagnostic surfaces; tests and benchmarks use each one, and doc 05 names none. The declaration the host builds takes the same label; the records the functions return are IMPLEMENTATION / SUPPORT.

Table 13.4 — Labels assigned in Chapter 13

Symbol	Label	Rule
analyze_sequence	PUBLIC / ADVANCED	R2 (diagnostic)
bounded_viability_oracle	PUBLIC / ADVANCED	R2 (diagnostic)
classify_multidomain_stress	PUBLIC / ADVANCED	R2 (diagnostic)
compare_contaminated_substrate_cycles	PUBLIC / ADVANCED	R2 (diagnostic)
compare_projection_causal_state_across_cycles	PUBLIC / ADVANCED	R2 (diagnostic)
capture_projection_attribution_snapshot	PUBLIC / ADVANCED	R2 (diagnostic)
compare_projection_attribution_across_cycles	PUBLIC / ADVANCED	R2 (diagnostic)
ProjectionCausalStateConsistencyDeclaration	PUBLIC / ADVANCED	R6
SequenceDiagnostics	IMPLEMENTATION / SUPPORT	R3
BoundedViabilityWitness	IMPLEMENTATION / SUPPORT	R3
MultidomainStressClassification	IMPLEMENTATION / SUPPORT	R3
ContaminatedSubstratePairedAssessment	IMPLEMENTATION / SUPPORT	R3
ProjectionCausalStateCrossCycleAssessment	IMPLEMENTATION / SUPPORT	R3
ProjectionAttributionSnapshot	IMPLEMENTATION / SUPPORT	R3

Symbol	Label	Rule
ProjectionCrossCycleAttributionAssessment	IMPLEMENTATION / SUPPORT	R3

Chapter 14 — Statuses, Errors, and Hostile Invariants

The earlier chapters give each status, exception, and refusal where it arises. This chapter collects them. It explains the four ways v0.141 reports failure, lists every status string by family with the section that documents it, lists the exceptions the runtime raises, describes the three forms validators return, and catalogs the hostile invariants—the misuse the runtime refuses—each with a test. It closes with a procedure for tracing any refusal to the stage and record that explain it.

A reader who receives any status or exception must be able to find what it means, which stage produced it, and what to do next.

14.1 How Failure Is Reported

v0.141 separates two kinds of failure. A gate that fails on the facts—no feasible candidate, an invalid frame, an unsatisfiable joint-recovery set—is an outcome, reported as a status in a result. Misuse of the API or a broken host contract—a missing hook, a copied license, a record for the wrong policy—is an error, raised as an exception. Table 14.1 gives the four forms.

Table 14.1 — Forms of failure report

Form	Used for	What the caller does
Status in a result (stopped_at and status)	A canonical gate failing on the facts; a cycle, sequence, or ϵ episode ending short of its goal	Read the status and the stage record; the facts, not the call, must change
Exception	A violated precondition, a host record that breaks its contract, forged or stale authority, an internal invariant	Fix the call or the host code; do not retry unchanged
Validator report	A validate_, assess_, or audit_ call; the caller decides what to do with the result	Read violations; see 14.9 for the three return forms
Invalid planning or executor result (valid false)	Re-entry planning, bundles, executors, change wrappers, triggers	Read status and violations; nothing was executed or changed beyond what the result reports

Messages carried by exceptions and violation lists name the failed condition, but their text is not part of the API (1.7). Match on the exception type, the status string, and the stage; do not parse message text.

14.2 Stage Status Vocabulary

Table 14.2 lists the statuses the canonical stages set on their result records, in stack order. Where a status stops the canonical cycle, CanonicalDecisionCycleAssessment carries it with stopped_at naming the stage (Table 9.2).

Table 14.2 — Stage statuses

Stage and record	Statuses	Section
$\mathcal{R}$: RegimeAssessment trigger codes (not statuses; the regime labels are Mission, Stabilization, Survival, Existential)	"contextual_interruption", "governing_min_horizon_existential", "governing_max_pressure_existential", "governing_min_horizon_survival", "governing_max_pressure_survival", "governing_multiplicity_survival", "governing_min_horizon_stabilization", "governing_max_pressure_stabilization", "governing_multiplicity_stabilization", "mission_default", "hysteresis_escalation_monotonicity", "hysteresis_deescalation_hold"	3.4; 9.7
Graph: GraphConstructionAssessment.status	"PASS", "CONDITIONAL PASS", "FAIL"; failure codes G-001 to G-005, G-007, and G-010	8.2; Table 8.1
Graph: transformation reachability	"reachable", "unreachable", "uncertain"	Table 8.2
Π: PiConstructionAssessment.status	"pi_constructed", "pi_blocked_by_graph_invalidity", "pi_boundedness_overflow", "pi_missing_mandatory_continuation"	3.5; 9.8
Π completeness: PiCompletenessAssessment.status	"pi_completeness_passed", "pi_completeness_failed_missing_material_policy_classes"	3.6; 9.8
Π completeness, substrate-qualified: QualifiedPiCompletenessAssessment	"qualified_pi_pass", "qualified_pi_conditional_warning", "qualified_pi_fail", "qualified_pi_fail_with_layered_attribution"; the substrate qualification it reads is "clean", "uncertain", "epistemically_contaminated"	9.8
Constraints: ConstraintsAssessment.status	"constraints_passed_all_candidates", "constraints_filtered_candidates", "constraints_no_feasible_candidates"	3.7; 9.9
Domain Framing: DomainFramingAssessment.status	"domain_framing_passed", "domain_framing_failed"	3.8; 9.9

Stage and record	Statuses	Section
Joint recovery: JointRecoveryFeasibilityAssessment.status	"no_active_recovery_corridors", "active_corridors_have_no_represented_capacity_demands", "active_capacity_authority_required", "active_jointly_feasible", "active_jointly_infeasible"	Table 5.2
Joint recovery: cycle stop statuses	"joint_recovery_capacity_authority_required", "joint_recovery_feasibility_failed"	Table 9.2
Joint recovery: execution binding	"current_period_joint_recovery_bound", "no_current_period_joint_recovery_binding"	3.9
Adequacy: AdequacyAssessment.status	"adequacy_passed_some_policies", "adequacy_no_recovery_capable_policy"	3.10; 9.11
Σ standard and terminal: selection status	"sigma_standard_policy_selected", "sigma_terminal_policy_selected"	3.11; 9.11
Σ fallback: SigmaFallbackAssessment.status	"fallback_singleton_maximal_selected", "fallback_stage1_selected", "fallback_stage2_override_selected", "fallback_stage3_selected", "fallback_stage4_tie_selected", "fallback_multiway_override_not_formalized"	3.11; 15.1
Σ: selection modes (SigmaAssessment.mode; license selection_mode)	"standard", "recovery_unavailable_fallback", "terminal_infeasibility"	Table 3.14; Table 11.2

"fallback_multiway_override_not_formalized" is the one Σ status without a selection: fallback Σ met an unresolved conflict among three or more candidates and stopped rather than invent a rule (15.1). The integrated cycle then reports
 "integrated_cycle_stopped_before_unique_sigma_selection".

14.3 Whole-Cycle and Sequence Statuses

Table 14.3 — Integrated, memory-conditioned, and sequence statuses

Entry	Statuses	Section
evaluate_integrated_canonical_cycle	"integrated_cycle_stopped_before_unique_sigma_selection", "integrated_cycle_selected_not_executed", "integrated_cycle_execution_blocked", "integrated_cycle_execution_active", "integrated_cycle_transition_applied", "integrated_cycle_transition_invalid"	Table 9.3

Entry	Statuses	Section
evaluate_memory_conditioned_integrated_cycle	"memory_cycle_stopped_before_unique_sigma_selection", "memory_cycle_selected_not_executed", "memory_cycle_execution_blocked", "memory_cycle_execution_active", "memory_cycle_transition_applied", "memory_cycle_transition_and_epistemic_update_applied"	9.5
evaluate_canonical_cycle_sequence	"sequence_completed", "sequence_stopped_at_active_epsilon", "sequence_invariant_failure"	9.5
evaluate_memory_conditioned_cycle_sequence	"memory_sequence_completed", "memory_sequence_stopped_at_active_epsilon", "memory_sequence_invariant_failure"	9.5

"integrated_cycle_transition_invalid" is declared but cannot occur. The integrated cycle applies the transition through `apply_layer1_transition`, which raises `ValueError` on an invalid result before the status is chosen. An invalid host transition therefore reaches the caller as an exception, as it does on the memory-conditioned path, which declares no such status. Recorded in Chapter 15.

14.4 Execution Vocabulary

Table 14.4 — Execution statuses and states

Vocabulary	Values	Section
ϵ status (<code>EpsilonStepResult.status</code>)	"active", "completed", "completed_staged", "partial_realization", "aborted_return", "failed", "emergency_return"	Table 11.4
License ledger	"issued", "instantiated"	11.2–11.3
Episode ledger	"current", "inactive", "terminal", "invalidated"	11.3–11.6
Native authorization status	"issued", "consumed", "invalidated"	Table 11.6
Native outcome (<code>NativeExecutionResult.status</code>)	"succeeded", "failed_cleanly", "indeterminate", "completed_invalid"	Table 11.8
Native failure_stage	"local_callable", "transport", "external_unknown", "validation"	Table 11.8
Control state	"ready", "paused", "aborted", "reauthorization_required"	Table 11.9

14.5 Structural, Information, and Change Statuses

Table 14.5 — Statuses of structural, information, and change surfaces

Surface	Values	Section
Graph transformation admission, novel function, recovery necessity admission	"ADMITTED", "REJECTED"	8.5–8.7
Graph transformation and recovery necessity revision	"REVISED", "REJECTED"	8.5, 8.7
Reachability refresh	"APPLIED", "REJECTED"	8.4
Structural coverage observation (coverage_status)	"sufficient", "incomplete", "uncertain"	8.8
Structural coverage disposition	"coverage_sufficient", "admission_definition_required", "unsupported_uncertain", "invalid"	8.8
Control-provenance topology update	"UPDATED", "NO_CHANGE", "REJECTED"	6.8
Information disposition	"current", "rejected", "superseded", "defective", "prohibited_for_use", "historically_valid_noncurrent", "unresolved", "contradicted"	6.1
Evidence authority loss (loss_reason, in precedence order)	"source_compromised", "source_revoked", "source_stale", "source_controlled_by_governed_agent", "source_controlled_by_descendant", "source_expired", "source_authority_unresolved"	6.7
Objective status	"active", "suspended", "completed", "abandoned", "expired"	7.2
Objective responsiveness relation	"supports", "neutral", "conflicts", "unresolved"	7.4

14.6 Re-entry Statuses

Table 14.6 — Re-entry statuses

Record	Values	Section
GovernanceReentryExecutionResult, all executors	"reentry_pass_completed", "reentry_pass_stopped", and per boundary "invalid_<stage>_reentry", "<stage>_reentry_not_executable", and, where the pass re-projects, "<stage>_reentry_projection_authority_failed"	12.5

Record	Values	Section
Stage names in those statuses	"sigma", "adequacy", "domain_framing", "joint_recovery", "constraints", "pi_completeness", "pi", "graph", "regime", "governing", "horizon", "phi", "ppp", "memory_conditioned_ppp"; projection-authority failure exists for domain_framing, joint_recovery, constraints, and pi_completeness	Table 12.2
Graph trigger executors	"graph_reachability_reentry_not_executable", "graph_structural_admission_reentry_not_executable"	12.6
execute_governance_reentry	"no_reentry_required", "reentry_pass_completed", "invalid_reentry_plan", "partial_stage_reentry_not_yet_supported"	12.3
assess_joint_recovery_reentry_readiness	"joint_recovery_reentry_ready", "joint_recovery_reentry_dependency_scope_insufficient"	12.8
Decision status when an executor stops at the joint-recovery gate	"active_capacity_authority_required", "active_jointly_infeasible", the joint-recovery record's own status, not the cycle stop status; "joint_recovery_infeasible" is tested for but never produced (defect K2)	12.5; 15.2
execute_epistemic_reentry_plan	"epistemic_reentry_dispatched", "epistemic_reentry_executor_rejected", "epistemic_reentry_not_executable"	12.7

The testing utilities have their own diagnostic statuses (Chapter 13). Statuses produced only by the LEGACY / COMPATIBILITY evaluators and the older assess path—for example "unique_feasible_adequate_policy_set", "no_adequate_policy_set", "registered_structure_filtered_policy_sets", and "resolved_by_governing_consequences"—are listed with those entry points in Chapter 15 and never appear in a canonical cycle result.

14.7 Exceptions

v0.141 raises only built-in exception types; it defines no exception hierarchy of its own. The three exception classes it does define—CleanExecutionFailure, IndeterminateExecutionFailure, and CompletedInvalidExecution—are raised by host bindings, not by the runtime, and are converted into native outcomes (11.7).

Table 14.7 — Exceptions raised by v0.141

Type	Raised for	Examples
ValueError	Almost every refusal: unmet preconditions, invalid host records, identity mismatches, forged or stale licenses and episodes, invalid frames or objective sets, projection licensing failures, record mutation, integrity failures, duplicate registrations	367 raise sites across kernel.py, registry.py, execution.py, reentry.py, feasibility.py, and testing.py
RuntimeError	A host-contract or lifecycle failure: "steady" not projectable; an authorization already consumed or invalidated; a non-ready control state; the deprecated ExecutionBindingRegistry.invoke; a missing TransferHistoryService; the G invariant "horizon seed is empty"; the oracle's visit limit	kernel.py, execution.py, testing.py
KeyError	An unknown native authorization, or an action with no registered binding	Native execution (11.6–11.7)
TypeError	An object of the wrong type where an episode, authorization, native result, telemetry, or callable binding is required	Native execution (11.5–11.8)
TypeError from Python itself	The PPP branch of execute_epistemic_reentry_plan (12.7)	Defect K1 (15.2), not a deliberate refusal

Missing optional services are not reported uniformly: most raise ValueError ("no ... is attached"), but record_transfer_history raises RuntimeError("transfer_history_service required"). Catch both where a service may be absent.

Table 14.7 describes the decision core and is unchanged in v2.1.167. The supervision layer defines eleven exception classes of its own and also raises PermissionError, KeyError, ValueError, and, in three places, sqlite3.IntegrityError; 16.11 lists them and says how to handle each.

14.8 Which Calls Raise and Which Return

Canonical stage gates return statuses; stage-call preconditions raise. Planning, re-entry, and change wrappers never raise for invalid inputs: they return a result with valid false and violations. Native invocation converts every exception from the host callable into a result, but raises for its own checks before the callable is entered. The integrated cycle raises for invalid prior provenance and for errors of the calls it composes, and returns statuses for gate outcomes.

14.9 Validation Failures

Validators return one of three forms. Code that treats every validator as returning an assessment will misread the second form.

Table 14.8 — Validator return forms

Form	Validators	Invalid when
Assessment record with valid, violations, and notes	All other validate_, assess_, and audit_ methods	valid is false
Tuple of violation strings	validate_transfer_history_event, validate_memory_state_reference, validate_memory_retrieval_package	The tuple is non-empty. An empty tuple means valid
The validated object, or an exception	validate_projection_record	ValueError is raised

One assessment type is reused: validate_actual_persistent_state returns a Layer1TransitionValidationAssessment, the same record apply_layer1_transition uses for a transition result.

14.10 Hostile Invariants

A hostile invariant is a refusal of misuse that could otherwise pass for legitimate use: forged or reused authority, spliced evidence, laundered invalidity, incomplete state, and downstream repair. v0.141's tests exercise several hundred such refusals; Table 14.9 lists the principal ones with a demonstrating test each.

Table 14.9 — Hostile invariants

Misuse	Refusal	Test
Copy or hand-build an execution license	instantiate_execution refuses a license the runtime did not issue	test_v2_canonical_lineage_authority_v0131: test_v0131_canonical_license_fabrication_rejected_at_epsilon
Fabricate or copy an episode to mint native authority	issue_native_execution_authorization refuses	test_v2_canonical_lineage_authority_v0131: test_v0131_reconstructed_genuine_episode_is_non_authorizing
Replay a consumed authorization	RuntimeError; the callable is not entered	test_v2_authority_bound_callthrough_v0130: test_abct05_replay_rejected
Race two invocations of one authorization	At most one enters the callable	test_v2_authority_bound_callthrough_v0130: test_abct21_concurrent_replay_race_at_most_one_call

Misuse	Refusal	Test
Invoke through a context with authority metadata	The public invoke fails closed	test_v2_authority_bound_callthrough_v0130: test_abct17_metadata_smuggling_has_no_authority
Execute a terminal-infeasibility selection	The license blocks entry	test_mode_aware_epsilon_license_v037: test_terminal_sigma_selection_produces_blocked_not_laundered_license
Relabel license flags to launder a fallback	Mode-aware checks refuse	test_mode_aware_epsilon_license_v037: test_mode_flags_cannot_launder_invalid_fallback_posture
License from a spliced or stale cycle	The licensing audit re-runs and refuses	test_cycle_artifact_identity_integrity_v064: test_execution_license_reaudits_instead_of_trusting_stored_valid_flag
Splice projection or adequacy sets from another cycle	The artifact audit refuses	test_cycle_artifact_identity_integrity_v064: test_projection_and_adequacy_policy_sets_cannot_be_spliced_from_other_cycle
Mutate a Q record during Adequacy or Σ	ValueError	test_projection_state_and_consistency_v059: test_adequacy_mutation_of_nested_q_record_fails_closed, test_sigma_mutation_of_nested_q_record_fails_closed
Splice a transition onto non-terminal ϵ or another state	The lifecycle audit detects it	test_integrated_cycle_lifecycle_integrity_v069: test_transition_cannot_follow_nonterminal_epsilon, test_prior_state_splice_is_detected

Misuse	Refusal	Test
Splice realized evidence into prediction error or memory update	Evidence signatures refuse	test_execution_evidence_integrity_v067: test_nested_evidence_mutation_after_request_build_is_detected
Start a cycle from a state other than the recorded transition result	Provenance validation refuses before perception	test_execution_transition_provenance_v065: test_next_integrated_cycle_fails_before_perception_on_stale_provenance
Let a Π pass completeness while a material class is missing	Completeness fails; substrate qualification reports contamination	test_contaminated_substrate_paired_diagnostics_v053
Bypass an uncertainty-invalid graph	Π is blocked	test_graph_uncertainty_integrity_v057: test_pi_cannot_bypass_uncertainty_invalid_graph
Admit structure against a stale substrate	The admission is rejected without mutation	test_v2_graph_structural_admission_v0104: test_stale_substrate_fails_without_mutation
Smuggle retrieval or expectation into a non-memory $P_i(t)$	Perceived-state validation refuses	test_perceived_decision_state_boundary_v056: test_nonmemory_pds_cannot_smuggle_retrieval_or_expectation_into_stack
Let ϵ repair upstream invalidity	ϵ returns upstream; it never re-selects	test_epsilon_bounded_realization_v022: test_unlicensed_envelope_cannot_be_laundered_by_epsilon

Misuse	Refusal	Test
Invent objective ordering or prune on responsiveness	Resp records carry no gate; conflicting objectives coexist	test_v2_1_objective_responsiveness_discovery_v0134: test_conflicting_objectives_do_not_create_runtime_ordering_in_resp
Restart re-entry at PPP to avoid a partial boundary	The generic executor refuses	test_v2_bounded_reentry_execution_v084

Refusals the runtime does not make are as important: they are the host obligations and limitations collected in Chapter 15, among them one execution per selection, advancing only the latest ϵ episode, identity-preserving perceive, and bundle currency. A hostile test suite for a host should cover those as well.

14.11 Diagnosing a Refusal

- **Exception or status?** An exception means the call or the host code must change; a status means the facts produced the outcome. Do not retry an exception unchanged.
- **Find the stage.** For a cycle, `stopped_at` names the stage and `pipeline_trace` the stages run; the stage's record on the result (Table 9.4) holds its reasons. For a re-entry, `recompute_from_stage` and the executor status name the boundary.
- **Read the record, not the message.** Violations, notes, failure codes (Table 8.1), and per-candidate results carry the explanation. Messages are for people.
- **Check identity next.** Most refusals after Σ are identity refusals: a copied object, a stale cycle, a second license, a different state. Compare ids and object identity before suspecting logic.
- **Check the host contract.** Use the conformance checklist (Table 10.5) and the host obligations of Chapter 15; many unexplained outcomes come from a hook that returns a different policy id, a non-identity perceive, or a stale bundle.

Chapter 15 — Limitations, Compatibility, and Legacy Entry Points

This chapter states what the runtime does not do and what a reader must not attribute to it. Its registers describe the decision core under the name v0.141 (1.4) and add what build 167 changes. It collects the limitations, the two known defects, and the host obligations found in Chapters 4–14; gives the legacy entry points with their replacements; records the runtime constructs that serve framework rules without a framework owner; compares the three runtime generations; and lists the evidence that does not transfer and the documentation discrepancies. It also closes the reconciliation-register entries that earlier chapters routed here.

A reader must never mistake a framework requirement, a host obligation, or an earlier generation's result for runtime behavior.

15.1 Limitation Register

A limitation is a place where the framework requires or permits something that v0.141 does not implement, or implements only in part. None of them authorizes a framework change: the framework remains the authority, and the limitation belongs to the build (C45/D2 §8).

Table 15.1 — Limitations of the runtime

#	Limitation	Framework position	Runtime behavior
L1	Synchronous execution	The framework permits governance of in-flight change where instrumentation and execution architecture exist; such capability is prospective until a runtime implements it (C45/D2 §8)	Decision core (v0.141, unchanged in v2.1.167): no asynchronous sensing, instrumentation hooks, in-flight change detection, distributed execution, or guaranteed mid-flight cancellation; control states are synchronous pre-invocation gates (11.9). v2.1.167 narrows L1 without removing it: its supervision layer admits host-reported observations, finds affected executions from declared dependencies, governs continuation at host-called checkpoints, and records stop requests that host controllers carry out (Chapter 16). It still senses, schedules, and stops nothing itself, and does not govern a native call once entered (16.12)
L2	Fallback Σ cardinality	Recovery-unavailable fallback selection (C15/D6; parameterization addendum C15/D17); fallback refinement is the historically less locked area (C45)	Resolves singleton and two-candidate cases; stops with "fallback_multiway_override_not_formalized" for unresolved conflicts among three or more candidates rather than invent a rule (14.2). Not authority for a new Σ rule

#	Limitation	Framework position	Runtime behavior
L3	Joint-feasibility scale	Computational and search boundaries are an open question (C00, Open Question D)	Exact search, described in its own code as small-prototype; cost grows quickly with active corridors and calendar periods (5.8)
L4	Execution authority bound to objects, not decisions	ε instantiates the selected policy as an execution episode (C15/D9 §5)	Each license, episode, and native authorization is single-use, but one cycle can yield several executable licenses, episode ids may be reused, and an earlier episode object can still be advanced (11.2–11.4). Ledgers are in memory in one runtime instance and are not persisted
L5	Re-entry is not enforcement-equivalent to a full canonical rerun	No owner file (15.6); upstream invalidity is not laundered downstream (C20/D4); projection licensing (C10/D6)	Executors check projection model authority but not causal-input licensing; their decisions carry no artifact-integrity record yet can be licensed; Adequacy re-entry enters fallback Σ without host structural profiles; <code>execute_governance_reentry</code> executes only PPP boundaries; and, by defect K2, executors at P_i Completeness or earlier do not stop for an infeasible active recovery set (12.3, 12.5, 15.2)
L6	Compatibility projection route	Projections are licensed facts (C10/D6)	The <code>project_policy_record</code> hook receives no finite-horizon, model-authority, causal-input, or record-validation enforcement (10.3)
L7	Declared scope boundaries	Not framework requirements; <code>RELEASE_NOTES.md</code> lists them so they are not inferred from Version 2.1	No reachability-at-scale optimization, autonomous semantic discovery, automatic invention of PP domains, unrestricted self-modification, universal persistent runtime database, or replacement of conventional security and enforcement mechanisms

#	Limitation	Framework position	Runtime behavior
L8	Supervised execution and the canonical Layer-1 bridge	Layer 1 owns the transition and ε ends at realization outputs (C10/D2; C15/D9)	A checkpoint returns a ContinuationAssessment, not the ε step result that build_layer1_transition_handoff requires, so a supervised episode does not reach the Layer-1 transition service through any public return; the layer records the host's own transition instead (16.8). And a checkpoint needs the authorization still issued: after invoke_registered_native, or after fencing, the execution can no longer be checkpointed (16.3)
L9	Supervision resource bounds	Not a framework requirement	No byte-size limit on observations; observation content is stored in more than one record (Table 16.15). A resource boundary, not a correctness defect: the host limits size (16.12)
L10	Durability of live bindings	Not a framework requirement	The bridge's bindings live in memory. After a restart, operations that need live canonical state fail closed for earlier executions, and no public call rebinds them (16.9)

The limitations cluster for a reason. The canonical decision path from $P_i(t)$ through Σ , and the licensing and native-authorization path after it, carry strong identity, lineage, licensing, and hostile-use protections. Enforcement thins, and host responsibility grows, at the surfaces around that path: re-entry, compatibility interfaces, execution bookkeeping, and host-mediated lifecycle boundaries. This is a characterization of the frozen implementation, not a framework limitation. The supervision layer of build 167 follows the same pattern: strong checks on the chain it records, and duties left to the host at its edges (16.12).

15.2 Known Defects

Table 15.2 — Defects in the decision core (builds 131, 141, and 167)

Defect	Effect	Workaround	Found
K1. execute_epistemic_reentry_plan, PPP branch: calls execute_ppp_reentry without horizon_inputs, and its input check omits horizon_inputs	Every argument after ppp_inputs shifts by one; the call raises TypeError (missing pi_inputs). No test covers the branch	Call execute_ppp_reentry directly with orchestration.governance_inv alidation.plan (12.7)	Chapter 12; confirmed against the frozen code

Defect	Effect	Workaround	Found
K2. execute_pi_completeness_reentry, joint-recovery gate: stops for "active_capacity_authority_required" and for "joint_recovery_infeasible", a status no record produces; an infeasible active set is reported as "active_jointly_infeasible"	Re-entry at Π Completeness or any earlier boundary—every earlier executor, including the graph trigger executors, delegates to this one—continues past an infeasible active recovery set to Σ . The decision can be licensed and instantiated. A full cycle on the same inputs stops with "joint_recovery_feasibility_failed". No test covers the case	After such a re-entry, if decision.joint_recovery_feasibility is runtime-verified and not jointly feasible, treat the pass as stopped and do not license the decision; or rerun the canonical cycle (12.5)	Appendix C source scan; confirmed against the frozen code

The defects are recorded, not fixed: the runtime is frozen, and build 167 corrected neither. Both are present, unchanged, in v2.1.167, whose kernel.py and reentry.py are byte-identical to build 141's (1.4); both were present in build 131 as well. K2 matters more than K1: K1 fails loudly, K2 returns a decision that looks valid. The supervision layer does not call either executor: a checkpoint plans re-entry with `assess_governance_reentry` and `plan_governance_reentry` and executes no re-entry pass (16.6). A host that executes the planned re-entry is exposed to both, exactly as in build 141.

15.3 Host Obligations the Runtime Does Not Check

Each obligation is something the host must guarantee because the decision core cannot detect its violation, or detects it only partly. Violating one may raise later, may fail closed, or may produce a result that looks valid and is not; silent success is possible, not certain, and is why each row is listed. The supervision layer's duties, classified by whether the layer fails closed or relies on the host, are in Table 16.31.

Table 15.3 — Host obligations

Obligation	Why	Section
perceive returns an already-represented state unchanged	The cycle re-applies perceive after $P_i(t)$ is validated	10.2
Hooks keep no hidden state and change no host state; domain_value is deterministic within a cycle	The runtime checks only that its own Q records are unchanged	10.10
Validate any objective set passed directly to evaluate_canonical_decision_cycle	Only the snapshot entries validate objective sets	7.2; 9.3

Obligation	Why	Section
Set execute=True only after establishing entry sufficiency	The integrated entry asserts entry_sufficient on the host's behalf	9.3
Use PolicyProjectionService rather than project_policy_record for new integrations	The compatibility route forgoes projection licensing	10.3
Pass runtime_version="0.141" to build_decision_replay_package	The default is "0.137"	6.4
Before removing a graph transformation, revise or remove every composition that references it	No cascade and no refusal; the next construct_graph fails with G-005	8.5
Change registered structure through the planning wrappers once a cycle has run	The bare registry methods plan no re-entry	12.6
Execute a selection at most once	One cycle can yield several executable licenses	11.2
Keep ϵ episode ids unique	A reused id replaces the earlier episode's ledger entry	11.3
Advance only the latest episode object; treat the first terminal result as final	An earlier object can still be advanced to a divergent or second terminal result	11.4
Rebuild the ExecutionContext from the native result and authorization	invoke_authorized_native does not return it	11.8
Record recovery bookkeeping once per validated transition	No deduplication by episode	11.11
Pass every Joint Recovery Feasibility plan through apply_executable_dependency_scope	Planning does not apply the executable-order correction	12.4
After re-entry at Pi Completeness or earlier, inspect the returned joint-recovery record; if it is runtime-verified and jointly infeasible, treat the pass as stopped and never license or execute the decision	Defect K2: those executors do not stop there, and their decision can be licensed and executed—a governance bypass	12.5; 15.2
Build reusable-artifact bundles only from the result of the cycle being re-entered	Bundles are matched by declared ids; payloads are not checked for origin or currency	12.4
Catch both ValueError and RuntimeError where an optional service may be absent	record_transfer_history raises RuntimeError; the other services raise ValueError	14.7

Obligation	Why	Section
When an agent or configuration is copied, pass configuration and information, pass permissions only by declaration, and never pass current PP: the copy re-establishes its PP from its own evidence	The runtime has no inheritance construct; a copied registry or state carries whatever the host copies (Guide 5.12; Getting Started 1.4)	5.7; 8.9

The supervision layer adds its own duties, eighteen of them, in Table 16.31 (16.12).

15.4 Legacy and Superseded Entry Points

These entry points are LEGACY / COMPATIBILITY: superseded by a newer entry according to their own docstrings or the release lineage, or retained from V1 or V2 for compatibility (rule R5). They still run, and they are tested; they do not implement the canonical stack, and their results must not be cited as v0.141 canonical behavior. Table 15.4 names each replacement or states that there is none.

Table 15.4 — Legacy entry points and replacements

Legacy entry	Why legacy	Replacement
PVPPRuntime.assess, decide, evaluate_proposal	The V1-generation assess path: prototype Φ , H , G , $\mathcal{R}$ (9.13), action evaluation by sequential projection, discretionary capacity selection	evaluate_integrated_canonical_cycle, or evaluate_canonical_decision_cycle (9.3)
PVPPRuntime.evaluate_policy_sets	Docstring: compatibility evaluator for pre-v0.18 callers; no Π completeness basis	The canonical cycle entries (9.3)
PVPPRuntime.evaluate_policy_space, evaluate_recovery_policy_space	Pre-canonical evaluators over a caller-built policy space; prototype G ; a comparator their notes call not canonical Σ (H.5-5)	The canonical cycle entries (9.3)

Legacy entry	Why legacy	Replacement
PVPPRuntime.evaluate_framed_policy_space	Docstring: compatibility path retaining the pre-v0.23 horizon adequacy heuristic; not a canonical Adequacy result	The canonical cycle entries (9.3)
PVPPRuntime.evaluate_constraints	Feasibility through action projection, not the host constraint profile (H.5-5)	evaluate_constraint_system (9.9)
PVPPRuntime.validate_domain_framing	Checks the frame against the prototype G, not canonical G (H.5-5)	No public replacement: the canonical cycle and the re-entry executors check framing against canonical G internally
PVPPRuntime.build_execution_license	Licenses from a pre-canonical selection. Its licenses never enter the license ledger, so instantiate_execution refuses them in v0.141	build_execution_license_from_cycle (11.2)
PVPPRuntime.record_execution	Docstring: legacy pre- ϵ compatibility recorder	ϵ , a validated Layer-1 transition, and record_recovery_bookkeeping_from_transition (11.11)
PVPPRuntime.record_layer1_execution_commit	Docstring: legacy v0.38 compatibility surface	As above
ExecutionBindingRegistry.invoke	Docstring: deprecated; always raises	invoke_authorized_native (11.7)

Legacy entry	Why legacy	Replacement
ExecutionBindingRegistry.build_layer1_handoff	Bypasses the ε trace made canonical by v0.79 (H.5-7)	epsilon_observation, advance_execution, build_layer1_transition_handoff (11.8, 11.10)
PVPPRegistry.register_domain_relation, register_dependency_requirement	Docstrings call them prototypes; consumed only by the legacy entries above (H.5-2)	None in the canonical cycle; G's closure uses registered recovery necessities (4.5, 8.7)
WorldAdapter.represented_state_from_actual, represented_state_from_perception_inputs	Compatibility perception hooks	perceived_decision_state_from_actual, perceived_decision_state_from_perception_inputs (10.2)
WorldAdapter.project_policy_record	Compatibility projection hook without licensing	PolicyProjectionService (10.3)
WorldAdapter.project_policy, execute	Used only by legacy paths, or never called	None needed (10.2)
CanonicalRuntimeBridge.invoke_authorized_native	Compatibility guard in build 167: allowed only for an authorization that has not entered supervision	invoke_registered_native (16.3)
CanonicalRuntimeBridge.advance_execution	Compatibility guard in build 167: refuses a supervised episode	ContinuationService.checkpoint (16.6)
SQLiteSupervisoryStore.bind_canonical_bridge, register_active_execution	Always refuse; kept as explicit refusals	CanonicalRuntimeBridge.register_active_execution (16.3)

The operator prototypes in `pvpp_runtime.operators.prototype` stay `INTERNAL`: they serve only the legacy path and are not part of any public contract. The records that only legacy entries return—`PVPPAssessment`, `ActionEvaluation`, `RecoveryAdequacyAssessment`, `SelectedPolicy`, `DiscretionarySelectionAssessment`, and `StructuralPolicyAssessment`—and `ExecutionResult`, the argument of `record_execution`, take the legacy label.

Statuses produced only on legacy paths never appear in a canonical cycle result:
 "unique_feasible_adequate_policy_set", "unique_structurally_admissible_policy_set",
 "unique_governing_consequence_dominance", "no_adequate_policy_set",
 "no_structurally_admissible_policy_set", "governing_consequences_unresolved",
 "crossing_governing_consequences_unresolved",
 "equivalent_governing_consequences_unresolved", "pi_completeness_basis_not_supplied",
 "registered_structure_filtered_policy_sets", "registered_structure_nonbinding",
 "no_registered_policy_structure", "all_eligible_fit", "capacity_not_supplied",
 "capacity_uniquely_determines_subset", "competing_maximal_sets_unresolved",
 "no_eligible_discretionary_actions", "no_active_corridors", and
 "resolved_by_governing_consequences".

Fifteen of the seventeen example scripts in the tree run legacy entries: every tiny_* world uses assess, decide, evaluate_policy_sets, or evaluate_policy_space. Only minimal_battery_service and authority_bound_callthrough_v0130 use the canonical cycle. Learn the canonical API from those two.

15.5 Declared-Only Protocols

The nine protocols of 10.9 are exported, but no v0.141 method accepts or calls them (H.5-6). They are not superseded—several name roles that configuration provenance already uses, and four have exactly the signature of a protocol the runtime calls—so LEGACY / COMPATIBILITY would misdescribe them. Disposition: they keep the default label IMPLEMENTATION / SUPPORT and are recorded as reserved roles. A host may implement them to organize its code; doing so changes nothing the runtime does. Whether a successor build wires them in, withdraws them, or documents them as reserved is a successor decision, not a v0.141 one.

15.6 Runtime Constructs Without Dedicated Framework Ownership

Some v0.141 surfaces serve framework rules without being defined by a V2.1 owner file. Their PUBLIC labels describe the runtime API; they are not framework primitives, and no framework owner should be created merely to regularize them.

Table 15.5 — Runtime constructs and the rules they serve

Construct	Serves	Section
Control-provenance topology	Source authority in perception and memory (C10/D19; C10/D16)	6.8
Security-evidence qualification	Evidence authority and information governance (C10/D16)	6.9
License, episode, and native-authorization ledgers; control states	Runtime detail between $\pi_{\text{sel}}(t)$ and $x_{\text{exec}}(t)$ stays subordinate to ε (C15/D9 §10B)	11.1–11.9
Artifact-integrity, lifecycle, and identity-chain audits	One shared projection record set; one state per cycle (C10/D6)	9.12
Configuration provenance and adapter roles	Portability without new primitives (C00/D1 §20; C30/D3)	4.9

Construct	Serves	Section
Reusable-artifact bundles, stage executors, and re-entry planning	Upstream invalidity is not repaired or laundered downstream (C20/D4); no V2.1 owner file defines dependency-scoped re-entry	Chapter 12
Testing diagnostics	Validation discipline (C45)	Chapter 13
Supervision layer: admission, dependencies, continuation, ownership, control, effect, recovery, audit	Governance of in-flight change where instrumentation exists (C45/D2 §8); successor capability separately implemented and versioned (C00/D3 §15)	Chapter 16

15.7 Compatibility Across Runtime Generations

Four frozen releases sit side by side in the repository, each in its own directory, so they can be compared but not imported under one name: three runtime generations, and v2.1.167, which carries the V2.1 decision core unchanged and adds the supervision layer.

Table 15.6 — Runtime generations

Generation	Directory and package	Exports	PVPPRuntime methods	Regression
V1, v0.70	runtime-v1/pvpp_runtime-build-v0-70	152	66	496 passed (its README)
V2, v0.131	runtime-v2/pvpp_runtime-build-v0-131	306	78	967 passed (its README)
V2.1, v0.141	runtime-v2.1/pvpp_runtime	335	89	1,093 passed (13.8)
V2.1, v2.1.167 (build 167)	runtime-v2.1.167/pvpp_runtime	335, and 51 in supervision	89	1,320 passed (13.8)

At the API surface each step was additive. Comparing the exports and the PVPPRuntime methods of the three packages found nothing removed. From V2 to V2.1, 29 exports and 11 methods were added, and six existing surfaces changed, all compatibly: three cycle entries gained keyword-only objective parameters, and CanonicalCycleSnapshot, CanonicalDecisionCycleAssessment, and MemoryRetrievalPackage gained fields appended after their existing ones. All 132 V2 test files are present, byte-identical, in the V2.1 suite, and the v0.132 identity check found the 143 inherited runtime and test files unchanged at the bootstrap (BASELINE_CODE_IDENTITY_CHECK.txt).

From v0.141 to v2.1.167 the step is additive in the strictest sense: the top-level package, its 335 exports, and the methods of its three core classes are byte-identical, and everything new is in pvpp_runtime.supervision. All 140 build-141 test files are present and byte-identical; the 28 new test files test only the supervision layer.

Additive is not the same as behavior-compatible. Guards added across the generations refuse usage that older code relied on: hand-built licenses (license lineage), stored-validity trust at licensing (the fresh artifact audit), and Σ ties without an explicit order. Three benchmark scripts

fail for exactly these reasons (13.8). Applications that need V2 behavior use runtime-v2; applications that need the v0.70 interface use runtime-v1; new Version 2.1 applications target runtime-v2.1.167, Runtime v2.1.167. An application that uses only the top-level package runs against byte-identical code; an application that registers executions for supervision opts into the guards of 16.3, which refuse direct advancement and native invocation of those executions. No application-compatibility test beyond the frozen suite stands behind this statement (RELEASE_NOTES.md).

15.8 Evidence That Does Not Transfer

Table 15.7 — Evidence that does not transfer

Evidence	Why it does not transfer
Regression results of V1 (496) and V2 (967)	Different builds; v2.1.167's result is 1,320 passed, of which 1,093 are build 141's suite
Historical results of benchmark_epsilon_v022, benchmark_sigma_standard_v024, benchmark_epsilon_mode_license_v037	These scripts fail against v0.141 (13.8)
Output of the tiny_* examples	They run legacy entries, not the canonical stack (15.4)
Examples, docs, README, release notes, and records	Outside the v0.141 hash set; some edited after the freeze (13.7). Evidence of use, not frozen behavior
Behavior of the pre-canonical evaluators and the assess path	LEGACY / COMPATIBILITY; not the canonical stack
Framework-permitted capabilities not built	Prospective until implemented (L1, L7)
Results of the supervision layer's 227 tests	Evidence about v2.1.167 only; v0.141 has no supervision layer
The Guide/Reference Supplement and the v0.167 Classification Record	Candidate-stage sources of intended use; checked against the frozen source before this book relies on them (Appendix F)

15.9 Known Documentation Discrepancies

Table 15.8 — Documentation discrepancies and resolutions

#	Discrepancy	Resolution
D1	The standard-Σ docstring cites Σ v1.13; the V2.1 owner file and ownership map name v1.14	Stale runtime documentation. Review found no change to the ordinary-case rule across that boundary, and the V2.1 change set does not change Σ by implication (C00/D9 §16). Σ v1.14 controls
D2	The Canonical Operator Ownership Map v0.9 (C20/D2) names the Restoration Adequacy owner as v0.9	The current file is v0.8 and no v0.9 exists. A project-control defect in compendium 20

#	Discrepancy	Resolution
D3	build_decision_replay_package defaults runtime_version to "0.137"	Stale metadata in frozen code; callers pass "0.141" (6.4)
D4	ExecutionTransitionAdapter's docstring says native call-through "remains a later phase"	Native invocation exists from v0.76; the protocol still confers no invocation authority (10.9)
D5	ExecutionBindingRegistry's docstring says it has no invoke method and a later build will invoke bindings	v0.141 invokes through invoke_authorized_native; the registry's invoke is deprecated (11.5)
D6	README.md names V2_1_BASELINE_AND_LINEAGE.md	The file is BASELINE_AND_LINEAGE.md (13.7)
D7	The integrated cycle declares "integrated_cycle_transition_invalid"	Unreachable: an invalid transition raises ValueError first (14.3)
D8	RELEASE_NOTES.md lists BASELINE_NOTICE.md among the V2.1 release records	No such file is in runtime-v2.1; runtime-v2 has one
D9	The example minimal_battery_service returns a positive expected_deterioration, and its comment says H expects a positive magnitude	H reads a negative value as deterioration: the horizon is the margin divided by max(epsilon_deterioration, -expected). The example's horizons are therefore effectively infinite; its decision is unaffected because it turns on projections and adequacy. Return the signed change (10.2)
D10	The supplement, the classification record, the README, and the package docstring call the release Runtime 2.2 v0.167, and the frozen archive unpacks to a folder named runtime-v2.2	One release: the Version Naming Migration Record names it Runtime v2.1.167 (Framework 2.1, Build 167), and the repository holds it in runtime-v2.1.167. This book uses those names
D11	The supervision package's __all__ omits 13 package-level names that the classification record labels SUPPORTED or REFERENCE	Export is not authority; they import by name (16.1; H.5-12)
D12	EvidenceUnverified is defined and exported but never raised	Unverified evidence appears as ValueError("evidence_unverified...") or an "unverified" attestation (16.11; H.5-11)
D13	EFFECT_STATES includes "resolved"	No public call produces it (Table 16.21)

None of these warrants a change to the frozen runtime. D2 is for project control to correct in compendium 20; the others are recorded here so a reader is not misled.

15.10 Register Decisions and Classification Record

This chapter closes the register entries routed to it. H.5-2 is decided: domain relations and dependency requirements are LEGACY / COMPATIBILITY, because every consumer is a legacy

entry. H.5-5 was decided in v0.14. H.5-6 is given its disposition in 15.5: the declared-only protocols keep the default label as reserved roles. H.5-3 is closed at the default label, IMPLEMENTATION / SUPPORT: the three capacity-authority flow methods are documented for direct host use but used only through the integrated cycle, and no owner file requires them to be application-callable, so the default rule decides. This is a conservative classification, not a judgment of importance; a successor build may expose them deliberately. The legacy entries of 15.4 not previously labeled are recorded as H.5-8.

Table 15.9 — Labels in Chapter 15

Symbol	Label	Rule
PVPPRuntime.assess	LEGACY / COMPATIBILITY	R5
PVPPRuntime.decide	LEGACY / COMPATIBILITY	R5
PVPPRuntime.evaluate_policy_sets	LEGACY / COMPATIBILITY	R5
PVPPRuntime.evaluate_proposal	LEGACY / COMPATIBILITY	R5
PVPPRuntime.evaluate_policy_space	LEGACY / COMPATIBILITY	R5 (H.5-5, revised in v0.14)
PVPPRuntime.evaluate_recovery_policy_space	LEGACY / COMPATIBILITY	R5 (H.5-5, revised in v0.14)
PVPPRuntime.evaluate_framed_policy_space	LEGACY / COMPATIBILITY	R5
PVPPRuntime.evaluate_constraints	LEGACY / COMPATIBILITY	R5 (H.5-5, revised in v0.14)
PVPPRuntime.validate_domain_framing	LEGACY / COMPATIBILITY	R5 (H.5-5, revised in v0.14)
PVPPRuntime.build_execution_license	LEGACY / COMPATIBILITY	R5
PVPPRuntime.record_execution	LEGACY / COMPATIBILITY	R5
PVPPRuntime.record_layer1_execution_commit	LEGACY / COMPATIBILITY	R5 (docstring: legacy v0.38 compatibility surface)
PVPPRegistry.register_domain_relation	LEGACY / COMPATIBILITY	R5 (H.5-2, decided in Chapter 15)
PVPPRegistry.register_dependency_requirement	LEGACY / COMPATIBILITY	R5 (H.5-2, decided in Chapter 15)
DomainRelationDefinition	LEGACY / COMPATIBILITY	R6, following its entry (H.5-2)

Symbol	Label	Rule
DependencyRequirementDefinition	LEGACY / COMPATIBILITY	R6, following its entry (H.5-2)
PVPPAssessment	LEGACY / COMPATIBILITY	R5 (result of a legacy entry)
ActionEvaluation	LEGACY / COMPATIBILITY	R5 (result of a legacy entry)
RecoveryAdequacyAssessment	LEGACY / COMPATIBILITY	R5 (result of a legacy entry)
SelectedPolicy	LEGACY / COMPATIBILITY	R5 (result of a legacy entry)
DiscretionarySelectionAssessment	LEGACY / COMPATIBILITY	R5 (result of a legacy entry)
StructuralPolicyAssessment	LEGACY / COMPATIBILITY	R5 (result of a legacy entry)
ExecutionResult	LEGACY / COMPATIBILITY	R5 (argument of record_execution and WorldAdapter.execute)
ExecutionBindingRegistry.invoke	LEGACY / COMPATIBILITY	R5 (docstring: deprecated; fails closed)
ExecutionBindingRegistry.build_layer1_hand off	LEGACY / COMPATIBILITY	R5 (superseded by the ε bridge of v0.79; H.5-7)

Chapter 16 — The Supervision Layer

Build 167 adds one package to the runtime, `pvpp_runtime.supervision`, and changes nothing else: the decision core of Chapters 3 through 15 is build 141's, byte for byte (1.4). The package lets a host govern an action while it runs. Host instrumentation reports facts; the layer admits them under source authority, finds the executions that declared a dependency on them, and, at a checkpoint the host calls, lets the canonical runtime decide whether the execution may continue. When it may not, the layer fences the execution's unused authority, records a stop request from the current supervisory owner, translates it for a registered controller, and records what the controller reports and, separately, what the world shows. This chapter documents every public name of the package, what each establishes, and what each leaves to the host.

The runtime decides; a monitor reports but does not decide; a controller stops; host code drives every step. Nothing in this chapter makes the decision core asynchronous.

What the layer does not do. It governs at checkpoints the host calls, not inside a native call once entered (16.3). It senses, schedules, and stops nothing itself (16.12). And it does not resume an execution after a restart: a reopened store can reconcile and record an earlier execution, but continuing requires a new decision and a new governed attempt (16.9).

Sources. Behavior is cited to the frozen source, `pvpp_runtime/supervision/` (14 files), and to the 227 successor tests in 28 test files (Appendix E, Table E.6). Intended use is taken from the v0.167 API Classification Record v1.0 and the Guide/Reference Supplement v1.0 (Appendix F). Where this chapter states behavior that no successor test covers, it says so and names the confirmation run, usually the tests of Appendix B.7. The Guide explains the layer in Chapter 12 and Getting Started in Chapter 6; this chapter does not repeat their examples.

16.1 Architecture, Authority, and Classification

The layer is a chain of services over one durable store. Each link records one kind of evidence and passes an identifier to the next; none of them substitutes for another. Table 16.1 lists the chain in the order a supervised execution meets it.

Table 16.1 — The supervision chain

Step	Service and call	Record produced	Who acts
Register the execution	<code>CanonicalRuntimeBridge.register_active_execution</code> (16.3)	<code>ActiveExecutionRecord</code>	Host, from a license and an issued native authorization
Report a fact	<code>ObservationService.submit_observation</code> , <code>assess_observation</code> (16.4)	<code>ExternalObservation</code> ; <code>ObservationAdmissionRecord</code>	Monitor reports; the service checks the source's registration

Step	Service and call	Record produced	Who acts
Commit the governed fact	ObservationService.commit_admission_change (16.4)	GovernedFactView; AdmittedFactChange	Service, for reports whose admission record is admissible
Find affected executions	DependencyService.resolve_affected_executions (16.5)	Tuple of active execution ids	Service, from declared dependencies only
Checkpoint	ContinuationService.checkpoint (16.6)	GovernanceSnapshotToken; ContinuationAssessment	Host calls; the canonical runtime decides the posture
Fence unused authority	ControlService.fence_execution_authority (16.7)	Invalidated native authorizations	Current supervisory owner
Request and dispatch a stop	ControlService.issue_control_request, dispatch_control_request (16.7)	ExecutionControlRequest; ControlAttemptRecord	Current owner; the posture limits the request
Carry out the stop	None: outside the runtime	—	The host's controller
Record the controller's report	ControlService.record_enforcement_outcome (16.7)	EnforcementRecord	Host relays; the trust provider verifies attestation
Reconcile the world effect	EffectService.submit_effect_evidence, reconcile_effect (16.8)	EffectEvidence; WorldEffectReconciliationState	Registered effect source
Record finality and Layer 1	EffectService.accept_canonical_finality, record_layer1_transition (16.8)	CanonicalEpisodeFinality; Layer1TransitionRecord	Host; the trust provider verifies Layer-1 authority
Recover after a crash	RecoveryService (16.9)	RecoveryReconciliationRecord	New owner, against a profile registered in advance

Authority. Three kinds of authority meet in the layer and none converts into another. Canonical authority—the license, the episode, the single-use native authorization—remains the decision core's (Chapter 11); the layer refers to it through a CanonicalExecutionHandle and never replaces it. Registration authority—who may register a source, a dependency, an owner, an adapter, a recovery profile, or reconcile a divergence—comes only from the HostTrustProvider's roots (16.2). Supervisory ownership—who may fence and request stops for one execution—is a fenced, renewable claim recorded in the store (16.7). A source that reports a fact gains none of these. Audit events record what happened and grant nothing (16.10).

Framework standing. The layer is a runtime construct without a dedicated framework owner, like the license and authorization ledgers (15.6). The framework permits governance of in-flight change where instrumentation and execution architecture exist (C45/D2 §8), and project control

requires that external instrumentation, asynchronous observation, and mid-flight enforcement be identified as successor capabilities unless separately implemented and versioned (C00/D3 §15). Build 167 is that separate implementation, for the parts this chapter documents and no others. It adds no operator and no selector, and every decision it records is made by the canonical runtime: ε for progress reports, and the re-entry planner of 12.3 for invalidations (16.6).

Classification. The labels of this chapter apply the decision rules of Table H.1, with one rule added for build 167: R7 assigns PUBLIC / REFERENCE to a supplied implementation that illustrates a contract a deployment may implement differently (1.5). Intended use is read from the v0.167 API Classification Record, which labels surfaces as wholes; this chapter labels each symbol and method. As recorded in H.7, the two agree on every service, entry, and exception. They differ in vocabulary for the records the services return, which the record's PUBLIC / SUPPORTED covers as parts of a supported surface and rule R3 labels IMPLEMENTATION / SUPPORT: applications receive and read them and do not build them (H.5-10). Table 16.2 summarizes; 16.14 and Appendix A.5 give every label.

Table 16.2 — Supervision surfaces and their labels

Surface	Label	Notes
The seven services and HostTrustProvider	PUBLIC / SUPPORTED	The integration surface; their methods are SUPPORTED except where 16.14 says otherwise
SourceRegistration, ExternalObservation	PUBLIC / SUPPORTED	Built by the host (R6)
Records the services return	IMPLEMENTATION / SUPPORT	Read them; do not construct them (R3; H.5-10)
Typed exceptions	PUBLIC / SUPPORTED	Handle them explicitly (16.11); EvidenceUnverified is never raised (H.5-11)
SQLiteSupervisoryStore	PUBLIC / ADVANCED	Direct store access is privileged; use the services. Its audit query is SUPPORTED
Bridge guards and store refusals	LEGACY / COMPATIBILITY	invoke_authorized_native and advance_execution on the bridge; bind_canonical_bridge and register_active_execution on the store (16.3, 16.10)
Raw table access and private helpers	INTERNAL	phase6_put, phase6_get, the bridge's advance permit, the observation-value helpers
Operational adapters and the simulation harness	PUBLIC / REFERENCE	Thin delegates over the same services (16.13)

Import route. Import from pvpp_runtime.supervision. Its `__all__` lists 38 of the 51 package-level names; the other 13—DependencyService, EffectService, their records, and the four operational adapters—import normally by name but are omitted by a star import (H.5-12). The five control records of 16.7 are not package-level names; import them, if at all, from `pvpp_runtime.supervision.models`; applications receive them from ControlService and need not

import them. The top-level package `pvpp_runtime` does not import the supervision package: its 335 exports are build 141's.

16.2 The Host Trust Provider

The trust provider is the deployment's trust boundary in one object. It holds the registration roots, the verifier for controller attestations, the verifier for Layer-1 authority, and the trusted clock. The store holds one, and every service consults it through the store. The runtime supplies no identity system and no cryptography: a proof is whatever opaque value the host configures, compared for equality, and a verifier is whatever callable the host supplies.

Operational requirement. `HostTrustProvider` is a configuration object, not a hardened identity system. A production deployment must independently protect where its roots come from, keep proofs secret, keep the clock correct, keep verifiers honest and their keys protected, control who can construct or replace the provider, and revoke roots and sources when they are compromised. The demonstration literals of B.7, Getting Started Chapter 6, and Guide Chapter 12 show the call shape only; they are not a trust design.

Entry — `HostTrustProvider`

Field	Content
Name and label	<code>HostTrustProvider</code> — class; <code>supervision/trust.py:12</code> . PUBLIC / SUPPORTED (rule R1; integration contract).
Purpose	Answers, for the supervision layer, who may register what, whether a controller's report and a Layer-1 transition are authentic, and what time it is.
Framework basis	Evidence authority (C10/D16): a report's authority comes from its source's registration, never from its content. A runtime construct (15.6).
Signature	<code>HostTrustProvider(roots: Mapping[str, Mapping[str, Any]] None = None, *, controller_attestation_verifier: Callable[..., bool] None = None, layer1_authority_verifier: Callable[..., bool] None = None, clock: Callable[[], datetime] None = None)</code>
Preconditions	Constructed by the host and passed to <code>SQLiteSupervisoryStore(trust_provider=...)</code> . <code>roots</code> maps each authority id to <code>{"proof": value, "roles": iterable, "configurations": iterable or None}</code> ; <code>"*"</code> is a wildcard for roles or configurations. Both verifiers receive keyword arguments only (Table 16.3).
Result	An object whose four methods the services call: <code>authorize_registration</code> , <code>verify_controller_attestation</code> , <code>verify_layer1_authority</code> , and <code>now</code> .
Invariants	<code>now()</code> always returns an aware UTC datetime; a naive clock value is read as UTC. A missing verifier answers <code>False</code> . <code>authorize_registration</code> answers <code>True</code> only for a configured root whose proof equals the one supplied, whose roles include the role or <code>"*"</code> , and whose configurations are <code>None</code> or include the configuration or <code>"*"</code> .

Field	Content
Failure behavior	Raises nothing itself. A refused registration surfaces as <code>PermissionError("trusted registration authority required for <role>")</code> from the calling service; a False controller-verifier answer surfaces as an "unverified" attestation, or "weak" when the adapter registered attestation method "none" (16.7) or <code>ValueError("layer1_conflict: ...")</code> (16.8).
Does not	Store, rotate, or protect credentials; check signatures; or strengthen weak evidence. A store built without a provider gets <code>HostTrustProvider()</code> with no roots and no verifiers: every registration is refused and no attestation verifies. The service code consults the provider, but code in the same process can replace it: in-process code is inside the trusted computing base (16.12).
Evidence	Direct use: test files 1; e.g. <code>r22_trust_fixture.py</code> ; Appendix B.7 (<code>b7_supervise.py</code>); named in the Guide/Reference Supplement.
See also	Guide 12.10; Getting Started 6.3; 16.10; B.7.

Table 16.3 — What the trust provider is asked

Method	Called by	Arguments	Answer used for
<code>authorize_registration(authority_id, *, role, configuration_id, proof)</code>	Every registration and <code>reconcile_continuation_divergence</code> , through the store	<code>role</code> is one of <code>source_registration</code> , <code>dependency_registration</code> , <code>owner_registration</code> , <code>adapter_registration</code> , <code>recovery_registration</code> , <code>continuation_reconciliation</code>	<code>PermissionError</code> when False
<code>verify_controller_attestation(*, adapter_registration, control_attempt, outcome, controller_evidence_ref, attestation_evidence)</code>	<code>ControlService.record_enforcement_outcome</code>	The adapter registration and attempt records, the reported outcome, and the host's attestation evidence	<code>EnforcementRecord.attestation_status</code>
<code>verify_layer1_authority(*, active_execution, effect_state, transition_time, evidence_ref, authority_evidence)</code>	<code>EffectService.record_layer1_transition</code>	The active execution record, the current effect state, and the host's authority evidence	Refusal with <code>ValueError</code> when False
<code>now()</code>	Ownership leases, trusted timestamps on control records	—	Lease expiry and <code>trusted_created_at</code> , <code>trusted_dispatched_at</code>

The caller cannot vouch for itself. `record_enforcement_outcome` accepts an `attestation_verified` argument and `record_layer1_transition` a `host_authoritative` argument; both are ignored, and only the provider's answer counts (`test_r22_phase16_v0157_round2_hardenig`: `test_v0157_08_caller_boolean_cannot_verify_controller_attestation`, `test_v0157_09_caller_boolean_cannot_create_layer1_authority`).

16.3 The Canonical Bridge and Active Executions

The bridge wraps one `PVPPRuntime` and records the exact objects that cross its public methods: each episode the runtime returns, each authorization it issues, and each handle the bridge resolves. It never reads or writes the runtime's private ledgers. A supervised execution is created in four calls: instantiate the episode and issue the authorization through the bridge, resolve a handle, and register the handle in a store. Registration binds the store to this bridge for that execution; from then on the episode advances only through `ContinuationService`. Governing an action while it runs means governing it at the checkpoints the host calls; a native call, once entered, is not governed, interrupted, or cancelled by anything in this chapter (15.1 L1, L8).

Entry — CanonicalRuntimeBridge

Field	Content
Name and label	<code>CanonicalRuntimeBridge</code> — class; <code>supervision/bridge.py:26</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Carries the decision core's execution objects into supervision without creating a second execution authority.
Framework basis	ε (C15/D9 §10B): runtime detail between $\pi_{\text{sel}}(t)$ and $x_{\text{exec}}(t)$ stays subordinate to ε . The bridge adds identity and fencing, not authority.
Signature	<code>CanonicalRuntimeBridge(runtime: Any) → None</code>
Preconditions	runtime is the <code>PVPPRuntime</code> that issued, or will issue, the license (11.2). Use one bridge per runtime object, and create every episode and authorization you will supervise through it: the bridge recognizes only objects it has itself returned.
Result	A bridge whose <code>instantiate_execution</code> and <code>issue_native_execution_authorization</code> return the runtime's own results, recorded as current.
Invariants	An episode, authorization, or handle is accepted only if it is the identical object the bridge recorded (identity, not equality). Once an <code>advance_execution</code> call has entered the runtime and raised, the episode is marked indeterminate and the bridge refuses to advance it, invoke for it, or issue for it again.
Failure behavior	<code>CanonicalResolutionError</code> for an unrecognized or stale object, an indeterminate episode, or a supervised episode advanced directly; <code>ConfigurationConflict</code> when a handle is requested under a configuration other than the authorization's; <code>TypeError</code> or <code>ValueError</code> from the pre-advance checks.

Field	Content
Does not	Persist anything. The bridge's records live in memory; a store reopened in another process has no bridge for its executions and cannot be given one (16.9, 16.12).
Evidence	Direct use: test files 7; e.g. test_r22_phase12_race_matrix.py, test_r22_phase15_strengthened_validation.py; Appendix B.7 (b7_supervise.py); named in the Guide/Reference Supplement.
See also	Guide 12.3; Getting Started 6.3; 11.3–11.7; B.7.

Entry — CanonicalRuntimeBridge.resolve_canonical_execution

Field	Content
Name and label	CanonicalRuntimeBridge.resolve_canonical_execution — method; supervision/bridge.py:115. PUBLIC / SUPPORTED (rule R1).
Purpose	Turns an issued native authorization into a handle that names the exact canonical lineage the supervisor will refer to.
Framework basis	Object-lineage authority (11.1); the handle is a reference, never authority.
Signature	CanonicalRuntimeBridge.resolve_canonical_execution(authorization: NativeExecutionAuthorization, *, configuration_id: str None = None) → CanonicalExecutionHandle
Preconditions	authorization is the exact object returned by this bridge's issue_native_execution_authorization; its status in the runtime is "issued"; its episode is current and active; configuration_id equals the authorization's; the authorization's action is in the episode's license.
Result	A CanonicalExecutionHandle with status "current", a new canonical_handle_id, and the episode, license, policy, action, cycle, configuration, attempt, binding, and authorization identities copied from the canonical objects.
Invariants	The handle is recorded by the bridge. register_active_execution accepts only a handle that is the identical recorded object.
Failure behavior	CanonicalResolutionError("canonical_resolution_failed: ...") for a non-exact authorization, a status other than "issued" (the message names it, for example "authorization status is consumed"), or a stale, terminal, or unlicensed episode; CanonicalResolutionError("canonical_state_indeterminate: ...") for an indeterminate episode; ConfigurationConflict("configuration_conflict").
Does not	Authorize anything or consume the authorization.
Evidence	Direct use: test files 13; e.g. test_r22_phase12_race_matrix.py, test_r22_phase14_conformance_closure.py; Appendix B.7 (b7_supervise.py); not named in the Supplement.
See also	16.6 (every checkpoint resolves the handle again).

Entry — CanonicalRuntimeBridge.register_active_execution

Field	Content
Name and label	CanonicalRuntimeBridge.register_active_execution — method; supervision/bridge.py:159. PUBLIC / SUPPORTED (rule R1).
Purpose	Registers one canonical execution attempt for supervision and binds the store to this bridge for it.
Framework basis	Runtime construct (15.6).
Signature	CanonicalRuntimeBridge.register_active_execution(store, handle: CanonicalExecutionHandle, *, execution_id: str, registered_at: str)
Preconditions	handle is the identical object this bridge returned and is still current (it is re-resolved); execution_id is non-empty—use PVPPRuntime.native_execution_authorization_execution_id(authorization_id) (11.6); the store has not registered this lineage before.
Result	An ActiveExecutionRecord with status "registered" and version 1, persisted in the store. The bridge records the execution against its authorization and episode, after which the bridge's direct advance_execution and invoke_authorized_native refuse that episode.
Invariants	One active execution per authorization: registering a new handle for an authorization already registered, or reusing an execution_id, raises ConcurrencyConflict("active execution already registered or lineage reused"); presenting the same handle again raises CanonicalResolutionError, because registration re-resolves the handle and replaces the recorded object. Uniqueness is per authorization, not per episode: a second authorization issued on the same episode can be registered, and the bridge then associates the episode with the later registration; register one authorization per episode. The store-bridge binding is private and cannot be replaced (SQLiteSupervisoryStore.bind_canonical_bridge always refuses).
Failure behavior	CanonicalResolutionError for a non-exact or stale handle; ValueError for an empty execution_id; ConcurrencyConflict for a reused lineage.
Does not	Track the execution's progress: the record's status stays "registered" whatever the episode does. Query the episode through the bridge, or the latest ContinuationAssessment.
Evidence	Direct use: test files 14; e.g. test_r22_phase11_api_misuse.py, test_r22_phase12_race_matrix.py; Appendix B.7 (b7_supervise.py); named in the Guide/Reference Supplement.
See also	Guide 12.3; Getting Started 6.3; test_r3_phase2_canonical_bridge; test_r22_phase20_v0161_round6_hardening; test_v0161_03_public_bridge_rebinding_cannot_forge_retry_authority.

Entry — `CanonicalRuntimeBridge.invoke_registered_native`

Field	Content
Name and label	<code>CanonicalRuntimeBridge.invoke_registered_native</code> — method; supervision/bridge.py:171. PUBLIC / SUPPORTED (rule R1).
Purpose	Enters a registered native binding for a supervised execution, recording invocation start durably before the callable runs.
Framework basis	Native invocation (11.7) under supervision.
Signature	<code>CanonicalRuntimeBridge.invoke_registered_native(store, active_execution_id: str, authorization: NativeExecutionAuthorization, binding_registry, *args, **kwargs)</code>
Preconditions	The execution is registered in store with this authorization; its episode is not indeterminate; its handle still resolves (the authorization is "issued"); authorization is the exact recorded object.
Result	Whatever <code>PVPPRuntime.invoke_authorized_native</code> returns (11.7). The store records a supervised native invocation first, which closes the retry-contract window (16.8).
Invariants	The invocation-start record is written before the runtime's own checks and before it consumes the authorization: a call the runtime then refuses—a configuration mismatch, a non-ready control state—has still closed the retry-contract window (16.8).
Failure behavior	<code>CanonicalResolutionError("pre_entry_registration_required")</code> , <code>("active_execution_authorization_mismatch")</code> , or a resolution failure; otherwise as 11.7.
Does not	Keep the execution checkpointable. Invocation consumes the authorization, and every later checkpoint must resolve the handle, which requires an "issued" authorization: after this call each checkpoint raises <code>CanonicalResolutionError("canonical_resolution_failed: authorization status is consumed")</code> . Nothing governs a native call once entered (15.1 L1, L8). A host that needs checkpoints while the action runs performs the action by its own means and reports progress at checkpoints, leaving the authorization issued (B.7).
Evidence	Direct use: test files 3; e.g. <code>test_r22_phase12_race_matrix.py</code> , <code>test_r22_phase16_v0157_round2_hardening.py</code> ; named in the Guide/Reference Supplement.
See also	Guide 7.15–7.16, 12.3; Getting Started 6.5; 15.1 L8.

Table 16.4 — Other bridge methods

Method	Label	Returns or does
<code>instantiate_execution(episode_id, license, **kwargs)</code>	SUPPORTED	<code>PVPPRuntime.instantiate_execution</code> 's result (11.3), recording the returned episode

Method	Label	Returns or does
issue_native_execution_authorization (episode, action_id, binding_registry, **kwargs)	SUPPORTED	PVPPRuntime.issue_native_execution_authorization's result (11.6), recording it; refuses an indeterminate or non-current episode
authorization_status (authorization_id)	ADVANCED	The runtime's lifecycle status for the authorization: "issued", "consumed", "invalidated", or None
current_handle (canonical_handle_id), refresh_canonical_handle (handle)	ADVANCED	The handle re-resolved from the current canonical objects, keeping its id; raise as resolve_canonical_execution. Each call replaces the recorded object, so an object returned earlier is then refused by register_active_execution: use the latest one
episode_state (episode_id), current_episode (episode_id), episode_step (episode_id)	ADVANCED	The latest recorded episode, the same only if still active, and its step_count; CanonicalResolutionError("stale_lineage: ...") when the episode is unknown, and from current_episode also when it is inactive
episode_is_indeterminate (episode_id), advance_call_was_entered (episode_id)	ADVANCED	Diagnostics: whether an entered advance raised, and whether an advance is in progress now
invoke_authorized_native (authorization, binding_registry, ...)	LEGACY / COMPATIBILITY	Guard: allowed only for an authorization not registered for supervision; otherwise CanonicalResolutionError("registered execution requires invoke_registered_native")
advance_execution (episode, observation)	LEGACY / COMPATIBILITY	Guard: for an unsupervised episode, advances it; for a supervised one, refuses with CanonicalResolutionError("supervised_advance_required: ...") unless ContinuationService holds the permit
authorize_supervised_advance , validate_advance_preconditions	INTERNAL	Used by ContinuationService; do not call

Table 16.5 — Fields of CanonicalExecutionHandle (supervision/models.py:79; IMPLEMENTATION / SUPPORT)

Field	Type	Meaning
canonical_handle_id	str	The handle's own id; kept across refreshes

Field	Type	Meaning
episode_id	str	The ϵ episode (11.3)
license_object_id	int	Python id() of the license object at resolution; meaningful only within the process
selected_policy_id	str	From the authorization
action_id	str	The licensed action authorized
decision_cycle_id	str	From the authorization
configuration_id	str None	From the authorization
attempt	int	From the authorization
binding_id	str	The registered binding at issuance
binding_implementation_version	str	Its implementation version at issuance
authorization_id	str	The native authorization (11.6)
status	str	"current"
resolved_at	str	Wall-clock time of resolution, ISO-8601 UTC

Table 16.6 — Fields of `ActiveExecutionRecord` ([supervision/models.py:97](#); [IMPLEMENTATION / SUPPORT](#))

Field	Type	Meaning
active_execution_id	str	The supervision layer's id for this attempt; every service call names it
canonical_handle_id	str	The handle registered
episode_id	str	As the handle
authorization_id	str	As the handle
execution_id	str	The native execution id supplied at registration
action_id	str	As the handle
decision_cycle_id	str	As the handle
configuration_id	str None	As the handle; every registration for this execution must use it
attempt	int	As the handle
binding_id	str	As the handle
status	str	"registered"; no public call changes it
version	int	1
registered_at	str	As supplied

16.4 Observation Intake and Admission

A report becomes a governed fact in three calls, each with its own record: `submit` stores the report as received; `assess` decides whether its source may be relied on for this fact and use; `commit` updates the current view of the fact and returns the change, if there is one. Reports that are not admissible, or that do not change the view, are kept as evidence and change nothing. A source is registered by a root before any of its reports can be admitted.

Entry — `ObservationService.register_source`

Field	Content
Name and label	<code>ObservationService.register_source</code> — method; <code>supervision/observation.py:47</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Records which source may report which fact kinds, about which subjects, in which configurations, for which uses, and when.
Framework basis	Information governance: authority, scope, permitted use, and temporal validity are explicit metadata (C10/D16).
Signature	<code>ObservationService.register_source(r: SourceRegistration, *, authority_proof=None) → None</code>
Preconditions	<code>r.status</code> is one of "current", "revoked", "superseded", "limited"; <code>r.registration_authority</code> is a root that the trust provider authorizes for source_registration with <code>authority_proof</code> . When <code>r.configuration_scope</code> names exactly one configuration the root must cover it; otherwise the root must cover all configurations (configurations None or "*").
Result	None. The registration is stored under (source_id, version).
Invariants	Versions are append-only and unique per source. The registration with the highest version is the source's current registration, whatever its status: to revoke a source, register a higher version with status "revoked". Only "current" makes a source authoritative; "limited", "revoked", and "superseded" do not.
Failure behavior	<code>ValueError("invalid source status")</code> ; <code>PermissionError</code> from the trust provider; <code>sqlite3.IntegrityError</code> for a duplicate registration id or (source_id, version).
Does not	Make the source a decision maker: an admitted report updates a fact view and nothing else. Check <code>fact_kinds</code> against any vocabulary.
Evidence	Direct use: test files 20; e.g. <code>r22_trust_fixture.py</code> , <code>test_r22_phase10_operational.py</code> ; Appendix B.7 (<code>b7_supervise.py</code>); not named in the Supplement.
See also	Guide 12.4; Table 16.7; <code>test_r22_phase16_v0157_round2_hardening</code> ; <code>test_v0157_10_source_registration_requires_host_root_proof</code> .

Table 16.7 — Fields of `SourceRegistration` (`supervision/observation.py:9`; PUBLIC / SUPPORTED)

Field	Type	Meaning
<code>source_registration_id</code>	str	Unique id of this registration version

Field	Type	Meaning
source_id	str	The source; matched against ExternalObservation.source_id and EffectService's source_id
evidence_roles	tuple[str, ...]	Must include "observation" for facts and "effect" for effect evidence
fact_kinds	tuple[str, ...]	Fact kinds the source may report; compared exactly as written
subject_scope	tuple[str, ...]	Subjects it may report on; empty means any
configuration_scope	tuple[str, ...]	Configurations; empty means any
permitted_uses	tuple[str, ...]	Must include the use asked of the report: "governance" by default; "effect_reconciliation" for effect evidence
attestation_method	str	Recorded; not checked
valid_from	str None	Optional; the report's effective_at must not be earlier, compared as text
valid_until	str None	Optional; the report's effective_at must not be later, compared as text
status	str	"current", "revoked", "superseded", or "limited"
registration_authority	str	The root that registers it
version	int	Higher versions supersede lower ones

Entry — [ObservationService.submit_observation](#)

Field	Content
Name and label	ObservationService.submit_observation — method; supervision/observation.py:60. PUBLIC / SUPPORTED (rule R1).
Purpose	Stores a report exactly as received, before any judgment about it.
Framework basis	Valid time versus record time (C10/D22): effective, observed, and received times are kept apart.
Signature	ObservationService.submit_observation(o: ExternalObservation) → ExternalObservation
Preconditions	o is an ExternalObservation (Table 16.8). Its content must serialize as JSON.
Result	o. A first submission is stored with a digest of its content; an identical resubmission is accepted and changes nothing.
Invariants	observation_id identifies content: reusing an id with different content is refused.

Field	Content
Failure behavior	ConcurrencyConflict("identity_conflict: observation_id reused with changed content"); TypeError if the content does not serialize.
Does not	Check the source, admit the report, or change any view. Limit size: no byte-size limit applies, and the content is stored more than once (16.12).
Evidence	Direct use: test files 13; e.g. test_r22_phase11_api_misuse.py, test_r22_phase12_race_matrix.py; Appendix B.7 (b7_supervise.py); not named in the Supplement.
See also	Guide 12.4; test_r3_phase3_observation_v0144.

Table 16.8 — Fields of ExternalObservation (supervision/observation.py:15; PUBLIC / SUPPORTED)

Field	Type	Meaning
observation_id	str	Unique id of the report; identifies its content
source_id	str	The reporting source
fact_kind	str	The kind of fact; write it in canonical form: lower case, single spaces, no surrounding space (16.5)
fact_id	str	Which fact of that kind
subject_id	str	Whom or what the fact is about
configuration_id	str	The configuration it belongs to
value	Any	The reported value; the layer stores it and never interprets it
confidence	float None	Optional; recorded, never used for admission
effective_at	str	When the fact became true; orders the view. Use one fixed ISO-8601 form throughout (16.12)
observed_at	str	When the source observed it
received_at	str	When the host received it
payload	Mapping[str, Any]	Any further content, stored with the report

Entry — ObservationService.assess_observation

Field	Content
Name and label	ObservationService.assess_observation — method; supervision/observation.py:70. PUBLIC / SUPPORTED (rule R1).
Purpose	Decides whether a submitted report may be relied on for one use.
Framework basis	Evidence authority and permitted use (C10/D16).

Field	Content
Signature	ObservationService.assess_observation(o: ExternalObservation, *, assessed_at: str, permitted_use: str = governance) → ObservationAdmissionRecord
Preconditions	o was submitted. permitted_use names the use; the default is "governance".
Result	An ObservationAdmissionRecord, stored, whose disposition is "admissible" when all four checks of Table 16.9 pass and "retained_noncurrent" otherwise; reason is "admissible", "source_not_current", or "scope_or_use_or_time_invalid".
Invariants	Every assessment is recorded, admissible or not, with its four check results.
Failure behavior	None for an unregistered or out-of-scope source: the report is assessed as not admissible.
Does not	Verify the report's truth, check its value, or use confidence. A source with no registration, or one with a self-chosen name, is simply not authoritative (test_r22_phase11_api_misuse: test_source_authority_self_named_and_out_of_scope_remain_nonauthoritative).
Evidence	Direct use: test files 13; e.g. test_r22_phase11_api_misuse.py, test_r22_phase12_race_matrix.py; Appendix B.7 (b7_supervise.py); not named in the Supplement.
See also	Guide 12.4; Getting Started 6.6.

Table 16.9 — Admission checks

Check	Passes when	Field of the record
Authority	The source's current registration has status "current"	authoritative
Applicability	fact_kind is in fact_kinds; subject and configuration are in scope, or the scope is empty	applicable
Permitted use	permitted_use is in permitted_uses and "observation" is in evidence_roles	permitted
Temporal validity	effective_at is not before valid_from and not after valid_until, compared as text	temporally_valid

Entry — ObservationService.commit_admission_change

Field	Content
Name and label	ObservationService.commit_admission_change — method; supervision/observation.py:88. PUBLIC / SUPPORTED (rule R1).
Purpose	Applies an admissible report to the governed current view of its fact, and returns the change for fan-out.

Field	Content
Framework basis	Current governed fact view; history is retained, not rewritten (C10/D22).
Signature	ObservationService.commit_admission_change(o: ExternalObservation, a: ObservationAdmissionRecord, *, committed_at: str) → AdmittedFactChange None
Preconditions	a is the record assess_observation returned for o, after o was submitted.
Result	An AdmittedFactChange when the view changed, or None. None is returned for a report that is not admissible, that is older than the current view, or that repeats a current view. Table 16.10 gives the rule.
Invariants	One serialized write per commit; the view's version increases by one per change. A report with the same effective time as the current view but a different value makes the view "unresolved", with value None; a later report resolves it.
Failure behavior	ValueError("admission/observation mismatch").
Does not	Verify that a was produced by assess_observation, or that o was submitted: only the observation ids are compared, and the disposition of the record passed in is trusted. Pass the record assess_observation returned for a report you submitted; in-process code is inside the trusted computing base (16.12). Fan out: pass the change to DependencyService.resolve_affected_executions. Interpret an "unresolved" view: a checkpoint proceeds on it, and deciding what an unresolved fact means is the host's.
Evidence	Direct use: test files 13; e.g. test_r22_phase11_api_misuse.py, test_r22_phase12_race_matrix.py; Appendix B.7 (b7_supervise.py); not named in the Supplement.
See also	Guide 12.4; test_r22_phase12_race_matrix: test_race10_duplicate_observation_is_idempotent, test_race11_out_of_order_observation_does_not_displace_newer_view.

Table 16.10 — How a committed report changes the view

Current view	Report	Result
None	Admissible	View created, state "current", version 1; change returned
Any	effective_at earlier than the view's, as text	No change; None
Any	Same effective_at, different value	State "unresolved", value None, both reports listed; change returned
Any	Later effective_at	State "current", the new value; change returned
"current"	Same effective_at and value, or already listed	No change; None

Current view	Report	Result
"unresolved"	Same effective_at, any report, including one already listed	Stays "unresolved"; version +1; change returned
Any	Not admissible	No change; None

Table 16.11 — Other observation records and queries

Name	Fields or signature	Meaning
ObservationAdmissionRecord	admission_id, observation_id, source_registration_id, disposition, reason, authoritative, applicable, permitted, temporally_valid, assessed_at	One assessment (Table 16.9)
GovernedFactView	fact_kind, fact_id, subject_id, configuration_id, state, value, effective_at, observation_ids, version	The current view of one fact; state "current" or "unresolved"
AdmittedFactChange	change_id, subject_id, fact_kind, changed_fact_ids, prior_view_version, new_view_version, observation_ids, admission_ids, committed_at	One committed change; the input to fan-out
read_current_fact_view	(*, configuration_id, subject_id, fact_kind, fact_id) → GovernedFactView None	The current view, matched exactly as written
current_registration	(source_id) → SourceRegistration None	The highest-version registration
observation_count	() → int	Reports stored

16.5 Dependencies, Fan-Out, and Snapshots

An execution is affected by a fact only if it declared a dependency on it. Dependencies are never discovered. Each binding names one fact exactly—configuration, subject, fact kind, and fact id—and every registration or retirement advances the execution's dependency descriptor, so a snapshot taken before the change no longer validates. A snapshot records the version of every fact the execution depends on; a consequential step validates it inside its own transaction, so a fact that changes in between is caught rather than missed.

Entry — `DependencyService.register_dependency_binding`

Field	Content
Name and label	<code>DependencyService.register_dependency_binding</code> — method; supervision/dependency.py:128. PUBLIC / SUPPORTED (rule R1).
Purpose	Declares that an active execution depends on one governed fact.
Framework basis	Upstream invalidity is not repaired or laundered downstream (C20/D4): what a decision relied on is explicit.

Field	Content
Signature	<code>DependencyService.register_dependency_binding(*, active_execution_id, configuration_id, fact_kind, fact_id, registration_authority, registered_at, subject_id=None, authority_proof=None)</code>
Preconditions	The execution is registered and <code>configuration_id</code> is its configuration; <code>registration_authority</code> is a root authorized for <code>dependency_registration</code> in that configuration. <code>subject_id</code> should be given. If it is omitted, it is inferred only when exactly one current fact view matches the configuration, fact kind, and fact id.
Result	An <code>ExecutionDependencyBinding</code> with status "current". The execution's descriptor version increases by one (the first binding creates the descriptor).
Invariants	<code>fact_kind</code> is canonicalized—lower case, internal whitespace collapsed, ends trimmed—and <code>fact_id</code> is trimmed. One binding per (execution, subject, kind, id).
Failure behavior	<code>ValueError("unknown active execution")</code> , <code>("configuration_conflict")</code> , <code>("subject_id required for exact dependency identity")</code> , <code>("fact_kind required")</code> , <code>("fact_id required")</code> ; <code>PermissionError</code> from the trust provider; <code>sqlite3.IntegrityError</code> for a duplicate binding.
Does not	Check that the fact exists; a later snapshot does. Canonicalize the fact views: views keep the kind as reported, so a binding for "Door_State" looks for a view under "door_state" and a report written "Door_State" never satisfies it; the checkpoint then fails closed with "missing current governed fact" (16.12; confirmation run B.7, <code>test_fact_kinds_must_be_written_in_canonical_form</code>).
Evidence	Direct use: test files 10; e.g. <code>test_r22_phase11_api_misuse.py</code> , <code>test_r22_phase14_conformance_closure.py</code> ; Appendix B.7 (<code>b7_supervise.py</code>); not named in the Supplement.
See also	Guide 12.5; Getting Started 6.3; <code>test_r22_phase15_v0156_hardening</code> ; <code>test_v0156_07_dependency_lookup_is_exact_and_subject_scoped</code> .

Entry — `DependencyService.resolve_affected_executions`

Field	Content
Name and label	<code>DependencyService.resolve_affected_executions</code> — method; supervision/dependency.py:252. PUBLIC / SUPPORTED (rule R1).
Purpose	Finds every active execution with a current binding to a fact that a committed change touched.
Framework basis	Fan-out of one admitted change to every dependent execution; each then commits its own checkpoint against the common basis.
Signature	<code>DependencyService.resolve_affected_executions(change: AdmittedFactChange, *, configuration_id)</code>

Field	Content
Preconditions	change is an AdmittedFactChange returned by commit_admission_change; configuration_id is the configuration to search.
Result	A sorted tuple of active_execution_id values, possibly empty. A "dependency_fanout_resolved" audit event is recorded.
Invariants	Matching is exact on configuration, subject, canonicalized kind, and fact id, over current bindings only.
Failure behavior	Errors from the store only.
Does not	Checkpoint the executions, decide anything, or exclude executions that have finished: the host checkpoints each id returned, and a checkpoint of a finished execution fails with CanonicalResolutionError.
Evidence	Direct use: test files 6; e.g. test_r22_phase10_operational.py, test_r22_phase13_crash_injection.py; Appendix B.7 (b7_supervise.py); not named in the Supplement.
See also	Guide 12.5; test_r22_phase14_conformance_closure: test_phase14_second_fact_change_during_shared_fanout.

Entry — [DependencyService.capture_governance_snapshot](#)

Field	Content
Name and label	DependencyService.capture_governance_snapshot — method; supervision/dependency.py:207. PUBLIC / SUPPORTED (rule R1).
Purpose	Records the exact version of every fact an execution currently depends on.
Framework basis	One state per decision: a continuation decision is tied to the facts it was made on.
Signature	DependencyService.capture_governance_snapshot(active_execution_id, *, created_at)
Preconditions	A binding has been registered for the execution, which creates its descriptor, and every current binding has a fact view.
Result	A GovernanceSnapshotToken, persisted. ContinuationService.checkpoint calls this itself; call it directly only to use evaluate_continuation.
Invariants	The snapshot lists every current binding; none can be omitted.
Failure behavior	ValueError("no registered dependency set"); ValueError("missing current governed fact: <subject>/<kind>/<id>").
Does not	Read fact values: it records versions. Require a current binding: once every binding is retired, an empty snapshot is captured and a checkpoint proceeds with no material facts. A retired identity cannot be bound again for the same execution (sqlite3.IntegrityError): the uniqueness constraint includes retired bindings.

Field	Content
Evidence	Direct use: test files 17; e.g. test_r22_phase10_operational.py, test_r22_phase11_api_misuse.py; not named in the Supplement.
See also	16.6; test_r22_phase11_api_misuse: test_dependency_snapshot_cannot_omit_registered_material_fact.

Entry — `DependencyService.validate_snapshot`

Field	Content
Name and label	<code>DependencyService.validate_snapshot</code> — method; supervision/dependency.py:238. PUBLIC / SUPPORTED (rule R1).
Purpose	Checks that nothing an execution depends on has changed since a snapshot.
Framework basis	As <code>capture_governance_snapshot</code> .
Signature	<code>DependencyService.validate_snapshot(s)</code>
Preconditions	<code>s</code> is a <code>GovernanceSnapshotToken</code> .
Result	True.
Invariants	Valid only if the descriptor id, version, and configuration are unchanged, the material set is the same, and every fact's version is unchanged.
Failure behavior	<code>ConcurrencyConflict("snapshot_conflict: dependency descriptor changed")</code> , <code>("...material set mismatch")</code> , or <code>("...material fact changed")</code> .
Does not	Rewrite history: a stale snapshot fails; the decisions made on it stand as recorded.
Evidence	Direct use: test files 3; e.g. test_r22_phase11_api_misuse.py, test_r22_phase15_strengthened_validation.py; not named in the Supplement.
See also	16.6–16.7; test_r22_phase11_api_misuse: test_snapshot_laundering_stale_descriptor_rejected.

Table 16.12 — Dependency records and other calls

Name	Fields or signature	Meaning
ExecutionDependencyBinding	<code>binding_id</code> , <code>active_execution_id</code> , <code>configuration_id</code> , <code>subject_id</code> , <code>fact_kind</code> , <code>fact_id</code> , <code>registration_authority</code> , <code>registered_at</code> , <code>status</code>	One declared dependency; status "current" or "retired"
DependencySetDescriptor	<code>dependency_set_id</code> , <code>active_execution_id</code> , <code>configuration_id</code> , <code>binding_refs</code> , <code>canonical_fact_identities</code> , <code>descriptor_version</code> , <code>status</code> , <code>derived_at</code>	The execution's current material set, derived on request

Name	Fields or signature	Meaning
GovernanceSnapshotToken	<code>governance_snapshot_id</code> , <code>active_execution_id</code> , <code>configuration_id</code> , <code>dependency_set_id</code> , <code>descriptor_version</code> , <code>current_view_version</code> , <code>material_fact_versions</code> , <code>created_at</code>	<code>material_fact_versions</code> holds (subject, kind, id, version) for each dependency; <code>current_view_version</code> is the highest of them
retire_dependency_binding	(<code>binding_id</code> , *, <code>registration_authority</code> , <code>retired_at</code> , <code>authority_proof=None</code>) → None	Retires a current binding; only the authority that registered it may (<code>PermissionError("unauthorized dependency retirement")</code>); advances the descriptor
derive_dependency_set	(<code>active_execution_id</code> , *, <code>derived_at</code>) → <code>DependencySetDescriptor</code>	The current descriptor; <code>ValueError</code> if none
get_snapshot	(<code>snapshot_id</code>) → <code>GovernanceSnapshotToken</code> None	A stored snapshot

16.6 Continuation and Checkpoints

A checkpoint is the one place where the layer asks the decision core a question. The host calls it with one of two inputs. With an `ExecutionObservation`—a progress report—the checkpoint advances the ε episode by one step through the bridge and maps ε 's answer to a continuation posture. With `GovernanceInvalidationSignals`—a governed fact has changed—it does not advance the episode; it asks the canonical re-entry planner (12.3) which stage the change invalidates. In both cases the posture is the runtime's: a caller or monitor that proposes one is refused.

Entry — `ContinuationService.checkpoint`

Field	Content
Name and label	<code>ContinuationService.checkpoint</code> — method; supervision/continuation.py:495. PUBLIC / SUPPORTED (rule R1).
Purpose	Governs whether a running execution may continue, from a progress report or from changed facts, against a fresh snapshot of everything it depends on.
Framework basis	Governance of in-flight change where instrumentation exists (C45/D2 §8); ε decides progress (C15/D9); re-entry planning decides invalidation (C20/D4; 12.3).
Signature	<code>ContinuationService.checkpoint</code> (*, <code>active_execution_id</code> , <code>observation=None</code> , <code>invalidation_signals=()</code> , <code>trigger_ids=()</code> , <code>checkpoint_at</code>)

Field	Content
Preconditions	checkpoint_at parses as ISO-8601 (a naive time is read as UTC). Pass one input. If invalidation_signals is non-empty, the observation is not passed to ϵ , but its event_id is still recorded as consumed, with no content digest, so a later checkpoint with it raises ObservationIdentityConflict: do not combine the two. Otherwise observation is required. The execution's handle still resolves: its authorization is "issued" and its episode active (16.3). No continuation intent for it is pending reconciliation or terminally fenced.
Result	A ContinuationAssessment (Table 16.14), persisted with its snapshot, its canonical_step, and a "continuation_assessed" audit event in one transaction. Table 16.13 gives the posture. A repeated observation—same event_id, identical content—returns the original assessment without advancing again, provided the handle still resolves: after a terminal step ϵ invalidates the authorization, and a replay raises CanonicalResolutionError.
Invariants	The snapshot is captured inside the call and validated inside the write transaction, before the canonical step; a fact that changed in between makes the call fail rather than decide on stale facts. The observation is frozen once into plain data (Table 16.15), and that one snapshot is digested, stored, validated, and passed to ϵ . A pending or fenced intent is checked before any replay is returned, so an old "continue" never masks a current fence.
Failure behavior	ValueError for an invalid time, a missing observation, or an invalid re-entry plan; ObservationValidationError for an observation outside the plain-data domain; ObservationIdentityConflict for a reused event_id with changed content; ConcurrencyConflict for a snapshot conflict, a checkpoint in progress elsewhere, or a lost race (retry); ContinuationReconciliationRequired when a previous checkpoint's outcome is unknown or the execution is terminally fenced (do not retry; 16.11); CanonicalResolutionError when the lineage no longer resolves; ValueError("no registered dependency set") or ("missing current governed fact: ...") from the snapshot capture (16.5). Errors from ϵ propagate; if ϵ raised after the call was entered, the episode becomes indeterminate.
Does not	Run on a schedule, stop anything, fence authority, or execute re-entry: an invalidation produces a plan reference, and re-entry or a new decision is the host's next step. Map a raw report to a stage: the host builds the GovernanceInvalidationSignal and names the stage the change invalidates.
Evidence	Direct use: test files 10; e.g. test_r22_phase18_v0159_round4_hardening.py, test_r22_phase19_v0160_round5_hardening.py; Appendix B.7 (b7_supervise.py); named in the Guide/Reference Supplement.
See also	Guide 12.6; Getting Started 6.3–6.5; test_r3_phase5_continuation; test_r22_phase23_v0164_round9_hardening; test_r22_phase24_v0165_round10_hardening.

Table 16.13 — How a checkpoint sets the posture

Input	Canonical answer	continuation_posture	authority_fence_required
Invalidation signals	The re-entry plan names a stage to recompute from	"reauthorization_required"; reentry_plan_ref is "reentry:<stage>:<signal ids>"	True
Invalidation signals	No stage invalidated	"continue"	False
Observation	ϵ status "active"	"continue"	False
Observation	ϵ status "emergency_return"	"emergency_return"	True
Observation	Any other status that returns upstream: "aborted_return", "partial_realization", "failed"	"abort_return"	True
Observation	Terminal without return: "completed", "completed_staged"	"continue"; epsilon_status carries the terminal status	False
Observation	Anything else; ϵ produces no such status, so this row is not reached in v2.1.167	"indeterminate"	False

A completed status gives the posture "continue" because no governance-derived control is needed: the episode has ended. Every other terminal status returns upstream and fences. "continue" is therefore not permission to resume or repeat work: read the posture together with epsilon_status, and for a terminal assessment settle canonical finality, the world effect, and Layer 1 (16.8) before anything further is done for the decision. Every governed path into re-entry—K1 and K2 included—is the host's next step, not the checkpoint's: a checkpoint plans re-entry and executes no re-entry executor, so the inherited defects of 15.2 reach a supervised application only if it executes the plan (12.5).

Table 16.14 — Fields of ContinuationAssessment (supervision/continuation.py:22; IMPLEMENTATION / SUPPORT)

Field	Type	Meaning
assessment_id	str	Id of this assessment; a control request names it
active_execution_id	str	The execution
execution_id	str	From the active execution record
attempt	int	From the active execution record
canonical_handle_id	str	The handle resolved for this checkpoint
input_snapshot_id	str	The snapshot the decision used
trigger_ids	tuple[str, ...]	As supplied, for example the AdmittedFactChange id

Field	Type	Meaning
continuation_posture	str	Table 16.13
epsilon_status	str None	ϵ 's status after an observation; None after signals
adaptation_ref	str None	Always None in v2.1.167
reentry_plan_ref	str None	Set when signals invalidate a stage
authority_fence_required	bool	Table 16.13; the owner fences when it is True (16.7)
reason_refs	tuple[str, ...]	ϵ 's notes, or the re-entry reason
evidence_refs	tuple[str, ...]	The signals' evidence ids
committed_snapshot_id	str	Equal to input_snapshot_id; control requests validate it again
assessed_at	str	checkpoint_at as supplied
canonical_step	int None = None	The episode's step_count after the checkpoint; a control request is refused once the step moves on

Table 16.15 — The supervised observation domain

Rule	v2.1.167	On violation
Scalars	None; exact bool; exact int of at most 4,096 bits; exact float, with NaN and $\pm$ infinity encoded explicitly; str of valid Unicode	ObservationValidationError, before any intent or canonical call
Containers	list and tuple, kept distinct; Mapping with str keys, copied once into a sorted plain dict	As above
Depth and cycles	Nesting at most 64; cycles rejected	As above
Type strictness	1 and 1.0 are different content; so are a list and a tuple	A replay with the other type is an ObservationIdentityConflict
event_id	Non-empty str of valid Unicode	ObservationValidationError
Size	No byte limit; content is stored in more than one record	None: the host limits size (16.12)
Other fields	The observation is an ExecutionObservation; its seven flags are exact bool; realized_pv_bundle and information are Mappings	ObservationValidationError

This domain is stricter than ϵ 's own (11.4), which accepts any Mapping. The difference is deliberate: idempotent replay needs a deterministic identity, so an object that is not plain data is refused rather than represented by its repr (RELEASE_NOTES.md, v0.166–v0.167).

Entry — `ContinuationService.evaluate_continuation`

Field	Content
Name and label	<code>ContinuationService.evaluate_continuation</code> — method; supervision/continuation.py:398. PUBLIC / SUPPORTED (rule R1).
Purpose	The checkpoint's decision step, for a caller that captured the snapshot itself.
Framework basis	As checkpoint.
Signature	<code>ContinuationService.evaluate_continuation(*, active_execution_id, snapshot: GovernanceSnapshotToken, observation: ExecutionObservation None = None, invalidation_signals: Iterable[GovernanceInvalidationSignal] = (), trigger_ids: Iterable[str] = (), assessed_at: str, requested_posture: str None = None)</code>
Preconditions	snapshot belongs to this execution and is still valid; assessed_at parses; requested_posture is None. Otherwise as checkpoint.
Result	As checkpoint. checkpoint is capture_governance_snapshot followed by this call.
Invariants	As checkpoint.
Failure behavior	<code>ValueError("caller/monitor continuation posture is not authoritative")</code> for any requested_posture; <code>ConcurrencyConflict("snapshot_conflict: execution scope mismatch")</code> for another execution's snapshot; otherwise as checkpoint.
Does not	Accept a posture from the caller. Prefer checkpoint, which cannot be given a stale snapshot.
Evidence	Direct use: test files 16; e.g. test_r22_phase10_operational.py, test_r22_phase11_api_misuse.py; named in the Guide/Reference Supplement.
See also	test_r3_phase5_continuation: test_caller_cannot_supply_continuation_posture, test_snapshot_execution_scope_cannot_be_laundered.

Cross-system atomicity. The decision core and the store are not one transaction, so the layer does not claim to roll back an episode that has advanced. Before each advance it writes a durable continuation intent; the advance then runs inside the write transaction, and the assessment, the observation's identity, the intent's close, and the audit event commit together. When that sequence is interrupted, the intent records what is known (Table 16.16), and further checkpoints and control requests for the execution are refused until the intent is resolved.

Table 16.16 — Continuation intent states

State	Meaning	What happens next
prepared	A checkpoint is in progress, here or in another supervisor, or was interrupted	<code>ConcurrencyConflict</code> to other callers; do not infer abandonment from elapsed time
closed	The checkpoint committed	Nothing pending

State	Meaning	What happens next
abandoned	Failed before the canonical call was entered: nothing moved	A fresh checkpoint may proceed
reconciliation_required	The canonical call may have moved the episode—it advanced but the assessment was not persisted, or it was entered and raised, or the episode state cannot be read	ContinuationReconciliationRequired until reconcile_continuation_divergence
terminally_fenced	The canonical state cannot be reconstructed	ContinuationReconciliationRequired permanently; start a new governed attempt

Entry — `ContinuationService.reconcile_continuation_divergence`

Field	Content
Name and label	<code>ContinuationService.reconcile_continuation_divergence</code> — method; supervision/continuation.py:306. PUBLIC / SUPPORTED (rule R1).
Purpose	Resolves the most recent unresolved continuation intent of an execution, under root authority, by comparing durable evidence with the live canonical step.
Framework basis	Upstream uncertainty is not laundered: an unknown canonical outcome is either proved or fenced (C20/D4).
Signature	<code>ContinuationService.reconcile_continuation_divergence(*, active_execution_id: str, authority_id: str, authority_proof, reconciled_at: str)</code>
Preconditions	<code>authority_id</code> is a root authorized for continuation_reconciliation in the execution's configuration; the live bridge is available for an outcome other than terminal fencing.
Result	For a prepared, reconciliation_required, or terminally_fenced intent: a dict with <code>reconciliation_id</code> , <code>active_execution_id</code> , <code>intent_id</code> , <code>expected_step</code> , <code>observed_step</code> , <code>outcome</code> , <code>authority_id</code> , <code>reconciled_at</code> , and <code>canonical_state_indeterminate</code> . <code>outcome</code> is "reconciled_closed" when the episode is still at the intent's expected step, or when the latest assessment already records the current step and the intent's snapshot; otherwise "terminally_fenced". A record and an audit event commit with the status change; reconciling a terminally fenced intent again records it again.
Invariants	The decision and the status change are one serialized transaction with compare-and-swap; caller timestamps never order it. An abandoned intent never becomes a fence basis; "reconciliation_required" and "terminally_fenced" are never erased by later history.

Field	Content
Failure behavior	PermissionError from the trust provider; CanonicalResolutionError("stale_lineage: active execution missing"); ConcurrencyConflict when another transaction holds the write boundary or changed the intent (retry); ValueError("continuation_reconciliation_not_required") when nothing is pending—also after it closes an abandoned or superseded intent, which it reports this way.
Does not	Revive a terminally fenced execution, or decide anything about the world: a fenced execution is reconciled outside the runtime and any further action needs a new decision and a new governed attempt.
Evidence	Direct use: test files 6; e.g. test_r22_phase17_v0158_round3_hardening.py, test_r22_phase18_v0159_round4_hardening.py; named in the Guide/Reference Supplement.
See also	Guide 12.9; test_r22_phase17_v0158_round3_hardening; test_r22_phase22_v0163_round8_hardening.

Table 16.17 — `get_assessment`

Method	Returns	Notes
<code>get_assessment(assessment_id)</code>	ContinuationAssessment None	From memory or the store; survives reopening

16.7 Ownership, Control, and Enforcement

Stopping an execution is four records, kept apart because each can be true while the next is not. A control request says the current owner asked, on the strength of an assessment, for a semantic control of a named target. A control attempt says the request was translated, through a registered adapter, into one exact vendor command. An enforcement record says what the controller reported, and whether its attestation verified. What the world shows is a fourth matter, reconciled in 16.8.

Entry — `ControlService.register_supervisory_owner`

Field	Content
Name and label	<code>ControlService.register_supervisory_owner</code> — method; supervision/control.py:34. PUBLIC / SUPPORTED (rule R1).
Purpose	Authorizes an owner id to own executions of one configuration.
Framework basis	Runtime construct (15.6).
Signature	<code>ControlService.register_supervisory_owner(*, owner_id, configuration_id, registration_authority, registered_at, authority_proof=None)</code>
Preconditions	<code>registration_authority</code> is a root authorized for <code>owner_registration</code> in <code>configuration_id</code> ; <code>owner_id</code> and <code>configuration_id</code> are non-empty.
Result	None. The owner is recorded as "current" for the configuration. <code>register_owner_authority</code> is the same call.

Field	Content
Invariants	Re-registering an owner refreshes its record.
Failure behavior	ValueError("owner authority registration fields required"); PermissionError from the trust provider.
Does not	Give ownership of any execution: acquire_ownership does.
Evidence	Direct use: test files 4; e.g. test_r22_phase15_strengthened_validation.py, test_r22_phase15_v0156_hardening.py; Appendix B.7 (b7_supervise.py); not named in the Supplement.
See also	Guide 12.7.

Entry — `ControlService.acquire_ownership`

Field	Content
Name and label	<code>ControlService.acquire_ownership</code> — method; supervision/control.py:36. PUBLIC / SUPPORTED (rule R1).
Purpose	Makes a registered owner the current supervisory owner of one execution, with a new fence.
Framework basis	Runtime construct (15.6). Fencing makes a superseded owner's later acts detectable.
Signature	<code>ControlService.acquire_ownership(*, active_execution_id, owner_id, acquired_at, expires_at=None)</code>
Preconditions	<code>owner_id</code> is registered for the execution's configuration; <code>acquired_at</code> parses; <code>expires_at</code> , if given, parses, is later than <code>acquired_at</code> , and is later than the trusted clock.
Result	A <code>SupervisoryOwnershipRecord</code> with status "current" and <code>owner_fence</code> one more than the previous fence (1 for the first owner).
Invariants	Acquisition supersedes the current owner, whoever it is and whatever its lease: every later act by the previous owner is refused as stale. Lease expiry is judged by the trust provider's clock, never by caller timestamps.
Failure behavior	ValueError for an empty <code>owner_id</code> , an unparseable time, or an expired or inverted lease; PermissionError("owner_unregistered"); KeyError("active execution not found").
Does not	Coordinate supervisors. Any registered owner can take over; deciding who may, and when, is the deployment's (Table 16.27, Profile C).
Evidence	Direct use: test files 8; e.g. test_r22_phase11_api_misuse.py, test_r22_phase12_race_matrix.py; Appendix B.7 (b7_supervise.py); not named in the Supplement.
See also	Guide 12.7; test_r22_phase16_v0157_round2_hardening: test_v0157_01_lease_expiry_uses_trusted_clock_not_caller_timestamp.

Entry — `ControlService.register_adapter`

Field	Content
-------	---------

Entry — `ControlService.issue_control_request`

Field	Content
Name and label	<code>ControlService.issue_control_request</code> — method; supervision/control.py:96. PUBLIC / SUPPORTED (rule R1).
Purpose	Records the current owner's request for one semantic control of one target, on the strength of one committed assessment.
Framework basis	Control is subordinate to governance: the posture limits the request (Table 16.18).
Signature	<code>ControlService.issue_control_request(*, assessment_id, active_execution_id, owner_id, owner_fence, requested_control, target_handle, created_at)</code>
Preconditions	<code>requested_control</code> is in the vocabulary; <code>target_handle</code> is 1 to 64 characters from <code>[A-Za-z0-9_-]</code> ending in <code>:"</code> , <code>/"</code> , or <code>#</code> , followed by a suffix of up to 128 characters from letters, digits, <code>."</code> , <code>_"</code> , <code>:"</code> , and <code>-</code> , with no wildcard, path, or <code>..</code> ; no continuation intent is pending; the caller is the current owner with the current fence and an unexpired lease; the assessment belongs to the execution and its posture permits the control; its snapshot is still valid; the episode is still at the assessment's <code>canonical_step</code> ; <code>created_at</code> parses.
Result	An <code>ExecutionControlRequest</code> , persisted, with the trusted clock time beside the caller's, and a <code>"control_requested"</code> audit event.
Invariants	A request is tied to one assessment, snapshot, owner, and fence.
Failure behavior	<code>ValueError("unsupported semantic control")</code> , <code>("authoritative committed continuation assessment required")</code> , <code>("control_not_permitted_by_continuation_posture")</code> ; <code>TargetScopeViolation</code> ; <code>StaleOwner</code> ; <code>ConcurrencyConflict</code> for a pending intent, a stale snapshot, or a moved canonical step (checkpoint again); <code>CanonicalResolutionError("stale_lineage: episode unavailable")</code> if the bridge no longer holds the episode.
Does not	Send anything, or fence authority.
Evidence	Direct use: test files 13; e.g. <code>test_r22_phase10_operational.py</code> , <code>test_r22_phase11_api_misuse.py</code> ; Appendix B.7 (<code>b7_supervise.py</code>); not named in the Supplement.
See also	Guide 12.7; Getting Started 6.3; <code>test_r22_phase16_v0157_round2_hardening</code> ; <code>test_v0157_07_continue_posture_remains_non_authorizing_for_control</code> .

Table 16.18 — Controls each posture permits

continuation_posture	Permitted controls
continue	None. A healthy execution cannot be stopped through governance; an operator stop would need a separate authority this build does not add

continuation_posture	Permitted controls
reauthorization_required	hold, pause, cancel, reauthorization_required
abort_return	hold, pause, cancel, reauthorization_required, terminate
emergency_return	hold, pause, cancel, reauthorization_required, terminate
indeterminate	hold, pause, reauthorization_required

Entry — [ControlService.dispatch_control_request](#)

Field	Content
Name and label	ControlService.dispatch_control_request — method; supervision/control.py:115. PUBLIC / SUPPORTED (rule R1).
Purpose	Translates a request, through one registered adapter, into one exact vendor command, and records the attempt.
Framework basis	As issue_control_request.
Signature	ControlService.dispatch_control_request(*, control_request_id, adapter_registration_id, owner_id, owner_fence, target_handle, control_region_at_request, capability_evidence_ref, mapping_identity, dispatched_at, dispatch_result=dispatched)
Preconditions	As issue_control_request, re-checked now: no pending intent, current owner and fence, same canonical step, valid snapshot. In addition, the adapter is current, has the request's configuration, and supports the control; target_handle equals the request's and lies in the adapter's namespace; the mapping does not widen.
Result	A ControlAttemptRecord whose mapped_command is the vendor command, with dispatch_result as supplied (default "dispatched"), trusted time, and a "control_dispatched" audit event.
Invariants	Exact target, exact command, recorded owner and fence.
Failure behavior	KeyError("control request not found"); StaleOwner; ConcurrencyConflict; CanonicalResolutionError as issue_control_request; AdapterUnregistered; TargetScopeViolation; ControlMappingWidened.
Does not	Contact the controller. The host sends mapped_command by its own means and reports the result.
Evidence	Direct use: test files 9; e.g. test_r22_phase11_api_misuse.py, test_r22_phase12_race_matrix.py; Appendix B.7 (b7_supervise.py); not named in the Supplement.
See also	Guide 12.7; test_r22_phase11_api_misuse: test_target_swap_outside_exact_request_is_rejected.

Entry — [ControlService.record_enforcement_outcome](#)

Field	Content
Name and label	ControlService.record_enforcement_outcome — method; supervision/control.py:141. PUBLIC / SUPPORTED (rule R1).

Table 16.19 — Enforcement outcomes

Outcome	Controller's report
accepted	Received the command; has not finished
completed	Carried it out
rejected	Refused it
too_late	The target had already finished or passed the point where the control applies
timed_out	No answer in time; the effect stays unknown
unknown	No usable report

Entry — `ControlService.fence_execution_authority`

Field	Content
Name and label	<code>ControlService.fence_execution_authority</code> — method; supervision/control.py:149. PUBLIC / SUPPORTED (rule R1).
Purpose	Invalidates every issued, unused native authorization of the execution's episode.
Framework basis	Stale-lineage retirement for native authority (11.6).
Signature	<code>ControlService.fence_execution_authority(*, active_execution_id, owner_id, owner_fence, reason)</code>
Preconditions	The caller is the current owner with the current fence and an unexpired lease.
Result	The tuple of authorization ids invalidated, from <code>PVPPRuntime.invalidate_native_execution_authorizations(episode_id=...)</code> . The runtime also retires the episode (Table 11.6), so no new authorization can be issued for it.
Invariants	Only "issued" authorizations change.
Failure behavior	StaleOwner.
Does not	Consult the posture: fence when an assessment's <code>authority_fence_required</code> is True, and not otherwise. Revoke an authorization already consumed, or stop a call already entered. After fencing, the execution's handle no longer resolves, so it cannot be checkpointed again; requests based on assessments already made remain possible.
Evidence	Direct use: test files 3; e.g. <code>test_r22_phase12_race_matrix.py</code> , <code>test_r22_phase6_control.py</code> ; Appendix B.7 (<code>b7_supervise.py</code>); not named in the Supplement.
See also	Guide 12.7; Getting Started 6.3; B.7.

Table 16.20 — Control records

Record	Fields	Notes
SupervisoryOwnershipRecord	ownership_id, active_execution_id, execution_id, attempt, owner_id, owner_fence, acquired_at, expires_at, store_version, status	status "current"; owner_fence increases with each acquisition
AdapterRegistration	adapter_registration_id, adapter_id, configuration_id, target_handle_namespace, supported_semantic_controls, command_mapping, vendor_command_mapping, controller_attestation_method, control_region_evidence_method, status, version, registration_authority, registered_at	One registration per call; no public call retires one
ExecutionControlRequest	control_request_id, active_execution_id, execution_id, attempt, owner_id, owner_fence, governance_snapshot_id, continuation_assessment_id, requested_control, target_handle, configuration_id, created_at, trusted_created_at	trusted_created_at is the trust provider's clock
ControlAttemptRecord	control_attempt_id, control_request_id, adapter_registration_id, adapter_version, target_handle, requested_control, mapped_command, control_region_at_request, capability_evidence_ref, mapping_identity, dispatched_at, dispatch_result, owner_id, owner_fence, trusted_dispatched_at	mapped_command is the vendor command
EnforcementRecord	enforcement_record_id, control_request_id, control_attempt_id, adapter_registration_id, outcome, attestation_status, controller_evidence_ref, observed_control_region, recorded_at, owner_fence_at_attempt	attestation_status "verified", "weak", or "unverified"

16.8 Effect, Finality, and Layer 1

Three facts about an execution are recorded separately and never derived from one another. Canonical finality is what ϵ concluded: once accepted, it is immutable. The world-effect state is what authoritative evidence shows the world did; each reconciliation replaces it. The Layer-1 record is the host's statement that it applied the transition to actual state, verified by the trust provider. A controller's acknowledgment is none of these.

Entry — `EffectService.submit_effect_evidence`

Field	Content
Name and label	<code>EffectService.submit_effect_evidence</code> — method; supervision/effect.py:71. PUBLIC / SUPPORTED (rule R1).

Field	Content
Purpose	Stores one report of what the world shows about an execution's effect.
Framework basis	Layer 1 owns the transition (C10/D2); evidence authority (C10/D16).
Signature	EffectService.submit_effect_evidence(*, active_execution_id, source_id, configuration_id, state, realized_bundle_ref, evidence_ref, provenance, observed_at, effective_at, received_at)
Preconditions	The execution exists and has configuration_id; source_id's current registration is "current", has evidence role "effect", permitted use "effect_reconciliation", and covers the configuration; state is one of the five reportable states of Table 16.21.
Result	An EffectEvidence record and an "effect_evidence_admitted" audit event.
Invariants	Evidence is keyed to the execution and attempt.
Failure behavior	KeyError("active execution not found"); ValueError("evidence_unverified: source lacks current effect role"), ("source_scope_violation"), ("configuration_conflict"), ("invalid effect evidence state").
Does not	Change the effect state: reconcile_effect does.
Evidence	Direct use: test files 9; e.g. test_r22_phase11_api_misuse.py, test_r22_phase12_race_matrix.py; Appendix B.7 (b7_supervise.py); not named in the Supplement.
See also	Guide 12.8; test_r22_phase11_api_misuse: test_unregistered_effect_source_cannot_close_effect.

Entry — EffectService.reconcile_effect

Field	Content
Name and label	EffectService.reconcile_effect — method; supervision/effect.py:112. PUBLIC / SUPPORTED (rule R1).
Purpose	Moves the execution's world-effect state forward to what one piece of evidence shows.
Framework basis	As submit_effect_evidence.
Signature	EffectService.reconcile_effect(*, active_execution_id, effect_evidence_id, updated_at, control_enforcement_refs=(), compensation_refs=())
Preconditions	effect_evidence_id names stored evidence. That it is evidence for this execution is not checked: pass only this execution's evidence.
Result	A WorldEffectReconciliationState with version one higher, the evidence's state, unresolved True for "effect_unknown" and "partial_effect", the accumulated authoritative sources, the enforcement and compensation references supplied, and the canonical finality id if one was accepted; an "effect_reconciled" audit event.

Field	Content
Invariants	The most recently reconciled evidence replaces the state, whatever its effective time: reconcile evidence in the order it should count. Canonical finality is never rewritten (test_r22_phase12_race_matrix: test_race08_late_effect_after_canonical_finality_reconciles_forward).
Failure behavior	KeyError; ValueError("evidence_unverified") for unknown evidence.
Does not	Infer an effect from enforcement, or turn a timeout into no effect.
Evidence	Direct use: test files 9; e.g. test_r22_phase11_api_misuse.py, test_r22_phase12_race_matrix.py; Appendix B.7 (b7_supervise.py); not named in the Supplement.
See also	Guide 12.8; Getting Started 6.5; B.7.

Table 16.21 — World-effect states

State	Reportable	Unresolved	Meaning
open	No	Yes	No evidence yet: what current_effect_state returns before the first reconciliation
no_material_effect_evidenced	Yes	No	Evidence shows no material effect
partial_effect	Yes	Yes	Part of the effect happened
completed_effect	Yes	No	The effect happened
invalid_effect	Yes	No	An effect happened that the license did not cover
effect_unknown	Yes	Yes	Evidence cannot settle it
resolved	No	—	Declared in the vocabulary; no public call produces it

Entry — EffectService.accept_canonical_finality

Field	Content
Name and label	EffectService.accept_canonical_finality — method; supervision/effect.py:85. PUBLIC / SUPPORTED (rule R1).
Purpose	Records, once and immutably, how the canonical episode ended.
Framework basis	ε ends at realization outputs (C15/D9).
Signature	EffectService.accept_canonical_finality(*, active_execution_id, terminalized_at, continuation_assessment_id=None, epsilon_terminal_status=None, execution_path_ref=None, realized_bundle_ref=None, terminal_evidence_ref=None, governance_snapshot_id=None, layer1_handoff_ref=None)

Field	Content
Preconditions	continuation_assessment_id names a stored assessment of this execution whose epsilon_status is terminal: "completed", "completed_staged", "partial_realization", "aborted_return", "failed", or "emergency_return". epsilon_terminal_status, if given, equals it.
Result	A CanonicalEpisodeFinality with immutable True; the snapshot defaults to the assessment's; a "canonical_finality_accepted" audit event.
Invariants	One finality per execution; the status comes from the assessment, never from the caller.
Failure behavior	ValueError for a missing, mismatched, or non-terminal assessment, or a caller status that differs; ConcurrencyConflict("canonical_finality_conflict") for a second acceptance.
Does not	Settle the world effect or apply Layer 1.
Evidence	Direct use: test files 6; e.g. test_r22_phase11_api_misuse.py, test_r22_phase12_race_matrix.py; Appendix B.7 (b7_supervise.py); not named in the Supplement.
See also	Guide 12.8; test_r22_phase15_v0156_hardening: test_v0156_04_canonical_finality_cannot_be_asserted_while_episode_active; B.7.

Entry — `EffectService.record_layer1_transition`

Field	Content
Name and label	<code>EffectService.record_layer1_transition</code> — method; supervision/effect.py:161. PUBLIC / SUPPORTED (rule R1).
Purpose	Records the host's transition of actual state for an execution, after verifying the host's authority for it.
Framework basis	Layer 1 is the host's mutation boundary (C10/D2; 10.4).
Signature	<code>EffectService.record_layer1_transition(*, active_execution_id, effect_reconciliation_ref, prior_actual_state_ref, next_actual_state_ref, transition_time, evidence_ref, provenance, configuration_id, authority_evidence=None, host_authoritative=None)</code>
Preconditions	configuration_id is the execution's; effect_reconciliation_ref is the id of the current effect state, which has been reconciled at least once; the trust provider's Layer-1 verifier accepts authority_evidence.
Result	A Layer1TransitionRecord naming the prior and next actual-state references, the effect state, and the canonical finality if any; a "layer1_transition_committed" audit event.
Invariants	host_authoritative is ignored: only the verifier's answer counts.
Failure behavior	KeyError; ValueError("configuration_conflict"), ("layer1_conflict: current effect reconciliation required"), ("layer1_conflict: verified host-authoritative transition required").

Field	Content
Does not	Apply the transition, call a <code>Layer1TransitionService</code> , or build a <code>Layer1TransitionHandoff</code> . A supervised checkpoint returns an assessment, not the ϵ step result that <code>build_layer1_transition_handoff</code> needs, so the host applies its transition by its own means and records it here (15.1 L8).
Evidence	Direct use: test files 8; e.g. <code>test_r22_phase10_operational.py</code> , <code>test_r22_phase12_race_matrix.py</code> ; Appendix B.7 (<code>b7_supervise.py</code>); not named in the Supplement.
See also	Guide 12.8; <code>test_r22_phase12_race_matrix</code> : <code>test_race09_layer1_waits_for_host_authoritative_transition</code> ; B.7.

Entry — `EffectService.register_retry_contract`

Field	Content
Name and label	<code>EffectService.register_retry_contract</code> — method; <code>supervision/effect.py:147</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Declares, before anything can have happened, whether the execution's action may be retried without reconciliation.
Framework basis	Retry safety is established before uncertainty, never after.
Signature	<code>EffectService.register_retry_contract(*, active_execution_id, idempotent, reconciliation_required, registered_at)</code>
Preconditions	The execution's authorization is "issued" according to the live bridge; no supervised native invocation, control dispatch, or effect evidence exists for it.
Result	None. The contract is stored once per execution attempt.
Invariants	Once only: a second registration raises <code>sqlite3.IntegrityError</code> .
Failure behavior	<code>ValueError("retry contract must be pre-execution/pre-dispatch; ...")</code> when any precondition fails, including after a store is reopened without its live bridge.
Does not	Use <code>reconciliation_required</code> : it is stored and not consulted by <code>assert_retry_safe</code> .
Evidence	Direct use: test files 6; e.g. <code>test_r22_phase15_v0156_hardening.py</code> , <code>test_r22_phase16_v0157_round2_hardening.py</code> ; Appendix B.7 (<code>b7_supervise.py</code>); not named in the Supplement.
See also	<code>test_r22_phase16_v0157_round2_hardening</code> : <code>test_v0157_02_retry_contract_cannot_be_added_after_dispatch</code> , <code>test_v0157_11_retry_contract_cannot_be_added_after_supervised_native_invocation</code> ; B.7.

Entry — `EffectService.assert_retry_safe`

Field	Content
Name and label	<code>EffectService.assert_retry_safe</code> — method; supervision/effect.py:154. PUBLIC / SUPPORTED (rule R1).
Purpose	Says whether retrying the execution's action is established as safe.
Framework basis	As <code>register_retry_contract</code> .
Signature	<code>EffectService.assert_retry_safe(*, active_execution_id)</code>
Preconditions	The execution exists.
Result	True when an idempotent contract was registered, or when the current effect state is "no_material_effect_evidenced".
Invariants	An idempotent contract makes a retry safe whatever the effect state; without one, only evidence of no material effect does.
Failure behavior	<code>ValueError("effect_unresolved")</code> when the state is unresolved, including "open"; <code>ValueError("retry_not_established_safe")</code> otherwise.
Does not	Retry anything, or, without an idempotent contract, make a timed-out or unknown effect safe.
Evidence	Direct use: test files 3; e.g. <code>test_r22_phase11_api_misuse.py</code> , <code>test_r22_phase13_crash_injection.py</code> ; Appendix B.7 (<code>b7_supervise.py</code>); not named in the Supplement.
See also	Guide 12.8; <code>test_r22_phase11_api_misuse</code> : <code>test_retry_safety_cannot_be_declared_post_hoc_after_unknown_effect</code> .

Table 16.22 — Effect records and queries

Name	Fields or signature	Notes
EffectEvidence	<code>effect_evidence_id</code> , <code>execution_id</code> , <code>attempt</code> , <code>source_id</code> , <code>configuration_id</code> , <code>state</code> , <code>realized_bundle_ref</code> , <code>observed_at</code> , <code>effective_at</code> , <code>received_at</code> , <code>evidence_ref</code> , <code>provenance</code>	One report
WorldEffectReconciliationState	<code>effect_record_id</code> , <code>execution_id</code> , <code>attempt</code> , <code>version</code> , <code>state</code> , <code>realized_bundle_ref</code> , <code>authoritative_source_refs</code> , <code>control_enforcement_refs</code> , <code>observed_at</code> , <code>effective_at</code> , <code>received_at</code> , <code>unresolved</code> , <code>canonical_finality_ref</code> , <code>compensation_refs</code> , <code>updated_at</code>	Version 0 and state "open" before evidence
CanonicalEpisodeFinality	<code>canonical_finality_id</code> , <code>active_execution_id</code> , <code>execution_id</code> , <code>attempt</code> , <code>episode_id</code> , <code>epsilon_terminal_status</code> , <code>execution_path_ref</code> , <code>realized_bundle_ref</code> , <code>terminal_evidence_ref</code> , <code>terminalized_at</code> , <code>governance_snapshot_id</code> , <code>layer1_handoff_ref</code> , <code>immutable</code>	<code>terminal_evidence_ref</code> defaults to the assessment id

Name	Fields or signature	Notes
Layer1TransitionRecord	layer1_transition_id, execution_id, attempt, canonical_finality_ref, effect_reconciliation_ref, prior_actual_state_ref, next_actual_state_ref, transition_time, evidence_ref, provenance, configuration_id	References, not states
current_effect_state	(active_execution_id) → WorldEffectReconciliationState	Current state, or a fresh "open" state with version 0
get_canonical_finality	(active_execution_id) → CanonicalEpisodeFinality None	

16.9 Recovery

After a crash, durable records survive and live state does not: the bridge, its episodes and authorizations, and every in-memory cache are gone. Recovery therefore never resumes from history. A new owner begins recovery against a profile registered, by the host's discipline, before the execution ran; reconciles control, effect, and Layer 1 from evidence, takes a fresh snapshot, and only then may resume supervisory control. An execution whose continuation was terminally fenced cannot be revived at all.

Table 16.23 — The recovery sequence

Call	Requires	Establishes
register_recovery_profile (before execution, by the host's discipline)	Root authority for recovery_registration	What reconciliation must show before resumption
begin_recovery (active_execution_id, new_owner_id, started_at)	A profile; a registered owner	Ownership with a new fence; a record in state "recovering" with control status "unknown" and Layer 1 "unresolved"
reconcile_recovery (..., external_control_status, effect_status, layer1_world_state_status)	The current owner and fence	Each requirement checked from records (Table 16.24); state "reconciled", "unresolved", or "conflict"
capture_recovery_snapshot (...)	The current owner and fence; call it after the final reconcile_recovery, which clears any earlier snapshot	A fresh governance snapshot on the record
complete_recovery (...)	State "reconciled"; every requirement met; a fresh snapshot that still validates, if the profile requires freshness	control_resumption_allowed True

Call	Requires	Establishes
assert_resumption_allowed(...)	The current owner and fence—any current owner, including one that acquired ownership after recovery completed	True, or <code>ValueError("recovery_requirement_unsatisfied")</code>

Entry — `RecoveryService.register_recovery_profile`

Field	Content
Name and label	<code>RecoveryService.register_recovery_profile</code> — method; <code>supervision/recovery.py:43</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Fixes in advance what a recovering owner must reconcile before resuming control.
Framework basis	Runtime construct (15.6).
Signature	<code>RecoveryService.register_recovery_profile(*, active_execution_id, required_control_reconciliation, required_effect_evidence, required_world_layer1_checks, retry_idempotency_prerequisites=False, compensation_prerequisites=False, freshness_dependency_requirements=True, unresolved_policy=remain_fenced, registration_authority, registered_at, authority_proof=None)</code>
Preconditions	<code>registration_authority</code> is a root authorized for <code>recovery_registration</code> ; no profile exists for the execution's action, binding, and configuration.
Result	A <code>RecoveryRequirementProfile</code> with version 1 and status "current".
Invariants	One profile per (action, binding, configuration), shared by every execution of that combination; once registered it cannot be replaced, so the recovering owner cannot weaken it. The call does not check timing: a first profile can be registered at any time, even after invocation, by a <code>recovery_registration</code> root.
Failure behavior	<code>KeyError</code> ; <code>PermissionError</code> ; <code>ConcurrencyConflict("recovery profile already established; changes are prospective/versioned")</code> .
Does not	Use <code>retry_idempotency_prerequisites</code> , <code>compensation_prerequisites</code> , or <code>unresolved_policy</code> : they are recorded; <code>reconcile_recovery</code> checks control, effect, and Layer 1, and <code>complete_recovery</code> checks freshness.
Evidence	Direct use: test files 4; e.g. <code>test_r22_phase11_api_misuse.py</code> , <code>test_r22_phase15_strengthened_validation.py</code> ; not named in the Supplement.
See also	Guide 12.9; <code>test_r22_phase16_v0157_round2_hardening</code> ; <code>test_v0157_12_recovery_profile_requires_host_root_proof</code> .

Entry — `RecoveryService.reconcile_recovery`

Field	Content
Name and label	<code>RecoveryService.reconcile_recovery</code> — method; <code>supervision/recovery.py:100</code> . PUBLIC / SUPPORTED (rule R1).
Purpose	Checks the profile's requirements against durable evidence and records the result.
Framework basis	History is evidence, never authority.
Signature	<code>RecoveryService.reconcile_recovery(*, active_execution_id, owner_id, owner_fence, external_control_status, effect_status, layer1_world_state_status, evidence_refs=(), provenance=live-reconciliation)</code>
Preconditions	Recovery has begun; the caller is the current owner, with the fence recovery began under.
Result	The updated <code>RecoveryReconciliationRecord</code> , with <code>requirement_results</code> <code>(("control", bool), ("effect", bool), ("layer1", bool))</code> and <code>recovery_state</code> "reconciled" when all hold, "conflict" when any of the three status arguments is "conflict", and "unresolved" otherwise.
Invariants	The caller's status strings cannot satisfy a requirement (Table 16.24).
Failure behavior	<code>ValueError("recovery not begun"); PermissionError("stale_owner")</code> .
Does not	Resume anything: <code>complete_recovery</code> decides.
Evidence	Direct use: test files 2; e.g. <code>test_r22_phase15_v0156_hardening.py</code> , <code>test_r22_phase8_recovery.py</code> ; not named in the Supplement.
See also	Guide 12.9; <code>test_r22_phase15_v0156_hardening</code> : <code>test_v0156_06_recovery_claim_strings_cannot_substitute_for_control_and_layer1_evidence</code> .

Table 16.24 — What `reconcile_recovery` checks

Requirement	Met when the profile does not require it, or when
control	Some enforcement record for the execution is attested "verified" with outcome "completed", "rejected", or "too_late"
effect	The effect state has been reconciled, is not unresolved, and equals the <code>effect_status</code> argument
layer1	A <code>Layer1TransitionRecord</code> exists for the execution, attempt, and configuration

Table 16.25 — Recovery records and other calls

Name	Fields or signature	Notes
RecoveryRequirement Profile	recovery_profile_id, action_id, binding_id, configuration_id, required_control_reconciliation, required_effect_evidence, required_world_layer1_checks, retry_idempotency_prerequisites, compensation_prerequisites, freshness_dependency_requirements, unresolved_policy, registration_authority, version, status, registered_at	freshness_dependency_requirements defaults to True; unresolved_policy to "remain_fenced"
RecoveryReconciliationRecord	recovery_id, active_execution_id, execution_id, attempt, new_owner_id, owner_fence, recovery_profile_ref, durable_state_version, canonical_finality_status, canonical_finality_ref, external_control_status, effect_status, effect_ref, layer1_world_state_status, requirement_results, recovery_state, control_resumption_allowed, governance_snapshot_id, evidence_refs, provenance, started_at, completed_at	canonical_finality_status "terminal" or "active"
begin_recovery	(*, active_execution_id, new_owner_id, started_at, provenance='durable-restart')	ValueError("recovery_profile_missing") ; PermissionError("owner_unregistered")
capture_recovery_snapshot	(*, active_execution_id, owner_id, owner_fence, created_at) → GovernanceSnapshotToken	PermissionError("stale_owner")
complete_recovery	(*, active_execution_id, owner_id, owner_fence, completed_at)	ValueError("recovery_requirement_unsatisfied...")
assert_resumption_owed	(*, active_execution_id, owner_id, owner_fence) → True	PermissionError("stale_owner") or ValueError
current_profile, get_recovery	(active_execution_id)	The profile or record, or None

Live bindings are not durable. A store reopened in a new process has its records but no bridge for them, and there is no public way to give it one. Every operation that needs the live canonical state therefore fails closed for an execution from before the restart: checkpoints, control requests, and a retry contract (the last tested in test_r22_phase19_v0160_round5_hardening:

test_v0160_04_reopened_store_fails_closed_when_canonical_status_unavailable). Recovery reconciles and records such an execution; further action needs a new decision and a new governed attempt.

16.10 The Store, Concurrency, and Deployment

Every service writes to one SQLiteSupervisoryStore. The store is the layer's transaction boundary: consequential steps take a serialized write transaction (BEGIN IMMEDIATE) and change status by compare-and-swap, so of two supervisors racing for the same step one wins and the other receives ConcurrencyConflict. A store at a file path uses write-ahead logging with full synchronization; ":memory:" is for tests and in-process use and does not survive the process. These are local guarantees of one store: they do not provide distributed coordination, crash-free external effects, or production-grade security, and the successor tests that exercise them (Table E.6) establish no more than that.

Entry — SQLiteSupervisoryStore

Field	Content
Name and label	SQLiteSupervisoryStore — class; supervision/store.py:23. PUBLIC / ADVANCED (rule R2; direct store access is privileged).
Purpose	Holds every supervisory record of one deployment boundary, its audit trail, and its trust provider.
Framework basis	Runtime construct (15.6).
Signature	SQLiteSupervisoryStore(path: str Path, *, trust_provider: HostTrustProvider None = None) → None
Preconditions	path is a file path or ":memory:"; trust_provider is the deployment's HostTrustProvider (16.2). Construct the services over the same store object.
Result	An open store; tables are created as needed. It is a context manager and closes on exit.
Invariants	Supervisory records are written by the services; the audit table is append-only. Live canonical bindings are held in memory only (16.9).
Failure behavior	sqlite3 errors for an unusable path; the services translate contention into ConcurrencyConflict where they serialize.
Does not	Provide a coordination service across processes or machines, or authority: direct store access is privileged and the services, not the store, enforce the contracts of this chapter. Its INTERNAL and default-labeled methods (Table 16.26) are not an application surface.
Evidence	Direct use: test files 10; e.g. r22_trust_fixture.py, test_r22_phase12_race_matrix.py; Appendix B.7 (b7_supervise.py); named in the Guide/Reference Supplement.
See also	Guide 12.10; test_r3_phase1_supervisory_store; test_r22_phase12_race_matrix.

Table 16.26 — Store methods

Method	Label	Use
audit_events(*, configuration_id=None) → list[AuditEvent]	SUPPORTED	The audit trail in commit order, optionally for one configuration
create_configuration, get_configuration, compare_and_swap_configuration	ADVANCED	A versioned configuration record; <code>compare_and_swap_configuration</code> raises <code>ConcurrencyConflict</code> unless <code>expected_version</code> is current
get_active_execution(active_execution_id)	ADVANCED	The <code>ActiveExecutionRecord</code> , or <code>None</code>
current_supervisory_ownership(active_execution_id)	ADVANCED	The current ownership as a dict, or <code>None</code>
now(), close()	ADVANCED	The trust provider's clock; close the connection
bind_canonical_bridge, register_active_execution	LEGACY / COMPATIBILITY	Always refuse (<code>PermissionError</code>): register through the bridge (16.3)
phase6_put, phase6_get	INTERNAL	Raw table access; do not use
acquire_supervisory_ownership, register_supervisory_owner_authority, supervisory_owner_authorized, require_registration_authority, record_supervisory_operation, append_audit_event, canonical_authorization_status, continuation_intent_is_prepared, continuation_reconciliation_required, has_supervised_native_invocation, record_supervised_native_invocation	IMPLEMENTATION / SUPPORT	Called by the services; use the service methods instead (H.5-10). Writing audit events directly adds evidence, never authority

Table 16.27 — Deployment profiles

Profile	Shape	What the deployment must supply
A — in-process	One process, one supervisor; <code>":memory:"</code> or a local file	Nothing more; every piece of code in the process is inside the trusted computing base
B — durable single service	A store file and one logical supervisor service	Restart handling as 16.9: durable records survive, live executions do not resume—continuing needs a new decision and a new governed attempt; the reference operational target

Profile	Shape	What the deployment must supply
C — distributed	Several supervisors	A coordination and fencing authority visible at every consequential step. The API semantics allow it; build 167 supplies no coordinator

Table 16.28 — Audit event kinds

Event kind	Recorded by
observation_submitted, observation_assessed, fact_change_committed	ObservationService
dependency_fanout_resolved	DependencyService.resolve_affected_executions
continuation_assessed, continuation_divergence_reconciled	ContinuationService, in the same transaction as the change
control_requested, control_dispatched, enforcement_recorded	ControlService
effect_evidence_admitted, effect_reconciled, canonical_finality_accepted, layer1_transition_committed	EffectService

AuditEvent has event_id, event_kind, occurred_at (the caller's time for the operation), configuration_id, subject_id (usually the active execution), and payload, which names the initiating event, the operation, the result reference, the invariant status, and the authority source where one applies. Ownership acquisition, registrations, dependency retirement, fencing, the retry contract, the native-invocation record, and recovery steps write their records but no audit event. ConfigurationState has configuration_id, version, payload, and updated_at. SemanticId, the generator of the layer's prefixed ids, is INTERNAL.

16.11 Errors

Unlike the decision core, which raises only built-in exceptions (14.7), the supervision layer defines typed exceptions, and an application should handle them by type. Three distinctions matter. A ConcurrencyConflict is a race or a stale basis: retry, usually after a fresh checkpoint. A ContinuationReconciliationRequired, which is a kind of ConcurrencyConflict, means a retry cannot succeed: test for it first, because it is a ConcurrencyConflict too. Everything else is a refusal on the facts or the caller's authority: a blind retry of the same call will not succeed, though a later, authorized change—a new admitted fact, a new registration, a new owner, a new decision—may legitimately change the outcome.

Table 16.29 — Supervision exceptions

Exception	Base	Raised by	Meaning and handling
ConcurrencyConflict	RuntimeError	Every service and the store	Contention, a stale snapshot, step, or version, a duplicate record, or a second finality. Retry after a fresh checkpoint

Exception	Base	Raised by	Meaning and handling
ContinuationReconciliationRequired	ConcurrencyConflict	ContinuationService	A previous checkpoint's outcome is unknown, or the execution is terminally fenced. Do not retry; reconcile (16.6)
ObservationIdentityConflict	ConcurrencyConflict	ContinuationService	An event_id reused with different content. A caller error, not a race
StaleOwner	ConcurrencyConflict	ControlService	The caller is not the current owner, its fence is old, its lease has expired, or its authority was withdrawn
ObservationValidationError	ValueError	ContinuationService	The observation is outside the plain-data domain (Table 16.15); no intent, assessment, or canonical call was made (the checkpoint's snapshot is already stored)
CanonicalResolutionError	RuntimeError	CanonicalRuntimeBridge, ContinuationService, ControlService	The canonical lineage no longer resolves: stale, terminal, consumed, invalidated, indeterminate, or a guard refusal
ConfigurationConflict	CanonicalResolutionError	resolve_canonical_execution	A handle requested under the wrong configuration
TargetScopeViolation	RuntimeError	ControlService	A malformed namespace or target, or a target outside the request or adapter
ControlMappingWidened	RuntimeError	ControlService	A mapping to a stronger control or outside the vocabulary
AdapterUnregistered	RuntimeError	ControlService	No current adapter, wrong configuration, or unsupported control
EvidenceUnverified	RuntimeError	None	Defined and exported; no code raises it. Unverified evidence appears as ValueError("evidence_unverified...") or an "unverified" attestation status (H.5-11)

Table 16.30 — Built-in exceptions the layer raises

Type	Raised for
PermissionError	Registration authority refused by the trust provider; an unregistered owner; retirement by another authority; a stale owner in RecoveryService (which raises PermissionError("stale_owner"), not StaleOwner); the store's refusals
ValueError	Invalid times, statuses, controls, and states; posture refusals; missing records; snapshot capture with a missing fact; retry and recovery refusals; Layer-1 refusals; "continuation_reconciliation_not_required"
KeyError	An unknown active execution in ControlService, EffectService, and RecoveryService (DependencyService raises ValueError, and reconcile_continuation_divergence CanonicalResolutionError); an unknown control request or control attempt
TypeError	A non-episode or non-observation passed to the bridge directly; content that does not serialize
sqlite3.IntegrityError	A duplicate source registration, dependency binding, or retry contract: these inserts are not translated

Handle a stale owner from both services: catch StaleOwner from ControlService and PermissionError from RecoveryService. Match on type and context, never on message text (1.7).

16.12 Boundaries and Host Duties

The layer governs only what the host reports, at checkpoints the host calls, for dependencies the host declares, and stops only what the host's controllers stop. Table 16.31 lists what the host must therefore do. A duty marked "Fails closed" is one the layer detects and refuses; "Relied on" means a violation produces results that look valid. The decision core's duties are in 15.3.

Table 16.31 — Host duties for supervised execution

Duty	If neglected	Section
Report the facts an execution relies on; the runtime senses nothing	Relied on: the change is never seen	16.4
Call checkpoints, after each fan-out and at each progress point; the runtime schedules nothing	Relied on: the execution runs ungoverned	16.6
Declare every material dependency before the first checkpoint	Relied on: an undeclared fact never affects the execution	16.5
Map a reported change to the canonical stage it invalidates when building the signal	Relied on: a wrong stage gives a wrong plan	16.6
Fence when authority_fence_required is True	Relied on: fencing is not gated by the posture	16.7

Duty	If neglected	Section
Carry out every dispatched command and report what the controller says, with its attestation	Relied on: a request stops nothing by itself	16.7
Reconcile the world effect from authoritative evidence; never treat an acknowledgment or a timeout as an effect	Relied on	16.8
Keep the authorization issued while checkpoints are needed: act by your own means, or invoke natively only when none will follow	Fails closed: checkpoints raise CanonicalResolutionError	16.3; 15.1 L8
Apply the Layer-1 transition yourself, then record it	Relied on: the record is not the transition	16.8
Supply real trust: roots, proofs kept secret, verifiers that check, a trusted clock	Relied on: weak verifiers give weak trust	16.2
Write fact kinds in canonical form	Fails closed at the checkpoint	16.5; B.7
Write every observation time in one fixed ISO-8601 form, offset, and precision	Relied on: times are compared as text; an older report can displace a newer view	16.4; B.7
Limit observation size; store bulky evidence by reference	Relied on: storage grows	Table 16.15
Register retry contracts before execution	Fails closed afterwards	16.8
Register recovery profiles before execution	Relied on: a first profile is accepted at any time	16.9
After a restart, reconcile and start a new governed attempt; do not expect to resume	Fails closed	16.9
Coordinate supervisors across processes (Profile C)	Relied on: any registered owner can take over	16.10
Advance and invoke supervised executions only through the services; treat in-process code as trusted	Fails closed for the public guards; in-process code can bypass anything	16.3
Treat audit as evidence only	Relied on	16.10

What remains outside the build entirely: universal physical preemption; exactly-once external effects; automatic dependency discovery; authoritative knowledge of the actual world; governance of a native call once entered; autonomous sensing; and distributed supervision without an external coordinator. An effect that cannot be interrupted, such as a payment sent or a message delivered, is not undone by any stop request. These are the residue of limitation L1 (15.1).

16.13 Simulation and Operational Reference Adapters

The package supplies two sets of thin classes, both PUBLIC / REFERENCE. The operational adapters show the shape of a host boundary; the simulation classes let a test control timing, inject external outcomes, and simulate a crash. Every adapter, source, and injector delegates to the services of this chapter, so simulation runs under the same authority model and records as operation; the scheduler calls only the callables it is given, and the audit oracle reads the store directly (test_r22_phase14_conformance_closure: test_phase14_simulation_operational_parity). A deployment may use them or write its own.

Table 16.32 — Reference adapters and the simulation harness

Class and method	Delegates to
OperationalObservationAdapter(service).admit(observation, *, assessed_at, committed_at, permitted_use='governance')	submit_observation, assess_observation, commit_admission_change; returns (assessment, change)
OperationalEnforcementAdapter(service).dispatch_exact(**kw), report_outcome(**kw)	dispatch_control_request, record_enforcement_outcome
OperationalEffectAdapter(service).admit_and_reconcile(*, updated_at, control_enforcement_refs=(), compensation_refs=(), **kw)	submit_effect_evidence, reconcile_effect; returns (evidence, state)
HostLayer1Authority(effects).commit(**kw)	record_layer1_transition; the verifier still decides
DeterministicScheduler().schedule(at, label, fn), run()	Runs callables in (at, insertion) order; trace holds SimulationEvent(at, sequence, label) records
SimulationObservationSource(service).inject(o, *, assessed_at, committed_at, permitted_use='governance')	As OperationalObservationAdapter.admit
SimulationEnforcementAdapter(service).dispatch(**kw), outcome(**kw)	As OperationalEnforcementAdapter
SimulationEffectSource(service).inject_and_reconcile(...)	As OperationalEffectAdapter
CrashFailoverInjector(recovery).failover(*, active_execution_id, new_owner_id, started_at, provenance='simulation-crash')	RecoveryService.begin_recovery
SimulationAuditOracle(store).table_count(table), current_owner(active_execution_id)	Read-only counts of nine record tables, and the current ownership

16.14 Classification Record

Table 16.33 gives the label of every package-level name of pvpp_runtime.supervision and of the five control records reachable through supervision.models. Methods are labeled in Appendix A.5, and the decisions, with the register entries H.5-10 to H.5-12, are recorded in H.7.

Table 16.33 — Labels assigned in Chapter 16

Symbol	Label	Rule	Section
HostTrustProvider	PUBLIC / SUPPORTED	R1 (integration contract)	16.2
ActiveExecutionRecord	IMPLEMENTATION / SUPPORT	R3 (H.5-10)	16.3
CanonicalExecutionHandle	IMPLEMENTATION / SUPPORT	R3 (H.5-10)	16.3
CanonicalRuntimeBridge	PUBLIC / SUPPORTED	R1	16.3
AdmittedFactChange	IMPLEMENTATION / SUPPORT	R3 (H.5-10)	16.4
ExternalObservation	PUBLIC / SUPPORTED	R6 (built by the host)	16.4
GovernedFactView	IMPLEMENTATION / SUPPORT	R3 (H.5-10)	16.4
ObservationAdmissionRecord	IMPLEMENTATION / SUPPORT	R3 (H.5-10)	16.4
ObservationService	PUBLIC / SUPPORTED	R1	16.4
SourceRegistration	PUBLIC / SUPPORTED	R6 (built by the host)	16.4
DependencyService	PUBLIC / SUPPORTED	R1 (not in __all__; H.5-12)	16.5
DependencySetDescriptor	IMPLEMENTATION / SUPPORT	R3 (not in __all__; H.5-10, H.5-12)	16.5
ExecutionDependencyBinding	IMPLEMENTATION / SUPPORT	R3 (not in __all__; H.5-10, H.5-12)	16.5
GovernanceSnapshotToken	IMPLEMENTATION / SUPPORT	R3 (not in __all__; H.5-10, H.5-12)	16.5
ContinuationAssessment	IMPLEMENTATION / SUPPORT	R3 (H.5-10)	16.6
ContinuationService	PUBLIC / SUPPORTED	R1	16.6
AdapterRegistration	IMPLEMENTATION / SUPPORT	R3 (supervision.models only; H.5-10)	16.7
ControlAttemptRecord	IMPLEMENTATION / SUPPORT	R3 (supervision.models only; H.5-10)	16.7
ControlService	PUBLIC / SUPPORTED	R1	16.7
EnforcementRecord	IMPLEMENTATION / SUPPORT	R3 (supervision.models only; H.5-10)	16.7
ExecutionControlRequest	IMPLEMENTATION / SUPPORT	R3 (supervision.models only; H.5-10)	16.7

Symbol	Label	Rule	Section
SupervisoryOwnershipRecord	IMPLEMENTATION / SUPPORT	R3 (supervision.models only; H.5-10)	16.7
CanonicalEpisodeFinality	IMPLEMENTATION / SUPPORT	R3 (not in __all__; H.5-10, H.5-12)	16.8
EffectEvidence	IMPLEMENTATION / SUPPORT	R3 (not in __all__; H.5-10, H.5-12)	16.8
EffectService	PUBLIC / SUPPORTED	R1 (not in __all__; H.5-12)	16.8
Layer1TransitionRecord	IMPLEMENTATION / SUPPORT	R3 (not in __all__; H.5-10, H.5-12)	16.8
WorldEffectReconciliationState	IMPLEMENTATION / SUPPORT	R3 (not in __all__; H.5-10, H.5-12)	16.8
RecoveryReconciliationRecord	IMPLEMENTATION / SUPPORT	R3 (H.5-10)	16.9
RecoveryRequirementProfile	IMPLEMENTATION / SUPPORT	R3 (H.5-10)	16.9
RecoveryService	PUBLIC / SUPPORTED	R1	16.9
AuditEvent	IMPLEMENTATION / SUPPORT	R3 (evidence, never authority)	16.10
ConfigurationState	IMPLEMENTATION / SUPPORT	R3	16.10
SQLiteSupervisoryStore	PUBLIC / ADVANCED	R2 (direct store access is privileged)	16.10
SemanticId	INTERNAL	R4	16.10
AdapterUnregistered	PUBLIC / SUPPORTED	R1 (applications handle it)	16.11
CanonicalResolutionError	PUBLIC / SUPPORTED	R1 (applications handle it)	16.11
ConcurrencyConflict	PUBLIC / SUPPORTED	R1 (applications handle it)	16.11
ConfigurationConflict	PUBLIC / SUPPORTED	R1 (applications handle it)	16.11
ContinuationReconciliationRequired	PUBLIC / SUPPORTED	R1 (applications handle it)	16.11
ControlMappingWidened	PUBLIC / SUPPORTED	R1 (applications handle it)	16.11
EvidenceUnverified	IMPLEMENTATION / SUPPORT	Default (never raised; H.5-11)	16.11
ObservationIdentityConflict	PUBLIC / SUPPORTED	R1 (applications handle it)	16.11
ObservationValidationError	PUBLIC / SUPPORTED	R1 (applications handle it)	16.11
StaleOwner	PUBLIC / SUPPORTED	R1 (applications handle it)	16.11

Symbol	Label	Rule	Section
TargetScopeViolation	PUBLIC / SUPPORTED	R1 (applications handle it)	16.11
CrashFailoverInjector	PUBLIC / REFERENCE	R7 (simulation harness)	16.13
DeterministicScheduler	PUBLIC / REFERENCE	R7 (simulation harness)	16.13
HostLayer1Authority	PUBLIC / REFERENCE	R7 (not in __all__; H.5-12)	16.13
OperationalEffectAdapter	PUBLIC / REFERENCE	R7 (not in __all__; H.5-12)	16.13
OperationalEnforcementAdapter	PUBLIC / REFERENCE	R7 (not in __all__; H.5-12)	16.13
OperationalObservationAdapter	PUBLIC / REFERENCE	R7 (not in __all__; H.5-12)	16.13
SimulationAuditOracle	PUBLIC / REFERENCE	R7 (simulation harness)	16.13
SimulationEffectSource	PUBLIC / REFERENCE	R7 (simulation harness)	16.13
SimulationEnforcementAdapter	PUBLIC / REFERENCE	R7 (simulation harness)	16.13
SimulationEvent	PUBLIC / REFERENCE	R7 (simulation harness)	16.13
SimulationObservationSource	PUBLIC / REFERENCE	R7 (simulation harness)	16.13

Appendix A — Quick API Index

This appendix lists every name the audits found on the callable surface: the 335 top-level exports of `pvpp_runtime`, the 133 public methods of `PVPPRuntime`, `PVPPRegistry`, and `ExecutionBindingRegistry`, and the 11 names reachable only through a submodule—479 in all. A.5 adds the supervision package of build 167: its 51 package-level names, the five control records reachable through its `models` module, and the 100 public methods of its classes. Each appears exactly once, with its kind, its label, and the section that holds its primary entry. The index is generated at each build from the audit evidence (`audit/evidence.json`), the classification record (`audit/routing.py`), and the chapters themselves.

Look a name up here first. The label tells you whether to build on it; the section tells you where its contract is written.

A.1 Reading the Index

Labels are abbreviated: SUP is PUBLIC / SUPPORTED, ADV is PUBLIC / ADVANCED, IMPL is IMPLEMENTATION / SUPPORT, INT is INTERNAL, and LEG is LEGACY / COMPATIBILITY (Appendix H). Primary is the section with the name's full entry, record table, or compact row; where a name has no table of its own, it is the section whose text or summary table documents it. A primary of 15.4 marks a legacy entry point and names its replacement there.

Table A.1 — Names by label

Label	Names
PUBLIC / SUPPORTED (SUP)	78
PUBLIC / ADVANCED (ADV)	218
IMPLEMENTATION / SUPPORT (IMPL)	150
INTERNAL (INT)	8
LEGACY / COMPATIBILITY (LEG)	25
Total	479

Exports are the names `pvpp_runtime` makes importable at the top level; export is not API authority (H.2). Nothing in this index is PUBLIC unless its label says so.

A.2 Exported Symbols

Table A.2 — Top-level exports of `pvpp_runtime`

Name	Kind	Label	Primary
AccessOperabilityState	dataclass	ADV	5.4
AccessOperabilityValidationAssessment	dataclass	IMPL	5.10
ActionDefinition	dataclass	SUP	4.4
ActionEvaluation	dataclass	LEG	15.4
ActionProjection	dataclass	SUP	9.6

Name	Kind	Label	Primary
ActionStructureAdapter	class	IMPL	10.9
ActiveRecoveryCorridor	dataclass	IMPL	11.13
ActualPersistentStateEnvelope	dataclass	SUP	5.1
AdapterIdentity	dataclass	ADV	4.9
AdequacyAssessment	dataclass	IMPL	9.14
AdequacyReusableDomainFramingArtifact	dataclass	ADV	12.5
AdequacyReusableProjectionArtifact	dataclass	ADV	12.5
admit_graph_transformation_and_plan_reentry	function	ADV	12.6
admit_novel_function_and_plan_reentry	function	ADV	12.6
admit_recovery_necessity_and_plan_reentry	function	ADV	12.6
analyze_sequence	function	ADV	13.2
apply_executable_dependency_scope	function	ADV	12.4
ArchitectureAdaptationAssessment	dataclass	IMPL	4.12
ArchitectureInvarianceProfile	dataclass	ADV	4.10
assess_capability_formation_prediction_error	function	ADV	8.11
assess_control_provenance_topology	function	ADV	6.8
assess_evidence_dependency_authority	function	ADV	6.7
assess_governance_reentry	function	ADV	12.3
assess_graph_structural_coverage_and_plan_reentry	function	ADV	12.6
assess_joint_recovery_reentry_readiness	function	ADV	12.8
assess_security_evidence_observation	function	ADV	6.9
bounded_viability_oracle	function	ADV	13.3
BoundedViabilityWitness	dataclass	IMPL	13.3
CandidatePolicySet	dataclass	IMPL	9.14
CandidatePolicySpace	dataclass	IMPL	9.14
CANONICAL_REENTRY_STAGE_ORDER	constant	ADV	12.1
CanonicalCycleDirective	dataclass	ADV	9.5
CanonicalCycleLedgerEntry	dataclass	IMPL	9.14
CanonicalCycleSequenceAssessment	dataclass	IMPL	9.14
CanonicalCycleSnapshot	dataclass	IMPL	9.14
CanonicalDecisionCycleAssessment	dataclass	IMPL	9.14
CanonicalDecisionCycleRequest	dataclass	SUP	9.2

Name	Kind	Label	Primary
CanonicalIntegratedCycleResult	dataclass	IMPL	9.14
CapabilityChangeObservation	dataclass	ADV	8.9
CapabilityChangeValidationAssessment	dataclass	IMPL	8.12
CapabilityDrivenGraphConstructibilityAssessment	dataclass	IMPL	8.12
CapabilityDrivenGraphRefreshAssessment	dataclass	IMPL	8.12
CapabilityFormationExpectation	dataclass	ADV	8.11
CapabilityFormationPredictionError	dataclass	IMPL	8.12
CapabilityFormationPredictionErrorAssessment	dataclass	IMPL	8.12
CapabilityGraphConstructibilityDependency	dataclass	ADV	8.10
CapabilityGraphReachabilityDependency	dataclass	ADV	8.10
CapabilityPredictionArtifactDependency	dataclass	ADV	8.11
CapabilityTrajectoryProjection	dataclass	IMPL	8.12
CapabilityTrajectoryProjectionAssessment	dataclass	IMPL	8.12
CapacityAuthorityEnvelope	dataclass	IMPL	5.10
CapacityAuthorityRequest	dataclass	IMPL	5.10
CapacityAuthorityService	class	ADV	10.7
CapacityAuthorityValidationAssessment	dataclass	IMPL	5.10
CapacityCalendar	dataclass	SUP	5.8
capture_projection_attribution_snapshot	function	ADV	13.6
classify_multidomain_stress	function	ADV	13.4
CleanExecutionFailure	exception	SUP	11.7
compare_contaminated_substrate_cycles	function	ADV	13.5
compare_projection_attribution_across_cycles	function	ADV	13.6
compare_projection_causal_state_across_cycles	function	ADV	13.6
CompletedInvalidExecution	exception	SUP	11.7
ConstraintAssessment	dataclass	IMPL	9.14
ConstraintEvidenceAdapter	class	IMPL	10.9
ConstraintObservation	dataclass	SUP	9.6
ConstraintRuleDefinition	dataclass	SUP	4.5
ConstraintsAssessment	dataclass	IMPL	9.14
ConstraintsReusablePiArtifact	dataclass	ADV	12.5
ConstraintViolationProfile	dataclass	IMPL	9.14

Name	Kind	Label	Primary
construct_graph_from_capability_trajectory	function	ADV	8.10
ContaminatedSubstratePairedAssessment	dataclass	IMPL	13.5
control_provenance_topology_identity	function	ADV	6.8
ControlProvenanceArtifactDependency	dataclass	ADV	12.2
ControlProvenanceGovernanceInvalidationAssessment	dataclass	IMPL	12.9
ControlProvenanceRelation	dataclass	ADV	6.8
ControlProvenanceTopologyAssessment	dataclass	IMPL	6.10
ControlProvenanceTopologyUpdateAssessment	dataclass	IMPL	6.10
ControlProvenanceTopologyUpdateRequest	dataclass	ADV	6.8
CorridorFeasibilityReport	dataclass	IMPL	5.10
CrossObjectExecutionIdentityAssessment	dataclass	IMPL	9.14
CycleArtifactIntegrityAssessment	dataclass	IMPL	9.14
DecisionReplayPackage	dataclass	IMPL	6.10
DependencyRequirementDefinition	dataclass	LEG	4.12
derive_control_provenance_governance_invalidations	function	ADV	12.2
derive_dependency_invalidations	function	ADV	12.2
derive_evidence_authority_governance_invalidations	function	ADV	12.2
derive_objective_lifecycle_invalidations	function	ADV	7.6
DiscretionarySelectionAssessment	dataclass	LEG	15.4
disposition_graph_structural_coverage	function	ADV	8.8
DomainAdequacyResult	dataclass	IMPL	9.14
DomainAssessment	dataclass	IMPL	9.14
DomainDefinition	dataclass	SUP	4.3
DomainFrame	dataclass	SUP	9.2
DomainFrameTarget	dataclass	SUP	9.2
DomainFramingAssessment	dataclass	IMPL	9.14
DomainRelationDefinition	dataclass	LEG	4.12
EpistemicInvalidationOrchestrationAssessment	dataclass	IMPL	12.9
EpistemicReentryExecutionAssessment	dataclass	IMPL	12.9
EpistemicSubstrateQualification	dataclass	ADV	9.8
EpistemicUpdateValidationAssessment	dataclass	IMPL	6.10

Name	Kind	Label	Primary
EpsilonStepResult	dataclass	IMPL	11.4
EvidenceAuthorityGovernanceInvalidationAssessment	dataclass	IMPL	12.9
EvidenceCanonicalArtifactDependency	dataclass	ADV	12.2
EvidenceDependencyAuthorityAssessment	dataclass	IMPL	6.10
EvidenceSourceAuthorityState	dataclass	ADV	6.7
execute_adequacy_reentry	function	ADV	12.5
execute_constraints_reentry	function	ADV	12.5
execute_domain_framing_reentry	function	ADV	12.5
execute_epistemic_reentry_plan	function	ADV	12.7
execute_governance_reentry	function	ADV	12.3
execute_governing_reentry	function	ADV	12.5
execute_graph_reachability_refresh_reentry	function	ADV	12.6
execute_graph_reentry	function	ADV	12.5
execute_graph_structural_admission_reentry	function	ADV	12.6
execute_horizon_reentry	function	ADV	12.5
execute_joint_recovery_reentry	function	ADV	12.5
execute_memory_conditioned_ppp_reentry	function	ADV	12.5
execute_phi_reentry	function	ADV	12.5
execute_pi_completeness_reentry	function	ADV	12.5
execute_pi_reentry	function	ADV	12.5
execute_ppp_reentry	function	ADV	12.5
execute_regime_reentry	function	ADV	12.5
execute_sigma_reentry	function	ADV	12.5
execution	module	IMPL	A.4
ExecutionBindingAssessment	dataclass	IMPL	11.13
ExecutionBindingIdentity	dataclass	SUP	11.5
ExecutionBindingRegistry	class	SUP	11.5
ExecutionBookkeepingAssessment	dataclass	IMPL	11.13
ExecutionContext	dataclass	ADV	11.8
ExecutionEpisode	dataclass	IMPL	11.3
ExecutionLicenseEnvelope	dataclass	IMPL	1.6
ExecutionObservation	dataclass	SUP	11.4

Name	Kind	Label	Primary
ExecutionResult	dataclass	LEG	15.4
ExecutionTraceNode	dataclass	IMPL	11.13
ExecutionTransitionAdapter	class	IMPL	10.9
ExecutionTransitionProvenance	dataclass	IMPL	11.13
ExecutionTransitionProvenanceAssessment	dataclass	IMPL	11.13
ExpectationStateReference	dataclass	ADV	6.2
FallbackConfiguration	dataclass	SUP	4.6
FallbackStructuralProfile	dataclass	ADV	9.6
feasibility	module	INT	A.4
GovernanceArtifactDependency	dataclass	ADV	12.2
GovernanceDependencyChange	dataclass	ADV	12.2
GovernanceDependencyInvalidationAssessment	dataclass	IMPL	12.9
GovernanceEvidenceDependency	dataclass	ADV	6.7
GovernanceInvalidationSignal	dataclass	ADV	12.2
GovernanceReentryAssessment	dataclass	IMPL	12.9
GovernanceReentryExecutionResult	dataclass	IMPL	12.3
GovernanceReentryPlan	dataclass	IMPL	12.3
GovernanceReusableArtifactBundle	dataclass	ADV	12.4
GovernanceReusableArtifactBundleAssessment	dataclass	IMPL	12.9
GoverningAssessment	dataclass	IMPL	9.14
GoverningConfiguration	dataclass	SUP	4.6
GoverningReusableIdentificationInputs	dataclass	ADV	12.5
GraphCompositionDefinition	dataclass	SUP	4.7
GraphConstructionAssessment	dataclass	IMPL	8.12
GraphConstructionConfig	dataclass	SUP	8.2
GraphInstanceDefinition	dataclass	SUP	4.7
GraphReachabilityAdapter	class	IMPL	10.9
GraphReachabilityGovernanceTrigger	dataclass	IMPL	12.9
GraphReachabilityObservation	dataclass	ADV	8.4
GraphReachabilityRefreshAssessment	dataclass	IMPL	8.12
GraphReachabilityRefreshRequest	dataclass	ADV	8.4
GraphReusableConstructionInputs	dataclass	ADV	12.5

Name	Kind	Label	Primary
GraphStructuralAdmissionGovernanceTrigger	dataclass	IMPL	12.9
GraphStructuralCoverageAdmissionHandoff	dataclass	IMPL	8.12
GraphStructuralCoverageAssessment	dataclass	IMPL	8.12
GraphStructuralCoverageDisposition	dataclass	IMPL	8.12
GraphStructuralCoverageGovernanceTrigger	dataclass	IMPL	12.9
GraphStructuralCoverageObservation	dataclass	ADV	8.8
GraphStructuralRevisionGovernanceTrigger	dataclass	IMPL	12.9
GraphTransformationAdmissionAssessment	dataclass	IMPL	8.12
GraphTransformationAdmissionRequest	dataclass	ADV	8.5
GraphTransformationDefinition	dataclass	SUP	4.7
GraphTransformationRevisionAssessment	dataclass	IMPL	8.12
GraphTransformationRevisionRequest	dataclass	ADV	8.5
HorizonAssessment	dataclass	IMPL	9.14
HorizonConfiguration	dataclass	SUP	4.6
HorizonDomainResult	dataclass	IMPL	9.14
HorizonReusableEvaluationInputs	dataclass	ADV	12.5
IndeterminateExecutionFailure	exception	SUP	11.7
InformationGovernanceAssessment	dataclass	IMPL	6.10
InformationGovernanceMetadata	dataclass	ADV	6.1
InformationTemporalProvenance	dataclass	ADV	6.1
IntegratedCycleLifecycleIntegrityAssessment	dataclass	IMPL	9.14
interfaces	module	SUP	A.4
JointRecoveryExecutionBindingAssessment	dataclass	IMPL	9.14
JointRecoveryReentryReadinessAssessment	dataclass	IMPL	12.9
kernel	module	IMPL	A.4
Layer1ExecutionCommit	dataclass	IMPL	11.13
Layer1TransitionHandoff	dataclass	IMPL	11.10
Layer1TransitionResult	dataclass	SUP	11.10
Layer1TransitionService	class	SUP	10.4
Layer1TransitionValidationAssessment	dataclass	IMPL	11.13
MemoryConditionedCycleDirective	dataclass	ADV	9.5
MemoryConditionedCycleLedgerEntry	dataclass	IMPL	9.14

Name	Kind	Label	Primary
MemoryConditionedCycleSequenceAssessment	dataclass	IMPL	9.14
MemoryConditionedCycleSnapshot	dataclass	IMPL	9.14
MemoryConditionedIntegratedCycleResult	dataclass	IMPL	9.14
MemoryConditionedPPPreentryInputs	dataclass	ADV	12.5
MemoryRetrievalPackage	dataclass	ADV	6.2
MemoryRetrievalRequest	dataclass	IMPL	6.10
MemoryStateReference	dataclass	ADV	6.2
models	module	IMPL	A.4
MultidomainStressClassification	dataclass	IMPL	13.4
NativeExecutionAuthorization	dataclass	IMPL	11.13
NativeExecutionControlEvent	dataclass	IMPL	11.13
NativeExecutionControlState	dataclass	IMPL	11.13
NativeExecutionResult	dataclass	IMPL	11.13
NativeExecutionTelemetry	dataclass	IMPL	11.13
NativeReauthorizationHandoff	dataclass	IMPL	11.13
NovelFunctionAdmissionAssessment	dataclass	IMPL	8.12
NovelFunctionAdmissionGovernanceTrigger	dataclass	IMPL	12.9
NovelFunctionAdmissionRequest	dataclass	ADV	8.6
ObjectiveDiscoveryAdapter	class	IMPL	10.9
ObjectiveDiscoveryAssessment	dataclass	IMPL	7.7
ObjectiveDiscoveryHint	dataclass	SUP	7.5
ObjectiveInterfaceAdapter	class	IMPL	10.9
ObjectiveLifecycleChange	dataclass	ADV	7.6
ObjectiveLifecycleInvalidationAssessment	dataclass	IMPL	7.7
ObjectiveRelation	dataclass	SUP	7.2
ObjectiveResponsivenessAdapter	class	ADV	10.8
ObjectiveResponsivenessAssessment	dataclass	IMPL	7.7
ObjectiveResponsivenessRecord	dataclass	ADV	7.4
ObjectiveSet	dataclass	SUP	7.2
ObjectiveSetValidationAssessment	dataclass	IMPL	7.7
ObjectiveTemporalBounds	dataclass	SUP	7.2
operators	module	INT	A.4

Name	Kind	Label	Primary
orchestrate_epistemic_invalidiation_reentry	function	ADV	12.7
PerceivedDecisionState	dataclass	SUP	5.3
PerceivedDecisionStateValidationAssessment	dataclass	IMPL	5.10
PhiReusableEvaluationInputs	dataclass	ADV	12.5
PiCompletenessAssessment	dataclass	IMPL	9.14
PiConstructionAssessment	dataclass	IMPL	9.14
PiConstructionConfig	dataclass	SUP	9.2
PiReusableConstructionInputs	dataclass	ADV	12.5
plan_governance_reentry	function	ADV	12.3
PolicyAdequacyResult	dataclass	IMPL	9.14
PolicyConstraintProfile	dataclass	SUP	9.6
PolicyProjectionRecord	dataclass	SUP	9.6
PolicyProjectionService	class	SUP	10.3
PolicySeedDefinition	dataclass	ADV	4.6
PolicySelectionAssessment	dataclass	IMPL	9.14
PolicySetEvaluation	dataclass	ADV	9.11
PostExecuteEpistemicUpdateRequest	dataclass	IMPL	6.10
PostExecuteEpistemicUpdateResult	dataclass	ADV	6.6
PPPreentryInputs	dataclass	ADV	12.5
PredictionErrorAssessment	dataclass	ADV	6.5
PredictionErrorComponent	dataclass	ADV	6.5
PredictionErrorRequest	dataclass	IMPL	6.10
PredictionErrorService	class	ADV	10.6
PredictionErrorValidationAssessment	dataclass	IMPL	6.10
PreliminaryPreservationObject	dataclass	SUP	8.2
PressureDomainResult	dataclass	IMPL	9.14
PressureFactors	dataclass	SUP	9.6
PressureFieldAssessment	dataclass	IMPL	9.14
PressureInvariantAssessment	dataclass	IMPL	9.14
PreviousRegimeState	dataclass	SUP	9.2
ProductiveConfigurationState	dataclass	ADV	5.5
ProductiveConfigurationValidationAssessment	dataclass	IMPL	5.10

Name	Kind	Label	Primary
ProductivePowerDefinition	dataclass	SUP	4.4
ProductivePowerState	dataclass	SUP	5.2
project_capability_trajectory	function	ADV	8.9
ProjectionAttributionSnapshot	dataclass	IMPL	13.6
ProjectionCausalStateConsistencyDeclaration	dataclass	ADV	13.6
ProjectionCausalStateCrossCycleAssessment	dataclass	IMPL	13.6
ProjectionCausalStateInput	dataclass	ADV	9.10
ProjectionConsistencyAudit	dataclass	IMPL	9.14
ProjectionCrossCycleAttributionAssessment	dataclass	IMPL	13.6
ProjectionInformationQualityClaim	dataclass	ADV	9.10
ProjectionInputLicenseAssessment	dataclass	IMPL	9.14
ProjectionModelAuthority	dataclass	ADV	9.10
ProjectionModelAuthorityAssessment	dataclass	IMPL	9.14
ProjectionRequest	dataclass	IMPL	9.14
ProjectionWorldModelAdapter	class	IMPL	10.9
PVPPAssessment	dataclass	LEG	15.4
PVPPRegistry	class	SUP	4.2
PVPPRuntime	class	SUP	4.1
QualifiedObjectiveRecord	dataclass	SUP	7.2
QualifiedPiCompletenessAssessment	dataclass	IMPL	9.14
QualifiedUnitState	dataclass	ADV	5.5
QualifiedUnitValidationAssessment	dataclass	IMPL	5.10
qualify_evidence_source_from_control_topology	function	ADV	6.7
RecoveryActionConfirmation	dataclass	ADV	11.11
RecoveryAdequacyAssessment	dataclass	LEG	15.4
RecoveryCorridorProjection	dataclass	SUP	9.6
RecoveryNecessityAdmissionAssessment	dataclass	IMPL	8.12
RecoveryNecessityAdmissionGovernanceTrigger	dataclass	IMPL	12.9
RecoveryNecessityAdmissionRequest	dataclass	ADV	8.7
RecoveryNecessityDefinition	dataclass	SUP	4.5
RecoveryNecessityRevisionAssessment	dataclass	IMPL	8.12
RecoveryNecessityRevisionGovernanceTrigger	dataclass	IMPL	12.9

Name	Kind	Label	Primary
RecoveryNecessityRevisionRequest	dataclass	ADV	8.7
RecoveryPlanDefinition	dataclass	SUP	4.5
RecoveryResourceState	dataclass	ADV	5.6
RecoveryResourceValidationAssessment	dataclass	IMPL	5.10
reentry	module	IMPL	A.4
refresh_graph_from_capability_trajectory	function	ADV	8.10
refresh_graph_reachability_and_plan_reentry	function	ADV	12.6
RegimeAssessment	dataclass	IMPL	9.14
RegimeConfiguration	dataclass	SUP	4.6
RegimeReusableClassificationInputs	dataclass	ADV	12.5
registry	module	IMPL	A.4
ReplaySufficiencyAssessment	dataclass	IMPL	6.10
RetrievalQualityMetadata	dataclass	ADV	6.2
revise_graph_transformation_and_plan_reentry	function	ADV	12.6
revise_recovery_necessity_and_plan_reentry	function	ADV	12.6
RuntimeConfigurationProvenance	dataclass	ADV	4.9
RuntimeConfigurationProvenanceAssessment	dataclass	IMPL	4.12
ScheduledRecoveryStep	dataclass	IMPL	5.10
SecurityEvidenceAssessment	dataclass	IMPL	6.10
SecurityEvidenceObservation	dataclass	ADV	6.9
SelectedPolicy	dataclass	LEG	15.4
SequenceDiagnostics	dataclass	IMPL	13.2
SigmaAssessment	dataclass	IMPL	9.14
SigmaFallbackAssessment	dataclass	IMPL	9.14
SigmaOrderDefinition	dataclass	SUP	4.6
SigmaReusableAdequacyArtifact	dataclass	ADV	12.5
SigmaStageAssessment	dataclass	IMPL	9.14
StateMeasurementAdapter	class	IMPL	10.9
StateSufficiencyAdapter	class	IMPL	10.9
StructuralPolicyAssessment	dataclass	LEG	15.4
testing	module	ADV	A.4
TransferHistoryEvent	dataclass	ADV	6.3

Name	Kind	Label	Primary
TransferHistoryPackage	dataclass	ADV	6.3
TransferHistoryReference	dataclass	ADV	6.3
TransferHistoryService	class	ADV	10.6
TransferHistoryValidationAssessment	dataclass	IMPL	6.10
TransitionRelevantStateBasis	dataclass	SUP	5.9
TransitionRelevantStatePairAssessment	dataclass	IMPL	5.10
TransitionRelevantStateSufficiencyAssessment	dataclass	IMPL	5.10
update_control_provenance_topology	function	ADV	6.8
validate_capability_change_observation	function	ADV	8.9
validate_reusable_artifact_bundle	function	ADV	12.4
WorldState	dataclass	SUP	5.2

A.3 Core Class Methods

Methods are listed by class, without the class prefix.

Table A.3 — PVPPRuntime methods (89)

Name	Kind	Label	Primary
advance_execution	method	SUP	11.4
apply_layer1_transition	method	SUP	11.10
apply_post_execution_epistemic_update	method	ADV	6.6
apply_prediction_error	method	ADV	6.5
assess	method	LEG	15.4
assess_architecture_adaptation_invariance	method	ADV	4.10
assess_recovery_feasibility	method	ADV	9.9
assess_replay_sufficiency	method	ADV	6.4
assess_transition_relevant_state_pair	method	ADV	5.9
audit_cycle_artifact_integrity	method	ADV	9.12
audit_integrated_cycle_lifecycle_integrity	method	ADV	9.12
audit_post_selection_identity_chain	method	ADV	9.12
build_capacity_authority_request	method	ADV	5.7
build_decision_replay_package	method	ADV	6.4
build_execution_license	method	LEG	15.4
build_execution_license_from_cycle	method	SUP	11.2
build_execution_transition_provenance	method	SUP	11.11

Name	Kind	Label	Primary
build_layer1_execution_commit_from_transition	method	ADV	11.11
build_layer1_transition_handoff	method	SUP	11.10
build_post_execution_epistemic_update_request	method	ADV	6.6
build_prediction_error_request	method	ADV	6.5
classify_regime	method	ADV	9.7
construct_graph	method	ADV	8.2
construct_pi	method	ADV	9.8
construct_pi_from_graph	method	ADV	9.8
decide	method	LEG	15.4
derive_capacity_authority	method	IMPL	5.10
evaluate_canonical_cycle_sequence	method	ADV	9.5
evaluate_canonical_decision_cycle	method	SUP	9.3
evaluate_collapse_horizons	method	ADV	9.7
evaluate_constraint_system	method	ADV	9.9
evaluate_constraints	method	LEG	9.13
evaluate_framed_policy_space	method	LEG	15.4
evaluate_integrated_canonical_cycle	method	SUP	9.3
evaluate_memory_conditioned_cycle_sequence	method	ADV	9.5
evaluate_memory_conditioned_decision_cycle	method	ADV	9.5
evaluate_memory_conditioned_integrated_cycle	method	ADV	9.5
evaluate_objective_responsiveness	method	ADV	7.4
evaluate_policy_sets	method	LEG	15.4
evaluate_policy_space	method	LEG	9.13
evaluate_pressure_field	method	ADV	9.7
evaluate_proposal	method	LEG	15.4
evaluate_recovery_policy_space	method	LEG	9.13
evaluate_restoration_adequacy	method	ADV	9.11
evaluate_sigma_fallback	method	ADV	9.11
evaluate_sigma_standard	method	ADV	9.11
evaluate_sigma_terminal	method	ADV	9.11
identify_governing_domains	method	ADV	9.7
instantiate_execution	method	SUP	11.3

Name	Kind	Label	Primary
invalidate_native_execution_authorizations	method	ADV	11.6
invoke_authorized_native	method	SUP	11.7
issue_native_execution_authorization	method	SUP	11.6
native_execution_authorization_execution_id	method	ADV	11.6
native_execution_authorization_status	method	ADV	11.6
native_execution_authorizations	method	ADV	11.6
prepare_canonical_cycle_snapshot	method	SUP	9.3
prepare_integrated_request_capacity	method	IMPL	5.10
prepare_memory_conditioned_cycle_snapshot	method	ADV	9.5
qualify_pi_completeness_for_substrate	method	ADV	9.8
record_execution	method	LEG	15.4
record_layer1_execution_commit	method	LEG	11.11
record_recovery_bookkeeping_from_transition	method	ADV	11.11
record_transfer_history	method	ADV	6.3
validate_access_operability_state	method	ADV	5.4
validate_actual_persistent_state	method	ADV	5.1
validate_capacity_authority_envelope	method	IMPL	5.10
validate_domain_framing	method	LEG	9.13
validate_execution_transition_provenance	method	ADV	11.11
validate_information_governance_metadata	method	ADV	6.1
validate_layer1_transition_result	method	ADV	11.10
validate_memory_retrieval_package	method	ADV	6.2
validate_memory_state_reference	method	ADV	6.2
validate_objective_discovery_hints	method	ADV	7.5
validate_objective_set	method	ADV	7.2
validate_perceived_decision_state	method	ADV	5.3
validate_pi_completeness	method	ADV	9.8
validate_post_execution_epistemic_update_result	method	ADV	6.6
validate_prediction_error_assessment	method	ADV	6.5
validate_pressure_invariants	method	ADV	9.7
validate_productive_configuration_state	method	ADV	5.5
validate_projection_causal_inputs	method	ADV	9.10

Name	Kind	Label	Primary
validate_projection_model_authority	method	ADV	9.10
validate_projection_record	method	ADV	9.10
validate_qualified_unit_state	method	ADV	5.5
validate_recovery_resource_state	method	ADV	5.6
validate_runtime_configuration_provenance	method	ADV	4.9
validate_transfer_history_event	method	ADV	6.3
validate_transfer_history_package	method	ADV	6.3
validate_transition_relevant_state_basis	method	SUP	5.9

Table A.4 — PVPPRegistry methods (30)

Name	Kind	Label	Primary
action_registry_identity	method	ADV	4.8
admit_graph_transformation	method	ADV	8.5
admit_novel_function	method	ADV	8.6
admit_recovery_necessity	method	ADV	8.7
assess_graph_structural_coverage	method	ADV	8.8
governing_substrate_identity	method	ADV	4.8
graph_substrate_identity	method	ADV	4.8
horizon_configuration_identity	method	ADV	4.8
phi_substrate_identity	method	ADV	4.8
refresh_graph_reachability	method	ADV	8.4
regime_configuration_identity	method	ADV	4.8
register_action	method	SUP	4.4
register_constraint_rule	method	SUP	4.5
register_dependency_requirement	method	LEG	4.12
register_domain	method	SUP	4.3
register_domain_relation	method	LEG	4.12
register_fallback_configuration	method	SUP	4.6
register_governing_configuration	method	SUP	4.6
register_graph_composition	method	SUP	4.7
register_graph_instance	method	SUP	4.7
register_graph_transformation	method	SUP	4.7
register_horizon_configuration	method	SUP	4.6

Name	Kind	Label	Primary
register_policy_seed	method	ADV	4.6
register_power	method	SUP	4.4
register_recovery_necessity	method	SUP	4.5
register_recovery_plan	method	SUP	4.5
register_regime_configuration	method	SUP	4.6
register_sigma_order	method	SUP	4.6
revise_graph_transformation	method	ADV	8.5
revise_recovery_necessity	method	ADV	8.7

Table A.5 — ExecutionBindingRegistry methods (14)

Name	Kind	Label	Primary
assess_identity	method	ADV	11.5
build_layer1_handoff	method	LEG	11.8
build_reauthorization_handoff	method	ADV	11.9
control_history	method	ADV	11.9
control_state	method	ADV	11.9
create_context	method	ADV	11.5
epsilon_observation	method	ADV	11.8
identity	method	ADV	11.5
identity_for_action	method	ADV	11.5
invoke	method	LEG	11.5
register	method	SUP	11.5
registered_identities	method	ADV	11.5
set_control_state	method	ADV	11.9
telemetry	method	ADV	11.8

A.4 Submodule Namespaces and Submodule-Only Surface

Nine of the exports are the package's submodules. Every public class and function they define is also exported at the top level, except the names in Table A.6; import from the top level. The submodule itself carries a label only for what it is used for: interfaces is the import route for WorldAdapter, MemoryRetrievalService, and EpistemicUpdateService (H.5-1); testing holds the diagnostics of Chapter 13; models is the route to two submodule-only records; feasibility and operators hold internal code. kernel, registry, execution, and reentry take the default label (H.5-9): they are implementation modules whose contents are reached through top-level names.

Table A.6 — Names reachable only through a submodule

Name	Kind	Label	Primary
feasibility.assess_joint_feasibility	function	INT	5.10
interfaces.EpistemicUpdateService	class	ADV	10.5
interfaces.MemoryRetrievalService	class	ADV	10.5
interfaces.WorldAdapter	class	SUP	10.2
models.JointRecoveryFeasibilityAssessment	dataclass	IMPL	5.10
models.SigmaTerminalAssessment	dataclass	IMPL	9.14
operators.prototype.collapse_horizon	function	INT	9.13
operators.prototype.dependency_close	function	INT	9.13
operators.prototype.governing_from_horizon	function	INT	9.13
operators.prototype.pressure_from_margin	function	INT	9.13
operators.prototype.regime_label	function	INT	9.13

A.5 Supervision Surface

Labels are abbreviated as in A.1, with REF for PUBLIC / REFERENCE. Names are importable from `pvpp_runtime.supervision`; a name marked models is reachable only as `pvpp_runtime.supervision.models.<name>`. Methods are listed as `Class.method`.

Table A.7 — Supervision names by label

Label	Names and methods
PUBLIC / SUPPORTED (SUP)	67
PUBLIC / ADVANCED (ADV)	21
PUBLIC / REFERENCE (REF)	25
IMPLEMENTATION / SUPPORT (IMPL)	34
INTERNAL (INT)	5
LEGACY / COMPATIBILITY (LEG)	4
Total	156

Table A.8 — Package-level names of `pvpp_runtime.supervision`

Name	Kind	Label	Primary
ActiveExecutionRecord	dataclass	IMPL	16.3
AdapterRegistration	dataclass, models	IMPL	16.7
AdapterUnregistered	exception	SUP	16.11
AdmittedFactChange	dataclass	IMPL	16.4
AuditEvent	dataclass	IMPL	16.10

Name	Kind	Label	Primary
CanonicalEpisodeFinality	dataclass	IMPL	16.8
CanonicalExecutionHandle	dataclass	IMPL	16.3
CanonicalResolutionError	exception	SUP	16.11
CanonicalRuntimeBridge	class	SUP	16.3
ConcurrencyConflict	exception	SUP	16.11
ConfigurationConflict	exception	SUP	16.11
ConfigurationState	dataclass	IMPL	16.10
ContinuationAssessment	dataclass	IMPL	16.6
ContinuationReconciliationRequired	exception	SUP	16.11
ContinuationService	class	SUP	16.6
ControlAttemptRecord	dataclass, models	IMPL	16.7
ControlMappingWidened	exception	SUP	16.11
ControlService	class	SUP	16.7
CrashFailoverInjector	class	REF	16.13
DependencyService	class	SUP	16.5
DependencySetDescriptor	dataclass	IMPL	16.5
DeterministicScheduler	class	REF	16.13
EffectEvidence	dataclass	IMPL	16.8
EffectService	class	SUP	16.8
EnforcementRecord	dataclass, models	IMPL	16.7
EvidenceUnverified	exception	IMPL	16.11
ExecutionControlRequest	dataclass, models	IMPL	16.7
ExecutionDependencyBinding	dataclass	IMPL	16.5
ExternalObservation	dataclass	SUP	16.4
GovernanceSnapshotToken	dataclass	IMPL	16.5
GovernedFactView	dataclass	IMPL	16.4
HostLayer1Authority	class	REF	16.13
HostTrustProvider	class	SUP	16.2
Layer1TransitionRecord	dataclass	IMPL	16.8
ObservationAdmissionRecord	dataclass	IMPL	16.4
ObservationIdentityConflict	exception	SUP	16.11
ObservationService	class	SUP	16.4

Name	Kind	Label	Primary
ObservationValidationError	exception	SUP	16.11
OperationalEffectAdapter	class	REF	16.13
OperationalEnforcementAdapter	class	REF	16.13
OperationalObservationAdapter	class	REF	16.13
RecoveryReconciliationRecord	dataclass	IMPL	16.9
RecoveryRequirementProfile	dataclass	IMPL	16.9
RecoveryService	class	SUP	16.9
SQLiteSupervisoryStore	class	ADV	16.10
SemanticId	dataclass	INT	16.10
SimulationAuditOracle	class	REF	16.13
SimulationEffectSource	class	REF	16.13
SimulationEnforcementAdapter	class	REF	16.13
SimulationEvent	dataclass	REF	16.13
SimulationObservationSource	class	REF	16.13
SourceRegistration	dataclass	SUP	16.4
StaleOwner	exception	SUP	16.11
SupervisoryOwnershipRecord	dataclass, models	IMPL	16.7
TargetScopeViolation	exception	SUP	16.11
WorldEffectReconciliationState	dataclass	IMPL	16.8

Table A.9 — Public methods of the supervision classes

Name	Label	Primary
CanonicalRuntimeBridge.advance_call_was_entered	ADV	16.3
CanonicalRuntimeBridge.advance_execution	LEG	16.3
CanonicalRuntimeBridge.authorization_status	ADV	16.3
CanonicalRuntimeBridge.authorize_supervised_advance	INT	16.3
CanonicalRuntimeBridge.current_episode	ADV	16.3
CanonicalRuntimeBridge.current_handle	ADV	16.3
CanonicalRuntimeBridge.episode_is_indeterminate	ADV	16.3
CanonicalRuntimeBridge.episode_state	ADV	16.3
CanonicalRuntimeBridge.episode_step	ADV	16.3
CanonicalRuntimeBridge.instantiate_execution	SUP	16.3
CanonicalRuntimeBridge.invoke_authorized_native	LEG	16.3

Name	Label	Primary
CanonicalRuntimeBridge.invoke_registered_native	SUP	16.3
CanonicalRuntimeBridge.issue_native_execution_authorization	SUP	16.3
CanonicalRuntimeBridge.refresh_canonical_handle	ADV	16.3
CanonicalRuntimeBridge.register_active_execution	SUP	16.3
CanonicalRuntimeBridge.resolve_canonical_execution	SUP	16.3
CanonicalRuntimeBridge.validate_advance_preconditions	INT	16.3
ContinuationService.checkpoint	SUP	16.6
ContinuationService.evaluate_continuation	SUP	16.6
ContinuationService.get_assessment	SUP	16.6
ContinuationService.reconcile_continuation_divergence	SUP	16.6
ControlService.acquire_ownership	SUP	16.7
ControlService.dispatch_control_request	SUP	16.7
ControlService.fence_execution_authority	SUP	16.7
ControlService.issue_control_request	SUP	16.7
ControlService.record_enforcement_outcome	SUP	16.7
ControlService.register_adapter	SUP	16.7
ControlService.register_owner_authority	ADV	16.7
ControlService.register_supervisory_owner	SUP	16.7
CrashFailoverInjector.failover	REF	16.13
DependencyService.capture_governance_snapshot	SUP	16.5
DependencyService.derive_dependency_set	SUP	16.5
DependencyService.get_snapshot	SUP	16.5
DependencyService.register_dependency_binding	SUP	16.5
DependencyService.resolve_affected_executions	SUP	16.5
DependencyService.retire_dependency_binding	SUP	16.5
DependencyService.validate_snapshot	SUP	16.5
DeterministicScheduler.run	REF	16.13
DeterministicScheduler.schedule	REF	16.13
EffectService.accept_canonical_finality	SUP	16.8
EffectService.assert_retry_safe	SUP	16.8
EffectService.current_effect_state	SUP	16.8
EffectService.get_canonical_finality	SUP	16.8

Name	Label	Primary
EffectService.reconcile_effect	SUP	16.8
EffectService.record_layer1_transition	SUP	16.8
EffectService.register_retry_contract	SUP	16.8
EffectService.submit_effect_evidence	SUP	16.8
HostLayer1Authority.commit	REF	16.13
HostTrustProvider.authorize_registration	ADV	16.2
HostTrustProvider.now	ADV	16.2
HostTrustProvider.verify_controller_attestation	ADV	16.2
HostTrustProvider.verify_layer1_authority	ADV	16.2
ObservationService.assess_observation	SUP	16.4
ObservationService.commit_admission_change	SUP	16.4
ObservationService.current_registration	SUP	16.4
ObservationService.observation_count	SUP	16.4
ObservationService.read_current_fact_view	SUP	16.4
ObservationService.register_source	SUP	16.4
ObservationService.submit_observation	SUP	16.4
OperationalEffectAdapter.admit_and_reconcile	REF	16.13
OperationalEnforcementAdapter.dispatch_exact	REF	16.13
OperationalEnforcementAdapter.report_outcome	REF	16.13
OperationalObservationAdapter.admit	REF	16.13
RecoveryService.assert_resumption_allowed	SUP	16.9
RecoveryService.begin_recovery	SUP	16.9
RecoveryService.capture_recovery_snapshot	SUP	16.9
RecoveryService.complete_recovery	SUP	16.9
RecoveryService.current_profile	SUP	16.9
RecoveryService.get_recovery	SUP	16.9
RecoveryService.reconcile_recovery	SUP	16.9
RecoveryService.register_recovery_profile	SUP	16.9
SQLiteSupervisoryStore.acquire_supervisory_ownership	IMPL	16.10
SQLiteSupervisoryStore.append_audit_event	IMPL	16.10
SQLiteSupervisoryStore.audit_events	SUP	16.10
SQLiteSupervisoryStore.bind_canonical_bridge	LEG	16.10

Name	Label	Primary
SQLiteSupervisoryStore.canonical_authorization_status	IMPL	16.10
SQLiteSupervisoryStore.close	ADV	16.10
SQLiteSupervisoryStore.compare_and_swap_configuration	ADV	16.10
SQLiteSupervisoryStore.continuation_intent_is_prepared	IMPL	16.10
SQLiteSupervisoryStore.continuation_reconciliation_required	IMPL	16.10
SQLiteSupervisoryStore.create_configuration	ADV	16.10
SQLiteSupervisoryStore.current_supervisory_ownership	ADV	16.10
SQLiteSupervisoryStore.get_active_execution	ADV	16.10
SQLiteSupervisoryStore.get_configuration	ADV	16.10
SQLiteSupervisoryStore.has_supervised_native_invocation	IMPL	16.10
SQLiteSupervisoryStore.now	ADV	16.10
SQLiteSupervisoryStore.phase6_get	INT	16.10
SQLiteSupervisoryStore.phase6_put	INT	16.10
SQLiteSupervisoryStore.record_supervised_native_invocation	IMPL	16.10
SQLiteSupervisoryStore.record_supervisory_operation	IMPL	16.10
SQLiteSupervisoryStore.register_active_execution	LEG	16.10
SQLiteSupervisoryStore.register_supervisory_owner_authority	IMPL	16.10
SQLiteSupervisoryStore.require_registration_authority	IMPL	16.10
SQLiteSupervisoryStore.supervisory_owner_authorized	IMPL	16.10
SimulationAuditOracle.current_owner	REF	16.13
SimulationAuditOracle.table_count	REF	16.13
SimulationEffectSource.inject_and_reconcile	REF	16.13
SimulationEnforcementAdapter.dispatch	REF	16.13
SimulationEnforcementAdapter.outcome	REF	16.13
SimulationObservationSource.inject	REF	16.13

Appendix B — Code Skeletons

Seven skeletons cover the common host tasks: registry setup, one decision cycle, the execution lineage, a Layer-1 transition service, re-entry after a change, a test harness, and supervised execution. Together they form one small application: a battery-powered sensor whose actions are steady (the baseline continuation), sample, and recharge. Each listing is a complete file, reproduced verbatim from the files that were run; nothing is elided.

Standing. The skeletons are host code written for this Reference. They are not part of the frozen runtime, are not covered by its hash manifest, and are not frozen evidence (1.2, 15.8). What they establish is narrower: that the calls shown, in the order shown, run against the unmodified, hash-verified v2.1.167 tree and produce the results banked in B.6. They live outside the runtime tree; running them wrote nothing into it. Exact copies are in the repository's `programmers-reference/skeletons` folder.

What they follow. Each skeleton uses only canonical entries and follows the Reference's recommendations rather than the shortest path the runtime accepts:

- the typed perception hook `perceived_decision_state_from_actual`, with `perceive` returning an already-represented state unchanged (10.2);
- a `PolicyProjectionService` that returns one projection record per request and never selects (10.3);
- an explicit Σ order for Stage-3 ties (3.11, 4.7);
- one execution per selected decision, unique episode ids, and the first terminal result treated as final (the object-lineage rule, 11.1, 15.3);
- the `ExecutionContext` rebuilt from the native result and its authorization before ϵ observation (11.8);
- a re-entry bundle built from the same cycle whose artifacts it reuses, with the `executable-scope` call always made (12.4).

No skeleton uses a `LEGACY / COMPATIBILITY` entry (15.4). New programmers should read these alongside the two canonical shipped examples, `minimal_battery_service` and `authority_bound_callthrough_v0130`, and not the `tiny_*` examples (15.4). The skeletons import each other in order: B.2 imports B.1 and B.4; B.3, B.5, and B.7 import B.2; the tests in B.6 and B.7 import the rest.

B.1 Registry Setup

The registry holds everything the runtime consults but does not compute: domains with thresholds, productive powers, actions, recovery plans, stage configurations, constraint rules, graph instances and transformations, and the Σ order (4.2–4.7). Three points are easy to miss. The constructor of `PVPPRuntime` requires an action with id `steady`, the baseline continuation used by pressure and horizon projection (4.1). A transformation marked `structurally_required` yields a required seed whether or not the request names its graph family (9.8). And a residual Σ tie among candidates with no registered order index is not broken silently: it raises `ValueError` (3.11). In this application only one candidate survives, so the order is registered but not consulted; real candidate sets will tie.

Listing B.1 — `b1_registry.py` (79 lines)

```
"""B.1 Registry setup: domains, actions, constraints, configurations, graph, and Sigma order."""
```

```

from pvpp_runtime import (
    PVPPRegistry,
    DomainDefinition,
    ProductivePowerDefinition,
    ActionDefinition,
    RecoveryPlanDefinition,
    GoverningConfiguration,
    RegimeConfiguration,
    ConstraintRuleDefinition,
    GraphInstanceDefinition,
    GraphTransformationDefinition,
    SigmaOrderDefinition,
)

DOMAIN, POWER, FUNCTION = "energy", "battery_capacity", "measurement_service"

def build_registry() -> PVPPRegistry:
    r = PVPPRegistry()
    r.register_domain(DomainDefinition(DOMAIN, "usable battery energy", threshold=2.0))
    r.register_power(ProductivePowerDefinition(POWER, DOMAIN, "capacity to power sensing"))
    # "steady" is required by the PVPPRuntime constructor: it is the baseline continuation.
    for aid, desc in (
        ("steady", "baseline drain"),
        ("sample", "take one measurement"),
        ("recharge", "restore the battery"),
    ):
        r.register_action(ActionDefinition(aid, desc, (DOMAIN,)))
    r.register_recovery_plan(
        RecoveryPlanDefinition(
            "battery_recovery", DOMAIN, FUNCTION, "recharge", (), deadline_offset=2.0
        )
    )
    r.register_governing_configuration(GoverningConfiguration(epsilon_h=0.0))
    r.register_regime_configuration(
        RegimeConfiguration(
            tau_h_existential=1.0,
            tau_h_survival=3.0,
            tau_h_stabilization=8.0,
            tau_phi_existential=10.0,
            tau_phi_survival=2.0,
            tau_phi_stabilization=0.5,
        )
    )
    r.register_constraint_rule(ConstraintRuleDefinition("action_allowed", "hard"))
    r.register_graph_instance(
        GraphInstanceDefinition("sensor", "productive_system", (DOMAIN,), "active")
    )
    r.register_graph_transformation(
        GraphTransformationDefinition(
            "continue_sampling",
            "sensor",
            "sensor",
            "continuation",
            (DOMAIN,),
            "reachable",
            "sample",
            ("continuation",),
        )
    )
    r.register_graph_transformation(
        GraphTransformationDefinition(
            "restore_battery",
            "sensor",
            "sensor",
            "corrective_repair",
            (DOMAIN,),
            "reachable",
            "recharge",
            ("recovery",),
            structurally_required=True,
        )
    )
    # Standard Sigma needs an explicit order for Stage-3 ties (3.11). Lower index wins.
    r.register_sigma_order(SigmaOrderDefinition("graph:continue_sampling", 20))

```



```
r.register_sigma_order(SigmaOrderDefinition("graph:restore_battery", 10))
return r
```

B.2 One Decision Cycle

The host supplies two adapters and one request. World is the world adapter (10.2): it implements the typed perception hook, the pressure and horizon hooks in their strict forms, projection of the baseline continuation, and a constraint profile for each policy (9.6). Projection is the policy projection service (10.3): for each candidate it returns a PolicyProjectionRecord that echoes the request's state id, time, candidate mode, horizon, and input trace, and states its model version. The request names the preliminary preservation object, the domain frame, and the graph families and policy classes that must be present (9.2).

run_cycle calls evaluate_integrated_canonical_cycle with execute left at its default of False (9.3). At a battery of 3.0 the energy domain is under pressure, the recovery transformation is structurally required, and the cycle stops at Σ having selected graph:restore_battery. Status integrated_cycle_selected_not_executed means what it says: a decision exists and nothing has been done (14.3).

Listing B.2 — b2_cycle.py (152 lines)

```
"""B.2 One decision cycle: host adapter, projection service, request, and the integrated entry."""

from pvpp_runtime import (
    PVPPRuntime,
    WorldState,
    ProductivePowerState,
    PerceivedDecisionState,
    ActualPersistentStateEnvelope,
    ActionProjection,
    PressureFactors,
    PolicyConstraintProfile,
    ConstraintObservation,
    PolicyProjectionRecord,
    RecoveryCorridorProjection,
    CanonicalDecisionCycleRequest,
    PreliminaryPreservationObject,
    DomainFrame,
    DomainFrameTarget,
)
from b1_registry import build_registry, DOMAIN, POWER, FUNCTION
from b4_layer1 import Layer1Service

class World:
    """WorldAdapter (10.2). States facts; decides nothing."""

    def perceived_decision_state_from_actual(self, actual): # typed perception hook (preferred)
        b = float(actual.pp["battery_units"])
        rep = WorldState(
            actual.time, {POWER: ProductivePowerState(POWER, b)}, {"state_id": actual.state_id}
        )
        return PerceivedDecisionState(
            actual.actor_id, actual.state_id, actual.time, rep, ppp={POWER: b}
        )

    def perceive(self, state): # must return an already-represented state unchanged (10.2)
        return state

    def domain_value(self, state, domain_id):
        return state.powers[POWER].value

    def project(self, state, action): # baseline continuation with "steady"
        b = self.domain_value(state, DOMAIN) + {"steady": -1.0, "sample": -1.0}.get(action.id, 0.0)
        if action.id == "recharge":
```

```

        b = 8.0
    return ActionProjection(
        action.id,
        WorldState(
            state.time + 1, {POWER: ProductivePowerState(POWER, b)}, dict(state.metadata)
        ),
        True,
    )

def pressure_factors(self, state, domain_id, baseline_drift):
    return PressureFactors(domain_id, self.domain_value(state, domain_id) - 2.0, baseline_drift)

def pressure_value(self, domain_id, factors):
    return 1.0 / max(factors.margin, 0.1)

def expected_deterioration(
    self, state, domain_id, baseline_drift, pressure_value, pressure_factors
):
    return float(baseline_drift) # signed change per period; negative when falling

def constraint_profile(self, state, policy_id, action_ids, regime, governing_domain_ids):
    return PolicyConstraintProfile(policy_id, (ConstraintObservation("action_allowed", False),))

class Projection:
    """PolicyProjectionService (10.3): one Q_t(pi) per request; never selects."""

    def project(self, req):
        start = req.represented_state.powers[POWER].value
        recharge = "recharge" in req.action_ids
        end = 8.0 if recharge else start - 1.0
        horizon = (
            req.projection_horizon if recharge else max(0.0, min(req.projection_horizon, end - 2.0))
        )
        corridor = RecoveryCorridorProjection(
            DOMAIN,
            FUNCTION,
            pre_recovery_viability_preserved=start >= 2.0,
            recovery_capable_region_reached=recharge,
            joint_sustainment_supported=True,
            recovery_entry_time=1.0 if recharge else None,
            projected_extinction_time=None if recharge else horizon,
        )
        terminal = WorldState(
            req.represented_state.time + 1,
            {POWER: ProductivePowerState(POWER, end)},
            dict(req.represented_state.metadata),
        )
        return PolicyProjectionRecord(
            req.policy_id,
            False,
            terminal,
            {DOMAIN: horizon},
            (corridor,),
            projection_horizon=req.projection_horizon,
            right_censored_domain_ids=(DOMAIN,) if recharge else (),
            state_id=req.state_id,
            state_time=req.represented_state.time,
            candidate_mode=req.candidate_mode,
            model_version="skeleton-q1",
            projection_input_trace=dict(req.projection_input_trace),
        )

def make_runtime():
    return PVPPRuntime(
        build_registry(),
        World(),
        projection_service=Projection(),
        layer1_transition_service=Layer1Service(),
    )

```

```

def make_request():
    return CanonicalDecisionCycleRequest(
        PreliminaryPreservationObject("keep_service", "preserve measurement-service continuity"),
        DomainFrame((DomainFrameTarget(DOMAIN, FUNCTION),)),
        required_graph_family_ids=("continuation", "corrective_repair"),
        materially_required_policy_class_ids=("continuation", "recovery"),
        projection_horizon=10.0,
        projection_input_trace={"source": "appendix-b"},
    )

def make_actual(battery=3.0):
    return ActualPersistentStateEnvelope(
        "sensor-A",
        "state-0",
        0.0,
        {"battery_units": battery},
        {"stored_charge": battery},
        {"service_available": True},
        {},
    )

def run_cycle(rt=None):
    rt = rt or make_runtime()
    integrated = rt.evaluate_integrated_canonical_cycle(
        make_actual(), make_request()
    ) # execute=False
    return rt, integrated

if __name__ == "__main__":
    _, out = run_cycle()
    d = out.decision
    print(out.status, d.stopped_at, d.selection.selected_policy_id if d.selection else None)

```

B.3 Execution Lineage

Two routes lead from a decision to a validated transition (11.1). The observation route suits a host that acts by its own means: license from the cycle (11.2), instantiate an episode (11.3), perform the action, report it as an `ExecutionObservation` (11.4), then hand off to Layer 1, apply, validate, and record provenance (11.10–11.11). Recovery bookkeeping is recorded once per transition, with an explicit confirmation naming the recovery plan it advances.

The native route suits a host that registers callables: bind the action (11.5), issue a native authorization (11.6), invoke it (11.7), rebuild the `ExecutionContext`—`invoke_authorized_native` does not return it—and pass the binding registry's ϵ observation to `advance_execution` (11.8). From there the two routes converge on the same handoff. Each route runs its own cycle: v0.141 lets one decision yield several licenses, so one execution per decision is the host's rule to keep, not the runtime's (11.2, 15.1 L4).

Listing B.3 — `b3_lineage.py` (84 lines)

```

"""B.3 Execution lineage: license to transition provenance, observation route and native route."""

from pvpp_runtime import (
    ExecutionObservation,
    ExecutionBindingRegistry,
    ExecutionBindingIdentity,
    ExecutionContext,
    RecoveryActionConfirmation,
)
from b2_cycle import run_cycle, make_request, make_actual

```

```

def observation_route():
    rt, integrated = run_cycle()
    actual, frame = make_actual(), make_request().domain_frame
    license = rt.build_execution_license_from_cycle(integrated.decision, frame) # 11.2
    step = rt.instantiate_execution(
        "episode-1", license, entry_sufficient=True, max_steps=2 # host asserts sufficiency
    ) # 11.3
    # The host performs the action by its own means, then reports what happened (11.4).
    step = rt.advance_execution(step.episode, ExecutionObservation("obs-1", completed=True))
    assert step.terminal and step.status == "completed" # treat the first terminal result as final
    handoff = rt.build_layer1_transition_handoff(step) # 11.10
    transition = rt.apply_layer1_transition(actual, handoff) # validated inside
    validation = rt.validate_layer1_transition_result(actual, handoff, transition)
    provenance = rt.build_execution_transition_provenance(
        actual, handoff, transition, validation
    ) # 11.11
    bookkeeping = rt.record_recovery_bookkeeping_from_transition( # once per transition
        actual, step, transition, (RecoveryActionConfirmation("recharge", "battery_recovery"),)
    )
    return rt, transition, provenance, bookkeeping

def native_route():
    rt, integrated = run_cycle() # a separate decision: execute each selection at most once
    actual, frame = make_actual(), make_request().domain_frame
    license = rt.build_execution_license_from_cycle(integrated.decision, frame)
    episode = rt.instantiate_execution(
        "episode-n1", license, entry_sufficient=True, max_steps=1
    ).episode
    bindings = ExecutionBindingRegistry(tuple(rt.registry.actions)) # 11.5
    action = license.action_ids[0]
    bindings.register(
        ExecutionBindingIdentity("recharge-v1", action, "1.0", ("appendix-b",)),
        lambda context: "charged",
    ) # the callable receives the ExecutionContext
    auth = rt.issue_native_execution_authorization(
        episode, action, bindings, decision_cycle_id="cycle-1"
    ) # 11.6
    result = rt.invoke_authorized_native(auth, bindings) # 11.7; consumes auth
    # invoke_authorized_native does not return the context;
    # rebuild it from the result and the authorization (11.8).
    ctx = ExecutionContext(
        result.execution_id,
        auth.action_id,
        auth.binding_id,
        auth.decision_cycle_id,
        auth.attempt,
        configuration_id=auth.configuration_id,
    )
    step = rt.advance_execution(episode, bindings.epsilon_observation(ctx, result))
    handoff = rt.build_layer1_transition_handoff(step)
    transition = rt.apply_layer1_transition(actual, handoff)
    return rt, auth, result, step, transition

if __name__ == "__main__":
    _, t, p, b = observation_route()
    print(
        "observation:",
        t.next_state.state_id,
        t.next_state.pp,
        p.next_state_id,
        b.started_corridor_ids,
    )
    rt, a, r, s, t2 = native_route()
    print(
        "native:",
        r.status,
        rt.native_execution_authorization_status(a.authorization_id),
        s.status,
        t2.next_state.pp,
    )

```

)

B.4 Layer-1 Transition Service

Layer 1 is the host's only mutation boundary (10.4). The service receives the current actual state and the handoff, applies the licensed actions, and returns a `Layer1TransitionResult` that echoes the handoff's episode and policy ids and the prior state id, gives the next state a new id, and reports the host invariants it checked. `apply_layer1_transition` validates the result against the handoff before returning it; the skeleton validates again explicitly because provenance requires the validation record (11.10–11.11).

Listing B.4 — `b4_layer1.py` (31 lines)

```
"""B.4 Layer-1 transition service: the host's only mutation boundary."""

from pvpp_runtime import ActualPersistentStateEnvelope, Layer1TransitionResult

class Layer1Service:
    def transition(self, current, handoff):
        pp = dict(current.pp)
        acts = set(handoff.licensed_action_ids)
        if "recharge" in acts:
            pp["battery_units"] = 8.0
        elif "sample" in acts:
            pp["battery_units"] = max(0.0, float(pp["battery_units"]) - 1.0)
        nxt = ActualPersistentStateEnvelope(
            current.actor_id,
            current.state_id + ":next",
            current.time + 1.0,
            pp,
            current.spv,
            current.avs,
            dict(current.context),
        )
        # Echo the handoff's identities; report host invariants; give the new state a new id.
        return Layer1TransitionResult(
            handoff.episode_id,
            handoff.selected_policy_id,
            current.state_id,
            nxt,
            True,
            {"battery_nonnegative": pp["battery_units"] >= 0.0},
        )
```

B.5 Re-entry After Change

The listing is the Chapter 12 host recipe in order. Register what an artifact depends on when it is produced (12.2); later, report the change; derive invalidations from explicit matches only; plan from the earliest invalidated stage (12.3); always apply the executable dependency scope, which alters only Joint Recovery Feasibility plans but costs nothing otherwise (12.4, 12.8); bundle the reusable artifacts of the same cycle, one payload per reusable stage in plan order; validate the bundle against the expected cycle, initial state, and configuration; then call the executor for the boundary (12.5).

Here a change to tie-break evidence invalidates only Σ , so the executor recomputes Σ alone and the selection is unchanged. The Adequacy payload is a `SigmaReusableAdequacyArtifact` carrying the cycle's projection records, which the Σ executor requires. Three cautions from Chapter 12 apply to any re-entry the skeleton is adapted to. The result is selective recomputation and is not enforcement-equivalent to a full canonical rerun (12.1, 15.1 L5). And for a PPP boundary, call `execute_ppp_reentry` directly: the dispatcher `execute_epistemic_reentry_plan` fails on that branch (defect K1). And after re-entry at Pi Completeness or earlier, check

decision.joint_recovery_feasibility before licensing: those executors do not stop for an infeasible active recovery set (defect K2, 12.5).

Listing B.5 — b5_reentry.py (87 lines)

```

"""B.5 Re-entry after change: dependency, change, invalidation, plan, bundle, Sigma executor."""

from pvpp_runtime import (
    GovernanceArtifactDependency,
    GovernanceDependencyChange,
    derive_dependency_invalidations,
    plan_governance_reentry,
    apply_executable_dependency_scope,
    GovernanceReusableArtifactBundle,
    validate_reusable_artifact_bundle,
    SigmaReusableAdequacyArtifact,
    execute_sigma_reentry,
)
from b2_cycle import run_cycle, make_actual

CYCLE_ID, CONFIG_ID = "cycle-1", "config-1"

def reenter_sigma():
    rt, integrated = run_cycle()
    d = integrated.decision
    # 1. Register, when the artifact is produced, what it depends on (12.2).
    deps = (
        GovernanceArtifactDependency("dep-sel", "selection-1", "Sigma", "evidence", ("ev-order",)),
    )
    # 2. Later, report which facts changed.
    changes = (
        GovernanceDependencyChange(
            "chg-1", "evidence", ("ev-order",), "tie-break evidence revised"
        ),
    )
    # 3. Derive invalidations from explicit matches only.
    # 4. Plan from the earliest invalidated stage (12.3).
    inv = derive_dependency_invalidations(deps, changes)
    plan = plan_governance_reentry(inv.reentry, inv.invalidation_signals)
    plan = apply_executable_dependency_scope(
        plan
    ) # changes only Joint Recovery Feasibility plans (12.4)
    assert plan.recompute_from_stage == "Sigma"
    # 5. Bundle the reusable results of THIS cycle: one payload per reusable stage,
    #     in plan order (12.4).
    payloads = {
        # Payloads for stages this executor does not consume are not inspected;
        # use the cycle's own records.
        "PPP": make_actual(),
        "Phi": d.pressure_field,
        "H": d.horizon_assessment,
        "G": d.governing_assessment,
        "R": d.regime_assessment,
        "Graph/Seed": d.graph_assessment,
        "Pi": d.pi_construction,
        "Pi Completeness": d.pi_completeness,
        "Constraints": d.constraints,
        "Domain Framing": d.domain_framing,
        "Joint Recovery Feasibility": d.joint_recovery_feasibility,
        "Adequacy": SigmaReusableAdequacyArtifact(d.adequacy, tuple(d.projection_records)),
    }
    bundle = GovernanceReusableArtifactBundle(
        "bundle-1",
        CYCLE_ID,
        integrated.initial_actual_state_id,
        CONFIG_ID,
        tuple((s, payloads[s]) for s in plan.reusable_upstream_stages),
    )
    check = validate_reusable_artifact_bundle(
        plan,
        bundle,
        expected_cycle_id=CYCLE_ID,

```

```

        expected_initial_state_id=integrated.initial_actual_state_id,
        expected_configuration_id=CONFIG_ID,
    )
    assert check.valid, check.violations
    # 6. Execute through the executor for this boundary (12.5). The result is selective
    #   recomputation: it is not
    #   enforcement-equivalent to a full canonical rerun.
    return d, execute_sigma_reentry(rt, plan, bundle, check)

if __name__ == "__main__":
    original, out = reenter_sigma()
    print(
        out.valid,
        out.status,
        out.recomputed_stage_ids,
        out.decision.selection.selected_policy_id,
        original.selection.selected_policy_id,
    )

```

B.6 Test Harness

The harness does two things. `verify_runtime_identity` recomputes the SHA-256 of every file in a manifest and returns the entries that are missing or differ; with `V0167_FILE_HASHES.sha256` an empty list identifies the tree as frozen v2.1.167, and with build 141's manifest the one expected entry, `pyproject.toml`, shows that the decision core is build 141's (13.7). Running it first means any later failure is a host failure, not a changed runtime. The tests then bank the expectations shown by the three runnable skeletons: the cycle selects recovery without executing; the observation route reaches a transition whose provenance validates and which starts the recovery corridor; the native route consumes its authorization, and a replay is refused before any binding lookup; and Σ re-entry recomputes only Σ .

Listing B.6 — `b6_harness.py` (29 lines)

```

"""B.6 Test harness: runtime identity check and banked expectations."""

import hashlib, os
import pvpp_runtime

def runtime_root():
    return os.path.dirname(os.path.dirname(os.path.abspath(pvpp_runtime.__file__)))

def verify_runtime_identity(root=None, manifest="V0167_FILE_HASHES.sha256"):
    """Return manifest entries that are missing or whose SHA-256 differs.

    An empty list means the tree is frozen Runtime v2.1.167 (13.7). With the build-141
    manifest, the one expected entry is pyproject.toml: the decision core is unchanged.
    """
    root = root or runtime_root()
    bad = []
    with open(os.path.join(root, manifest)) as fh:
        for line in fh:
            digest, rel = line.split(None, 1)
            path = os.path.join(root, rel.strip())
            if not os.path.exists(path):
                bad.append(rel.strip())
                continue
            with open(path, "rb") as f:
                if hashlib.sha256(f.read()).hexdigest() != digest:
                    bad.append(rel.strip())
    return bad

```

Listing B.7 — test_skeletons.py (52 lines)

```

"""Banked expectations for the Appendix B skeletons.

Run: PYTHONDONTWRITEBYTECODE=1 python3 -m pytest -q -p no:cacheprovider
"""

import pytest
from b6_harness import verify_runtime_identity
from b2_cycle import run_cycle
from b3_lineage import observation_route, native_route
from b5_reentry import reenter_sigma

def test_runtime_is_frozen_v2_1_167():
    assert verify_runtime_identity() == []

def test_decision_core_is_build_141():
    assert verify_runtime_identity(manifest="V0141_FILE_HASHES.sha256") == ["pyproject.toml"]

def test_one_cycle_selects_recovery_without_executing():
    _, out = run_cycle()
    assert out.status == "integrated_cycle_selected_not_executed"
    assert out.decision.stopped_at == "Sigma"
    assert out.decision.selection.selected_policy_id == "graph:restore_battery"

def test_observation_route_reaches_a_validated_transition():
    rt, transition, provenance, bookkeeping = observation_route()
    assert transition.next_state.pp["battery_units"] == 8.0
    assert rt.validate_execution_transition_provenance(provenance, transition.next_state).valid
    assert bookkeeping.started_corridor_ids == ("battery_recovery",)

def test_native_route_consumes_authorization_once():
    rt, auth, result, step, transition = native_route()
    assert result.status == "succeeded" and step.status == "completed"
    assert rt.native_execution_authorization_status(auth.authorization_id) == "consumed"

def test_native_replay_is_refused():
    rt, auth, *_ = native_route()
    # A consumed authorization is refused before any binding lookup.
    with pytest.raises(RuntimeError, match="consumed"):
        rt.invoke_authorized_native(auth, None)

def test_sigma_reentry_recomputes_only_sigma():
    original, out = reenter_sigma()
    assert out.valid and out.status == "reentry_pass_completed"
    assert out.recomputed_stage_ids == ("Sigma",)
    assert out.decision.selection.selected_policy_id == original.selection.selected_policy_id

```

Banked result. Against the hash-verified tree (346 of 346 files matching), the skeleton suite, including B.7's tests, reports 15 passed, and the four runnable skeletons print the lines below. The run disables bytecode caching so that nothing is written beside the runtime; the tree's hashes were unchanged afterwards (13.8).

Listing B.8 — Running the skeletons

```

$ R="<repository>/runtime-v2.1.167"      # the hash-verified v2.1.167 tree (13.7)
$ cd <skeleton directory>                # outside the runtime tree
$ PYTHONPATH="$R:." PYTHONDONTWRITEBYTECODE=1 python3 -m pytest -q -p no:cacheprovider
.....                                  [100%]

```


15 passed

```
$ PYTHONPATH="$R:." python3 b2_cycle.py
integrated_cycle_selected_not_executed Sigma graph:restore_battery
$ PYTHONPATH="$R:." python3 b3_lineage.py
observation: state-0:next {'battery_units': 8.0} state-0:next ('battery_recovery',)
native: succeeded consumed completed {'battery_units': 8.0}
$ PYTHONPATH="$R:." python3 b5_reentry.py
True reentry_pass_completed ('Sigma',) graph:restore_battery graph:restore_battery
$ PYTHONPATH="$R:." python3 b7_supervise.py
interrupted: reauthorization_required True invalidated
  stop: dock_stop completed verified
  effect: no_material_effect_evidenced False True
completed: completed completed completed_effect state-0:next
```

B.7 Supervised Execution

The listing supervises one licensed recharge while it runs, through the whole chain of Chapter 16: a trust provider with registration roots, a controller verifier, a Layer-1 verifier, and a fixed clock (16.2); a bridge and an active execution (16.3); a permit monitor and a charging dock registered as sources (16.4); a declared dependency (16.5); a retry contract registered before anything runs (16.8); an owner and a dock adapter (16.7). The host starts the charge by its own means and reports progress at checkpoints, leaving the native authorization issued, so the execution stays checkpointable (16.3; 15.1 L8). Nothing in the skeleton interrupts work in progress: the stop is carried out by the dock, and a native call once entered would not be governed at all.

`interrupted()` withdraws the permit mid-charge: the change fans out to the execution, the checkpoint plans re-entry from Constraints and requires reauthorization, the owner fences the unused authority and requests a cancel, the adapter maps it to `dock_stop`, the dock's attested report is recorded, and the dock's evidence of no material effect settles retry safety. `completed()` lets the charge finish: the terminal checkpoint is accepted as canonical finality, the effect is reconciled, and the host's own Layer-1 transition is recorded under the Layer-1 verifier.

What it shows. The tests bank both paths and six boundaries: idempotent checkpoint replay and the typed conflict for a reused event id; refusal of a superseded owner; a continue posture that authorizes no control; a refused Layer-1 record; and two host duties of 16.12 that the successor suite does not test—fact kinds must be written in canonical form, and observation times are compared as written, so an older report written with a different offset displaces a newer view.

Listing B.9 — `b7_supervise.py` (164 lines)

```
"""B.7 Supervised execution: one licensed recharge, governed while it runs (Chapter 16)."""

from datetime import datetime, timezone

from pvpp_runtime import ExecutionBindingRegistry, ExecutionBindingIdentity
from pvpp_runtime import ExecutionObservation, GovernanceInvalidationSignal
from pvpp_runtime.supervision import (
    HostTrustProvider, SQLiteSupervisoryStore, CanonicalRuntimeBridge, ObservationService,
    SourceRegistration, ExternalObservation, DependencyService, ContinuationService,
    ControlService, EffectService,
)
from b2_cycle import run_cycle, make_request

CFG, SUBJECT = "site-1", "sensor-A"
ROOT, PROOF = "site-admin", "proof-from-host-vault" # stands in for the host's identity system

def at(minute): # every time in one fixed ISO-8601 form: times are compared as written (16.4)
```

```

return f"2026-10-07T12:{minute:02d}:00+00:00"

def trust(): # 16.2: registration roots, controller and Layer-1 verifiers, trusted clock
    return HostTrustProvider(
        {ROOT: {"proof": PROOF, "roles": ("*",), "configurations": (CFG,)}},
        controller_attestation_verifier=lambda **r: r["attestation_evidence"] == "dock-signed",
        layer1_authority_verifier=lambda **r: r["authority_evidence"] == "ledger-signed",
        clock=lambda: datetime(2026, 10, 7, 12, 30, tzinfo=timezone.utc))

class Supervisor:
    def __init__(self):
        rt, integrated = run_cycle() # selects graph:restore_battery (B.2)
        license = rt.build_execution_license_from_cycle(integrated.decision,
                                                    make_request().domain_frame)
        self.rt, self.bridge = rt, CanonicalRuntimeBridge(rt) # 16.3
        episode = self.bridge.instantiate_execution("episode-s1", license, entry_sufficient=True,
                                                    max_steps=5).episode
        bindings = ExecutionBindingRegistry(tuple(rt.registry.actions))
        action = license.action_ids[0]
        bindings.register(ExecutionBindingIdentity("recharge-v1", action, "1.0"), lambda c: None)
        self.auth = self.bridge.issue_native_execution_authorization(
            episode, action, bindings, decision_cycle_id="cycle-1", configuration_id=CFG)
        self.store = SQLiteSupervisoryStore(":memory:", trust_provider=trust()) # 16.10
        handle = self.bridge.resolve_canonical_execution(self.auth, configuration_id=CFG)
        self.aid = self.bridge.register_active_execution(
            self.store, handle, registered_at=at(0),
            execution_id=rt.native_execution_authorization_execution_id(
                self.auth.authorization_id)).active_execution_id
        self.obs, self.deps = ObservationService(self.store), DependencyService(self.store)
        self.cont = ContinuationService(self.store, self.deps, self.bridge)
        self.cntl = ControlService(self.store, self.deps, self.cont, self.bridge)
        self.eff = EffectService(self.store, self.obs)
        for reg, source, role, kind, use in ( # 16.4: who may report what, for what use
            ("reg-m", "permit-monitor", "observation", "charge_permit", "governance"),
            ("reg-d", "dock", "effect", "charge_job", "effect_reconciliation")):
            self.obs.register_source(SourceRegistration(
                reg, source, (role,), (kind,), (SUBJECT,), (CFG,), (use,), "signed", None, None,
                "current", ROOT, 1), authority_proof=PROOF)
        self.report("permit-1", "granted", at(0))
        self.deps.register_dependency_binding( # 16.5: declared, never discovered
            active_execution_id=self.aid, configuration_id=CFG, subject_id=SUBJECT,
            fact_kind="charge_permit", fact_id="site", registration_authority=ROOT,
            authority_proof=PROOF, registered_at=at(0))
        self.eff.register_retry_contract( # 16.8: only before anything can have happened
            active_execution_id=self.aid, idempotent=False, reconciliation_required=True,
            registered_at=at(0))
        self.cntl.register_supervisory_owner(owner_id="sup-1", configuration_id=CFG,
            registration_authority=ROOT, authority_proof=PROOF,
            registered_at=at(0))
        self.owner = self.cntl.acquire_ownership(active_execution_id=self.aid, owner_id="sup-1",
            acquired_at=at(0)) # 16.7
        self.adapter = self.cntl.register_adapter(
            adapter_id="dock", configuration_id=CFG, target_handle_namespace="dock:",
            supported_semantic_controls=("hold", "cancel"),
            command_mapping={"hold": "hold", "cancel": "cancel"},
            vendor_command_mapping={"hold": "dock_hold", "cancel": "dock_stop"},
            controller_attestation_method="signed", control_region_evidence_method="dock-state",
            registration_authority=ROOT, authority_proof=PROOF, registered_at=at(0))
        self.job = "dock:job-1" # the host starts the charge by its own means (16.12)

    def report(self, observation_id, value, when, source="permit-monitor"):
        o = ExternalObservation(observation_id, source, "charge_permit", "site", SUBJECT, CFG,
            value, None, when, when, when, {})
        self.obs.submit_observation(o)
        admission = self.obs.assess_observation(o, assessed_at=when)
        return self.obs.commit_admission_change(o, admission, committed_at=when)

    def tick(self, event_id, minute, **flags): # 16.6: progress reported at a checkpoint
        return self.cont.checkpoint(active_execution_id=self.aid, checkpoint_at=at(minute),
            observation=ExecutionObservation(event_id, **flags))

```

```

def withdraw(self, minute=5): # the monitor reports; the host checkpoints each execution
    change = self.report("permit-2", "withdrawn", at(minute))
    signal = GovernanceInvalidationSignal(
        "permit-withdrawn", "Constraints", "charging permit withdrawn",
        "monitor:permit-monitor", change.observation_ids, (), None, CFG)
    return [self.cont.checkpoint(active_execution_id=aid, invalidation_signals=(signal,),
                                trigger_ids=(change.change_id,), checkpoint_at=at(minute))
            for aid in self.deps.resolve_affected_executions(change, configuration_id=CFG)]

def stop(self, assessment, control="cancel", owner=None): # 16.7
    owner = owner or self.owner
    who = dict(owner_id=owner.owner_id, owner_fence=owner.owner_fence)
    if assessment.authority_fence_required:
        self.ctl.fence_execution_authority(active_execution_id=self.aid, reason="withdrawn",
                                           **who)
    req = self.ctl.issue_control_request(
        assessment_id=assessment.assessment_id, active_execution_id=self.aid,
        requested_control=control, target_handle=self.job, created_at=at(6), **who)
    attempt = self.ctl.dispatch_control_request(
        control_request_id=req.control_request_id, target_handle=self.job,
        adapter_registration_id=self.adapter.adapter_registration_id,
        control_region_at_request="interruptible", capability_evidence_ref="dock-caps",
        mapping_identity="dock-map-1", dispatched_at=at(6), **who)
    # The dock carries out attempt.mapped_command and reports; its report is evidence.
    return attempt, self.ctl.record_enforcement_outcome(
        control_attempt_id=attempt.control_attempt_id, outcome="completed",
        controller_evidence_ref="dock:job-1:ack", observed_control_region="stopped",
        recorded_at=at(7), attestation_evidence="dock-signed")

def reconcile(self, state, minute=8): # 16.8: what the world shows, from the dock
    e = self.eff.submit_effect_evidence(
        active_execution_id=self.aid, source_id="dock", configuration_id=CFG, state=state,
        realized_bundle_ref=None, evidence_ref=f"dock:job-1:report-{minute}",
        provenance="dock report", observed_at=at(minute), effective_at=at(minute),
        received_at=at(minute))
    return self.eff.reconcile_effect(active_execution_id=self.aid,
                                    effect_evidence_id=e.effect_evidence_id,
                                    updated_at=at(minute))

def interrupted():
    s = Supervisor()
    s.tick("charge-10pct", 2)
    (assessment,) = s.withdraw()
    attempt, enforcement = s.stop(assessment)
    return s, assessment, attempt, enforcement, s.reconcile("no_material_effect_evidenced")

def completed():
    s = Supervisor()
    s.tick("charge-10pct", 2)
    done = s.tick("charge-done", 9, completed=True)
    finality = s.eff.accept_canonical_finality(active_execution_id=s.aid, terminalized_at=at(9),
                                              continuation_assessment_id=done.assessment_id)
    effect = s.reconcile("completed_effect", minute=10)
    layer1 = s.eff.record_layer1_transition( # the host applied it; this records it (16.8)
        active_execution_id=s.aid, effect_reconciliation_ref=effect.effect_record_id,
        prior_actual_state_ref="state-0", next_actual_state_ref="state-0:next",
        transition_time=at(11), evidence_ref="ledger:entry-7", provenance="host ledger",
        configuration_id=CFG, authority_evidence="ledger-signed")
    return s, done, finality, effect, layer1

if __name__ == "__main__":
    s, a, attempt, enf, eff = interrupted()
    print("interrupted:", a.continuation_posture, a.authority_fence_required,
          s.rt.native_execution_authorization_status(s.auth.authorization_id))
    print(" stop:", attempt.mapped_command, enf.outcome, enf.attestation_status)
    safe = s.eff.assert_retry_safe(active_execution_id=s.aid)
    print(" effect:", eff.state, eff.unresolved, safe)
    s, done, fin, eff, l1 = completed()
    print("completed:", done.epsilon_status, fin.epsilon_terminal_status, eff.state,

```

```
ll.next_actual_state_ref)
```

Listing B.10 — test_b7_supervise.py (86 lines)

```
"""Banked expectations for B.7. Run with the other skeleton tests (Listing B.11)."""

import pytest
from pvpp_runtime.supervision import ObservationIdentityConflict, StaleOwner
from pvpp_runtime.supervision import SourceRegistration, ExternalObservation
from b7_supervise import Supervisor, interrupted, completed, at, CFG, SUBJECT, ROOT, PROOF

def test_withdrawal_stops_the_running_job_and_fences_unused_authority():
    s, a, attempt, enforcement, _ = interrupted()
    assert a.continuation_posture == "reauthorization_required" and a.authority_fence_required
    assert s.rt.native_execution_authorization_status(s.auth.authorization_id) == "invalidated"
    assert attempt.mapped_command == "dock_stop"
    assert (enforcement.outcome, enforcement.attestation_status) == ("completed", "verified")

def test_effect_is_reconciled_separately_and_settles_retry():
    s, *_ = effect = interrupted()
    assert effect.state == "no_material_effect_evidenced" and not effect.unresolved
    assert s.eff.assert_retry_safe(active_execution_id=s.aid)

def test_completion_records_finality_effect_and_layer1():
    s, done, finality, effect, layer1 = completed()
    assert done.continuation_posture == "continue" and done.epsilon_status == "completed"
    assert finality.epsilon_terminal_status == "completed" and finality.immutable
    assert layer1.canonical_finality_ref == finality.canonical_finality_id
    with pytest.raises(ValueError, match="layer1_conflict"): # the host verifier refuses
        s.eff.record_layer1_transition(
            active_execution_id=s.aid, effect_reconciliation_ref=effect.effect_record_id,
            prior_actual_state_ref="x", next_actual_state_ref="y", transition_time=at(12),
            evidence_ref="forged", provenance="none", configuration_id=CFG,
            authority_evidence="unsigned")

def test_checkpoint_replay_is_idempotent_and_reuse_with_new_content_is_refused():
    s = Supervisor()
    first = s.tick("charge-10pct", 2)
    assert s.tick("charge-10pct", 3).assessment_id == first.assessment_id
    with pytest.raises(ObservationIdentityConflict):
        s.tick("charge-10pct", 4, information={"pct": 11})

def test_a_superseded_owner_is_refused():
    s = Supervisor()
    (a,) = s.withdraw()
    s.ctl.register_supervisory_owner(owner_id="sup-2", configuration_id=CFG,
                                     registration_authority=ROOT, authority_proof=PROOF,
                                     registered_at=at(5))
    s.ctl.acquire_ownership(active_execution_id=s.aid, owner_id="sup-2", acquired_at=at(5))
    with pytest.raises(StaleOwner):
        s.stop(a)

def test_a_continue_posture_authORIZES_no_control():
    s = Supervisor()
    with pytest.raises(ValueError, match="control_not_permitted_by_continuation_posture"):
        s.stop(s.tick("charge-10pct", 2), control="hold")

def test_fact_kinds_must_be_written_in_canonical_form():
    s = Supervisor() # the fact view keeps the kind as written; a dependency lower-cases it
    s.obs.register_source(SourceRegistration(
        "reg-m2", "door-monitor", ("observation",), ("Door_State",), (SUBJECT,), (CFG,),
```

```

        ("governance"), "signed", None, None, "current", ROOT, 1), authority_proof=PROOF)
o = ExternalObservation("door-1", "door-monitor", "Door_State", "bay", SUBJECT, CFG, "shut",
                        None, at(1), at(1), at(1), {}))
s.obs.submit_observation(o)
s.obs.commit_admission_change(o, s.obs.assess_observation(o, assessed_at=at(1)),
                             committed_at=at(1))
s.deps.register_dependency_binding(
    active_execution_id=s.aid, configuration_id=CFG, subject_id=SUBJECT,
    fact_kind="Door_State", fact_id="bay", registration_authority=ROOT,
    authority_proof=PROOF, registered_at=at(1))
with pytest.raises(ValueError, match="missing current governed fact"):
    s.tick("charge-10pct", 2) # fails closed: there is no view under "door_state"

def test_observation_times_are_compared_as_written():
    s = Supervisor()
    s.report("permit-2", "withdrawn", at(5)) # 12:05 UTC
    # 13:01+01:00 is 12:01 UTC, older than the withdrawal, but it sorts later as text.
    assert s.report("permit-3", "granted", "2026-10-07T13:01:00+01:00") is not None
    view = s.obs.read_current_fact_view(configuration_id=CFG, subject_id=SUBJECT,
                                       fact_kind="charge_permit", fact_id="site")
    assert view.value == "granted"

```

Appendix C — Status and Error Index

An alphabetical index of every status string, outcome, lifecycle and control state, mode, reason code, and validated input vocabulary in v0.141, and of every exception it raises or defines. Chapter 14 groups the same values by family; this appendix is the lookup table. It holds 237 rows for 227 distinct values in 50 families; a value used by several families, such as "REJECTED" or "active", has one row per family.

Reading a row. Kind says how to treat the value (Table C.1). Family names the record and field that carry it (Table C.2). Section is where the value is explained. Every status is reported as a value in a result record—form one of Table 14.1; exceptions and validator returns are the other forms (C.4, C.5). Messages, notes, violation strings, and invalidation-signal reasons are free text and are not indexed: they are not part of the API (1.7, 14.1).

C.1 Kinds

Table C.1 — Kinds of value

Kind	Meaning
pass	The gate, call, or pass succeeded; processing continues or the result is usable
stop	A canonical gate failed on the facts; the cycle, sequence, or pass stopped short and reports where
branch	Not a failure: the cycle takes a defined alternative path (terminal or fallback Σ)
refusal	The input or plan was invalid or not executable; nothing was computed beyond the report
state	A lifecycle, ledger, mode, or label value, not an outcome
code	A reason or trigger code explaining another value
input	A value the host supplies and the runtime validates
diag	A testing-layer diagnostic; never input to a decision (Chapter 13)
legacy	Produced only by a LEGACY / COMPATIBILITY entry (15.4); never in a canonical cycle result
unreachable	Declared or tested for, but never produced by v0.141

C.2 Families

Table C.2 — Families and the records that carry them

Family	Carried by	Section
Regime trigger	RegimeAssessment trigger code	3.4; 9.7
Regime	RegimeAssessment.regime	3.4
Graph	GraphConstructionAssessment.status	8.2; Table 9.2
Graph failure code	GraphConstructionAssessment failure codes	Table 8.1

Family	Carried by	Section
Reachability	Transformation reachability	Table 8.2; 8.4
Π construction	PiConstructionAssessment.status	3.5; 9.8
Π completeness	PiCompletenessAssessment.status	3.6; 9.8
Qualified Π completeness	QualifiedPiCompletenessAssessment.status	9.8
Substrate qualification	EpistemicSubstrateQualification.status	9.8
Constraints	ConstraintsAssessment.status	3.7; 9.9
Domain framing	DomainFramingAssessment.status	3.8; 9.9
Joint recovery	JointRecoveryFeasibilityAssessment.status	Table 5.2
Joint-recovery stop	CanonicalDecisionCycleAssessment.status	Table 9.2
Joint-recovery binding	Joint-recovery execution binding status	3.9
Joint diagnostic mode	joint_recovery_diagnostic_mode	5.8
Adequacy	AdequacyAssessment.status	3.10; 9.11
Σ selection	Selection status	3.11; 9.11
Σ fallback	SigmaFallbackAssessment.status	3.11; 15.1
Σ mode	SigmaAssessment.mode; license selection_mode	Table 3.14; Table 11.2
Candidate mode	ProjectionRequest.candidate_mode	10.3
Integrated cycle	CanonicalIntegratedCycleResult.status	Table 9.3
Memory-conditioned cycle	Memory-conditioned integrated result status	9.5
Sequence	Sequence result status	9.5
ϵ	EpsilonStepResult.status	Table 11.4
License ledger	Execution-license ledger	11.2–11.3
Episode ledger	ϵ episode ledger	11.3–11.6
Native authorization	Native authorization lifecycle	Table 11.6
Native outcome	NativeExecutionResult.status	Table 11.8
Native failure stage	NativeExecutionResult.failure_stage	Table 11.8
Control state	Native execution control state	Table 11.9
Recovery corridor	ActiveRecoveryCorridor.status	11.11

Family	Carried by	Section
Structural change	Admission, revision, refresh, and topology-update status	8.4–8.7; 6.8
Coverage observation	Structural coverage observation coverage_status	8.8
Coverage disposition	GraphStructuralCoverageDisposition.status	8.8
Information disposition	InformationGovernanceMetadata disposition	6.1
Evidence source	EvidenceSourceAuthorityState authority_state and control_state	6.7
Evidence authority loss	EvidenceDependencyAuthorityAssessment.loss_reason	6.7
Security evidence	SecurityEvidenceObservation.authority_status	6.9
Objective	Objective status	7.2
Responsiveness	Objective responsiveness relation	7.4
Re-entry execution	GovernanceReentryExecutionResult.status	12.5
Governance re-entry	execute_governance_reentry status	12.3
Joint-recovery readiness	JointRecoveryReentryReadinessAssessment.status	12.8
Epistemic re-entry	EpistemicReentryExecutionAssessment.status	12.7
Stress outcome	MultidomainStressClassification.outcome	13.4
Contaminated pair	ContaminatedSubstratePairedAssessment.status	13.5
Causal-state flag	ProjectionCausalStateCrossCycleAssessment.status (flags joined with "+")	13.6
Projection attribution	ProjectionCrossCycleAttributionAssessment.status	13.6
Attribution input class	ProjectionCrossCycleAttributionAssessment changed input classes	13.6
Legacy	Legacy entry results	15.4

C.3 Alphabetical Index

Templated re-entry statuses are indexed under their template: <stage> is one of sigma, adequacy, domain_framing, joint_recovery, constraints, pi_completeness, pi, graph, regime, governing, horizon, phi, ppp, or memory_conditioned_ppp (Table 12.2), and they sort by the text after <stage>_.

Table C.3 — Status and value index

Value	Kind	Family	Meaning	Section
abandoned	input	Objective	Not active	7.2
aborted	state	Control state	As for paused	Table 11.9
aborted_return	stop	ε	Nothing realized; returns upstream, including blocked or insufficient entry	Table 11.4
active	input	Objective	The only status that may remain in $O_i(t)$	7.2
active	state	Recovery corridor	Continuation actions remain	11.11
active	state	ε	Not terminal; the runtime's working status	Table 11.4
active_capacity_authority_required	stop	Joint recovery	Capacity is demanded but no calendar was supplied. Also the decision status when a re-entry executor stops here (12.5)	Table 5.2
active_corridors_have_no_represented_capacity_demands	pass	Joint recovery	Active obligations demand no shared capacity	Table 5.2
active_jointly_feasible	pass	Joint recovery	A schedule exists for the whole active set	Table 5.2
active_jointly_infeasible	stop	Joint recovery	No schedule exists. Also the decision status when a re-entry executor stops here; executors at P_i Completeness or earlier do not stop for it (defect K2)	Table 5.2
adequacy_no_recovery_capable_policy	branch	Adequacy	No adequate policy; fallback Σ	3.10 ; 9.11

Value	Kind	Family	Meaning	Section
adequacy_passed_some_policies	pass	Adequacy	At least one policy is restoration-adequate; standard Σ	3.10 ; 9.11
admission_definition_required	stop	Coverage disposition	Graph re-entry blocked; an admission handoff lists the gaps	8.8
ADMITTED	pass	Structural change	Admission applied to the registry	8.4– 8.7; 6.8
all_eligible_fit	legacy	Legacy	assess: every eligible discretionary action fits	15.4
all_minimal	input	Joint diagnostic mode	Every inclusion-minimal infeasible set	5.8
all_of	legacy	Legacy	DependencyRequirementDefinition.mode: every source domain required (H.5-2)	15.4
any_of	legacy	Legacy	DependencyRequirementDefinition.mode: any source domain suffices (H.5-2)	15.4
APPLIED	pass	Structural change	Reachability refresh applied	8.4– 8.7; 6.8
authoritative	input	Security evidence	Usable as governance evidence	6.9
bounded_nonterminal_survival	diag	Stress outcome	Neither recovered, collapsed, nor oscillated	13.4
candidate_policy_set	diag	Attribution input class	The candidate set changed	13.6
capacity_not_supplied	legacy	Legacy	assess: corridors active but no calendar supplied	15.4
capacity_uniquely_determines_subset	legacy	Legacy	assess: capacity leaves one subset	15.4

Value	Kind	Family	Meaning	Section
clean	input	Substrate qualification	Substrate not known to be compromised	9.8
collapse_despite_viable_alternative	diag	Stress outcome	Collapsed although a feasible path existed	13.4
competing_maximal_sets_unresolved	legacy	Legacy	assess: several maximal subsets	15.4
complete	state	Recovery corridor	No continuation actions remain	11.11
completed	input	Objective	Not active	7.2
completed	pass	ϵ	Completed as licensed	Table 11.4
completed_invalid	stop	Native outcome	Completed, but the transition is invalid	Table 11.8
completed_staged	pass	ϵ	Completed through a licensed staged path	Table 11.4
compromised	input	Evidence source	Authority lost	6.7
CONDITIONAL PASS	pass	Graph	No failure code; some transformation is uncertain. The cycle continues	8.2; Table 9.2
conflicts	input	Responsiveness	π works against the objective	7.4
consistent_progression	diag	Causal-state flag	No flag and no change	13.6
consistent_progression_with_updates	diag	Causal-state flag	No flag; inputs appeared, disappeared, or changed	13.6
constraints_filtered_candidates	pass	Constraints	Some candidates are infeasible; the rest continue	3.7; 9.9

Value	Kind	Family	Meaning	Section
constraints_no_feasible_candidates	branch	Constraints	No feasible candidate; the cycle goes to terminal Σ	3.7; 9.9
constraints_passed_all_candidates	pass	Constraints	Every candidate is feasible	3.7; 9.9
consumed	state	Native authorization	Used; any later use raises RuntimeError	Table 11.6
contaminated_extra_structure_observed	diag	Contaminated pair	The contaminated side shows extra structure	13.5
contaminated_omission_observed	diag	Contaminated pair	The contaminated side omits structure	13.5
contextual_interruption	code	Regime trigger	Existential because the host declared a contextual interruption	3.4; 9.7
contradicted	input	Information disposition	Non-current	6.1
corroborated	input	Security evidence	Usable as governance evidence	6.9
coverage_sufficient	pass	Coverage disposition	No block and no handoff	8.8
crossing_governing_consequences_unresolved	legacy	Legacy	Consequences cross between sets	15.4
current	state	Episode ledger	The live episode for its id	11.3 – 11.6
current	input	Information disposition	The only disposition usable as current positive evidence	6.1
current_period_joint_recovery_bound	state	Joint-recovery binding	Mandatory current-period recovery actions are bound into every candidate	3.9

Value	Kind	Family	Meaning	Section
defective	input	Information disposition	Non-current	6.1
degraded	input	Evidence source	Authority unresolved	6.7
descendant	input	Evidence source	Controlled by a descendant; authority lost	6.7
disputed	input	Security evidence	Kept for traceability; not usable	6.9
domain_framing_failed	stop	Domain framing	The frame is invalid; the cycle stops at Domain Framing	3.8; 9.9
domain_framing_passed	pass	Domain framing	The frame is valid for the governing set	3.8; 9.9
downstream_epistemic_divergence_observed	diag	Contaminated pair	Adequacy, selection, or G diverged	13.5
emergency_return	stop	ϵ	Execution exposed immediate governing risk	Table 11.4
epistemic_reentry_dispatched	pass	Epistemic re-entry	The executor ran; read its result	12.7
epistemic_reentry_executor_rejected	refusal	Epistemic re-entry	The executor ran and returned valid false	12.7
epistemic_reentry_not_executable	refusal	Epistemic re-entry	Invalid plan, no recomputation stage, or missing inputs	12.7
epistemically_contaminated	input	Substrate qualification	Substrate known to be contaminated	9.8
equivalent_governing_consequences_unresolved	legacy	Legacy	Consequences are equivalent	15.4

Value	Kind	Family	Meaning	Section
Existential	state	Regime	Most severe; no hysteresis rule suppresses an Existential trigger	3.4
expired	input	Objective	Not active	7.2
external_unknown	code	Native failure stage	IndeterminateExecution Failure or TimeoutError	Table 11.8
FAIL	stop	Graph	A failure code applies; the cycle stops at Graph/Seed	8.2; Table 9.2
failed	stop	ϵ	The licensed path was not realized	Table 11.4
failed_cleanly	stop	Native outcome	CleanExecutionFailure: no external effect	Table 11.8
fallback_multiway_override_not_formalized	stop	Σ fallback	Unresolved conflict among three or more candidates; no selection (L2)	3.11; 15.1
fallback_singleton_maximal_selected	pass	Σ fallback	One maximal candidate	3.11; 15.1
fallback_stage1_selected	pass	Σ fallback	Resolved at fallback stage 1	3.11; 15.1
fallback_stage2_override_selected	pass	Σ fallback	Resolved by the stage-2 override	3.11; 15.1
fallback_stage3_selected	pass	Σ fallback	Resolved at fallback stage 3	3.11; 15.1
fallback_stage4_tie_selected	pass	Σ fallback	Tie broken by the registered order	3.11; 15.1
forced_or_observed_collapse	diag	Stress outcome	Collapsed with no structurally feasible alternative	13.4
G-001	code	Graph failure code	A transformation's family is outside the eight graph families	Table 8.1

Value	Kind	Family	Meaning	Section
G-002	code	Graph failure code	A required family has no reachable transformation or valid composition	Table 8.1
G-003	code	Graph failure code	An uncertain transformation is structurally required or the only source of a required family	Table 8.1
G-004	code	Graph failure code	Uncertainty decides whether a family is present	Table 8.1
G-005	code	Graph failure code	A required path class is missing, or a composition is invalid	Table 8.1
G-007	code	Graph failure code	Seeds exceed max_seeds; no handoff	Table 8.1
G-010	code	Graph failure code	Preservation object undeclared, unknown governing domains, or no graph instances	Table 8.1
governed_agent	input	Evidence source	Controlled by a governed agent; authority lost	6.7
governing_consequences_unresolved	legacy	Legacy	No unique set on governing consequences	15.4
governing_max_pressure_existential	code	Regime trigger	Existential: highest governing pressure at or above τ_Φ existential	3.4; 9.7
governing_max_pressure_stabilization	code	Regime trigger	Stabilization: highest governing pressure at or above τ_Φ stabilization	3.4; 9.7
governing_max_pressure_survival	code	Regime trigger	Survival: highest governing pressure at or above τ_Φ survival	3.4; 9.7
governing_min_horizon_existential	code	Regime trigger	Existential: shortest governing horizon at or below τ_H existential	3.4; 9.7

Value	Kind	Family	Meaning	Section
governing_min_horizon_stabilization	code	Regime trigger	Stabilization: shortest governing horizon at or below τ_H stabilization	3.4; 9.7
governing_min_horizon_survival	code	Regime trigger	Survival: shortest governing horizon at or below τ_H survival	3.4; 9.7
governing_multiplicity_stabilization	code	Regime trigger	Stabilization: many governing domains with a short horizon	3.4; 9.7
governing_multiplicity_survival	code	Regime trigger	Survival: many governing domains with a short horizon	3.4; 9.7
graph_context	diag	Attribution input class	The graph context changed	13.6
graph_reachability_reentry_not_executable	refusal	Re-entry execution	Reachability trigger not executable	12.5
graph_structural_admission_reentry_not_executable	refusal	Re-entry execution	Structural-admission trigger not executable	12.5
historically_valid_noncurrent	input	Information disposition	Non-current	6.1
host_declared_contradiction_present	diag	Causal-state flag	A host-declared contradiction group is present	13.6
host_declared_stale_inputs_present	diag	Causal-state flag	Host-declared stale inputs are present	13.6
hysteresis_deescalation_hold	code	Regime trigger	Previous regime held: improvement is below the de-escalation margin	3.4; 9.7
hysteresis_escalation_monotonicity	code	Regime trigger	Previous regime held: the horizon worsened materially, so no de-escalation	3.4; 9.7
inactive	state	Episode ledger	Instantiated with entry not permitted	11.3 – 11.6

Value	Kind	Family	Meaning	Section
incomplete	input	Coverage observation	Known missing structure	8.8
independent	input	Evidence source	Not controlled by a governed agent	6.7
indeterminate	stop	Native outcome	Effect unknown; treated as possibly effective	Table 11.8
instantiated	state	License ledger	License used; it cannot be instantiated again	11.2 – 11.3
integrated_cycle_execution_active	state	Integrated cycle	ε instantiated but not terminal	Table 9.3
integrated_cycle_execution_blocked	stop	Integrated cycle	The license blocks entry (terminal infeasibility)	Table 9.3
integrated_cycle_selected_not_executed	pass	Integrated cycle	Selected; execute was False	Table 9.3
integrated_cycle_stopped_before_unique_sigma_selection	stop	Integrated cycle	No selection	Table 9.3
integrated_cycle_transition_applied	pass	Integrated cycle	Terminal ε and a valid Layer-1 transition	Table 9.3
integrated_cycle_transition_invalid	unreachable	Integrated cycle	Declared, never set: an invalid transition raises ValueError first (14.3; 15.9)	Table 9.3
invalid	refusal	Coverage disposition	Invalid coverage evidence or trigger	8.8
invalid_comparison	diag	Causal-state flag	The comparison had violations	13.6
invalid_<stage>_reentry	refusal	Re-entry execution	Plan, bundle, or payload types failed validation; <stage> as in Table 12.2	12.5

Value	Kind	Family	Meaning	Section
invalid_reentry_plan	refusal	Governance re-entry	Invalid plan, a plan asking to execute, or a PPP plan declaring reusable stages	12.3
invalidated	state	Episode ledger	Retired by authorization invalidation	11.3 – 11.6
invalidated	state	Native authorization	Retired; any later use raises RuntimeError	Table 11.6
issued	state	License ledger	License built from a cycle; not yet instantiated	11.2 – 11.3
issued	state	Native authorization	May be consumed once	Table 11.6
joint_recovery_capacity_authority_required	stop	Joint-recovery stop	Cycle stopped: capacity authority missing	Table 9.2
joint_recovery_feasibility_failed	stop	Joint-recovery stop	Cycle stopped: the active set is infeasible	Table 9.2
joint_recovery_infeasible	unreachable	Re-entry execution	Tested for by the Pi Completeness executor, never produced (defect K2)	12.5
joint_recovery_reentry_dependency_scope_insufficient	refusal	Joint-recovery readiness	Apply <code>apply_executable_dependency_scope</code> first	12.8
joint_recovery_reentry_ready	pass	Joint-recovery readiness	Plan, cycle, and scope support joint-recovery re-entry	12.8
local_callable	code	Native failure stage	An unclassified exception from the callable	Table 11.8
material_omission_with_apparent_completeness	diag	Contaminated pair	A material class is missing yet contaminated Π passed	13.5

Value	Kind	Family	Meaning	Section
memory_cycle_execution_active	state	Memory-conditioned cycle	ε instantiated but not terminal	9.5
memory_cycle_execution_blocked	stop	Memory-conditioned cycle	The license blocks entry	9.5
memory_cycle_selected_not_executed	pass	Memory-conditioned cycle	Selected; not executed	9.5
memory_cycle_stopped_before_unique_sigma_selection	stop	Memory-conditioned cycle	No selection	9.5
memory_cycle_transition_and_epistemic_update_applied	pass	Memory-conditioned cycle	Transition and epistemic update applied	9.5
memory_cycle_transition_applied	pass	Memory-conditioned cycle	Transition applied	9.5
memory_sequence_completed	pass	Sequence	Every memory-conditioned cycle ran	9.5
memory_sequence_invariant_failure	stop	Sequence	A sequence invariant failed	9.5
memory_sequence_stopped_at_active_epsilon	stop	Sequence	A cycle left ε active	9.5
Mission	state	Regime	Least severe regime	3.4
mission_default	code	Regime trigger	Mission: no threshold met	3.4; 9.7
neutral	input	Responsiveness	No bearing	7.4
no_active_corridors	legacy	Legacy	assess: RecoveryAdequacyAssessment with no active corridor	15.4
no_active_recovery_corridors	pass	Joint recovery	Nothing to compose	Table 5.2

Value	Kind	Family	Meaning	Section
no_adequate_policy_set	legacy	Legacy	No adequate set. Also a guarded branch of the canonical standard- Σ helper that every caller makes unreachable	15.4
NO_CHANGE	pass	Structural change	Valid update; topology identity unchanged	8.4–8.7; 6.8
no_current_period_joint_recovery_binding	state	Joint-recovery binding	Nothing to bind this period	3.9
no_eligible_discretionary_actions	legacy	Legacy	assess: nothing eligible	15.4
no_reentry_required	pass	Governance re-entry	The plan establishes no invalidated stage; nothing runs	12.3
no_registered_policy_structure	legacy	Legacy	No relations or requirements registered	15.4
no_structural_or_downstream_divergence_observed	diag	Contaminated pair	No difference found	13.5
no_structurally_admissible_policy_set	legacy	Legacy	Registered structure excluded every set	15.4
ordinary_adequacy	state	Candidate mode	The only mode v0.141 sends; the projection record must echo it	10.3
partial_realization	stop	ϵ	Some PV realized without completion; returns upstream	Table 11.4
partial_stage_reentry_not_yet_supported	refusal	Governance re-entry	Any boundary other than PPP; use a stage executor	12.3
PASS	pass	Graph	No failure code and no uncertain transformation	8.2; Table 9.2
paused	state	Control state	Invocation invalidates the authorization and raises RuntimeError	Table 11.9
periodic_survival_cycling_with_viable_recovery_path	diag	Stress outcome	Oscillated although recovery was feasible	13.4

Value	Kind	Family	Meaning	Section
pi_blocked_by_graph_invalidity	stop	Π construction	The graph failed; no policy space	3.5; 9.8
pi_boundedness_overflow	stop	Π construction	More unique candidates than max_candidates	3.5; 9.8
pi_completeness_basis_not_supplied	legacy	Legacy	evaluate_policy_sets makes no completeness claim	15.4
pi_completeness_failed_missing_material_policy_classes	stop	Π completeness	A required class is missing; the cycle stops	3.6; 9.8
pi_completeness_passed	pass	Π completeness	Every materially required policy class is present	3.6; 9.8
pi_constructed	pass	Π construction	A policy space was built	3.5; 9.8
pi_missing_mandatory_continuation	stop	Π construction	No mandatory continuation candidate	3.5; 9.8
predictive_model_authority	diag	Attribution input class	The model authority changed	13.6
prohibited_for_use	input	Information disposition	Non-current	6.1
projection_and_observed_authoritative_inputs_unchanged	diag	Projection attribution	Nothing changed	13.6
projection_causal_inputs	diag	Attribution input class	Projection causal inputs changed	13.6
projection_changed_with_observed_authoritative_input_change	diag	Projection attribution	Projection and inputs both changed	13.6

Value	Kind	Family	Meaning	Section
projection_changed_without_observed_authoritative_input_change	diag	Projection attribution	Determinism anomaly	13.6
projection_horizon	diag	Attribution input class	The projection horizon changed	13.6
projection_unchanged_with_observed_authoritative_input_change	diag	Projection attribution	Inputs changed; projection did not	13.6
provisional	input	Security evidence	Never usable directly; requires independent confirmation	6.9
qualified_pi_conditional_warning	pass	Qualified Π completeness	Locally complete on an uncertain or contaminated substrate; not a full pass	9.8
qualified_pi_fail	stop	Qualified Π completeness	Incomplete on a clean substrate	9.8
qualified_pi_fail_with_layered_attribution	stop	Qualified Π completeness	Incomplete on an uncertain or contaminated substrate	9.8
qualified_pi_pass	pass	Qualified Π completeness	Complete on a clean substrate	9.8
reachable	state	Reachability	Realizable from the current state and configuration	Table 8.2; 8.4
ready	state	Control state	Invocation proceeds (default)	Table 11.9
reauthorization_required	state	Control state	As for paused; a reauthorization handoff can be built	Table 11.9

Value	Kind	Family	Meaning	Section
recovery	diag	Stress outcome	The run recovered	13.4
recovery_unavailable_fallback	state	Σ mode	Fallback selection; no adequate policy	Table 3.14 ; Table 11.2
reentry_pass_completed	pass	Governance re-entry	PPP boundary: one integrated cycle with execute False	12.3
reentry_pass_completed	pass	Re-entry execution	The bounded pass reached Σ ; ϵ not executed	12.5
reentry_pass_stopped	stop	Re-entry execution	A gate stopped the bounded pass; the decision says where	12.5
registered_structure_filtered_policy_sets	legacy	Legacy	Domain relations or dependency requirements filtered sets	15.4
registered_structure_nonbinding	legacy	Legacy	Registered structure excluded nothing	15.4
rejected	input	Information disposition	Non-current	6.1
REJECTED	refusal	Structural change	Request refused; registry or topology unchanged	8.4–8.7; 6.8
representation_time_regression	diag	Causal-state flag	An input's own time went backward	13.6
represented_state	diag	Attribution input class	The represented state changed	13.6
resolved_by_governing_consequences	legacy	Legacy	assess/decide resolved by governing consequences	15.4

Value	Kind	Family	Meaning	Section
REVISED	pass	Structural change	Revision applied to the registry	8.4–8.7; 6.8
revoked	input	Evidence source	Authority lost	6.7
same_timestamp_value_change	diag	Causal-state flag	A value changed at the same represented time	13.6
sequence_completed	pass	Sequence	Every cycle ran	9.5
sequence_invariant_failure	stop	Sequence	A sequence invariant failed	9.5
sequence_stopped_at_active_epsilon	stop	Sequence	A cycle left ϵ active	9.5
sigma_standard_policy_selected	pass	Σ selection	Standard Σ selected a policy	3.11; 9.11
sigma_terminal_policy_selected	pass	Σ selection	Terminal Σ selected under infeasibility; the license blocks entry	3.11; 9.11
source_authority_unresolved	code	Evidence authority loss	Degraded authority or unknown control	6.7
source_compromised	code	Evidence authority loss	Highest precedence	6.7
source_controlled_by_descendant	code	Evidence authority loss	Source controlled by a descendant	6.7
source_controlled_by_governed_agent	code	Evidence authority loss	Source controlled by a governed agent	6.7
source_expired	code	Evidence authority loss	as_of is after valid_until	6.7
source_revoked	code	Evidence authority loss	Source revoked	6.7
source_stale	code	Evidence authority loss	Source stale	6.7

Value	Kind	Family	Meaning	Section
Stabilization	state	Regime	Second regime	3.4
<stage>_reentry_not_executable	refusal	Re-entry execution	Wrong boundary or missing reusable passes or inputs	12.5
<stage>_reentry_projection_authority_failed	refusal	Re-entry execution	Projection model authority invalid; for domain_framing, joint_recovery, constraints, and pi_completeness	12.5
stale	input	Evidence source	Authority lost	6.7
standard	state	Σ mode	Ordinary selection among adequate policies	Table 3.14 ; Table 11.2
structurally_forced_periodic_survival_cycling	diag	Stress outcome	Oscillated with no feasible recovery	13.4
succeeded	pass	Native outcome	The callable returned	Table 11.8
sufficient	input	Coverage observation	No material gap and no uncertainty	8.8
superseded	input	Information disposition	Non-current	6.1
supports	input	Responsiveness	π supports the objective	7.4
Survival	state	Regime	Third regime	3.4
suspended	input	Objective	Not active	7.2
terminal	state	Episode ledger	Reached a terminal ϵ status	11.3 – 11.6

Value	Kind	Family	Meaning	Section
terminal_infeasibility	state	Σ mode	Terminal selection; no feasible policy	Table 3.14 ; Table 11.2
transport	code	Native failure stage	ConnectionError	Table 11.8
trusted	input	Evidence source	Authority intact	6.7
uncertain	input	Coverage observation	Unresolved structural uncertainty	8.8
uncertain	state	Reachability	Unresolved; can make the graph CONDITIONAL PASS or, where decisive, FAIL	Table 8.2; 8.4
uncertain	input	Substrate qualification	Substrate integrity unresolved	9.8
unique_feasible_adequate_policy_set	legacy	Legacy	One feasible adequate set (evaluate_policy_sets, evaluate_policy_space, or evaluate_recovery_policy_space)	15.4
unique_governing_consequence_dominance	legacy	Legacy	One set dominates on governing consequences	15.4
unique_structurally_admissible_policy_set	legacy	Legacy	One set survives registered structure	15.4
unknown	input	Evidence source	Control unresolved	6.7

Value	Kind	Family	Meaning	Section
unreachable	state	Reachability	Not realizable now; excluded from seeds	Table 8.2; 8.4
unresolved	input	Information disposition	Non-current	6.1
unresolved	input	Responsiveness	Not determined	7.4
unsupported_uncertain	stop	Coverage disposition	Graph re-entry blocked; no handoff	8.8
untrusted	input	Security evidence	Kept for traceability; not usable	6.9
UPDATED	pass	Structural change	Control-provenance topology changed	8.4–8.7; 6.8
validation	code	Native failure stage	Completed but invalid: CompletedInvalidExecution	Table 11.8
witness	input	Joint diagnostic mode	Default: one inclusion-minimal infeasible set	5.8

C.4 Exceptions

The decision core raises only built-in exception types and defines no exception hierarchy of its own; Table C.4 is unchanged from build 141. Raise-site counts are from the frozen source; a site may serve several conditions. Match on the type and the context, never on message text. The supervision layer's exceptions are in Table C.6.

Table C.4 — Exceptions

Type	Raised for	Raise sites	Section
ValueError	Almost every refusal: unmet preconditions, invalid host records, identity mismatches, forged or stale authority, record mutation, integrity failures, duplicate registrations	367: kernel.py 274, registry.py 56, execution.py 22, reentry.py 8, testing.py 6, feasibility.py 1	14.7

Type	Raised for	Raise sites	Section
RuntimeError	"steady" not projectable; horizon seed empty; transfer_history_service missing; authorization consumed, invalidated, or blocked by control state; paused, aborted, or reauthorization-required execution; deprecated ExecutionBindingRegistry.invoke; oracle visit limit	11: kernel.py 6, execution.py 4, testing.py 1	14.7; 11.6; 13.3
KeyError	Unknown native authorization; no binding registered for an action or binding id	5: kernel.py 3, execution.py 2	11.5–11.7
TypeError	Wrong object where an episode, authorization, native result, telemetry, or callable binding is required	7: execution.py 5, kernel.py 2	11.5–11.8
TypeError (from Python)	The PPP branch of execute_epistemic_reentry_plan	Not a raise site: defect K1	12.7; 15.2
CleanExecutionFailure	Raised by a host binding to assert that no external effect occurred; converted to "failed_cleanly"	Defined in models.py; never raised by the runtime	11.7
IndeterminateExecutionFailure	Raised by a host binding when the effect is unknown; converted to "indeterminate"	As above	11.7
CompletedInvalidExecution	Raised by a host binding when execution completed but is invalid; converted to "completed_invalid"	As above	11.7

C.5 Validator Return Forms

Table C.5 — Which form each validator returns

Form	Validators	Invalid when
Assessment with valid, violations, notes	Every validate_, assess_, and audit_ method not listed below	valid is false
Tuple of violation strings	validate_transfer_history_event, validate_memory_state_reference, validate_memory_retrieval_package	The tuple is non-empty
The validated object, or an exception	validate_projection_record	ValueError is raised

C.6 How the Index Was Checked

The index was built from the frozen source, not from Chapter 14, and then compared with it. Three scans of the runtime package found candidate values: string constants in keyword arguments, comparisons, and assignments whose name ends in status, state, outcome, mode, disposition, relation, or reason; string constants passed, by position or keyword, to a field of that kind in any runtime record; and every snake_case constant with a status-like suffix. Each candidate was either indexed or set aside as a field name, message fragment, free-text reason, or the private return code of an internal helper. In the other direction, every value in Chapter 14 was found in the source, directly or through its template. Static scans cannot prove a negative, but three independent routes to the same set make an omission unlikely.

The check found five errors in the earlier chapters, all now corrected:

- Table 14.5 omitted the coverage disposition "invalid" and five of the seven evidence-authority loss reasons.
- Table 14.2 listed "epistemically_contaminated" as a qualified-completeness status; it is a substrate qualification input.
- Table 14.2 gave the graph failure codes as a range, G-001 to G-010; v0.141 defines seven of them.
- Table 14.6 listed "joint_recovery_infeasible" as a status. No record produces it. The Pi Completeness executor tests for it, so that executor, and every earlier one that delegates to it, does not stop for an infeasible active recovery set. This is defect K2, confirmed against the frozen code and recorded in 12.5 and 15.2.
- Chapter 6 paraphrased two loss reasons; it now gives their exact names.

C.7 Supervision Exceptions and Values

Raise-site counts are from pvpp_runtime/supervision. Section is where the value or exception is explained.

Table C.6 — Supervision exceptions

Type	Base	Raise sites	Section
ConcurrencyConflict	RuntimeError	34 across seven modules	16.11
CanonicalResolutionError	RuntimeError	30: bridge.py, continuation.py	16.3; 16.11
ConfigurationConflict	CanonicalResolutionError	1: bridge.py	16.3
ContinuationReconciliationRequired	ConcurrencyConflict	3: continuation.py	16.6
ObservationIdentityConflict	ConcurrencyConflict	2: continuation.py	16.6
ObservationValidationError	ValueError	12: observation_value.py	Table 16.15
StaleOwner	ConcurrencyConflict	3: control.py	16.7

Type	Base	Raise sites	Section
AdapterUnregistered, ControlMappingWidened, TargetScopeViolation	RuntimeError	3 each: control.py	16.7
EvidenceUnverified	RuntimeError	0	16.11; H.5-11
ValueError	built-in	68	16.11
PermissionError	built-in	15: dependency.py, recovery.py, store.py	16.11
KeyError	built-in	9	16.11
sqlite3.IntegrityError	built-in	Untranslated duplicate inserts	16.11

Table C.7 — Supervision value vocabularies

Family	Values	Section
Continuation posture	continue, reauthorization_required, abort_return, emergency_return, indeterminate	Table 16.13
Semantic control	hold, pause, cancel, reauthorization_required, terminate	Table 16.18
Enforcement outcome	accepted, completed, rejected, too_late, timed_out, unknown	Table 16.19
Attestation status	verified, weak, unverified	16.7
World-effect state	open, no_material_effect_evidenced, partial_effect, completed_effect, invalid_effect, effect_unknown, resolved	Table 16.21
Terminal ϵ status accepted as finality	completed, completed_staged, partial_realization, aborted_return, failed, emergency_return	16.8
Admission disposition; reason	admissible, retained_noncurrent; admissible, source_not_current, scope_or_use_or_time_invalid	16.4
Fact view state	current, unresolved	Table 16.10
Source registration status	current, revoked, superseded, limited	16.4
Continuation intent state	prepared, closed, abandoned, reconciliation_required, terminally_fenced	Table 16.16
Divergence reconciliation outcome	reconciled_closed, terminally_fenced, not_required_abandoned_closed, not_required_superseded_closed	16.6
Recovery state	recovering, reconciled, unresolved, conflict	16.9
Registration role	source_registration, dependency_registration, owner_registration, adapter_registration, recovery_registration, continuation_reconciliation	Table 16.3

Family	Values	Section
Audit event kind	Thirteen kinds	Table 16.28

Appendix D — Version Matrix

Every claim in this Reference depends on some version: of the framework, a compendium document, the runtime, the proof program, or this book. This appendix lists each version the book cites, with where it comes from, and shows every conflict found between sources rather than resolving it silently. Document versions are taken from the compendium tables of contents and checked against each document's own header; runtime versions from the frozen tree; proof versions from the proof repository.

D.1 Framework and Compendia

Table D.1 — Framework versions

Version	Master file set	Standing	Source
Framework Version 2.1	MFS11	Current; frozen	C00/D8
Framework Version 2.0	MFS10	Preserved closed baseline	C00/D8; C00/D5
Compendia 00, 10, 15, 20, 30, 40, 45	Version 2.1 editions	Controlling (00–30); runtime and testing record (40, 45)	Project files

Table D.2 lists every compendium document the Reference cites, with its version. Documents cite one another at older versions in their revision histories; the Reference always means the version below. For compendium 00 the versions are those of the rebuilt September 2026 edition, which records the v0.141 runtime build (V6).

Table D.2 — Cited compendium documents

Citation	Document	Version	Note
C00/D1	Architectural Evolution and Authority Timeline	v0.5	Frozen; header still says 0.4 (V6)
C00/D2	Current State Assessment and Next Workplan	v0.5	Frozen
C00/D3	Document Header and Status Standard	v2.4	Frozen
C00/D6	Repository Structure Standard	v2.4	Frozen
C00/D7	Unsafe Stale Reference Checklist	v0.4	Adds post-v0.141 stale signals (§12)
C00/D8	Version 2.1 Freeze Declaration and Closeout Record	v0.2	Freeze authority for MFS11; §10 records v0.141
C00/D9	Versioning and Change Control Protocol	v2.4	Frozen; header still says 2.3 (V6)
C10/D1	Canonical State Object and Symbol Table	v0.5	
C10/D2	Layer 1 Canonical Core Formal System	v1.9.4	

Citation	Document	Version	Note
C10/D6	Layer 2 Shared Policy-Conditional Projection Service	v1.1	Cites Σ v1.11 and "v1.13 or later" (V1)
C10/D7	Viability Domain Framing Rule	v0.5	
C10/D9	Actor Purpose and Objective Interface	v0.1	New V2.1 owner surface
C10/D10	Capital Assets and Environmental Capability Classification Note	v0.1	
C10/D11	Collective Entity Qualification, Membership, and Nested Viability	v0.2	
C10/D12	Execution-Contingent Capability Revision and Adaptive Interaction Control	v0.3 or v0.2	Title and header disagree (V3)
C10/D16	Memory Interface and Pipeline Integration	v0.5	
C10/D18	Opportunity Closure, Reachability, and Delay	v0.2	
C10/D19	Perception and Signal Interpretation Architecture	v0.9	
C10/D20	Resource Transformation, Transfer, and Capability Change	v0.2	
C10/D21	System State Dynamics	v0.6	
C10/D22	Transfer History Object Specification	v0.2	
C15/D1	Targeted Revision Plan for Π Completeness Validation	v0.1	Planning note, not a specification
C15/D2	Governing Domain Identification Operator G	v0.6	
C15/D3	Regime Classification Operator $\mathcal{R}$	v0.5	
C15/D4	Collapse Horizon Operator H	v1.7	
C15/D5	Constraint System	v0.7	
C15/D6	Σ Selection Operator	v1.14	Controls Σ (V1)
C15/D7	Policy Construction Operator Π	v0.8	
C15/D8	Pressure Field Operator Φ	v0.7	
C15/D9	Realization Operator ε	v1.7	
C15/D10	Restoration Adequacy Operator	v0.8	Current; C20/D2 names v0.9 (V2)
C15/D11	Instance-Transformation Graph Layer	v0.4	
C15/D12	Π Completeness Validation Rule	v0.4	

Citation	Document	Version	Note
C15/D14	Graph Construction Protocol	v0.4	
C15/D17	Σ and Fallback Structure Parameterization Addendum	v0.2	
C20/D2	Canonical Operator Ownership Map	v0.9	Names Adequacy v0.9 in error (V2)
C20/D3	Canonical Operator Stack Reference	v0.8 Merged	
C20/D4	Canonical Operator System Specification, Master Architecture Document	v1.4	
C30/D3	Cross-Domain Core Schema and Application Mapping Reference	v0.1	
C30/D4	Layer 3 Applications and Structured Case Analyses	v1.2	
C45/D2	Version 2.1 Consolidated Regression and Hostile Consistency Audit	v0.1	
C45/D8	Multi-Scenario Hardening Tests for Π Completeness on Contaminated Substrates	v0.1	
C45/D9	Replay Sufficiency and Historical Decision Reconstruction	v0.1	
C45/D27	August 2026 Canonical Framework Incorporation and Promotion Audit	v0.6	Classifies C10/D8 as exploratory and deferred (V4)

D.2 Runtime Generations and Builds

Table D.3 — Runtime generations

Generation	Frozen build	Directory	Exports / methods / tests	Standing
Runtime V1	v0.70	runtime-v1	152 / 66 / 496	Preserved
Runtime V2	v0.131	runtime-v2	306 / 78 / 967	Preserved; unchanged
Runtime V2.1	v0.141	runtime-v2.1	335 / 89 / 1,093	Preserved; its decision core is carried unchanged in v2.1.167
Runtime V2.1, build 167	v2.1.167 (developed as v0.167)	runtime- v2.1.167	335 + 51 / 89 / 1,320	Current; frozen; subject of this book

The book cites the builds in Table D.4, each for the surface it introduced. Builds before v0.132 are not described in the V2.1 release notes; their attribution rests on build-numbered test and benchmark files and on source comments, which is evidence of when a surface was added, not of its current behavior. Behavior is always v0.141's.

Table D.4 — Builds cited

Build	Cited for	Evidence	Where
v0.32	Canonical decision cycle; the pre-canonical evaluators become legacy	Test and benchmark named _v032	9.13; H.5-5
v0.38	record_layer1_execution_commit	Its docstring	11.11; 15.4
v0.64	Fresh artifact-integrity audit at licensing	test_cycle_artifact_identity_integrity_v064	13.8
v0.74	Portable-adapter separation; most declared-only protocols	test_v2_adapter_configuration_v074	10.9
v0.76	Native invocation	test_v2_native_invocation_v076	10.9; 15.9
v0.77	ExecutionBindingRegistry.build_layer1_handoff	test_v2_native_layer1_handoff_v077	H.5-7
v0.79	ϵ bridge: the ϵ trace becomes the canonical execution evidence	test_v2_native_epsilon_bridge_v079	11.8; 15.4
v0.84	execute_governance_reentry, PPP boundary only	Source comment and docstring	12.3
v0.110	Security-evidence qualification	Source comment	6.9
v0.111 – v0.112	Evidence-source authority and fact-to-source dependencies	Source comments	6.7
v0.124 – v0.126	Control-provenance topology	Source comments	6.7; 6.8
v0.131	Runtime V2 frozen; runtime-private license lineage	runtime-v2 release notes; source comment	1.4; 13.8
v0.132	V2.1 successor bootstrap; inherited-file identity check	RELEASE_NOTES.md; BASELINE_CODE_IDENTITY_CHECK.txt	1.4; 15.7
v0.133 – v0.136	Objective interface and responsiveness; information governance; transfer history	RELEASE_NOTES.md	Chapters 6–7

Build	Cited for	Evidence	Where
v0.13 7	Replay sufficiency; still the default runtime_version of build_decision_replay_package (V5)	RELEASE_NOTES.md; kernel.py	6.4
v0.13 8– v0.14 0	Transition-relevant state sufficiency; objective lifecycle re-entry; Small Economy benchmark	RELEASE_NOTES.md	5.9; 7.6; 13.8
v0.14 1	Hardening and freeze; pyproject version "0.141"	RELEASE_NOTES.md; V0141_FILE_HASHES.sha256	Through out

D.3 Σ Versions by Source

Table D.5 — The Σ version each source cites

Source	Σ version	Reading
C15/D6, the Σ owner file	v1.14	Controls
C20/D2, the ownership map	v1.14	Agrees with the owner
C10/D6, the projection service	v1.11; requires "v1.13 or later" for censor-aware dominance	Historical reconciliation list; consistent with v1.14
Proof-program documents	v1.11 and v1.13	Cited as of their writing
v0.141 standard-Σ docstring (kernel.py)	v1.13	Stale provenance (15.9 D1)
This Reference and the Guide	v1.14	Table 2.1; 3.11

D.4 Proof Program

Table D.6 — Proof-program versions

Object	Version	Standing
Combined Theorem Packet	V3.25 Final	Controlling theorem authority (1.2; 1.9)
Status and boundary layer	V3.26	Status and boundary controls; does not replace V3.25
Internal closeout	September 11, 2026	Finite program internally closed; Direction 1 proved locally only, Direction 2 strong but conditional
Published paper, When Is One Number Enough?	Version 2.0, September 2026; DOI 10.5281/zenodo.22715202	Publishes Direction 1; not theorem authority over V3.25
Public proof repository	CITATION.cff v1.0 (released August 7, 2026)	Packaging of the program

D.5 Companion Volumes and Retired Documents

Table D.7 — Book and documentation versions

Document	Version	Standing
PV-PP Getting Started	Edition 2.1, for Runtime v2.1.167	Companion volume: entry point and tutorial
PV-PP Programmer's Guide	Edition 2.1, for Runtime v2.1.167	Companion volume: the Guide this Reference accompanies (Appendix G)
This Reference	Edition 2.1, for Runtime v2.1.167	Exact description of runtime v2.1.167; previous version described v0.141
Doc 05, Runtime API Reference V2.1	V2.1 edition, describing v0.141	Shipped in runtime-v2.1/docs; superseded by this Reference
Reader documents 01–10	V2.1 editions	Superseded as sources; see Appendix F

D.6 Verification Environment

Tree identity is not environment identity (4.9). Every run reported for this edition used Python 3.13.16 and pytest 9.1.1 against a fresh extraction of the frozen archive PVPP_Runtime_v2.1.167_FROZEN.zip (SHA-256 7359f88ba83004af078e1f0cd26f1bb045bbb137c96e70044c3e0eb93087a85e) on 7 October 2026; the runtime declares requires-python ">=3.11". Runs reported for build 141 in the previous edition used Python 3.11.15. The hash check (13.7) identifies the runtime tree; these values identify the environment.

D.7 Version Conflicts

Table D.8 — Conflicts between sources

#	Subject	What the sources say	Resolution
V1	Σ version	C15/D6 and C20/D2 give v1.14; C10/D6, the proof documents, and the runtime docstring cite v1.11 or v1.13	v1.14 controls. The ordinary-case rule is unchanged across v1.11–v1.14 (Table 2.8; 15.9 D1)
V2	Adequacy version	C20/D2 names v0.9; C15/D10 is v0.8	v0.8 is current and no v0.9 exists; a project-control defect in compendium 20 (15.9 D2)
V3	C10/D12 version	The compendium table of contents and title give v0.3; the document's own header gives v0.2	Unresolved. The Reference cites C10/D12 by number only (8.9–8.11); project control should reconcile the header

#	Subject	What the sources say	Resolution
V4	Authority for re-entry	Working versions of this Reference cited C10/D8 as the framework basis of dependency-scoped re-entry. C10/D8 is a v0.1 proposed clinical bridge, marked for exclusion from public release, that does not specify stage re-entry; C45/D27 classifies it as exploratory and deferred	Corrected: no V2.1 owner file defines re-entry; it is a runtime construct serving C20/D4's non-laundering rule and C20/D3's order (12.1; 15.6). C10/D8 is no longer cited anywhere
V5	Replay runtime version	build_decision_replay_package defaults runtime_version to "0.137"; the tree is v0.141	Pass "0.141" (6.4; 15.9 D3)
V6	C00 headers	The rebuilt compendium 00 gives C00/D1 as v0.5 and C00/D9 as v2.4 in its titles; their headers still read 0.4 and 2.3. The pre-rebuild edition lacks C00/D1 §23, C00/D3 §15, C00/D6 §14, C00/D8 §10, and C00/D9 §18	The Reference follows the rebuilt edition, which holds every section it cites (F.2), and takes the version numbers from the titles
V7	Three proof-program version numbers	Theorem packet V3.25 Final; published paper version 2.0; repository CITATION.cff version v1.0	Not a conflict: three different objects (D.5). Only V3.25 Final is theorem authority

Three more title-and-header mismatches lie outside the cited set: the Framework Reader Guide (C20/D11, v0.5 against v0.4), the Forest versus Trees memo (C00/D4, v1.6 against V1.5), and the Open Questions Register (C00/D10, v0.4 against 0.3). The Reference cites none of them.

D.8 Feature Matrix by Build

Table D.9 sets what the framework provides for beside what three frozen builds implement: v0.131 (Runtime V2, Version 2 framework), v0.141 (Runtime V2.1), and v2.1.167. Getting Started Section 2.5 gives the same story at a glance, by what you can do rather than by build. For the framework, Yes means the framework defines or permits the feature; for a build, Yes means the build implements it, Partly that it implements part, and Host that it is a host duty the build does not check; n/a marks an implementation matter outside the framework. "Partly" for governance of a running action in v0.131 and v0.141 means changes the host reports between ϵ steps and before a call; nothing while a call is under way.

Table D.9 — The framework and three builds

Feature	Framework V2.1	v0.131	v0.141	v2.1.167	Reference
MODELING AND STATE					
Actual state kept apart from the perceived state decisions use	Yes	Yes	Yes	Yes	5.1–5.3

Feature	Framework V2.1	v0.131	v0.141	v2.1.167	Reference
Objectives as a governed interface, never hidden utility	Yes	No	Yes	Yes	Chapter 7
Information governance: authority, scope, permitted use, freshness	Yes	Partly	Yes	Yes	6.1; 16.4
Transfer history: when a fact was true vs. when it was known	Yes	No	Yes	Yes	6.3
Inheritance: configuration may pass, current PP never	Yes	Host	Host	Host	15.3
DECIDING					
Staged decision cycle that stops with a stated reason	Yes	Yes	Yes	Yes	Chapter 9
Recovery-aware adequacy and joint-recovery feasibility	Yes	Yes	Yes	Yes	5.8; 9.11
Projections checked for model authority and licensed inputs	Yes	Yes	Yes	Yes	9.10
Selective re-entry from the earliest invalidated stage (K1, K2)	Yes	Yes	Yes	Yes	Chapter 12; 15.2
Replay of a past decision from its records	Yes	No	Yes	Yes	6.4
Transition-relevant state sufficiency	Yes	No	Yes	Yes	5.9
AUTHORITY AND EXECUTION					
Permission to act only from a completed decision	Yes	Yes	Yes	Yes	11.2
Single-use authority, checked at the moment of the call	Yes	Yes	Yes	Yes	11.6
Host-owned state transition, validated and attributed	Yes	Yes	Yes	Yes	11.10
WHILE AN ACTION RUNS					
Governance of a running action when its facts change	Yes	Partly	Partly	Yes, at host checkpoints	11.9; 16.6

Feature	Framework V2.1	v0.131	v0.141	v2.1.167	Reference
External monitor hooks with authority-checked admission	Yes	No	No	Yes	16.4
Declared dependencies; every affected execution found	Yes	Partly	Partly	Yes	12.2; 16.5
Governed stop requests carried out by the host's controller	Yes	No	No	Yes	16.7
Enforcement kept apart from world effect	Yes	Partly	Partly	Yes	16.8
Governance of a native call after it has started	Yes	No	No	No	15.1 L1, L8
Autonomous sensing; distributed supervision	Yes	No	No	No	16.12
ENGINEERING AND ASSURANCE					
Supervisory ownership with fencing	n/a	No	No	Yes	16.7
Durable records, crash recovery, divergence reconciliation	n/a	No	No	Yes	16.6; 16.9
Append-only audit of the supervision chain	n/a	Partly	Partly	Yes	16.10
Deterministic simulation harness	n/a	No	No	Yes	16.13
Regression tests passing at release	n/a	967	1,093	1,320	13.8; 15.7

Appendix E — Test and Benchmark Matrix

This appendix lists every test file and benchmark script in the frozen tree, the chapters whose surfaces each one exercises, and the sections of this book that cite it. It also says what each kind of evidence establishes and what it does not, and lists the behaviors this book reports that no test in the tree covers. Chapter 13 explains how the suite and benchmarks are run; this appendix is the lookup table.

E.1 The Evidence Base

Table E.1 — Evidence in the frozen tree and in this book

Evidence	Count	Result
Regression test files (tests/)	168, and 2 fixtures	1,320 tests collected; 1,320 passed (13.8): build 141's 1,093 in 140 files and 227 for the supervision layer in 28 (Table E.6)
Benchmark directory (benchmarks/)	54	50 benchmarks, 2 scenario modules, 2 performance checkpoints; 51 run to completion, 3 fail on later guards
Example scripts (examples/)	17	All run; 2 use the canonical API, 15 use legacy entries (15.4)
Example test file	1	6 tests passed (test_minimal_battery_service.py)
Hash manifest	346 files (build 167)	All match; build 141's manifest: 205 of 206, pyproject.toml differing (13.7)
Appendix B skeleton tests	15	15 passed; host code, not frozen evidence (B.6, B.7)
Confirmation runs for this book	—	Scratch scripts outside the tree; each result is reported where it is used (Table E.5)

E.2 What Each Kind of Evidence Establishes

Table E.2 — What evidence establishes and does not

Evidence	Establishes	Does not establish
A passing regression test	That v0.141 behaves as the test asserts, for the cases the test constructs	Behavior for other inputs; conformance to a framework rule the test does not assert; anything about a successor build
A test of legacy entries only	Legacy behavior	Anything about the canonical stack
A benchmark that runs	That the script's own assertions hold against v0.141; the Small Economy outputs also match their banked record (13.8)	Performance or timing beyond the machine it ran on; general correctness

Evidence	Establishes	Does not establish
A benchmark that fails	That a later guard refuses the usage the script relies on (13.8)	A runtime defect: the regression suite passes
Historical benchmark or V1/V2 results	What an earlier build did	v0.141 behavior (15.8)
The hash manifest	Which files make up the frozen tree	Behavior; the environment (4.9, D.6)
A confirmation run for this book	The specific behavior reported, on the frozen tree	Anything beyond that case; it is reproducible from its description, not banked in the tree

Surface in Tables E.3 and E.4 is derived from the audit record (Appendix H): canonical when the file calls no LEGACY / COMPATIBILITY entry directly, legacy when it calls LEGACY / COMPATIBILITY entries and no PUBLIC ones, mixed otherwise. 9 test files (36 tests) are legacy only and 17 are mixed. Those 36 tests are part of the 1,093 but say nothing about the canonical stack. Chapters are those whose entries document the names the file calls directly; Chapter 4 is omitted when a file also reaches later chapters, since nearly every file builds a registry. Files that call the runtime only through a helper module take that module's chapters.

E.3 Test Files

Table E.3 — Test files

Test file	Tests	Chapters	Surface	Cited in
test_assessment_recovery_integration.py	3	5, 15	mixed	—
test_authoritative_recovery_adequacy_v023.py	9	9, 15	mixed	—
test_branching_converging_structure.py	4	15	legacy	—
test_buffered_multistep_recovery_v048.py	10	13	canonical	—
test_canonical_decision_cycle_v032.py	7	5, 8, 9	canonical	10.10
test_canonical_joint_recovery_feasibility_v050.py	10	5, 9, 11	canonical	—
test_concurrent_recovery_execution_binding_v055.py	8	5, 8, 9, 11	canonical	1.6, 12.5
test_constraint_system_v031.py	10	5, 9	canonical	10.10
test_constraints_gate_v020.py	4	9, 15	mixed	—
test_contaminated_substrate_paired_diagnostics_v053.py	7	13	canonical	13.5, 14.10
test_crop_corridor.py	2	15	legacy	—

Test file	Test s	Chapter s	Surface	Cited in
test_cross_object_execution_identity_v068.py	8	6, 9, 11	canonic al	6.5, 6.6, 9.12, 11.12
test_crossing_governing_consequences.py	2	15	legacy	—
test_cycle_artifact_identity_integrity_v064.py	8	9, 11	canonic al	1.6, 9.12, 14.10
test_dependency_closure_provenance.py	4	15	legacy	—
test_dependency_requirement_modes.py	5	9, 15	mixed	—
test_discretionary_competition.py	1	5, 15	mixed	—
test_domain_assessment_projection_scaling_v019.py	1	5, 9	canonic al	—
test_domain_framing_gate_v019.py	7	9, 15	mixed	—
test_epsilon_bounded_realization_v022.py	11	1, 9, 11, 15	mixed	11.4, 14.10
test_epsilon_trace_performance_architecture_v022.py	1	1, 9, 11	canonic al	—
test_execution_evidence_integrity_v067.py	7	6	canonic al	6.5, 6.6, 11.12, 14.10
test_execution_evidence_provenance_v066.py	8	6	canonic al	6.5, 6.6
test_execution_transition_provenance_v065.py	8	5, 9, 11	canonic al	11.11, 14.10
test_governing_operator_v027.py	12	9	canonic al	—
test_graph_composition_sufficiency_v039.py	7	8	canonic al	—
test_graph_construction_v026.py	10	8, 9	canonic al	9.8
test_graph_uncertainty_integrity_v057.py	7	8, 9	canonic al	9.8, 14.10
test_integrated_canonical_cycle_v041.py	8	5, 8, 9, 10, 11	canonic al	—
test_integrated_cycle_lifecycle_integrity_v069.py	8	9	canonic al	9.12, 14.10

Test file	Test s	Chapter s	Surface	Cited in
test_joint_feasibility.py	4	9, 11	canonical	—
test_layer1_actual_state_boundary_v040.py	10	1, 5, 10, 11	canonical	5.1, 10.10
test_layer1_transition_handoff_v033.py	8	1, 9, 11	canonical	11.10
test_memory_conditioned_sequence_v045.py	9	5, 6, 8, 9, 10, 11	canonical	10.5
test_memory_retrieval_boundary_v043.py	10	5, 6, 8, 9	canonical	9.5, 10.10
test_mission_structural_required_v049.py	9	8, 9	canonical	—
test_mixed_dependency_requirements_v017.py	8	15	legacy	—
test_mode_aware_epsilon_license_v037.py	5	1, 5, 8, 9, 11	canonical	1.6, 1.8, 14.10
test_multicycle_sequence_v042.py	8	5, 8, 9, 11	canonical	9.2
test_no_hidden_single_action_priority_v018.py	3	5, 9, 15	mixed	—
test_perceived_decision_state_boundary_v056.py	9	5, 6, 9	canonical	5.3, 9.5, 10.10, 14.10
test_persistent_state_capacity_authority_v051.py	10	5, 9, 10, 11	canonical	5.7, 10.7, 10.10
test_phi_h_strict_v028.py	10	5, 9	canonical	9.7, 10.10
test_pi_completeness_gate_v018.py	6	9, 15	mixed	—
test_pi_construction_v025.py	7	9	canonical	—
test_policy_set_selection.py	3	15	legacy	—
test_post_execution_bookkeeping_v038.py	8	1, 11	mixed	—
test_post_execution_epistemic_update_v044.py	12	1, 5, 6, 10, 11	canonical	10.5, 10.10

Test file	Tests	Chapters	Surface	Cited in
test_prediction_error_boundary_v052.py	9	6, 9, 10, 11	canonical	9.5, 10.6, 10.10
test_projection_causal_input_licensing_v060.py	8	5, 9	canonical	—
test_projection_causal_state_cross_cycle_v061.py	8	5, 9, 13	canonical	13.6
test_projection_cross_cycle_attribution_v063.py	8	5, 9, 13	canonical	13.6
test_projection_information_quality_v058.py	8	5, 9	canonical	—
test_projection_model_authority_v062.py	8	9	canonical	9.10
test_projection_service_v036.py	7	5, 8, 9	canonical	10.3, 10.10
test_projection_state_and_consistency_v059.py	8	5, 9	canonical	14.10
test_qualified_pi_substrate_protocol_v054.py	10	9	canonical	—
test_regime_operator_v021.py	9	5, 9, 15	mixed	9.2
test_registered_structure_policy_resolution.py	3	9, 15	legacy	—
test_residual_capacity_discretionary.py	4	5, 15	mixed	—
test_scheduler_performance_architecture.py	3	9	canonical	—
test_selected_policy_and_residual_capacity.py	3	5, 15	mixed	—
test_sigma_censor_aware_v030.py	9	9	mixed	—
test_sigma_fallback_v034.py	8	9	canonical	10.10
test_sigma_stage3_order_v029.py	5	9	canonical	—
test_sigma_staged_standard_v024.py	9	9	canonical	—
test_sigma_terminal_v035.py	6	9	canonical	10.10
test_structural_resolution.py	2	9, 15	legacy	—
test_three_domain_oscillation_stress_v047.py	8	13	canonical	13.2, 13.3

Test file	Test s	Chapter s	Surface	Cited in
test_tiny_food_world.py	8	15	legacy	—
test_transition_authoritative_bookkeeping_v046.py	9	1, 5, 11	mixed	11.11
test_v0124_control_provenance_topology.py	8	6	canonic al	6.7, 6.8
test_v0125_control_provenance_topology_lifecycle.py	8	6	canonic al	6.7, 6.8
test_v0126_control_provenance_governance_dependencies.py	8	6, 12	canonic al	6.8
test_v2_1_memory_governance_v0135.py	16	5, 6	canonic al	6.1, 6.2
test_v2_1_objective_interface_v0133.py	13	5, 7, 9	canonic al	7.2
test_v2_1_objective_lifecycle_reentry_v0139.py	18	7, 12	canonic al	12.3
test_v2_1_objective_responsiveness_discovery_v0134.py	15	5, 7, 8, 9	canonic al	7.5, 10.10 , 14.10
test_v2_1_replay_sufficiency_v0137.py	18	1, 5, 6, 9, 11	canonic al	6.1, 6.3, 6.4
test_v2_1_small_economy_benchmark_v0140.py	11	5, 6, 7, 8, 9, 12	canonic al	—
test_v2_1_state_sufficiency_v0138.py	18	5, 6	canonic al	5.9
test_v2_1_transfer_history_v0136.py	17	5, 6	canonic al	6.3, 10.10
test_v2_adapter_configuration_v074.py	6	4	canonic al	4.9
test_v2_adequacy_partial_reentry_v087.py	8	9, 12	canonic al	12.2, 12.3, 12.4, 12.5
test_v2_architecture_invariance_v0129.py	8	4	canonic al	4.10

Test file	Test s	Chapter s	Surface	Cited in
test_v2_authority_bound_callthrough_v0130.py	23	1, 9, 11	mixed	1.6, 11.6, 11.7, 11.9, 14.10
test_v2_bounded_reentry_execution_v084.py	6	12	canonic al	12.1, 12.2, 12.3, 14.10
test_v2_canonical_lineage_authority_v0131.py	5	1, 11	canonic al	11.6, 14.10
test_v2_capability_change_v0116.py	8	8	canonic al	8.9
test_v2_capability_driven_graph_refresh_v0118.py	8	8	canonic al	8.9, 8.10
test_v2_capability_graph_constructibility_v0128.py	8	8	canonic al	8.9, 8.10
test_v2_capability_prediction_error_v0119.py	8	8	canonic al	8.11
test_v2_capability_trajectory_v0117.py	8	8	canonic al	—
test_v2_constraints_partial_reentry_v091.py	7	9, 12	canonic al	12.3, 12.4, 12.5
test_v2_dependency_invalidation_v082.py	9	12	canonic al	12.2
test_v2_domain_framing_partial_reentry_v0115.py	8	9, 12	canonic al	12.5, 12.7
test_v2_epistemic_orchestration_v0113.py	8	6, 12	canonic al	6.7, 12.7
test_v2_epistemic_reentry_execution_v0114.py	8	12	canonic al	12.7
test_v2_evidence_artifact_dependency_v0112.py	8	6, 12	canonic al	—
test_v2_evidence_source_dependency_v0111.py	8	6	canonic al	6.7
test_v2_executable_dependency_scope_v089.py	7	9, 12	canonic al	12.4, 12.8

Test file	Tests	Chapters	Surface	Cited in
test_v2_execution_binding_v075.py	7	11	canonical	—
test_v2_execution_bookkeeping_v080.py	8	11	canonical	11.9
test_v2_governance_reentry_v081.py	9	12	canonical	12.3
test_v2_governing_identity_ordering.py	2	8, 9	canonical	9.12
test_v2_governing_reentry_v096.py	7	9, 12	canonical	12.5
test_v2_graph_reachability_execution_v0103.py	7	8, 9, 12	canonical	8.4
test_v2_graph_reachability_governance_trigger_v0102.py	8	8, 12	canonical	8.4
test_v2_graph_reachability_refresh_v0101.py	8	8	canonical	8.4
test_v2_graph_reentry_v094.py	7	8, 9, 12	canonical	12.5
test_v2_graph_structural_admission_execution_v0106.py	7	8, 12	canonical	8.5
test_v2_graph_structural_admission_governance_v0105.py	8	8, 12	canonical	8.5
test_v2_graph_structural_admission_v0104.py	8	8	canonical	8.5, 14.10
test_v2_graph_structural_coverage_disposition_v0109.py	8	8, 12	canonical	8.8
test_v2_graph_structural_coverage_reentry_v0108.py	8	8, 12	canonical	8.8
test_v2_graph_structural_coverage_v0107.py	8	8	canonical	8.8
test_v2_graph_structural_revision_v0127.py	8	8, 12	canonical	8.5
test_v2_horizon_reentry_v097.py	7	9, 12	canonical	12.5
test_v2_joint_recovery_reentry_readiness_v088.py	6	9, 12	canonical	12.4, 12.8
test_v2_joint_recovery_reentry_v090.py	6	9, 12	canonical	12.4, 12.5

Test file	Test s	Chapter s	Surface	Cited in
test_v2_memory_conditioned_ppp_reentry_v0100.py	8	5, 6, 12	canonical	12.5
test_v2_native_epsilon_bridge_v079.py	7	1, 11	canonical	11.8
test_v2_native_invocation_v076.py	9	11	canonical	11.7
test_v2_native_layer1_handoff_v077.py	6	5, 11	mixed	—
test_v2_native_telemetry_control_v078.py	9	11	canonical	11.9
test_v2_novel_function_admission_v0121.py	8	8, 12	canonical	8.6
test_v2_partial_reentry_sigma_completion_v0120.py	6	9, 12	canonical	—
test_v2_phi_reentry_v098.py	7	12	canonical	12.5
test_v2_pi_completeness_reentry_v092.py	7	9, 12	canonical	12.5
test_v2_pi_reentry_v093.py	7	9, 12	canonical	12.5
test_v2_ppp_reentry_v099.py	7	5, 12	canonical	12.5
test_v2_recovery_and_units_v073.py	8	5, 9	canonical	5.5, 5.6
test_v2_recovery_necessity_admission_v0122.py	8	8, 12	canonical	8.7
test_v2_recovery_necessity_revision_v0123.py	8	8, 12	canonical	8.7
test_v2_reentry_plan_v083.py	8	12	canonical	12.1, 12.3
test_v2_regime_reentry_v095.py	7	9, 12	canonical	12.5
test_v2_reusable_artifact_bundle_v085.py	8	12	canonical	12.4
test_v2_security_evidence_v0110.py	8	6	canonical	6.9
test_v2_sigma_partial_reentry_v086.py	8	9, 12	canonical	12.5

Test file	Test s	Chapter s	Surface	Cited in
test_v2_state_interface_reconciliation_v072.py	7	5	canonical	5.1, 5.3, 5.4, 5.5

E.4 Benchmarks

Table E.4 — Benchmark directory

Script	Kind	Result	Chapter s	Cited in
benchmark_branching_structure_v015.py	benchmark	runs	9	9.11
benchmark_buffered_multistep_recovery_v048.py	benchmark	runs	5, 8, 9, 11, 13	—
benchmark_canonical_cycle_v032.py	benchmark	runs	5, 8, 9	8.2, 9.2
benchmark_canonical_joint_recovery_v050.py	benchmark	runs	5, 9, 11	—
benchmark_capacity_authority_v051.py	benchmark	runs	5, 9, 11	—
benchmark_concurrent_recovery_binding_v055.py	benchmark	runs	9	—
benchmark_constraints_v020.py	benchmark	runs	5, 9	—
benchmark_constraints_v031.py	benchmark	runs	5, 9	9.9
benchmark_contaminated_pair_v053.py	benchmark	runs	13	13.5
benchmark_dependency_requirements_v016.py	benchmark	runs	9	9.11
benchmark_domain_framing_v019.py	benchmark	runs	5, 9	—
benchmark_epsilon_mode_license_v037.py	benchmark	fails (13.8)	9, 11	1.6, 13.8, 15.8
benchmark_epsilon_v022.py	benchmark	fails (13.8)	1, 9, 11	13.8, 15.8
benchmark_execution_transition_provenance_v065.py	benchmark	runs	9, 11	11.11
benchmark_governing_operator_v027.py	benchmark	runs	9	4.5, 9.7
benchmark_graph_composition_sufficiency_v039.py	benchmark	runs	8	4.7, 8.2
benchmark_graph_construction_v026.py	benchmark	runs	8, 9	8.2
benchmark_graph_uncertainty_v057.py	benchmark	runs	8	—

Script	Kind	Result	Chapters	Cited in
benchmark_integrated_boundary_v041.py	benchmark	runs	5, 8, 9, 11	—
benchmark_layer1_actual_state_boundary_v040.py	benchmark	runs	5, 11	11.10
benchmark_layer1_handoff_v033.py	benchmark	runs	1, 9, 11	11.3, 11.4
benchmark_memory_conditioned_sequence_v045.py	benchmark	runs	5, 6, 8, 9, 11	6.2, 9.5
benchmark_memory_retrieval_boundary_v043.py	benchmark	runs	5, 6, 9	6.2, 9.5
benchmark_mission_required_pi_v049.py	benchmark	runs	9	4.6, 9.2, 9.8
benchmark_mixed_dependency_requirements_v017.py	benchmark	runs	9	—
benchmark_multicycle_sequence_v042.py	benchmark	runs	5, 8, 9, 11	9.5
benchmark_perceived_decision_state_v056.py	benchmark	runs	5, 9	—
benchmark_phi_h_v028.py	benchmark	runs	5, 9	4.6
benchmark_pi_completeness_v018.py	benchmark	runs	9	9.8
benchmark_pi_construction_v025.py	benchmark	runs	9	4.6, 9.8
benchmark_post_execution_bookkeeping_v038.py	benchmark	runs	1, 11	11.3, 11.4, 11.11
benchmark_post_execution_epistemic_update_v044.py	benchmark	runs	1, 5, 6, 11	6.2, 6.6
benchmark_prediction_error_v052.py	benchmark	runs	6, 9	6.5
benchmark_projection_attribution_v063.py	benchmark	runs	5, 9, 13	13.6
benchmark_projection_causal_input_licensing_v060.py	benchmark	runs	5, 9	9.10
benchmark_projection_causal_state_cross_cycle_v061.py	benchmark	runs	5, 9, 13	9.10, 13.6
benchmark_projection_information_quality_v058.py	benchmark	runs	5, 9	9.10
benchmark_projection_service_v036.py	benchmark	runs	5, 9	9.10
benchmark_projection_state_consistency_v059.py	benchmark	runs	9	—
benchmark_qualified_pi_v054.py	benchmark	runs	9	9.8

Script	Kind	Result	Chapters	Cited in
benchmark_regime_operator_v021.py	benchmark	runs	9	9.7
benchmark_restoration_adequacy_v023.py	benchmark	runs	9	9.11
benchmark_sigma_fallback_v034.py	benchmark	runs	9	9.11
benchmark_sigma_stage3_v029.py	benchmark	runs	9	9.11
benchmark_sigma_standard_v024.py	benchmark	fails (13.8)	9	9.11, 13.8, 15.8
benchmark_sigma_terminal_v035.py	benchmark	runs	9	9.11
benchmark_single_action_selection_v018.py	benchmark	runs	5, 9, 15	—
benchmark_small_economy_v2_1_v0140.py	benchmark	runs	5, 6, 7, 8, 9, 12	6.1, 7.2, 7.4, 7.6, 12.2, 13.8
benchmark_three_domain_stress_v047.py	benchmark	runs	5, 8, 9, 11, 13	—
benchmark_transition_authoritative_bookkeeping_v046.py	benchmark	runs	1, 5, 11	11.11
performance_checkpoint_v0_11.py	performance checkpoint	runs	5, 9, 11, 15	—
performance_checkpoint_v0_12.py	performance checkpoint	runs	5, 9, 11, 15	—
three_domain_buffered_multistep_scenario_v048.py	scenario module	runs	5, 8, 9, 11, 13	4.6, 4.7, 9.6, 13.2, 13.4
three_domain_oscillation_scenario_v047.py	scenario module	runs	5, 8, 9, 11, 13	4.6, 9.6, 13.3, 13.4

The three failing scripts—`benchmark_epsilon_mode_license_v037`, `benchmark_epsilon_v022`, `benchmark_sigma_standard_v024`—are reconciled in 13.8: each builds or relies on something a later build's guard now refuses (a hand-built license, stored-validity trust at licensing, or a Σ tie without an explicit order). Their historical results do not transfer (15.8).

E.5 Behaviors No Test Covers

The book reports these behaviors of the runtime. Each rests on source reading and a confirmation run, not on the frozen suite. A search of the suite found no test that exercises any of them.

Table E.5 — Reported behaviors without a frozen test

Behavior	Evidence	Section
Defect K1: the PPP branch of <code>execute_epistemic_reentry_plan</code> raises <code>TypeError</code>	Confirmation run	12.7; 15.2
Defect K2: re-entry at Pi Completeness or earlier does not stop for an infeasible active recovery set	Confirmation run with the v055 fixture	12.5; 15.2
One decision yields several executable licenses	Confirmation run; the suite tests single use of each license object only	11.2; L4
A reused episode id replaces the earlier ledger entry	Confirmation run	11.3; L4
An earlier episode object can still be advanced	Confirmation run	11.4; L4
A re-entry decision can be licensed and executed	Confirmation run (Σ -only re-entry)	12.5; L5
"<code>integrated_cycle_transition_invalid</code>" is unreachable	Source reading	14.3; 15.9 D7
The nine declared-only protocols are never called	Source reading; no test implements them	10.9; 15.5
The capacity-authority flow methods are used only inside the integrated cycle	No test calls them directly	5.7; H.5-3
Fact kinds are compared as written: a report in non-canonical form never satisfies a dependency, and the checkpoint fails closed	Confirmation run: B.7 <code>test_fact_kinds_must_be_written_in_canonical_form</code>	16.5; 16.12
Observation times are compared as text; an older report with a different offset displaces a newer view	Confirmation run: B.7 <code>test_observation_times_are_compared_as_written</code>	16.4; 16.12
Duplicate source registrations, dependency bindings, and retry contracts raise <code>sqlite3.IntegrityError</code>	Source reading	16.11

E.6 Resolution Check

Every test file, test function, and benchmark named anywhere in Chapters 1–15 was resolved against the frozen tree at build time: 97 of the 140 test files, 74 individual test functions, 39 of the 54 benchmark-directory scripts, and the example test. All resolve. Files not cited by any

section still appear in Tables E.3 and E.4, so every evidence field in the book resolves to a row here.

E.7 Supervision-Layer Tests

The 28 test files below test only the supervision layer; they are the 227 tests by which the 1,320 of build 167 exceed build 141's 1,093. Each also passes with warnings promoted to errors. Two fixture modules, `r22_real_cycle_fixture.py` and `r22_trust_fixture.py`, supply a real decision cycle and a trust provider. Sections are those whose surfaces each file exercises; hardening rounds 2 to 12 each close findings of an independent hostile review (RELEASE_NOTES.md).

Table E.6 — Supervision test files

Test file	Tests	Sections	Covers
<code>test_r3_phase1_supervisory_store.py</code>	5	16.10	Store records, configuration CAS, audit
<code>test_r3_phase2_canonical_bridge.py</code>	6	16.3	Exact-object resolution and registration
<code>test_r3_phase3_observation_v0144.py</code>	8	16.4	Registration, admission, current view
<code>test_r3_phase4_dependencies.py</code>	8	16.5	Bindings, fan-out, snapshots
<code>test_r3_phase5_continuation.py</code>	8	16.6	Postures, re-entry signals, snapshot scope
<code>test_r22_phase6_control.py</code>	10	16.7	Ownership, adapters, requests, enforcement
<code>test_r22_phase7_effect.py</code>	10	16.8	Effect, finality, retry, Layer 1
<code>test_r22_phase8_recovery.py</code>	10	16.9	Profiles and recovery sequence
<code>test_r22_phase9_simulation.py</code>	10	16.13	Simulation harness
<code>test_r22_phase10_operational.py</code>	10	16.13	Operational adapters
<code>test_r22_phase11_api_misuse.py</code>	14	16.4–16.9	Fourteen misuse cases
<code>test_r22_phase12_race_matrix.py</code>	12	16.6–16.8	Twelve orderings
<code>test_r22_phase13_crash_injection.py</code>	10	16.9	Ten crash points
<code>test_r22_phase14_conformance_closure.py</code>	4	16.1	Requirement manifest, shared fan-out, parity, audit trace
<code>test_r22_phase15_real_crash_matrix.py</code>	11	16.6; 16.9	Real process death
<code>test_r22_phase15_strengthened_validation.py</code>	3	16.1; 16.9	Real cycle drives supervision; behavioral gate

Test file	Tests	Sections	Covers
test_r22_phase15_v0156_hardening.py	9	16.5–16.9	Round 1 hardening
test_r22_phase16_v0157_round2_hardening.py	12	16.6–16.7	Hardening round 2
test_r22_phase17_v0158_round3_hardening.py	8	16.6–16.7	Hardening round 3
test_r22_phase18_v0159_round4_hardening.py	7	16.6–16.7	Hardening round 4
test_r22_phase19_v0160_round5_hardening.py	5	16.6–16.7	Hardening round 5
test_r22_phase20_v0161_round6_hardening.py	9	16.6–16.7	Hardening round 6
test_r22_phase21_v0162_round7_hardening.py	6	16.6–16.7	Hardening round 7
test_r22_phase22_v0163_round8_hardening.py	5	16.6–16.7	Hardening round 8
test_r22_phase23_v0164_round9_hardening.py	6	16.6–16.7	Hardening round 9
test_r22_phase24_v0165_round10_hardening.py	10	16.6–16.7	Hardening round 10
test_r22_phase25_v0166_round11_hardening.py	6	16.6–16.7	Hardening round 11
test_r22_phase26_v0167_round12_hardening.py	5	16.6–16.7	Hardening round 12
Total	227		

Appendix F — Source and Authority Map

This appendix lists every source the Reference cites, with the authority each carries. Two kinds differ in what a reader can check: the runtime, its tests, and its records are public and can be verified independently (13.7); the framework compendia are maintained by the author and are not published with this book, so claims cited to them can be checked only by a reader who has them. Appendix D gives the versions; Appendix E gives the tests and benchmarks. The rule for conflicts is in 1.2: the framework compendia control what the framework requires, the frozen runtime controls what v0.141 does, and neither overrides the other.

F.1 Authority Levels

Project control assigns every document an authority level (C00/D3 §6). Level 0 is project control; Level 1 canonical architecture; Level 2 operator and proof owners; Level 3 controlled support; Level 4 application, benchmark, and memory records; Level 5 public and outreach material; Level 6 archive. An owner file prevails over a supporting file on that owner's subject (C00/D9 §3). This Reference is Level 3: it defines nothing and cites upward.

Table F.1 — The compendia

Compendium	Role	Authority	Edition used
00 Project Control	Repository control, versioning, freeze records, stale-reference checks	Level 0	Rebuilt September 2026 edition (D.7 V6)
10 Core Framework	Ontology, Layer 1, state, perception, and supporting specifications	Levels 1 and 3	V2.1
15 Operators	Owning Layer 2 operator specifications and supporting files	Levels 2 and 3	V2.1
20 Stack and Layer Governance	Stack order, ownership, master architecture	Level 1	V2.1
30 Applications and Extensions	Applications; the application and adapter boundary	Level 4	V2.1
40 Runtime and Execution Formalization	Runtime and agent prototypes subordinate to the framework	Level 3	V2.1; cited only to mark what this book does not cover (Table 1.1)
45 Testing and Ledger	Conformance, audits, replay sufficiency, test records	Levels 3 and 4	V2.1

F.2 Compendium Citations

Table F.2 lists every compendium document the book cites, with its standing, its authority level, the number of citations, and the chapters and appendices that make them. There are 327

citations to 46 documents or whole compendia. Counts are recomputed from the text at every build; Appendices D and F, which describe the citations, are not counted.

Table F.2 — Cited compendium documents

Citation	Standing	Level	Count	Cited in
C00	Project-control compendium, cited as a whole	0	2	1, 15
C00/D1	Project control	0	11	1, 4, 5, 6, 15
C00/D2	Project control	0	5	1, 2, 3, 9
C00/D3	Project-control standard	0	6	1
C00/D6	Project-control standard	0	1	1
C00/D7	Project-control checklist	0	2	1, 3
C00/D8	Freeze record	0	2	1
C00/D9	Project-control standard	0	5	1, 15
C10/D1	Canonical state and symbol table	1	21	2, 3, 4, 5, 6, 9, 10
C10/D2	Canonical Layer 1 formal system	1	12	2, 3, 5, 9, 10, 11
C10/D6	Canonical projection service	1	11	2, 3, 9, 10, 15
C10/D7	Supporting specification (framing rule)	3	3	2, 3
C10/D9	Active supporting owner surface	3	21	2, 3, 7, 8, 9, 10
C10/D10	Classification note	3	2	5, 10
C10/D11	Active supporting specification	3	1	5
C10/D12	Supporting specification	3	3	8
C10/D16	Active supporting specification	3	18	2, 6, 9, 10, 15
C10/D18	Active supporting specification	3	2	8
C10/D19	Canonical perception architecture	1	9	2, 5, 6, 9, 10, 15
C10/D20	Active supporting specification	3	4	5, 8, 10
C10/D21	Active supporting specification	3	3	5
C10/D22	Active supporting specification	3	6	2, 6, 10
C15	Operators compendium, cited as a whole	2	1	4
C15/D1	Planning note; not a specification	3	1	9
C15/D2	Operator owner	2	6	2, 3, 4, 8, 9
C15/D3	Operator owner	2	4	2, 3, 4, 9
C15/D4	Operator owner	2	6	2, 3, 4, 9
C15/D5	Operator owner	2	4	2, 3, 4, 9

Citation	Standing	Level	Count	Cited in
C15/D6	Operator owner	2	12	2, 3, 4, 9, 10, 11, 15
C15/D7	Operator owner	2	9	2, 3, 4, 8, 9
C15/D8	Operator owner	2	6	2, 3, 9
C15/D9	Operator owner	2	28	1, 2, 3, 4, 6, 9, 10, 11, 15
C15/D10	Operator owner	2	10	2, 3, 4, 5, 9, 11
C15/D11	Supporting specification (graph layer)	3	13	2, 3, 4, 7, 8
C15/D12	Supporting specification (completeness)	3	5	2, 3, 8, 9, 13
C15/D14	Supporting specification (graph protocol)	3	9	3, 4, 8
C15/D17	Supporting addendum (fallback Σ)	3	5	3, 4, 9, 11, 15
C20/D2	Ownership map	1	4	2, 3, 15
C20/D3	Stack reference	1	17	2, 3, 4, 7, 8, 9, 12
C20/D4	Master architecture	1	12	12, 15
C30/D3	Cross-domain schema reference	4	5	4, 10, 15
C30/D4	Layer 3 applications	4	2	10
C45	Testing and ledger compendium, cited as a whole	4	11	4, 13, 15
C45/D2	Consolidated audit record	4	2	15
C45/D8	Test record	4	1	9
C45/D9	Supporting specification (replay)	3	4	6, 11

Four checks were run on these citations. Every document exists in the edition used. Every section cited with § exists under that number—checked against the document's headings for every section citation, about seventy, including those to the rebuilt compendium 00. Every cited Guide section exists in the PV-PP Programmer's Guide. And the claim each citation supports was compared with the cited document's text, with low-overlap cases read in full. That last check found one misattribution, C10/D8, since corrected (12.1; D.7 V4). It is a keyword screen followed by reading, not a proof that every citation is apt.

F.3 Runtime and Repository Sources

The frozen tree controls what the runtime does, but not every file in the repository is part of it. Build 167's manifest covers 346 files: the 25 source files, pyproject.toml, the 170 files in tests/, the 54 in benchmarks/, and the examples, shipped docs, and release records. Build 141's manifest, kept in the tree, covered 206 files and none of the documentation. Examples, notes, and records are evidence of use or intent, not runtime behavior, even when hash-covered (13.7; 15.8).

Table F.3 — Runtime and repository sources

Source	Authority	Hash-covered	Where
Runtime source: pvpp_runtime/ (25 files: the 11 of the decision core and the 14 of supervision/)	What v2.1.167 does	Yes	Throughout; symbol citations module:line
Regression tests: tests/ (168 files and 2 fixtures)	What has been verified, for the cases tested	Yes	Appendix E
Benchmarks: benchmarks/ (54 files)	Evidence of use; 3 fail on later guards	Yes	13.8; Appendix E
pyproject.toml	Package version "0.167"; requires-python	Yes	D.2; D.6
V0167_FILE_HASHES.sha256	Identity of 346 files, including the documentation rows below	Is the manifest	1.4; 13.7
Examples: examples/ (17 scripts)	Evidence of use only; 15 use legacy entries	Yes, from build 167	13.8; 15.4
RELEASE_NOTES.md	What each V2.1 build added; claims checked against source	Yes, from build 167	1.4; D.2
README.md, BASELINE_AND_LINEAGE.md	Orientation and lineage; README names the wrong lineage file	Yes, from build 167	13.7; 15.9 D6
BASLINE_CODE_IDENTITY_CHECK.txt	Inherited-file identity at the v0.132 bootstrap	Yes, from build 167	15.7
BENCHMARK_SMALL_ECONOMY_v0_140.txt; HARDENING_SMALL_ECONOMY_v0_141.txt	Banked Small Economy results	Yes, from build 167	13.8
docs/01-09 (V2.1 reader documents)	Retired as sources (F.5)	Yes, from build 167	F.5
V0141_FILE_HASHES.sha256	Build 141's 206 files; 205 match in v2.1.167, pyproject.toml differs	Yes	1.4; 13.7

Source	Authority	Hash-covered	Where
BUILD_NOTES_v0_142 to v0_167; R3_CONFORMANCE_v0_155 to v0_167	Development and hardening history of the supervision layer	Yes	Chapter 16; E.7
runtime-v1 (v0.70), runtime-v2 (v0.131), runtime-v2.1 (v0.141)	Earlier generations; compatibility comparison only	Their own records	15.7

F.4 Proof Program and Companion Sources

Table F.4 — Other sources

Source	Authority	Where
Combined Theorem Packet V3.25 Final	Theorem statements and their conditions (Level 2)	1.2; 1.9; D.4
Proof program closeout and status documents (V3.26 layer; September 2026 closeout)	Program status; do not replace V3.25	1.9; D.4
PV-PP Programmer's Guide, Edition 2.1	Explanation and worked code; companion volume, same level	Throughout, as "Guide n.n"; Appendix G
Appendix B skeletons	Host code written for this book; not frozen evidence	Appendix B
Confirmation runs for this book	The specific behavior reported, on the frozen tree	Table E.5
The project owner's rulings	Recorded decisions where sources were silent or in conflict, for example Adequacy v0.8	Table 2.8; D.7
v0.167 API Classification Record v1.0 (2 October 2026)	Intended use of the supervision surface; candidate-stage, reconciled in H.7	16.1; 16.14; H.7
Runtime 2.2 v0.167 Guide/Reference Supplement v1.0	Description of the supervision layer; candidate-stage; checked against the frozen source	Chapter 16
v2.1.167 release records: Official Release Record, Owner Freeze Declaration, Frozen Source-of-Truth Manifest, Version Naming Migration Record, Candidate Freeze Audit Record, Conformance Reference Reconciliation	Release identity, naming, freeze status	1.4; 15.9 D10; D.2

Source	Authority	Where
PV-PP Getting Started (10.5281/zenodo.23238468) and PV-PP Programmer's Guide (10.5281/zenodo.23238712), Edition 2.1 for v2.1.167	Companion volumes	Appendix G

F.5 Retired Reader Documents

The ten V2.1 reader-facing documents, shipped in runtime-v2.1/docs, are superseded as sources: the Guide and this Reference replace them. Their material now lives as follows.

Table F.5 — Where the reader documents now live

Document	Guide	Reference
01 What Is It and What Is It Good For	Chapters 1, 11	—
02 Framework Executive Overview	Chapters 1, 2	Chapter 2
03 Compact Architecture Skeleton	Chapters 2, 6	Chapters 2–3
04 Scenario and Model Architecture Guide	Chapters 3–5; Appendix C	Chapters 4–5, 10
05 Runtime API Reference	Chapters 6–8	The whole Reference, which supersedes it
06 Structural Comparison and Theoretical Positioning	Chapter 11	1.9
07 Benchmark Design Guide	Chapter 9; Appendix E	Chapter 13
08 Benchmark Programming and Runtime Integration Guide	Chapters 8, 9; Appendix E	Chapters 10, 13; Appendix E
09 Safe Margin versus Fast Gain Teaching Benchmark	Section 10.6	—
10 Architecture Map	Chapter 2	Chapter 2; Table 2.1

Appendix G — Guide–Reference Crosswalk

The Guide explains how to understand and build with the Productive Value–Productive Power (PV-PP) framework; this Reference states exactly what v0.141 provides. This appendix maps each Guide chapter and section to the Reference sections it relies on, maps each Reference chapter back to the Guide sections that explain it, resolves every Guide pointer to the Reference, and checks the Guide against the legacy surface. It uses the PV-PP Programmer's Guide, Edition 2.1.

G.1 By Guide Chapter

Table G.1 — Guide chapters and the Reference

Guide chapter or appendix	Reference
1 What the PV-PP Framework Is and Why It Exists	1.1; 1.9; Chapter 2
2 Conceptual Architecture and Construct Map	Chapters 2–3
3 Modeling a Productive System	Chapters 4–5
4 Information, Memory, History, and Evidence	Chapter 6; 12.7
5 Representing Change, Capability, and Reachability	Chapters 7–8
6 The Canonical Decision Cycle	Chapters 3, 9, 10
7 Selection, Execution, and Environmental Realization	Chapters 11, 14
8 Building a PV-PP Application	Chapters 4, 9–12; Appendix B
9 Testing and Benchmarking	Chapters 13–14; Appendix E
10 Worked Examples	Chapters 8, 12; Appendix B
11 Positioning, Reduction, and Advanced Topics	1.9; Chapter 15; D.4
Appendix A AI Interview Prompts	Appendix A
Appendix B Notation, Symbols, and Glossary	Chapter 2
Appendix C Modeling Checklist	Chapters 4–5
Appendix D Implementation Checklist	10.10; Chapter 15 (host obligations)
Appendix E Testing and Benchmark Checklist	Chapter 13; Appendix E
Appendix F Sources and Open Items	Appendices D and F; 15.1–15.2
12 Supervising Execution While It Runs	Chapter 16; 15.1; 15.3
Appendix G Code Files	Appendix B; G.4

G.2 By Guide Section

Table G.2 lists each Guide section linked to specific Reference sections: either the Reference cites it as “Guide n.n”, or the section names a runtime symbol whose primary entry is in the

Reference. 36 Guide sections have such links. Most Guide sections explain concepts and name no symbol; for them Table G.1 gives the chapter-level mapping.

Table G.2 — Guide sections and Reference sections

Guide section	Reference sections
1.10 One Architecture Across Very Different Domains	4.10
1.12 Using This Guide with AI	1.1
3.6 Separate Possession, Access, Permission, Control, and Operability	5.4
3.7 Model Configuration-Dependent and Joint Capability	5.5
3.8 Define Objectives Without Turning Them Into Hidden Utility	7.2
3.10 Model Recovery as Structure, Not as a Hope	5.6
3.11 Decide Whether a Group Is Actually a Productive Entity	5.5
4.4 Quality and Confidence Do Not Create Authority	6.1, 6.9
4.5 Applicability and Permitted Use	6.1
4.6 Disposition, Supersession, and Contradiction	6.1
4.7 Temporal Provenance: When Was It True, and When Was It Known?	6.1
4.8 Memory Is Not the Same as History	6.2
4.9 Transfer History: Record Events Without Turning Them Into Current Truth	6.3
4.10 Historical Residue and Transition-Relevant State Sufficiency	5.9
4.11 Replay Sufficiency: Reconstruct the Decision That Was Actually Made	6.4
4.12 Prospective Sufficiency and Replay Sufficiency Side by Side	6.4
6.3 Pressure: Φ	4.3
6.4 Collapse Horizons: H	4.4, 7.4
6.6 Regime: $\mathcal{R}$	4.6
6.8 Constraints	4.5
6.10 Recovery in the Cycle, and the Runtime's Joint-Recovery Check	4.5
6.12 Selection: Σ	4.6

Guide section	Reference sections
6.15 Your First Program: Evaluating a Decision Cycle	4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7, 5.2, 8.2, 9.2, 9.6, 10.3
7.4 The Execution License and the Episode	1.6, 1.8, 9.3, 11.3
7.5 Registration Is Not Authorization	1.3
7.8 From Result to Evidence	1.3
7.9 The Layer-1 Transition	1.3
7.12 Program: Executing the Coffee Purchase	4.1, 5.1, 11.5, 11.7, 11.8, 11.10
8.3 Registration and Runtime Prerequisites	4.1, 10.2
8.5 Perception from Evidence	5.3
8.6 The Store and the Layer-1 Transition	5.1
8.7 Observation Intake and Enforcement Interfaces	7.6
8.10 Program: The Coffee Application	5.2, 5.3, 9.6, 10.3, 11.8, 11.10, 12.2
9.9 Reproducibility	4.9
9.12 Program: Testing the Coffee Application	4.6
10.5 Example 4: An Autonomous Operations Agent	4.4, 4.7, 8.4, 8.6, 9.6, 10.3, 12.6

G.3 By Reference Chapter

Table G.3 — Reference chapters and the Guide sections that explain them

Reference	Guide sections
Chapter 1	1.12, 7.4, 7.5, 7.8, 7.9
Chapter 4	1.10, 6.3, 6.4, 6.6, 6.8, 6.10, 6.12, 6.15, 7.12, 8.3, 9.9, 9.12, 10.5
Chapter 5	3.6, 3.7, 3.10, 3.11, 4.10, 6.15, 7.12, 8.5, 8.6, 8.10
Chapter 6	4.4, 4.5, 4.6, 4.7, 4.8, 4.9, 4.11, 4.12
Chapter 7	3.8, 6.4, 8.7
Chapter 8	6.15, 10.5
Chapter 9	6.15, 7.4, 8.10, 10.5
Chapter 10	6.15, 8.3, 8.10, 10.5
Chapter 11	7.4, 7.12, 8.10
Chapter 12	8.10, 10.5

Reference chapters with no section-level Guide link: 2, 3, 13, 14, 15. Their Guide counterparts are at chapter level in Table G.1.

G.4 The Guide's Code Files

Each Guide program was parsed for the runtime names it imports or calls; the Reference chapters are those holding the primary entries for those names. The programs of Chapters 6 to 10 run against a copy of the runtime package byte-identical to the frozen v0.141 source, which is v2.1.167's decision core; the Chapter 12 programs run against frozen v2.1.167.

Table G.4 — Guide programs and the Reference

File	Guide chapter	Reference chapters	Note
coffee_decision.py	6	4, 5, 8, 9	Supplies a PolicyProjectionService
coffee_execute.py	7	4, 5, 9, 11	—
coffee_app.py	8	4, 5, 8, 9, 11, 12	Supplies a PolicyProjectionService
test_coffee_hostile.py	9	4, 9, 11	Supplies a PolicyProjectionService
agent_ops.py	10	4, 5, 8, 9, 11, 12	Supplies a PolicyProjectionService
ops_model.py	12	4, 5, 8, 9	The Chapter 10 agent reduced to the failover decision
supervised_failover.py	12	11, 16	Supervision of the failover; tests in test_supervised_failover.py

G.5 Guide Pointers to the Reference

Every place the Guide defers to the PV-PP Programmer's Reference resolves to a section here.

Table G.5 — Guide pointers

Guide	What the Guide defers	Where it is in the Reference
7.3	Exact method signatures, field lists, and result-normalization tables	11.5–11.8; Table 11.8
8.1	Exact constructors and fields	Chapters 4–5 and 10; Appendix A
Appendix A	Attach the Reference to an AI interview for exact interface detail	Appendix A; Chapters 4–12
Appendix D	Exact constructors and fields	Chapters 4, 10; Appendix A

G.6 Legacy Surface

The Guide was checked for use of any LEGACY / COMPATIBILITY entry (15.4), in its prose and in its code. The prose names none: no tiny_* example, no pre-canonical evaluator, no assess or decide method, no hand-built license, no direct Layer-1 handoff builder. Its runtime-version

language is consistent with this Reference: evidence from v0.70 is marked as such and never attributed to v0.141 (Guide 9.16, 9.18). Every Guide program supplies projections through a projection service; none uses the compatibility projection hook.

The two volumes agree: the Guide teaches no surface this Reference tells new code to avoid.

G.7 Supervision Crosswalk

Getting Started Chapter 6 and Guide Chapter 12 explain the supervision layer; Table G.6 maps their sections to the sections of Chapter 16 that state the contracts.

Table G.6 — Supervision sections across the three volumes

Reference	Guide	Getting Started
16.1 Architecture, authority, classification	12.1–12.2	6.1, 6.7
16.2 Host trust provider	12.10	6.3
16.3 Bridge and active executions	12.3; 7.15–7.16	6.3, 6.5
16.4 Observation intake	12.4	6.3, 6.6
16.5 Dependencies, fan-out, snapshots	12.5	6.3
16.6 Continuation and checkpoints	12.6	6.3–6.5
16.7 Ownership, control, enforcement	12.7	6.3–6.4
16.8 Effect, finality, Layer 1	12.8	6.5
16.9 Recovery	12.9	—
16.10 Store, concurrency, deployment	12.10	—
16.11 Errors	12.14	6.6
16.12 Boundaries and host duties	12.13; 7.17	6.5, 6.7; Chapter 7
16.13 Simulation and reference adapters	12.10	—
B.7 Supervised execution skeleton	12.11–12.12; D.4	6.3–6.6

Appendix H — Export-Surface Audit

This appendix is the method and complete record of the audit that assigns the classification labels of 1.5. The audit is a standing part of the Reference: the label in every entry, compact row, and Appendix A line is read from this record at build time, so they cannot disagree.

Complete. Every one of the 479 symbols of the decision core, and of the 156 symbols of the supervision layer (H.7), has a label and a rule; every default label (17 symbols) cites a register entry; all nine register entries are closed; and the evidence reproduces from Listing H.1. The build enforces the first two conditions and fails if either breaks.

H.1 Scope and Inputs

The audited surface is every name an application can reach: the 335 top-level exports of `pvpp_runtime`; the 133 public methods of `PVPPRuntime` (89), `PVPPRegistry` (30), and `ExecutionBindingRegistry` (14); and 11 public names reachable only through submodules—three interface protocols, the joint-feasibility function, two result records, and the five operator prototypes. 479 symbols in all. The inputs are the hash-verified frozen tree (13.7) and doc 05.

H.2 Evidence Collected

For each symbol: kind, module, and line; signature or fields; first docstring line; the runtime source files that reference it; the test, benchmark, and example files that name it; and how often doc 05 names it. Doc 05 names only about twenty methods, so its silence is weak evidence and never decisive alone. Usage counts corroborate a label but never define one.

H.3 Decision Rules

Intended use, established from documentation, is the primary criterion. Each label cites the rule it rests on. Rules R1, R2, and R6 were revised in v0.8 after review; the rules have not changed since.

Table H.1 — Decision rules

Rule	Evidence pattern	Label
R1	Intended as an ordinary application entry point, or as a host protocol that a PUBLIC constructor or entry requires, as shown by documentation (docstrings, interface definitions, doc 05, release notes); corroborated by direct use in examples or benchmarks; not superseded	PUBLIC / SUPPORTED
R2	Intended for specialized host, integration, or diagnostic work—re-entry, admission, provenance, integrity, explicit validation that the ordinary path already performs—as shown by documentation; direct test use corroborates but does not define it; not superseded	PUBLIC / ADVANCED
R3	A record returned by, or carried in the result of, a PUBLIC entry	IMPLEMENTATION / SUPPORT

Rule	Evidence pattern	Label
R4	Referenced only within its own module or by tests of internals, and neither argument nor result of a PUBLIC entry	INTERNAL
R5	Superseded by a newer entry point according to its own docstring or the release lineage, or retained from V1 or V2 for compatibility	LEGACY / COMPATIBILITY
R6	A record whose documented construction contract assigns construction to the application or a host protocol: an entry's argument, or a host hook's return	The label of the entry or protocol that assigns the construction. The consumer's label is only a presumption; the construction contract must be confirmed
Default	Evidence conflicts or is insufficient	IMPLEMENTATION / SUPPORT, entered in the register (H.5)

H.4 Classification Record

Table H.2 — Labels by scope

Scope	SUP	ADV	IMPL	INT	LEG	Total
export	50	129	145	2	9	335
method	27	87	3	0	16	133
submodule-only	1	2	2	6	0	11
All	78	218	150	8	25	479

Table H.3 is the per-symbol record, grouped by label and rule. Methods are written `Class.method` and submodule-only names `module.name`; a note after a dash gives the register entry or the evidence behind a revised or qualified label. Appendix A gives each symbol's kind and primary section.

Table H.3 — Symbols by label and rule

Label · rule · count	Symbols
SUP · R1 · 34	ExecutionBindingRegistry; ExecutionBindingRegistry.register; interfaces — import route for WorldAdapter; H.5-1; interfaces.WorldAdapter — required host protocol; export gap H.5-1; Layer1TransitionService — host protocol the supported integrated entry needs to execute; PolicyProjectionService — documented route for $Q_t(\pi)$ in the ordinary cycle; PVPPRegistry; PVPPRegistry.register_action; PVPPRegistry.register_constraint_rule; PVPPRegistry.register_domain; PVPPRegistry.register_fallback_configuration; PVPPRegistry.register_governing_configuration; PVPPRegistry.register_graph_composition; PVPPRegistry.register_graph_instance; PVPPRegistry.register_graph_transformation; PVPPRegistry.register_horizon_configuration; PVPPRegistry.register_power; PVPPRegistry.register_recovery_necessity; PVPPRegistry.register_recovery_plan; PVPPRegistry.register_regime_configuration; PVPPRegistry.register_sigma_order; PVPPRuntime; PVPPRuntime.advance_execution; PVPPRuntime.apply_layer1_transition; PVPPRuntime.build_execution_license_from_cycle; PVPPRuntime.build_execution_transition_provenance; PVPPRuntime.build_layer1_transition_handoff; PVPPRuntime.evaluate_canonical_decision_cycle; PVPPRuntime.evaluate_integrated_canonical_cycle; PVPPRuntime.instantiate_execution
SUP · R1 (cont.)	PVPPRuntime.invoke_authorized_native; PVPPRuntime.issue_native_execution_authorization; PVPPRuntime.prepare_canonical_cycle_snapshot; PVPPRuntime.validate_transition_relevant_state_basis
SUP · R6 · 44	ActionDefinition; ActionProjection; ActualPersistentStateEnvelope; CanonicalDecisionCycleRequest; CapacityCalendar; CleanExecutionFailure — raised by the host binding; CompletedInvalidExecution — raised by the host binding; ConstraintObservation; ConstraintRuleDefinition; DomainDefinition; DomainFrame; DomainFrameTarget; ExecutionBindingIdentity; ExecutionObservation; FallbackConfiguration; GoverningConfiguration; GraphCompositionDefinition; GraphConstructionConfig; GraphInstanceDefinition; GraphTransformationDefinition; HorizonConfiguration; IndeterminateExecutionFailure — raised by the host binding; Layer1TransitionResult; ObjectiveDiscoveryHint; ObjectiveRelation; ObjectiveSet; ObjectiveTemporalBounds; PerceivedDecisionState; PiConstructionConfig; PolicyConstraintProfile

Label · rule · count	Symbols
SUP · R6 (cont.)	PolicyProjectionRecord; PreliminaryPreservationObject; PressureFactors; PreviousRegimeState; ProductivePowerDefinition; ProductivePowerState; QualifiedObjectiveRecord; RecoveryCorridorProjection; RecoveryNecessityDefinition; RecoveryPlanDefinition; RegimeConfiguration; SigmaOrderDefinition; TransitionRelevantStateBasis; WorldState
ADV · R2 · 149	admit_graph_transformation_and_plan_reentry; admit_novel_function_and_plan_reentry; admit_recovery_necessity_and_plan_reentry; analyze_sequence — diagnostic; apply_executable_dependency_scope; assess_capability_formation_prediction_error; assess_control_provenance_topology; assess_evidence_dependency_authority; assess_governance_reentry; assess_graph_structural_coverage_and_plan_reentry; assess_joint_recovery_reentry_readiness; assess_security_evidence_observation; bounded_viability_oracle — diagnostic; CANONICAL_REENTRY_STAGE_ORDER; CapacityAuthorityService — optional host service; capture_projection_attribution_snapshot — diagnostic; classify_multidomain_stress — diagnostic; compare_contaminated_substrate_cycles — diagnostic; compare_projection_attribution_across_cycles — diagnostic; compare_projection_causal_state_across_cycles — diagnostic; construct_graph_from_capability_trajectory; control_provenance_topology_identity; derive_control_provenance_governance_invalidations; derive_dependency_invalidations; derive_evidence_authority_governance_invalidations; derive_objective_lifecycle_invalidations; disposition_graph_structural_coverage; execute_adequacy_reentry; execute_constraints_reentry; execute_domain_framing_reentry

Label · rule · count	Symbols
ADV · R2 (cont.)	execute_epistemic_reentry_plan; execute_governance_reentry; execute_governing_reentry; execute_graph_reachability_refresh_reentry; execute_graph_reentry; execute_graph_structural_admission_reentry; execute_horizon_reentry; execute_joint_recovery_reentry; execute_memory_conditioned_ppp_reentry; execute_phi_reentry; execute_pi_completeness_reentry; execute_pi_reentry; execute_ppp_reentry; execute_regime_reentry; execute_sigma_reentry; ExecutionBindingRegistry.assess_identity; ExecutionBindingRegistry.build_reauthorization_handoff; ExecutionBindingRegistry.control_history; ExecutionBindingRegistry.control_state; ExecutionBindingRegistry.create_context; ExecutionBindingRegistry.epsilon_observation; ExecutionBindingRegistry.identity; ExecutionBindingRegistry.identity_for_action; ExecutionBindingRegistry.registered_identities; ExecutionBindingRegistry.set_control_state; ExecutionBindingRegistry.telemetry; interfaces.EpistemicUpdateService — optional host service; interfaces.MemoryRetrievalService — optional host service; ObjectiveResponsivenessAdapter — optional host service; orchestrate_epistemic_invalidation_reentry

Label · rule · count	Symbols
ADV · R2 (cont.)	plan_governance_reentry; PredictionErrorService — optional host service; project_capability_trajectory; PVPPRegistry.action_registry_identity; PVPPRegistry.admit_graph_transformation; PVPPRegistry.admit_novel_function; PVPPRegistry.admit_recovery_necessity; PVPPRegistry.assess_graph_structural_coverage; PVPPRegistry.governing_substrate_identity; PVPPRegistry.graph_substrate_identity; PVPPRegistry.horizon_configuration_identity; PVPPRegistry.phi_substrate_identity; PVPPRegistry.refresh_graph_reachability; PVPPRegistry.regime_configuration_identity; PVPPRegistry.register_policy_seed — revised (H.5-4); PVPPRegistry.revise_graph_transformation; PVPPRegistry.revise_recovery_necessity; PVPPRuntime.apply_post_execution_epistemic_update; PVPPRuntime.apply_prediction_error; PVPPRuntime.assess_architecture_adaptation_invariance; PVPPRuntime.assess_recovery_feasibility; PVPPRuntime.assess_replay_sufficiency; PVPPRuntime.assess_transition_relevant_state_pair; PVPPRuntime.audit_cycle_artifact_integrity; PVPPRuntime.audit_integrated_cycle_lifecycle_integrity; PVPPRuntime.audit_post_selection_identity_chain; PVPPRuntime.build_capacity_authority_request; PVPPRuntime.build_decision_replay_package; PVPPRuntime.build_layer1_execution_commit_from_transition; PVPPRuntime.build_post_execution_epistemic_update_request

Label · rule · count	Symbols
ADV · R2 (cont.)	PVPPRuntime.build_prediction_error_request; PVPPRuntime.classify_regime; PVPPRuntime.construct_graph; PVPPRuntime.construct_pi; PVPPRuntime.construct_pi_from_graph; PVPPRuntime.evaluate_canonical_cycle_sequence; PVPPRuntime.evaluate_collapse_horizons; PVPPRuntime.evaluate_constraint_system; PVPPRuntime.evaluate_memory_conditioned_cycle_sequence; PVPPRuntime.evaluate_memory_conditioned_decision_cycle; PVPPRuntime.evaluate_memory_conditioned_integrated_cycle; PVPPRuntime.evaluate_objective_responsiveness; PVPPRuntime.evaluate_pressure_field; PVPPRuntime.evaluate_restoration_adequacy; PVPPRuntime.evaluate_sigma_fallback; PVPPRuntime.evaluate_sigma_standard; PVPPRuntime.evaluate_sigma_terminal; PVPPRuntime.identify_governing_domains; PVPPRuntime.invalidate_native_execution_authorizations; PVPPRuntime.native_execution_authorization_execution_id; PVPPRuntime.native_execution_authorization_status; PVPPRuntime.native_execution_authorizations; PVPPRuntime.prepare_memory_conditioned_cycle_snapshot; PVPPRuntime.qualify_pi_completeness_for_substrate; PVPPRuntime.record_recovery_bookkeeping_from_transition; PVPPRuntime.record_transfer_history; PVPPRuntime.validate_access_operability_state; PVPPRuntime.validate_actual_persistent_state; PVPPRuntime.validate_execution_transition_provenance; PVPPRuntime.validate_information_governance_metadata

Label · rule · count	Symbols
ADV · R2 (cont.)	PVPPRuntime.validate_layer1_transition_result; PVPPRuntime.validate_memory_retrieval_package; PVPPRuntime.validate_memory_state_reference; PVPPRuntime.validate_objective_discovery_hints; PVPPRuntime.validate_objective_set; PVPPRuntime.validate_perceived_decision_state; PVPPRuntime.validate_pi_completeness; PVPPRuntime.validate_post_execution_epistemic_update_result; PVPPRuntime.validate_prediction_error_assessment; PVPPRuntime.validate_pressure_invariants; PVPPRuntime.validate_productive_configuration_state; PVPPRuntime.validate_projection_causal_inputs; PVPPRuntime.validate_projection_model_authority; PVPPRuntime.validate_projection_record; PVPPRuntime.validate_qualified_unit_state; PVPPRuntime.validate_recovery_resource_state; PVPPRuntime.validate_runtime_configuration_provenance; PVPPRuntime.validate_transfer_history_event; PVPPRuntime.validate_transfer_history_package; qualify_evidence_source_from_control_topology; refresh_graph_from_capability_trajectory; refresh_graph_reachability_and_plan_reentry; revise_graph_transformation_and_plan_reentry; revise_recovery_necessity_and_plan_reentry; testing — diagnostics namespace; TransferHistoryService — optional host service; update_control_provenance_topology; validate_capability_change_observation; validate_reusable_artifact_bundle
ADV · R6 · 69	AccessOperabilityState; AdapterIdentity; AdequacyReusableDomainFramingArtifact; AdequacyReusableProjectionArtifact; ArchitectureInvarianceProfile; CanonicalCycleDirective; CapabilityChangeObservation; CapabilityFormationExpectation; CapabilityGraphConstructibilityDependency; CapabilityGraphReachabilityDependency; CapabilityPredictionArtifactDependency; ConstraintsReusablePiArtifact; ControlProvenanceArtifactDependency; ControlProvenanceRelation; ControlProvenanceTopologyUpdateRequest; EpistemicSubstrateQualification; EvidenceCanonicalArtifactDependency; EvidenceSourceAuthorityState; ExecutionContext; ExpectationStateReference; FallbackStructuralProfile; GovernanceArtifactDependency; GovernanceDependencyChange; GovernanceEvidenceDependency; GovernanceInvalidationSignal; GovernanceReusableArtifactBundle; GoverningReusableIdentificationInputs; GraphReachabilityObservation; GraphReachabilityRefreshRequest; GraphReusableConstructionInputs

Label · rule · count	Symbols
ADV · R6 (cont.)	GraphStructuralCoverageObservation; GraphTransformationAdmissionRequest; GraphTransformationRevisionRequest; HorizonReusableEvaluationInputs; InformationGovernanceMetadata; InformationTemporalProvenance; MemoryConditionedCycleDirective; MemoryConditionedPPPReentryInputs; MemoryRetrievalPackage; MemoryStateReference; NovelFunctionAdmissionRequest; ObjectiveLifecycleChange; ObjectiveResponsivenessRecord; PhiReusableEvaluationInputs; PiReusableConstructionInputs; PolicySeedDefinition — revised (H.5-4); PolicySetEvaluation; PostExecuteEpistemicUpdateResult; PPPReentryInputs; PredictionErrorAssessment; PredictionErrorComponent; ProductiveConfigurationState; ProjectionCausalStateConsistencyDeclaration; ProjectionCausalStateInput; ProjectionInformationQualityClaim; ProjectionModelAuthority; QualifiedUnitState; RecoveryActionConfirmation; RecoveryNecessityAdmissionRequest; RecoveryNecessityRevisionRequest
ADV · R6 (cont.)	RecoveryResourceState; RegimeReusableClassificationInputs; RetrievalQualityMetadata; RuntimeConfigurationProvenance; SecurityEvidenceObservation; SigmaReusableAdequacyArtifact; TransferHistoryEvent; TransferHistoryPackage; TransferHistoryReference
IMPL · R3 · 133	AccessOperabilityValidationAssessment; ActiveRecoveryCorridor; AdequacyAssessment; ArchitectureAdaptationAssessment; BoundedViabilityWitness; CandidatePolicySet; CandidatePolicySpace; CanonicalCycleLedgerEntry; CanonicalCycleSequenceAssessment; CanonicalCycleSnapshot; CanonicalDecisionCycleAssessment; CanonicalIntegratedCycleResult; CapabilityChangeValidationAssessment; CapabilityDrivenGraphConstructibilityAssessment; CapabilityDrivenGraphRefreshAssessment; CapabilityFormationPredictionError; CapabilityFormationPredictionErrorAssessment; CapabilityTrajectoryProjection; CapabilityTrajectoryProjectionAssessment; CapacityAuthorityRequest; CapacityAuthorityValidationAssessment; ConstraintAssessment; ConstraintsAssessment; ConstraintViolationProfile; ContaminatedSubstratePairedAssessment; ControlProvenanceGovernanceInvalidationAssessment; ControlProvenanceTopologyAssessment; ControlProvenanceTopologyUpdateAssessment; CorridorFeasibilityReport; CrossObjectExecutionIdentityAssessment

Label · rule · count	Symbols
IMPL · R3 (cont.)	CycleArtifactIntegrityAssessment; DecisionReplayPackage; DomainAdequacyResult; DomainAssessment; DomainFramingAssessment; EpistemicInvalidationOrchestrationAssessment; EpistemicReentryExecutionAssessment; EpistemicUpdateValidationAssessment; EpsilonStepResult; EvidenceAuthorityGovernanceInvalidationAssessment; EvidenceDependencyAuthorityAssessment; ExecutionBindingAssessment; ExecutionBookkeepingAssessment; ExecutionEpisode; ExecutionLicenseEnvelope; ExecutionTraceNode; ExecutionTransitionProvenance; ExecutionTransitionProvenanceAssessment; GovernanceDependencyInvalidationAssessment; GovernanceReentryAssessment; GovernanceReentryExecutionResult; GovernanceReentryPlan; GovernanceReusableArtifactBundleAssessment; GoverningAssessment; GraphConstructionAssessment; GraphReachabilityGovernanceTrigger; GraphReachabilityRefreshAssessment; GraphStructuralAdmissionGovernanceTrigger; GraphStructuralCoverageAdmissionHandoff; GraphStructuralCoverageAssessment
IMPL · R3 (cont.)	GraphStructuralCoverageDisposition; GraphStructuralCoverageGovernanceTrigger; GraphStructuralRevisionGovernanceTrigger; GraphTransformationAdmissionAssessment; GraphTransformationRevisionAssessment; HorizonAssessment; HorizonDomainResult; InformationGovernanceAssessment; IntegratedCycleLifecycleIntegrityAssessment; JointRecoveryExecutionBindingAssessment; JointRecoveryReentryReadinessAssessment; Layer1ExecutionCommit; Layer1TransitionHandoff; Layer1TransitionValidationAssessment; MemoryConditionedCycleLedgerEntry; MemoryConditionedCycleSequenceAssessment; MemoryConditionedCycleSnapshot; MemoryConditionedIntegratedCycleResult; MemoryRetrievalRequest; models — route to two submodule-only records; models.JointRecoveryFeasibilityAssessment — submodule-only; models.SigmaTerminalAssessment — submodule-only; MultidomainStressClassification; NativeExecutionAuthorization; NativeExecutionControlEvent; NativeExecutionControlState; NativeExecutionResult; NativeExecutionTelemetry; NativeReauthorizationHandoff; NovelFunctionAdmissionAssessment

Label · rule · count	Symbols
IMPL · R3 (cont.)	NovelFunctionAdmissionGovernanceTrigger; ObjectiveDiscoveryAssessment; ObjectiveLifecycleInvalidationAssessment; ObjectiveResponsivenessAssessment; ObjectiveSetValidationAssessment; PerceivedDecisionStateValidationAssessment; PiCompletenessAssessment; PiConstructionAssessment; PolicyAdequacyResult; PolicySelectionAssessment; PostExecuteEpistemicUpdateRequest; PredictionErrorRequest; PredictionErrorValidationAssessment; PressureDomainResult; PressureFieldAssessment; PressureInvariantAssessment; ProductiveConfigurationValidationAssessment; ProjectionAttributionSnapshot; ProjectionCausalStateCrossCycleAssessment; ProjectionConsistencyAudit; ProjectionCrossCycleAttributionAssessment; ProjectionInputLicenseAssessment; ProjectionModelAuthorityAssessment; ProjectionRequest; QualifiedPiCompletenessAssessment; QualifiedUnitValidationAssessment; RecoveryNecessityAdmissionAssessment; RecoveryNecessityAdmissionGovernanceTrigger; RecoveryNecessityRevisionAssessment; RecoveryNecessityRevisionGovernanceTrigger
IMPL · R3 (cont.)	RecoveryResourceValidationAssessment; RegimeAssessment; ReplaySufficiencyAssessment; RuntimeConfigurationProvenanceAssessment; ScheduledRecoveryStep; SecurityEvidenceAssessment; SequenceDiagnostics; SigmaAssessment; SigmaFallbackAssessment; SigmaStageAssessment; TransferHistoryValidationAssessment; TransitionRelevantStatePairAssessment; TransitionRelevantStateSufficiencyAssessment
IMPL · Default · 17	ActionStructureAdapter — H.5-6; CapacityAuthorityEnvelope — H.5-3; ConstraintEvidenceAdapter — H.5-6; execution — H.5-9; ExecutionTransitionAdapter — H.5-6; GraphReachabilityAdapter — H.5-6; kernel — H.5-9; ObjectiveDiscoveryAdapter — H.5-6; ObjectiveInterfaceAdapter — H.5-6; ProjectionWorldModelAdapter — H.5-6; PVPPRuntime.derive_capacity_authority — H.5-3; PVPPRuntime.prepare_integrated_request_capacity — H.5-3; PVPPRuntime.validate_capacity_authority_envelope — H.5-3; reentry — H.5-9; registry — H.5-9; StateMeasurementAdapter — H.5-6; StateSufficiencyAdapter — H.5-6
INT · R4 · 8	feasibility; feasibility.assess_joint_feasibility — submodule-only; operators; operators.prototype.collapse_horizon — submodule-only; operators.prototype.dependency_close — submodule-only; operators.prototype.governing_from_horizon — submodule-only; operators.prototype.pressure_from_margin — submodule-only; operators.prototype.regime_label — submodule-only

Label · rule · count	Symbols
LEG · R5 · 23	ActionEvaluation — result of a legacy entry; DiscretionarySelectionAssessment — result of a legacy entry; ExecutionBindingRegistry.build_layer1_handoff — superseded by the ε bridge of v0.79; H.5-7; ExecutionBindingRegistry.invoke — docstring: deprecated; fails closed; ExecutionResult — argument of record_execution and WorldAdapter.execute; PVPPAssessment — result of a legacy entry; PVPPRegistry.register_dependency_requirement — H.5-2, decided in Chapter 15; PVPPRegistry.register_domain_relation — H.5-2, decided in Chapter 15; PVPPRuntime.assess; PVPPRuntime.build_execution_license; PVPPRuntime.decide; PVPPRuntime.evaluate_constraints — H.5-5, revised in v0.14; PVPPRuntime.evaluate_framed_policy_space; PVPPRuntime.evaluate_policy_sets; PVPPRuntime.evaluate_policy_space — H.5-5, revised in v0.14; PVPPRuntime.evaluate_proposal; PVPPRuntime.evaluate_recovery_policy_space — H.5-5, revised in v0.14; PVPPRuntime.record_execution; PVPPRuntime.record_layer1_execution_commit — docstring: legacy v0.38 compatibility surface; PVPPRuntime.validate_domain_framing — H.5-5, revised in v0.14; RecoveryAdequacyAssessment — result of a legacy entry; SelectedPolicy — result of a legacy entry; StructuralPolicyAssessment — result of a legacy entry
LEG · R6 · 2	DependencyRequirementDefinition — following its entry (H.5-2); DomainRelationDefinition — following its entry (H.5-2)

H.5 Reconciliation Register

Symbols whose evidence conflicted or was insufficient, with the question, the decision, and its status. All entries date from September 2026.

Table H.4 — Reconciliation register

Entry	Question	Decision	Status
H.5-1	WorldAdapter, MemoryRetrievalService, EpistemicUpdateService: accepted by the PVPPRuntime constructor but not exported at the top level	WorldAdapter PUBLIC / SUPPORTED under R1, imported from pvpp_runtime.interfaces; the two services PUBLIC / ADVANCED (10.5). The export gap is recorded, not a label conflict	Settled
H.5-2	register_domain_relation, register_dependency_requirement, and their records: used by examples and benchmarks, but docstrings call them prototypes and only the policy-space evaluators consume them	Default in v0.6; LEGACY / COMPATIBILITY under R5 in v0.19, because every consumer is a legacy entry (15.10)	Closed

Entry	Question	Decision	Status
H.5-3	derive_capacity_authority, validate_capacity_authority_envelope, prepare_integrated_request_capacity, CapacityAuthorityEnvelope: documented for host use, but called only through evaluate_integrated_canonical_cycle	IMPLEMENTATION / SUPPORT by default; closed at that label in v0.20, since no owner file requires direct application use (15.10)	Closed at default
H.5-4	register_policy_seed and PolicySeedDefinition: labeled SUPPORTED in Chapter 4, but registered seeds feed only construct_pi, which the canonical cycle never calls	PUBLIC / ADVANCED under R2 and R6	Closed
H.5-5	validate_domain_framing, evaluate_policy_space, evaluate_recovery_policy_space, evaluate_constraints: pre-canonical evaluators with prototype governing domains and, in two cases, non-canonical fallbacks	Default in v0.12; LEGACY / COMPATIBILITY under R5 in v0.14, superseded by the canonical cycle (v0.32; 9.13)	Closed
H.5-6	Nine exported adapter protocols that no runtime method accepts or calls, and nothing implements by name	IMPLEMENTATION / SUPPORT by default, kept as reserved roles; not LEGACY, since nothing supersedes them (15.5)	Closed at default
H.5-7	ExecutionBindingRegistry.build_layer1_handoff (v0.77): a Layer-1 handoff built directly from a native result, bypassing the ϵ trace that v0.79 made canonical	LEGACY / COMPATIBILITY under R5	Closed
H.5-8	assess, decide, evaluate_proposal, evaluate_policy_sets, evaluate_framed_policy_space, build_execution_license, record_execution, and the records only they return: were unlabeled when this question was opened	LEGACY / COMPATIBILITY under R5; build_execution_license's licenses cannot be instantiated in v0.141 (15.4)	Closed
H.5-9	Submodules kernel, registry, execution, reentry: exported names whose public contents are all exported at the top level	IMPLEMENTATION / SUPPORT by default; import from the top level (A.4)	Closed at default
H.5-10	Records the supervision services return, and store methods the services call: the v0.167 classification record labels surfaces as wholes, SUPPORTED, while rule R3 labels returned records and the default rule labels unsettled store methods	IMPLEMENTATION / SUPPORT under R3 for returned records; default for the store methods the services call. The record and this book agree on every service, entry, and exception (16.1)	Closed

Entry	Question	Decision	Status
H.5-11	EvidenceUnverified: exported and listed among the integration-boundary errors, but never raised	IMPLEMENTATION / SUPPORT by default; documented in 16.11 and 15.9 D12	Closed at default
H.5-12	Thirteen package-level names absent from the supervision __all__, including DependencyService and EffectService	Labels unaffected: export is not authority. Recorded as an export gap (15.9 D11)	Settled

H.6 Reproduction

Listing H.1 is the evidence collector. It imports the frozen package without writing bytecode, reads source, tests, benchmarks, examples, and doc 05, and writes evidence.json; it writes nothing into the runtime tree. Run it only after the hash check (13.7). The labels themselves are judgments recorded in Table H.3, not outputs of the collector; a reviewer reproduces them by applying Table H.1 to the evidence.

Listing H.1 — collect.py

```

"""Export-surface audit evidence collector (Appendix H).

Usage: python3 collect.py <runtime-v2.1 directory>
Reads the hash-verified frozen tree; writes evidence.json in the current directory.
"""

import os, re, sys, json, inspect, dataclasses, enum, importlib

if len(sys.argv) != 2:
    sys.exit("usage: python3 collect.py <runtime-v2.1 directory>")
ROOT = os.path.abspath(sys.argv[1])
sys.dont_write_bytecode = True
sys.path.insert(0, ROOT)
import pvpp_runtime as P
from docx import Document

def read(p):
    return open(p, encoding="utf-8", errors="replace").read()

def files(sub, ext=".py"):
    out = []
    for dp, dn, fn in os.walk(os.path.join(ROOT, sub)):
        dn[:] = [d for d in dn if d != "__pycache__"]
        out += [os.path.join(dp, f) for f in fn if f.endswith(ext)]
    return sorted(out)

SRC = {os.path.relpath(f, ROOT): read(f) for f in files("pvpp_runtime")}
TESTS = {os.path.relpath(f, ROOT): read(f) for f in files("tests")}
BENCH = {os.path.relpath(f, ROOT): read(f) for f in files("benchmarks")}
EXAMP = {os.path.relpath(f, ROOT): read(f) for f in files("examples")}
DOC05 = "\n".join(
    p.text
    for p in Document(os.path.join(ROOT, "docs/05_PVPP_Runtime_API_Reference_V2_1.docx")).paragraphs
)
d = Document(os.path.join(ROOT, "docs/05_PVPP_Runtime_API_Reference_V2_1.docx"))
DOC05 += "\n" + "\n".join(c.text for t in d.tables for r in t.rows for c in r.cells)

def kind(o):

```



```

    if inspect.ismodule(o):
        return "module"
    if inspect.isclass(o):
        if issubclass(o, BaseException):
            return "exception"
        if issubclass(o, enum.Enum):
            return "enum"
        if dataclasses.is_dataclass(o):
            return "dataclass"
        return "class"
    if inspect.isfunction(o):
        return "function"
    return "constant"

def loc(o):
    try:
        f = os.path.relpath(inspect.getsourcefile(o), ROOT)
        ln = inspect.getsourcelines(o)[1]
        return f, ln
    except Exception:
        return None, None

def count(corpus, pat):
    rx = re.compile(pat)
    return sorted(f for f, t in corpus.items() if rx.search(t))

def evidence(name, obj, qual=None, method=False, defmod=None):
    k = "method" if method else kind(obj)
    f, ln = loc(obj)
    pat = (r"\.%s%s*" % re.escape(name)) if method else (r"\b%s\b" % re.escape(name))
    src_hits = [x for x in count(SRC, pat) if not (x == f and len(re.findall(pat, SRC[x])) <= 1)]
    doc = inspect.getdoc(obj) or ""
    sig = ""
    try:
        if k in ("function", "method", "class", "exception"):
            sig = str(inspect.signature(obj))
    except Exception:
        pass
    fields = (
        [
            (
                x.name,
                str(x.type),
                (
                    repr(x.default)
                    if x.default is not dataclasses.MISSING
                    else ("empty" if x.default_factory is not dataclasses.MISSING else None)
                ),
            ),
        ]
        for x in dataclasses.fields(obj)
    )
    if k == "dataclass"
    else []
    )
    frozen = bool(k == "dataclass" and obj.__dataclass_params__.frozen)
    return dict(
        name=qual or name,
        kind=k,
        file=f,
        line=ln,
        signature=sig,
        fields=fields,
        frozen=frozen,
        doc_first=doc.split("\n")[0] if doc else "",
        src_files=src_hits,
        tests=count(TESTS, pat),
        benchmarks=count(BENCH, pat),
        examples=count(EXAMP, pat),
        doc05=len(re.findall(re.escape(name), DOC05)),
    )

```

```

rows = []
exports = sorted(n for n in dir(P) if not n.startswith("_"))
for n in exports:
    rows.append(dict(evidence(n, getattr(P, n)), scope="export"))
for cls in (P.PVPPRuntime, P.PVPPRegistry, P.ExecutionBindingRegistry):
    for n, m in inspect.getmembers(cls, predicate=inspect.isfunction):
        if n.startswith("_"):
            continue
        rows.append(dict(evidence(n, m, qual=f"{cls.__name__}.{n}", method=True), scope="method"))
# submodule-only public names
for modname in (
    "interfaces",
    "feasibility",
    "models",
    "kernel",
    "reentry",
    "execution",
    "registry",
    "testing",
    "operators",
    "operators.prototype",
):
    mod = importlib.import_module("pvpp_runtime." + modname)
    for n in sorted(vars(mod)):
        if n.startswith("_") or n in exports:
            continue
        o = getattr(mod, n)
        if inspect.ismodule(o):
            continue
        if getattr(o, "__module__", None) != mod.__name__:
            continue
        rows.append(dict(evidence(n, o, qual=f"{modname}.{n}"), scope="submodule-only"))
json.dump(rows, open("evidence.json", "w"), indent=1)
from collections import Counter

print(len(rows), Counter(r["scope"] for r in rows), Counter(r["kind"] for r in rows))

```

Listing H.2 — Running the collector

```

$ pip install python-docx # reads doc 05
$ python3 collect.py "<repository>/runtime-v2.1" # writes evidence.json
479 Counter({'export': 335, 'method': 133, 'submodule-only': 11}) Counter({'dataclass': 252, 'method':
133, 'function': 60, 'class': 21, 'module': 9, 'exception': 3, 'constant': 1})

```

H.7 Supervision-Layer Classification

Scope. The 51 package-level names of `pvpp_runtime.supervision`, the five control records reachable only through its `models` module, and the 100 public methods of its 19 classes: 156 symbols. Underscore-prefixed names are not classified. The evidence was collected by `gen_api.py`, which imports the hash-verified tree without writing bytecode and records each symbol's kind, module, line, signature or fields, and the test, benchmark, and example files that name it.

Rules. Table H.1 applies unchanged, with one addition: R7 assigns PUBLIC / REFERENCE to a supplied implementation that illustrates a contract a deployment may implement differently, as documented by the v0.167 API Classification Record. Intended use is read from that record and the Guide/Reference Supplement; the source and the 227 successor tests corroborate it. Every symbol has a label and a rule, and every default label cites a register entry (H.5-10, H.5-11). These are audit conclusions resting on that evidence and on the author's judgment; a reviewer reproduces them by applying the rules to `gen_api.py`'s output. Intended use comes from the classification record; observed behavior from the source and tests. A PUBLIC label—above all PUBLIC / REFERENCE—describes the API, not a production deployment contract.

Table H.5 — Supervision labels by scope

Scope	SUP	ADV	REF	IMPL	INT	LEG	Total
Package-level and models names	20	1	11	23	1	0	56
Methods	47	20	14	11	4	4	100

Table 16.33 gives the label of each name, and Tables A.8 and A.9 the label and primary section of each name and method. The decisions above are subject to review by the project owner, like every label in this appendix (1.5).